

```
async function 재기(횟수, 할일) {  
  const 잔것 = [];  
  const 시작 = performance.now();  
  await 할일(회차);  
  잔것.push(performance.now() - 시작);  
  const 만들기시작 = performance.now();  
  await db.exec(`
```

2 권 · 성능 과 운영

데이터베이스

표를 만들고, 넣고, 잇고, 빠르게 만듭니다.

이 책의 모든 출력은 실제로 실행해서 얻은 값입니다.

5

단원

25

개념

173

직접 해보기

0

문제

차례

06	색인과 실행계획	13
01	왜 느린가	14
02	색인을 겁니다	15
03	실행계획 읽는 법	30
04	복합 색인과 순서	45
05	색인이 안 쓰일 때, 그리고 N+1	60
07	트랜잭션과 동시성	83
01	트랜잭션이 무엇인가	84
02	동시에 고치면 사라집니다	100
03	격리수준 네 가지	116
04	잠금	131
05	데드락	149
08	애플리케이션과 잇기	155
01	서버에서 데이터베이스 부르기	156
02	커넥션 풀	173
03	저장소 계층으로 묶기	189
04	마이그레이션	192
05	ORM 은 언제 쓰나	207
09	MySQL과 무엇이 다른가	227
01	왜 둘로 갈렸나	228
02	옛날 이야기를 뒤집습니다	242
03	조용히 다른 결과	258
04	기능과 문법의 차이	277
05	웁길 때 확인할 것	298
10	운영과 종합 실습	325

01	백업과 복구	326
02	느린 쿼리 찾기	344
03	청소와 통계	361
04	멈추지 않고 스키마 바꾸기	362
05	종합 실습	373

부록 가	연습문제·종합연습 정답	403
------	---------------------	-----

부록 나	심화문제 정답	491
------	----------------	-----

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작하기

```
npm install
cd 01_왜_데이터베이스인가
node 개념01_파일에_저장하면_생기는_일.js
```

01~06단원은 이게 전부입니다. 데이터베이스를 설치하지 않습니다.

npm install 하나로 진짜 PostgreSQL 18 이 옵니다. PGLite 라는 것인데, Postgres 를 WebAssembly 로 컴파일한 것입니다. 흥내가 아니라 진짜입니다.

```
PGLite 0.5.7 = PostgreSQL 18.3 (wasm32)
```

07단원부터는 진짜 서버가 필요합니다. 동시에 두 사람이 접속하는 실습을 하기 때문입니다.

```
docker compose up -d      # postgres:18 (5434) + mysql:8.4 (3307)
docker compose ps          # 둘 다 healthy 가 될 때까지 기다립니다
```

필요한 것

무엇	언제부터	비고
Node 20 이상	01단원	npm install 만 하면 됩니다
Docker	07단원	★ 01~06단원은 필요 없습니다

포트가 5432 도 5433 도 아닌 **5434** 입니다. 이미 깔린 Postgres 와 안 부딪히게 비켜 뵈었습니다. 그 포트도 쓰고 있으면 docker-compose.yml 의 숫자를 바꾸세요. (port is already allocated 라는 에러가 납니다)

이 자료가 다른 점

모든 출력은 실제로 실행해서 얻은 값입니다

자료에 적힌 // 출력: 은 전부 진짜 실행 결과입니다. 기계가 대조합니다.

```
npm run 검증
```

사람 눈으로는 절대 못 잡습니다. 자료에 적힌 출력이 실제와 다르면 학생이 자기가 틀린 줄 알고 헤맬니다.

틀린 것을 실제로 돌려서 보여 줍니다

- SQL 인젝션을 **진짜로 뚫습니다** (로그인 우회, UNION 유출, 표 이름 캐내기)
- 데드락을 **진짜로 만듭니다** (40P01 deadlock detected)
- UPDATE 에서 WHERE 를 빠뜨려 **전부 바꿔 봅니다**
- 표를 **진짜로 날리고 복구합니다**

감으로 쓰지 않았습니다

아래는 이 자료를 만들면서 감으로 적었다가 측정에 뒤집힌 것들입니다. 전부 다시 재서 고쳤습니다.

★ 실측으로 뒤집힌 것들

타입

무엇	처음 생각	실제
BIGINT 가 JS 로 오면	항상 bigint	★★★ PGlite: 안전정수(2^53-1) 이하면 number, 넘으면 bigint / pg 드라이버: 항상 string
COUNT(*)	bigint	PGlite 는 number, pg 는 string
0.1 + 0.2 !== 0.3 재현	REAL 로 됨	★ REAL 로는 안 됩니다. 정밀도가 낮아 오차가 반올림에 묻힙니다. DOUBLE PRECISION 이어야 합니다
DATE 의 시간대 함정	toISOString()	★ PGlite 는 getDate() 쪽 / pg 는 로컬 자정이라 toISOString() 도 하루 밀립니다

★★ BIGINT 함정이 생각보다 무섭습니다. 개발할 때는 id 가 작아 number 로 오다가, 운영에서 커지면 bigint 로 바뀌어 JSON.stringify 가 터집니다. 테스트로는 절대 안 잡힙니다. 그래서 이 자료는 전부 count(*)::int 를 씁니다.

성능

무엇	처음 생각	실제
----	-------	----

DB 가 파일보다 빠르다	항상	★ 10만 건이면 메모리 배열 filter 가 8배 빠릅니다. 그래도 DB 를 쓰는 이유는 따로 있습니다 (01단원 개념02)
색인 효과	벽시계로 재면 됨	★★ 벽시계로는 3342배인데 Execution Time 으로는 400600배 **, Buffers 로는 220분의 1
복합색인 앞칸이 아니면	절대 안 쓰임	★★ PG18 부터 skip scan 이 생겨서 뒷칸만 줘도 씁니다. 다만 앞칸 값 종류가 많으면 여전히 Seq Scan
OR 는 색인을 못 씀	항상	한 표 안에서 양쪽에 색인이 있으면 BitmapOr 로 잘 씁니다. 두 표에 걸칠 때가 진짜 문제
커버링 색인	만들면 Index Only Scan	★ VACUUM 을 해야 가시성 지도가 채워집니다. 그 전엔 Bitmap Heap Scan
N+1	로컬에서도 크게 느림	로컬 5.5배. 망 왕복 3ms 를 흥내 내면 33~40배
상관 서브쿼리	조금 느림	JOIN 보다 120~220배 (설비 1000 / 점검 20000)
OFFSET 이 커지면 느려짐	항상	★ 정렬키에 색인이 있어야 재현됩니다 (없으면 2.9배, 있으면 65~70배)
10만 건 넣기	한 건씩도 할 만함	한 건씩 16.1초 / 묶음 0.68초 / generate_series 0.34초 / COPY 0.23초 (71배)

동시성

무엇	처음 생각	실제
REPEATABLE READ 와 팬텀	교과서대로 허용	★★★ Postgres 는 막습니다. 스냅샷 격리라서요. 표준은 최소 요구조건입니다
스냅샷이 찍히는 시점	BEGIN	★ 첫 문장입니다. BEGIN 뒤에 남이 커밋한 것이 보입니다
데드락 희생자	규칙이 있음	★ 완전히 비결정적 입니다. 4회 재니 가/나/가/나
SERIAL 번호	롤백하면 돌아옴	★ 안 돌아옵니다. 실패한 INSERT 도 번호를 먹습니다
긴 트랜잭션의 영향	좀 느려짐	★★ 청소를 막습니다. 168KB → 1096KB (안 열면 280KB). 3.9배
파일에 20건 동시 쓰기	몇 건 사라짐	1건만 남습니다 (5회 전부)

MySQL

무엇	처음 생각	실제
MySQL 은 관대해서 위험	이상한 값이 조용히 들어감	★★★ MySQL 8.4 는 전부 거절합니다. 타입·길이·범위 ·GROUP BY·CHECK 전부 PG 와 같습니다. 8.0 부터 엄격 모드가 기본입니다
대소문자 무시가 위험	MySQL 이 뚫림	★★★ 반대입니다. 'Admin' 가입 뒤 'admin' 재가입 → PG 는 통과(계정 2개), MySQL 은 1062 로 거절
MySQL 은 뒤 공백을 무시	항상	★ collation 에 따라 다릅니다. 5.7 기본은 무시, 8.0 기본은 구분합니다
'a' 'b' 가 MySQL 에서	0	★ 뒷값이 숫자로 읽히면 1 이 됩니다. "0 이면 문제"로 점검하면 못 잡습니다

★ "MySQL 은 허술하다"고 배웠다면 그건 옛날 이야기입니다. 낡은 정보를 가르치면 안 됩니다.
09단원에서 둘을 나란히 띄워 놓고 직접 재 봅니다.

그 밖에

- **NATURAL JOIN** 은 쓰면 안 됩니다. 칸을 하나씩 추가하니 결과가 $15 \rightarrow 0 \rightarrow 7 \rightarrow 3$ 으로 무너졌습니다. 같은 데이터에 **ON** 조인은 내내 15줄
- **NOT IN** 에 **NULL** 이 섞이면 0건입니다. 에러도 안 납니다
- ★★ **JSONB** 안의 값에는 제약을 못 겁니다. 외래키·UNIQUE·NOT NULL 전부 42601 — "제약 위반" 이 아니라 문법 자체가 없습니다. 그리고 JSONB 검색은 생각보다 안 느립니다(20만 줄에서 1.8배). **JSONB** 의 진짜 위험은 느린 게 아니라 틀린 데이터가 들어가는 것입니다
- ★★ 자기 참조에서 "자기가 자기 상위" 인 것을 외래키가 못 막습니다. **UPDATE** 조직 **SET** 상위번호 = 번호 가 그냥 통과합니다. **CHECK** (상위번호 <> 번호) 가 필요하고, 순환(**A**→**B**→**A**)은 **CHECK** 로도 못 막습니다
- ★★ **ON DELETE RESTRICT** 와 **NO ACTION** 은 같은 것이 아닙니다. 에러 코드부터 다릅니다 (23001 vs 23503). **DEFERRABLE INITIALLY DEFERRED** 를 걸면 **NO ACTION** 은 커밋까지 미뤄져 통과하고, **RESTRICT** 는 미뤄도 즉시 막습니다
- **LEFT JOIN** 뒤의 **WHERE** 는 **INNER JOIN** 이 됩니다. ... **OR IS NULL** 로 고치면 절반만 고쳐집니다
- **FROM** 절 서브쿼리 별칭은 **PostgreSQL 16**부터 선택 사항입니다 (MySQL 8.0 은 여전히 에러)
- **SUM(점수)** / **COUNT(*)** 는 캐스팅 없이 쓰면 정수 나눗셈이라 소수점이 통째로 잘립니다
- **EXPLAIN ANALYZE** 는 **DELETE** 를 진짜로 실행합니다. 66,666건이 진짜 지워졌습니다
- **PGlite** 에서 **COPY** 가 됩니다 (/dev/blob). 10만 건 0.23초
- **console.log** 안의 % 는 **printf** 서식으로 해석됩니다. %95% 가 %95 로 줄어듭니다

표 이름을 한글로 쓰는 이유

```
CREATE TABLE 설비 (  
  id      INTEGER PRIMARY KEY,  
  이름    TEXT NOT NULL,  
  라인    TEXT NOT NULL  
)
```

읽기 쉬우라고 그렇게 했습니다. 회사에서는 영어를 쓰는 곳이 더 많습니다. 팀 규칙을 따르세요.

★ 다만 접속 정보(계정·DB 이름·컨테이너 이름)는 영어입니다. 도구들이 한글을 거절하는 곳이 많습니다. 도커는 컨테이너 이름에 한글이 들어가면 아예 안 만들어 줍니다. `application_name` 에 한글을 넣으면 `pg_stat_activity` 에 16진수로 보입니다.

색인과 실행계획

OPEN 06_색인과_실행계획

```
async function 재기(횟수, 할일) {  
  const 잔것 = [];  
  const 시작 = performance.now();  
  await 할일(회차);
```

06

01 왜 느린가

02 색인을 겁니다

03 실행계획 읽는 법

04 복합 색인과 순서

05 색인이 안 쓰일 때, 그리고 N+1

- 연습문제

왜 느린가

RUN node 개념01_왜_느린가.js

★ 이 파일은 25초쯤 걸립니다. 20만 건을 진짜로 만들고 진짜로 잡니다.

05단원까지 표를 만들고, 넣고, 읽고, 이어 봤습니다. 전부 잘 돌아갔습니다. 데이터가 몇십 건이었으니까요.

현장은 다릅니다. 점검기록은 하루에도 수천 건씩 쌓입니다. 1년이면 수십만 건, 3년이면 백만 건이 넘습니다.

이 단원에서는 딱 하나를 합니다.

느린 질의를 어떻게 찾아내고, 어떻게 빠르게 만드는가.

그런데 “색인을 걸면 빨라집니다” 같은 말은 이 단원에 한 줄도 안 나옵니다. 전부 재서 숫자로 보여 드립니다. 색인이 **소용없는 경우도** 똑같이 재서 보여 드립니다.

★★ 먼저 밝혀 둘 것이 있습니다. 이 자료는 PGLite 를 씁니다. PGLite 는 PostgreSQL 을 WebAssembly 로 옮긴 것이라 진짜 서버용 PostgreSQL 보다 느립니다. 그리고 데이터를 전부 메모리에 둡니다.

그래서 **절대값은 여러분 회사 서버와 다릅니다**. 하지만 “색인 있을 때 대 없을 때” 같은 **상대 비교는 그대로 유효합니다**. 이 단원에서 보는 배수와 계획 모양은 진짜 PostgreSQL 에서도 똑같습니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

재는 도구

★ 한 번 재고 결론 내리면 안 됩니다. 첫 실행은 캐시가 비어 있어서 느립니다. 그래서 한 번 예열하고 시작합니다. 그리고 여러 번 재서 **중앙값**을 씁니다. 평균은 어쩌다 한 번 느린 것에 크게 흔들립니다.

```
async function 재기(횟수, 할일) {
  await 할일(0); // 예열. 이 값은 버립니다 const 잔것 = [];

  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    await 할일(회차);
    잔것.push(performance.now() - 시작);
  }
}
```

색인을 겁니다

RUN node 개념02_색인을_겁니다.js

★ 이 파일은 60초쯤 걸립니다. 100만 건까지 진짜로 만들어서 켜니다.

개념01 에서 봤습니다. 열 줄 찾자고 2642 쪽을 전부 읽었습니다.

이 파일에서는 **색인(index)** 을 겁니다. 그리고 네 가지를 켜니다. ① 얼마나 빨라지는가 ② 왜 그렇게 빨라지는가(B-tree 깊이) ③ 얼마나 느려지는가(넣고 고치는 쪽) ④ 자리를 얼마나 먹는가

★★ 색인은 공짜가 아닙니다. **읽기를 사고 쓰기를 파는 거래**입니다. 그래서 “일단 다 걸어 두자” 가 왜 나쁜지 이 파일에서 숫자로 확인합니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();

async function 재기(횟수, 할일) {
  await 할일(0); // 예열 const 잔것 = [];

  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    await 할일(회차);
    잔것.push(performance.now() - 시작);
  }

  return 잔것.sort((가, 나) => 가 - 나)[Math.floor(횟수 / 2)];
}

async function 계획(sql, 옵션 = "") {
  const 결과 = await db.query(`EXPLAIN ${옵션} ${sql}`);
  return 결과.rows.map((줄) => 줄["QUERY PLAN"]).join("\n");
}

const 메가 = (바이트) => `${(Number(바이트) / 1024 / 1024).toFixed(2)} MB`;
```

기준값을 먼저 잡습니다

섹션 1

```
await db.exec(`
  CREATE TABLE 점검기록 (
```

```

설비번호 INT NOT NULL,
라인      TEXT NOT NULL,
상태      TEXT NOT NULL,
점검자    TEXT NOT NULL,
점검일    DATE NOT NULL,
비고      TEXT
);

INSERT INTO 점검기록 (설비번호, 라인, 상태, 점검자, 점검일, 비고)
SELECT
  (i % 20000) + 1,
  (ARRAY['A','B','C','D'][(i % 4) + 1]),
  (ARRAY['정상','주의','고장'][(i % 3) + 1]),
  '점검원' || ((i % 50) + 1),
  DATE '2024-01-01' + ((i % 365) || ' days')::interval,
  '특이사항 없음. 기록번호 ' || i
FROM generate_series(1, 200000) AS i;
`);

```

```
await db.exec("ANALYZE 점검기록");
```

```
const 찾기 = () => db.query("SELECT * FROM 점검기록 WHERE 설비번호 = $1", [12345]);
```

```
const 색인전ms = await 재기(9, 찾기);
```

```
console.log(`색인 없이: ${색인전ms.toFixed(2)} ms`);
```

```

출력          색인 없이: 18.61 ms
값이 다를 수 있음

```

CREATE INDEX

섹션 2

문법은 짧습니다.

```
CREATE INDEX 색인이름 ON 표이름 (칸이름);
```

이름은 마음대로 정합니다. 안 적으면 PostgreSQL 이 표_칸_idx 로 지어 줍니다. ★ 회사에서는 idx_점검 기록_설비번호 처럼 규칙을 정해 둡니다. 나중에 지울 때 안 헛갈리려고요.

```

const 만들기시작 = performance.now();
await db.exec("CREATE INDEX 점검_설비번호 ON 점검기록 (설비번호)");
const 만드는데 = performance.now() - 만들기시작;

console.log(`20만 건에 색인 만드는 데: ${만드는데.toFixed(0)} ms`);

```

```

출력          20만 건에 색인 만드는 데: 85 ms
값이 다를 수 있음

```



```
await db.exec("ANALYZE 점검기록");
```

★★ 얼마나 빨라졌나

섹션 3

```
const 색인후ms = await 재기(9, 찾기);

console.log(`색인 후: ${색인후ms.toFixed(2)} ms`);
```

출력 색인 후: 0.52 ms
값이 다를 수 있음

```
console.log(`몇 배: ${(색인전ms / 색인후ms).toFixed(0)}배`);
```

출력 몇 배: 36배
값이 다를 수 있음

★★ 값은 기계마다 다릅니다. 판정은 따로 찍습니다.

```
console.log("색인이 더 빠른가:", 색인후ms < 색인전ms);
```

출력 색인이 더 빠른가: true

```
console.log("10배 이상 빨라졌나:", 색인전ms / 색인후ms > 10);
```

출력 10배 이상 빨라졌나: true

계획도 바뀌었습니다.

```
const 색인후계획 = await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 12345", "
(ANALYZE)");
```

```
console.log(색인후계획);
```

출력 Bitmap Heap Scan on "점검기록" (cost=4.37..42.65 rows=10 width=72)
값이 다를 수 있음 (actual time=0.083..0.149 rows=10.00 loops=1)
 Recheck Cond: ("설비번호" = 12345)
 Heap Blocks: exact=10
 Buffers: shared hit=12
 -> Bitmap Index Scan on "점검_설비번호" (cost=0.00..4.37 rows=10
width=0) (actual time=0.060..0.061 rows=10.00 loops=1)
 Index Cond: ("설비번호" = 12345)
 Index Searches: 1
 Buffers: shared hit=2
Planning Time: 0.182 ms
Execution Time: 0.425 ms

★★ **Buffers: shared hit=12** — **12** 쪽입니다. 색인 없을 때는 2642 쪽이었습니다. 220분의 1 입니다.
그리고 Rows Removed by Filter 줄이 **사라졌습니다**. 버린 줄이 없습니다.

★★★ 그런데 왜 시간은 220배가 아니라 수십 배만 빨라졌을까요? 재기() 로 켜 값에는 **자바스크립트에서 WASM 으로 오가는 왕복 비용**(0.4ms 쯤)이 섞여 있습니다. 실제 일이 0.05ms 로 줄어도 겉으로 켜 값은 0.4ms 밑으로 안 내려갑니다. **데이터베이스가 한 일만** 보려면 계획의 **Execution Time** 을 봐야 합니다.

```
const 실행시간뽑기 = (계획글) => Number(계획글.match(/Execution Time: ([\d.]+) ms/)[1]);

const 색인전실행 = 실행시간뽑기(await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 12345", "(ANALYZE)"));

await db.exec("SET enable_bitmapscan = off; SET enable_indexscan = off;");
const 훑기실행 = 실행시간뽑기(await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 12345", "(ANALYZE)"));
await db.exec("SET enable_bitmapscan = on; SET enable_indexscan = on;");

console.log(`Execution Time - 순차 훑기 ${훑기실행.toFixed(3)} ms / 색인 ${색인전실행.toFixed(3)} ms`);
```

출력 Execution Time - 순차 훑기 27.850 ms / 색인 0.054 ms
값이 다를 수 있음

```
console.log(`데이터베이스가 한 일만 보면 몇 배: ${(훑기실행 / 색인전실행).toFixed(0)} 배`);
```

출력 데이터베이스가 한 일만 보면 몇 배: 516배
값이 다를 수 있음

```
console.log("100배 이상인가:", 훑기실행 / 색인전실행 > 100);
```

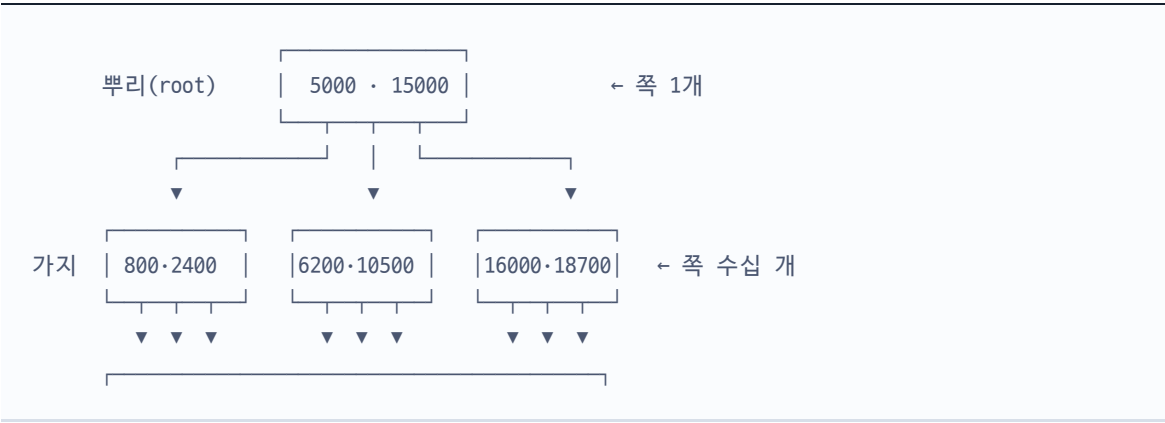
출력 100배 이상인가: true

★ 색인 효과를 켈 때는 **Execution Time** 이나 **Buffers** 로 재세요. 벽시계 시간에는 연결·왕복 비용이 섞여서 효과가 작아 보입니다.

★ B-tree 는 어떻게 생겼나

섹션 4

PostgreSQL 의 기본 색인은 **B-tree** 입니다. (B 는 balanced, 균형) 정렬된 값이 층층이 쌓인 나무입니다.



앞 | 실제 값 + 표의 어느 쪽 몇 번째 줄인지 | ← 쪽 수백 개



12345 를 찾는 과정:

① 뿌리에서 본다: 12345 는 5000 과 15000 사이 → 가운데 가지로 ② 가지에서 본다: 12345 는 10500 보다 크다 → 오른쪽 앞으로 ③ 앞에서 찾는다: 12345 → 표의 1580쪽 7번째 줄 ④ 표의 그 쪽만 읽는다

쪽 네 개를 읽었습니다. 2642 쪽이 아니라 4 쪽입니다.

★★ 왜 이렇게 얇은가 — 한 쪽(8KB)에 색인 항목이 500개쯤 들어갑니다.

깊이 2 → 500 × 500 = 25만 개 / 깊이 3 → 1억 2500만 개

1억 건짜리 표도 깊이 3~4 입니다. $\log_{500}(1\text{억}) \approx 3$ 이니까요.

말로만 하면 안 믿습니다. 규모를 1000배로 늘려 가며 실제로 몇 쪽을 읽는지 켓니다.

```
await db.exec("CREATE TABLE 깊이실험 (id INT PRIMARY KEY)");

for (const 건수 of [1000, 20000, 200000, 1000000]) {
  await db.exec("DROP TABLE 깊이실험");
  await db.exec("CREATE TABLE 깊이실험 (id INT PRIMARY KEY)");
  await db.exec(`INSERT INTO 깊이실험 SELECT i FROM generate_series(1, ${건수}) AS i`);
}
```

★ VACUUM 은 트랜잭션 안에서 못 돛니다. db.exec 에 여러 문장을 넣으면 트랜잭션으로 묶이므로 반드시 따로 부릅니다. 묶어서 부르면: "VACUUM cannot run inside a transaction block"

```
await db.exec("VACUUM ANALYZE 깊이실험");

await db.query("SELECT id FROM 깊이실험 WHERE id = 777"); // 예열 const 잔계획 =
await 계획("SELECT id FROM 깊이실험 WHERE id = 777", "(ANALYZE)");
const 읽은쪽 = 잔계획.match(/Buffers: shared hit=(\d+)/)[1];
const 색인쪽 = (await db.query("SELECT relpages FROM pg_class WHERE relname =
'깊이실험_pkey'")).rows[0].relpages;

console.log(`${건수.toLocaleString()} 건 · 색인 ${색인쪽} 쪽 · 한 건 찾는 데 읽은
쪽 ${읽은쪽}`);
}
```

출력	1,000 건 · 색인 5 쪽 · 한 건 찾는 데 읽은 쪽 3
깊이 다를 수 있음	20,000 건 · 색인 57 쪽 · 한 건 찾는 데 읽은 쪽 3
	200,000 건 · 색인 551 쪽 · 한 건 찾는 데 읽은 쪽 4
	1,000,000 건 · 색인 2745 쪽 · 한 건 찾는 데 읽은 쪽 4

★★★ 데이터가 1000배 늘었는데 읽은 쪽은 3 → 4 로 하나 늘었습니다.

순차 훑기였다면 1000배 느려졌을 일입니다. 이게 $O(n)$ 과 $O(\log n)$ 의 차이입니다. 이 표 한 줄이 색인의 전부입니다.

★ 그래서 “데이터가 늘어날수록 색인의 값어치가 커집니다.” 100건짜리 표에 색인을 걸면 별 차이가 없고, 1000만 건짜리 표에 색인을 안 걸면 서비스가 멈춥니다.

★ ANALYZE — 통계가 없으면 계획이 틀립니다

섹션 5

플래너는 데이터를 직접 세지 않습니다. 통계를 보고 짐작합니다. 그 통계를 모으는 명령이 ANALYZE 입니다.

통계가 낡으면 어떻게 되는지 만들어 봅니다. ‘고장’ 이 스무 건뿐일 때 통계를 모아 두고, 그 뒤에 90% 를 고장으로 바꿉니다.

```
await db.exec(`
  CREATE TABLE 편향 (id SERIAL PRIMARY KEY, 상태 TEXT NOT NULL, 비교 TEXT);
  INSERT INTO 편향 (상태, 비교)
  SELECT CASE WHEN i <= 20 THEN '고장' ELSE '정상' END, '점검 특이사항 없음 기록번호
  ' || i
  FROM generate_series(1, 200000) AS i;
  CREATE INDEX 편향_상태 ON 편향 (상태);
  ANALYZE 편향;
`);
```

이 시점의 통계: 고장 20건 (0.01%)

```
await db.exec("UPDATE 편향 SET 상태 = '고장' WHERE id % 10 <> 0");
```

이제 진짜로는 고장 18만 건 (90%). 그런데 ANALYZE 를 안 했습니다.

```
const 편향질의 = "SELECT sum(length(비고)) FROM 편향 WHERE 상태 = '고장'";
const 낚은계획 = await 계획(편향질의, "(ANALYZE)");

console.log(낚은계획.split("\n").slice(2, 4).join("\n"));
```

```
출력          -> Index Scan using "편향_상태" on "편향" (cost=0.42..9.31 rows=51
값이 다를 수 있음 width=46) (actual time=0.050..62.759 rows=180002.00 loops=1)
                  Index Cond: ("상태" = '고장'::text)
```

계획에서 색인 스캔 줄만 뽑아 추정치와 실제값을 건춥니다.

```
const 색인줄 = 낚은계획.split("\n").find((줄) => 줄.includes("Index Scan"));
const 추정 = Number(색인줄.match(/rows=(\d+) width/)[1]);
const 실제 = Number(색인줄.match(/actual time=[\d.]+\.[\d.]+ rows=(\d+)/)[1]);

console.log(`플래너 추정 ${추정} 줄 / 실제 ${실제} 줄 → ${Math.round(실제 /
추정).toLocaleString()}배 어긋남`);
```

```
출력          플래너 추정 51 줄 / 실제 180002 줄 → 3,529배 어긋남
값이 다를 수 있음
```

```
console.log("추정이 1000배 이상 어긋났나:", 실제 / 추정 > 1000);
```

```
출력          추정이 1000배 이상 어긋났나: true
```

```
await db.exec("ANALYZE 편향");

const 새계획 = await 계획(편향질의, "(ANALYZE)");

console.log(새계획.split("\n").slice(2, 4).join("\n"));
```

```
출력          -> Seq Scan on "편향" (cost=0.00..6799.00 rows=180520 width=46)
값이 다를 수 있음 (actual time=0.452..79.591 rows=180002.00 loops=1)
                  Filter: ("상태" = '고장'::text)
```

```
console.log("ANALYZE 전에는 색인을 썼나:", 낚은계획.includes("Index Scan"));
```

```
출력          ANALYZE 전에는 색인을 썼나: true
```

```
console.log("ANALYZE 후에는 순차 훑기로 바뀌었나:", 새계획.includes("Seq Scan"));
```

```
출력          ANALYZE 후에는 순차 훑기로 바뀌었나: true
```

★★ 계획이 완전히 뒤집혔습니다.

통계가 “고장은 수십 줄뿐” 이라고 하니 플래너는 색인으로 콧 집으려 했습니다. 실제로는 18만 줄이었습니다. 18만 번을 색인으로 왔다 갔다 한 것입니다.

★★★ 여기서 정직하게 말씀드릴 것이 있습니다. PGlite 에서는 이 두 계획의 시간 차이가 거의 안 납니다. 데이터가 전부 메모리에 있어서 “무작위로 왔다 갔다” 하는 비용이 거의 0 이기 때문입니다. 진짜 서버는 디스크에서 읽습니다. 18만 번 무작위 접근은 순서대로 읽기보다 수십 배 비쌉니다. 그래서 이게 운영에서 사고가 됩니다.

★ 그러니 이 실습에서 봐야 할 것은 시간이 아니라 **rows 추정치와 actual rows** 의 차이입니다.

수천 배 어긋났다는 것 - 그게 신호입니다. (개념03 에서 자세히 봅니다)

★ 실무에서는 autovacuum 이 알아서 ANALYZE 를 돌립니다. 문제는 **대량으로 넣거나 고친 직후**입니다. autovacuum 이 오기 전까지는 통계가 낡아 있습니다. 대량 작업 뒤에는 손으로 ANALYZE 표 이름; 을 돌리세요. (10단원에서 운영 이야기를 합니다)

★★ 색인의 대가 ㉠ — 넣고 고치는 게 느려집니다

섹션 6

색인은 표와 **따로** 관리됩니다. 표에 한 줄 넣으면 색인에도 넣어야 합니다. 색인이 다섯 개면 다섯 번 더 씁니다. 똑같이 생긴 표 둘로 재 봅니다.

```
const 표만들기 = (이름) => `
  CREATE TABLE ${이름} (
    id SERIAL PRIMARY KEY, 설비번호 INT, 라인 TEXT, 상태 TEXT, 점검자 TEXT, 점검일
    DATE
  );

await db.exec(표만들기("색인없는표"));
await db.exec(표만들기("색인다섯표"));

await db.exec(`
  CREATE INDEX 다섯_1 ON 색인다섯표 (설비번호);
  CREATE INDEX 다섯_2 ON 색인다섯표 (라인);
  CREATE INDEX 다섯_3 ON 색인다섯표 (상태);
  CREATE INDEX 다섯_4 ON 색인다섯표 (점검자);
  CREATE INDEX 다섯_5 ON 색인다섯표 (점검일);
`);

const 십만건넣기 = (표) => `
  INSERT INTO ${표} (설비번호, 라인, 상태, 점검자, 점검일)
  SELECT (i % 20000) + 1,
    (ARRAY['A','B','C','D'])[ (i % 4) + 1],
    (ARRAY['정상','주의','고장'])[ (i % 3) + 1],
    '점검원' || ((i % 50) + 1),
```

```
DATE '2024-01-01' + ((i % 365) || ' days')::interval
FROM generate_series(1, 100000) AS i`;
```

```
const 없이시작 = performance.now();
await db.exec(십만건넣기("색인없는표"));
const 없이ms = performance.now() - 없이시작;
```

```
const 다섯시작 = performance.now();
await db.exec(십만건넣기("색인다섯표"));
const 다섯ms = performance.now() - 다섯시작;
```

```
console.log(`10만 건 INSERT - 색인 0개: ${없이ms.toFixed(0)} ms / 색인 5개:
${다섯ms.toFixed(0)} ms`);
```

```
출력          10만 건 INSERT - 색인 0개: 559 ms / 색인 5개: 1280 ms
값이 다를 수 있음
```

```
console.log(`색인 5개가 몇 배 느린가: ${다섯ms / 없이ms}.toFixed(2)}배`);
```

```
출력          색인 5개가 몇 배 느린가: 2.29배
값이 다를 수 있음
```

```
console.log("색인이 있으면 넣기가 느려지나:", 다섯ms > 없이ms);
```

```
출력          색인이 있으면 넣기가 느려지나: true
```

UPDATE 도 봅니다. 색인이 걸린 칸을 고치면 색인도 고쳐야 합니다.

```
const u없이 = await 재기(5, (회차) =>
  db.query("UPDATE 색인없는표 SET 상태 = '점검' WHERE id BETWEEN $1 AND $1 + 999",
    [회차 * 2000 + 1]),
);
const u다섯 = await 재기(5, (회차) =>
  db.query("UPDATE 색인다섯표 SET 상태 = '점검' WHERE id BETWEEN $1 AND $1 + 999",
    [회차 * 2000 + 1]),
);
```

```
console.log(`1000건 UPDATE - 색인 0개: ${u없이.toFixed(1)} ms / 색인 5개:
${u다섯.toFixed(1)} ms`);
```

```
출력          1000건 UPDATE - 색인 0개: 4.9 ms / 색인 5개: 17.8 ms
값이 다를 수 있음
```

```
console.log("색인이 있으면 고치기도 느려지나:", u다섯 > u없이);
```

```
출력          색인이 있으면 고치기도 느려지나: true
```

★★★ 읽기를 사면 쓰기를 팝니다.

점검기록처럼 하루 종일 쌓이는 표에 색인을 열 개 걸면 넣는 게 몇 배 느려집니다. “혹시 쓸까 봐” 걸어 둔 색인 하나하나가 INSERT 마다 값을 치릅니다.

색인의 대가 ② — 자리를 먹습니다

섹션 7

```
const 크기 = (  
  await db.query(`  
    SELECT pg_relation_size('색인다섯표') AS 표, pg_indexes_size('색인다섯표') AS  
    색인들,  
           pg_relation_size('점검_설비번호') AS 하나  
  `)  
)>.rows[0];  
  
console.log(`색인다섯표 - 표 ${메가(크기.표)} / 색인 6개 합계 ${메가(크기.색인들)}`);
```

출력 색인다섯표 - 표 6.77 MB / 색인 6개 합계 7.06 MB
값이 다를 수 있음

```
console.log("색인이 표보다 큰가:", Number(크기.색인들) > Number(크기.표));
```

출력 색인이 표보다 큰가: true

```
console.log(`점검기록의 설비번호 색인 하나: ${메가(크기.하나)}`);
```

출력 점검기록의 설비번호 색인 하나: 1.81 MB
값이 다를 수 있음

★★ 색인 여섯 개(기본키 포함)가 표보다 큼니다.

백업도 그만큼 커지고, 메모리 캐시도 그만큼 차지합니다. “디스크 싸잖아요” 라고 하기 쉬운데, 진짜 비싼 건 **메모리 캐시 자리**입니다. 자주 쓰는 데이터가 캐시에서 밀려나면 전체가 느려집니다.

색인 목록과 크기는 이렇게 봅니다. 운영에서 자주 씁니다.

```
const 색인목록 = (  
  await db.query(`  
    SELECT indexrelname AS 이름, pg_relation_size(indexrelid) AS 바이트  
    FROM pg_stat_user_indexes WHERE relname = '색인다섯표' ORDER BY 이름  
  `)  
)>.rows;  
  
for (const { 이름, 바이트 } of 색인목록) {  
  console.log(`· ${이름} - ${메가(바이트)}`);  
}
```


출력	· 다섯_1 – 1.73 MB
값이 다를 수 있음	· 다섯_2 – 0.67 MB
	· 다섯_3 – 0.69 MB
	· 다섯_4 – 0.80 MB
	· 다섯_5 – 0.89 MB
	· 색인다섯표_pkey – 2.29 MB

UNIQUE 색인과 UNIQUE 제약

섹션 8

UNIQUE 를 걸면 중복을 막으면서 색인도 생깁니다. 하나로 두 가지를 합니다.

```
await db.exec(`
  CREATE TABLE 설비 (
    id          SERIAL PRIMARY KEY,
    관리번호 TEXT NOT NULL UNIQUE,
    이름       TEXT NOT NULL
  );
  INSERT INTO 설비 (관리번호, 이름) VALUES ('EQ-001', '컨베이어 1호');
`);

try {
  await db.query("INSERT INTO 설비 (관리번호, 이름) VALUES ('EQ-001', '가짜 설비')");
} catch (에러) {
  console.log("중복을 막았나:", 에러.code === "23505");
}
```

출력	중복을 막았나: true
----	---------------

```
}

const 설비색인들 = (
  await db.query("SELECT indexname FROM pg_indexes WHERE tablename = '설비' ORDER BY
indexname")
).rows.map((줄) => 줄.indexname);

console.log("설비 표의 색인:", 설비색인들.join(", "));
```

출력	설비 표의 색인: 설비_pkey, 설비_관리번호_key
----	--------------------------------

★ UNIQUE 제약(constraint)과 UNIQUE 색인(index)의 관계 · UNIQUE 제약을 걸면 → PostgreSQL 이 UNIQUE 색인을 자동으로 만듭니다 · CREATE UNIQUE INDEX 로 색인만 만들어도 → 중복은 똑같이 막힙니다 · 다른 점: 제약은 외래키가 참조할 수 있고, 색인만 만든 것은 참조 대상이 못 됩니다 ★ 그래서 UNIQUE 는 제약으로 거는 게 낫습니다. 색인은 따라옵니다.

06

```

await db.exec("CREATE UNIQUE INDEX 설비_이름_유일 ON 설비 (이름)");

try {
  await db.query("INSERT INTO 설비 (관리번호, 이름) VALUES ('EQ-002', '컨베이어 1호')");
} catch (에러) {
  console.log("UNIQUE 색인만으로도 중복이 막히나:", 에러.code === "23505");
}

```

출력 UNIQUE 색인만으로도 중복이 막히나: true

```

}

```

★★ 기본키에는 색인이 생기고, 외래키에는 안 생깁니다

섹션 9

여기서 아주 많이 당합니다. 실제로 확인합니다.

```

await db.exec(`
  CREATE TABLE 점검2 (
    id          SERIAL PRIMARY KEY,
    설비번호 INT NOT NULL REFERENCES 설비(id),
    상태       TEXT NOT NULL
  );
  INSERT INTO 점검2 (설비번호, 상태)
  SELECT 1, (ARRAY['정상', '주의', '고장'][(i % 3) + 1])
  FROM generate_series(1, 100000) AS i;
  ANALYZE 점검2;
`);

const 점검2색인 = (
  await db.query("SELECT indexname FROM pg_indexes WHERE tablename = '점검2' ORDER BY indexname")
).rows.map((줄) => 줄.indexname);

console.log("점검2 의 색인:", 점검2색인.join(", "));

```

출력 점검2 의 색인: 점검2_pkey

```

console.log("기본키 색인이 자동으로 생겼나:", 점검2색인.includes("점검2_pkey"));

```

출력 기본키 색인이 자동으로 생겼나: true

```

console.log("외래키 칸(설비번호)에도 색인이 생겼나:", 점검2색인.some((이름) =>
  이름.includes("설비번호")));

```

출력 외래키 칸(설비번호)에도 색인이 생겼나: false

```
console.log(await 계획("SELECT count(*) FROM 점검2 WHERE 설비번호 = 1"));

출력      Aggregate  (cost=2041.00..2041.01 rows=1 width=8)
값이 다를 수 있음  -> Seq Scan on "점검2"  (cost=0.00..1791.00 rows=100000 width=0)
                        Filter: ("설비번호" = 1)
```

★★★ 외래키를 걸어도 색인은 안 생깁니다.

왜 문제가 되나:

- ① 조인이 느려집니다. 설비 JOIN 점검2 ON 점검2.설비번호 = 설비.id 는 거의 모든 화면에서 씁니다
- ② 부모를 지울 때 자식 표를 전부 훑습니다. DELETE FROM 설비 WHERE id = 1 을 하면

"이 설비를 참조하는 점검2 줄이 있나" 를 확인해야 합니다. 색인이 없으면 통째로 훑습니다.
자식이 1000만 건이면 그대로 멈춥니다.

★ 규칙: 외래키 칸에는 색인을 직접 거세요. 예외는 거의 없습니다.

```
await db.exec("CREATE INDEX 점검2_설비번호 ON 점검2 (설비번호);

const 지우기전 = await 계획("DELETE FROM 설비 WHERE id = 99999");

console.log("색인을 걸면 조인/삭제 확인이 색인을 타나:", !(await 계획("SELECT count(*) FROM 점검2 WHERE 설비번호 = 99999")).includes("Seq Scan"));

출력      색인을 걸면 조인/삭제 확인이 색인을 타나: true

console.log("DELETE 계획도 나오나:", 지우기전.length > 0);

출력      DELETE 계획도 나오나: true
```

MySQL 은 여기가 다릅니다

· MySQL(InnoDB)은 외래키를 만들면 색인을 자동으로 만들어 줍니다. 이걸 다릅니다 · 기본키가 곧 데이터 저장 순서입니다(클러스터드 인덱스). PostgreSQL 은 안 그렇습니다 · ANALYZE TABLE 표 이름; 으로 통계를 모읍니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 색인을 걸면 무엇이 생기고 무엇을 잃나

무엇 재 본 결과

20만 건 한 건 찾기 2642 쪽 → 12 쪽 (220분의 1), Execution Time 400배 1000배 큰 표에서 찾기 읽은 쪽 3 → 4 (하나만 늘어남) 10만 건 INSERT 색인 5개면 2~3배 느려짐 1000건 UPDATE 색인 5개면

2~3배 느려짐 저장 공간 색인 여섯 개가 표보다 큼 통계가 낡으면 추정이 수천 배 어긋나고 계획이 뒤집힘
명령 무엇을 하나

CREATE INDEX 이름 ON 표(칸) 색인을 만든다 CREATE UNIQUE INDEX ... 중복도 막는다 ANALYZE
표이름 통계를 모은다 (대량 작업 뒤에는 꼭)

<code>pg_relation_size('이름')</code>	표나 색인 하나의 크기
<code>pg_indexes_size('표')</code>	그 표의 색인 전부 합친 크기

★★★ 색인은 읽기를 사고 쓰기를 파는 거래입니다. 무엇을 사고 무엇을 파는지 모르고 하면 손해 보는 거래가 됩니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 6 에서 색인 개수를 1개, 3개, 5개, 10개로 바꿔 보세요. INSERT 시간이 색인 개수에 비례하나요?

▫ 직접 해보기 2

섹션 4 의 깊이실험에 500만 건을 추가해 보세요. 읽은 쪽이 5 가 되나요? (몇 초 더 걸립니다)

▫ 직접 해보기 3

색인 이름을 안 주고 `CREATE INDEX ON 점검기록 (라인);` 이라고 해 보세요. 어떤 이름이 붙나요?
(`pg_indexes` 로 확인)

▫ 직접 해보기 4

섹션 6 의 UPDATE 대상 칸을 상태(색인 있음) 대신 색인이 없는 칸으로 바꿔 보세요. 차이가 줄어드나요? ★ 힌트: HOT 갱신이라는 최적화가 있습니다

▫ 직접 해보기 5

`DROP INDEX 점검_설비번호;` 로 지우고 섹션 3 을 다시 재 보세요. 원래 속도로 돌아오나요?

▫ 직접 해보기 6

섹션 9 에서 점검2 를 100만 건으로 늘리고, 색인 없이 `DELETE FROM 설비 WHERE id = 99999` 를
`EXPLAIN ANALYZE` 해 보세요. Trigger 라는 줄이 보이나요? 얼마나 걸리나요?

자주 하는 실수

이런 실수를 합니다

“일단 자주 쓰는 칸에 다 걸어 두자”

섹션 6 에서 재 봤습니다. 색인 다섯 개면 INSERT 가 세 배 가까이 느려집니다. **안 쓰이는 색인은 순수한 손해**입니다. 읽기는 안 빨라지고 쓰기만 느려집니다. 무엇을 안 쓰는지 찾는 법은 개념04 에서 합니다.

이런 실수를 합니다

외래키에 색인이 자동으로 생기는 줄 얕

섹션 9 에서 확인했습니다. **PostgreSQL 은 안 만들어 줍니다.** MySQL 은 만들어 줍니다. 그래서 MySQL 하다 온 사람이 특히 잘 당합니다.

이런 실수를 합니다

대량으로 넣고 ANALYZE 를 안 함

섹션 5 에서 봤습니다. 통계가 낡으면 추정이 수천 배 어긋납니다. 대량 INSERT/UPDATE/DELETE 뒤에는 **ANALYZE** 표 이름; 을 돌려주세요.

이런 실수를 합니다

VACUUM 을 db.exec 에 다른 문장과 같이 넣음

섹션 4 의 주석대로 "VACUUM cannot run inside a transaction block" 이 납니다. VACUUM 은 항상 혼자 부르세요.

이런 실수를 합니다

색인을 걸어 두고 빨라졌는지 안 재 봄

섹션 3 처럼 **전/후를 같은 방법으로** 재세요. 그리고 시간이 아니라 **Buffers** 의 쪽 수를 보세요. 시간은 기계마다 다릅니다.

이런 실수를 합니다

색인 크기를 무시함

섹션 7 에서 색인 합계가 표보다 컸습니다. 백업 시간, 복구 시간, 메모리 캐시가 전부 그만큼 늘어납니다.

```
await db.close();
```

실행계획 읽는 법

RUN node 개념03_실행계획_읽는_법.js

★ 이 파일은 30초쯤 걸립니다.

개념02 에서 색인을 걸고 빨라진 것을 확인했습니다. 그런데 색인을 걸어도 안 빨라지는 날이 옵니다. 반드시 옵니다.

그때 필요한 것이 **실행계획을 읽는** 눈입니다. 이 파일에서 다섯 가지를 합니다.

① EXPLAIN 과 EXPLAIN ANALYZE 의 차이 — 그리고 위험한 점 ② 출력 한 줄을 토막 내서 전부 해석하기 ③ ★★ 추정과 실재가 어긋나는 것 — 문제의 신호 ④ 스캔 네 종류를 직접 만들어 보기 ⑤ 조인 세 종류와, 데이터가 커지면 계획이 바뀌는 것

★ 계획을 읽는 순서를 먼저 알려 드립니다. **안쪽부터, 들여쓰기가 깊은 것부터 읽습니다.** 맨 위 줄이 마지막에 일어나는 일입니다. 거꾸로 읽으세요.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();

async function 계획(sql, 옵션 = "") {
  const 결과 = await db.query(`EXPLAIN ${옵션} ${sql}`);
  return 결과.rows.map((줄) => 줄["QUERY PLAN"]).join("\n");
}
```

계획에서 스캔·조인 노드 이름만 뽑습니다. 무엇을 골랐는지 한눈에 보려고요.

```
function 노드들(계획글) {
  return 계획글
    .split("\n")
    .map((줄) => 줄.match(/^s*(?:->s+)?([A-Z][A-Za-z ]+)?(?:s+(?:on|using)s|s+(cost))/))
    .filter(Boolean)
    .map((맞음) => 맞음[1].trim())
    .join(" / ");
}
```

```
await db.exec(`
  CREATE TABLE 설비 (
    id      SERIAL PRIMARY KEY,
    이름    TEXT NOT NULL,
    라인    TEXT NOT NULL,
    상태    TEXT NOT NULL
  );
  INSERT INTO 설비 (이름, 라인, 상태)
  SELECT '설비' || i, (ARRAY['A','B','C','D'][(i % 4) + 1],
  (ARRAY['가동','정지','점검'][(i % 3) + 1]
  FROM generate_series(1, 20000) AS i;

  CREATE TABLE 점검기록 (
    id      SERIAL PRIMARY KEY,
    설비번호 INT NOT NULL,
    상태    TEXT NOT NULL,
    점검자  TEXT NOT NULL,
    점검일  DATE NOT NULL,
    비고    TEXT
  );
  INSERT INTO 점검기록 (설비번호, 상태, 점검자, 점검일, 비고)
  SELECT (i % 20000) + 1,
         (ARRAY['정상','주의','고장'][(i % 3) + 1],
         '점검원' || ((i % 50) + 1),
         DATE '2024-01-01' + ((i % 365) || ' days')::interval,
         '특이사항 없음. 기록번호 ' || i
  FROM generate_series(1, 200000) AS i;

  CREATE INDEX 점검_설비번호 ON 점검기록 (설비번호);
  ANALYZE;
`);
```

EXPLAIN 과 EXPLAIN ANALYZE

EXPLAIN — 계획만 세워서 보여 줍니다. **실행하지 않습니다**. EXPLAIN ANALYZE — **진짜로 실행합니다**. 그리고 실제 값까지 붙여 줍니다.

```
console.log(await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 777"));
```

```
출력      Bitmap Heap Scan on "점검기록" (cost=4.37..42.63 rows=10 width=70)
값이 다를 수 있음      Recheck Cond: ("설비번호" = 777)
                        -> Bitmap Index Scan on "점검_설비번호" (cost=0.00..4.37 rows=10
                        width=0)
                        Index Cond: ("설비번호" = 777)
```

```
console.log(await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 777", "(ANALYZE)"));
```

```
출력      Bitmap Heap Scan on "점검기록" (cost=4.37..42.63 rows=10 width=70)
값이 다를 수 있음      (actual time=0.121..0.161 rows=10.00 loops=1)
                        Recheck Cond: ("설비번호" = 777)
                        Heap Blocks: exact=10
                        Buffers: shared hit=10 read=2
                        -> Bitmap Index Scan on "점검_설비번호" (cost=0.00..4.37 rows=10
                        width=0) (actual time=0.067..0.067 rows=10.00 loops=1)
                        Index Cond: ("설비번호" = 777)
                        Index Searches: 1
                        Buffers: shared read=2
                        Planning Time: 0.061 ms
                        Execution Time: 0.295 ms
```

★★★ ANALYZE 는 진짜로 실행합니다.

SELECT 면 상관없습니다. 그런데 UPDATE 나 DELETE 에 붙이면 **진짜로 고치고 진짜로 지웁니다.** 계획만 보려던 건데 데이터가 바뀝니다.

실제로 운영 데이터베이스에서 이걸로 사고가 납니다. "계획만 보려고 했는데요" 는 변명이 안 됩니다.

★ 안전하게 보는 법: 트랜잭션으로 감싸고 되돌립니다.

```
const 지우기전건수 = (await db.query("SELECT count(*) AS 수 FROM 점검기록")).rows[0].수;

await db.exec("BEGIN");
const 지우기계획 = await 계획("DELETE FROM 점검기록 WHERE 상태 = '정상'", "(ANALYZE)");
const 되돌리기전건수 = (await db.query("SELECT count(*) AS 수 FROM 점검기록")).rows[0].수;
await db.exec("ROLLBACK");

const 되돌린뒤건수 = (await db.query("SELECT count(*) AS 수 FROM 점검기록")).rows[0].수;

console.log(`ROLLBACK 전: ${되돌리기전건수} 건 / 원래: ${지우기전건수} 건`);
```

```
출력      ROLLBACK 전: 133334 건 / 원래: 200000 건
```



```
console.log("EXPLAIN ANALYZE 만으로도 진짜 지워졌나:", 되돌리기전건수 <
지우기전건수);
```

```
출력      EXPLAIN ANALYZE 만으로도 진짜 지워졌나: true
```

```
console.log("ROLLBACK 으로 되살아났나:", 되돌린뒤건수 === 지우기전건수);
```

```
출력      ROLLBACK 으로 되살아났나: true
```

```
console.log("계획에 Delete 노드가 있나:", 지우기계획.includes("Delete on"));
```

```
출력      계획에 Delete 노드가 있나: true
```

★★ 6만 6천 건이 진짜로 지워졌습니다. ROLLBACK 을 안 했으면 그대로 남아있습니다. **UPDATE / DELETE / INSERT** 에 **EXPLAIN ANALYZE** 를 쓸 때는 반드시

```
BEGIN;
EXPLAIN (ANALYZE) DELETE ...;
ROLLBACK;
```

순서로 하세요. 예외 없습니다.

★ 출력 한 줄을 토막 내기

섹션 2

이 한 줄에 여섯 가지 정보가 들어 있습니다.

Bitmap Heap Scan on "점검기록" (cost=4.37..42.65 rows=10 width=72)

```
(actual time=0.088..0.110 rows=10.00 loops=1)
```

```
└─────────── ① ───────────┐ └── ② ───┐ └── ③ ─┐ └── ④ ─┐
```

```
└────────── ⑤ ─────────┐ └── ⑥ ─┐
```

① 노드 이름 + 대상 — 무엇을 어떻게 읽나

② cost=4.37..42.65

플래너의 ****비용 점수****. 밀리초가 아닙니다.

4.37 = 첫 줄이 나오기까지 드는 비용 (시작 비용)

42.65 = 마지막 줄까지 드는 비용 (총 비용)

* 기준은 "꼭 하나를 순서대로 읽는 비용 = 1.0" 입니다.

* LIMIT 이 붙으면 시작 비용이 중요해집니다. 빨리 첫 줄을 내는 계획이 이깁니다.

③ rows=10

****추정**** 결과 줄 수. 통계에서 계산한 짐작입니다.

④ width=72

한 줄의 평균 바이트. 가져오는 칸만 셉니다.
SELECT * 대신 필요한 칸만 고르면 이 값이 줄어듭니다.

⑤ actual time=0.088..0.110 rows=10.00

****실제**** 값. 첫 줄까지 0.088ms, 끝까지 0.110ms, 실제로 10줄.
★ ANALYZE 를 붙였을 때만 나옵니다.

⑥ loops=1

이 노드가 ****몇 번 실행됐나.**** 1이면 한 번입니다.
★ 조인 안쪽에 들어가면 이 값이 커집니다. 아래에서 봅니다.
★ 그리고 ``actual time`` 은 ****한 번 실행할 때의 평균****입니다.
총 시간을 알려면 `actual time × loops` 를 해야 합니다. 여기서 많이 착각합니다.

loops 가 1이 아닌 계획을 만들어 봅니다.

```
const 반복계획 = await 계획(  
  `SELECT s.이름, p.상태  
  FROM 설비 s JOIN 점검기록 p ON p.설비번호 = s.id  
  WHERE s.id BETWEEN 1 AND 50`,  
  "(ANALYZE)",  
);  
  
console.log(반복계획.split("\n").filter((줄) => 줄.includes("loops=")).join("\n"));
```

출력	Nested Loop (cost=0.58..2032.45 rows=500 width=18) (actual
값이 다를 수 있음	time=0.123..1.633 rows=500.00 loops=1)
	-> Index Scan using "설비_pkey" on "설비" s (cost=0.29..9.29
	rows=50 width=15) (actual time=0.094..0.116 rows=50.00 loops=1)
	-> Index Scan using "점검_설비번호" on "점검기록" p
	(cost=0.29..40.36 rows=10 width=11) (actual time=0.002..0.023
	rows=10.00 loops=50)

★★ 마지막 줄의 `loops=50` 을 보세요. 안쪽 색인 스캔이 **50번** 돌았습니다. 바깥에서 설비 50대가 나왔으니까요. `actual time=0.008..0.011` 은 **한 번당** 값입니다. 총 시간은 $0.011 \times 50 = 0.55\text{ms}$ 쯤입니다.

★ 느린 질의를 볼 때 `loops` 가 큰 노드를 먼저 의심하세요.

한 번은 0.01ms 라도 10000번이면 100ms 입니다.

★★ 추정과 실재가 어긋나면 그게 신호입니다

섹션 3

플래너는 통계로 짐작합니다. 짐작이 틀리면 계획도 틀립니다. 짐작이 얼마나 틀렸는지는 `rows=` 와 `actual ... rows=` 를 비교하면 보입니다.

일부러 어긋나게 만들어 봅니다. 라인이 100개 있고, 라인마다 **라인장이 딱 한 명씩** 있는 표입니다. 사람 눈에는 “라인을 알면 라인장도 안다” 가 뻔한데, 플래너는 그걸 모릅니다.

```
await db.exec(`
  CREATE TABLE 라인별기록 (id SERIAL PRIMARY KEY, 라인 TEXT NOT NULL, 라인장 TEXT NOT NULL);
  INSERT INTO 라인별기록 (라인, 라인장)
  SELECT 'L' || lpad(((i % 100) + 1)::text, 3, '0'),
        '반장' || lpad(((i % 100) + 1)::text, 3, '0')
  FROM generate_series(1, 100000) AS i;
  ANALYZE 라인별기록;
`);
```

라인 L007 은 항상 반장007 입니다. 둘 다 걸어도 결과는 하나도 안 줄어듭니다.

```
const 어긋난계획 = await 계획(
  "SELECT count(*) FROM 라인별기록 WHERE 라인 = 'L007' AND 라인장 = '반장007'",
  "(ANALYZE)",
);
const 스캔줄 = 어긋난계획.split("\n").find((줄) => 줄.includes("Seq Scan"));

console.log(스캔줄.trim());
```

```
출력          -> Seq Scan on "라인별기록" (cost=0.00..2137.00 rows=10 width=0)
값이 다를 수 있음 (actual time=0.016..13.053 rows=1000.00 loops=1)
```

```
const 추정줄수 = Number(스캔줄.match(/rows=(\d+) width/)[1]);
const 실제줄수 = Number(스캔줄.match(/actual time=[\d.]+\.[\d.]+ rows=(\d.+)/)[1]);

console.log(`추정 ${추정줄수} 줄 / 실제 ${실제줄수} 줄 → ${Math.round(실제줄수 / 추정줄수)}배 어긋남`);
```

```
출력          추정 10 줄 / 실제 1000 줄 → 100배 어긋남
값이 다를 수 있음
```

```
console.log("50배 이상 어긋났나:", 실제줄수 / 추정줄수 > 50);
```

```
출력          50배 이상 어긋났나: true
```

★★★ 왜 이렇게 어긋나나.

플래너는 각 칸을 **따로따로** 봅니다. 그리고 곱합니다.

```
라인 = 'L007'      → 100분의 1
라인장 = '반장007' → 100분의 1
둘 다니까 → 100000 × (1/100) × (1/100) = **10 줄**
```

실제로는 라인이 L007 이면 라인장은 무조건 반장007 입니다. 하나도 안 줄어듭니다. 그래서 1000 줄이 나옵니다.

플래너가 왜 모르나: **칸들이 서로 독립이라고 가정**하기 때문입니다. 현실의 표에는 이런 짝(우편번호-시, 모델명-제조사, 라인-공장)이 아주 많습니다.

★★ 이게 왜 큰 문제인가: “10줄쯤 나오겠지” 하고 세운 계획이 실제로는 1000줄을 다룹니다. 조인 안쪽에 들어가면 10번 예상이 1000번 반복으로 바뀝니다.

★ 느린 질의를 볼 때 가장 먼저 볼 것이 이 어긋남입니다.

rows 와 actual rows 가 크게 벌어진 노드를 찾으세요. 거기가 범인입니다.

★ 고치는 법 (PostgreSQL 10 이상)

```
CREATE STATISTICS 이름 (dependencies) ON 라인, 라인장 FROM 라인별기록;
ANALYZE 라인별기록;
```

“이 두 칸은 관계가 있다” 고 알려 주는 것입니다. 실제로 해 봅니다.

```
await db.exec("CREATE STATISTICS 라인_라인장 (dependencies) ON 라인, 라인장 FROM
라인별기록");
await db.exec("ANALYZE 라인별기록");

const 고친계획 = await 계획(
  "SELECT count(*) FROM 라인별기록 WHERE 라인 = 'L007' AND 라인장 = '반장007'",
  "(ANALYZE)",
);
const 고친줄 = 고친계획.split("\n").find((줄) => 줄.includes("Seq Scan"));
const 고친추정 = Number(고친줄.match(/rows=(\d+) width/)[1]);

console.log(`확장 통계를 준 뒤 추정: ${고친추정} 줄 (실제 ${실제줄수} 줄)`);
```

출력 확장 통계를 준 뒤 추정: 1030 줄 (실제 1000 줄)
값이 다를 수 있음

```
console.log("추정이 실제에 훨씬 가까워졌나:", Math.abs(고친추정 - 실제줄수) <
Math.abs(추정줄수 - 실제줄수));
```

출력 추정이 실제에 훨씬 가까워졌나: true

스캔 네 종류를 직접 만들어 봅니다

섹션 4

① Seq Scan — 색인이 없거나, 있어도 대부분을 읽을 때

```
console.log("① 색인 없는 칸:", 노드들(await 계획("SELECT * FROM 점검기록 WHERE 점검자 = '점검원7'")));
```

출력 ① 색인 없는 칸: Seq Scan

② Index Scan — 색인으로 콕 집고, 나오는 줄이 아주 적을 때 ★ 기본키처럼 **유일한** 값을 찾으면 거의 항상 이겁니다.

```
console.log("② 기본키로 한 건:", 노드들(await 계획("SELECT * FROM 점검기록 WHERE id = 12345")));
```

출력 ② 기본키로 한 건: Index Scan

- 1 색인을 훑어서 “표의 몇 쪽 몇 번째” 목록을 **비트맵**으로 모읍니다
- 2 그 비트맵을 **쪽 번호 순서로 정렬**합니다
- 3 표를 쪽 번호 순서대로 한 번에 읽습니다 → 왔다 갔다 하지 않고 순서대로 읽으니 디스크에 훨씬 친절합니다.

```
console.log("③ 색인으로 열 줄:", 노드들(await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 777")));
```

출력 ③ 색인으로 열 줄: Bitmap Heap Scan / Bitmap Index Scan

```
console.log("④ 색인으로 삼천 줄:", 노드들(await 계획("SELECT * FROM 점검기록 WHERE 설비번호 BETWEEN 1 AND 300")));
```

출력 ④ 색인으로 삼천 줄: Bitmap Heap Scan / Bitmap Index Scan

★ **Recheck Cond** 가 왜 있나 비트맵이 너무 커지면 PostgreSQL 은 “줄 단위” 대신 “쪽 단위” 로 기억합니다(lossy). 그러면 그 쪽 안의 어느 줄인지 모르니 다시 조건을 확인해야 합니다. 그게 Recheck 입니다.

★ Index Only Scan — 표를 아예 안 봅니다

섹션 5

지금까지의 색인 스캔은 전부 두 단계였습니다.

① 색인에서 “표의 몇 쪽 몇 번째 줄” 을 찾는다 ② **표로 가서 그 줄을 읽는다** ← 이게 비쌉니다

②를 안 해도 되는 경우가 있습니다. **필요한 칸이 전부 색인 안에 있을 때** 입니다. 그런 색인을 **커버링 색인 (covering index)** 이라고 합니다.

```
await db.exec("CREATE INDEX 점검_커버 ON 점검기록 (설비번호) INCLUDE (상태, 점검일);");
await db.exec("DROP INDEX 점검_설비번호");
await db.exec("ANALYZE 점검기록");

const 진공전 = await 계획("SELECT 설비번호, 상태 FROM 점검기록 WHERE 설비번호 = 777", "(ANALYZE)");

console.log("VACUUM 전:", 노드들(진공전));
```

출력 VACUUM 전: Bitmap Heap Scan / Bitmap Index Scan

★★ 커버링 색인을 만들었는데도 Index Only Scan 이 안 나왔습니다. 왜일까요?

색인에는 **그 줄이 지금 보여도 되는 줄인지** 가 안 적혀 있습니다. (누가 지웠는데 아직 정리가 안 된 줄일 수 있습니다 — 07단원에서 다룹니다) 그래서 PostgreSQL 은 **가시성 지도(visibility map)** 라는 걸 따로 봅니다. 그 지도는 VACUUM 이 채웁니다. VACUUM 을 한 번도 안 했으면 지도가 비어 있습니다.

```
await db.exec("VACUUM ANALYZE 점검기록");

const 진공후 = await 계획("SELECT 설비번호, 상태 FROM 점검기록 WHERE 설비번호 = 777", "(ANALYZE)");

console.log("VACUUM 후:", 노드들(진공후));
```

출력 VACUUM 후: Index Only Scan

```
console.log(진공후.split("\n").slice(0, 5).join("\n"));
```

출력
값이 다를 수 있음

```
Index Only Scan using "점검_커버" on "점검기록" (cost=0.42..4.59
rows=10 width=11) (actual time=0.114..0.119 rows=10.00 loops=1)
  Index Cond: ("설비번호" = 777)
  Heap Fetches: 0
  Index Searches: 1
  Buffers: shared hit=4
```

```
const 쪽수뽑기 = (계획글) => Number(계획글.match(/Buffers: shared hit=(\d+)/)[1]);

console.log(`읽은 쪽 - 표까지 보는 경우 ${쪽수뽑기(진공전)} / 색인만 보는 경우
${쪽수뽑기(진공후)}`);
```

출력
값이 다를 수 있음

읽은 쪽 - 표까지 보는 경우 10 / 색인만 보는 경우 4

```
console.log("색인만 보는 쪽이 더 적은 쪽을 읽나:", 쪽수뽑기(진공후) < 쪽수뽑기(진공전));
```

```
출력      색인만 보는 쪽이 더 적은 쪽을 읽나: true
```

```
console.log("Heap Fetches 가 0 인가:", 진공후.includes("Heap Fetches: 0"));
```

```
출력      Heap Fetches 가 0 인가: true
```

★★ Heap Fetches: 0 — 표를 한 번도 안 봤습니다.

커버링 색인 만드는 법 두 가지:

```
CREATE INDEX ... ON 점검기록 (설비번호, 상태);          -- 둘 다 찾기에 쓸 수 있음
CREATE INDEX ... ON 점검기록 (설비번호) INCLUDE (상태); -- 상태는 '들고만 있음'
```

INCLUDE 쪽이 색인이 조금 더 작고, 정렬 순서에 영향을 안 줍니다. 찾는 조건에 안 쓰고 **결과로만 필요한** 칸은 INCLUDE 에 넣으세요.

★ 대가: 색인이 커집니다. 그리고 그 칸을 고칠 때마다 색인도 고쳐야 합니다.

조인 세 종류

섹션 6

두 표를 잇는 방법은 세 가지입니다. 플래너가 **크기를 보고** 고릅니다.

Nested Loop — 바깥 표를 한 줄씩 돌면서, 안쪽 표를 그때그때 찾는다

```
바깥이 작을 때 최고. 안쪽에 색인이 있어야 함
```

Hash Join — 작은 쪽을 통째로 메모리에 해시표로 올려놓고,

```
큰 쪽을 한 번 훑으면서 맞춰 본다. 둘 다 클 때 최고
```

Merge Join — 양쪽을 정렬해 놓고 지퍼 채우듯 맞춰 나간다

```
이미 정렬돼 있거나(색인) 결과가 정렬돼 있어야 할 때
```

```
await db.exec("CREATE INDEX 점검_설비번호2 ON 점검기록 (설비번호); ANALYZE;");

const 작은조인 = await 계획(
  "SELECT s.이름, p.상태 FROM 설비 s JOIN 점검기록 p ON p.설비번호 = s.id WHERE s.id BETWEEN 1 AND 20",
);

console.log("설비 20대만 조인:", 노드들(작은조인));
```

출력 설비 20대만 조인: Nested Loop / Index Scan / Index Only Scan

```
const 큰조인 = await 계획("SELECT s.이름, p.상태 FROM 설비 s JOIN 점검기록 p ON p.설비번호 = s.id");

console.log("설비 전체를 조인:", 노드들(큰조인));
```

출력 설비 전체를 조인: Hash Join / Seq Scan / Hash / Seq Scan

```
console.log("작을 때 Nested Loop 를 골랐나:", 작은조인.includes("Nested Loop"));
```

출력 작을 때 Nested Loop 를 골랐나: true

```
console.log("클 때 Hash Join 으로 바뀌었나:", 큰조인.includes("Hash Join"));
```

출력 클 때 Hash Join 으로 바뀌었나: true

★★ 같은 SQL 인데 조건 하나 바꾸니 조인 방식이 통째로 바뀌었습니다. 플래너가 크기를 보고 스스로 정한 것입니다. 이게 정상입니다.

Merge Join 도 보고 싶으면 다른 방식을 꺼서 억지로 만들 수 있습니다. (실습용입니다)

```
await db.exec("SET enable_hashjoin = off; SET enable_nestloop = off;");
const 병합조인 = await 계획("SELECT s.이름, p.상태 FROM 설비 s JOIN 점검기록 p ON p.설비번호 = s.id");
await db.exec("SET enable_hashjoin = on; SET enable_nestloop = on;");

console.log("억지로 만든 Merge Join:", 노드들(병합조인));
```

출력 억지로 만든 Merge Join: Merge Join / Index Scan / Index Only Scan

★ 양쪽이 이미 색인으로 정렬돼 있으니 정렬 없이 바로 지퍼를 채웠습니다. Merge Join 은 이럴 때 좋습니다. 정렬부터 해야 하면 보통 Hash Join 이 이깁니다.

BUFFERS — 실제로 몇 쪽을 읽었나

섹션 7

PostgreSQL 18 부터는 EXPLAIN ANALYZE 에 Buffers 가 자동으로 나옵니다. 옛날 버전에서는 EXPLAIN (ANALYZE, BUFFERS) 라고 써야 했습니다.

shared hit=N 메모리(캐시)에 이미 있어서 바로 쓴 쪽 shared read=N 디스크에서 새로 읽어 온 쪽 ← 이게 진짜 비용 shared dirtied 이번에 고쳐서 다시 써야 할 쪽

★★ 시간보다 이 숫자를 믿으세요. 시간은 그때 서버가 바빴는지, 캐시가 따뜻했는지에 따라 흔들립니다. 읽은 쪽 수는 같은 데이터·같은 계획이면 어디서 돌려도 같습니다.


```
const 훑기버퍼 = 쪽수뽑기(await 계획("SELECT count(*) FROM 점검기록 WHERE 점검자 = '점검원7'", "(ANALYZE)"));
const 색인버퍼 = 쪽수뽑기(await 계획("SELECT count(*) FROM 점검기록 WHERE 설비번호 = 777", "(ANALYZE)"));

console.log(`점검자로 찾기(색인 없음) ${훑기버퍼} 쪽 / 설비번호로 찾기(색인 있음) ${색인버퍼} 쪽`);
```

출력 점검자로 찾기(색인 없음) 2568 쪽 / 설비번호로 찾기(색인 있음) 2 쪽
값이 다를 수 있음

```
console.log("색인 쪽이 훨씬 적게 읽나:", 색인버퍼 * 100 < 훑기버퍼);
```

출력 색인 쪽이 훨씬 적게 읽나: true

★ 계획 읽는 순서

섹션 8

```
const 복잡한계획 = await 계획(
  `SELECT s.라인, count(*) AS 건수
   FROM 설비 s JOIN 점검기록 p ON p.설비번호 = s.id
   WHERE p.상태 = '고장'
   GROUP BY s.라인
   ORDER BY 건수 DESC`,
);

console.log(복잡한계획);
```

출력 Sort (cost=6160.43..6160.44 rows=4 width=10)
값이 다를 수 있음 Sort Key: (count(*)) DESC
 -> HashAggregate (cost=6160.35..6160.39 rows=4 width=10)
 Group Key: s."라인"
 -> Hash Join (cost=588.00..5829.65 rows=66140 width=2)
 Hash Cond: (p."설비번호" = s.id)
 -> Seq Scan on "점검기록" p (cost=0.00..5068.00
 rows=66140 width=4)
 Filter: ("상태" = '고장'::text)
 -> Hash (cost=338.00..338.00 rows=20000 width=6)
 -> Seq Scan on "설비" s (cost=0.00..338.00
 rows=20000 width=6)

★★★ 읽는 순서는 아래에서 위로, 안쪽에서 바깥으로입니다.

⑤ Sort ← 마지막. 건수 순으로 정렬 ④ HashAggregate ← 라인별로 묶어서 센다 ③ Hash Join ← 둘을 잇는다 ② Hash ← 설비를 통째로 해시표로 올린다 ① Seq Scan (설비) ← **여기가 제일 먼저** 일어납니다 ① Seq Scan (점검기록) ← 그리고 여기

요령:

- `->` 가 붙은 줄은 "이걸 먼저 해서 위로 올려 준다" 는 뜻입니다
- 들여쓰기가 가장 깊은 줄부터 읽으세요
- 같은 깊이가 둘이면 **아래쪽이 먼저** 실행되는 경우가 많습니다 (Hash Join 은 아래의 Hash 를 먼저 만듭니다)

★ 그리고 항상 **가장 안쪽 스캔**부터 확인하세요. 맨 위의 Sort 가 느린 게 아니라, 맨 아래 Seq Scan 이 6만 줄을 퍼 올려서 느린 것입니다.

그 밖의 유용한 옵션

EXPLAIN (ANALYZE, VERBOSE) 각 노드가 어떤 칸을 내보내는지 보여 줍니다 EXPLAIN (COSTS OFF) cost 를 지웁니다. 계획 모양만 볼 때 깔끔합니다 EXPLAIN (FORMAT JSON) 도구로 파싱할 때 씁니다 EXPLAIN (ANALYZE, SETTINGS) 기본값과 다른 설정을 같이 보여 줍니다

```
console.log(await 계획("SELECT count(*) FROM 점검기록 WHERE 설비번호 = 777", "(COSTS OFF)"));
```

출력	Aggregate
값이 다를 수 있음	-> Index Only Scan using "점검_설비번호2" on "점검기록"
	Index Cond: ("설비번호" = 777)

★ 계획이 길어지면 <https://explain.dank.systems> 같은 시각화 도구에 FORMAT JSON 결과를 붙여 넣어 보세요. 어디가 오래 걸렸는지 색으로 보입니다.

MySQL 은 여기가 다릅니다

· EXPLAIN 출력이 표 모양입니다. type, key, rows, Extra 칸을 봅니다 · EXPLAIN ANALYZE 는 8.0.18 부터. 그 전에는 실제값을 못 봅니다 · Using index 가 PostgreSQL 의 Index Only Scan 에 해당합니다 · Hash Join 은 8.0.18 부터입니다. 그 전에는 전부 Nested Loop 였습니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 계획을 읽는 법

무엇을 볼 때 어디를 보나

짐작이 맞았나 rows 대 actual rows ← 제일 먼저 몇 번 돌았나 loops ← 곱해서 봐야 함 진짜 비용
Buffers 의 read/hit ← 시간보다 믿을 만함 무슨 방법으로 읽었나 Seq / Index / Index Only / Bitmap
무슨 방법으로 이었나 Nested Loop / Hash / Merge
스캔 종류 언제 나오나

Seq Scan 색인이 없거나, 표의 대부분을 읽을 때 Index Scan 색인으로 콧 집고 나오는 줄이 아주 적을 때 Bitmap Heap Scan 색인으로 찾는데 줄이 좀 많을 때 Index Only Scan 필요한 칸이 전부 색인 안에 있고, VACUUM 이 돼 있을 때

조인 종류 언제 나오나

Nested Loop 한쪽이 작고 다른 쪽에 색인이 있을 때 Hash Join 둘 다 크고 등호로 이을 때 Merge Join 양쪽이 이미 정렬돼 있을 때

★★★ EXPLAIN ANALYZE 는 **진짜로 실행합니다**. UPDATE / DELETE 에는 반드시 BEGIN ... ROLLBACK 으로 감싸세요.

직접 해 볼 것

▫ 직접 해보기 1

섹션 2 의 조인 범위를 1~50 에서 1~5000 으로 넓혀 보세요. loops 가 5000 이 되나요, 아니면 조인 방식이 바뀌나요?

▫ 직접 해보기 2

섹션 3 에서 CREATE STATISTICS 를 지우고 다시 돌려 보세요. 추정치가 다시 어긋나나요? ★ DROP STATISTICS 라인_라인장; 뒤에 ANALYZE 를 꼭 하세요.

▫ 직접 해보기 3

섹션 5 에서 SELECT 설비번호, 상태, 점검자 로 바꿔 보세요. 점검자는 색인에 없습니다. Index Only Scan 이 유지되나요?

▫ 직접 해보기 4

EXPLAIN (ANALYZE, VERBOSE) 로 섹션 8 의 질의를 보세요. Output 줄에 무엇이 적혀 있나요?

▫ 직접 해보기 5

섹션 6 에서 설비 표를 200건으로 줄이고 다시 보세요. Hash Join 이 Nested Loop 로 바뀌나요?

▫ 직접 해보기 6

BEGIN 없이 EXPLAIN (ANALYZE) DELETE FROM 점검기록; 을 돌려 보세요. ★ 진짜로 다 지워집니다. 이 파일 끝에서 해 보고 결과를 확인하세요. (겁내지 마세요. 메모리에만 있는 실습용 데이터입니다)

이런 실수를 합니다

EXPLAIN ANALYZE 를 운영 **UPDATE** 에 그냥 씀

섹션 1 에서 6만 건이 진짜로 지워졌습니다. **BEGIN ... ROLLBACK** 으로 감싸세요. 이건 습관으로 만들어야 합니다.

이런 실수를 합니다

actual time 을 총 시간으로 읽음

`actual time=0.008..0.011 loops=50` 은 한 번당 0.011ms 입니다. 총 시간은 0.011×50 입니다. `loops` 를 안 곱하면 원인을 못 찾습니다.

이런 실수를 합니다

rows 추정치가 틀린 걸 그냥 넘어감

섹션 3 에서 100배 어긋났습니다. 이건 “계획이 틀린 전제로 세워졌다” 는 뜻입니다. 느린 질의를 볼 때 **가장 먼저** 확인해야 할 곳입니다.

이런 실수를 합니다

커버링 색인을 만들었는데 **Index Only Scan** 이 안 나온다고 당황함

섹션 5 에서 봤습니다. **VACUUM** 을 해야 가시성 지도가 채워집니다. 그리고 `Heap Fetches` 가 0 이 아니면 반쪽짜리입니다.

이런 실수를 합니다

Nested Loop 를 무조건 나쁘게 봄

바깥이 20줄이면 Nested Loop 가 최고입니다. 문제는 바깥이 클 때입니다. `loops` 숫자를 보고 판단하세요.

이런 실수를 합니다

cost 만 보고 좋아졌다고 판단함

`cost` 는 플래너의 짐작입니다. 짐작이 틀렸을 수도 있습니다. 반드시 **ANALYZE** 를 붙여서 `actual` 값을 보세요.

```
await db.close();
```

복합 색인과 순서

RUN node 개념04_복합_색인과_순서.js

★ 이 파일은 30초쯤 걸립니다.

지금까지 건 색인은 전부 **칸 하나짜리**였습니다. 그런데 현장 질의는 조건이 여러 개입니다.

“A 라인에서 3월에 고장 난 것” → 조건 세 개

칸마다 색인을 하나씩 걸면 될까요? 아닙니다. **여러 칸을 한 색인에 묶는 것이 훨씬 낫습니다.** 그걸 복합 색인이라고 합니다.

그리고 여기서 이 단원에서 가장 많이 틀리는 것이 나옵니다.

★★★ 칸을 어떤 순서로 넣느냐에 따라 색인이 쓰이기도 하고 안 쓰이기도 합니다.

이 파일에서 그걸 실측으로 확인하고, 순서를 정하는 규칙을 세웁니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();

async function 계획(sql, 옵션 = "") {
  const 결과 = await db.query(`EXPLAIN ${옵션} ${sql}`);
  return 결과.rows.map((줄) => 줄["QUERY PLAN"]).join("\n");
}

function 노드들(계획글) {
  return 계획글
    .split("\n")
    .map((줄) => 줄.match(/^s*(?:->s+)?([A-Z][A-Za-z ]+)?(?:\s+(?:on|using)\s|\s+(cost)/))
    .filter(Boolean)
    .map((맞음) => 맞음[1].trim())
    .join(" / ");
}
```

색인 노드가 읽은 쪽 수만 뽑습니다. 색인 효율을 견줄 때 제일 정직한 숫자입니다.

```
function 색인쪽수(계획글) {
  const 줄들 = 계획글.split("\n");
  const 자리 = 줄들.findIndex((줄) => /Index (Scan|Only Scan)|Bitmap Index
```

```

Scan/.test(줄));
if (자리 < 0) return null;

for (let i = 자리 + 1; i < 줄들.length; i += 1) {
  const 맞춤 = 줄들[i].match(/Buffers: shared (?:(hit=(\d+))?\s?(?:read=(\d+))?/);
  if (맞음) return Number(맞음[1] || 0) + Number(맞음[2] || 0);
}
return null;
}

const 메가 = (바이트) => `${(Number(바이트) / 1024 / 1024).toFixed(2)} MB`;

```

데이터

섹션 0

```

await db.exec(`
  CREATE TABLE 점검기록 (
    id          SERIAL PRIMARY KEY,
    설비번호 INT NOT NULL,
    라인       TEXT NOT NULL,
    상태       TEXT NOT NULL,
    점검자     TEXT NOT NULL,
    점검일     DATE NOT NULL,
    비고       TEXT
  );
  INSERT INTO 점검기록 (설비번호, 라인, 상태, 점검자, 점검일, 비고)
  SELECT (i % 20000) + 1,
         (ARRAY['A','B','C','D'][(i % 4) + 1],
         (ARRAY['정상','주의','고장'][(i % 3) + 1],
         '점검원' || ((i % 50) + 1),
         DATE '2024-01-01' + ((i % 365) || ' days')::interval,
         '특이사항 없음. 기록번호 ' || i
  FROM generate_series(1, 200000) AS i;
  ANALYZE 점검기록;
`);

console.log("칸별 값 종류:", JSON.stringify(
  (await db.query(`
    SELECT count(DISTINCT 설비번호) AS 설비번호, count(DISTINCT 라인) AS 라인,
           count(DISTINCT 상태) AS 상태, count(DISTINCT 점검자) AS 점검자,
           count(DISTINCT 점검일) AS 점검일
    FROM 점검기록
  `)).rows[0],
));

```

```

출력      칸별 값 종류:
          {"설비번호":20000,"라인":4,"상태":3,"점검자":50,"점검일":365}

```

★ 이 숫자를 **선택도(selectivity)** 라고 합니다. 값 종류가 많을수록 조건 하나로 많이 걸러집니다. 설비번호로 찾으면 20000분의 1로 줄고, 라인으로 찾으면 4분의 1 밖에 안 줄어듭니다.

복합 색인 만들기

섹션 1

문법은 칸을 쉼표로 나열하는 것입니다.

```
CREATE INDEX 이름 ON 표 (첫째칸, 둘째칸, 셋째칸);
```

PostgreSQL의 B-tree는 이 순서로 **정렬된 하나의 색인**을 만듭니다. (첫째칸으로 정렬하고, 같으면 둘째칸으로, 또 같으면 셋째칸으로)

```
await db.exec("CREATE INDEX 점검_설비_라인 ON 점검기록 (설비번호, 라인);");
await db.exec("ANALYZE 점검기록");

console.log("두 칸 다 주면:", 노드들(await 계획("SELECT * FROM 점검기록 WHERE
설비번호 = 500 AND 라인 = 'A'")));
```

출력 두 칸 다 주면: Bitmap Heap Scan / Bitmap Index Scan

★★★ 칸 순서가 결정적입니다

섹션 2

지금 색인은 (설비번호, 라인) 입니다. 앞칸만 주는 것과 뒷칸만 주는 것을 비교합니다.

```
const 앞칸만 = await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 500");
const 뒷칸만 = await 계획("SELECT * FROM 점검기록 WHERE 라인 = 'A'");

console.log("앞칸(설비번호)만 주면:", 노드들(앞칸만));
```

출력 앞칸(설비번호)만 주면: Bitmap Heap Scan / Bitmap Index Scan

```
console.log("뒷칸(라인)만 주면:", 노드들(뒷칸만));
```

출력 뒷칸(라인)만 주면: Seq Scan

```
console.log("앞칸만 줬을 때 색인을 썼나:", 앞칸만.includes("점검_설비_라인"));
```

출력 앞칸만 줬을 때 색인을 썼나: true

```
console.log("뒷칸만 줬을 때 색인을 썼나:", 뒷칸만.includes("점검_설비_라인"));
```

출력 뒷칸만 줬을 때 색인을 썼나: false

★★★ 같은 색인인데 앞칸으로 찾으면 쓰이고, 뒷칸으로 찾으면 안 쓰입니다.

★ 전화번호부를 생각하세요.

전화번호부는 (성, 이름) 순서로 정렬돼 있습니다.

"김" 씨를 찾아라	→ 됩니다. 김 씨 구역으로 바로 갑니다
"김철수" 를 찾아라	→ 됩니다. 김 씨 구역에서 철수를 찾습니다
"철수" 를 찾아라	→ **안 됩니다.** 이 철수가 김철수인지 박철수인지 모르니 처음부터 끝까지 다 봐야 합니다

복합 색인도 똑같습니다. **왼쪽부터 이어지는 만큼만 쓸 수 있습니다.** 이걸 "왼쪽 접두사 규칙(leftmost prefix)"이라고 합니다.

(설비번호, 라인, 상태) 색인이 있을 때	
WHERE 설비번호=?	→ 쓸 수 있음
WHERE 설비번호=? AND 라인=?	→ 쓸 수 있음
WHERE 설비번호=? AND 라인=? AND 상태=?	→ 전부 쓸 수 있음
WHERE 라인=?	→ 못 씀
WHERE 라인=? AND 상태=?	→ 못 씀
WHERE 설비번호=? AND 상태=?	→ 설비번호까지만 쓰고 상태는 걸러냄

★★ 그런데 요즘은 반쯤 쓰이기도 합니다 (PostgreSQL 18)

섹션 3

위에서 "뒷칸만 주면 못 쓴다" 고 했습니다. **항상 그런 건 아닙니다.** 순서를 뒤집어서 (라인, 설비번호) 로 만들고 뒷칸(설비번호)만 줘 봅니다.

```
await db.exec("DROP INDEX 점검_설비_라인");
await db.exec("CREATE INDEX 점검_라인_설비 ON 점검기록 (라인, 설비번호)");
await db.exec("ANALYZE 점검기록");

const 건너뛰기 = await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 500", "
(ANALYZE)");

console.log("(라인, 설비번호) 색인에 뒷칸만 줬을 때 색인을 쓰나:",
  건너뛰기.includes("점검_라인_설비"));
```

출력 (라인, 설비번호) 색인에 뒷칸만 줬을 때 색인을 쓰나: true

```
console.log("무슨 방법으로:", 노트들(건너뛰기));
```

출력 무슨 방법으로: Index Scan
값이 다를 수 있음

```
console.log(건너뛰기.split("\n").filter((줄) => /Index Cond|Index
Searches/.test(줄)).map((줄) => 줄.trim()).join(" | "));
```


출력 Index Cond: ("설비번호" = 500) | Index Searches: 9

★★ Index Searches: 9 를 보세요. 색인을 **아홉 번** 뒤졌습니다.

PostgreSQL 18 에 **건너뛰기 스캔(skip scan)** 이 들어왔습니다. 앞칸(라인)의 값이 A, B, C, D 네 가지뿐이니까 "A 구역에서 500 찾고, B 구역에서 500 찾고..." 를 반복한 것입니다.

전화번호부로 치면 성이 네 개뿐일 때 "김 철수, 이 철수, 박 철수, 최 철수" 를 각각 찾아보는 것입니다. (구역 경계를 확인하는 몫까지 더해 아홉 번이 됐습니다) 성이 5000개면 못 합니다.

★ 그래서 섹션 2 에서는 안 쓰였던 것입니다. (**설비번호**, **라인**) 의 앞칸 설비번호는 값이 **20000**까지입니다. 2만 번 건너뛰느니 그냥 표를 훑는 게 쌉니다. 플래너가 옳습니다.

★★★ 그래도 순서 규칙은 그대로 지키세요. 건너뛰기 스캔은 "운이 좋으면 살려 주는 안전망" 이지 설계 원칙이 아닙니다. 제대로 된 순서로 만든 색인과 견줘 봅니다.

```
const 이상적 = "SELECT * FROM 점검기록 WHERE 설비번호 = 500";

const 뒤집힌쪽수 = 색인쪽수(await 계획(이상적, "(ANALYZE)"));

await db.exec("DROP INDEX 점검_라인_설비");
await db.exec("CREATE INDEX 점검_설비_라인 ON 점검기록 (설비번호, 라인)");
await db.exec("ANALYZE 점검기록");
await db.query(이상적);

const 제대로쪽수 = 색인쪽수(await 계획(이상적, "(ANALYZE)"));

console.log(`색인이 읽은 쪽 - (라인,설비번호) ${뒤집힌쪽수} 쪽 / (설비번호,라인)
${제대로쪽수} 쪽`);
```

출력 색인이 읽은 쪽 - (라인,설비번호) 28 쪽 / (설비번호,라인) 2 쪽
값이 다를 수 있음

```
console.log("순서를 제대로 준 쪽이 덜 읽나:", 제대로쪽수 < 뒤집힌쪽수);
```

출력 순서를 제대로 준 쪽이 덜 읽나: true

★★ 순서를 정하는 규칙 — 등호 먼저, 범위 나중

섹션 4

조건이 **라인 = 'A'** (등호) 와 **점검일 BETWEEN ...** (범위) 두 개일 때 어느 칸을 앞에 뒤야 할까요? 재 봅니다.

```
const 섞인질의 = `
SELECT count(*) FROM 점검기록
WHERE 라인 = 'A' AND 점검일 BETWEEN DATE '2024-03-01' AND DATE '2024-03-31';
```

```
const 순서결과 = [];

for (const [설명, 칸들] of [["등호 먼저 (라인, 점검일)", "라인, 점검일"], ["범위 먼저 (점검일, 라인)", "점검일, 라인"]]) {
  await db.exec(`CREATE INDEX 순서실험 ON 점검기록 (${칸들})`);
  await db.exec("ANALYZE 점검기록");
  await db.query(섞인질의); // 예열 const 쪽 =
  색인쪽수(await 계획(섞인질의, "(ANALYZE)"));

  순서결과.push({ 설명, 쪽 });
  console.log(`${설명} → 색인에서 ${쪽} 쪽`);

  await db.exec("DROP INDEX 순서실험");
}
```

출력 등호 먼저 (라인, 점검일) → 색인에서 6 쪽
값이 다를 수 있음 범위 먼저 (점검일, 라인) → 색인에서 18 쪽

```
console.log("등호를 앞에 둔 쪽이 덜 읽나:", 순서결과[0].쪽 < 순서결과[1].쪽);
```

출력 등호를 앞에 둔 쪽이 덜 읽나: true

★★ 왜 그런가.

색인은 정렬된 목록입니다. 범위 조건을 만나는 순간 그 뒤 칸은 정렬이 깨집니다.

(라인, 점검일) 로 만들면:

라인 = 'A' 인 구역으로 점프 → 그 안에서 점검일이 정렬돼 있음 → 3월 구간만 딱 읽음

(점검일, 라인) 로 만들면:

3월 구간으로 점프 → 그 안에 A·B·C·D 가 섞여 있음 → 전부 읽고 A 만 골라냄

★★★ 규칙: 등호(=, IN) 조건 칸을 앞에, 범위(>, <, BETWEEN, LIKE 'x%') 칸을 뒤에. 그리고 등호 칸이 여러 개면 값 종류가 많은 칸을 앞에 두는 게 대체로 낫습니다. (많이 걸러지는 칸을 먼저 쓰면 뒤에서 볼 게 적어집니다)

그리고 마지막 규칙: 정렬(ORDER BY)에 쓰는 칸은 맨 뒤에. 섹션 8 에서 봅니다.

★ 부분 색인 — 필요한 줄만 색인합니다

섹션 5

현장 화면 대부분은 “고장 난 것” 만 봅니다. 정상 기록은 안 봅니다. 그런데 색인은 20만 줄을 전부 담고 있습니다. 3분의 1만 담으면 안 될까요?

```
CREATE INDEX ... ON 표 (칸) WHERE 조건;
```

조건에 맞는 줄만 색인에 넣습니다. 이것 **부분 색인(partial index)** 이라고 합니다.

```
await db.exec("CREATE INDEX 점검_점검일_전체 ON 점검기록 (점검일)");
await db.exec("CREATE INDEX 점검_고장만 ON 점검기록 (점검일) WHERE 상태 = '고장'");
await db.exec("ANALYZE 점검기록");

const 색인크기 = (
  await db.query(`
    SELECT pg_relation_size('점검_점검일_전체') AS 전체, pg_relation_size('점검_
    고장만') AS 부분
  `)
).rows[0];

console.log(`전체 색인 ${메가(색인크기.전체)} / 고장만 담은 부분 색인 ${메가(
  색인크기.부분)}`);
```

출력 전체 색인 1.45 MB / 고장만 담은 부분 색인 0.46 MB
값이 다를 수 있음

```
console.log("부분 색인이 더 작은가:", Number(색인크기.부분) < Number(색인크기.전체));
```

출력 부분 색인이 더 작은가: true

```
const 고장질의 = "SELECT * FROM 점검기록 WHERE 상태 = '고장' AND 점검일 = DATE '2024-
03-01'";

console.log("고장 조회에 무엇을 골랐나:", (await 계획(고장질의)).includes("점검_
고장만") ? "부분 색인" : "다른 것");
```

출력 고장 조회에 무엇을 골랐나: 부분 색인

★★ 부분 색인의 좋은 점

① 작습니다 → 메모리 캐시를 덜 먹고, 뒤지는 깊이도 얕습니다 ② 조건에 안 맞는 줄을 넣거나 고칠 때는 색인을 안 건드립니다 → 쓰기가 덜 느려집니다

현장에서 잘 맞는 곳:

- 상태가 '진행중' 인 주문만 (끝난 주문은 안 봄)
- 삭제표시 = false 인 줄만 (지운 건 안 봄)
- 오류 로그만 (전체 로그의 0.1%)

★ 주의: 질의의 조건이 색인의 WHERE 를 포함해야 씁니다. WHERE 상태 = '고장' 을 안 쓰면 이 색인은 못 씁니다.

```
console.log("조건을 빼면 부분 색인을 쓰나:", (await 계획("SELECT * FROM 점검기록
WHERE 점검일 = DATE '2024-03-01'")).includes("점검_고장만"));
```

```
출력          조건을 빼면 부분 색인을 쓰나: false
```

```
await db.exec("DROP INDEX 점검_점검일_전체");
```

★ 표현식 색인 — 함수를 씌운 검색을 살립니다

섹션 6

대소문자를 무시하고 이름을 찾고 싶습니다.

WHERE lower(점검자) = '점검원7'

이러면 색인이 안 쓰입니다. 색인에는 점검원7 이 들어 있지 lower(점검원7) 이 없으니까요. (개념05 에서 이 함정을 자세히 봅니다)

```
await db.exec("CREATE INDEX 점검_점검자 ON 점검기록 (점검자)");
await db.exec("ANALYZE 점검기록");

console.log("lower() 를 씌우면:", 노트들(await 계획("SELECT count(*) FROM 점검기록
WHERE lower(점검자) = '점검원7'")));
```

```
출력          lower() 를 씌우면: Aggregate / Seq Scan
```

함수를 씌운 결과 자체를 색인으로 만들면 됩니다.

```
await db.exec("CREATE INDEX 점검_소문자점검자 ON 점검기록 (lower(점검자))");
await db.exec("ANALYZE 점검기록");

console.log("표현식 색인을 만든 뒤:", 노트들(await 계획("SELECT count(*) FROM
점검기록 WHERE lower(점검자) = '점검원7'")));
```

```
출력          표현식 색인을 만든 뒤: Aggregate / Bitmap Heap Scan / Bitmap Index Scan
```

★ 쓸 데가 많습니다.

CREATE INDEX ON 회원 (lower(전자우편)); -- 대소문자 무시 로그인 CREATE INDEX ON 주문
(date_trunc('day', 시각)); -- 날짜별 집계 CREATE INDEX ON 설비 ((설정 -> '제조사')); -- JSONB
안의 값

★★ 주의: 질의에 쓴 표현식과 글자 그대로 같아야 쓰입니다. lower(점검자) 로 만들었으면 질의도
lower(점검자) 여야 합니다. upper(점검자) 나 점검자 로는 안 됩니다. 그리고 그 함수는 IMMUTABLE
이어야 합니다. (같은 입력이면 항상 같은 출력)

커버링 색인 (INCLUDE)

섹션 7

개념03 에서 봤습니다. 필요한 칸이 전부 색인 안에 있으면 표를 안 뵙니다. 찾는 데는 안 쓰고 **결과로만 필요한 칸**은 INCLUDE 에 넣습니다.

```
await db.exec("CREATE INDEX 점검_커버 ON 점검기록 (설비번호) INCLUDE (상태, 점검일);");
await db.exec("ANALYZE 점검기록");
await db.exec("VACUUM 점검기록"); // 가시성 지도를 채워야
합니다 console.log("커버링 색인:", 노드들(await 계획("SELECT 설비번호, 상태, 점검일
FROM 점검기록 WHERE 설비번호 = 500"))));
```

출력 커버링 색인: Index Only Scan

```
console.log("색인에 없는 칸을 하나 더 달라고 하면:", 노드들(await 계획("SELECT
설비번호, 상태, 비고 FROM 점검기록 WHERE 설비번호 = 500"))));
```

출력 색인에 없는 칸을 하나 더 달라고 하면: Bitmap Heap Scan / Bitmap Index Scan

★ (설비번호, 상태) 와 (설비번호) INCLUDE (상태) 의 차이

앞쪽: 상태로도 정렬됩니다. WHERE 설비번호=? AND 상태=? 에서 상태까지 색인으로 찾습니다 뒤쪽: 상태는 그냥 얹혀 있습니다. 찾는 데는 못 쓰고 **꺼내는 데만** 씁니다

찾는 조건에 안 쓰는 칸은 INCLUDE 쪽이 낫습니다. 색인이 조금 더 작고 UNIQUE 색인에 없을 때는 유일성 판정에 영향을 안 줍니다.

★ ORDER BY 를 색인으로 처리하기

섹션 8

정렬은 비쌉니다. 20만 줄을 정렬하려면 다 모아서 줄을 세워야 합니다. 그런데 **색인은 이미 정렬돼 있습니다**. 순서대로 읽기만 하면 정렬이 끝납니다.

```
await db.exec("DROP INDEX 점검_커버; DROP INDEX 점검_설비_라인;");
await db.exec("ANALYZE 점검기록");

const 정렬질의 = "SELECT * FROM 점검기록 WHERE 라인 = 'A' ORDER BY 점검일 DESC LIMIT 20";

const 색인전정렬 = await 계획(정렬질의, "(ANALYZE)");

console.log("색인 없이 정렬:", 노드들(색인전정렬));
```

출력 색인 없이 정렬: Limit / Sort / Seq Scan

```
await db.exec("CREATE INDEX 점검_라인_점검일 ON 점검기록 (라인, 점검일 DESC);");
await db.exec("ANALYZE 점검기록");
await db.query(정렬질의);
```

```
const 색인후정렬 = await 계획(정렬질의, "(ANALYZE)");
```

```
console.log("맞는 색인을 걸고:", 노드들(색인후정렬));
```

출력 맞는 색인을 걸고: Limit / Index Scan

```
console.log("Sort 노드가 사라졌나:", 색인전정렬.includes("Sort") && !
색인후정렬.includes("Sort"));
```

출력 Sort 노드가 사라졌나: true

```
const 실행시간 = (계획글) => Number(계획글.match(/Execution Time: ([\d.]+) ms/)[1]);
```

```
console.log(`Execution Time - 정렬함 ${실행시간(색인전정렬).toFixed(2)} ms / 색인으로
${실행시간(색인후정렬).toFixed(2)} ms`);
```

출력 Execution Time - 정렬함 45.03 ms / 색인으로 0.04 ms
값이 다를 수 있음

```
console.log("색인으로 정렬한 쪽이 빠른가:", 실행시간(색인후정렬) < 실행시간
(색인전정렬));
```

출력 색인으로 정렬한 쪽이 빠른가: true

★★ ORDER BY ... LIMIT 20 에서 특히 큼니다. 정렬을 하면 20줄을 얻으려고 5만 줄을 다 모아서 줄 세웁니다. 색인 순서대로 읽으면 앞에서 20줄만 읽고 멈춥니다.

★ 색인 방향도 맞아야 합니다. ORDER BY 점검일 DESC 인데 색인이 (라인, 점검일) 오름차순이어도 PostgreSQL 은 색인을 거꾸로 읽을 수 있어서 대개 괜찮습니다. 문제는 방향이 섞일 때입니다: ORDER BY 라인 ASC, 점검일 DESC 는 (라인, 점검일 DESC) 색인이 있어야 정렬 없이 됩니다.

★ 색인을 몇 개까지 걸어야 하나

섹션 9

답: **재 보고 정합니다.** 정해진 개수는 없습니다. 대신 판단할 수 있는 도구가 있습니다.

일부러 아무도 안 쓸 색인을 하나 만들어 둡니다.

```
await db.exec("CREATE INDEX 점검_비고 ON 점검기록 (비고);");
```

★ 통계는 **진짜로 실행한 질의**만 셉니다. EXPLAIN 만 해서는 안 늘어납니다. 그래서 위에서 EXPLAIN 만 잔뜩 했는데도 숫자가 0 이었던 것입니다. 실제로 몇 번 돌려 봅니다.

```

for (let 회차 = 0; 회차 < 5; 회차 += 1) {
  await db.query("SELECT * FROM 점검기록 WHERE 라인 = 'A' ORDER BY 점검일 DESC LIMIT 20");
  await db.query("SELECT count(*) FROM 점검기록 WHERE lower(점검자) = '점검원7'");
  await db.query("SELECT * FROM 점검기록 WHERE id = 12345");
}

const 색인사용 = (
  await db.query(`
    SELECT indexrelname AS 이름, idx_scan AS 사용횟수, pg_relation_size(indexrelid)
    AS 크기
    FROM pg_stat_user_indexes WHERE relname = '점검기록' ORDER BY 사용횟수 DESC, 이름
  `)
).rows;

for (const { 이름, 사용횟수, 크기 } of 색인사용) {
  console.log(`· ${이름} - ${사용횟수}번 쓰임 · ${메가(크기)}`);
}

```

출력
값이 다를 수 있음

- 점검_고장만 - 0번 쓰임 · 0.46 MB
- 점검_라인_점검일 - 0번 쓰임 · 1.29 MB
- 점검_비고 - 0번 쓰임 · 12.16 MB
- 점검_소문자점검자 - 0번 쓰임 · 1.38 MB
- 점검_점검자 - 0번 쓰임 · 1.38 MB
- 점검기록_pkey - 0번 쓰임 · 4.30 MB

```

const 안쓰인것 = 색인사용.filter((줄) => Number(줄.사용횟수) === 0).map((줄) => 줄.이름);

```

```

console.log("한 번도 안 쓰인 색인:", 안쓰인것.join(", "));

```

출력
값이 다를 수 있음

한 번도 안 쓰인 색인: 점검_고장만, 점검_라인_점검일, 점검_비고, 점검_소문자점검자, 점검_점검자, 점검기록_pkey

```

console.log("점검_비고 가 안 쓰인 목록에 있나:", 안쓰인것.includes("점검_비고"));

```

출력

점검_비고 가 안 쓰인 목록에 있나: true

★★ 점검_비고 는 이 표에서 가장 큰 색인인데 한 번도 안 쓰였습니다. 표의 절반쯤 되는 덩어리가 순수한 손해로 앉아 있는 것입니다. INSERT 마다 여기에도 써야 하고, 백업에도 들어가고, 캐시도 먹습니다.

★ 그런데 점검_점검자 도 0 입니다. 이걸 조심해야 합니다. 이 파일에서 그 질의를 안 돌렸을 뿐이지, 운영에서는 쓰일 수도 있습니다. 며칠 돌려 본 통계로 판단하면 안 됩니다.

★★ 운영에서 안 쓰이는 색인을 찾는 질의입니다. 그대로 쓰세요.

SELECT relname AS 표, indexrelname AS 색인, idx_scan AS 사용횟수,

```
pg_size_pretty(pg_relation_size(indexrelid)) AS 크기
```

```
FROM pg_stat_user_indexes WHERE idx_scan = 0
```

```
AND indexrelid NOT IN (SELECT conindid FROM pg_constraint WHERE contype IN ('p','u'))
```

```
ORDER BY pg_relation_size(indexrelid) DESC;
```

★ 주의할 점 세 가지

- ① 이 숫자는 **통계를 마지막으로 초기화한 뒤**부터 셉니다.
서버를 다시 켜거나 `pg_stat_reset()` 을 하면 0 부터 시작합니다.
최소 몇 주는 지켜보고 판단하세요. 월말 결산에만 쓰는 색인도 있습니다
- ② 기본키·UNIQUE 색인은 사용횟수가 0 이어도 **지우면 안 됩니다.** 규칙을 지키는 역할이 있습니다
- ③ 지우기 전에 `DROP INDEX CONCURRENTLY` 를 쓰세요. 표를 잠그지 않습니다

★ 색인을 더 걸지 말지 판단하는 순서

① 느린 질의를 찾는다 (10단원에서 pg_stat_statements 로 합니다) ② EXPLAIN ANALYZE 로 어디가 문제인지 본다 ③ 색인을 만들고 **같은 방법으로 다시 켜** ④ 안 빨라졌으면 **지웁니다**. 미련 두지 마세요 ⑤ 쓰기가 얼마나 느려졌는지도 켜다 (개념02 섹션 6)

★★ 그리고 **이미 있는 색인을 늘리는 것을 먼저** 생각하세요. (설비번호) 색인이 있는데 (설비번호, 라인) 이 필요하면 새로 만들고 **옛것을 지우면** 색인이 하나로 유지됩니다. (설비번호, 라인) 은 (설비번호) 가 하던 일을 전부 할 수 있습니다.

MySQL 은 여기가 다릅니다

· 왼쪽 접두사 규칙은 똑같습니다. 건너뛰기 스캔은 8.0 의 일부 경우만 지원합니다 · **부분 색인(WHERE 붙은 색인)이 없습니다**. 이건 PostgreSQL 만 있습니다 · **표현식 색인**은 8.0.13 부터 가능합니다. 그 전에는 생성 칸을 만들어 색인했습니다 · **INCLUDE** 가 없습니다. 대신 기본키가 모든 색인에 자동으로 붙습니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 복합 색인 설계

규칙 이유

왼쪽부터 이어지는 만큼만 쓰인다 색인은 (첫칸, 둘째칸) 순서로 정렬돼 있다 등호 조건 칸을 앞에 범위 조건을 만나면 뒤 칸의 정렬이 깨진다 값 종류가 많은 칸을 앞에 많이 걸러지는 조건을 먼저 쓴다 ORDER BY 칸은 맨 뒤에 정렬을 색인으로 대신할 수 있다 찾는 데 안 쓰는 칸은 INCLUDE 로 색인이 작아지고 Index Only Scan 이 된다

종류 언제 쓰나

복합 색인 (가, 나) 조건이 여러 개일 때 부분 색인 ... WHERE 조건 일부만 자주 볼 때 (진행중, 오류, 미삭제)
표현식 색인 (lower(칸)) 함수를 씌워 검색할 때 커버링 색인 ... INCLUDE (칸) 결과로만 필요한 칸이 있을 때

재 본 것 결과

(설비번호,라인) 에 라인만 조건 색인 안 쓰임 → Seq Scan (라인,설비번호) 에 설비번호만 조건 건너뛰기 스캔으로 쓰이긴 함 (Index Searches: 9)

단, 제대로 된 순서보다 열 배쯤 더 읽음

등호 먼저 대 범위 먼저 등호 먼저가 색인 쪽을 3분의 1 만 읽음 부분 색인 전체 색인의 3분의 1 크기 ORDER BY 를 색인으로 Sort 노드가 사라짐

직접 해 볼 것

▫ 직접 해보기 1

(설비번호, 라인, 상태) 색인을 만들고 아래 다섯 가지 조건으로 EXPLAIN 해 보세요. 어디까지 색인을 쓰나요? ① 설비번호=? ② 설비번호=? AND 라인=? ③ 라인=? ④ 설비번호=? AND 상태=? ⑤ 라인=? AND 상태=?

▫ 직접 해보기 2

섹션 4 를 점검일 = DATE '2024-03-01' (등호 두 개) 로 바꿔 보세요. 이번에도 순서가 중요한가요?

▫ 직접 해보기 3

섹션 5 의 부분 색인 조건을 WHERE 상태 <> '정상' 으로 바꿔 보세요. 크기가 얼마나 되나요? WHERE 상태 = '고장' 질의에 쓰이나요?

▫ 직접 해보기 4

CREATE INDEX ON 점검기록 (upper(점검자)); 를 만들고 WHERE lower(점검자) = ... 로 찾아 보세요. 쓰이나요? 왜 그럴까요?

▫ 직접 해보기 5

섹션 8 에서 ORDER BY 점검일 DESC 를 ORDER BY 점검일 ASC 로 바꿔 보세요. Sort 가 다시 생기나요? (힌트: 색인을 거꾸로 읽을 수 있습니다. Backward 라는 말을 찾아보세요)

섹션 3 의 앞칸을 값이 아주 많은 칸(점검자 50가지)으로 바꿔서 (점검자, 설비번호) 를 만들고 설비번호만으로 찾아 보세요. Index Searches 가 몇 번인가요? 그래도 색인을 쓰나요?

자주 하는 실수

이런 실수를 합니다

칸마다 색인을 하나씩 만듦

(설비번호), (라인), (상태) 세 개보다 (설비번호, 라인, 상태) 하나가 낫습니다. PostgreSQL 이 BitmapAnd 로 여러 색인을 합칠 수는 있지만 복합 색인만 못합니다. 그리고 색인 세 개는 쓰기를 세 배 느리게 합니다.

이런 실수를 합니다

순서를 아무렇게나 정함

섹션 2 에서 봤습니다. 순서가 틀리면 아예 안 쓰입니다. 가장 자주 쓰는 질의의 WHERE 를 보고 순서를 정하세요.

이런 실수를 합니다

건너뛰기 스캔을 믿고 순서를 대충 정함

섹션 3 에서 확인했습니다. 쓰이긴 하는데 열 배쯤 더 읽습니다. 그리고 앞칸 값이 많으면 아예 안 됩니다. 안전망이지 설계가 아닙니다.

이런 실수를 합니다

부분 색인을 만들어 놓고 질의에 조건을 안 씀

WHERE 상태='고장' 으로 만든 색인은 질의에도 그 조건이 있어야 쓰입니다. 섹션 5 끝에서 확인했습니다.

이런 실수를 합니다

표현식이 조금 다른데 쓰일 거라고 생각함

lower(점검자) 색인은 lower(점검자) 에만 쓰입니다. lower(trim(점검자)) 나 점검자 에는 안 쓰입니다. 글자 그대로 같아야 합니다.

이런 실수를 합니다

안 쓰이는 색인을 안 지움

pg_stat_user_indexes 로 확인하세요. 안 쓰이는 색인은 순수한 손해입니다. 단, 최소 몇 주는 지켜보고, 기본키·UNIQUE 는 건드리지 마세요.

이런 실수를 합니다

색인을 만들 때 표를 잠그는 걸 모름

`CREATE INDEX` 는 그동안 그 표에 **쓰기를 막습니다**. 운영 중이면 `CREATE INDEX CONCURRENTLY` 를 쓰세요.
느리지만 안 막습니다. (PGlite 는 연결이 하나뿐이라 `CONCURRENTLY` 실습은 07단원에서 합니다)

```
await db.close();
```

색인이 안 쓰일 때, 그리고 N+1

RUN node 개념05_색인이_안_쓰일_때와_N더하기1.js

★ 이 파일은 40초쯤 걸립니다. 망 지연을 흉내 내는 부분이 있어 좀 걸립니다.

“색인을 걸었는데 안 빨라져요.”

이 단원에서 가장 많이 듣는 말입니다. 대부분 원인은 정해져 있습니다. **색인은 있는데 질의가 색인을 못 쓰게 생겼기 때문**입니다.

이 파일의 앞부분에서 그 경우를 여섯 가지 전부 **실제로 재현**하고 고칩니다. 뒷부분에서는 색인과 아무 상관 없는데 가장 흔한 성능 문제, **N+1** 을 다룹니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();

async function 재기(횟수, 할일) {
  await 할일(0); // 예열 const 잔것 = [];

  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    await 할일(회차);
    잔것.push(performance.now() - 시작);
  }

  return 잔것.sort((가, 나) => 가 - 나)[Math.floor(횟수 / 2)];
}

async function 계획(sql, 옵션 = "") {
  const 결과 = await db.query(`EXPLAIN ${옵션} ${sql}`);
  return 결과.rows.map((줄) => 줄["QUERY PLAN"]).join("\n");
}

const 색인썼나 = async (sql) => !(await 계획(sql)).includes("Seq Scan on");
```

데이터와 색인

섹션 0

```
await db.exec(`
  CREATE TABLE 설비 (
```

```

이름 TEXT NOT NULL,
라인 TEXT NOT NULL,
관리번호 TEXT NOT NULL
);
INSERT INTO 설비 (이름, 라인, 관리번호)
SELECT '설비' || i, (ARRAY['A','B','C','D'][(i % 4) + 1], lpad(i::text, 8, '0'))
FROM generate_series(1, 20000) AS i;
CREATE INDEX 설비_이름 ON 설비 (이름);
CREATE INDEX 설비_관리번호 ON 설비 (관리번호);

CREATE TABLE 점검기록 (
  id SERIAL PRIMARY KEY,
  설비번호 INT NOT NULL,
  상태 TEXT NOT NULL,
  점검자 TEXT NOT NULL,
  점검일 DATE NOT NULL
);
INSERT INTO 점검기록 (설비번호, 상태, 점검자, 점검일)
SELECT (i % 20000) + 1,
       (ARRAY['정상','주의','고장'][(i % 3) + 1],
       '점검원' || ((i % 50) + 1),
       DATE '2024-01-01' + ((i % 365) || ' days')::interval
FROM generate_series(1, 200000) AS i;
CREATE INDEX 점검_설비번호 ON 점검기록 (설비번호);
CREATE INDEX 점검_점검자 ON 점검기록 (점검자);
ANALYZE;
`);

```

색인은 전부 걸려 있습니다. 이제부터 색인을 못 쓰게 만드는 실수를 하나씩 해 봅니다.

함정 1: 칸에 함수를 씌웠습니다

```
console.log("그냥 찾기:", await 색인했나("SELECT * FROM 점검기록 WHERE 점검자 = '점검원7'"));
```

출력 그냥 찾기: true

```
console.log("lower() 를 씌우면:", await 색인했나("SELECT * FROM 점검기록 WHERE lower(점검자) = '점검원7'"));
```

출력 lower() 를 씌우면: false

```
console.log(await 계획("SELECT * FROM 점검기록 WHERE lower(점검자) = '점검원7'"));
```

출력 Seq Scan on "점검기록" (cost=0.00..4471.00 rows=1000 width=30)
 값이 다를 수 있음 Filter: (lower("점검자") = '점검원7'::text)

★ 왜: 색인 안에는 점검원7 이 들어 있습니다. lower(점검원7) 은 없습니다. 색인은 저장된 값 그대로 정렬돼 있습니다. 함수를 씌우면 그 정렬이 소용없어집니다.

★ 자주 하는 형태 — 전부 같은 함정입니다

```
WHERE lower(이름) = ...
WHERE 설비번호 + 0 = 500
WHERE 설비번호::text = '500'
WHERE date_trunc('day', 점검일) = DATE '2024-03-01'
WHERE substring(관리번호, 1, 3) = '000'
```

```
console.log("설비번호 + 0 으로 찾으려면:", await 색인썼나("SELECT * FROM 점검기록 WHERE 설비번호 + 0 = 500"));
```

출력 설비번호 + 0 으로 찾으려면: false

★ 고치는 법 두 가지 ① 조건을 옮긴다 — 칸 쪽은 그대로 두고 값 쪽을 바꿉니다

```
WHERE 설비번호 + 0 = 500 → WHERE 설비번호 = 500
WHERE date_trunc('day', 점검일) = DATE '2024-03-01'
→ WHERE 점검일 >= DATE '2024-03-01' AND 점검일 < DATE '2024-03-02'
```

② 표현식 색인을 만든다 (개념04 섹션 6)

```
CREATE INDEX ON 점검기록 (lower(점검자));
```

```
await db.exec("CREATE INDEX 점검_소문자점검자 ON 점검기록 (lower(점검자));");
await db.exec("ANALYZE 점검기록");
```

```
console.log("표현식 색인을 만든 뒤:", await 색인썼나("SELECT * FROM 점검기록 WHERE lower(점검자) = '점검원7'"));
```

출력 표현식 색인을 만든 뒤: true

```
console.log("범위로 바꾼 날짜 조건:", await 색인썼나("SELECT * FROM 점검기록 WHERE 설비번호 >= 500 AND 설비번호 < 501"));
```

출력 범위로 바꾼 날짜 조건: true

함정 2: LIKE 의 앞이 열려 있습니다 —————

```
console.log("LIKE '설비77%' (앞이 고정):", await 색인썼나("SELECT * FROM 설비 WHERE 이름 LIKE '설비77%'"));
```

출력 LIKE '설비77%' (앞이 고정): true

```
console.log("LIKE '%77%' (앞이 열림):", await 색인했나("SELECT * FROM 설비 WHERE 이름 LIKE '%77%'"));
```

```
출력      LIKE '%77%' (앞이 열림): false
```

```
console.log(await 계획("SELECT * FROM 설비 WHERE 이름 LIKE '설비77%'"));
```

```
출력      Bitmap Heap Scan on "설비" (cost=5.30..146.62 rows=2 width=26)
값이 다를 수 있음      Filter: ("이름" ~~ '설비77'::text)
                        -> Bitmap Index Scan on "설비_이름" (cost=0.00..5.30 rows=101
                        width=0)
                        Index Cond: (("이름" >= '설비77'::text) AND ("이름" <
                        '설비78'::text))
```

★★ Index Cond 를 보세요. PostgreSQL 이 LIKE ‘설비77%’ 를 >= ‘설비77’ AND < ‘설비78’ 이라는 범위 조건으로 바꿨습니다. 색인은 정렬돼 있으니 그 구간만 읽으면 됩니다.

그런데 ‘%77%’ 은 그렇게 못 바꿉니다. ‘77’ 로 시작하는 게 아니라 아무 데나 있으면 되니까요.

전화번호부에서 “이름에 철 자가 들어가는 사람” 을 찾는 것과 같습니다. 다 봐야 합니다.

★ 고치는 법 ① 앞을 고정할 수 있으면 고정한다 (LIKE ‘설비77%’) ② 뒤에서부터 찾는 거면 거꾸로 저장한 칸을 만들고 색인 (reverse(이름)) ③ 진짜 전문 검색이면 전문 검색 색인(GIN) 이나 pg_trgm 확장을 쓴다

(PGlite 에는 확장이 없어 실습은 못 합니다. 10단원에서 언급합니다)

★ 대소문자 무시 + 앞 고정을 같이 하려면 표현식 색인을 씁니다

```
CREATE INDEX ON 설비 (lower(이름) text_pattern_ops);
WHERE lower(이름) LIKE '설비77%'
```

함정 3: 타입이 안 맞습니다

★ PostgreSQL 은 여기서 다른 데이터베이스보다 훨씬 친절합니다. 맞는 게 없으면 조용히 느껴지는 대신 에러를 냅니다.

```
try {
  await db.query("SELECT * FROM 설비 WHERE 관리번호 = 500");
} catch (에러) {
  console.log("문자 칸에 숫자를 주면:", 에러.code, "-", 에러.message.slice(0, 40));
}
```

```
출력      문자 칸에 숫자를 주면: 42883 – operator does not exist: text = integer
```

```
}
```

숫자 칸에 문자를 주는 것은 알아서 맞춰 줍니다. 색인도 그대로 씁니다.

```
console.log("숫자 칸에 문자를 주면:", await 색인썼나("SELECT * FROM 점검기록 WHERE  
설비번호 = '500'"));
```

출력 숫자 칸에 문자를 주면: true

★★ PostgreSQL 에서 타입 때문에 색인이 죽는 건 '칸 쪽에 형변환이 붙을 때' 입니다.

```
console.log("칸을 형변환하면:", await 색인썼나("SELECT * FROM 점검기록 WHERE  
설비번호::text = '500'"));
```

출력 칸을 형변환하면: false

★ 애플리케이션에서 이렇게 만들어지기 쉽습니다

```
WHERE CAST(설비번호 AS TEXT) = $1          ← 코드에서 문자열로 받아서 그냥 맞춘 경우
```

고치는 법: 값 쪽을 바꾸세요. WHERE 설비번호 = \$1::int

MySQL 은 여기가 다릅니다

MySQL 은 문자 칸에 숫자를 주면 **에러 없이 조용히 전부 훑습니다**. 그게 훨씬 위험합니다. 09단원에서 실제로 재 봅니다.

함정 4: OR

★ 먼저 오해를 풀니다. **OR 이 항상 나쁜 게 아닙니다**. 양쪽에 색인이 있으면 PostgreSQL 은 둘 다 씁니다.

```
console.log(await 계획("SELECT id FROM 점검기록 WHERE 설비번호 = 500 OR 점검자 =  
'점검원7'"));
```

출력 Bitmap Heap Scan on "점검기록" (cost=53.65..1586.41 rows=4116
값이 다를 수 있음 width=4)
 Recheck Cond: (("설비번호" = 500) OR ("점검자" = '점검원7'::text))
 -> BitmapOr (cost=53.65..53.65 rows=4117 width=0)
 -> Bitmap Index Scan on "점검_설비번호" (cost=0.00..4.37
 rows=10 width=0)
 Index Cond: ("설비번호" = 500)
 -> Bitmap Index Scan on "점검_점검자" (cost=0.00..47.22
 rows=4107 width=0)
 Index Cond: ("점검자" = '점검원7'::text)

★ BitmapOr — 색인 두 개로 각각 찾아서 결과를 합쳤습니다. 잘 돌아갑니다.

★★ 진짜 문제는 **OR 이 두 표에 걸칠 때**입니다.


```
const 오알질의 = `
  SELECT p.id FROM 점검기록 p JOIN 설비 s ON p.설비번호 = s.id
  WHERE s.이름 = '설비77' OR p.점검자 = '점검원7'`;

const 오알계획 = await 계획(오알질의, "(ANALYZE)");

console.log(오알계획.split("\n").slice(0, 4).map((줄) => 줄.trim()).join("\n"));
```

```
출력      Hash Join  (cost=597.00..4593.00 rows=4099 width=4) (actual
값이 다를 수 있음  time=13.497..148.188 rows=4010.00 loops=1)
                  Hash Cond: (p."설비번호" = s.id)
                  Join Filter: ((s."이름" = '설비77'::text) OR (p."점검자" =
                  '점검원7'::text))
                  Rows Removed by Join Filter: 195990
```

★★★ Rows Removed by Join Filter: 195990 을 보세요. **20만 줄을 전부 이어 붙인 다음** OR 조건으로 걸렸습니다. 색인은 하나도 안 썼습니다. 조건이 두 표에 걸쳐 있으면 색인으로 미리 못 거릅니다.

★ 고치는 법: **UNION 으로 쪼갭니다.** 각각은 한 표의 조건이니 색인을 쓸 수 있습니다.

```
const 유니온질의 = `
  SELECT p.id FROM 점검기록 p JOIN 설비 s ON p.설비번호 = s.id WHERE s.이름 =
  '설비77'
  UNION
  SELECT p.id FROM 점검기록 p JOIN 설비 s ON p.설비번호 = s.id WHERE p.점검자 =
  '점검원7'`;

const 오알ms = await 재기(5, () => db.query(오알질의));
const 유니온ms = await 재기(5, () => db.query(유니온질의));

console.log(`OR ${오알ms.toFixed(1)} ms / UNION ${유니온ms.toFixed(1)} ms → ${오알ms
/ 유니온ms).toFixed(1)}배`);
```

```
출력      OR 73.4 ms / UNION 13.9 ms → 5.3배
값이 다를 수 있음
```

```
console.log("UNION 이 더 빠르냐:", 유니온ms < 오알ms);
```

```
출력      UNION 이 더 빠르냐: true
```

```
const 오알건수 = (await db.query(`SELECT count(*) AS 수 FROM (${오알질의}) AS
t`)).rows[0].수;
const 유니온건수 = (await db.query(`SELECT count(*) AS 수 FROM (${유니온질의}) AS
t`)).rows[0].수;

console.log(`결과 건수 - OR ${오알건수} / UNION ${유니온건수}`);
```

출력 결과 건수 - OR 4010 / UNION 4010

```
console.log("결과가 같은가:", 오알건수 === 유니온건수);
```

출력 결과가 같은가: true

★★ 주의: **UNION** 은 중복을 지웁니다(정렬/해시 비용이 듭니다). 중복이 없는 게 확실하면 **UNION ALL** 이 더쌘니다. 하지만 **OR** 은 원래 중복을 안 내므로, 겹치는 줄이 있으면 **UNION** 을 써야 결과가 같습니다.

함정 5: 대부분의 줄을 읽습니다

이건 **함정**이 아닙니다. 플래너가 일부러 색인을 안 쓴 것입니다. (개념01 섹션 6)

```
await db.exec("CREATE INDEX 점검_상태 ON 점검기록 (상태); ANALYZE 점검기록;");

console.log("고장만 (33%):", await 색인썼나("SELECT id FROM 점검기록 WHERE 상태 = '고장'"));
```

출력 고장만 (33%): true

```
console.log("정상이 아닌 것 (67%):", await 색인썼나("SELECT id FROM 점검기록 WHERE 상태 <> '정상'"));
```

출력 정상이 아닌 것 (67%): false

★ 이럴 때 “왜 색인을 안 쓰지?” 하고 억지로 쓰게 만들면 **더 느려집니다**. 개념01 에서 재 봤습니다.

★ 진짜 고치는 법은 색인이 아닙니다 ① 조건을 더 좁힌다 (기간을 넣는다) ② **LIMIT** 을 건다 (화면에 20줄만 보여 주면 되잖아요) ③ 미리 세어 둔 요약표를 만든다 (집계는 04단원의 이야기입니다)

함정 6: 통계가 낡았습니다

개념02 섹션 5 에서 자세히 봤습니다. 여기서는 짧게 확인만 합니다.

```
await db.exec(`
  CREATE TABLE 신규기록 (id SERIAL PRIMARY KEY, 상태 TEXT NOT NULL);
  INSERT INTO 신규기록 (상태) SELECT CASE WHEN i <= 10 THEN '고장' ELSE '정상' END
  FROM generate_series(1, 100000) AS i;
  CREATE INDEX 신규_상태 ON 신규기록 (상태);
  ANALYZE 신규기록;
`);

await db.exec("UPDATE 신규기록 SET 상태 = '고장' WHERE id % 10 <> 0"); // 이제 90%
가 고장 const 낡은계획 = await 계획("SELECT count(*) FROM 신규기록 WHERE 상태 = '고장'", "(ANALYZE)");
```

```
const 낱은줄 = 낱은계획.split("\n").find((줄) => /Scan/.test(줄));

console.log("ANALYZE 전:", 낱은줄.trim().slice(0, 30), "...");
```

출력 ANALYZE 전: -> Index Only Scan using "신규_ ...
값이 다를 수 있음

```
console.log(`추정 ${낱은줄.match(/rows=(\d+) width/)[1]} 줄 / 실제  
${낱은줄.match(/loops/) ? 낱은줄.match(/rows=(\d.+)+ loops/)[1] : "?"} 줄`);
```

출력 추정 19 줄 / 실제 90001.00 줄
값이 다를 수 있음

```
await db.exec("ANALYZE 신규기록");

console.log("ANALYZE 후:", (await 계획("SELECT count(*) FROM 신규기록 WHERE 상태 =  
'고장'")).includes("Seq Scan") ? "Seq Scan 으로 바뀜" : "그대로");
```

출력 ANALYZE 후: Seq Scan 으로 바뀜

★ 대량으로 넣거나 고친 뒤에는 ANALYZE 표이름; 을 손으로 돌리세요.

후반부 — N+1 문제

지금부터는 색인 이야기가 아닙니다. 색인을 아무리 잘 걸어도 안 풀리는 문제입니다.

화면 하나를 만듭니다.

"A 라인 설비 100대를 보여 주고, 각 설비의 최근 점검기록도 같이 보여 준다"

코드로 짜면 자연스럽게 이렇게 됩니다.

```
const 설비목록질의 = "SELECT id, 이름 FROM 설비 WHERE 라인 = 'A' LIMIT 100";

async function 한대씩읽기() {
  const 설비들 = (await db.query(설비목록질의)).rows; // ← 1번 const 결과 =
  [];

  for (const 설비 of 설비들) {
    const 기록 = await db.query( // ← N번
      (100번) "SELECT id, 상태 FROM 점검기록 WHERE 설비번호 = $1",
      [설비.id],
    );
    결과.push({ ...설비, 기록수: 기록.rows.length });
  }

  return 결과;
}
```

```
const 한대씩결과 = await 한대씩읽기();
```

```
console.log(`설비 ${한대씩결과.length} 대, 데이터베이스 왕복 ${1 + 한대씩결과.length} 번`);
```

출력 설비 100 대, 데이터베이스 왕복 101 번

★★ 이게 **N+1 문제**입니다. 목록 1번 + 각 줄마다 1번씩 N번 = N+1 번. 설비가 1000대면 1001번입니다. 화면에 보이는 줄 수만큼 왕복이 늘어납니다.

고치는 법은 두 가지입니다.

고치기 ① — 한 번에 다 가져오기 (IN / = ANY)

```
async function 한번에읽기() {
  const 설비들 = (await db.query(설비목록질의)).rows;           // ← 1번 const 기록들
  = (
    await db.query(                                             // ← 1번. 끝 "SELECT
      설비번호, id, 상태 FROM 점검기록 WHERE 설비번호 = ANY($1)",
      [설비들.map((설비) => 설비.id)],
    )
  ).rows;
```

자바스크립트에서 설비별로 묶습니다

```
const 묶음 = new Map();
for (const 기록 of 기록들) {
  if (!묶음.has(기록.설비번호)) 묶음.set(기록.설비번호, []);
  묶음.get(기록.설비번호).push(기록);
}

return 설비들.map((설비) => ({ ...설비, 기록수: (묶음.get(설비.id) ?? []).length })));
}
```

★ = ANY(\$1) 은 배열을 통째로 넘깁니다. IN (1,2,3,...) 과 결과가 같습니다. ★★ 파라미터로 넘길 때는 IN 보다 = ANY 를 쓰세요.

``IN`` 은 개수만큼 ``$1, $2, ...`` 를 만들어야 해서 개수가 바뀔 때마다 질의 글자가 달라집니다. 그러면 계획 캐시가 매번 새로 만들어집니다.

고치기 ② — JOIN 한 번으로

```
async function 조인으로읽기() {
  const 줄들 = (
    await db.query(`
      SELECT s.id, s.이름, count(p.id) AS 기록수
```

```

LEFT JOIN 점검기록 p ON p.설비번호 = s.id
    GROUP BY s.id, s.이름
  `
  ).rows;                                     // ← 1번. 끝 return
출들;
}

```

```

const 한번에결과 = await 한번에읽기();
const 조인결과 = await 조인으로읽기();

```

```

console.log("세 방법의 결과 건수:", 한대씩결과.length, 한번에결과.length,
  조인결과.length);

```

```

출력      세 방법의 결과 건수: 100 100 100

```

```

console.log("기록수 합계가 같은가:",
  한대씩결과.reduce((합, 줄) => 합 + 줄.기록수, 0) === 한번에결과.reduce((합, 줄) =>
  합 + 줄.기록수, 0));

```

```

출력      기록수 합계가 같은가: true

```

얼마나 차이 나나 — 같은 기계에서

```

const 한대씩ms = await 재기(5, 한대씩읽기);
const 한번에ms = await 재기(5, 한번에읽기);
const 조인ms = await 재기(5, 조인으로읽기);

console.log(`N+1 ${한대씩ms.toFixed(1)} ms / ANY 한 번 ${한번에ms.toFixed(1)} ms /
  JOIN 한 번 ${조인ms.toFixed(1)} ms`);

```

```

출력      N+1 22.3 ms / ANY 한 번 4.1 ms / JOIN 한 번 1.4 ms
값이 다를 수 있음

```

```

console.log(`같은 기계에서 몇 배: ${(한대씩ms / 한번에ms).toFixed(1)}배`);

```

```

출력      같은 기계에서 몇 배: 5.5배
값이 다를 수 있음

```

```

console.log("한 번에 읽는 쪽이 빠른가:", 한번에ms < 한대씩ms);

```

```

출력      한 번에 읽는 쪽이 빠른가: true

```

★ 5배쯤입니다. “생각보다 별거 아니네” 싶으시죠. **그건 데이터베이스가 같은 컴퓨터 안(그것도 같은 프로세스 안)에 있어서 그렇습니다.**

★★ 진짜 서버에서는 — 망 왕복을 흉내 냅니다 —

회사에서는 애플리케이션 서버와 데이터베이스 서버가 다른 기계입니다. 같은 데이터센터 안이라도 왕복 0.5~1ms, 다른 지역이면 수십 ms 가 듭니다.

여기서는 **한 번 왕복에 3ms** 를 흉내 내 봅니다. 클라우드에서 흔한 값입니다.

```
const 왕복지연 = 3;
const 지연 = () => new Promise((끝내기) => setTimeout(끝내기, 왕복지연));

async function 지연질의(sql, 값들) {
  await 지연(); // 가는 길 + 오는 길 return db.query(sql,
  값들);
}

async function 느린N더하기1() {
  const 설비들 = (await 지연질의(설비목록질의)).rows;

  for (const 설비 of 설비들) {
    await 지연질의("SELECT id, 상태 FROM 점검기록 WHERE 설비번호 = $1", [설비.id]);
  }

  return 설비들.length;
}

async function 느린한번에() {
  const 설비들 = (await 지연질의(설비목록질의)).rows;
  await 지연질의("SELECT 설비번호, id, 상태 FROM 점검기록 WHERE 설비번호 = ANY($1)",
  [
    설비들.map((설비) => 설비.id),
  ]);
  return 설비들.length;
}

const 망N더하기1 = await 재기(3, 느린N더하기1);
const 망한번에 = await 재기(3, 느린한번에);

console.log(`망 왕복 ${왕복지연}ms 를 흉내 냈을 때 - N+1 ${망N더하기1.toFixed(0)} ms
/ 한 번에 ${망한번에.toFixed(0)} ms`);
```

출력 망 왕복 3ms 를 흉내 냈을 때 - N+1 403 ms / 한 번에 12 ms
값이 다를 수 있음

```
console.log(`몇 배: ${망N더하기1 / 망한번에}.toFixed(0)}배`);
```

출력 몇 배: 33배
값이 다를 수 있음

```
console.log("망이 끼면 차이가 훨씬 커지나:", 망N더하기1 / 망한번에 > 한대씩ms / 한번에ms);
```

```
출력      망이 끼면 차이가 훨씬 커지나: true
```

★★★ 계산으로도 확인해 봅시다.

N+1 : 101번 왕복 × 3ms = 303ms (+ 데이터베이스가 실제로 일한 시간) 한 번에 : 2번 왕복 × 3ms = 6ms (+ 데이터베이스가 실제로 일한 시간)

망 시간만 놓고 봐도 50배입니다. 질의 하나 하나는 0.05ms 로 아주 빠릅니다. 색인도 잘 걸려 있습니다. 그런데 화면 하나 그리는 데 0.4초가 걸립니다.

★ 이게 N+1 이 무서운 이유입니다. **느린 질의가 하나도 없는데 화면이 느립니다.** 느린 질의 로그를 봐도 아무것도 안 걸립니다. 전부 빠르니까요. 왕복 횟수를 세어 봐야 보입니다.

```
console.log(`계산으로: N+1 은 ${101 * 왕복지연}ms, 한 번에는 ${2 * 왕복지연}ms → ${((101 * 왕복지연) / (2 * 왕복지연)).toFixed(0)}배`);
```

```
출력      계산으로: N+1 은 303ms, 한 번에는 6ms → 51배
```

★ ORM 을 쓰면 자기도 모르게 납니다

이 코드는 for 문이 눈에 보여서 그나마 낫습니다. ORM(객체 관계 매핑 도구)을 쓰면 **for 문 없이도** N+1 이 납니다.

```
const 설비들 = await 설비.findAll({ where: { 라인: 'A' }, limit: 100 });
```

```
for (const 설비 of 설비들) {
```

```
console.log(설비.점검기록들.length); // ← 이 한 줄이 질의를 한 번 더 보냅니다
```

```
}
```

설비.점검기록들 은 그냥 속성처럼 생겼습니다. 그런데 읽는 순간 질의가 나갑니다. 이것 지연 로딩(lazy loading) 이라고 합니다.

★ 고치는 법은 ORM 마다 이름이 다르지만 하는 일은 같습니다.

```
Sequelize → include: [점검기록]
Prisma → include: { 점검기록들: true }
TypeORM → relations: ['점검기록들']
Django → prefetch_related('점검기록들')
```

전부 "미리 한 번에 가져와라" 는 뜻입니다.

★★ 08단원에서 애플리케이션과 이을 때 이 문제를 다시, 더 깊게 봅니다. 그때는 **날아간 질의를 전부 찍어 보는 법**도 같이 합니다.

정리 — 색인이 안 쓰이는 여섯 가지와 N+1

증상 원인 고치는 법

lower(칸) = ? 칸에 함수를 씌움 조건을 옮기거나 표현식 색인 칸 + 0 = ?, 칸::text = ? 칸에 계산/형변환 값 쪽을 바꾼다 LIKE '%중간%' 앞이 열려 있음 앞을 고정하거나 전문 검색 색인 문자칸 = 숫자 타입 불일치 PostgreSQL 은 에러를 냄(다행) 두 표에 걸친 OR 조인 뒤에 걸려야 함 UNION 으로 쪼갬다 대부분을 읽는 조건 플래너가 일부러 안 씌움 조건을 좁히거나 LIMIT 통계가 낫음 ANALYZE 를 안 함 ANALYZE 표이름

N+1 무엇 고치는 법

목록 1번 + 줄마다 1번 왕복 횟수가 줄 수만큼 늘어남 = ANY(\$1) 한 번 / JOIN 한 번 같은 기계에서 5배 망이 없으면 티가 잘 안 남 망 3ms 면 50배 질의는 다 빠르는데 화면이 느림

★★★ 느린 화면을 볼 때 순서 ① 왕복을 몇 번 하나 (N+1 인가) ← 여기가 제일 크다 ② EXPLAIN ANALYZE 로 rows 어긋남 확인 ③ 색인이 쓰이나 (위 여섯 가지 중 하나인가) ④ 정말로 색인이 없나

직접 해 볼 것

▫ 직접 해보기 1

WHERE date_trunc('day', 점검일) = DATE '2024-03-01' 로 찾아 보세요. 색인이 쓰이나요? 범위 조건 (>= AND <)으로 바꾸면요?

▫ 직접 해보기 2

함정 2 의 LIKE '설비77%' 를 LIKE '설비%' 로 바꿔 보세요. (2만 건 중 2만 건이 걸립니다) 색인을 쓰나요? 왜 그럴까요?

▫ 직접 해보기 3

함정 4 의 UNION 을 UNION ALL 로 바꿔 보세요. 결과 건수가 달라지나요? 왜 그럴까요?

▫ 직접 해보기 4

후반부의 설비 개수를 100 에서 500 으로 늘려 보세요. N+1 시간이 5배가 되나요? 한 번에 읽는 쪽은요?

▫ 직접 해보기 5

왕복지연을 3 에서 30 으로 바꿔 보세요. (다른 지역 서버라고 치고) 배수가 어떻게 되나요? ★ 재는 데 시간이 좀 걸립니다.

▫ 직접 해보기 6

= ANY(\$1) 에 넘기는 배열을 10000개로 만들면 어떻게 되나요? 한 번에 넘길 수 있는 개수에 한계가 있을까요? (힌트: 있습니다. 보통 1000개쯤에서 나눠 보내는 게 안전합니다)

▫ 직접 해보기 7

함정 6 에서 UPDATE 를 한 뒤 ANALYZE 를 하기 전과 후의 Execution Time 을 재 보세요. 차이가 크게 나타나요? ★ PGLite 는 전부 메모리라 차이가 작습니다. 왜 그런지 생각해 보세요.

자주 하는 실수

이런 실수를 합니다

“색인을 걸었으니 됐다” 고 생각함

질의가 색인을 못 쓰게 생겼으면 소용없습니다. **EXPLAIN** 으로 확인하세요. Seq Scan 이 보이면 이 파일의 여섯 가지를 하나씩 대 보세요.

이런 실수를 합니다

애플리케이션에서 값을 가공해서 조건을 만들

WHERE lower(이름) = \$1 처럼 칸 쪽을 건드리면 색인이 죽습니다. 가공은 값 쪽에서 하세요. WHERE 이름 = lower(\$1) 은 괜찮습니다.

이런 실수를 합니다

“OR 은 무조건 나쁘다” 고 외움

함정 4 에서 봤습니다. 한 표 안의 OR 은 BitmapOr 로 잘 돕니다. 문제는 두 표에 걸친 OR 입니다. 그때만 UNION 으로 쪼개세요.

이런 실수를 합니다

Seq Scan 을 보면 무조건 색인을 더 검

함정 5 를 보세요. 표의 절반을 읽는 질의에 색인을 걸어 봐야 안 쓰입니다. 쓰이게 강제하면 더 느려집니다.

이런 실수를 합니다

N+1 을 느린 질의 로그로 찾으려 함

못 찾습니다. 질의 하나하나는 다 빠릅니다. 왕복 횟수를 세야 보입니다. 08단원에서 세는 법을 합니다.

이런 실수를 합니다

로컬에서 재고 “N+1 이 별거 아니네” 라고 결론 냄

같은 기계에서는 5배, 망이 끼면 50배입니다. 로컬에서 **젠 값으로 망 성능을 판단하면 안 됩니다.**

이런 실수를 합니다

N+1 을 고친다고 데이터를 전부 가져옴

설비 100대의 기록만 필요한데 20만 건을 다 가져오면 그것도 문제입니다. = `ANY($1)` 로 **필요한 것만** 한 번에 가져오세요.

```
await db.close();
```

색인과 실행계획

RUN node 연습문제.js

★ 이 파일은 20초쯤 걸립니다. 20만 건을 만들어 놓고 시작합니다.

15문제입니다. 뒤로 갈수록 어렵습니다. 마지막 셋은 [도전] 입니다.

푸는 법: · 각 문제의 // TODO: 아래 변수에 답을 채웁니다 · node 연습문제.js 를 돌리면 채점해 줍니다 · 처음 돌리면 전부 “아직 안 풀었습니다” 라고 나옵니다. 정상입니다

★ 막히면 개념01~05 를 다시 보세요. 답을 먼저 보지 마세요. 정답은 연습문제_정답.js 에 있습니다. 왜 그런지 설명이 길게 붙어 있습니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();

await db.exec(`
  CREATE TABLE 설비 (
    id          SERIAL PRIMARY KEY,
    이름        TEXT NOT NULL,
    라인        TEXT NOT NULL,
    관리번호    TEXT NOT NULL
  );
  INSERT INTO 설비 (이름, 라인, 관리번호)
  SELECT '설비' || i, (ARRAY['A','B','C','D'])[i % 4] + 1, lpad(i::text, 8, '0')
  FROM generate_series(1, 20000) AS i;

  CREATE TABLE 점검기록 (
    id          SERIAL PRIMARY KEY,
    설비번호    INT  NOT NULL REFERENCES 설비(id),
    상태        TEXT NOT NULL,
    점검자      TEXT NOT NULL,
    소속반      TEXT NOT NULL,
    점검일      DATE NOT NULL,
    비고        TEXT
  );
  -- ★ 점검원N 은 반드시 반N 소속입니다. 둘은 짝입니다. (문제 13 에서 씁니다)
  INSERT INTO 점검기록 (설비번호, 상태, 점검자, 소속반, 점검일, 비고)
  SELECT (i % 20000) + 1,
         (ARRAY['정상','주의','고장'])[i % 3] + 1,
         '점검원' || ((i % 50) + 1),
```

```
DATE '2024-01-01' + ((i % 365) || ' days')::interval,
    '특이사항 없음. 기록번호 ' || i
FROM generate_series(1, 200000) AS i;
ANALYZE;
`);
```

채점 도구 (고치지 마세요)

```
const 결과 = [];

async function 계획(sql, 옵션 = "") {
    return (await db.query(`EXPLAIN ${옵션} ${sql}`)).rows.map((줄) => 줄["QUERY PLAN"]).join("\n");
}

async function 채점(번호, 제목, 답, 검사) {
    const 비었나 = 답 === null || 답 === undefined || (typeof 답 === "string" &&
    답.trim() === "");

    if (비었나) {
        결과.push(false);
        console.log(`${번호}. ${제목} - 아직 안 풀었습니다`);
        return;
    }

    try {
        const 통과 = await 검사(답);
        결과.push(통과 === true);
        console.log(`${번호}. ${제목} - ${통과 === true ? "통과" : "틀렸습니다"}`);
    } catch (에러) {
        결과.push(false);
        console.log(`${번호}. ${제목} - 에러가 났습니다 (${에러.code ?? "?"})`);
    }
}
```

문제 1 — 색인 만들기

점검기록의 점검자 칸으로 찾는 질의가 느립니다. 색인을 만드는 SQL 을 쓰세요. 색인 이름은 점검_점검자로 하세요.

CREATE INDEX ... 문장을 문자열로 쓰세요

```
const 답1 = "";

await 채점(1, "점검자에 색인 만들기", 답1, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 점검기록");
  const 계획글 = await 계획("SELECT * FROM 점검기록 WHERE 점검자 = '점검원7'");
  return 계획글.includes("점검_점검자");
});
```

출력 1. 점검자에 색인 만들기 – 아직 안 풀었습니다

문제 2 — Seq Scan 인지 확인하기

아래 질의의 실행계획에서 **최상위 스캔 노드 이름**을 문자열로 쓰세요. (색인이 없는 칸으로 찾습니다)

SELECT * FROM 점검기록 WHERE 비고 = '없음'

“Seq Scan” 처럼 노드 이름만 쓰세요

```
const 답2 = "";

await 채점(2, "어떤 스캔이 나오나", 답2, async (이름) => {
  const 계획글 = await 계획("SELECT * FROM 점검기록 WHERE 비고 = '없음'");
  return 계획글.split("\n")[0].startsWith(이름.trim());
});
```

출력 2. 어떤 스캔이 나오나 – 아직 안 풀었습니다

문제 3 — 외래키에 색인 걸기

점검기록의 **설비번호**는 설비를 참조하는 외래키입니다. 그런데 PostgreSQL 은 외래키에 색인을 자동으로 안 만들어 줍니다. 직접 만드세요. 이름은 **점검_설비번호**로 하세요.

여기에 코드를 쓰세요

```
const 답3 = "";

await 채점(3, "외래키에 색인 걸기", 답3, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 점검기록");
  const 계획글 = await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 777");
  return 계획글.includes("점검_설비번호") && !계획글.includes("Seq Scan");
});
```

출력 3. 외래키에 색인 걸기 - 아직 안 풀었습니다

문제 4 — 복합 색인의 칸 순서

“3월에 고장 난 기록”을 뽑는 화면이 있습니다.

```
SELECT * FROM 점검기록 WHERE 상태 = '고장' AND 점검일 BETWEEN DATE '2024-03-01' AND
DATE '2024-03-31'
```

이 질의를 가장 잘 처리할 복합 색인을 만드세요. 이름은 점검_복합 입니다.

★ 등호 조건(상태)과 범위 조건(점검일) 중 어느 쪽이 앞이어야 할까요? 두 조건이 **모두** Index Cond 에 들어가야 통과입니다.

여기에 코드를 쓰세요

```
const 답4 = "";

await 채점(4, "복합 색인 순서", 답4, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 점검기록");

  const 계획글 = await 계획(
    `SELECT * FROM 점검기록
      WHERE 상태 = '고장' AND 점검일 BETWEEN DATE '2024-03-01' AND DATE '2024-03-31'`,
  );
```

색인을 쓰면서, 두 조건 모두 Index Cond 로 들어가야 통과입니다

```
const 색인조건 = 계획글.split("\n").filter((줄) => 줄.includes("Index Cond")).join("
");
return 계획글.includes("점검_복합") && 색인조건.includes("상태") &&
색인조건.includes("점검일");
});
```

출력 4. 복합 색인 순서 - 아직 안 풀었습니다

문제 5 — 부분 색인

현장 화면은 **고장 난 기록만** 봅니다. 전체의 3분의 1입니다. **점검일** 로 찾되 **고장인 줄만** 담는 부분 색인을 만드세요. 이름은 **점검_고장부분** 입니다.

조건: 만들어진 색인의 크기가 전체 색인의 절반보다 작아야 통과입니다.

여기에 코드를 쓰세요

```
const 답5 = "";

await 채점(5, "부분 색인", 답5, async (sql) => {
  await db.exec(sql);
  await db.exec("CREATE INDEX 비교용_전체 ON 점검기록 (점검일)");
  await db.exec("ANALYZE 점검기록");

  const 크기 = (
    await db.query("SELECT pg_relation_size('점검_고장부분') AS 부분,
pg_relation_size('비교용_전체') AS 전체")
  ).rows[0];

  const 계획글 = await 계획("SELECT * FROM 점검기록 WHERE 상태 = '고장' AND 점검일 =
DATE '2024-03-01'");

  await db.exec("DROP INDEX 비교용_전체");

  return 계획글.includes("점검_고장부분") && Number(크기.부분) < Number(크기.전체) /
2;
});
```

출력 5. 부분 색인 - 아직 안 풀었습니다

문제 6 — 표현식 색인

대소문자를 무시하고 설비 이름을 찾습니다.

```
SELECT * FROM 설비 WHERE lower(이름) = '설비777'
```

지금은 Seq Scan 입니다. 색인이 쓰이게 만드세요. 이름은 설비_소문자이름 입니다.

여기에 코드를 쓰세요

```
const 답6 = "";

await 채점(6, "표현식 색인", 답6, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 설비");
  const 계획글 = await 계획("SELECT * FROM 설비 WHERE lower(이름) = '설비777'");
  return 계획글.includes("설비_소문자이름");
});
```

출력 6. 표현식 색인 - 아직 안 풀었습니다

문제 7 — 커버링 색인으로 Index Only Scan 만들기

이 질의가 표를 아예 안 보게 만드세요.

```
SELECT 설비번호, 상태 FROM 점검기록 WHERE 설비번호 = 777
```

색인 이름은 점검_커버 입니다. ★ 색인만 만들면 안 됩니다. 개념03 섹션 5 에서 뭘 더 해야 했나요?
(채점기가 VACUUM 은 대신 돌려 줍니다)

여기에 코드를 쓰세요


```
const 답7 = "";

await 채점(7, "Index Only Scan 만들기", 답7, async (sql) => {
  await db.exec(sql);
  await db.exec("VACUUM ANALYZE 점검기록");
  const 계획글 = await 계획("SELECT 설비번호, 상태 FROM 점검기록 WHERE 설비번호 = 777", "(ANALYZE)");
  return 계획글.includes("Index Only Scan") && 계획글.includes("Heap Fetches: 0");
});
```

출력 7. Index Only Scan 만들기 - 아직 안 풀었습니다

문제 8 — 색인을 못 쓰게 만든 질의 고치기 ①

이 질의는 색인이 있는데도 Seq Scan 입니다.

SELECT * FROM 점검기록 WHERE 설비번호::text = '777'

색인을 쓰도록 질의를 고쳐 쓰세요. 결과는 같아야 합니다. (색인을 새로 만들지 마세요. 문제 3 에서 만든 색인을 쓰면 됩니다)

고친 SELECT 문을 문자열로 쓰세요

```
const 답8 = "";

await 채점(8, "형변환 때문에 죽은 색인 살리기", 답8, async (sql) => {
  const 계획글 = await 계획(sql);
  const 줄수 = (await db.query(sql)).rows.length;
  return !계획글.includes("Seq Scan") && 줄수 === 10;
});
```

출력 8. 형변환 때문에 죽은 색인 살리기 - 아직 안 풀었습니다

문제 9 — 색인을 못 쓰게 만든 질의 고치기 ②

설비 이름이 '설비777' 로 **시작하는** 것을 찾고 싶습니다. 지금 코드는 이렇게 돼 있습니다.

SELECT * FROM 설비 WHERE 이름 LIKE '%설비777%'

색인을 쓰도록 고쳐 쓰세요. (설비_이름 색인은 채점기가 미리 만들어 둡니다)

여기에 코드를 쓰세요

```
await db.exec("CREATE INDEX 설비_이름 ON 설비 (이름)");
await db.exec("ANALYZE 설비");

const 답9 = "";

await 채점(9, "LIKE 앞을 고정하기", 답9, async (sql) => {
  const 계획글 = await 계획(sql);
  const 줄수 = (await db.query(sql)).rows.length;
  return 계획글.includes("설비_이름") && 줄수 >= 1;
});
```

출력 9. LIKE 앞을 고정하기 - 아직 안 풀었습니다

문제 10 — 두 표에 걸친 OR 고치기

이 질의는 20만 줄을 전부 이어 붙인 뒤에 거릅니다.

SELECT p.id FROM 점검기록 p JOIN 설비 s ON p.설비번호 = s.id WHERE s.이름 = '설비77' OR p.점검자 = '점검원7'

결과가 같으면서 색인을 쓰는 질의로 고쳐 쓰세요.

여기에 코드를 쓰세요

```
const 답10 = "";

await 채점(10, "두 표에 걸친 OR 고치기", 답10, async (sql) => {
  const 원래 = (
    await db.query(`
      SELECT p.id FROM 점검기록 p JOIN 설비 s ON p.설비번호 = s.id
      WHERE s.이름 = '설비77' OR p.점검자 = '점검원7'`)
  ).rows.length;

  const 고친것 = (await db.query(sql)).rows.length;
  const 계획글 = await 계획(sql);
```

트랜잭션과 동시성

OPEN 07_트랜잭션과_동시성

```
import pg from "pg";  
const 접속정보 = {  
  host: "127.0.0.1",  
  user: "factory",
```

01 트랜잭션이 무엇인가

02 동시에 고치면 사라집니다

03 격리수준 네 가지

04 잠금

05 데드락

- 연습문제

트랜잭션이 무엇인가

RUN node 개념01_트랜잭션이_무엇인가.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

01단원에서 파일 저장이 못 하는 것 다섯 가지를 봤습니다. 그중 두 번째가 이것이었습니다.

"중간 실패를 못 되돌림 — 설비를 지우다 중간에 죽으면 점검기록만 남습니다"

이 단원은 그것을 푼니다.

설비 하나를 폐기하는 일은 **한 문장이 아닙니다.**

① 그 설비의 점검기록을 정리한다 ② 설비 목록에서 지운다 ③ 폐기대장에 남긴다

세 문장입니다. 그런데 현실에서 이건 **하나의 일**입니다. ①만 되고 ②③이 안 되면, 점검기록만 날아간 채 설비는 그대로 남습니다. 아무도 모릅니다. 몇 달 뒤에 "이 설비 점검 기록이 왜 없죠?" 로 발견됩니다.

트랜잭션은 이 세 문장을 **전부 되거나 전부 안 되게** 묶는 장치입니다.

★ 왜 06단원까지는 이걸 안 했나 PGlite 는 연결이 하나뿐입니다. 트랜잭션은 혼자서도 배울 수 있지만, 다음 개념부터 나오는 동시성은 연결이 둘 이상이어야 합니다. 그래서 07단원부터 Docker 로 진짜 PostgreSQL 서버를 씁니다.

```
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434, // * 5432 가 아닙니다. 이미 깔린 DB 와 안 부딪히게 비켜 뒀습니다 user:
  "factory",
  password: "secret",
  database: "factory_db",
};
```

★ Docker 가 안 떠 있으면 여기서 붙지 못합니다. 그때는 안내하고 곱게 끝냅니다.

```

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("Docker Postgres 에 못 붙었습니다. 먼저 이걸 실행하세요:");
  console.log(" docker compose up -d");
  console.log(` (원인: ${에러.message})`);
  process.exit(0);
}

```

★★ 07~10단원이 같은 데이터베이스를 씁니다. 단원끼리 표 이름이 부딪히지 않게 각 단원이 **자기 스키마**를 씁니다. 스키마(schema)는 표를 담는 폴더라고 생각하면 됩니다.

```

await 연결.query("CREATE SCHEMA IF NOT EXISTS 단원07");
await 연결.query("SET search_path TO 단원07");

```

여러 번 돌려도 같은 결과가 나오게 매번 새로 만듭니다.

```
await 연결.query("DROP TABLE IF EXISTS 폐기대장, 점검기록, 설비 CASCADE");
```

```
await 연결.query(`
  CREATE TABLE 설비 (
    번호 INT PRIMARY KEY,
    이름 TEXT NOT NULL,
    라인 TEXT NOT NULL,
    상태 TEXT NOT NULL DEFAULT '가동'
  )
`);
```

```
await 연결.query(`
  CREATE TABLE 점검기록 (
    번호 SERIAL PRIMARY KEY,
    설비번호 INT NOT NULL REFERENCES 설비(번호),
    내용 TEXT NOT NULL
  )
`);
```

```
await 연결.query(`
  CREATE TABLE 폐기대장 (
    번호 SERIAL PRIMARY KEY,
    설비이름 TEXT NOT NULL,
    사유 TEXT NOT NULL
  )
`);
```

```
await 연결.query(`
  INSERT INTO 설비 VALUES
    (1, '컨베이어 1호', 'A', '가동'),
    (2, '프레스 1호', 'B', '가동'),
    (3, '용접로봇 1호', 'C', '정지')
`);
```

```
await 연결.query(`
  INSERT INTO 점검기록 (설비번호, 내용) VALUES
    (1, '벨트 장력 점검'), (1, '모터 소음 확인'), (2, '유압 누유 점검')
`);
```

자주 쓸 것들을 짧은 함수로 만들어 둡니다.

```
const 한값 = async (문장, 값들 = []) => Object.values((await 연결.query(문장,
값들)).rows[0])[0]);
const 설비수 = () => 한값("SELECT count(*) FROM 설비");
const 점검수 = () => 한값("SELECT count(*) FROM 점검기록");
const 대장수 = () => 한값("SELECT count(*) FROM 폐기대장");

console.log(`설비 ${await 설비수()}대 · 점검기록 ${await 점검수()}건 · 폐기대장
${await 대장수()}건`);
```

출력 설비 3대 · 점검기록 3건 · 폐기대장 0건

★ count(*) 가 문자열로 옵니다. 숫자가 아닙니다. Postgres 의 count 는 BIGINT 인데, BIGINT 를 number 로 바꾸면 큰 수에서 값이 깨집니다. 그래서 드라이버가 안전하게 문자열로 줍니다. 계산하려면 Number() 로 바꾸세요.

★ 트랜잭션 없이 폐기해 봅니다

섹션 1

설비 1번(컨베이어 1호)을 폐기합니다. 세 문장입니다. 그런데 ③에서 실수를 합니다. 사유를 안 넣었습니다.

```
try {
  await 연결.query("DELETE FROM 점검기록 WHERE 설비번호 = 1"); // ① 점검기록
  정리 await 연결.query("DELETE FROM 설비 WHERE 번호 = 1"); // ② 설비 지우기 await
  연결.query("INSERT INTO 폐기대장 (설비이름) VALUES ('컨베이어 1호')"); // ③ 사유를
  빠뜨렸습니다
} catch (에러) {
  console.log(`③ 에서 실패: ${에러.code} ${에러.message}`);
}
```

출력 ③ 에서 실패: 23502 null value in column "사유" of relation "폐기대장"
violates not-null constraint

```
}

console.log(`남은 것 - 설비 ${await 설비수()}대 · 점검기록 ${await 점검수()}건 ·
폐기대장 ${await 대장수()}건`);
```

출력 남은 것 - 설비 2대 · 점검기록 1건 · 폐기대장 0건

★★★ 이게 최악의 상태입니다.

설비는 지워졌습니다. (되돌릴 수 없습니다) 점검기록도 지워졌습니다. (되돌릴 수 없습니다) 폐기대장에는 없습니다. (이 설비가 왜 사라졌는지 아무 기록이 없습니다)

①②는 됐고 ③만 안 됐습니다. **반쯤 된 상태** 입니다. 에러 메시지는 ③ 이야기만 하지, ①②를 되돌려 주지 않습니다.

★ 코드로 되돌리면 되지 않나요?

`catch` 안에서 다시 `INSERT` 하면 됩니다. 그런데

- 되돌리는 코드 자체가 실패하면?
- 되돌리는 중에 프로세스가 죽으면?
- `SERIAL` 로 매긴 점검기록 번호는 어떻게 되살리나요?

결국 데이터베이스가 해 줘야 합니다.

원상복구하고 다시 시작합니다.

```
await 연결.query("INSERT INTO 설비 VALUES (1, '컨베이어 1호', 'A', '가동')");
await 연결.query("INSERT INTO 점검기록 (설비번호, 내용) VALUES (1, '벨트 장력 점검'),
(1, '모터 소음 확인')");
```

BEGIN · COMMIT · ROLLBACK

섹션 2

트랜잭션은 세 마디로 끝납니다.

BEGIN	여기서부터 한 덩어리
COMMIT	전부 확정
ROLLBACK	전부 없던 일로

같은 실수를 트랜잭션 안에서 해 봅니다.

```
await 연결.query("BEGIN");

try {
  await 연결.query("DELETE FROM 점검기록 WHERE 설비번호 = 1");
  await 연결.query("DELETE FROM 설비 WHERE 번호 = 1");
  await 연결.query("INSERT INTO 폐기대장 (설비이름) VALUES ('컨베이어 1호')"); // 또
  빠뜨립니다 await 연결.query("COMMIT");
} catch (에러) {
  console.log(`실패했으니 되돌립니다: ${에러.code}`);
}
```

출력 실패했으니 되돌립니다: 23502

```
await 연결.query("ROLLBACK");
}

console.log(`되돌린 뒤 - 설비 ${await 설비수()}대 · 점검기록 ${await 점검수()}건 ·
폐기대장 ${await 대장수()}건`);
```

출력 되돌린 뒤 - 설비 3대 · 점검기록 3건 · 폐기대장 0건

★★ 아무것도 안 바뀌었습니다. 처음 그대로입니다. `DELETE` 두 개가 분명히 성공했는데도 남아 있습니다. 트랜잭션 안에서 한 일은 `COMMIT` 전까지는 **아직 일어나지 않은 일**입니다.

이번엔 제대로 해 봅시다.

```
await 연결.query("BEGIN");
await 연결.query("DELETE FROM 점검기록 WHERE 설비번호 = 1");
await 연결.query("DELETE FROM 설비 WHERE 번호 = 1");
await 연결.query("INSERT INTO 폐기대장 (설비이름, 사유) VALUES ('컨베이어 1호', '노후')");
await 연결.query("COMMIT");

console.log(`제대로 폐기 - 설비 ${await 설비수()}대 · 점검기록 ${await 점검수()}건 · 폐기대장 ${await 대장수()}건`);
```

출력 제대로 폐기 - 설비 2대 · 점검기록 1건 · 폐기대장 1건

세 문장이 **전부** 됐습니다. 이제 되돌릴 수 없습니다. 그게 COMMIT 입니다.

★ BEGIN 을 안 쓰면 — 자동 커밋

섹션 3

“그럼 지금까지 06단원에서 쓴 문장들은 트랜잭션이 아니었나요?” 아닙니다. **전부 트랜잭션이었습니다.**

★ BEGIN 없이 보낸 문장은 그 문장 하나가 통째로 하나의 트랜잭션입니다.

보내는 순간 시작하고, 끝나는 순간 커밋합니다. 이걸 자동 커밋이라고 합니다.

그래서 UPDATE 한 줄이 반만 되는 일은 없습니다. 문제는 **여러 문장을 묶고 싶을 때** 생깁니다.

옆에서 지켜볼 두 번째 연결을 하나 만듭니다.

```
const 옆사람 = new pg.Client(접속정보);
await 옆사람.connect();
await 옆사람.query("SET search_path TO 단위07");

const 옆에서본상태 = async () =>
  (await 옆사람.query("SELECT 상태 FROM 설비 WHERE 번호 = 2")).rows[0].상태;
```

먼저 BEGIN 없이 바꿔 봅시다.

```
await 연결.query("UPDATE 설비 SET 상태 = '점검중' WHERE 번호 = 2");
console.log("BEGIN 없이 바꾼 직후, 옆사람 눈에는:", await 옆에서본상태());
```

출력 BEGIN 없이 바꾼 직후, 옆사람 눈에는: 점검중

이번엔 BEGIN 안에서 바꿉니다.

```
await 연결.query("BEGIN");
await 연결.query("UPDATE 설비 SET 상태 = '고장' WHERE 번호 = 2");

console.log("트랜잭션 안, 나에게는:", (await 연결.query("SELECT 상태 FROM 설비 WHERE
번호 = 2")).rows[0].상태);
```

출력 트랜잭션 안, 나에게는: 고장

```
console.log("트랜잭션 안, 옆사람 눈에는:", await 옆에서본상태());
```

출력 트랜잭션 안, 옆사람 눈에는: 점검중

```
await 연결.query("ROLLBACK");

console.log("롤백한 뒤, 나에게도:", (await 연결.query("SELECT 상태 FROM 설비 WHERE
번호 = 2")).rows[0].상태);
```

출력 롤백한 뒤, 나에게도: 점검중

★★ 커밋하지 않은 변경은 **나만 보입니다**. 내 화면에서는 이미 '고장' 인데, 남의 화면에서는 아직 '점검중' 입니다. 이 성질에 이름이 붙어 있습니다. **격리성** 입니다. 개념03 에서 자세히 봅니다.

ACID 를 외우지 말고 눌러 봅니다

섹션 4

트랜잭션의 성질을 네 글자로 줄여 ACID 라고 부릅니다. 시험에는 나오지만 외워도 쓸모가 없습니다. 하나씩 실제로 확인합니다.

A · 원자성(Atomicity) — 전부 되거나 전부 안 되거나

섹션 2 에서 이미 봤습니다. 다시 한 번 짧게 확인합니다.

```
const 폐기전설비수 = Number(await 설비수());

await 연결.query("BEGIN");
await 연결.query("DELETE FROM 설비 WHERE 번호 = 3");
await 연결.query("ROLLBACK");

console.log("A · 지웠다 되돌린 뒤 설비 수가 그대로인가:", Number(await 설비수()) ===
폐기전설비수);
```

출력 A · 지웠다 되돌린 뒤 설비 수가 그대로인가: true

C · 일관성(Consistency) — 규칙을 어긴 채로는 커밋이 안 됩니다

설비 2번에는 점검기록이 달려 있습니다. 외래키가 막습니다.

```
await 연결.query("BEGIN");

try {
  await 연결.query("DELETE FROM 설비 WHERE 번호 = 2");
  await 연결.query("COMMIT");
  console.log("C · 지워졌습니다");
} catch (에러) {
  console.log(`C · 규칙이 막았습니다: ${에러.code} - ${에러.detail}`);
}
```

출력 C · 규칙이 막았습니다: 23503 - Key (번호)=(2) is still referenced from table "점검기록".

```
await 연결.query("ROLLBACK");
}
```

★ 04단원에서 만든 외래키가 여기서 일합니다. 제약이 없으면 트랜잭션이 있어도 이상한 데이터가 그대로 커밋됩니다. 트랜잭션은 규칙을 지켜 주지 않습니다. 규칙을 어기면 막아 줄 뿐입니다.

I · 격리성(Isolation) — 남의 미완성이 나에게 안 보입니다

섹션 3 에서 봤습니다. 커밋 전에는 옆사람에게 '점검중' 으로 보였습니다.

```
await 연결.query("BEGIN");
await 연결.query("UPDATE 설비 SET 이름 = '없는이름' WHERE 번호 = 2");
const 남에게보이나 = (await 옆사람.query("SELECT 이름 FROM 설비 WHERE 번호 = 2")).rows[0].이름;
await 연결.query("ROLLBACK");

console.log("I · 커밋 전 변경이 남에게 보이나:", 남에게보이나 === "없는이름");
```

출력 I · 커밋 전 변경이 남에게 보이나: false

D · 지속성(Durability) — 커밋한 것은 살아남습니다

연결을 끊고 새로 붙여도 남아 있어야 합니다. 프로세스가 죽었다 살아난 셈입니다.

```

await 연결.query("BEGIN");
await 연결.query("UPDATE 설비 SET 상태 = '폐기예정' WHERE 번호 = 3");
await 연결.query("COMMIT");

await 옆사람.end(); // 연결을 진짜로 끊습니다 const 새사람 = new pg.Client(접속정보);
await 새사람.connect();
await 새사람.query("SET search_path TO 단원07");

const 살아남았나 = (await 새사람.query("SELECT 상태 FROM 설비 WHERE 번호 = 3")).rows[0].상태;
console.log("D · 연결을 끊었다 새로 붙어도 남아 있나:", 살아남았나);

```

출력 D · 연결을 끊었다 새로 붙어도 남아 있나: 폐기예정

★ 지속성은 “커밃했다고 대답했으면 정전이 나도 남는다” 는 약속입니다. Postgres 는 데이터 파일을 고치기 **전에** 변경 내역을 WAL 이라는 기록장에 먼저 씁니다. 그래서 갑자기 꺼져도 다시 켤 때 WAL 을 읽어 복구합니다. 10단원에서 다시 봅니다.

★★ 에러가 난 트랜잭션은 그 뒤로 아무것도 못 합니다

섹션 5

여기서 사람들이 가장 많이 넘어집니다.

```

await 연결.query("BEGIN");
await 연결.query("UPDATE 설비 SET 상태 = '가동' WHERE 번호 = 2");

try {
  await 연결.query("SELECT * FROM 없는표");
} catch (에러) {
  console.log(`1) 일부러 낸 에러: ${에러.code}`);
}

```

출력 1) 일부러 낸 에러: 42P01

```

}

```

이제 아주 멀쩡한 문장을 보내 봅니다.

```

try {
  await 연결.query("SELECT 1");
} catch (에러) {
  console.log(`2) 멀쩡한 SELECT 1 도 거부: ${에러.code}`);
}

```

출력 2) 멀쩡한 SELECT 1 도 거부: 25P02

```

console.log(`   ${에러.message}`);

```

```
출력      current transaction is aborted, commands ignored until end of
          transaction block
```

```
}
```

COMMIT 을 보내도 커밋이 안 됩니다. 롤백으로 처리됩니다.

```
await 연결.query("COMMIT");

console.log("3) COMMIT 했는데 UPDATE 가 남았나:", (await 연결.query("SELECT 상태 FROM
설비 WHERE 번호 = 2")).rows[0].상태);
```

```
출력      3) COMMIT 했는데 UPDATE 가 남았나: 점검중
```

★★★ 정리하면 이렇습니다.

트랜잭션 안에서 문장 하나가 실패하면 → 그 트랜잭션은 **오염(aborted)** 상태가 됩니다 → 그 뒤 모든 문장이 25P02 로 거부됩니다 → COMMIT 을 보내도 ROLLBACK 처럼 처리됩니다

★ 왜 이렇게 만들었나

```
"실패한 문장만 빼고 나머지는 커밋" 은 반쯤 된 상태를 만듭니다.
그건 섹션 1 에서 본 그 지옥입니다. 그래서 통째로 막습니다.
```

★★ 노드 드라이버가 알아서 롤백해 주지 않습니다.

```
catch 에서 ROLLBACK 을 안 보내면 그 연결은 오염된 채로 남습니다.
연결 풀을 쓰면 그 연결이 다음 요청에 그대로 넘어갑니다.
그러면 **아무 죄 없는 다음 요청이 25P02 로 죽습니다.**
실무에서 원인 찾기 제일 어려운 종류의 버그입니다.
```

MySQL 은 여기가 다릅니다 —————

```
MySQL 은 문장 하나가 실패해도 트랜잭션이 오염되지 않습니다.
그 문장만 취소되고 나머지는 계속됩니다.
* 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다
```

SAVEPOINT — 트랜잭션 안의 중간 저장

섹션 6

“일부만 실패해도 되는 경우” 는 실제로 있습니다. 점검기록 열 건을 넣는데 그중 하나가 형식이 틀렸다고 아홉 건을 버릴 이유는 없습니다.

SAVEPOINT 는 트랜잭션 안에 되돌아올 지점을 찍어 둡니다.

```

await 연결.query("BEGIN");
await 연결.query("INSERT INTO 점검기록 (설비번호, 내용) VALUES (2, '정상 기록 1')");

await 연결.query("SAVEPOINT 여기까지는_확실");

try {

```

없는 설비 9번에 기록을 담니다. 외래키가 막습니다.

```

await 연결.query("INSERT INTO 점검기록 (설비번호, 내용) VALUES (9, '잘못된 기록')");
} catch (에러) {
  console.log(`이상한 건 하나 실패: ${에러.code}`);

```

출력 이상한 건 하나 실패: 23503

★ 여기서 세이브포인트로 되돌리면 오염이 풀립니다.

```

await 연결.query("ROLLBACK TO SAVEPOINT 여기까지는_확실");
}

```

오염이 풀렸으니 다시 일할 수 있습니다.

```

await 연결.query("INSERT INTO 점검기록 (설비번호, 내용) VALUES (2, '정상 기록 2')");
await 연결.query("COMMIT");

console.log("점검기록 건수:", await 점검수());

```

출력 점검기록 건수: 3

★ 잘못된 한 건만 빠지고 나머지 두 건은 들어갔습니다. ROLLBACK TO SAVEPOINT 는 **세이브포인트 이후만** 되돌립니다. 그 앞에서 한 일('정상 기록 1')은 그대로 살아 있습니다.

★ RELEASE SAVEPOINT 이름 — 더 안 쓸 세이브포인트를 지웁니다. 안 지워도 COMMIT 하면 같이 사라집니다.

★★ 남발하지 마세요. 세이브포인트 하나마다 내부 트랜잭션이 하나씩 생깁니다. 반복문 안에서 수만 번 찍으면 눈에 띄게 느려집니다. “한 건씩 실패를 허용해야 한다” 는 요구가 진짜 있을 때만 쓰세요.

노드에서 안전하게 쓰는 형태

섹션 7

실무에서 트랜잭션은 거의 항상 이 모양입니다. 이 틀을 외우세요.

```

async function 트랜잭션으로(연결, 할일) {
  await 연결.query("BEGIN");

  try {
    const 결과 = await 할일(연결);
    await 연결.query("COMMIT");
    return 결과;
  } catch (에러) {

```

★★ 여기가 핵심입니다. 실패하면 반드시 되돌립니다. ROLLBACK 자체도 실패할 수 있으니(연결이 끊긴 경우) 감싸 둡니다.

```

    try {
      await 연결.query("ROLLBACK");
    } catch {

```

이미 연결이 끊겼으면 서버가 알아서 롤백합니다. 여기서 할 일은 없습니다.

```

    }

    throw 에러; // * 삼키지 마세요. 부르는 쪽이 실패를 알아야 합니다
  }
}

```

잘 되는 경우

```

const 폐기결과 = await 트랜잭션으로(연결, async (연결) => {
  await 연결.query("DELETE FROM 점검기록 WHERE 설비번호 = 3");
  await 연결.query("DELETE FROM 설비 WHERE 번호 = 3");
  const 넣은것 = await 연결.query(
    "INSERT INTO 폐기대장 (설비이름, 사유) VALUES ($1, $2) RETURNING 번호",
    ["용접로봇 1호", "라인 폐쇄"],
  );
  return 넣은것.rows[0].번호;
});

console.log("폐기대장 번호:", 폐기결과, "· 남은 설비:", await 설비수());

```

```
출력      폐기대장 번호: 4 · 남은 설비: 1
```

★★ 잠깐. 폐기대장에는 지금 두 건밖에 없는데 번호가 4 입니다.

앞에서 폐기대장에 INSERT 를 네 번 시도했습니다. 그중 두 번은 사유를 빠뜨려 실패했고, 한 번은 롤백까지 했습니다. 그런데 **번호는 네 개가 다 쓰였습니다.**

★ 순번(SEQUENCE)은 롤백되지 않습니다.

되돌린다면 어떻게 될까요? 동시에 열 명이 번호를 받아 갔는데 그중 세 명이 롤백하면, 되돌리려고 서로를 기다려야 합니다. 그러면 번호 하나 받는 데 줄을 서게 됩니다. 그래서 안 되돌립니다.

★★ 그래서 SERIAL 번호에는 **구멍이 생깁니다**.

"1번부터 빈틈없이 이어져야 한다" 는 요구(영수증 번호, 세금계산서 번호)에는 SERIAL 을 쓰면 안 됩니다. 감사에서 "3번이 왜 없죠?" 를 설명하지 못합니다. 그런 번호는 따로 표를 두고 잠금을 걸어서 매겨야 합니다.

실패하는 경우

```
try {
  await 트랜잭션으로(연결, async (연결) => {
    await 연결.query("DELETE FROM 설비 WHERE 번호 = 2"); // 점검기록이 달려 있어
    실패합니다
  });
} catch (에러) {
  console.log("실패가 밖으로 전달됐나:", 에러.code === "23503");
```

출력 실패가 밖으로 전달됐나: true

```
}

console.log("실패 뒤에도 연결이 멀쩡한가:", (await 연결.query("SELECT 1 AS
값")).rows[0].값 === 1);
```

출력 실패 뒤에도 연결이 멀쩡한가: true

★ ROLLBACK 을 했기 때문에 이 연결을 계속 쓸 수 있습니다. 안 했다면 위 SELECT 1 이 25P02 로 죽었을 것입니다.

★ 연결 풀을 쓸 때는 finally 에서 반납까지 해야 합니다. 08단원에서 합니다.

```
const 연결 = await 풀.connect();
try { ... } finally { 연결.release(); }
```

★★ 트랜잭션 안에서 외부 API 를 부르지 마세요. BEGIN 을 열어 둔 채 결제 서버 응답을 3초 기다리면 그 3초 동안 잠금을 쥐고 있습니다. 개념04 에서 무슨 일이 나는지 봅니다.

읽기 전용 트랜잭션

섹션 8

여러 표를 한 시점 기준으로 읽고 싶을 때 씁니다. 보고서를 만드는 도중에 숫자가 바뀌면 합계가 안 맞습니다.


```
await 연결.query("BEGIN READ ONLY");
const 설비합 = await 한값("SELECT count(*) FROM 설비");

try {
  await 연결.query("DELETE FROM 설비 WHERE 번호 = 1");
} catch (에러) {
  console.log(`읽기 전용에서 쓰기 시도: ${에러.code}`);
}
```

출력 읽기 전용에서 쓰기 시도: 25006

```
}

await 연결.query("ROLLBACK");

console.log("읽기 전용으로 읽은 설비 수:", 설비합);
```

출력 읽기 전용으로 읽은 설비 수: 1

★ 실수로 쓰는 것을 막아 줍니다. 통계·보고서 코드에는 붙여 두면 좋습니다.

정리

명령 하는 일

BEGIN	여기부터 한 덩어리로 묶는다
COMMIT	묶은 것을 전부 확정한다
ROLLBACK	묶은 것을 전부 없던 일로 한다

SAVEPOINT 이름 트랜잭션 안에 되돌아올 지점을 찍는다 ROLLBACK TO SAVEPOINT 이름 그 지점
뒤만 되돌린다 (오염도 풀린다) RELEASE SAVEPOINT 이름 더 안 쓸 지점을 지운다 BEGIN READ ONLY
읽기만 하는 트랜잭션

ACID 실제로는 이런 뜻

A 원자성 세 문장 중 하나가 실패하면 세 문장 다 없던 일이 된다 C 일관성 제약을 어기면 커밋이 거부된다
(지켜 주는 게 아니라 막아 준다) I 격리성 커밋 전 변경은 나만 보인다 — 자세한 건 개념03 D 지속성
커밋했다고 답했으면 정전이 나도 남는다 (WAL 덕분)

★★★ 이 단원에서 제일 중요한 세 가지 ① BEGIN 없이 보낸 문장은 각각 하나의 트랜잭션입니다 (자동
커밋) ② 트랜잭션 안에서 에러가 나면 그 뒤 모든 문장이 25P02 로 거부됩니다 ③ catch 에서 반드시
ROLLBACK 하세요. 안 하면 다음 요청이 대신 죽습니다

직접 해 볼 것

▫ 직접 해보기 1

섹션 2 의 첫 트랜잭션에서 ROLLBACK 을 지우고 돌려 보세요. 그 뒤 문장들이 어떻게 되나요? 몇 번 에러가 나나요?

▫ 직접 해보기 2

섹션 3 에서 ROLLBACK 을 COMMIT 으로 바꿔 보세요. 옆사람 눈에 언제부터 '고장' 이 보이나요?

▫ 직접 해보기 3

섹션 5 에서 COMMIT 대신 ROLLBACK 을 보내 보세요. 결과가 달라지나요? (힌트: 안 달라집니다. 왜일까요?)

▫ 직접 해보기 4

섹션 6 에서 ROLLBACK TO SAVEPOINT 를 지워 보세요. 그 뒤 INSERT 가 어떻게 되나요? 몇 건이 남나요?

▫ 직접 해보기 5

트랜잭션으로() 안에서 ROLLBACK 을 지우고, 실패하는 트랜잭션을 돌린 뒤 SELECT 1 을 해 보세요. 무슨 코드가 나오나요?

▫ 직접 해보기 6

BEGIN 을 두 번 연속 보내면 어떻게 되나요? (힌트: 경고가 나옵니다. Postgres 에는 중첩 트랜잭션이 없습니다)

▫ 직접 해보기 7

CREATE TABLE 을 트랜잭션 안에서 하고 ROLLBACK 해 보세요. 표가 사라지나요? (Postgres 는 사라집니다. MySQL 은 안 사라집니다)

자주 하는 실수

이런 실수를 합니다

catch 에서 ROLLBACK 을 안 함

가장 흔하고 가장 아픕니다. 그 연결이 오염된 채 풀로 돌아갑니다. 다음 요청이 25P02 로 죽고, 로그에는 아무 관계 없는 코드가 찍힙니다. ★ 트랜잭션은 반드시 감싼 함수로 쓰세요. 손으로 BEGIN/COMMIT 쓰지 마세요.

이런 실수를 합니다**“에러가 났으니 나머지는 알아서 취소되겠지”**

BEGIN 을 안 썼다면 취소되지 않습니다. 앞 문장들은 이미 커밋됐습니다. 섹션 1 이 정확히 그 상황입니다.

이런 실수를 합니다**트랜잭션 안에서 외부 호출을 함**

결제 API, 메일 발송, 파일 업로드를 BEGIN 과 COMMIT 사이에 넣으면 그동안 잠금을 줍니다. 상대 서버가 느리면 우리 DB 가 같이 멈춥니다. ★ 외부 호출은 트랜잭션 밖에서 하세요.

이런 실수를 합니다**COMMIT 을 보냈으니 성공했다고 믿음**

오염된 트랜잭션에서는 COMMIT 이 롤백으로 처리됩니다. 에러도 안 납니다. 섹션 5 의 3) 이 그 경우입니다. **COMMIT 이 성공을 보장하지 않습니다.**

이런 실수를 합니다**트랜잭션을 너무 크게 잡음**

“안전하니까” 하고 백만 건 처리를 통째로 하나로 묶으면 그동안 잠금을 쥐고, 청소(VACUUM)를 막고, 실패하면 처음부터 다시 합니다. ★ 나눌 수 있으면 나누세요. 개념04 에서 긴 트랜잭션의 대가를 쥅니다.

이런 실수를 합니다**트랜잭션이 동시성 문제를 다 풀어 준다고 생각함**

안 풀어 줍니다. 트랜잭션을 제대로 쓰기도 데이터가 사라집니다. 개념02 에서 실제로 사라뜨려 보겠습니다.

이런 실수를 합니다**SAVEPOINT 를 반복문마다 찍음**

만 건 반복에 만 개를 찍으면 눈에 띄게 느려집니다. 실패를 건별로 허용해야 할 때만 쓰세요.

이런 실수를 합니다**SERIAL 번호가 빈틈없이 이어질 거라고 믿음**

롤백해도 순번은 되돌아오지 않습니다. 섹션 7 에서 번호가 3 을 건너뛰고 4 였습니다. 영수증 번호처럼 구멍이 있으면 안 되는 번호에 SERIAL 을 쓰면 나중에 곤란해집니다.

★ 연결을 반드시 닫습니다. 안 닫으면 이 파일이 끝나지 않습니다.

```
await 연결.end();
await 새사람.end();
```

동시에 고치면 사라집니다

```
RUN node 개념02_동시에_고치면_사라집니다.js
```

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

01단원 개념01 에서 JSON 파일에 20건을 동시에 올렸더니 **1건만 남았습니다**. 그때 “데이터베이스는 트랜잭션과 잠금으로 풀니다. 07단원에서 합니다” 라고 적었습니다. 여기가 그 07단원입니다. 값을 차례입니다.

그런데 이 개념의 진짜 목적은 따로 있습니다.

★★★ 트랜잭션을 제대로 써도 데이터는 사라집니다.

실제로 사라지게 해 보는 것이 이 파일의 절반입니다. 트랜잭션은 만능이 아닙니다. 무엇을 해 주고 무엇을 안 해 주는지 정확히 알아야 합니다.

```

import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
};

let 주연결;

try {
  주연결 = new pg.Client(접속정보);
  await 주연결.connect();
} catch (에러) {
  console.log("Docker Postgres 에 못 붙었습니다. 먼저 이걸 실행하세요:");
  console.log(" docker compose up -d");
  console.log(` (원인: ${에러.message})`);
  process.exit(0);
}

await 주연결.query("CREATE SCHEMA IF NOT EXISTS 단원07");
await 주연결.query("SET search_path TO 단원07");

await 주연결.query("DROP TABLE IF EXISTS 생산실적, 재고 CASCADE");

await 주연결.query(`
  CREATE TABLE 생산실적 (
    번호 SERIAL PRIMARY KEY,
    작업자 TEXT NOT NULL,
    수량 INT NOT NULL
  )
`);

await 주연결.query(` CREATE TABLE 재고 (
  부품번호 INT PRIMARY KEY,
  이름 TEXT NOT NULL,
  수량 INT NOT NULL,
  버전 INT NOT NULL DEFAULT 0
)
`);

```

“동시에”를 흉내 내려면 **연결이 여러 개** 있어야 합니다.

★ 연결 하나로 Promise.all 을 해 봐야 소용없습니다. 하나의 연결은 문장을

한 줄로 세워서 보냅니다. 겹치질 않으니 동시성 문제가 안 납니다.
PgLite 로 이 실습을 못 하는 이유가 정확히 이것입니다.

연결을 20개 만듭니다. 진짜 작업자 20명이라고 생각하세요.

```
const 사람수 = 20;
const 사람들 = [];

for (let 번호 = 0; 번호 < 사람수; 번호 += 1) {
  const 사람 = new pg.Client(접속정보);
  await 사람.connect();
  await 사람.query("SET search_path TO 단원07");
  사람들.push(사람);
}

console.log(`연결 ${사람들.length}개 준비`);
```

출력 연결 20개 준비

```
const 한값 = async (문장, 값들 = []) => Object.values((await 주연결.query(문장,
값들)).rows[0])[0];
```

★ 01단원의 그 실험을 DB 로 다시 합니다

먼저 파일로 합니다. 01단원과 똑같은 코드입니다.

```
const 작업방 = fs.mkdtempSync(path.join(os.tmpdir(), "db자료-07-"));
const 실적파일 = path.join(작업방, "실적.json");

async function 파일에추가(값) {
  const 목록 = JSON.parse(fs.readFileSync(실적파일, "utf8"));
```

읽고 쓰는 사이의 틈입니다. 진짜 서버에서는 저절로 생깁니다.

```
await new Promise((끝) => setTimeout(끝, 1));

목록.push(값);
fs.writeFileSync(실적파일, JSON.stringify(목록));
```

```

}

const 파일결과 = [];

for (let 회차 = 1; 회차 <= 3; 회차 += 1) {
  fs.writeFileSync(실적파일, JSON.stringify([]));

  await Promise.all(
    Array.from({ length: 사람수 }, (_, 번호) => 파일에추가({ 작업자: `작업자${번호}`,
    수량: 10 })),
  );

  파일결과.push(JSON.parse(fs.readFileSync(실적파일, "utf8")).length);
}

console.log(`파일에 ${사람수}건 동시 저장 → 남은 건수 (3회):`, 파일결과.join(", "));

```

출력 파일에 20건 동시 저장 → 남은 건수 (3회): 1, 1, 1
값이 다를 수 있음

```
console.log("파일 - 20건 전부 남았나:", 파일결과.every((건수) => 건수 === 사람수));
```

출력 파일 - 20건 전부 남았나: false

```
fs.rmSync(작업방, { recursive: true, force: true });
```

이번엔 데이터베이스로 똑같이 합니다. 연결 20개가 동시에 INSERT 합니다.

```

const DB결과 = [];

for (let 회차 = 1; 회차 <= 3; 회차 += 1) {
  await 주연결.query("TRUNCATE 생산실적");

  await Promise.all(
    사람들.map((사람, 번호) =>
      사람.query("INSERT INTO 생산실적 (작업자, 수량) VALUES ($1, $2)", [`작업자
      ${번호}`, 10]),
    ),
  );

  DB결과.push(Number(await 한값("SELECT count(*) FROM 생산실적")));
}

console.log(`DB 에 ${사람수}건 동시 저장 → 남은 건수 (3회):`, DB결과.join(", "));

```

출력 DB 에 20건 동시 저장 → 남은 건수 (3회): 20, 20, 20

```
console.log("DB - 20건 전부 남았나:", DB결과.every((건수) => 건수 === 사람수));
```

출력 DB - 20건 전부 남았나: true

★★ 같은 실험, 같은 20명, 같은 틸. 파일은 1건, DB 는 20건입니다. 파일은 "전체를 읽고 → 전체를 다시 쓴다" 라서 나중 사람이 앞사람 것을 통째로 덮습니다. DB 의 INSERT 는 "한 줄을 덧붙인다" 라서 덮을 것이 없습니다. 게다가 각 INSERT 는 자동 커밋으로 각자 하나의 트랜잭션입니다(개념01 섹션3).

★ 여기까지가 "데이터베이스 쓰면 해결" 이야기입니다. 여기서 끝나면 좋겠지만 아닙니다.

★★★ 트랜잭션을 써도 사라집니다

섹션 2

이번엔 INSERT 가 아니라 **UPDATE** 입니다. 부품 재고를 20명이 한 개씩 채워 넣습니다. 0 에서 시작해 20 이 돼야 합니다. 코드는 아주 흔한 모양입니다. ① 지금 수량을 읽고 ② JS 에서 +1 하고 ③ 그 값을 씁니다. 트랜잭션도 제대로 씁니다. BEGIN 하고 COMMIT 합니다.

```
async function 읽고_더하고_쓰기(사람) {
  await 사람.query("BEGIN");

  const 지금 = (await 사람.query("SELECT 수량 FROM 재고 WHERE 부품번호 = 1")).rows[0].수량;
```

읽고 쓰는 사이의 틸입니다. 파일 실험과 똑같습니다.

```
await new Promise((끝) => setTimeout(끝, 5));

  await 사람.query("UPDATE 재고 SET 수량 = $1 WHERE 부품번호 = 1", [지금 + 1]);
  await 사람.query("COMMIT");
}

async function 재고초기화() {
  await 주연결.query("TRUNCATE 재고");
  await 주연결.query("INSERT INTO 재고 VALUES (1, '베어링 6203', 0, 0)");
}

const 손실결과 = [];

for (let 회차 = 1; 회차 <= 3; 회차 += 1) {
  await 재고초기화();
  await Promise.all(사람들.map(읽고_더하고_쓰기));
  손실결과.push(Number(await 한값("SELECT 수량 FROM 재고 WHERE 부품번호 = 1")));
}

console.log(`트랜잭션 안에서 ${사람수}명이 +1 → 결과 (3회):`, 손실결과.join(", "));
```


출력 트랜잭션 안에서 20명이 +1 → 결과 (3회): 1, 1, 1
값이 다를 수 있음

```
console.log("20 이 됐나:", 손실결과.every((값) => 값 === 사람수));
```

출력 20 이 됐나: false

```
console.log("몇 건이나 사라졌나 (첫 회):", 사람수 - 손실결과[0], "건");
```

출력 몇 건이나 사라졌나 (첫 회): 19 건
값이 다를 수 있음

★★★ BEGIN 도 했고 COMMIT 도 했는데 사라졌습니다. 에러는 하나도 안 났습니다. 20명 전부 “저장했습니다” 소리를 들었습니다. 01단원의 파일 실험과 **똑같은 일** 이 벌어졌습니다.

이걸 **갱신 손실(lost update)** 이라고 합니다.

왜 사라지나

섹션 3

20명은 많으니 두 명으로 줄여서 그려 봅니다. 수량은 0 에서 시작합니다.

시각 김반장 이반장 DB 의 수량

1	BEGIN	0
2	SELECT 수량 → 0	0
3	BEGIN	0
4	SELECT 수량 → 0	0
5	UPDATE 수량 = 0+1 = 1	0 (아직 안 보임)
6	COMMIT	1
7	UPDATE 수량 = 0+1 = 1	1
8	COMMIT	1 ← 2 가 아닙니다

두 사람 다 **0** 을 읽었습니다. 둘 다 1 을 썼습니다. 결과는 1 입니다. 이반장의 작업은 흔적도 없이 사라졌습니다.

★ 트랜잭션이 왜 못 막았나 트랜잭션이 보장하는 것은 “내 문장들이 전부 되거나 전부 안 되거나” 뿐입니다. 김반장 것도 전부 됐고 이반장 것도 전부 됐습니다. **둘 다 성공했습니다.** 문제는 **읽은 값이 쓸 때까지 그대로일 거라고 믿은 것** 입니다. 그건 트랜잭션이 약속한 적 없는 일입니다.

★★ 이 버그의 무서운 점 혼자 눌러 보면 절대 안 나고, 테스트에서도 거의 안 나고, 로그에 아무것도 안 남고, 손님이 많아질수록 자주 납니다. 재고, 잔액, 포인트, 조회수, 좋아요 — 전부 이 버그의 단골입니다.

가장 간단하고 가장 빠릅니다. 그리고 대부분의 경우 이걸로 끝납니다.

나쁨: SELECT 수량 → JS 에서 +1 → UPDATE 수량 = 21
 좋음: UPDATE 재고 SET 수량 = 수량 + 1

오른쪽은 값을 자바스크립트로 가져오지 않습니다. DB 안에서 읽고 더하고 씁니다.

```
async function 원자적으로(사람) {
  await 사람.query("UPDATE 재고 SET 수량 = 수량 + 1 WHERE 부품번호 = 1");
}

const 원자결과 = [];

for (let 회차 = 1; 회차 <= 3; 회차 += 1) {
  await 재고초기화();
  await Promise.all(사람들.map(원자적으로));
  원자결과.push(Number(await 한값("SELECT 수량 FROM 재고 WHERE 부품번호 = 1")));
}

console.log(`원자적 UPDATE → 결과 (3회):`, 원자결과.join(", "));
```

출력 원자적 UPDATE → 결과 (3회): 20, 20, 20

```
console.log("20 이 됐나:", 원자결과.every((값) => 값 === 사람수));
```

출력 20 이 됐나: true

★ 왜 이걸 되나 — 잠금이 자동으로 걸립니다 UPDATE 는 그 줄에 잠금을 걸고 커밋할 때까지 안 놓습니다. 남의 UPDATE 는 그 앞에서 기다렸다가, 풀리면 **새 값을 다시 읽어서** 더합니다. 이 “다시 읽는다” 를 직접 확인해 봅시다.

```
await 재고초기화();
await 주연결.query("UPDATE 재고 SET 수량 = 100 WHERE 부품번호 = 1");

const 가 = 사람들[0];
const 나 = 사람들[1];

await 가.query("BEGIN");
await 가.query("UPDATE 재고 SET 수량 = 수량 - 5 WHERE 부품번호 = 1"); // 100 → 95,
잠금을 씁니다
```

나는 여기서 막힙니다. await 하지 않고 띄워만 둡니다.

```
const 나의작업 = 나.query("UPDATE 재고 SET 수량 = 수량 - 3 WHERE 부품번호 = 1");

await new Promise((끝) => setTimeout(끝, 200)); // 나가 확실히 막히도록 잠깐 둡니다
await 가.query("COMMIT"); // 잠금을 놓습니다
await 나의작업; // 이제 나가 진행됩니다 console.log("100 에서 5 빼고 3 뺀 결과:",
await 한값("SELECT 수량 FROM 재고 WHERE 부품번호 = 1"));
```

출력 100 에서 5 빼고 3 뺀 결과: 92

★★ 92 입니다. 97 이 아닙니다. 나는 처음에 100 을 보고 있었습니다. 그런데 기다렸다 풀리고 나서 **95** 를 다시 읽어서 3 을 뺐습니다. 그래서 92 입니다. 이 “잠금이 풀리면 다시 읽는” 동작을 Postgres 는 자동으로 해 줍니다.

★ 원자적 UPDATE 의 함정 — 조건이 붙으면 다릅니다

섹션 5

재고가 모자라면 빼면 안 됩니다. 조건을 붙입니다.

UPDATE 재고 SET 수량 = 수량 - 3 WHERE 부품번호 = 1 AND 수량 >= 3

이건 안전할까요? 확인해 봅시다.

```
await 재고초기화();
await 주연결.query("UPDATE 재고 SET 수량 = 100 WHERE 부품번호 = 1");

await 가.query("BEGIN");
await 가.query("UPDATE 재고 SET 수량 = 0 WHERE 부품번호 = 1"); // 가가 재고를 다
    씁니다 const 나의시도 = 나.query("UPDATE 재고 SET 수량 = 수량 - 3 WHERE 부품번호 = 1
    AND 수량 >= 3");

await new Promise((끝) => setTimeout(끝, 200));
await 가.query("COMMIT");

const 나의결과 = await 나의시도;

console.log("나가 실제로 고친 줄 수:", 나의결과.rowCount);
```

출력 나가 실제로 고친 줄 수: 0

```
console.log("최종 수량:", await 한값("SELECT 수량 FROM 재고 WHERE 부품번호 = 1"));
```

출력 최종 수량: 0

★★★ 수량이 -3 이 되지 않았습니다. 조건을 **다시 확인** 했기 때문입니다.

나는 100 을 보고 시작했지만, 잠금이 풀린 뒤 0 을 다시 읽었고 수량 >= 3 이 이제 거짓이라 **그 줄을 건너뛰었습니다**.

★ 그래서 rowCount 를 반드시 보세요. 0 이면 "재고가 모자라 못 뺐다" 는 뜻입니다.

안 보고 넘어가면 손님에게 "출고 완료" 라고 답하게 됩니다.

```
const 결과 = await 연결.query("UPDATE ... AND 수량 >= $1", [주문수]);
if (결과.rowCount === 0) throw new Error("재고 부족");
```

★ 이 한 줄짜리 패턴이 실무에서 재고 차감의 정석입니다. SELECT 로 확인하고 UPDATE 하지 마세요. 그 사이에 남이 다 가져갑니다.

해법 ② SELECT ... FOR UPDATE — 비관적 잠금

섹션 6

원자적 UPDATE 로 안 되는 경우가 있습니다. · 읽은 값으로 자바스크립트에서 복잡한 계산을 해야 할 때 · 여러 표를 읽어서 판단한 뒤 써야 할 때

그럴 때는 **읽을 때부터 줄을 잡아둡니다**. SELECT 뒤에 FOR UPDATE 를 붙입니다.

SELECT 수량 FROM 재고 WHERE 부품번호 = 1 FOR UPDATE → 이 줄을 커밋할 때까지 아무도 못 고칩니다. 남은 여기서 기다립니다.

“남이 고칠 것이라고 미리 걱정하고 미리 막는다” 고 해서 **비관적 잠금** 이라고 합니다.

```
async function 비관적으로(사람) {
  await 사람.query("BEGIN");

  const 지금 = (await 사람.query("SELECT 수량 FROM 재고 WHERE 부품번호 = 1 FOR
  UPDATE")).rows[0].수량;

  await new Promise((끝) => setTimeout(끝, 1)); // 계산하는 시간이라고 칩시다
  await 사람.query("UPDATE 재고 SET 수량 = $1 WHERE 부품번호 = 1", [지금 + 1]);
  await 사람.query("COMMIT");
}

const 비관결과 = [];

for (let 회차 = 1; 회차 <= 3; 회차 += 1) {
  await 재고초기화();
  await Promise.all(사람들.map(비관적으로));
  비관결과.push(Number(await 한값("SELECT 수량 FROM 재고 WHERE 부품번호 = 1")));
}

console.log("FOR UPDATE → 결과 (3회):", 비관결과.join(", "));
```

출력 FOR UPDATE → 결과 (3회): 20, 20, 20

```
console.log("20 이 됐나:", 비관결과.every((값) => 값 === 사람수));
```

출력 20 이 됐나: true

★ 섹션 2 와 코드가 거의 같습니다. FOR UPDATE 여섯 글자만 붙였습니다. 그 여섯 글자가 20명을 한 줄로 세웁니다.

★★ 대가가 있습니다. 20명이 줄을 섭니다. 한 명이 계산하는 동안 나머지 19명이 기다립니다. 그리고 잠금을 쥔 채로 오래 있으면 전부 멈춥니다. 개념04 에서 잡니다.

해법 ③ 버전 칸 — 낙관적 잠금

섹션 7

반대로 갑니다. “부딪히는 일은 드물 것이다” 라고 낙관하고, 잠그지 않고 쓸 때 확인만 합니다.

① 수량과 버전을 같이 읽는다 (수량 5, 버전 3) ② 계산한다 ③ UPDATE ... WHERE 버전 = 3 ← 내가 읽은 그 버전일 때만 쓴다 ④ rowCount 가 0 이면 남이 먼저 고친 것 → 처음부터 다시

```
async function 낙관적으로(사람) {
  for (let 시도 = 1; 시도 <= 100; 시도 += 1) {
    const 읽은것 = (await 사람.query("SELECT 수량, 버전 FROM 재고 WHERE 부품번호 = 1")).rows[0];

    await new Promise((끝) => setTimeout(끝, 1));

    const 결과 = await 사람.query(
      "UPDATE 재고 SET 수량 = $1, 버전 = 버전 + 1 WHERE 부품번호 = 1 AND 버전 = $2",
      [읽은것.수량 + 1, 읽은것.버전],
    );

    if (결과.rowCount === 1) return 시도; // 성공
```

rowCount 가 0 이면 남이 먼저 고쳤습니다. 다시 읽고 다시 합니다.

```
}

throw new Error("100번 시도했는데도 실패했습니다");
}

await 재고초기화();
const 시도들 = await Promise.all(사람들.map(낙관적으로));

console.log("버전 칸 → 결과:", await 한값("SELECT 수량 FROM 재고 WHERE 부품번호 = 1"));
```

출력 버전 칸 → 결과: 20

```
console.log("가장 운 나쁜 사람의 시도 횟수:", Math.max(...시도들));
```

출력
값이 다를 수 있음

가장 운 나쁜 사람의 시도 횟수: 19

```
console.log("한 번에 성공한 사람 수:", 시도들.filter((횟수) => 횟수 === 1).length);
```

출력
값이 다를 수 있음

한 번에 성공한 사람 수: 1

★ 20 이 됐습니다. 그런데 누군가는 16번을 다시 했습니다. 같은 줄에 20명이 몰렸으니 당연합니다. 낙관이 틀린 상황입니다.

★★ 버전 칸의 진짜 값어치는 다른 데 있습니다. 화면에서 폼을 열어 두고 5분 뒤에 저장하는 경우를 생각해 보세요. FOR UPDATE 로 5분을 잠글 수는 없습니다. 버전 칸이면 잠그지 않고도 “그사이 남이 고쳤습니다”를 알려 줍니다. 웹 화면의 그 메시지가 이것입니다.

★ 셋을 나란히 재 봅니다 — 언제 무엇을 쓰나

섹션 8

총돌이 찾을 때와 드물 때를 나눠서 잡니다.

총돌 찾을 : 20명이 **한 줄** 에 몰림 총돌 드물 : 20명이 **각자 다른 줄** 을 고침

```

await 주연결.query("DROP TABLE IF EXISTS 비교시험 CASCADE");
await 주연결.query("CREATE TABLE 비교시험 (번호 INT PRIMARY KEY, 수량 INT NOT NULL,
버전 INT NOT NULL DEFAULT 0)");

async function 비교준비(줄수) {
  await 주연결.query("TRUNCATE 비교시험");
  await 주연결.query("INSERT INTO 비교시험 SELECT g, 0, 0 FROM generate_series(1, $1)
g", [줄수]);
}

async function 비교재기(방식, 줄수) {
  await 비교준비(줄수);

  let 재시도 = 0;
  const 시작 = performance.now();

  await Promise.all(
    사람들.map(async (사람, 자리) => {
      const 대상 = (자리 % 줄수) + 1;

      if (방식 === "비관적") {
        await 사람.query("BEGIN");
        const 지금 = (await 사람.query("SELECT 수량 FROM 비교시험 WHERE 번호 = $1 FOR
UPDATE", [대상])).rows[0].수량;
        await 사람.query("UPDATE 비교시험 SET 수량 = $1 WHERE 번호 = $2", [지금 + 1,
대상]);
        await 사람.query("COMMIT");
        return;
      }

      for (let 시도 = 1; 시도 <= 100; 시도 += 1) {
        const 읽은것 = (await 사람.query("SELECT 수량, 버전 FROM 비교시험 WHERE 번호
= $1", [대상])).rows[0];
        const 결과 = await 사람.query(
          "UPDATE 비교시험 SET 수량 = $1, 버전 = 버전 + 1 WHERE 번호 = $2 AND 버전 =
$3",
          [읽은것.수량 + 1, 대상, 읽은것.버전],
        );
        if (결과.rowCount === 1) return;
        재시도 += 1;
      }
    }),
  ),

```

```
);

const 걸린ms = performance.now() - 시작;
const 합계 = Number(await 한값("SELECT sum(수량) FROM 비교시험"));

return { 걸린ms, 합계, 재시도 };
}
```

중앙값을 씁니다. 한 번 재고 판단하면 안 됩니다.

```
async function 다섯번재기(방식, 줄수) {
  const 잔것 = [];
  for (let 회차 = 0; 회차 < 5; 회차 += 1) 잔것.push(await 비교재기(방식, 줄수));
  잔것.sort((가, 나) => 가.걸린ms - 나.걸린ms);
  return 잔것[2];
}

const 잦은비관 = await 다섯번재기("비관적", 1);
const 잦은낙관 = await 다섯번재기("낙관적", 1);
const 드문비관 = await 다섯번재기("비관적", 20);
const 드문낙관 = await 다섯번재기("낙관적", 20);

console.log("네 경우 모두 합계가 20 인가:",
  [잦은비관, 잦은낙관, 드문비관, 드문낙관].every((결과) => 결과.합계 === 20));
```

출력 네 경우 모두 합계가 20 인가: true

```
console.log(`충돌 잦음 · 비관적: ${잦은비관.걸린ms.toFixed(0)} ms · 헛시도
${잦은비관.재시도}회`);
```

출력 충돌 잦음 · 비관적: 23 ms · 헛시도 0회
값이 다를 수 있음

```
console.log(`충돌 잦음 · 낙관적: ${잦은낙관.걸린ms.toFixed(0)} ms · 헛시도
${잦은낙관.재시도}회`);
```

출력 충돌 잦음 · 낙관적: 33 ms · 헛시도 130회
값이 다를 수 있음

```
console.log(`충돌 드묾 · 비관적: ${드문비관.걸린ms.toFixed(0)} ms · 헛시도
${드문비관.재시도}회`);
```

출력 충돌 드묾 · 비관적: 5 ms · 헛시도 0회
값이 다를 수 있음

```
console.log(`충돌 드묾 · 낙관적: ${드문낙관.걸린ms.toFixed(0)} ms · 헛시도
${드문낙관.재시도}회`);
```


출력 충돌 드물 · 낙관적: 4 ms · 헛시도 0회
값이 다를 수 있음

시간은 컴퓨터마다 다릅니다. 값 대신 **성질**을 봐야 합니다.

```
console.log("충돌이 잦으면 낙관적이 헛일을 하나:", 잦은낙관.재시도 > 0);
```

출력 충돌이 잦으면 낙관적이 헛일을 하나: true

```
console.log("충돌이 드물면 낙관적이 헛일을 안 하나:", 드문낙관.재시도 === 0);
```

출력 충돌이 드물면 낙관적이 헛일을 안 하나: true

★★ 이게 “충돌이 잦으면 비관적, 드물면 낙관적”의 실제 근거입니다. 낙관적 잠금은 부딪히면 **한 일을 버리고 처음부터 다시** 합니다. 위에서 헛시도가 130회였습니다. 20번 성공하는 데 150번을 시도한 셈입니다. 반대로 안 부딪히면 잠금 비용도 기다림도 0입니다. 대부분의 화면 수정은 이쪽입니다. 같은 줄을 동시에 고칠 일이 거의 없습니다.

정리 — 갱신 손실을 막는 세 가지

방법 코드 언제

① 원자적 UPDATE UPDATE ... SET 수량 = 수량 + 1 할 수 있으면 무조건 이것 ② 비관적 (FOR UPDATE) SELECT ... FOR UPDATE 로 잡고 고침 충돌이 잦을 때 ③ 낙관적 (버전 칸) UPDATE ... WHERE 버전 = 읽은버전 충돌이 드물 때 / 화면이 오래 열려 있을 때

방법 장점 단점

① 원자적 제일 빠름, 코드 한 줄 복잡한 계산을 못 함 ② 비관적 확실함, 재시도 없음 줄을 세움. 오래 쥐면 전부 멈춤. 데드락 위험 ③ 낙관적 잡그지 않음, 잘 늘어남 부딪히면 버리고 다시 함. 재시도 코드가 필요

★★★ 반드시 기억할 것 ① 트랜잭션은 갱신 손실을 막아 주지 않습니다. BEGIN/COMMIT 을 해도 사라집니다 ② “SELECT 로 확인하고 UPDATE” 는 거의 항상 버그입니다. 그 사이에 남이 바꿨습니다 ③ 조건부 UPDATE 를 했으면 **rowCount** 를 꼭 보세요. 0 이면 아무것도 안 된 것입니다

직접 해 볼 것

▸ 직접 해보기 1

섹션 2 의 setTimeout(5) 을 지우면 결과가 어떻게 되나요? 20 이 나오나요? 그렇다면 이 버그가 사라진 걸까요? (힌트: 틸이 줄었을 뿐입니다. 진짜 서버에는 늘 틸이 있습니다)

▫ 직접 해보기 2

사람수를 20 에서 100 으로 늘려 보세요. 섹션 2 의 결과가 어떻게 되나요? 섹션 7 의 최대 시도 횟수는요?

▫ 직접 해보기 3

섹션 5 에서 AND 수량 >= 3 을 지우고 돌려 보세요. 최종 수량이 얼마가 되나요? 재고가 음수가 되는 걸 확인하세요.

▫ 직접 해보기 4

섹션 6 의 FOR UPDATE 를 지우면 결과가 어떻게 되나요? 섹션 2 와 같아지나요?

▫ 직접 해보기 5

섹션 7 에서 버전 조건 AND 버전 = \$2 를 지워 보세요. rowCount 가 항상 1 이 됩니다. 결과는 어떻게 되나요?

▫ 직접 해보기 6

섹션 8 의 줄수를 5 로 바꿔 보세요 (20명이 5줄에 나눠 몰림). 비관적과 낙관적 중 어느 쪽이 유리해지나요?

▫ 직접 해보기 7

낙관적으로() 의 재시도 상한 100 을 3 으로 줄이면 무슨 일이 나나요?

자주 하는 실수

이런 실수를 합니다

“트랜잭션을 썼으니 안전하다”

이 파일의 절반이 이 오해를 깨려고 있습니다. 트랜잭션은 내 문장들이 통째로 되거나 안 되거나 를 보장할 뿐입니다. “내가 읽은 값이 그대로 있을 것” 은 보장하지 않습니다.

이런 실수를 합니다

SELECT 로 확인하고 UPDATE 함

```
if (재고 >= 주문수) { UPDATE ... } ← 그 사이에 남아 다 가져갑니다.
```

조건을 UPDATE 의 WHERE 로 옮기세요. 한 문장이 되면 안전합니다.

이런 실수를 합니다**rowCount 를 안 봄**

조건부 UPDATE 가 0줄을 고쳐도 에러가 안 납니다. 성공처럼 보입니다. 재고가 없는데 “출고 완료” 를 답하는 사고가 여기서 납니다.

이런 실수를 합니다**낙관적 잠금에 재시도를 안 붙임**

버전 칸만 만들어 두고 rowCount 가 0 일 때 그냥 에러를 던지면 손님은 “잠시 후 다시 시도해 주세요” 만 보게 됩니다. 충돌이 드문 곳에는 재시도를 붙여 두면 손님이 모르고 지나갑니다.

이런 실수를 합니다**재시도 상한이 없음**

while (true) 로 돌리면 충돌이 심할 때 영원히 돕니다. 상한을 두고, 넘으면 실패로 처리하세요. 그리고 기다리는 시간을 조금씩 늘리세요.

이런 실수를 합니다**FOR UPDATE 를 트랜잭션 밖에서 씀**

BEGIN 이 없으면 그 SELECT 문장 하나가 끝나는 순간 잠금이 풀립니다. 잠근 의미가 없습니다. FOR UPDATE 는 반드시 **BEGIN 안에서** 쓰세요.

이런 실수를 합니다**연결 하나로 동시성을 시험함**

같은 연결로 Promise.all 을 돌리면 문장이 한 줄로 서서 나가 버그가 재현되지 않습니다.

★ 연결을 전부 닫습니다. 하나라도 남으면 이 파일이 끝나지 않습니다.

```
for (const 사람 of 사람들) await 사람.end();
await 주연결.end();
```

격리수준 네 가지

RUN node 개념03_격리수준_네_가지.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

개념01 에서 ACID 의 I(격리성)를 봤습니다. “커밋 전 변경은 남에게 안 보인다” 였습니다.

그런데 격리성은 켜고 끄는 스위치가 아닙니다. **손잡이** 입니다. 세게 돌리면 안전한 대신 느리고, 약하게 돌리면 빠른 대신 이상한 일이 생깁니다. 그 눈금이 네 개 있습니다. 그게 격리수준입니다.

이 파일에서는 눈금을 낮춰 놓고 **이상한 일을 일부러 일으켜 봅니다**. 그리고 눈금을 올려서 그 일이 사라지는 것을 확인합니다.

★★★ 미리 말해 둘 것이 하나 있습니다. 교과서에 실린 표와 PostgreSQL 의 실제 동작이 **다릅니다**. 이 파일에서 직접 돌려서 확인합니다. 시험에 나오는 표만 외우면 틀립니다.

```
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
};

let 가;

try {
  가 = new pg.Client(접속정보);
  await 가.connect();
} catch (에러) {
  console.log("Docker Postgres 에 못 붙었습니다. 먼저 이걸 실행하세요:");
  console.log(" docker compose up -d");
  console.log(` (원인: ${에러.message})`);
  process.exit(0);
}

await 가.query("CREATE SCHEMA IF NOT EXISTS 단원07");
await 가.query("SET search_path TO 단원07");
```

두 번째 사람. 이 사람이 옆에서 끼어듭니다.

```

const 나 = new pg.Client(접속정보);
await 나.connect();
await 나.query("SET search_path TO 단원07");

await 가.query("DROP TABLE IF EXISTS 계측값, 당직 CASCADE");

await 가.query(`
  CREATE TABLE 계측값 (
    번호 INT PRIMARY KEY,
    라인 TEXT NOT NULL,
    온도 INT NOT NULL
  )
`);

await 가.query("INSERT INTO 계측값 VALUES (1, 'A', 100), (2, 'A', 200)");

const 온도읽기 = async (연결) => (await 연결.query("SELECT 온도 FROM 계측값 WHERE
번호 = 1")).rows[0].온도;
const A라인건수 = async (연결) => Number((await 연결.query("SELECT count(*) FROM
계측값 WHERE 라인 = 'A'")).rows[0].count);

async function 처음상태로() {
  await 나.query("DELETE FROM 계측값 WHERE 번호 >= 3");
  await 나.query("UPDATE 계측값 SET 온도 = 100 WHERE 번호 = 1");
}

console.log("기본 격리수준:", (await 가.query("SHOW
default_transaction_isolation")).rows[0].default_transaction_isolation);

```

출력 기본 격리수준: read committed

이상 현상 세 가지

섹션 1

격리를 약하게 하면 생기는 이상한 일에 이름이 붙어 있습니다. 세 가지입니다.

① 더티 읽기(dirty read)

남이 ****아직 커밋도 안 한**** 값을 읽습니다.
그 사람이 롤백하면, 나는 세상에 없던 값을 보고 판단한 것이 됩니다.

② 반복 불가 읽기(non-repeatable read)

같은 SELECT 를 한 트랜잭션 안에서 두 번 했는데 ****값이 달라집니다.****
사이에 남이 고치고 커밋했기 때문입니다.

③ 팬텀 읽기(phantom read)

같은 조건으로 두 번 썼는데 **줄 수가 달라집니다.**
사이에 남이 조건에 맞는 줄을 새로 넣었기 때문입니다.

②와 ③은 비슷해 보이지만 다릅니다. ② 는 이미 있던 줄의 값이 바뀌는 것 ③ 은 줄 자체가 새로 생기거나 사라지는 것 막는 방법이 달라서 구분합니다.

① 더티 읽기 — 일으키려 해도 안 됩니다

섹션 2

더티 읽기를 하려면 READ UNCOMMITTED 로 내려가야 합니다. 해 봅시다.

```
await 나.query("BEGIN");
await 나.query("UPDATE 계측값 SET 온도 = 777 WHERE 번호 = 1"); // 커밋 안
합니다 await 가.query("BEGIN ISOLATION LEVEL READ UNCOMMITTED");
console.log("READ UNCOMMITTED 로 남의 미커밋 값을 보면:", await 온도읽기(가));
```

출력 READ UNCOMMITTED 로 남의 미커밋 값을 보면: 100

```
console.log("가 요청한 격리수준:", (await 가.query("SHOW
transaction_isolation")).rows[0].transaction_isolation);
```

출력 가 요청한 격리수준: read uncommitted

```
await 가.query("COMMIT");
await 나.query("ROLLBACK");
```

★★ 777 이 아니라 100 이 나왔습니다. 더티 읽기가 안 일어났습니다.

PostgreSQL 에는 READ UNCOMMITTED 가 사실상 없습니다. 문법은 받아 줍니다. SHOW 로 물어보면 "read uncommitted" 라고 대답도 합니다. 그런데 동작은 **READ COMMITTED** 와 똑같습니다.

★ 왜 못 만들었냐가 아니라 안 만든 것입니다.

Postgres 는 잠금이 아니라 MVCC(섹션 7)로 격리를 구현했습니다.
MVCC 에서는 "커밋 안 된 버전을 읽는다" 는 동작 자체가 더 복잡하고 느립니다.
막는 것이 더 쉬웠기 때문에 그냥 막았습니다.

★★ 그래서 Postgres 를 쓰는 한 더티 읽기는 걱정할 필요가 없습니다.

실전에서 쓰는 눈금은 사실상 **세 개** 입니다.

② 반복 불가 읽기 — 기본값에서 실제로 일으킵니다

섹션 3

```
await 처음상태로();

await 가.query("BEGIN ISOLATION LEVEL READ COMMITTED");
const 첫번째읽기 = await 온도읽기(가);
```

여기서 나가 끼어듭니다. 고치고 커밋까지 합니다.

```
await 나.query("UPDATE 계측값 SET 온도 = 999 WHERE 번호 = 1");

const 두번째읽기 = await 온도읽기(가);
await 가.query("COMMIT");

console.log(`READ COMMITTED - 같은 SELECT 두 번: ${첫번째읽기} → ${두번째읽기}`);
```

```
출력      READ COMMITTED - 같은 SELECT 두 번: 100 → 999
```

```
console.log("값이 달라졌나:", 첫번째읽기 !== 두번째읽기);
```

```
출력      값이 달라졌나: true
```

★★ 한 트랜잭션 안에서 같은 문장을 두 번 실행했는데 답이 다릅니다.

무엇이 문제인가:

보고서를 만든다고 합시다. 위쪽에서 합계를 구하고, 아래쪽에서 항목을 나열합니다.
그 사이에 남은 값들을 고치면 **합계와 항목이 안 맞습니다.**
"이 보고서 숫자가 왜 안 맞죠?" 가 여기서 나옵니다.

★ READ COMMITTED 는 문장 하나마다 새로 사진을 찍습니다.

그래서 문장 하나 안에서는 일관되지만, 문장 사이에서는 아닙니다.

③ 팬텀 읽기 — 역시 기본값에서 일으킵니다

섹션 4

```
await 처음상태로();

await 가.query("BEGIN ISOLATION LEVEL READ COMMITTED");
const 첫번째건수 = await A라인건수(가);
```

나가 A라인에 줄을 하나 새로 넣습니다.

```

await 나.query("INSERT INTO 계측값 VALUES (3, 'A', 300)");

const 두번째건수 = await A라인건수(가);
await 가.query("COMMIT");

console.log(`READ COMMITTED - 같은 COUNT 두 번: ${첫번째건수}건 → ${두번째건수}건`);

```

출력 READ COMMITTED - 같은 COUNT 두 번: 2건 → 3건

```

console.log("없던 줄이 나타났나:", 두번째건수 > 첫번째건수);

```

출력 없던 줄이 나타났나: true

★ 유령처럼 없던 줄이 나타났다고 해서 팬텀(유령) 읽기입니다.

★★★ REPEATABLE READ 로 올려 봅니다

섹션 5

이제 눈금을 한 칸 올립니다. 같은 실험을 그대로 다시 합니다.

```

await 처음상태로();

await 가.query("BEGIN ISOLATION LEVEL REPEATABLE READ");
const RR첫값 = await 온도읽기(가);
await 나.query("UPDATE 계측값 SET 온도 = 999 WHERE 번호 = 1");
const RR둘값 = await 온도읽기(가);
await 가.query("COMMIT");

console.log(`REPEATABLE READ - 같은 SELECT 두 번: ${RR첫값} → ${RR둘값}`);

```

출력 REPEATABLE READ - 같은 SELECT 두 번: 100 → 100

```

console.log("반복 불가 읽기가 막혔나:", RR첫값 === RR둘값);

```

출력 반복 불가 읽기가 막혔나: true

여기까지는 교과서대로입니다. 팬텀은 어떨까요? 교과서는 **REPEATABLE READ** 에서 팬텀이 허용된다고 합니다.


```
await 처음상태로();

await 가.query("BEGIN ISOLATION LEVEL REPEATABLE READ");
const RR첫건수 = await A라인건수(가);
await 나.query("INSERT INTO 계측값 VALUES (3, 'A', 300)");
const RR둘건수 = await A라인건수(가);
await 가.query("COMMIT");

console.log(`REPEATABLE READ - 같은 COUNT 두 번: ${RR첫건수}건 → ${RR둘건수}건`);
```

```
출력      REPEATABLE READ - 같은 COUNT 두 번: 2건 → 2건
```

```
console.log("팬텀이 막혔나:", RR첫건수 === RR둘건수);
```

```
출력      팬텀이 막혔나: true
```

★★★ 여기가 이 파일에서 제일 중요한 곳입니다.

표준(SQL 표준)은 REPEATABLE READ 에서 팬텀을 **허용** 한다고 정합니다. 그런데 PostgreSQL 은 **막았습니다**. 방금 직접 확인했습니다.

“표에는 팬텀 허용이라던데요?” 라고 물으면 답은 이렇습니다.

★ 표준은 **최소 요구조건** 입니다. “여기까지는 반드시 막아라” 입니다.

구현이 그보다 ****더** 세게 막는 것은 허용** 됩니다.
Postgres 는 스냅샷 격리로 만들다 보니 팬텀까지 자동으로 막히게 됐습니다.
일부러 약하게 만들 이유가 없으니 그대로 뒀습니다.

★★ 그러니 시험 답은 “허용”, 실제 Postgres 답은 “안 생김” 입니다.

둘 다 알고 있어야 합니다. 그리고 ****다른 DB 로 옮길 때 조심해야 합니다.****
같은 코드가 MySQL 이나 Oracle 에서는 다르게 동작할 수 있습니다.

MySQL 은 여기가 다릅니다

MySQL(InnoDB)은 기본값이 REPEATABLE READ 이고, 팬텀을 갭 락(gap lock)이라는
다른 방식으로 막습니다. 잠그는 범위가 넓어서 대기가 더 잘 생깁니다.
* 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

표 두 개를 나란히

섹션 6

— 교과서 표 — SQL 표준

더티읽기 반복불가읽기 팬텀읽기

READ UNCOMMITTED 허용 허용 허용 READ COMMITTED 막음 허용 허용 REPEATABLE READ 막음
막음 ★ 허용 SERIALIZABLE 막음 막음 막음

— PostgreSQL 실제 — 위에서 직접 돌린 결과

더티읽기 반복불가읽기 팬텀읽기

READ UNCOMMITTED ★ 안 생김 허용 허용 ← READ COMMITTED 로 동작 READ COMMITTED 안
생김 허용 허용 REPEATABLE READ 안 생김 막음 ★★ 막음 SERIALIZABLE 안 생김 막음 막음 + 쓰기
왜곡까지 막음

★ 눈금이 실질적으로 세 개입니다. READ COMMITTED / REPEATABLE READ / SERIALIZABLE.

MVCC — 왜 읽기가 쓰기를 안 막나

섹션 7

Postgres 가 이렇게 동작하는 이유는 **MVCC** 때문입니다. 다중 버전 동시성 제어(Multi-Version
Concurrency Control)의 줄임말입니다.

핵심은 한 줄입니다.

★ 줄을 고칠 때 **덮어쓰지 않습니다**. 새 버전을 하나 더 만듭니다.

직접 봅시다. 줄이 물리적으로 어디에 있는지는 ctid 로 볼 수 있습니다.

```
await 가.query("DROP TABLE IF EXISTS 버전보기 CASCADE");
await 가.query("CREATE TABLE 버전보기 (번호 INT PRIMARY KEY, 값 INT)");
await 가.query("INSERT INTO 버전보기 VALUES (1, 100)");

const 위치보기 = async () =>
  (await 가.query("SELECT ctid::text AS 자리, xmin::text AS 만든트랜잭션 FROM
버전보기 WHERE 번호 = 1")).rows[0];

const 고치기전 = await 위치보기();
await 가.query("UPDATE 버전보기 SET 값 = 200 WHERE 번호 = 1");
const 고친후 = await 위치보기();

console.log(`UPDATE 전 자리: ${고치기전.자리} · UPDATE 후 자리: ${고친후.자리}`);
```

출력 UPDATE 전 자리: (0,1) · UPDATE 후 자리: (0,2)

```
console.log("줄이 다른 자리로 옮겨 갔나:", 고치기전.자리 !== 고친후.자리);
```

출력 줄이 다른 자리로 옮겨 갔나: true

```
console.log("만든 트랜잭션 번호가 바뀌었나:", 고치기전.만든트랜잭션 !== 고치후.
만든트랜잭션);
```

출력 만든 트랜잭션 번호가 바뀌었나: true

★★ UPDATE 한 번에 줄이 (0,1) 에서 (0,2) 로 갔습니다. 옛 버전은 (0,1) 에 그대로 남아 있습니다.

지워지지 않았습니다. xmin 은 "이 버전을 만든 트랜잭션 번호" 입니다. 새 버전은 새 번호를 답니다.

그래서 각 트랜잭션은 시작할 때 "지금 몇 번까지 커밋됐나" 를 적어 둡니다. 그게 스냅샷 입니다. 사진 한 장이라고 생각하면 됩니다. 그 뒤로는 자기 사진에 찍힌 버전만 봅니다. 남이 새 버전을 만들어도 안 보입니다.

★★★ 이래서 이런 일이 가능합니다.

쓰는 중에 읽어 봅니다.

```
await 나.query("BEGIN");
await 나.query("UPDATE 버전보기 SET 값 = 555 WHERE 번호 = 1"); // 잠금을 쥔 채 const
읽기시작 = performance.now();
const 읽은값 = (await 가.query("SELECT 값 FROM 버전보기 WHERE 번호 = 1")).rows[0].값;
const 읽기ms = performance.now() - 읽기시작;

await 나.query("ROLLBACK");

console.log(`남이 고치는 중에 읽은 값: ${읽은값} (${읽기ms.toFixed(0)} ms 만에)`);
```

출력 남이 고치는 중에 읽은 값: 200 (1 ms 만에)

값이 다를 수 있음

```
console.log("기다리지 않고 읽었나 (100ms 미만):", 읽기ms < 100);
```

출력 기다리지 않고 읽었나 (100ms 미만): true

★★★ 읽기는 쓰기를 기다리지 않습니다. 쓰기도 읽기를 기다리지 않습니다.

이게 Postgres 의 가장 큰 장점입니다. 옛 버전이 남아 있으니 읽는 사람은 옛 버전을 보면 됩니다. 잠금을 기다릴 이유가 없습니다.

★ 잠금으로 격리를 구현한 DB 는 이게 안 됩니다.

"누가 고치는 중이면 읽는 사람도 기다린다" 가 됩니다.
보고서 하나 돌리면 서비스 전체가 느려집니다.

★★ 공짜는 아닙니다. 옛 버전이 계속 쌓입니다.

누군가 치워야 합니다. 그게 VACUUM 입니다.
그리고 **긴 트랜잭션이 열려 있으면 치우지 못합니다.**
개념04 에서 실제로 못 치우는 것을 확인합니다.

★ 사진은 BEGIN 이 아니라 첫 문장에서 찍힙니다

섹션 8

흔한 오해입니다. "BEGIN 하는 순간 사진이 찍힌다" 고 생각합니다. 아닙니다.

```
await 처음상태로();

await 가.query("BEGIN ISOLATION LEVEL REPEATABLE READ"); // 아직 아무 문장도 안
보냈습니다 await 나.query("UPDATE 계측값 SET 온도 = 888 WHERE 번호 = 1"); // 나가
먼저 고치고 커밋합니다 const BEGIN뒤에본값 = await 온도읽기(가); // 여기가 첫
문장입니다
await 나.query("UPDATE 계측값 SET 온도 = 555 WHERE 번호 = 1"); // 또 고칩니다
const 그다음에본값 = await 온도읽기(가);

await 가.query("COMMIT");

console.log(`BEGIN 뒤에 남은 고친 값: ${BEGIN뒤에본값} · 첫 SELECT 뒤에 고친 값:
${그다음에본값}`);
```

출력 BEGIN 뒤에 남은 고친 값: 888 · 첫 SELECT 뒤에 고친 값: 888

★★ 888 이 보입니다. BEGIN 시점이 아니라 첫 **SELECT 시점**의 사진입니다. 그 뒤에 고친 555 는 안
보입니다. 사진은 그때 이미 찍혔으니까요.

★ 정확한 시점이 중요하면 BEGIN 직후에 아무 문장이나 하나 보내세요.

또는 Postgres 전용으로 `BEGIN; SELECT pg\_current\_snapshot();` 를 씁니다.

격리수준은 트랜잭션의 **첫 문장 전에** 정해야 합니다.

```
await 가.query("BEGIN");
await 가.query("SELECT 1");

try {
  await 가.query("SET TRANSACTION ISOLATION LEVEL REPEATABLE READ");
} catch (에러) {
  console.log(`문장을 보낸 뒤 격리수준 변경: ${에러.code} - ${에러.message}`);
}
```

출력 문장을 보낸 뒤 격리수준 변경: 25001 - SET TRANSACTION ISOLATION LEVEL
must be called before any query

```
}

await 가.query("ROLLBACK");
```

REPEATABLE READ 의 대가 — 40001

섹션 9

사진을 계속 보는 대신 값을 치릅니다. 내가 본 줄을 남이 먼저 고치면 커밋이 거부됩니다.

```
await 처음상태로();

await 가.query("BEGIN ISOLATION LEVEL REPEATABLE READ");
await 온도읽기(가); // 사진을 찍습니다 await 나.query("UPDATE 계측값 SET 온도 = 500
WHERE 번호 = 1"); // 나가 먼저 고칩니다 try {
  await 가.query("UPDATE 계측값 SET 온도 = 600 WHERE 번호 = 1");
  console.log("가의 UPDATE 가 성공했습니다");
} catch (에러) {
  console.log(`REPEATABLE READ 쓰기 충돌: ${에러.code} - ${에러.message}`);
}
```

```
출력      REPEATABLE READ 쓰기 충돌: 40001 - could not serialize access due to
          concurrent update
```

```
}

await 가.query("ROLLBACK");
```

★ 40001 은 “네 잘못이 아니다. 다시 해 봐라” 는 뜻입니다. READ COMMITTED 였다면 기다렸다가 새 값을 다시 읽고 성공했을 것입니다(개념02 섹션4). REPEATABLE READ 는 사진을 지켜야 하니 그럴 수가 없어서 포기합니다.

★★ 그래서 REPEATABLE READ 이상을 쓰면 **재시도 코드가 반드시 필요합니다**. 개념05 에서 제대로 만듭니다.

SERIALIZABLE — 사진만으로 못 막는 것

섹션 10

REPEATABLE READ 로도 못 막는 이상 현상이 있습니다. 쓰기 왜곡(write skew) 입니다.

상황: 당직은 항상 최소 한 명이 있어야 합니다. 김반장과 이반장이 둘 다 당직입니다. 둘 다 동시에 “나 말고 한 명 더 있네” 하고 뺍니다. 각자 보기에는 규칙을 안 어겼습니다. 그런데 합치면 당직이 0명이 됩니다.

```

await 가.query("CREATE TABLE 당직 (번호 INT PRIMARY KEY, 이름 TEXT NOT NULL, 당직여부
BOOLEAN NOT NULL)");

async function 당직초기화() {
  await 나.query("TRUNCATE 당직");
  await 나.query("INSERT INTO 당직 VALUES (1, '김반장', true), (2, '이반장', true)");
}

const 당직수 = async (연결) =>
  Number((await 연결.query("SELECT count(*) FROM 당직 WHERE
당직여부")).rows[0].count);

```

먼저 SERIALIZABLE 로 해 봅니다.

```

await 당직초기화();

await 가.query("BEGIN ISOLATION LEVEL SERIALIZABLE");
await 나.query("BEGIN ISOLATION LEVEL SERIALIZABLE");

console.log(`가가 센 당직: ${await 당직수(가)}명 · 나가 센 당직: ${await 당직수(나)}
명`);

```

출력 가가 센 당직: 2명 · 나가 센 당직: 2명

둘 다 “두 명이니 나 하나 빠져도 되겠다” 고 판단합니다.

```

await 가.query("UPDATE 당직 SET 당직여부 = false WHERE 번호 = 1");
await 나.query("UPDATE 당직 SET 당직여부 = false WHERE 번호 = 2");

await 가.query("COMMIT");
console.log("가는 커밋 성공");

```

출력 가는 커밋 성공

```

try {
  await 나.query("COMMIT");
  console.log("나도 커밋 성공");
} catch (에러) {
  console.log(`나는 커밋 거부: ${에러.code} - ${에러.message}`);
}

```

출력 나는 커밋 거부: 40001 – could not serialize access due to read/write
dependencies among transactions

```

console.log(`  힌트: ${에러.hint}`);

```

출력 힌트: The transaction might succeed if retried.

```
await 나.query("ROLLBACK");
}

console.log("남은 당직:", await 당직수(가), "명");
```

출력 남은 당직: 1 명

★★ 당직이 0명이 되는 것을 막았습니다.

★ 여기서 중요한 것: 두 사람은 **서로 다른 줄**을 고쳤습니다.

번호 1과 번호 2입니다. 잠금으로는 절대 못 막습니다. 부딪히는 줄이 없으니까요.
REPEATABLE READ 도 못 막습니다. 각자 자기 사진 안에서는 문제가 없습니다.

SERIALIZABLE 은 **읽은 것까지 추적** 합니다.
"나는 당직 목록을 읽고 판단했다" 를 기억해 뒀다가,
그 목록을 남이 바꿨으면 커밋을 거부합니다.

★★★ SERIALIZABLE 을 쓰면 **애플리케이션이 반드시 재시도해야 합니다**. "혹시 실패하면" 이 아닙니다.
정상 동작의 일부로 실패합니다. 재시도 코드가 없으면 손님이 그냥 에러 화면을 봅니다.

어떻게 정하고, 언제 올리나

섹션 11

07

정하는 방법은 세 가지입니다.

① 트랜잭션마다 BEGIN ISOLATION LEVEL REPEATABLE READ

② 트랜잭션 시작 직후 BEGIN; SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
③ 연결 전체 SET default_transaction_isolation = 'repeatable read';

③ 을 확인해 봅니다.

```
await 나.query("SET default_transaction_isolation = 'repeatable read'");
await 나.query("BEGIN");
console.log("연결 기본값을 바꾼 뒤:", (await 나.query("SHOW
transaction_isolation")).rows[0].transaction_isolation);
```

출력 연결 기본값을 바꾼 뒤: repeatable read

```
await 나.query("COMMIT");
await 나.query("SET default_transaction_isolation = 'read committed');"
```

★★ 언제 올려야 하나 — 대부분은 안 올려도 됩니다

READ COMMITTED (기본값) 로 충분한 경우

- 한 문장으로 끝나는 일 (UPDATE ... SET 수량 = 수량 - 1)
- 목록 조회, 상세 조회
- 갱신 손실은 개념02 의 세 가지 방법으로 막으면 됩니다
- ★ 실무 코드의 95% 는 여기입니다. 손대지 마세요.

REPEATABLE READ 로 올릴 때

- 여러 문장으로 한 시점의 자료를 읽어야 할 때 (월말 정산, 대사 작업)
- 백업이나 대용량 리포트
- ★ 재시도 코드를 같이 넣으세요. 40001 이 납니다.

SERIALIZABLE 로 올릴 때

- "읽고 판단해서 쓰는" 규칙이 있고, 그 규칙이 여러 줄에 걸쳐 있을 때 (당직 최소 한 명, 좌석 중복 예약, 잔액이 음수가 되면 안 됨)
- 그 규칙을 제약조건 하나로 표현할 수 없을 때
- ★★ 재시도 코드가 **필수** 입니다. 없으면 쓰지 마세요.

★ 격리수준을 올리기 전에 먼저 생각해 볼 것

제약조건(UNIQUE, CHECK)으로 표현할 수 있으면 그게 훨씬 쌉니다.
"좌석 중복 예약" 은 (공연, 좌석) 에 UNIQUE 를 걸면 끝입니다.
격리수준은 제약으로 안 되는 것에만 쓰세요.

정리

격리수준 막아 주는 것 대가

READ UNCOMMITTED (Postgres 에는 없음. READ COMMITTED 로 동작) READ COMMITTED
더티 읽기 없음 — 기본값 REPEATABLE READ + 반복 불가 읽기, 팬텀(Postgres 한정) 40001 재시도
필요 SERIALIZABLE + 쓰기 왜곡 (읽은 것까지 추적) 40001 재시도 필수, 조금 느림

에러코드 뜻 어떻게 해야 하나

40001	직렬화 실패	트랜잭션 전체를 처음부터 다시 (개념05)
25001	문장을 보낸 뒤 격리수준 변경	BEGIN 직후에 정하세요

★★★ 이 개념에서 꼭 기억할 것 ① Postgres 의 REPEATABLE READ 는 **팬텀도 막습니다**. 교과서 표와 다릅니다 ② Postgres 에는 READ UNCOMMITTED 가 사실상 없습니다. 더티 읽기는 안 납니다 ③ MVCC 덕분에 **읽기와 쓰기가 서로를 안 막습니다**. 대신 옛 버전이 쌓입니다 ④ 사진은 BEGIN 이 아니라 **첫 문장**에서 찍힙니다 ⑤ 기본값으로 충분한 경우가 대부분입니다. 올리면 재시도 코드가 달려 옵니다

직접 해 볼 것

◦ 직접 해보기 1

섹션 3 의 격리수준을 REPEATABLE READ 로 바꿔 보세요. 결과가 어떻게 되나요? 섹션 5 와 같아지나요?

◦ 직접 해보기 2

섹션 4 의 INSERT 를 DELETE 로 바꿔 보세요. 줄이 사라지는 것도 팬텀일까요? (힌트: 네, 그것도 팬텀입니다)

◦ 직접 해보기 3

섹션 7 에서 UPDATE 를 다섯 번 연속 해 보고 ctid 를 찍어 보세요. 자리가 어떻게 움직이나요? 옛 자리들은 어떻게 될까요?

◦ 직접 해보기 4

섹션 8 에서 BEGIN 바로 뒤에 SELECT 1 을 넣어 보세요. 결과가 888 에서 100 으로 바뀌나요? 왜 그럴까요?

◦ 직접 해보기 5

섹션 10 을 REPEATABLE READ 로 바꿔서 돌려 보세요. 당직이 몇 명 남나요? 쓰기 왜곡이 실제로 일어나는 것을 확인하세요.

◦ 직접 해보기 6

섹션 10 에서 가와 나가 같은 줄을 고치게 바꿔 보세요. 에러 코드가 40001 그대로인가요? 메시지는 어떻게 다른가요?

◦ 직접 해보기 7

SERIALIZABLE 두 개를 띄우고 읽기만 해 보세요. 40001 이 나나요? (힌트: 안 납니다. 읽기만 하면 충돌할 일이 없습니다)

자주 하는 실수

이런 실수를 합니다

교과서 표를 그대로 믿고 Postgres 에 적용함

"REPEATABLE READ 는 팬텀이 나니까 SERIALIZABLE 을 써야지" → Postgres 에서는 불필요합니다. 반대로 다른 DB 로 옮길 때는 그 표가 맞습니다. 어느 DB 인지가 답을 바꿉니다.

이런 실수를 합니다

격리수준을 올리면 갱신 손실도 막힌다고 생각함

REPEATABLE READ 는 갱신 손실을 막는 게 아니라 **에러로 알려 줍니다**(40001). 재시도를 안 붙이면 손님 화면에 에러가 뜰 뿐입니다. 개념02 의 세 가지 방법이 먼저입니다.

이런 실수를 합니다

40001 을 버그로 처리함

40001 은 정상 동작입니다. 로그에 error 로 쌓아 두고 원인을 찾으려 안 됩니다. 잡아서 다시 시도하세요. 개념05 에서 만듭니다.

이런 실수를 합니다

SERIALIZABLE 을 켜고 재시도를 안 붙임

가장 위험한 조합입니다. 평소에는 잘 되다가 손님이 몰릴 때만 실패합니다. 재시도가 없으면 SERIALIZABLE 은 켜지 마세요.

이런 실수를 합니다

트랜잭션 첫 문장 뒤에 격리수준을 바꾸려고 함

25001 이 납니다. BEGIN ISOLATION LEVEL ... 로 한 번에 쓰는 것이 안전합니다.

이런 실수를 합니다

사진이 BEGIN 에서 찍힌다고 믿음

BEGIN 과 첫 문장 사이에 남이 커밋한 것은 보입니다. 섹션 8 에서 확인했습니다. 연결 풀에서 꺼내고 BEGIN 하고 다른 일 하다가 SELECT 하면 그 사이가 꽤 벌어집니다.

이런 실수를 합니다

격리수준을 올려서 성능 문제를 풀려고 함

방향이 반대입니다. 올릴수록 재시도와 대기가 늘어납니다. 격리수준은 **정확성** 을 위한 손잡이지 성능 손잡이가 아닙니다.

```
await 가.end();
await 나.end();
```

잠금

RUN node 개념04_잠금.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 뒷부분에서 VACUUM 을 여러 번 돌려서 10초쯤 걸립니다.

개념02 에서 FOR UPDATE 를 잠깐 썼습니다. “줄을 잡아 둔다” 고만 하고 넘어갔습니다. 개념03 에서는 MVCC 덕분에 읽기가 쓰기를 안 막는다는 것을 봤습니다. 그러면 무엇이 무엇을 막는 걸까요? 이 파일에서 그 규칙을 봅니다.

· 잠금을 걸면 상대가 **얼마나** 기다리는지 재 봅니다 · 기다리지 않고 다른 일을 집어 가는 방법(SKIP LOCKED)으로 **작업 큐**를 만듭니다 · 지금 누가 누구를 막고 있는지 **보는 법**을 익힙니다 · 긴 트랜잭션이 왜 위험한지 **실제로 켜**입니다

★ 잠금은 직접 걸 일이 적습니다. UPDATE 나 DELETE 를 하면 알아서 걸립니다. 그런데 **무엇이 잠기는지 모르면** 서비스가 멈췄을 때 원인을 못 찾습니다.

```
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
}
```

★ 내 연결에 이름표를 겁니다. 섹션 6 에서 **내 세션만** 골라 보는 데 씁니다. 안 달면 pg_stat_activity 에 남의 세션까지 섞여 나옵니다. ★★ 이름표는 **영어로** 쓰세요. 한글을 넣으면 Postgres 가 \xeb\x8b\xa8... 처럼

16진수로 바꿔 버립니다. 이 자료에서 표 이름은 한글을 쓰지만 접속 정보는 영어입니다.

```

application_name: "unit07-lock",
};

let 가;

try {
  가 = new pg.Client(접속정보);
  await 가.connect();
} catch (에러) {
  console.log("Docker Postgres 에 못 붙었습니다. 먼저 이걸 실행하세요:");
  console.log(" docker compose up -d");
  console.log(` (원인: ${에러.message})`);
  process.exit(0);
}

await 가.query("CREATE SCHEMA IF NOT EXISTS 단원07");
await 가.query("SET search_path TO 단원07");

async function 새연결() {
  const 연결 = new pg.Client(접속정보);
  await 연결.connect();
  await 연결.query("SET search_path TO 단원07");
  return 연결;
}

const 나 = await 새연결();
const 감시 = await 새연결(); // 잠금 상태를 들여다보는 세 번째 사람 await
가.query("DROP TABLE IF EXISTS 작업지시, 청소시험 CASCADE");

await 가.query(`
  CREATE TABLE 작업지시 (
    번호 INT PRIMARY KEY,
    내용 TEXT NOT NULL,
    상태 TEXT NOT NULL DEFAULT '대기',
    처리자 TEXT
  )
`);

await 가.query("INSERT INTO 작업지시 (번호, 내용) SELECT g, '작업 ' || g FROM
generate_series(1, 12) g");

const 잠깐 = (밀리초) => new Promise((끝) => setTimeout(끝, 밀리초));

console.log("작업지시:", (await 가.query("SELECT count(*) FROM
작업지시")).rows[0].count, "건");

```

출력

작업지시: 12 건

SELECT ... FOR UPDATE — 얼마나 기다리나

섹션 1

가가 1번 줄을 잡습니다. 나가 같은 줄을 잡으려 하면 기다립니다. **얼마나** 기다리는지 재 봅니다. 가가 300ms 뒤에 커밋하게 해 두겠습니다.

```
await 가.query("BEGIN");
await 가.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR UPDATE");

const 기다리기시작 = performance.now();
```

★ await 를 하지 않습니다. 띄워만 두고 아래로 내려갑니다. await 를 붙이면 여기서 영원히 멈춥니다. 가가 아직 안 봤으니까요.

```
const 나의대기 = 나
  .query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR UPDATE")
  .then(() => performance.now() - 기다리기시작);

await 잠깐(300); // 가가 300ms 동안 잠금을 쥐고 있습니다
await 가.query("COMMIT"); // 놓습니다 const 기다린ms = await 나의대기;
await 나.query("ROLLBACK");

console.log(`나가 기다린 시간: ${기다린ms.toFixed(0)} ms`);
```

출력

나가 기다린 시간: 311 ms

값이 다를 수 있음

```
console.log("가가 쥔 300ms 만큼 기다렸나:", 기다린ms >= 290);
```

출력

가가 쥔 300ms 만큼 기다렸나: true

★★ 나는 가가 커밋할 때까지 기다렸습니다. 가가 3분을 쥐면 3분을 기다립니다. 그동안 나는 아무것도 못 합니다. 손님은 화면이 멈춘 것을 봅니다. ★ 그래서 트랜잭션은 **짧아야** 합니다. 잠금을 쥔 채 외부 API 를 부르면

그 API 가 느린 만큼 우리가 멈춥니다.

무엇이 무엇을 막나

섹션 2

잠금에는 종류가 있고, 종류끼리 부딪히는 규칙이 정해져 있습니다. 하나씩 확인합니다.

SELECT (그냥) 아무 줄 잠금도 안 걸. 아무도 안 막고 아무에게도 안 막힘 SELECT ... FOR SHARE “내가 읽는 동안 고치지 마”. 읽기끼리는 같이 됨 SELECT ... FOR UPDATE “내가 고칠 거야”. 아무도 못 건드림

UPDATE / DELETE FOR UPDATE 와 같은 잠금이 자동으로 걸림

① 순수 SELECT 는 안 막힙니다 (개념03 의 MVCC)

```
await 가.query("BEGIN");
await 가.query("UPDATE 작업지시 SET 내용 = '고치는 중' WHERE 번호 = 1");

const 읽기시작 = performance.now();
const 읽은것 = (await 나.query("SELECT 내용 FROM 작업지시 WHERE 번호 = 1")).rows[0].
내용;
const 읽기ms = performance.now() - 읽기시작;

await 가.query("ROLLBACK");

console.log(`① 남이 고치는 중에 그냥 SELECT: "${읽은것}" (${읽기ms.toFixed(0)}
ms)`);
```

출력 ① 남이 고치는 중에 그냥 SELECT: "작업 1" (1 ms)
값이 다를 수 있음

```
console.log("① 안 기다렸나:", 읽기ms < 100);
```

출력 ① 안 기다렸나: true

② FOR SHARE 끼리는 같이 됩니다

```
await 가.query("BEGIN");
await 가.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR SHARE");

await 나.query("BEGIN");
const 공유시작 = performance.now();
await 나.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR SHARE");
const 공유ms = performance.now() - 공유시작;

console.log("② FOR SHARE 둘이 같이 되나:", 공유ms < 100);
```

출력 ② FOR SHARE 둘이 같이 되나: true

③ FOR SHARE 위에 FOR UPDATE 는 막힙니다 기다리다 멈추면 곤란하니 lock_timeout 을 걸어 둡니다. 섹션 8 에서 자세히 봅니다.

```
await 나.query("SET lock_timeout = '200ms'");

try {
  await 나.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR UPDATE");
  console.log("③ FOR UPDATE 성공");
} catch (에러) {
  console.log(`③ FOR SHARE 위에 FOR UPDATE: ${에러.code} - ${에러.message}`);
}
```

출력 ③ FOR SHARE 위에 FOR UPDATE: 55P03 – canceling statement due to lock timeout

```
}

await 나.query("ROLLBACK");
await 나.query("SET lock_timeout = 0");
await 가.query("COMMIT");
```

★★ 정리하면 규칙은 한 줄입니다. 읽기는 아무것도 안 막습니다. 쓰려는 사람끼리만 부딪힙니다. FOR SHARE 는 “읽는 중이니 고치지 마” 라서 쓰려는 사람만 막습니다. FOR UPDATE 는 “내가 고칠 거야” 라서 FOR SHARE 도 FOR UPDATE 도 막습니다.

★ FOR SHARE 는 자식 줄을 넣는 동안 부모 줄이 사라지면 안 될 때 씁니다. 외래키가 있으면 Postgres 가 알아서 걸어 주므로 직접 쓸 일은 드뭅니다.

기다리지 않는 두 가지 — NOWAIT 과 SKIP LOCKED

섹션 3

기다리는 것 말고 두 가지 선택지가 더 있습니다. NOWAIT 잠겨 있으면 바로 에러를 냅니다 (55P03) SKIP LOCKED 잠긴 줄은 없는 셈 치고 건너뛰입니다

```
await 가.query("BEGIN");
await 가.query("SELECT * FROM 작업지시 WHERE 번호 IN (1, 2) FOR UPDATE"); // 1, 2번을
잡습니다 try {
  await 나.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR UPDATE NOWAIT");
  console.log("NOWAIT 성공");
} catch (에러) {
  console.log(`NOWAIT: ${에러.code} – ${에러.message}`);
```

출력 NOWAIT: 55P03 – could not obtain lock on row in relation "작업지시"

```
}

await 나.query("ROLLBACK");

const 건너뛴결과 = await 나.query("SELECT 번호 FROM 작업지시 ORDER BY 번호 FOR UPDATE
SKIP LOCKED");
await 나.query("ROLLBACK");
await 가.query("COMMIT");

console.log("SKIP LOCKED 로 잡은 번호:", 건너뛴결과.rows.map((줄) => 줄.번호).join(",
"));
```

출력 SKIP LOCKED 로 잡은 번호: 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

```
console.log("잠긴 1, 2번이 빠졌나:", !건너뛴결과.rows.some((줄) => 줄.번호 <= 2));
```

출력 잠긴 1, 2번이 빠졌나: true

★ 12건 중 1, 2번만 쏙 빠졌습니다. 에러도 대기도 없습니다. ★★ SKIP LOCKED 는 “지금 안 잠긴 것들”을 주므로 **결과가 매번 다릅니다**.

보고서나 합계에 쓰면 숫자가 틀립니다. 쓸 곳은 딱 하나, ****작업 큐**** 입니다.

★ SKIP LOCKED 로 작업 큐 만들기

섹션 4

실무에서 아주 자주 쓰는 패턴입니다. 이것 하나만 가져가도 이 파일은 본전입니다.

상황: 작업 12건을 일꾼 3명이 나눠서 처리합니다. ★ 두 사람이 **같은 작업**을 집으면 안 됩니다 (두 번 처리됩니다) ★ 한 사람이 느리다고 다른 사람이 **기다리면** 안 됩니다 이 둘을 동시에 만족시키는 것이 SKIP LOCKED 입니다.


```

await 가.query("UPDATE 작업지시 SET 상태 = '대기', 처리자 = NULL");

const 일꾼이름 = ["일꾼1", "일꾼2", "일꾼3"];
const 일꾼연결 = [];

for (const 이름 of 일꾼이름) 일꾼연결.push({ 이름, 연결: await 새연결() });

async function 일꾼돌리기({ 이름, 연결 }) {
  const 집은것 = [];

  while (true) {
    await 연결.query("BEGIN");

    const 집기 = await 연결.query(`
      SELECT 번호, 내용 FROM 작업지시
      WHERE 상태 = '대기'
      ORDER BY 번호
      LIMIT 1
      FOR UPDATE SKIP LOCKED
    `);

    if (집기.rowCount === 0) {
      await 연결.query("COMMIT"); // 더 집을 게 없습니다 break;
    }

    const 일 = 집기.rows[0];

    await 잠깐(10); // 진짜로 일하는 시간이라고 칩시다 await 연결.query("UPDATE
작업지시 SET 상태 = '완료', 처리자 = $1 WHERE 번호 = $2", [이름, 일.번호]);
    await 연결.query("COMMIT");

    집은것.push(일.번호);
  }

  return { 이름, 집은것 };
}

const 일꾼결과 = await Promise.all(일꾼연결.map(일꾼돌리기));

for (const { 이름, 집은것 } of 일꾼결과) {
  console.log(`${이름}: ${집은것.length}건 - ${집은것.join(", ")}`);
}

```

출력	일꾼1: 4건 - 2, 4, 8, 11
값이 다를 수 있음	일꾼2: 4건 - 1, 6, 7, 12
	일꾼3: 4건 - 3, 5, 9, 10

```
}
```

```
const 처리된번호 = 일꾼결과.flatMap((결과) => 결과.집은것);
```

```
console.log("전부 처리됐나:", 처리된번호.length === 12);
```

출력 전부 처리됐나: true

```
console.log("겹쳐서 처리한 것이 있나:", new Set(처리된번호).size !== 12);
```

출력 겹쳐서 처리한 것이 있나: false

```
console.log("남은 대기:", (await 가.query("SELECT count(*) FROM 작업지시 WHERE 상태 = '대기'")).rows[0].count, "건");
```

출력 남은 대기: 0 건

★★★ 12건이 3명에게 **겹치지 않게** 나눠졌습니다. 누가 몇 번을 집을지는 매번 다르지만 중복도 누락도 없습니다.

★ SKIP LOCKED 가 없으면

- 그냥 SELECT 로 집으면 → 세 명이 같은 1번을 집습니다. 세 번 처리됩니다
- FOR UPDATE 만 쓰면 → 두 명이 1번 앞에 줄을 섭니다. 한 명씩만 일합니다

★★ 큐 서버가 따로 필요 없습니다. 작은 서비스의 메일 발송·알림·정산 배치를

이걸로 다 돌립니다. 일꾼을 늘리려면 프로세스를 하나 더 띄우면 끝입니다.

★ 일꾼이 중간에 죽으면 롤백되어 그 작업은 '대기' 로 돌아옵니다. 좋은 일입니다.

대신 **한 번 처리한 일을 또 처리해도 괜찮게** 만들어 두세요.

줄 잠금 vs 표 잠금

섹션 5

지금까지는 전부 **줄 잠금**이었습니다. 표 전체를 잠글 수도 있습니다. LOCK TABLE 입니다. ★ 직접 쓸 일은 거의 없지만 ALTER TABLE, DROP TABLE, VACUUM FULL 이 자동으로 겁니다.

운영 중에 ALTER TABLE 을 했다가 서비스가 멈추는 사고가 여기서 납니다.

```
await 가.query("BEGIN");
await 가.query("LOCK TABLE 작업지시 IN SHARE MODE"); // 읽기는 되고 쓰기는 막는
모드 await 나.query("SET lock_timeout = '200ms'");

const 표잠금중읽기 = await 나.query("SELECT count(*) FROM 작업지시");
console.log("SHARE MODE 중에 읽기:", 표잠금중읽기.rows[0].count, "건 - 됩니다");
```

출력 SHARE MODE 중에 읽기: 12 건 - 됩니다

```
try {
  await 나.query("UPDATE 작업지시 SET 상태 = '대기' WHERE 번호 = 1");
  console.log("SHARE MODE 중에 쓰기: 됩니다");
} catch (에러) {
  console.log(`SHARE MODE 중에 쓰기: ${에러.code} - 막힙니다`);
}
```

출력 SHARE MODE 중에 쓰기: 55P03 - 막힙니다

```
}

await 가.query("COMMIT");
```

이번엔 제일 센 잠금입니다. ALTER TABLE 이 거는 것과 같습니다.

```
await 가.query("BEGIN");
await 가.query("LOCK TABLE 작업지시 IN ACCESS EXCLUSIVE MODE");

try {
  await 나.query("SELECT count(*) FROM 작업지시");
  console.log("ACCESS EXCLUSIVE 중에 읽기: 됩니다");
} catch (에러) {
  console.log(`ACCESS EXCLUSIVE 중에 읽기: ${에러.code} - 읽기조차 막힙니다`);
}
```

출력 ACCESS EXCLUSIVE 중에 읽기: 55P03 - 읽기조차 막힙니다

```
}

await 가.query("COMMIT");
await 나.query("SET lock_timeout = 0");
```

★★★ ACCESS EXCLUSIVE 는 읽기까지 막습니다. ALTER TABLE, DROP TABLE, TRUNCATE, VACUUM FULL 이 이걸 겁니다.

★ 운영 사고의 전형: ALTER TABLE 이 큰 표를 고치느라 30초를 잡습니다.

그동안 그 표를 읽으려는 요청이 전부 줄을 섭니다. 연결이 다 차고,
다른 표를 쓰는 요청까지 연결이 없어서 죽습니다.
표 하나 고치다 서비스 전체가 멈춥니다.

★ 그래서 운영 중 스키마 변경은 lock_timeout 을 짧게 걸고, 안 잡히면 포기하고,

CREATE INDEX CONCURRENTLY 처럼 안 막는 방법을 씁니다. 10단원에서 다시 다룹니다.

★ 지금 무엇이 잠겨 있나

섹션 6

서비스가 멈췄을 때 제일 먼저 봐야 하는 것입니다.

```
await 가.query("BEGIN");
await 가.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR UPDATE");

const 막힌작업 = 나.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR UPDATE"); //
    띄워만 둡니다
await 잠깐(400); // 확실히 막히도록 잠깐 둡니다
```

★ 이 DB 는 다른 단원도 같이 씁니다. 그래서 **내 이름표가 붙은 세션만** 봅니다.

```
const 내세션만 = "pid IN (SELECT pid FROM pg_stat_activity WHERE application_name =
    'unit07-lock')";
```

① pg_locks — 안 잡힌 잠금이 있으면 누군가 기다리는 중입니다

```
const 대기잠금 = await 감시.query(`
    SELECT locktype, mode FROM pg_locks WHERE NOT granted AND ${내세션만}
`);
```

```
console.log("기다리는 중인 잠금:", JSON.stringify(대기잠금.rows));
```

출력 기다리는 중인 잠금: [{"locktype":"transactionid","mode":"ShareLock"}]

② pg_stat_activity + pg_blocking_pids() — ★ 이게 제일 쓸모 있습니다

```
const 누가막나 = await 감시.query(`
  SELECT wait_event_type AS 왜기다림,
         cardinality(pg_blocking_pids(pid)) AS 막는사람수,
         left(query, 30) AS 문장
  FROM pg_stat_activity
  WHERE application_name = 'unit07-lock'
         AND cardinality(pg_blocking_pids(pid)) > 0
`);

console.log("막혀 있는 세션:", JSON.stringify(누가막나.rows));
```

```
출력      막혀 있는 세션: [{"왜기다림":"Lock","막는사람수":1,"문장":"SELECT * FROM
작업지시 WHERE 번호 = "}]
```

③ 아무 일도 안 하면서 트랜잭션만 열어 둔 세션 — 제일 위험한 놈입니다

```
const 놀고있는트랜잭션 = await 감시.query(`
  SELECT count(*) FROM pg_stat_activity
  WHERE state = 'idle in transaction' AND application_name = 'unit07-lock'
`);

console.log("idle in transaction 세션 수:", 놀고있는트랜잭션.rows[0].count);
```

```
출력      idle in transaction 세션 수: 1
```

```
await 가.query("COMMIT");
await 막힌작업;
await 나.query("ROLLBACK");
```

★ application_name 을 달아 두면 이렇게 내 것만 골라 볼 수 있습니다. 연결할 때 한 줄만 넣으면 됩니다. 서비스마다 다르게 달아 두면 “지금 누가 DB 를 붙잡고 있나” 를 바로 알 수 있습니다. 실무에서 꼭 하세요.

★★ 실무에서 쓰는 순서 ① pg_stat_activity 에서 state = ‘idle in transaction’ 을 찾는다

→ BEGIN 만 하고 잊어버린 세션입니다. 잠금을 쥔 채 놀고 있습니다

② pg_blocking_pids(pid) 로 누가 누구를 막는지 확인한다 ③ 정 급하면 pg_terminate_backend(막는pid) 로 끊는다 — 마지막 수단입니다.

그 트랜잭션은 롤백됩니다. 끊기 전에 query 칸을 꼭 보세요.

이유가 두 개입니다. ① 잠금을 오래 줍니다 — 섹션 1 에서 봤습니다 ② **청소(VACUUM)를 막습니다** — 이게 덜 알려져 있고 더 무섭습니다

개념03 에서 봤듯 Postgres 는 UPDATE 할 때 새 버전을 만들고 옛 버전을 남깁니다. 옛 버전은 VACUUM 이 치우는데, **아무도 안 볼 때만** 치울 수 있습니다. 긴 트랜잭션이 열려 있으면 “저 사람이 볼지도 모른다” 며 못 치웁니다.

재 봅니다. 2000줄짜리 표를 10번 통째로 UPDATE 하고, 매번 VACUUM 합니다.

```
const 방해꾼 = await 새연결();

const 표크기KB = async () =>
  Number((await 가.query("SELECT
pg_total_relation_size('청소시험')")).rows[0].pg_total_relation_size) / 1024;

const 죽은줄수 = async () =>
  Number((await 가.query("SELECT n_dead_tup FROM pg_stat_user_tables WHERE relname =
'청소시험'")).rows[0].n_dead_tup);

async function 청소시험(긴트랜잭션을열어둘까) {
  await 가.query("DROP TABLE IF EXISTS 청소시험 CASCADE");
  await 가.query("CREATE TABLE 청소시험 (번호 INT PRIMARY KEY, 값 INT NOT NULL)");
  await 가.query("INSERT INTO 청소시험 SELECT g, g FROM generate_series(1, 2000) g");
  await 가.query("VACUUM 청소시험");

  const 처음KB = await 표크기KB();

  if (긴트랜잭션을열어둘까) {
    await 방해꾼.query("BEGIN ISOLATION LEVEL REPEATABLE READ");
    await 방해꾼.query("SELECT count(*) FROM 청소시험"); // 여기서 사진이 찍힙니다
  }

  for (let 회차 = 0; 회차 < 10; 회차 += 1) {
    await 가.query("UPDATE 청소시험 SET 값 = 값 + 1");
    await 가.query("VACUUM 청소시험"); // 매번 청소를 시킵니다
  }

  const 끝KB = await 표크기KB();
  const 남은죽은줄 = await 죽은줄수();

  if (긴트랜잭션을열어둘까) await 방해꾼.query("COMMIT");

  return { 처음KB, 끝KB, 남은죽은줄 };
}
```

```
const 열어둠 = await 청소시험(true);
```

★ 이 DB 는 다른 단원도 같이 씁니다. **남이** 긴 트랜잭션을 열어 두면 비교 대상인 “없음” 쪽도 청소가 막혀서 결과가 흐려집니다. 그래서 깨끗하게 나올 때까지 다시 켭니다. 측정에서는 이런 방해를 걷어 내야 합니다.

```
let 안열어둠 = await 청소시험(false);
```

```
for (let 다시 = 0; 안열어둠.남은죽은줄 > 0 && 다시 < 10; 다시 += 1) {
  await 잠깐(500);
  안열어둠 = await 청소시험(false);
}
```

```
console.log(`긴 트랜잭션 열어 둠: ${열어둠.처음KB} KB → ${열어둠.끝KB} KB · 못 치운
죽은 줄 ${열어둠.남은죽은줄}개`);
```

```
출력      긴 트랜잭션 열어 둠: 168 KB → 1096 KB · 못 치운 죽은 줄 20000개
값이 다를 수 있음
```

```
console.log(`긴 트랜잭션 없음:  ${안열어둠.처음KB} KB → ${안열어둠.끝KB} KB · 못
치운 죽은 줄 ${안열어둠.남은죽은줄}개`);
```

```
출력      긴 트랜잭션 없음:   168 KB → 280 KB · 못 치운 죽은 줄 0개
값이 다를 수 있음
```

★★ 위 두 줄은 // 출력?: 입니다. 크기는 **기계와 상황에 따라 달라집니다**. 실제로 이 자료를 만들 때, 다른 사람이 같은 데이터베이스를 쓰고 있던 순간에 다시 켜더니 280 KB 가 664 KB 로, 3.9배가 1.7배로 바뀌었습니다. 남이 연 트랜잭션이 내 VACUUM 까지 막았기 때문입니다.

★ ‘못 치운 죽은 줄’ 숫자도 흔들립니다. 재는 순간 autovacuum 이 이미 치웠으면 0 이 됩니다.

★★ 그래서 진짜 검사는 아래 한 줄입니다. **크기 비교는 흔들리지 않습니다**.

```
긴 트랜잭션이 열려 있는 동안에는 죽은 줄을 못 치우고, 못 치우면 표가 커집니다.
그 인과는 상황이 바뀌어도 그대로입니다.
```

```
console.log("긴 트랜잭션 쪽이 더 부풀었나:", 열어둠.끝KB > 안열어둠.끝KB);
```

```
출력      긴 트랜잭션 쪽이 더 부풀었나: true
```

```
console.log("몇 배나 부풀었나:", (열어둠.끝KB / 안열어둠.끝KB).toFixed(1), "배");
```

```
출력      몇 배나 부풀었나: 3.9 배
값이 다를 수 있음
```

★★★ 같은 일을 했는데 표 크기가 4배 가까이 차이 납니다.

VACUUM 을 10번이나 돌렸는데도 소용없었습니다. **읽기만 하는** 트랜잭션 하나가 열려 있었을 뿐인데요.

이걸 표 부풀림(table bloat)이라고 합니다. 진행되면 같은 자료를 읽는 데 디스크를 더 많이 읽고, 색인도 같이 부풀고, 디스크가 차고, 심하면 트랜잭션 번호가 바닥나 DB 가 쓰기를 거부합니다.

★ 원인 1위는 **BEGIN** 하고 잊어버린 세션 입니다.

화면을 열어 놓고 점심 먹으러 간 관리자, 예외 처리를 빼먹은 코드, 연결 풀에 반납 안 된 연결. 전부 여기에 해당합니다.

★ 그래서 섹션 6 의 `idle in transaction` 확인과

섹션 8 의 `idle_in_transaction_session_timeout` 이 중요합니다.
`VACUUM` 과 부풀림을 제대로 다루는 것은 10단원에서 합니다.

언제까지 기다릴지 정하기

섹션 8

기본값은 **영원히 기다림** 입니다. 운영에서 위험합니다.

```
console.log("기본 lock_timeout:", (await 가.query("SHOW lock_timeout")).rows[0].lock_timeout, "(0 = 영원히)");
```

출력 기본 lock_timeout: 0 (0 = 영원히)

① lock_timeout — 잠금을 기다리는 시간만 제한합니다

```
await 가.query("BEGIN");
await 가.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR UPDATE");

await 나.query("SET lock_timeout = '150ms'");
const 잠금대기시작 = performance.now();

try {
  await 나.query("SELECT * FROM 작업지시 WHERE 번호 = 1 FOR UPDATE");
} catch (에러) {
  const 걸린 = performance.now() - 잠금대기시작;
  console.log(`lock_timeout: ${에러.code} - ${걸린.toFixed(0)} ms 만에 포기`);
}
```

출력 lock_timeout: 55P03 - 152 ms 만에 포기

값이 다를 수 있음

```
console.log("150ms 안팎에 포기했나:", 걸린 >= 140 && 걸린 < 1000);
```

출력 150ms 안팎에 포기했나: true


```

}

await 나.query("ROLLBACK");
await 가.query("COMMIT");
await 나.query("SET lock_timeout = 0");

```

② statement_timeout — 문장 하나가 걸리는 전체 시간을 제한합니다

```

await 나.query("SET statement_timeout = '150ms'");

try {
  await 나.query("SELECT pg_sleep(3)"); // 3초 자는 문장
} catch (에러) {
  console.log(`statement_timeout: ${에러.code} - ${에러.message}`);
}

```

```
출력      statement_timeout: 57014 - canceling statement due to statement timeout
```

```

}

await 나.query("SET statement_timeout = 0");

```

③ idle_in_transaction_session_timeout — BEGIN 하고 노는 세션을 끊습니다 ★ 이걸 세션을 진짜로 끊어 버립니다. 그래서 버릴 연결을 하나 따로 만듭니다.

```
const 희생양 = await 새연결();
```

★★ 서버가 연결을 끊으면 pg 드라이버가 'error' 이벤트를 냅니다. 이걸 안 받아 두면 **노드 프로세스가 통째로 죽습니다**. 반드시 달아 두세요. 그리고 끊긴 진짜 이유는 이 이벤트로만 옵니다. query 의 catch 로는 안 옵니다.

```

let 끊긴이유 = null;
희생양.on("error", (에러) => { 끊긴이유 ??= 에러.message; });

await 희생양.query("SET idle_in_transaction_session_timeout = '200ms'");
await 희생양.query("BEGIN");
await 희생양.query("SELECT 1");
await 잠깐(500); // 트랜잭션을 열어 둔 채 아무것도 안 합니다 try {
  await 희생양.query("SELECT 1");
  console.log("세션이 살아 있습니다");
} catch (에러) {
  console.log("query 가 알려 주는 것:", 에러.message);
}

```

```
출력      query 가 알려 주는 것: Client has encountered a connection error and is not queryable
```

```
console.log("서버가 알려 준 진짜 이유:", 끊긴이유);
```

```
출력      서버가 알려 준 진짜 이유: terminating connection due to idle-in-transaction timeout
```

```
}
```

★★ 세 가지를 구분하세요.

lock_timeout	잠금을 **기다리는** 시간만 쟀다
statement_timeout	문장 하나의 **전체** 시간을 쟀다
idle_in_transaction_session_timeout	BEGIN 하고 **노는** 시간을 쟀다

★ 운영 서버에는 셋 다 걸어 두세요. 특히 세 번째가 섹션 7의 부풀림을 막습니다. ★ ALTER TABLE 전에는 lock_timeout 을 짧게 거세요. 안 잡히면 빨리 포기하는 편이 낫습니다.

정리

문법 하는 일

— —————

SELECT ... FOR UPDATE 줄을 잡는다. 남은 기다린다 SELECT ... FOR SHARE "읽는 중이니 고치지 마". 읽기끼리는 같이 됨 SELECT ... FOR UPDATE NOWAIT 잠겨 있으면 바로 55P03 SELECT ... FOR UPDATE SKIP LOCKED 잠긴 줄은 건너뛰다 — 작업 큐의 핵심 LOCK TABLE ... IN ... MODE 표 전체를 잠근다 (직접 쓸 일은 드뭄)

설정 (셋 다 기본값 0 = 무제한) 무엇을 재나

— —————

lock_timeout	잠금 대기 시간
statement_timeout	문장 전체 시간
idle_in_transaction_session_timeout	BEGIN 하고 노는 시간

에러코드 뜻

— —————

55P03	잠금을 못 잡았다 (NOWAIT 또는 lock_timeout)
57014	문장이 취소됐다 (statement_timeout 또는 사용자 취소)

★★★ 꼭 기억할 것 ① 읽기는 아무것도 안 막습니다. 쓰려는 사람끼리만 부딪힙니다 ② SKIP LOCKED 로 작업 큐를 만들 수 있습니다. 큐 서버가 필요 없습니다 ③ 긴 트랜잭션은 잠금만 주는 게 아니라 **청소를 막아**

표를 부풀립니다 ④ 막혔을 때는 `pg_blocking_pids()` 로 누가 막는지 보세요 ⑤ 운영 서버에는 타임아웃 세가지를 걸어 두세요

직접 해 볼 것

▢ 직접 해보기 1

섹션 1 의 잠깐(300) 을 2000 으로 바꿔 보세요. 나는 몇 ms 를 기다리나요? 상한이 있나요?

▢ 직접 해보기 2

섹션 3 의 SKIP LOCKED 를 지우고 돌려 보세요. 몇 건이 나오나요? 왜 그럴까요?

▢ 직접 해보기 3

섹션 4 의 일꾼을 3명에서 6명으로 늘려 보세요. 12건이 어떻게 나뉘나요? 전체 시간이 반으로 줄었나요?

▢ 직접 해보기 4

섹션 4 에서 SKIP LOCKED 를 빼고 FOR UPDATE 만 남기면 결과는 맞나요? 시간은요? (힌트: 맞지만 줄을 섭니다)

▢ 직접 해보기 5

섹션 7 의 UPDATE 횟수를 10 에서 30 으로 늘리면 표가 몇 배로 부풀나요? 비례해서 늘어나나요?

▢ 직접 해보기 6

섹션 7 에서 방해꾼의 격리수준을 READ COMMITTED 로 바꿔 보세요. 여전히 청소를 막나요?

▢ 직접 해보기 7

`statement_timeout` 을 걸어 놓고 잠금을 기다리면 55P03 이 나오나요 57014 가 나오나요?

자주 하는 실수

이런 실수를 합니다

FOR UPDATE 를 BEGIN 없이 씀

자동 커밋이라 그 문장이 끝나는 순간 잠금이 풀립니다. 잠근 의미가 없습니다.

이런 실수를 합니다

SKIP LOCKED 를 조회에 씀

목록에서 줄이 빠져도 아무 표시가 안 납니다. 작업 큐 말고는 쓰지 마세요.

이런 실수를 합니다

잠금을 쥔 채로 외부 API 를 부름

상대가 5초 걸리면 그 줄을 쓰려는 모든 요청이 5초 멈춥니다. 밖으로 빼세요.

이런 실수를 합니다

운영 중에 ALTER TABLE 을 그냥 실행함

ACCESS EXCLUSIVE 를 잡습니다. 읽기까지 막힙니다. ★ lock_timeout 을 짧게 걸고 하세요. 안 잡히면 포기하고 다시 시도하세요.

이런 실수를 합니다

타임아웃을 안 걸어 둬

기본값이 "영원히 기다림" 입니다. 기다리는 요청이 쌓여 연결이 다 찹니다.

이런 실수를 합니다

pg_locks 에 안 보이니 안 잠겼다고 생각함

줄 잠금은 줄 자체(튜플 헤더)에 표시되고, 부딪힐 때만 pg_locks 에 나타납니다.

이런 실수를 합니다

긴 배치를 한 트랜잭션으로 돌림

백만 건을 한 번에 처리하면 그동안 청소가 멈추고 표가 부풀니다. ★ 만 건씩 끊어서 커밋하세요.

```
for (const { 연결 } of 일꾼연결) await 연결.end();
await 희생양.end().catch(() => {}); // 이미 서버가 끊었을 수 있습니다
await 방해꾼.end();
await 감시.end();
await 나.end();
await 가.end();
```

데드락

```
RUN node 개념05_데드락.js
```

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 데드락 감지에 1초씩 걸려서 이 파일은 15초쯤 걸립니다.

개념04 에서 잠금을 기다리는 것을 봤습니다. 상대가 놓을 때까지 기다렸습니다.

그런데 **상대도 나를 기다리고 있다면** 어떻게 될까요?

김반장: 베어링을 잡았다. 이제 벨트를 잡아야지 → 이반장이 쥐고 있다. 기다린다 이반장: 벨트를 잡았다.

이제 베어링을 잡아야지 → 김반장이 쥐고 있다. 기다린다

둘 다 영원히 기다립니다. 아무도 안 놓습니다. 이것이 **데드락(deadlock)** 입니다.

이 파일에서는 · 데드락을 **진짜로 만들어** 봅니다 (40P01 을 실제로 받습니다) · 어느 쪽이 죽는지 확인합니다

· 순서를 맞춰서 **안 나게** 만듭니다 · 그래도 날 때를 대비한 **재시도 코드**를 만듭니다

★ 데드락은 버그가 아닙니다. 여러 명이 여러 줄을 고치면 언젠가 납니다. 나는 것을 0 으로 만들 수는 없습니다. **덜 나게** 하고 **나면 다시 하게** 만드는 것이 답입니다.

```

import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit07-deadlock", // 이름표는 영어로 (개념04 섹션6)
};

let 가;

try {
  가 = new pg.Client(접속정보);
  await 가.connect();
} catch (에러) {
  console.log("Docker Postgres 에 못 붙었습니다. 먼저 이걸 실행하세요:");
  console.log(" docker compose up -d");
  console.log(` (원인: ${에러.message})`);
  process.exit(0);
}

await 가.query("CREATE SCHEMA IF NOT EXISTS 단원07");
await 가.query("SET search_path TO 단원07");

async function 새연결() {
  const 연결 = new pg.Client(접속정보);
  await 연결.connect();
  await 연결.query("SET search_path TO 단원07");
  return 연결;
}

const 나 = await 새연결();
const 잠깐 = (밀리초) => new Promise((끝) => setTimeout(끝, 밀리초));

```

```

await 가.query("DROP TABLE IF EXISTS 창고, 좌석예약 CASCADE");

await 가.query(`
  CREATE TABLE 창고 (
    번호 INT PRIMARY KEY,
    이름 TEXT NOT NULL,
    수량 INT NOT NULL
  )
`);

await 가.query("INSERT INTO 창고 VALUES (1, '베어링 6203', 10), (2, '벨트 A형', 10)");

const 창고보기 = async () => {
  (await 가.query("SELECT 번호, 수량 FROM 창고 ORDER BY 번호")).rows.map((줄)
=> `${줄.번호}번:${줄.수량}`).join(" ");

  console.log("시작 재고 -", await 창고보기());
}

```

출력 시작 재고 - 1번:10 2번:10

★ 데드락을 진짜로 만듭니다

섹션 1

두 사람이 **반대 순서**로 두 줄을 잠급니다. 그것뿐입니다.

가: 1번 → 2번 나: 2번 → 1번

```

async function 창고초기화() {
  await 가.query("UPDATE 창고 SET 수량 = 10");
}

async function 반대순서로() {
  await 창고초기화();
}

```

★ 둘을 await 없이 동시에 띄웁니다. 그래야 겹칩니다.

```

const 가의일 = (async () => {
  await 가.query("BEGIN");
  await 가.query("UPDATE 창고 SET 수량 = 수량 - 1 WHERE 번호 = 1"); // 1번을
  잠습니다 await 잠깐(150); // 나가 2번을 잠을 시간을 줍니다 await 가.query("UPDATE
  창고 SET 수량 = 수량 - 1 WHERE 번호 = 2"); // ← 여기서 막힙니다 await
  가.query("COMMIT");
  return "가 성공";
})();

const 나의일 = (async () => {
  await 나.query("BEGIN");
}

```

트랜잭션과 동시성

RUN node 연습문제.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

각 문제의 // TODO: 를 채우세요. 채우면 아래 채점이 "통과" 로 바뀝니다. 지금 실행하면 전부 "아직" 이 나옵니다. 정상입니다.

★ 순서대로 푸세요. 뒤로 갈수록 어렵습니다. 마지막 셋은 [도전] 입니다. ★ 막히면 개념01~05 를 다시 보세요. 문제마다 어느 개념인지 적어 뒀습니다. ★ 답을 보기 전에 **직접 돌려 보세요**. 동시성은 눈으로 봐야 이해됩니다.


```

import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit07-exercise",
};

let 가;

try {
  가 = new pg.Client(접속정보);
  await 가.connect();
} catch (에러) {
  console.log("Docker Postgres 에 못 붙었습니다. 먼저 이걸 실행하세요.");
  console.log(" docker compose up -d");
  console.log(` (원인: ${에러.message})`);
  process.exit(0);
}

await 가.query("CREATE SCHEMA IF NOT EXISTS 단위07");
await 가.query("SET search_path TO 단위07");

async function 새연결() {
  const 연결 = new pg.Client(접속정보);
  await 연결.connect();
  await 연결.query("SET search_path TO 단위07");
  return 연결;
}

const 나 = await 새연결();
const 다 = await 새연결();
const 모든연결 = [가, 나, 다];

```

```

const 잠깐 = (밀리초) => new Promise((끝) => setTimeout(끝, 밀리초));
const 한값 = async (문장, 값들 = []) => Object.values((await 가.query(문장,
값들)).rows[0])[0];

await 가.query("DROP TABLE IF EXISTS 연습_점검, 연습_설비, 연습_재고, 연습_큐, 연습_
당직 CASCADE");
await 가.query("CREATE TABLE 연습_설비 (번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
상태 TEXT NOT NULL)");
await 가.query("CREATE TABLE 연습_점검 (번호 SERIAL PRIMARY KEY, 설비번호 INT NOT
NULL REFERENCES 연습_설비(번호), 내용 TEXT NOT NULL)");
await 가.query("CREATE TABLE 연습_재고 (부품번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
수량 INT NOT NULL, 버전 INT NOT NULL DEFAULT 0)");
await 가.query("CREATE TABLE 연습_큐 (번호 INT PRIMARY KEY, 내용 TEXT NOT NULL, 상태
TEXT NOT NULL DEFAULT '대기', 처리자 TEXT)");
await 가.query("CREATE TABLE 연습_당직 (번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
당직여부 BOOLEAN NOT NULL)");

async function 초기화() {
  for (const 연결 of 모든연결) {
    try {
      await 연결.query("ROLLBACK");
    } catch {

```

열린 트랜잭션이 없으면 넘어갑니다

```

    }
  }

  await 가.query("TRUNCATE 연습_점검, 연습_설비, 연습_재고, 연습_큐, 연습_당직
RESTART IDENTITY CASCADE");
  await 가.query("INSERT INTO 연습_설비 VALUES (1, '컨베이어 1호', '가동'), (2,
'프레스 1호', '가동')");
  await 가.query("INSERT INTO 연습_점검 (설비번호, 내용) VALUES (1, '벨트 점검'), (2,
'유압 점검')");
  await 가.query("INSERT INTO 연습_재고 VALUES (1, '베어링', 100, 0), (2, '벨트',
100, 0)");
  await 가.query("INSERT INTO 연습_큐 (번호, 내용) SELECT g, '작업' || g FROM
generate_series(1, 9) g");
  await 가.query("INSERT INTO 연습_당직 VALUES (1, '김반장', true), (2, '이반장',
true)");
}

```

★ 채점하다가 실패해도 임시 연결은 **반드시** 닫아야 합니다. 안 닫으면 노드가 끝나지 않아서 채점이 멈춘 것처럼 보입니다. (이것도 실무에서 자주 하는 실수입니다. finally 를 쓰세요)

```

async function 연결여럿(개수, 할일) {
  const 연결들 = [];

```

애플리케이션과 잇기

OPEN 08_애플리케이션과_잇기

```
import pg from "pg";  
const 접속정보 = {  
  host: "127.0.0.1",  
  user: "factory",
```

01 서버에서 데이터베이스 부르기

02 커넥션 풀

03 저장소 계층으로 묶기

04 마이그레이션

05 ORM 은 언제 쓰나

- 연습문제

서버에서 데이터베이스 부르기

RUN node 개념01_서버에서_데이터베이스_부르기.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

여기까지 우리는 SQL 을 **손으로** 쳤습니다. 이제 자바스크립트 프로그램이 대신 치게 합니다.

브라우저 → (HTTP) → Node 서버 → (SQL) → PostgreSQL

이 단원은 가운데 화살표, **Node 서버가 데이터베이스를 부르는 부분**을 다룹니다.

이 파일에서 볼 것:

- ① 연결하는 두 가지 방법 — 설정 객체와 연결 문자열 ② ★★ 비밀번호를 코드에 적으면 안 되는 이유 ③ 연결이 실패하는 세 가지를 **일부러 내 보고** 구분하는 법 ④ query 가 돌려주는 것이 정확히 무엇인가 ⑤ ★★ Postgres 의 타입이 자바스크립트로 오면 무엇이 되는가 ⑥ 연결을 안 닫으면 생기는 일

```
import pg from "pg";
```

연결 설정

섹션 0

★ 표 이름·칸 이름은 한글로 쓰지만 **계정·DB 이름은 영어로** 씁니다. 도구들이 한글을 못 받는 곳이 많습니다. docker-compose.yml 도 그래서 영어입니다.

```
const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,          // ★ 5432 가 아닙니다. 이미 깔린 Postgres 와 안 부딪히게 비켜
  user: "factory",     뵈었습니다
  password: "secret",
  database: "factory_db",
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
}
```

```
process.exit(0);
}
```

★★ 08~10단원이 같은 데이터베이스를 씁니다. 단원끼리 표 이름이 부딪히지 않게 **자기 스키마**를 씁니다.

```
await 연결.query("CREATE SCHEMA IF NOT EXISTS 단원08");
await 연결.query("SET search_path TO 단원08");

console.log("연결됐습니다.");
```

출력 연결됐습니다.

```
const 서버판 = (await 연결.query("SELECT version()")).rows[0].version;

console.log("서버:", 서버판.split(" ").slice(0, 2).join(" "));
```

출력 서버: PostgreSQL 18.6
값이 다를 수 있음

★ 아무것도 안 적으면 pg 는 5432 로 붙으려 합니다. 우리 컨테이너는 5434 입니다.

```
console.log("pg 의 기본 포트:", pg.defaults.port);
```

출력 pg 의 기본 포트: 5432

세 줄이면 됩니다

섹션 1

연결하고 → 물어보고 → 답합니다. 그게 전부입니다.

```
const 연결 = new pg.Client({ ... }); await 연결.connect();
const 결과 = await 연결.query("SELECT ..."); await 연결.end();
```

★ pg.Client 는 **연결 하나**입니다. 전화선 한 가닥이라고 생각하세요. 서버가 손님을 여럿 받는다면 이걸로는 모자랍니다. 개념02 에서 풀립니다.

```
await 연결.query("DROP TABLE IF EXISTS 점검기록");
await 연결.query("DROP TABLE IF EXISTS 설비");

await 연결.query(`
  CREATE TABLE 설비 (
    설비번호 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인      TEXT NOT NULL,
    상태      TEXT NOT NULL DEFAULT '정지',
    도입가    NUMERIC(12, 2),
    누적가동  BIGINT,
```

```

)
`);

await 연결.query(
  `INSERT INTO 설비 (설비번호, 이름, 라인, 상태, 도입가, 누적가동, 설치일)
  VALUES ($1, $2, $3, $4, $5, $6, $7)`,
  ["EQ-001", "컨베이어 1호", "A", "가동", "12500000.50", "9007199254740993", "2026-03-02"],
);

await 연결.query(
  `INSERT INTO 설비 (설비번호, 이름, 라인, 상태, 도입가, 누적가동, 설치일)
  VALUES ($1, $2, $3, $4, $5, $6, $7)`,
  ["EQ-002", "프레스 1호", "B", "정지", "8300000.00", "120000", "2026-04-11"],
);

const 설비수 = (await 연결.query("SELECT count(*)::int AS 건수 FROM 설비")).rows[0].건수;

console.log("설비 건수:", 설비수);

```

출력 설비 건수: 2

★ 파라미터는 \$1, \$2 입니다. ? 가 아닙니다. ? 는 MySQL 과 SQLite 입니다. 09단원에서 비교합니다.

★★ 값을 문자열로 이어 붙이지 마세요. 그러면 뚫립니다.

```

const 나쁜입력 = "' OR '1'='1";

const 안전 = await 연결.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 이름 = $1", [나쁜입력]);
const 위험 = await 연결.query(`SELECT count(*)::int AS 건수 FROM 설비 WHERE 이름 = '${나쁜입력}'`);

console.log("파라미터로 넘기면:", 안전.rows[0].건수, "건");

```

출력 파라미터로 넘기면: 0 건

```

console.log("이어 붙이면:", 위험.rows[0].건수, "건");

```

출력 이어 붙이면: 2 건

★★★ 이어 붙인 쪽은 전부 나왔습니다. WHERE 이름 = ' ' OR '1'='1' 이 되어 조건이 통째로 무력화됐습니다. \$1 로 넘기면 값이 SQL 문장과 따로 전달되어 따옴표가 있어도 그냥 글자가 됩니다. ★ 이스케이프를 직접 하지 마세요. 뚫리는 방법이 계속 나옵니다. 드라이버에 맡기세요.

연결 문자열

섹션 2

위처럼 객체로 써도 되고, 한 줄짜리 문자열로 써도 됩니다.

postgres://사용자:비밀번호@호스트:포트/데이터베이스

클라우드 서비스들은 대개 이 한 줄을 줍니다. 그래서 이 모양을 알아야 합니다.

```
const 연결문자열 =
`postgres://${접속정보.user}:${접속정보.password}@${접속정보.host}:${접속정보.port}/${접속정보.database}`;

const 문자열연결 = new pg.Client({ connectionString: 연결문자열 });
await 문자열연결.connect();

console.log("연결 문자열로도 됩니다:", (await 문자열연결.query("SELECT 1 AS 값")).rows[0].값);
```

출력 연결 문자열로도 됩니다: 1

```
await 문자열연결.end();
```

★ 배포 환경에서는 연결 문자열(환경변수 하나로 끝납니다), 내 노트북에서는 객체를 씁니다.

★ 비밀번호에 @ 나 / 가 들어가면 연결 문자열이 깨집니다. `encodeURIComponent(비밀번호)` 로 감싸세요.

```
const 특수비번 = "a@b/c";

console.log("인코딩 전:", 특수비번, "→ 후:", encodeURIComponent(특수비번));
```

출력 인코딩 전: a@b/c → 후: a%40b%2Fc

★★ 비밀번호를 코드에 적으면 안 됩니다

섹션 3

위 코드에는 `password: "secret"` 이 그대로 적혀 있습니다. 실습이니까 그렇게 했습니다. **실무에서는 사고입니다.**

① git 에 올라갑니다. 한 번 올라가면 **지워도 히스토리에 남습니다** ② 저장소를 잠깐 공개로 바꾸면 그 순간 세상에 공개됩니다 ③ 비밀번호를 바꾸려면 코드를 고쳐서 다시 배포해야 합니다

★ 공개 저장소에 올라간 DB 비밀번호는 **몇 분 안에** 스캔됩니다. 봇이 24시간 돌립니다. 그래서 **환경변수로** 뱉습니다.

```
const 환경에서읽기 = process.env.DATABASE_URL ?? 연결문자열;

console.log("DATABASE_URL 이 설정돼 있나:", process.env.DATABASE_URL !== undefined);
```

```
출력      DATABASE_URL 이 설정돼 있나: false
```

실무 코드는 이렇게 씁니다. **없으면 아예 못 뜨게** 막는 것까지가 한 벌입니다. 조용히 기본값으로 도는 쪽이 더 위험합니다.

```
if (!process.env.DATABASE_URL) throw new Error("DATABASE_URL 이 없습니다");
const 연결 = new pg.Client({ connectionString: process.env.DATABASE_URL });
```

값은 .env 파일에 적고, Node 20.6+ 의 내장 기능으로 읽습니다. 패키지가 필요 없습니다.

.env ← ★★★ 반드시 .gitignore 에 넣으세요

```
DATABASE_URL=postgres://factory:secret@127.0.0.1:5434/factory_db
```

```
node --env-file=.env 서버.js
```

★ 팀원에게는 값을 비운 .env.example 을 줍니다. 이름만 알려 주는 것입니다.

★ 로그를 찍을 때도 조심해야 합니다. 연결 문자열을 그대로 찍으면 비밀번호가 남습니다.

```
function 비밀가리기(문자열) {
  return 문자열.replace(/:\w\/([^\:]+):([^\@]*)@/, "://$1:****@");
}

console.log("그냥 찍으면:", 연결문자열);
```

```
출력      그냥 찍으면: postgres://factory:secret@127.0.0.1:5434/factory_db
```

```
console.log("가리고 찍으면:", 비밀가리기(연결문자열));
```

```
출력      가리고 찍으면: postgres://factory:****@127.0.0.1:5434/factory_db
```

★ 에러 객체를 통째로 찍는 것도 위험합니다. console.log(에러) 가 설정 객체를 같이 뱉는 라이브러리가 있습니다. 에러.code 와 에러.message 만 찍는 습관을 들이세요.

```
console.log("환경변수에서 읽은 값이 있나:", 환경에서읽기.length > 0);
```

```
출력      환경변수에서 읽은 값이 있나: true
```

★ 연결이 실패하는 세 가지를 일부러 내 봅니다

섹션 4

처음 연결할 때 학생이 가장 많이 막히는 곳입니다. 에러 메시지를 읽을 줄 알면 대부분 1분 안에 풀립니다.


```

async function 연결해보기(설명, 바꿀것) {
  const 시험 = new pg.Client({ ...접속정보, ...바꿀것, connectionTimeoutMillis: 3000
});

  try {
    await 시험.connect();
    await 시험.end();
    return `${설명} → 성공`;
  } catch (에러) {
    try {
      await 시험.end();
    } catch {

```

이미 끊어진 것을 또 끊으려 한 것뿐입니다. 무시합니다.

```

    }

    return `${설명} → code=${에러.code} ${에러.message}`;
  }
}

for (const [설명, 바꿀것] of [
  ["포트가 틀림", { port: 5999 }],
  ["비밀번호가 틀림", { password: "틀린비번" }],
  ["없는 데이터베이스", { database: "없는디비" }],
  ["없는 호스트", { host: "없는호스트.invalid" }],
]) {
  console.log(await 연결해보기(설명, 바꿀것));
}

```

```

출력      포트가 틀림 → code=ECONNREFUSED  connect ECONNREFUSED 127.0.0.1:5999
          비밀번호가 틀림 → code=28P01  password authentication failed for user
          "factory"
          없는 데이터베이스 → code=3D000  database "없는디비" does not exist
          없는 호스트 → code=ENOTFOUND  getaddrinfo ENOTFOUND 없는호스트.invalid

```

★★ 코드가 두 종류라는 것을 눈여겨보세요. 이 구분이 진단의 절반입니다.

ECONNREFUSED / ENOTFOUND / ETIMEDOUT ← **Node** 가 낸 것. DB 까지 못 갔습니다 28P01 / 3D000 / 42601 ... ← **Postgres** 가 낸 것. DB 는 만났습니다

무엇을 보나 무슨 뜻인가 무엇을 확인하나

ECONNREFUSED	그 포트에 아무도 없음	docker compose ps / 포트 번호
ENOTFOUND	호스트 이름을 못 찾음	호스트 오타, DNS
ETIMEDOUT	대답이 없음	방화벽, 보안그룹
28P01	비밀번호가 틀림	계정·비밀번호
3D000	그 이름의 DB 가 없음	DB 이름 오타
28000	그 계정으로는 못 들어옴	pg_hba.conf

★ 없는 계정으로 붙어도 28P01 입니다. 계정이 있는지를 밖에서 못 알아내게 일부러 그렇게 만든 것입니다. 로그인 화면이 “아이디 또는 비밀번호가 틀립니다” 라고 쓰는 것과 같은 이유입니다.

query 가 돌려주는 것

섹션 5

```
const 결과 = await 연결.query("SELECT 설비번호, 이름, 라인 FROM 설비 ORDER BY 설비번호");
```

```
console.log("rowCount:", 결과.rowCount);
```

```
출력      rowCount: 2
```

```
console.log("command:", 결과.command);
```

```
출력      command: SELECT
```

```
console.log("rows:", JSON.stringify(결과.rows));
```

```
출력      rows: [{"설비번호":"EQ-001","이름":"컨베이어 1호","라인":"A"},
{"설비번호":"EQ-002","이름":"프레스 1호","라인":"B"}]
```

```
console.log("칸 이름:", 결과.fields.map((칸) => 칸.name).join(", "));
```

```
출력      칸 이름: 설비번호, 이름, 라인
```

```
const 첫칸 = 결과.fields[0];
```

```
console.log("첫 칸:", 첫칸.name, "· 타입번호", 첫칸.dataTypeID, "· 전송형식",
첫칸.format);
```

```
출력      첫 칸: 설비번호 · 타입번호 25 · 전송형식 text
```

★ rows 는 평범한 객체의 배열입니다. 그대로 res.json(결과.rows) 해도 됩니다. ★ dataTypeID 는 Postgres 의 타입 번호입니다. 25 = TEXT, 23 = INT4, 1700 = NUMERIC. 드라이버는 이 번호를 보고 무슨 자바스크립트 값으로 만들지 정합니다. 섹션 6 에서 봅니다.

SELECT 말고 다른 것들은 어떻게 오나 봅니다.

```
const 넣기 = await 연결.query("INSERT INTO 설비 (설비번호, 이름, 라인) VALUES ('EQ-003', '용접로봇 1호', 'C')");

console.log("INSERT:", 넣기.command, "/ rowCount", 넣기.rowCount, "/ rows",
JSON.stringify(넣기.rows));
```

출력 INSERT: INSERT / rowCount 1 / rows []

```
const 고치기 = await 연결.query("UPDATE 설비 SET 상태 = '점검중' WHERE 라인 = 'A'");

console.log("UPDATE:", 고치기.command, "/ rowCount", 고치기.rowCount);
```

출력 UPDATE: UPDATE / rowCount 1

```
const 지우기 = await 연결.query("DELETE FROM 설비 WHERE 설비번호 = 'EQ-999'");

console.log("아무것도 못 지웠을 때 rowCount:", 지우기.rowCount);
```

출력 아무것도 못 지웠을 때 rowCount: 0

★★ rowCount 가 0 인지 보는 것이 아주 중요합니다. “수정했습니다” 라고 응답해 놓고 실제로는 그 줄이 없어서 아무것도 안 바뀐 경우를 이걸로 잡습니다. 개념03 에서 다시 씁니다.

```
const 되돌려받기 = await 연결.query(
  "INSERT INTO 설비 (설비번호, 이름, 라인) VALUES ($1, $2, $3) RETURNING 설비번호, 상태",
  ["EQ-004", "검사기 1호", "A"],
);

console.log("RETURNING:", JSON.stringify(되돌려받기.rows));
```

출력 RETURNING: [{"설비번호":"EQ-004","상태":"정지"}]

★ RETURNING 은 Postgres 의 무기입니다. 넣으면서 자동 생성된 값을 바로 받습니다. MySQL 에는 없습니다. 09단원에서 비교합니다.

★ 칸 이름이 겹치는 JOIN 에서는 rowMode: “array” 로 배열을 받을 수도 있습니다. 연결.query({ text: “SELECT ...”, rowMode: “array” })

★★ 타입이 자바스크립트로 오면 무엇이 되는가

섹션 6

여기가 이 파일에서 가장 사고가 많이 나는 곳입니다. **재 봅시다**. 감으로 알면 안 됩니다.

```
const 한줄 = (await 연결.query("SELECT * FROM 설비 WHERE 설비번호 = 'EQ-001'")).rows[0];

for (const 칸이름 of ["설비번호", "도입가", "누적가동", "설치일"]) {
  const 값 = 한줄[칸이름];
  const 종류 = 값 instanceof Date ? "Date" : typeof 값;

  console.log(`${칸이름} → ${종류} · ${String(값 instanceof Date ? 값.toISOString() : 값)}`);
}
```

출력 설비번호 → string · EQ-001

출력 도입가 → string · 12500000.50
 값이 다를 수 있음 누적가동 → string · 9007199254740993
 설치일 → Date · 2026-03-01T15:00:00.000Z

```
console.log("도입가가 문자열인가:", typeof 한줄.도입가 === "string");
```

출력 도입가가 문자열인가: true

```
console.log("누적가동이 문자열인가:", typeof 한줄.누적가동 === "string");
```

출력 누적가동이 문자열인가: true

★★★ NUMERIC 도 BIGINT 도 문자열로 옵니다.

왜? 자바스크립트의 `number` 로는 정확히 담을 수 없기 때문입니다. 드라이버가 마음대로 `number` 로 바꾸면 값이 조용히 망가집니다. 그래서 일부러 문자열로 줍니다. “네가 알아서 정하라” 는 뜻입니다.

그대로 계산하면 이렇게 됩니다.

```
console.log("문자열 + 1 =", 한줄.도입가 + 1);
```

출력 문자열 + 1 = 12500000.501

★★★ 12500001.50 이 아니라 12500000.501 입니다. 더한 게 아니라 글자를 이어 붙였습니다. 에러도 안 납니다.

```
console.log("Number 로 바꾸면:", Number(한줄.도입가) + 1);
```

출력 Number 로 바꾸면: 12500001.5

★ 돈이라면 `Number` 도 위험합니다. 0.1 + 0.2 문제가 그대로 있습니다.

```
console.log("0.1 + 0.2 =", 0.1 + 0.2);
```

```
출력      0.1 + 0.2 = 0.30000000000000004
```

★ 돈은 **DB 안에서 NUMERIC** 으로 계산하거나 원 단위 정수로 다루세요.

```
const 합계 = (await 연결.query("SELECT sum(도입가) AS 합 FROM 설비")).rows[0].합;

console.log("DB 가 더한 합계:", 합계, "· 타입", typeof 합계);
```

```
출력      DB 가 더한 합계: 20800000.50 · 타입 string
```

★★ **BIGINT** 를 굳이 **bigint** 로 받고 싶으면 파서를 바꿀 수 있습니다. 그런데 그러면 다른 것이 터집니다.

```
pg.types.setTypeParser(20, (글자) => BigInt(글자)); // 20 = BIGINT 의 타입
번호 const 큰수 = (await 연결.query("SELECT 누적가동 FROM 설비 WHERE 설비번호 = 'EQ-001'")).rows[0].누적가동;

console.log("파서를 바꾼 뒤:", typeof 큰수, String(큰수));
```

```
출력      파서를 바꾼 뒤: bigint 9007199254740993
```

```
try {
  JSON.stringify({ 누적가동: 큰수 });
} catch (에러) {
  console.log("JSON.stringify:", 에러.constructor.name, "-", 에러.message);
}
```

```
출력      JSON.stringify: TypeError - Do not know how to serialize a BigInt
```

★★★ **res.json(rows)** 가 **500** 으로 터집니다. 조회는 됐는데 응답을 못 만듭니다. 그래서 **pg** 는 기본을 문자열로 둔 것입니다.

Number 로 바꾸는 파서는 더 위험합니다. 조용히 값이 바뀝니다.

```
pg.types.setTypeParser(20, (글자) => Number(글자));

const 숫자로 = (await 연결.query("SELECT 누적가동 FROM 설비 WHERE 설비번호 = 'EQ-001'")).rows[0].누적가동;

console.log("Number 파서:", 숫자로, "· 원래 값과 같은가:", String(숫자로) === "9007199254740993");
```

```
출력      Number 파서: 9007199254740992 · 원래 값과 같은가: false
```

★★★ 끝자리가 3 에서 2 로 바뀌었습니다. 에러도 경고도 없습니다. **Number.MAX_SAFE_INTEGER** 는 9007199254740991 입니다. 그 위는 못 셉니다.

```
pg.types.setTypeParser(20, (글자) => 글자); // 원래대로 되돌립니다
```

★★ DATE 의 시간대 함정 — 2026-03-02 로 넣었는데 위 출력은 2026-03-01T15:00:00.000Z 입니다. 하루
앞으로 밀렸습니다.

```
const 설치일 = 한줄.설치일;

console.log(
  "로컬:", `${설치일.getFullYear()}-${설치일.getMonth() + 1}-${설치일.getDate()}`,
  "· toISOString 앞 10글자:", 설치일.toISOString().slice(0, 10),
);
```

출력 로컬: 2026-3-2 · toISOString 앞 10글자: 2026-03-01
값이 다를 수 있음

```
console.log("둘이 같은 날인가:", 설치일.toISOString().slice(0, 10) === "2026-03-02");
```

출력 둘이 같은 날인가: false
값이 다를 수 있음

★★★ pg 는 DATE 를 그 기계의 로컬 자정으로 만듭니다. 한국(UTC+9)에서 2026-03-02 00:00 KST =
2026-03-01 15:00 UTC 입니다. UTC 기계에서는 안 밀립니다. 그래서 한국에서만 나는 버그가 되어 재현이
어렵습니다.

★ 해결책 두 가지

- ① SQL 에서 글자로 받기 - `to\_char(설치일, 'YYYY-MM-DD')`
- ② 파서를 글자로 바꾸기 - `pg.types.setTypeParser(1082, (글자) => 글자)`

```
const 글자날짜 = (await 연결.query(
  "SELECT to_char(설치일, 'YYYY-MM-DD') AS 날 FROM 설비 WHERE 설비번호 = 'EQ-001'",
)).rows[0].날;

console.log("SQL 에서 글자로 받으면:", 글자날짜);
```

출력 SQL 에서 글자로 받으면: 2026-03-02

★ 표로 정리합니다. (전부 이 파일에서 실제로 확인한 값입니다)

Postgres pg 가 주는 것 주의

INT / SMALLINT number 안전 BIGINT ★ string number 로 바꾸면 값이 깨질 수 있음 NUMERIC /
DECIMAL ★ string + 로 더하면 글자가 이어 붙음 REAL / DOUBLE number 0.1 + 0.2 문제 있음

BOOLEAN	boolean	안전
DATE	★ Date	로컬 자정. toISOString 하면 날짜가 밀림
TIMESTAMPZ Date		제대로 옵니다

TEXT / VARCHAR / UUID string 안전 JSONB / TEXT[] object / Array 자동으로 파싱해 줍니다
count(*) ★ string BIGINT 라서 문자열입니다

★★ count(*) 가 문자열인 것을 모르고 건수 === 0 을 쓰면 늘 false 입니다. 이 자료가 count(*)::int 라고 쓰는 이유입니다.

```
const 그냥카운트 = (await 연결.query("SELECT count(*) FROM 설비")).rows[0].count;
const 캐스팅 = (await 연결.query("SELECT count(*)::int AS 건수 FROM 설비")).rows[0].
  건수;

console.log("count(*) 그대로:", typeof 그냥카운트, ". ::int 붙이면:", typeof 캐스팅);
```

출력 count(*) 그대로: string . ::int 붙이면: number

★★ PGlite(01~06단원)는 BIGINT 를 bigint 로 주고, pg 는 string 으로 줍니다. 같은 PostgreSQL 이라도 드라이버가 다르면 오는 값이 다릅니다. 옮길 때 확인하세요.

여러 문장 한 번에

섹션 7

★ 파라미터가 없으면 query 가 세미콜론으로 이은 여러 문장을 받습니다.

```
const 여럿 = await 연결.query("SELECT 1 AS 가; SELECT 2 AS 나");

console.log("여러 문장 결과:", Array.isArray(여럿) ? `배열 ${여럿.length}개` : "객체 1개");
```

출력 여러 문장 결과: 배열 2개

★★ 그런데 파라미터를 하나라도 쓰면 거절합니다.

```
try {
  await 연결.query("SELECT $1::int AS 가; SELECT 2 AS 나", [1]);
} catch (에러) {
  console.log("파라미터 + 여러 문장:", 에러.code, "-", 에러.message);
}
```

출력 파라미터 + 여러 문장: 42601 - cannot insert multiple commands into a prepared statement

★★★ 여러 문장을 받는다는 것은 **인젝션의 피해가 커진다는** 뜻입니다. 이어 붙인 문자열에 ; DROP TABLE 설비; -- 가 들어가면 진짜로 실행됩니다. 파라미터를 쓰면 애초에 이 경로가 막힙니다.

`await 연결.end()` 를 안 하면 ① 프로그램이 안 끝나고(node 가 소켓을 붙잡습니다), ② 서버 쪽에 연결이 남아 쌓이면 `max_connections` 를 채워서 아무도 못 붙습니다. 실제로 재 봅니다.

```
const 내연결세기 = async () =>
  (await 연결.query(
    "SELECT count(*)::int AS 건수 FROM pg_stat_activity WHERE application_name = $1",
    ["unit08-demo"],
  )).rows[0].건수;

console.log("시작할 때 내 연결:", await 내연결세기());
```

출력 시작할 때 내 연결: 0

```
const 안닫은것 = [];

for (let 번째 = 0; 번째 < 3; 번째 += 1) {
  const 임시 = new pg.Client({ ...접속정보, application_name: "unit08-demo" });
  await 임시.connect();
  await 임시.query("SELECT 1");
  안닫은것.push(임시);
}

console.log("3개 열고 안 닫으면:", await 내연결세기());
```

출력 3개 열고 안 닫으면: 3

```
for (const 임시 of 안닫은것) await 임시.end();

console.log("전부 닫으면:", await 내연결세기());
```

출력 전부 닫으면: 0

★ `application_name` 은 **영어로** 쓰세요. 운영 중에 "누가 붙어 있나" 볼 때 이 칸이 생명줄인데, 한글을 넣으면 바이트로 깨져 보입니다.

```
const 한글이름 = new pg.Client({ ...접속정보, application_name: "내앱" });
await 한글이름.connect();

const 보이는이름 = (await 한글이름.query("SHOW
application_name")).rows[0].application_name;

console.log("한글 application_name 은 이렇게 보입니다:", 보이는이름);
```

출력 한글 application_name 은 이렇게 보입니다: \xeb\x82\xb4\xec\x95\xb1


```
await 한글이름.end();
```

★ 닫은 연결에 다시 물어보면 어떻게 되나

```
const 닫힌것 = new pg.Client(접속정보);
await 닫힌것.connect();
await 닫힌것.end();

try {
  await 닫힌것.query("SELECT 1");
} catch (에러) {
  console.log("닫은 뒤 query:", 에러.message);
}
```

```
출력      닫은 뒤 query: Client was closed and is not queryable
```

★★ Client 는 재사용이 안 됩니다. 한 번 닫으면 끝이고, 다시 connect() 해도 안 됩니다. 새 Client 를 만들어야 합니다. 개념02 의 풀이 이것도 대신 해 줍니다.

★★★ connect() 를 안 하고 query() 를 부르면 아무 일도 안 일어납니다. 에러도 안 나고 영원히 안 끝납니다. Node 는 종료 코드 13 으로 조용히 죽습니다. 3초만 기다려 보겠습니다.

```
const 연결안한것 = new pg.Client(접속정보);

const 삼초 = new Promise((풀기) => setTimeout(() => 풀기("시간초과"), 3000));
const 결말 = await Promise.race([연결안한것.query("SELECT 1").then(() => "응답옴"),
삼초]);

console.log("connect() 없이 query() 한 결과:", 결말);
```

```
출력      connect() 없이 query() 한 결과: 시간초과
```

```
연결안한것.end().catch(() => {}); // 붙은 적이 없으니 조용히 정리만 합니다
```

뒷정리

```
await 연결.query("DROP TABLE IF EXISTS 설비");
await 연결.end();

console.log("끝났습니다.");
```

```
출력      끝났습니다.
```

정리

무엇 쓰는 법 주의

연결 new pg.Client(설정) 연결 하나입니다 연결 문자열 postgres://user:pw@host:port/db
비밀번호에 특수문자 조심

비밀번호	process.env.DATABASE_URL	.env 를 .gitignore 에
파라미터	\$1, \$2	? 가 아닙니다
결과	rows / rowCount / fields	rows 는 평범한 객체 배열

NUMERIC · BIGINT 문자열로 옵니다 + 로 더하면 글자가 이어집니다

DATE	로컬 자정 Date	toISOString 하면 날짜가
밀립니다		
닫기	await 연결.end()	안 닫으면 안 끝납니다

★ 연결 하나로써는 손님을 여럿 못 받습니다. 개념02 에서 커넥션 풀로 풀립니다.

MySQL 은 여기가 다릅니다

· 드라이버가 mysql2 입니다 · 파라미터가 ? 입니다 · RETURNING 이 없습니다 (insertId 를 따로 받습니다) ·
결과가 { rows } 가 아니라 [rows, fields] 배열로 옵니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고
합니다

직접 해 볼 것

▫ 직접 해보기 1

섹션 4 의 연결해보기 에 { user: “없는계정” } 을 넣어 보세요. 무슨 코드가 나오나요? 왜 3D000 이
아닐까요?

▫ 직접 해보기 2

docker compose down 으로 DB 를 끄고 이 파일을 실행해 보세요. 맨 위의 안내가 제대로 나오나요? 다시
up -d 로 켜세요.

▫ 직접 해보기 3

DATABASE_URL=postgres://factory:secret@127.0.0.1:5434/factory_db node 개념01_....js 로
실행해 보세요. 섹션 3 의 출력이 어떻게 바뀌나요?

▫ 직접 해보기 4

도입가 NUMERIC 대신 DOUBLE PRECISION 으로 바꿔 보세요. 타입이 무엇으로 오나요? 12500000.50 이
그대로 나오나요?

▹ 직접 해보기 5

`pg.types.setTypeParser(1082, (글자) => 글자)` 를 섹션 6 앞에 넣어 보세요. (1082 는 DATE 의 타입 번호입니다) 설치일이 무엇으로 오나요?

▹ 직접 해보기 6

섹션 8 의 `안담은것` 반복을 3 에서 30 으로 늘려 보세요. `max_connections` 는 100 입니다. 몇 개까지 버티나요?

자주 하는 실수

이런 실수를 합니다

파라미터를 ? 로 씀

Postgres 에서 `42601` 문법 오류가 납니다. `?` 는 MySQL-SQLite 문법입니다.

이런 실수를 합니다

값을 문자열로 이어 붙임

섹션 1 에서 본 것처럼 그대로 뚫립니다. 그리고 파라미터로 넘기면 코드가 더 짧습니다. 안 쓸 이유가 없습니다.

이런 실수를 합니다

NUMERIC 을 그냥 더함

`"100" + "200"` 은 `"100200"` 입니다. 에러가 안 나서 몇 달 뒤에 발견됩니다. 돈은 DB 에서 `sum()` 으로 더하거나 원 단위 정수로 다루세요.

이런 실수를 합니다

DATE 를 `toISOString().slice(0, 10)` 으로 자름

한국에서는 하루가 밀리고 UTC 서버에서는 안 밀립니다. 재현이 어렵습니다. `to_char(칸, 'YYYY-MM-DD')` 로 글자를 받으세요.

이런 실수를 합니다

비밀번호를 코드에 적고 커밋함

git 히스토리는 지워도 남습니다. 되돌리려면 비밀번호 자체를 바꿔야 합니다. `.env` 를 만들기 전에 `.gitignore` 에 넣으세요. 순서가 중요합니다.

이런 실수를 합니다

end() 를 안 함

프로그램이 안 끝납니다. 테스트 러너에서 “왜 안 끝나지” 로 몇 시간을 씁니다.

이런 실수를 합니다

에러 객체를 통째로 로그에 찍음

설정이 같이 찍혀서 비밀번호가 로그에 남습니다. `code` 와 `message` 만 찍으세요.

커넥션 풀

```
RUN node 개념02_커넥션_풀.js
```

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 일부러 기다리는 실험이 있어서 15초쯤 걸립니다.

개념01 에서 pg.Client 하나로 데이터베이스를 불렀습니다. 혼자 쓰는 스크립트라면 그걸로 충분합니다.

그런데 서버는 손님을 **동시에** 받습니다. 김반장과 이반장이 같은 순간에 누릅니다.

이 파일에서 재 볼 것:

① 연결 하나를 계속 쓰면 요청이 **줄을 섭니다** — 얼마나 느려지나 ② 매번 새로 연결하면 **연결 만드는 값을** 냅니다 — 얼마나 비싼가 ③ ★★ 빌린 연결을 안 돌려주면 풀이 **마릅니다** — 진짜로 알려 줍니다 ④ ★★ 트랜잭션을 풀에 그냥 던지면 **롤백이 안 됩니다** — 진짜로 깨 줍니다

```
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
};

let 준비;

try {
  준비 = new pg.Client(접속정보);
  await 준비.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

await 준비.query("CREATE SCHEMA IF NOT EXISTS 단원08");
await 준비.query("SET search_path TO 단원08");
await 준비.query("DROP TABLE IF EXISTS 참고");
```

```
await 준비.query("CREATE TABLE 잔고 (계정 TEXT PRIMARY KEY, 값 INT NOT NULL)");
await 준비.query("INSERT INTO 잔고 VALUES ('A라인', 100)");

const 중앙값 = (잔것) => 잔것.slice().sort((가, 나) => 가 - 나)
[Math.floor(잔것.length / 2)];

console.log("준비됐습니다.");
```

출력 준비됐습니다.

★ 연결 하나로 여덟 명을 받으면

섹션 1

`pg.Client` 는 전화선 한 가닥입니다. 여덟 명이 동시에 걸어도 **한 명씩** 통화합니다. 나머지는 기다립니다.

재 봅시다. `pg_sleep(0.1)` 은 데이터베이스가 0.1초 동안 일하는 흉내입니다.

```
const 하나짜리 = new pg.Client({ ...접속정보, application_name: "unit08-one" });
await 하나짜리.connect();

const 하나로시작 = performance.now();
```

★ 여덟 개를 **동시에** 던집니다. 기다리지 않고 한꺼번에 보냅니다.

```
await Promise.all(Array.from({ length: 8 }, () => 하나짜리.query("SELECT
pg_sleep(0.1)"))));

const 하나로걸린ms = performance.now() - 하나로시작;

await 하나짜리.end();

console.log(`연결 1개로 0.1초짜리 8건: ${하나로걸린ms.toFixed(0)} ms`);
```

출력 연결 1개로 0.1초짜리 8건: 833 ms
값이 다를 수 있음

```
console.log("8건이 겹치지 않고 줄을 섰나 (0.7초 넘게 걸렸나):", 하나로걸린ms > 700);
```

출력 8건이 겹치지 않고 줄을 섰나 (0.7초 넘게 걸렸나): true

★★ 0.1초짜리 여덟 개인데 0.8초가 걸렸습니다. **하나도 안 겹쳤습니다.** `Promise.all` 로 동시에 보냈는데도 그렇습니다. 연결이 하나뿐이라 드라이버가 큐에 넣고 한 개씩 보냅니다.

★ 위 코드를 돌리면 터미널에 이런 경고가 뜹니다. `pg` 가 하지 말라고 말하는 것입니다. `pg 9` 부터는 아예 예러가 됩니다. 이 경고를 보면 풀을 떠올리세요.

```
DeprecationWarning: Calling client.query() when the client is already
executing a query is deprecated
```

★ 손님이 100명이면 100번째는 10초를 기다립니다. 그 사이 데이터베이스는 **놀고 있습니다**. 일을 시킬 방법이 없을 뿐입니다.

★ 그럼 매번 새로 연결하면?

섹션 2

“요청마다 새 Client 를 만들면 되잖아요” — 되기는 됩니다. 비쌉니다. 연결 하나를 만들 때 이 네 가지가 일어납니다.

① TCP 를 엽니다 ② (운영에서는) TLS 를 협상합니다 ③ 비밀번호로 인증합니다 (SCRAM-SHA-256 — 왕복이 여러 번입니다) ④ 서버가 **프로세스를 하나 새로 만듭니다** (Postgres 는 연결마다 프로세스입니다) 재 봅니다.

```
const 새로만들기ms = [];

for (let 회차 = 0; 회차 < 7; 회차 += 1) {
  const 시작 = performance.now();
  const 임시 = new pg.Client(접속정보);
  await 임시.connect();
  await 임시.query("SELECT 1");
  await 임시.end();
  새로만들기ms.push(performance.now() - 시작);
}
```

★ 풀이 만드는 연결에도 스키마를 걸어 둡니다. 풀.query(“SET search_path ...”) 로는 안 됩니다. 그 연결 하나에만 걸리기 때문입니다. 풀을 만들 때 options 로 주면 **모든 연결**에 걸립니다. (섹션 7 에서 다시 봅니다)

```
const 풀 = new pg.Pool({
  ...접속정보,
  max: 10,
  application_name: "unit08-pool",
  options: "-c search_path=단원08",
});

await 풀.query("SELECT 1"); // 첫 연결을 미리 만들어 둡니다 (워밍업) const 빌리기ms = [];

for (let 회차 = 0; 회차 < 7; 회차 += 1) {
  const 시작 = performance.now();
  const 빌린것 = await 풀.connect();
  await 빌린것.query("SELECT 1");
  빌린것.release();
  빌리기ms.push(performance.now() - 시작);
}
```

```
console.log(`새 Client 를 만들어서 한 번 물어보기: ${중양값(새로만들기ms).toFixed(1)} ms`);
```

출력 새 Client 를 만들어서 한 번 물어보기: 9.5 ms
값이 다를 수 있음

```
console.log(`풀에서 빌려서 한 번 물어보기: ${중양값(빌리기ms).toFixed(2)} ms`);
```

출력 풀에서 빌려서 한 번 물어보기: 0.55 ms
값이 다를 수 있음

```
console.log("풀이 더 빠른가:", 중양값(빌리기ms) < 중양값(새로만들기ms));
```

출력 풀이 더 빠른가: true

```
console.log(`몇 배 빠른가: ${((중양값(새로만들기ms) / 중양값(빌리기ms)).toFixed(0))} 배`);
```

출력 몇 배 빠른가: 17 배
값이 다를 수 있음

★★ 열 배가 넘게 차이 납니다. 그리고 이건 **같은 노트북 안에서** 잦 값입니다. 서버와 DB 가 다른 기계면 왕복이 여러 번이라 20~50ms 가 나오는 경우도 흔합니다. 시간만 문제가 아닙니다. 초당 100건이면 초당 100개의 프로세스를 만들었다 지웁니다.

풀은 이 둘을 동시에 해결합니다

섹션 3

풀(pool)은 **미리 만들어 둔 연결 여러 개를 돌려 쓰는 것**입니다. 빌리고 → 쓰고 → 돌려줍니다. 끊지 않습니다. 다음 요청은 **이미 만들어진 연결**을 쓰고, 연결이 여러 개라 줄서기도 사라집니다.

```
const 풀로시작 = performance.now();

await Promise.all(Array.from({ length: 8 }, () => 풀.query("SELECT pg_sleep(0.1)")));

const 풀로걸린ms = performance.now() - 풀로시작;

console.log(`풀(max 10)로 0.1초짜리 8건: ${풀로걸린ms.toFixed(0)} ms`);
```

출력 풀(max 10)로 0.1초짜리 8건: 124 ms
값이 다를 수 있음

```
console.log("풀이 연결 1개보다 빠른가:", 풀로걸린ms < 하나로걸린ms);
```

출력 풀이 연결 1개보다 빠른가: true

★★ 833ms → 124ms. 여덟 건이 **겹쳐서** 처리됐습니다. 0.1초짜리 여덟 개가 0.12초에 끝났습니다. 거의 완전히 병렬입니다.


```
console.log("풀 상태 - 만든 연결:", 풀.totalCount, "· 노는 연결:", 풀.idleCount, "· 기다리는 요청:", 풀.waitingCount);
```

```
출력      풀 상태 - 만든 연결: 8 · 노는 연결: 8 · 기다리는 요청: 0
값이 다를 수 있음
```

★ `totalCount` 가 8 입니다. 처음부터 10개를 만든 게 아니라, **필요할 때 하나씩** 만듭니다. 8건이 겹쳤으니 8개가 됐습니다. `max: 10` 은 “최대 10개까지” 라는 뜻이지 “항상 10개” 가 아닙니다.

쓰는 법 두 가지

섹션 4

① `풀.query(...)` — 한 문장이면 이게 제일 낫습니다. 빌리고, 쓰고, 돌려주는 것을 풀이 알아서 합니다.

```
const 한문장 = await 풀.query("SELECT 값 FROM 잔고 WHERE 계정 = $1", ["A라인"]);

console.log("pool.query 로 조회:", 한문장.rows[0].값);
```

```
출력      pool.query 로 조회: 100
```

② `풀.connect()` — **같은 연결**을 계속 써야 할 때만 씁니다. 트랜잭션이 대표적입니다. 임시 표나 `SET` 도 그렇습니다.

```
const 빌린연결 = await 풀.connect();

try {
  await 빌린연결.query("BEGIN");
  await 빌린연결.query("UPDATE 잔고 SET 값 = 값 + 50 WHERE 계정 = $1", ["A라인"]);
  await 빌린연결.query("COMMIT");
} catch (에러) {
  await 빌린연결.query("ROLLBACK");
  throw 에러;
} finally {
  빌린연결.release(); // ** 이 줄이 이 파일에서 제일 중요한 줄입니다
}

console.log("트랜잭션 뒤 잔고:", (await 풀.query("SELECT 값 FROM 잔고")).rows[0].값);
```

```
출력      트랜잭션 뒤 잔고: 150
```

★ `release()` 는 연결을 **끊는 게 아닙니다**. 풀에 돌려주는 것입니다. `end()` 와 헷갈리지 마세요. `client.end()` 를 부르면 풀이 망가집니다.

`release()` 를 빠뜨리면 그 연결은 **영원히 빌려 간 상태**로 남습니다. 풀이 `max` 만큼 다 빌려주고 나면, 그다음 요청은 아무도 못 받습니다.

진짜로 말려 보니다. 작은 풀(max 3)을 만들고 안 돌려주는 함수를 씁니다.

```
const 새는풀 = new pg.Pool({ ...접속정보, max: 3, connectionTimeoutMillis: 1500 });

const 새어나간것 = [];

async function 안돌려주는조회() {
  const 빌린것 = await 새는풀.connect();
  const 결과 = await 빌린것.query("SELECT 1 AS 값");

  새어나간것.push(빌린것); // 실습에서 정리하려고 보관만 해 둡니다 return
  결과.rows[0].값; // * release() 가 없습니다
}

for (let 번째 = 1; 번째 <= 3; 번째 += 1) {
  await 안돌려주는조회();
}

console.log("세 번 부른 뒤 - 만든 연결:", 새는풀.totalCount, "· 노는 연결:",
  새는풀.idleCount);
```

출력 세 번 부른 뒤 - 만든 연결: 3 · 노는 연결: 0

★ 노는 연결: 0 입니다. 세 개를 다 빌려 갔고 하나도 안 돌아왔습니다. 이제 네 번째를 부르면 어떻게 되나 보니다.

```
const 네번째시작 = performance.now();
let 네번째결과;

try {
  await 안돌려주는조회();
  네번째결과 = "성공";
} catch (에러) {
  네번째결과 = 에러.message;
}

const 기다린ms = performance.now() - 네번째시작;

console.log("네 번째 요청:", 네번째결과);
```

출력 네 번째 요청: timeout exceeded when trying to connect

```
console.log("얼마나 기다렸나 (1.5초쯤이면 맞습니다):", 기다린ms > 1400 && 기다린ms < 3000);
```

```
출력      얼마나 기다렸나 (1.5초쯤이면 맞습니다): true
```

★★★ 이게 운영에서 벌어지면 이렇게 보입니다.

· 서비스가 **멀쩡하다가 갑자기** 전부 멈춥니다 · CPU 도 메모리도 DB 도 한가합니다. 재시작하면 잠깐 괜찮았다가 또 멈춥니다 · 로그에는 “timeout exceeded when trying to connect” 만 쌓입니다

원인은 `release()` 를 빠뜨린 경로가 **하나** 있는 것입니다. ★ 특히 **에러가 난 경로**에서 잘 빠집니다. 정상 경로에는 썼는데 `throw` 뒤에는

```
안 쓴 코드가 혼합니다. 그래서 **에러가 몇 번 난 다음에** 죽습니다.
```

실습을 계속하려고 새어 나간 것들을 돌려줍니다.

```
for (const 빌린것 of 새어나간것) 빌린것.release();

console.log("돌려준 뒤 노는 연결:", 새는풀.idleCount);
```

```
출력      돌려준 뒤 노는 연결: 3
```

고친 모양입니다. **finally** 에 넣습니다.

```
async function 제대로조회() {
  const 빌린것 = await 새는풀.connect();

  try {
    return (await 빌린것.query("SELECT 1 AS 값")).rows[0].값;
  } finally {
    빌린것.release(); // 성공하든 던지든 반드시 돌아갑니다
  }
}

for (let 번째 = 1; 번째 <= 20; 번째 += 1) {
  await 제대로조회();
}

console.log("20번 불러도 - 만든 연결:", 새는풀.totalCount, "· 노는 연결:",
  새는풀.idleCount);
```

```
출력      20번 불러도 - 만든 연결: 3 · 노는 연결: 3
```

에러가 나는 경로도 새지 않는지 확인합니다.

```

async function 터지는조회() {
  const 빌린것 = await 새는풀.connect();

  try {
    await 빌린것.query("SELEC 1");
  } finally {
    빌린것.release();
  }
}

for (let 번째 = 1; 번째 <= 5; 번째 += 1) {
  await 터지는조회().catch(() => {});
}

console.log("다섯 번 터진 뒤에도 노는 연결:", 새는풀.idleCount);

```

출력 다섯 번 터진 뒤에도 노는 연결: 3

```

await 새는풀.end();

```

★★ 풀.end() 도 빌려준 연결이 있으면 안 끝납니다. 테스트가 안 끝나고 멈춰 있다면 십중팔구 release() 를 빠뜨린 것입니다.

풀 크기를 어떻게 정하나

섹션 6

★ 크게 잡으면 좋을 것 같지만 아닙니다.

먼저 데이터베이스가 받아 줄 수 있는 한계를 봅니다.

```

const 최대연결 = (await 풀.query("SHOW max_connections")).rows[0].max_connections;
const 예약분 = (await 풀.query("SHOW
superuser_reserved_connections")).rows[0].superuser_reserved_connections;

console.log("이 서버의 max_connections:", 최대연결, "· 관리자 예약:", 예약분);

```

출력 이 서버의 max_connections: 100 · 관리자 예약: 3

★★ 100 이 전부입니다. 그런데 이건 **서버 전체**의 숫자입니다. 서버 4대에 각자 max: 30 이면 120 이라 넘칩니다. 그러면 이 에러가 납니다.

```

53300 sorry, too many clients already

```

★ 계산: (서버 대수 × 풀 max) + 배치 + 마이그레이션 + 사람이 붙는 psql

이 합계가 (max_connections - superuser_reserved) 보다 작아야 합니다.

★ 그럼 얼마로 잡나 — 많이 잡는다고 빨라지지 않습니다. 연결을 100개 열어도 CPU 가 4개면 동시에 4개만 진짜로 일합니다. 나머지는 문맥 교환 비용만 늘립니다. 출발점으로 많이 쓰는 값은 이렇습니다.

풀 크기 = CPU 코어 수 × 2 + 디스크 개수 → 4코어면 10 안팎

pg.Pool 의 기본값도 10 입니다. 그 뒤에는 **재 보고** 정하세요.

★ 풀.waitingCount 가 꾸준히 0 보다 크면 풀이 작은 것입니다. 그런데 **쿼리가 느려서** 그런 경우가 더 많습니다. 그때는 06단원(색인)이 답입니다.

```
console.log("pg.Pool 의 기본 max:", new pg.Pool({ ...접속정보 }).options.max);
```

출력 pg.Pool 의 기본 max: 10

★ 서버 대수가 많아지면 애플리케이션 풀만으로 부족합니다. 그때는 DB 앞에 **PgBouncer** 같은 연결 중개기를 둡니다. 수천 개의 애플리케이션 연결을 수십 개의 실제 연결로 줄여 줍니다. 10단원(운영)에서 이름을 다시 봅니다.

★★★ 트랜잭션은 반드시 같은 연결에서

섹션 7

여기서 틀리면 **데이터가 깨집니다**. 이 파일에서 가장 중요한 섹션입니다.

이렇게 쓰고 싶어집니다. 짧고 깔끔해 보입니다.

```
await 풀.query("BEGIN");
await 풀.query("UPDATE ...");
await 풀.query("COMMIT");
```

안 됩니다. 풀.query 는 **매번 다른 연결을 쓸 수 있습니다**.

트랜잭션은 연결에 붙어 있는 상태입니다. BEGIN 을 A 연결에 보내고 UPDATE 를 B 연결에 보내면, B 는 트랜잭션 밖이라 **그 자리에서 커밋됩니다**.

진짜로 깨 봅니다.

```
await 풀.query("UPDATE 잔고 SET 값 = 100 WHERE 계정 = 'A라인'");
```

★ 두 문장을 await 없이 **동시에** 던집니다. 풀은 놓고 있는 연결이 있으면 그걸 주고, 없으면 새로 만듭니다. 그래서 이 둘은 **서로 다른 연결로** 갑니다.

```
const 시작보내기 = 풀.query("BEGIN");
const 고치기보내기 = 풀.query("UPDATE 잔고 SET 값 = 999 WHERE 계정 = 'A라인'");

await Promise.all([시작보내기, 고치기보내기]);

await 풀.query("ROLLBACK");

const 롤백뒤값 = (await 풀.query("SELECT 값 FROM 잔고 WHERE 계정 = 'A라인'")).rows[0].값;

console.log("ROLLBACK 을 했는데 잔고:", 롤백뒤값);
```

출력 ROLLBACK 을 했는데 잔고: 999

```
console.log("롤백이 됐나 (100 이어야 정상):", 롤백뒤값 === 100);
```

출력 롤백이 됐나 (100 이어야 정상): false

★★★ 롤백이 안 됐습니다. 에러도 안 났습니다. UPDATE 가 트랜잭션 밖의 다른 연결로 가서 곧바로 커밋됐고, ROLLBACK 은 또 다른 연결에서 경고 하나 남기고 끝났습니다. 07단원에서 배운 **원자성**이 통째로 사라졌습니다. 돈을 옮기는 코드라면 한쪽만 빠지고 한쪽은 안 들어옵니다.

★ 게다가 **부하가 없으면 안 납니다**. 혼자 테스트하면 늘 같은 연결이 잡혀서 잘 됩니다. 손님이 몰리는 순간에만 깨집니다. 최악의 버그입니다.

제대로 하는 법 — 연결을 하나 빌려서 **그 연결에만** 보냅니다.

```
async function 트랜잭션으로(할일) {
  const 빌린것 = await 풀.connect();

  try {
    await 빌린것.query("BEGIN");
    const 결과 = await 할일(빌린것);      // * 빌린 연결을 그대로 넘겨 줍니다 await
    빌린것.query("COMMIT");
    return 결과;
  } catch (에러) {
    await 빌린것.query("ROLLBACK");
    throw 에러;
  } finally {
    빌린것.release();
  }
}

await 풀.query("UPDATE 잔고 SET 값 = 100 WHERE 계정 = 'A라인'");
```

```
try {
  await 트랜잭션으로(async (연결) => {
    await 연결.query("UPDATE 잔고 SET 값 = 999 WHERE 계정 = 'A라인'");
    throw new Error("여기서 일이 터졌다고 침시다");
  });
} catch {
```

일부러 던진 것입니다

```
}

const 제대로롤백 = (await 풀.query("SELECT 값 FROM 잔고 WHERE 계정 =
'A라인'")).rows[0].값;

console.log("같은 연결로 하면 잔고:", 제대로롤백);
```

출력 같은 연결로 하면 잔고: 100

```
console.log("이번에는 롤백이 됐나:", 제대로롤백 === 100);
```

출력 이번에는 롤백이 됐나: true

★★ 트랜잭션으로 같은 껍데기 함수를 하나 만들어 두고 **그것만 쓰세요**. BEGIN 을 손으로 치는 곳이 코드에 남아 있으면 언젠가 위 사고가 납니다. 개념03 에서 이 함수를 저장소 계층과 어떻게 맞추는지 봅니다.

★ 같은 이유로 이런 것들도 풀.query 로 하면 안 됩니다. · SET search_path TO ... — 그 연결에만 걸립니다 · CREATE TEMP TABLE ... — 그 연결에만 보입니다 · LOCK TABLE ... — 그 연결의 트랜잭션에만 걸립니다 · SET LOCAL ... — 트랜잭션 안에서만 삽니다

★ 풀 전체에 걸고 싶은 설정은 풀을 만들 때 options 로 주세요.

```
`new pg.Pool({ ..., options: "-c search_path=단원08" })`
```

타임아웃 세 가지

섹션 8

기본값은 **전부 무제한**입니다. 그래서 아무 설정 없이 띄운 서버는 한 번 막히면 영원히 막힙니다. 세 개를 구분해서 알아 두세요.

① connectionTimeoutMillis — **풀에서 연결을 못 빌릴 때** 얼마나 기다리나 섹션 5 에서 이미 봤습니다. 이게 없으면 요청이 영원히 매달립니다.

② statement_timeout — **데이터베이스가** 쿼리를 얼마나 오래 돌리나 서버 쪽에서 잘라 줍니다. 그래서 확실합니다.

```
const 시간제한풀 = new pg.Pool({ ...접속정보, max: 2, statement_timeout: 300 });

try {
  await 시간제한풀.query("SELECT pg_sleep(2)");
} catch (에러) {
  console.log("statement_timeout:", 에러.code, "-", 에러.message);
}
```

```
출력      statement_timeout: 57014 - canceling statement due to statement timeout
```

```
console.log("풀은 살아 있나:", (await 시간제한풀.query("SELECT 1 AS 값")).rows[0].값
=== 1);
```

```
출력      풀은 살아 있나: true
```

★ 잘린 뒤에도 연결은 멀쩡합니다. 그 쿼리만 취소된 것입니다.

③ idleTimeoutMillis — **놓고 있는 연결**을 언제 정리하나 기본 10초입니다. 트래픽이 없을 때 연결을 붙잡고 있지 않게 해 줍니다.

```
const 노는것정리 = new pg.Pool({ ...접속정보, max: 2, idleTimeoutMillis: 300 });

await 노는것정리.query("SELECT 1");

console.log("쓴 직후 연결 수:", 노는것정리.totalCount);
```

```
출력      쓴 직후 연결 수: 1
```

```
await new Promise((풀기) => setTimeout(풀기, 700));

console.log("0.7초 논 뒤 연결 수:", 노는것정리.totalCount);
```

```
출력      0.7초 논 뒤 연결 수: 0
```

```
await 노는것정리.end();
await 시간제한풀.end();
```

★ 실무 출발점

```
new pg.Pool({

  connectionString: process.env.DATABASE_URL,
  max: 10,
  connectionTimeoutMillis: 5000,    // 5초 안에 연결을 못 빌리면 포기
  idleTimeoutMillis: 30000,        // 30초 놓면 정리
  statement_timeout: 10000,        // 10초 넘는 쿼리는 서버가 자름
```



```
});
```

★ 배치나 마이그레이션은 값이 다릅니다. 풀을 따로 만드세요. 웹 요청용 풀에 10분짜리 집계 쿼리를 태우면 다른 손님이 다 굶습니다.

풀도 닫아야 합니다

섹션 9

웹 서버는 켜 놓고 계속 쓰니 닫을 일이 없습니다. 스크립트나 테스트는 **반드시** 닫아야 프로그램이 끝납니다.

```
const 이벤트기록 = [];
const 구경할풀 = new pg.Pool({ ...접속정보, max: 2 });

구경할풀.on("connect", () => 이벤트기록.push("connect"));
구경할풀.on("acquire", () => 이벤트기록.push("acquire"));
구경할풀.on("release", () => 이벤트기록.push("release"));

await 구경할풀.query("SELECT 1");
await 구경할풀.query("SELECT 1");
await 구경할풀.end();

console.log("풀 이벤트 차례:", 이벤트기록.join(" → "));
```

출력 풀 이벤트 차례: connect → acquire → release → acquire → release

★ 첫 번째만 connect 가 있습니다. 두 번째는 있는 연결을 빌렸습니다. 이게 전부입니다.

★ 서버를 끝 때도 닫습니다. 안 닫으면 처리 중이던 요청이 잘립니다.

```
process.on("SIGTERM", async () => {
```

```
  await 서버닫기(); // 새 요청 그만 받기
  await 풀.end();   // 남은 쿼리 끝나면 연결 반납
  process.exit(0);
```

```
});
```

★ 풀.end() 는 빌려준 연결이 다 돌아올 때까지 기다립니다. 그래서 release() 를 빠뜨리면 여기서 영원히 멈춥니다.

뒷정리

```
await 풀.query("DROP TABLE IF EXISTS 잔고");
await 풀.end();
await 준비.end();

console.log("끝났습니다.");
```

출력 끝났습니다.

정리

무엇 언제 쓰나 주의

pg.Client 섭니다	스크립트, 마이그레이션	연결 하나. 요청이 줄을
pg.Pool	웹 서버	거의 항상 이것입니다
풀.query(...)	한 문장짜리	매번 다른 연결일 수
있습니다		

풀.connect() + release() 트랜잭션, SET, 임시 표 release 는 finally 에 풀.end() 스크립트 끝, 서버 종료
빌려준 게 있으면 안 끝납니다

설정 무엇을 재나 없으면

max	연결을 몇 개까지 만드나	기본 10
connectionTimeoutMillis	연결을 빌리는 대기 시간	영원히 기다립니다
idleTimeoutMillis	노는 연결을 언제 정리하나	기본 10초
statement_timeout	쿼리 한 개의 최대 시간	영원히 둡니다

★★★ 세 줄 요약

- ① 서버에서는 Pool 을 씁니다. Client 하나로는 손님을 못 받습니다
- ② 빌렸으면 finally 에서 돌려줍니다. 안 돌려주면 서비스가 멈춥니다
- ③ 트랜잭션은 반드시 **빌린 연결 하나**에만 보냅니다

MySQL 은 여기가 다릅니다

· mysql2/promise 의 createPool 을 씁니다 · 돌려줄 때 release() 가 아니라 connection.release() —
이름은 같습니다 · statement_timeout 이 없습니다. MAX_EXECUTION_TIME 힌트를 씁니다 ★ 자세한 비교는
09단원에서 합니다

직접 해 볼 것

▫ 직접 해보기 1

섹션 1의 8건을 40건으로 늘려 보세요. 몇 초가 걸리나요? 풀 쪽은 얼마나 걸리나요? (풀은 max 10 이하 4번에 나눠 줍니다)

▫ 직접 해보기 2

섹션 3의 풀을 max: 1로 만들어 보세요. 연결 하나짜리 Client와 시간이 같아지나요?

▫ 직접 해보기 3

섹션 5에서 `connectionTimeoutMillis`를 지워 보세요. 네 번째 요청이 어떻게 되나요? (★ Ctrl+C로 끄셔야 합니다)

▫ 직접 해보기 4

섹션 7의 깨지는 코드를 `Promise.all` 없이 `await 풀.query("BEGIN")`처럼 하나씩 기다리게 바꿔 보세요. 잔고가 100이 되나요? 그럼 이 코드는 안전한가요? (힌트: 손님이 한 명일 때만 그렇습니다)

▫ 직접 해보기 5

`statement_timeout`을 10밀리초로 줄여 보세요. 평범한 `SELECT 1`도 잘리나요?

▫ 직접 해보기 6

풀에 `풀.on("error", ...)`를 달고 `docker compose restart postgres`를 해 보세요. 어떤 에러가 오나요? (★ 이 핸들러가 없으면 서버가 통째로 죽습니다)

▫ 직접 해보기 7

30개 요청을 동시에 던지면서 `풀.waitingCount`를 찍어 보세요. 언제 0이 아니게 되나요?

자주 하는 실수

이런 실수를 합니다

요청마다 `new pg.Pool(...)`을 만들

풀은 **앱에 하나**입니다. 요청마다 만들면 연결이 폭발해 53300을 보게 됩니다. 모듈 하나에서 만들어 `export`하고 모두가 그걸 가져다 쓰세요.

이런 실수를 합니다

`release()`를 정상 경로에만 씀

`try`블록 끝에 두면 에러가 났을 때 안 돌아옵니다. 반드시 `finally`에 두세요. 이 실수 하나가 서비스를 멈춥니다.

이런 실수를 합니다

client.end() 를 부름

빌린 연결에 end() 를 부르면 풀은 그 연결이 돌아오기를 계속 기다립니다. 빌린 것에는 release(), 풀 자체에는 end() 입니다.

이런 실수를 합니다

트랜잭션을 풀.query 로 함

섹션 7 에서 본 대로 롤백이 안 됩니다. 그리고 **부하가 있을 때만** 깨집니다. 테스트에서는 절대 안 잡힙니다.

이런 실수를 합니다

풀 크기를 크게 잡으면 빨라진다고 생각함

DB 의 CPU 는 그대로입니다. 100 으로 잡으면 문맥 교환만 늘고 더 느려집니다. 그리고 서버를 여러 대 띄우면 max_connections 를 넘겨 버립니다.

이런 실수를 합니다

풀.on("error", ...) 를 안 달아 둬

놓고 있는 연결이 끊기면 (DB 재시작 등) 풀이 error 를 냅니다. 핸들러가 없으면 Node 의 규칙에 따라 프로세스가 통째로 죽습니다. 한 줄이면 됩니다. 풀.on("error", (에러) => console.error(에러.message));

이런 실수를 합니다

타임아웃을 하나도 안 걸어 둬

기본값이 전부 무제한입니다. 느린 쿼리 하나가 풀을 다 채우면 그 순간부터 아무 요청도 못 받습니다. 최소한 statement_timeout 은 거세요.

저장소 계층으로 묶기

RUN node 개념03_저장소_계층으로_묶기.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

개념02 까지로 데이터베이스를 부를 수는 있게 됐습니다. 이제 **어디에 그 코드를 두느냐** 의 문제가 남습니다.

처음에는 다들 라우트 안에 SQL 을 그냥 씁니다. 잘 돌아갑니다. 파일이 열 개쯤 되면 무너지기 시작합니다.

이 파일에서 할 것:

① SQL 을 여기저기 흩어 놓으면 무슨 일이 생이나 ② 저장소(repository) 로 묶기 — 목록·하나·추가·수정·삭제 ③ ★ 저장소는 **DB 를 모르는 모양**으로 돌려줘야 합니다 ④ ★★ 저장소 함수 여럿을 **한 트랜잭션**으로 묶는 법 ⑤ 에러를 도메인 말로 바꿔서 올리기 — 23505 를 “이미 있는 설비번호입니다” 로 ⑥ 트랜잭션으로 감싸고 롤백하는 **테스트 기법**

```
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
};

let 풀;

try {
  풀 = new pg.Pool({ ...접속정보, max: 5, options: "-c search_path=단원08" });
  await 풀.query("SELECT 1");
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}
```

★ 스키마는 풀 밖에서 한 번 만들어야 합니다. options 로 준 search_path 는 스키마가 이미 있어야 통합니다.

```
const 초기화 = new pg.Client(접속정보);
```

```

await 초기화.connect();
await 초기화.query("CREATE SCHEMA IF NOT EXISTS 단위08");
await 초기화.query("SET search_path TO 단위08");
await 초기화.query("DROP TABLE IF EXISTS 점검기록");
await 초기화.query("DROP TABLE IF EXISTS 설비");
await 초기화.query(`
    CREATE TABLE 설비 (
        설비번호 TEXT PRIMARY KEY,
        이름      TEXT NOT NULL,
        라인      TEXT NOT NULL,
        상태      TEXT NOT NULL DEFAULT '정지',
        등록시각 TIMESTAMPTZ NOT NULL DEFAULT now()
    )
`);
await 초기화.query(`
    CREATE TABLE 점검기록 (
        번호      SERIAL PRIMARY KEY,
        설비번호 TEXT NOT NULL REFERENCES 설비(설비번호) ON DELETE CASCADE,
        결과      TEXT NOT NULL,
        점검시각 TIMESTAMPTZ NOT NULL DEFAULT now()
    )
`);
await 초기화.end();

console.log("표를 만들었습니다.");

```

출력 표를 만들었습니다.

SQL 을 흠어 놓으면

섹션 1

처음에는 이렇게 씁니다. 나쁜 코드처럼 보이지도 않습니다.

```

app.get("/equipments", async (req, res) => {

    const 결과 = await 풀.query("SELECT * FROM 설비 WHERE 라인 = $1", [req.query.line]);
    res.json(결과.rows);

});

```

그리고 여섯 달이 지나면 이렇게 됩니다.

· `SELECT * FROM 설비` 가 코드 곳곳에 **열일곱 군데** 있습니다 · 칸 하나를 지우고 싶은데 어디가 깨질지 아무도 모릅니다 · `상태` 값이 하나 늘었는데 검사하는 곳이 제각각입니다 · 테스트를 쓰려면 매번 진짜 DB 를 띄워야 합니다 · `SELECT *` 라서 칸을 추가했더니 비밀번호 해시가 API 응답에 딸려 나갔습니다

★★ 마지막 것은 진짜로 자주 일어납니다. `SELECT *` 는 **오늘의 칸**이 아니라 **미래의 칸**까지 가져옵니다.

해결은 간단합니다. **설비 표를 다루는 코드를 한 파일에 모읍니다.** 그 파일을 저장소(repository)라고 부릅니다.

설비 저장소

섹션 2

★★ 첫 번째 인자를 **실행자**로 받는 것을 눈여겨보세요. 풀 이 될 수도 있고, 트랜잭션 중인 **빌린 연결**이 될 수도 있습니다. 이 한 가지 결정이 ④(트랜잭션 묶기)를 통째로 풀어 줍니다. 지금은 그냥 "물어볼 상대"라고 생각하세요.

DB 의 한 줄을 우리 앱이 쓰는 모양으로 바꿉니다.

```
function 설비로만들기(줄) {
  return {
    번호: 줄.설비번호,
    이름: 줄.이름,
    라인: 줄.라인,
    가동중: 줄.상태 === "가동",
  };
}

const 설비저장소 = {
  async 목록(실행자, { 라인 } = {}) {
    const 결과 = 라인
      ? await 실행자.query(
        "SELECT 설비번호, 이름, 라인, 상태 FROM 설비 WHERE 라인 = $1 ORDER BY 설비번호",
        [라인],
      )
      : await 실행자.query("SELECT 설비번호, 이름, 라인, 상태 FROM 설비 ORDER BY 설비번호");

    return 결과.rows.map(설비로만들기);
  },

  async 하나(실행자, 번호) {
    const 결과 = await 실행자.query(
      "SELECT 설비번호, 이름, 라인, 상태 FROM 설비 WHERE 설비번호 = $1",
      [번호],
    );
  },
};
```

★ 못 찾으면 null 입니다. undefined 가 아닙니다. (섹션 3 에서 이유를 봅니다)

```
return 결과.rows.length === 0 ? null : 설비로만들기(결과.rows[0]);
},

async 추가(실행자, { 번호, 이름, 라인 }) {
```

마이그레이션

RUN node 개념04_마이그레이션.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

지금까지 표를 만들 때 이렇게 했습니다.

```
await 연결.query("DROP TABLE IF EXISTS 설비");
await 연결.query("CREATE TABLE 설비 (...);");
```

실습이니깐 됩니다. 운영에서는 DROP TABLE 이 곧 사고입니다.

진짜 서비스에서는 **표가 이미 있고, 그 안에 자료가 들어 있는 채로** 칸을 하나 더 붙이거나 색인을 하나 더 만들어야 합니다. 그 일을 안전하게 하는 방법이 마이그레이션(migration)입니다.

이 파일에서 할 것:

- ① 스키마를 손으로 고치면 무슨 일이 나나 ② 마이그레이션 도구를 **직접 만듭니다** (100줄이면 됩니다) ③
- ★ Postgres 는 CREATE TABLE 도 롤백됩니다 — 실제로 확인합니다 ④ 되돌리기(down)를 만들 것인가 ⑤
- ★★ 배포 중에 옛 코드와 새 코드가 같이 둡니다 — 그때 안전한 순서

```
import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
}
```



```
process.exit(0);
}
```

★ 마이그레이션은 **연결 하나**로 합니다. 풀을 쓰지 않습니다. 순서가 중요하고, 트랜잭션과 잠금을 써야 하기 때문입니다. (개념02 섹션 7)

```
await 연결.query("DROP SCHEMA IF EXISTS 단위08_이사 CASCADE");
await 연결.query("CREATE SCHEMA 단위08_이사");
await 연결.query("SET search_path TO 단위08_이사");

console.log("연습용 스키마를 만들었습니다.");
```

출력 연습용 스키마를 만들었습니다.

손으로 고치면 왜 안 되나

섹션 1

급해서 psql 을 열고 이렇게 칩니다.

```
ALTER TABLE 설비 ADD COLUMN 담당자 TEXT;
```

그날은 잘 돌아갑니다. 다음 주에 이런 일이 벌어집니다.

· 새로 들어온 사람이 저장소를 받아 띄웠는데 **담당자** 칸이 없어서 터집니다 · 스테이징 서버에는 있고 운영 서버에는 없습니다. 누가 언제 쳤는지 모릅니다 · 되돌리려는데 뭘 쳤는지 기억이 안 납니다 · 두 사람이 각자 다른 이름의 칸을 만들었습니다 (담당자 / 관리자)

★★ 핵심은 이것입니다. **스키마는 코드입니다**. 코드는 저장소에 넣고, 리뷰하고, 순서대로 적용합니다. 손으로 친 것은 코드가 아닙니다. 기록이 없기 때문입니다.

그래서 두 가지를 만듭니다.

① 번호가 붙은 변경 파일들 — 001_설비표.sql, 002_점검기록표.sql, ... ② 어디까지 적용했는지 적는 표 — **_마이그레이션**

이게 전부입니다. 유명한 도구들도 이 두 가지를 합니다.

변경 파일을 만듭니다

섹션 2

진짜 프로젝트라면 **migrations/** 폴더에 손으로 만들어 둡니다. 여기서는 실습이라 임시 폴더에 코드로 만들겠습니다.

```
const 이사폴더 = fs.mkdtempSync(path.join(os.tmpdir(), "db자료-08-"));

const 변경파일들 = {
  "001_설비표.sql": `
```

```

설비번호 TEXT PRIMARY KEY,
    이름    TEXT NOT NULL,
    라인    TEXT NOT NULL
);
`,
"002_점검기록표.sql": `
CREATE TABLE 점검기록 (
    번호      SERIAL PRIMARY KEY,
    설비번호 TEXT NOT NULL REFERENCES 설비(설비번호),
    결과      TEXT NOT NULL,
    점검시각  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX 점검기록_설비번호_색인 ON 점검기록 (설비번호);
`,
"003_설비에_상태칸.sql": `
ALTER TABLE 설비 ADD COLUMN 상태 TEXT NOT NULL DEFAULT '정지';
`,
};

for (const [이름, 내용] of Object.entries(변경파일들)) {
    fs.writeFileSync(path.join(이사폴더, 이름), 내용.trim() + "\n");
}

console.log("변경 파일:", fs.readdirSync(이사폴더).sort().join(", "));

```

출력 변경 파일: 001_설비표.sql, 002_점검기록표.sql, 003_설비에_상태칸.sql

★ 이름 짓는 규칙 · 앞에 **자리를 맞춘 번호**를 붙입니다. 1_ 이 아니라 001_ 입니다

그래야 글자 순서 = 적용 순서가 됩니다. (`10\_` 이 `2\_` 보다 앞에 오는 사고를 막습니다)

· 뒤에는 **무엇을 하는지** 적습니다. 003_수정.sql 은 아무 정보가 없습니다 · 번호 대신 20260826103000... 처럼 시각을 쓰는 팀도 많습니다

여러 명이 동시에 만들 때 번호가 겹치지 않아서 그렇게 합니다

★★★ 한 번 적용한 파일은 절대 고치지 않습니다. 내 노트북에는 고친 게 적용되고, 운영에는 옛것이 적용된 채로 남습니다. 고칠 일이 있으면 **새 번호로** 파일을 하나 더 만듭니다.

★★ 마이그레이션 도구를 직접 만듭니다

섹션 3

하는 일은 네 줄로 요약됩니다.

① 적용 기록 표가 없으면 만든다 ② 폴더의 .sql 파일을 이름 순으로 읽는다 ③ 이미 적용한 것은 건너뛰다 ④ 나머지를 **하나씩 트랜잭션 안에서** 적용하고 기록을 남긴다

```
const 기록표 = "_마이그레이션";

async function 마이그레이션돌리기(연결, 폴더, { 조용히 = false } = {}) {
```

① 기록 표

```
await 연결.query(`
  CREATE TABLE IF NOT EXISTS ${기록표} (
    이름      TEXT PRIMARY KEY,
    적용시각  TIMESTAMPTZ NOT NULL DEFAULT now()
  )
`);
```

★★ 서버가 여러 대면 배포할 때 **동시에** 이 코드가 돌립니다. 둘 다 같은 파일을 적용하려 들면 표가 두 번 만들어지거나 에러가 납니다. Postgres 의 권고 잠금(advisory lock)으로 한 명씩만 들어오게 막습니다. 숫자는 아무거나 정하면 됩니다. 우리 팀 안에서만 안 겹치면 됩니다.

```
await 연결.query("SELECT pg_advisory_lock($1)", [880801]);

try {
```

② 이미 적용한 것

```
const 적용됨 = new Set(
  (await 연결.query(`SELECT 이름 FROM ${기록표}`)).rows.map((줄) => 줄.이름),
);
```

③ 폴더의 파일 목록 (이름 순)

```
const 파일들 = fs.readdirSync(폴더).filter((이름) => 이름.endsWith(".sql")).sort();

const 한것 = [];

for (const 이름 of 파일들) {
  if (적용됨.has(이름)) {
    if (!조용히) console.log(` 건너뛴 ${이름}`);
    continue;
  }

  const 내용 = fs.readFileSync(path.join(폴더, 이름), "utf8");
```

④ ★★ 한 파일 = 한 트랜잭션. 중간에 실패하면 그 파일이 한 일이 **통째로** 없던 일이 됩니다.

```

await 연결.query("BEGIN");

    try {
        await 연결.query(내용);
        await 연결.query(`INSERT INTO ${기록표} (이름) VALUES ($1)`, [이름]);
        await 연결.query("COMMIT");
    } catch (에러) {
        await 연결.query("ROLLBACK");
    }
}

```

★ 어느 파일에서 터졌는지 반드시 알려 줘야 합니다. 원래 코드도 같이 달아 둡니다. 부르는 쪽이 판단할 수 있게요.

```

throw Object.assign(new Error(`${이름} 적용 실패 - ${에러.message}`), {
    code: 에러.code,
    파일: 이름,
});
}

한것.push(이름);
if (!조용히) console.log(` 적용함 ${이름}`);
}

return 한것;
} finally {
    await 연결.query("SELECT pg_advisory_unlock($1)", [880801]);
}
}

console.log("— 처음 돌립니다 —");

```

출력 — 처음 돌립니다 —

```
const 첫번째 = await 마이그레이션돌리기(연결, 이사폴더);
```

출력 적용함 001_설비표.sql
 적용함 002_점검기록표.sql
 적용함 003_설비에_상태칸.sql

```
console.log("적용한 개수:", 첫번째.length);
```

출력 적용한 개수: 3

```
const 칸들 = await 연결.query(
  `SELECT column_name FROM information_schema.columns
   WHERE table_schema = '단원08_이사' AND table_name = '설비'
   ORDER BY ordinal_position`,
);

console.log("설비 표의 칸:", 칸들.rows.map((줄) => 줄.column_name).join(", "));
```

출력 설비 표의 칸: 설비번호, 이름, 라인, 상태

★ query 에 파라미터가 없으면 세미콜론으로 이은 여러 문장을 받습니다. 그래서 002_점검기록표.sql 의 CREATE TABLE + CREATE INDEX 가 한 번에 됐습니다. (개념01 섹션 7 에서 확인한 것입니다)

두 번 돌려도 안전해야 합니다

```
console.log("— 다시 돌립니다 —");
```

출력 — 다시 돌립니다 —

```
const 두번째 = await 마이그레이션돌리기(연결, 이사폴더);
```

출력 건너뛴 001_설비표.sql
 건너뛴 002_점검기록표.sql
 건너뛴 003_설비에_상태칸.sql

```
console.log("이번에 적용한 개수:", 두번째.length);
```

출력 이번에 적용한 개수: 0

★★ 이걸 **멥등(idempotent)** 이라고 합니다. 몇 번 돌려도 결과가 같습니다. 배포 스크립트가 매번 이걸 부르게 해도 안전하다는 뜻입니다. 실제로 많은 팀이 서버가 뜰 때마다 자동으로 부릅니다.

새 파일을 하나 더 얹어 봅니다. 실무의 흐름 그대로입니다.

```
fs.writeFileSync(
  path.join(이사폴더, "004_설비에_담당자칸.sql"),
  "ALTER TABLE 설비 ADD COLUMN 담당자 TEXT;\n",
);

console.log("— 새 파일을 하나 얹고 돌립니다 —");
```

출력 — 새 파일을 하나 얹고 돌립니다 —

```
const 세번째 = await 마이그레이션돌리기(연결, 이사폴더, { 조용히: true });

console.log("이번에 적용한 것:", 세번째.join(", "));
```

출력 이번에 적용한 것: 004_설비에_담당자칸.sql

```
const 기록 = await 연결.query(`SELECT 이름 FROM ${기록표} ORDER BY 이름`);

console.log("기록 표의 내용:", 기록.rows.map((줄) => 줄.이름).join(" · "));
```

출력 기록 표의 내용: 001_설비표.sql · 002_점검기록표.sql · 003_설비에_상태칸.sql · 004_설비에_담당자칸.sql

★ 이게 마이그레이션 도구의 전부입니다. 100줄이 안 됩니다.

★ 진짜 도구들이 더 하는 것 · 파일 내용의 해시를 같이 저장해서 **이미 적용한 파일이 바뀌면** 경고합니다 · [down](#) 을 지원합니다 (섹션 5) · 잠금이 안 풀린 채 죽었을 때 손으로 풀 수 있게 해 줍니다 · 아직 안 적용된 것이 있으면 목록만 보여 주는 `status` 명령이 있습니다

★★ 실패하면 어떻게 되나

섹션 4

일부러 틀린 파일을 하나 넣어 봅니다.

```
fs.writeFileSync(
  path.join(이사폴더, "005_틀린것.sql"),
  "CREATE TABLE 새표 (번호 INT);\nALTER TABLE 없는표 ADD COLUMN 뭔가 TEXT;\n",
);

const 실패 = await 마이그레이션돌리기(연결, 이사폴더, { 조용히: true })
  .then(() => null)
  .catch((에러) => 에러);

console.log("어느 파일에서:", 실패.파일, ". 코드:", 실패.code);
```

출력 어느 파일에서: 005_틀린것.sql · 코드: 42P01

```
const 새표있나 = (await 연결.query("SELECT to_regclass('단원08_이사.새표') AS 있나")).rows[0].있나;

console.log("앞부분의 CREATE TABLE 이 남았나:", 새표있나 !== null);
```

출력 앞부분의 CREATE TABLE 이 남았나: false

```
const 기록에있나 = (await 연결.query(
  `SELECT count(*)::int AS 건수 FROM ${기록표} WHERE 이름 = '005_틀린것.sql'`,
)).rows[0].건수;

console.log("기록 표에 남았나:", 기록에있나 === 1);
```

```
출력      기록 표에 남았나: false
```

★★★ 여기가 PostgreSQL 의 큰 장점입니다.

005_틀린것.sql 은 두 문장입니다. 첫 문장(CREATE TABLE 새표)은 **성공했습니다**. 두 번째에서 터졌고, ROLLBACK 이 첫 문장까지 되돌렸습니다.

그래서 **반쯤 적용된 상태가 없습니다**. 파일을 고치고 다시 돌리면 됩니다.

좀 더 또렷하게 확인해 봅시다. 표를 만들고 롤백합니다.

```
await 연결.query("BEGIN");
await 연결.query("CREATE TABLE 사라질표 (번호 INT)");

const 안에서 = (await 연결.query("SELECT to_regclass('단원08_이사.사라질표') AS
있나")).rows[0].있나;

await 연결.query("ROLLBACK");

const 밖에서 = (await 연결.query("SELECT to_regclass('단원08_이사.사라질표') AS
있나")).rows[0].있나;

console.log("트랜잭션 안에서 표가 보이냐:", 안에서 !== null);
```

```
출력      트랜잭션 안에서 표가 보이냐: true
```

```
console.log("롤백한 뒤에도 남아 있냐:", 밖에서 !== null);
```

```
출력      롤백한 뒤에도 남아 있냐: false
```

MySQL 은 여기가 다릅니다 ———

★★ MySQL 은 CREATE TABLE 같은 스키마 변경(DDL)을 만나면

```
**그 자리에서 자동으로 커밋합니다.** `ROLLBACK` 을 해도 표가 남습니다.
```

그래서 MySQL 에서는 마이그레이션이 중간에 실패하면 **반쯤 적용된 상태**가 그대로 남습니다. 손으로 치워야 합니다. 그래서 MySQL 쪽 도구들은 파일 하나에 한 문장만 넣기를 권합니다.

★ 09단원에서 둘 다 띄워 놓고 직접 확인합니다.

실패한 파일은 치우고 이어 갑니다.

```
fs.rmSync(path.join(이사폴더, "005_틀린것.sql"));
```

되돌리기(down)를 만들 것인가

섹션 5

많은 도구가 파일 하나에 up 과 down 을 같이 쓰게 해 줍니다.

-- up

```
ALTER TABLE 설비 ADD COLUMN 담당자 TEXT;
```

-- down

```
ALTER TABLE 설비 DROP COLUMN 담당자;
```

★ 실무에서는 down 을 거의 안 씁니다. 이유가 분명합니다.

① **자료가 사라집니다.** DROP COLUMN 의 down 은 칸을 되살릴 수 있어도

그 안에 있던 값은 못 되살립니다. 되돌린 게 아닙니다

② 배포는 이미 나갔습니다. 새 코드가 그 칸을 쓰고 있는데 칸을 지우면 서비스가 죽습니다 ③ down 은 테스트가 안 됩니다. 아무도 안 돌려 보는 코드가 저장소에 쌓입니다 ④ 진짜 사고가 났을 때는 down 보다 백업 복구가 빠르고 확실합니다

★★ 그래서 실무의 규칙은 이렇습니다.

앞으로만 갑니다. 잘못 했으면 되돌리는 게 아니라 고치는 새 마이그레이션을 하나 더 만들어서 앞으로 갑니다.

```
005_담당자칸_잘못됨.sql  ← 이미 나감
006_담당자칸_고침.sql   ← 되돌리지 말고 이걸 만듭니다
```

★ down 이 쓸모 있는 곳도 있습니다. **내 노트북입니다.** 개발하다 방금 만든 마이그레이션을 고치고 싶을 때는 편합니다. 운영에 쓰라고 만든 게 아니라고 생각하면 됩니다.

★★★ 위험한 변경 — 배포 중에는 옛 코드도 둡니다

섹션 6

이게 이 파일에서 제일 중요한 부분입니다.

서버를 새 버전으로 바꾸는 동안, 잠깐이지만 **두 버전이 같이 둡니다.**

시각 서버1 서버2 데이터베이스

10:00	옛 코드	옛 코드	옛 스키마
10:01	옛 코드	옛 코드	★ 새 스키마 (마이그레이션 적용)
10:02	새 코드	옛 코드	새 스키마 ← 여기가 위험합니다
10:03	새 코드	새 코드	새 스키마

10:02 에 **옛 코드가 새 스키마 위에서** 돕니다. 이때 깨지지 않아야 무중단 배포가 됩니다.

위험한 세 가지를 봅니다.

① 칸 이름 바꾸기 — `ALTER TABLE 설비 RENAME COLUMN 이름 TO 설비명`

옛 코드는 **이름** 을 SELECT 합니다. 바꾸는 순간 **전부 42703 으로** 터집니다.

★★ 안전한 순서: **확장 → 이전 → 축소** (expand / migrate / contract)

1단계 (확장)	새 칸 `설비명` 을 **추가만** 합니다. 옛 칸은 그대로 돕니다 → 옛 코드도 새 코드도 다 돕니다
2단계 (이전)	값을 복사합니다. 새 코드는 **양쪽에 다 씁니다** → 어느 쪽을 읽어도 맞습니다
3단계 (축소)	모든 서버가 새 코드가 된 뒤에 옛 칸을 지웁니다 → 다음 배포(며칠 뒤)에 합니다

배포가 세 번 필요합니다. 귀찮습니다. 그런데 이게 유일하게 안 죽는 방법입니다.

<pre>await 연결.query("ALTER TABLE 설비 ADD COLUMN 설비명 TEXT"); await 연결.query("UPDATE 설비 SET 설비명 = 이름"); const 두칸 = await 연결.query(`SELECT column_name FROM information_schema.columns    WHERE table_schema = '단원08_이사' AND table_name = '설비' AND column_name IN    ('이름', '설비명')    ORDER BY column_name`,); console.log("확장 단계 - 두 칸이 같이 있나:", 두칸.rows.map((줄) => 줄.column_name).join(", "));</pre>	
출력	확장 단계 - 두 칸이 같이 있나: 설비명, 이름

② 칸 지우기 — `ALTER TABLE 설비 DROP COLUMN 담당자`

옛 코드가 `INSERT INTO 설비 (설비번호, 이름, 라인, 담당자) ...` 를 하고 있으면 그 순간 전부 터집니다. 그리고 **되돌릴 수 없습니다**. 값이 사라졌습니다.

★ 순서: 새 코드에서 그 칸을 **안 쓰게 먼저 배포**하고,

며칠 지켜본 뒤에 지웁니다. 급할 이유가 없습니다.

③ NOT NULL 추가 — ALTER TABLE 설비 ALTER COLUMN 담당자 SET NOT NULL

두 가지가 겹칩니다.

· 이미 NULL 인 줄이 있으면 그 자리에서 실패합니다 · 옛 코드가 그 칸 없이 INSERT 하면 터집니다

```
await 연결.query("INSERT INTO 설비 (설비번호, 이름, 라인) VALUES ('EQ-001', '컨베이어 1호', 'A')");
```

```
const 낮널실패 = await 연결
  .query("ALTER TABLE 설비 ALTER COLUMN 담당자 SET NOT NULL")
  .then(() => "성공")
  .catch((에러) => 에러.code);
```

```
console.log("NULL 이 있는 채로 NOT NULL 걸기:", 낮널실패);
```

출력 NULL 이 있는 채로 NOT NULL 걸기: 23502

★★ 안전한 순서 (역시 확장 → 이전 → 축소)

1단계 기본값을 주면서 칸을 추가합니다 - `ADD COLUMN 담당자 TEXT DEFAULT '미지정'`
2단계 이미 있는 NULL 을 채웁니다 - `UPDATE 설비 SET 담당자 = '미지정' WHERE 담당자 IS NULL`
3단계 새 코드가 항상 값을 넣게 배포합니다
4단계 그다음에 NOT NULL 을 겁니다

```
await 연결.query("UPDATE 설비 SET 담당자 = '미지정' WHERE 담당자 IS NULL");
await 연결.query("ALTER TABLE 설비 ALTER COLUMN 담당자 SET NOT NULL");
```

```
const 낮널됐나 = (await 연결.query(
  `SELECT is_nullable FROM information_schema.columns
  WHERE table_schema = '단원08_이사' AND table_name = '설비' AND column_name =
  '담당자'`,
)).rows[0].is_nullable;
```

```
console.log("채운 뒤 NOT NULL 걸기:", 낮널됐나 === "NO" ? "성공" : "실패");
```

출력 채운 뒤 NOT NULL 걸기: 성공

★★ 또 하나 — 잠금(lock) 을 조심하세요.

ALTER TABLE ... ADD COLUMN 은 그 표에 **ACCESS EXCLUSIVE** 잠금을 겁니다. 그동안은 그 표를 아무도 못 읽고 못 씁니다.

· 기본값이 없는 칸 추가 → Postgres 11+ 에서는 빠릅니다 (메타데이터만 고침) · 기본값이 있는 칸 추가 → Postgres 11+ 도 빠릅니다 · 칸의 타입 바꾸기 → ★ 표 전체를 다시 씁니다. 큰 표면 몇 분씩 멈춥니다 · 색인 만들기 → ★ `CREATE INDEX CONCURRENTLY` 를 쓰세요. 안 멈춥니다

★★★ 그런데 `CREATE INDEX CONCURRENTLY` 는 트랜잭션 안에서 못 씁니다.

우리 도구는 파일마다 `BEGIN` 을 하므로 이 파일만 예외로 빼야 합니다.
진짜 도구들에도 "이 마이그레이션은 트랜잭션 없이" 표시가 있습니다.

```
const 동시색인 = await 연결
  .query("BEGIN; CREATE INDEX CONCURRENTLY 시험색인 ON 설비 (라인); COMMIT;")
  .then(() => "성공")
  .catch((에러) => `${에러.code} ${에러.message}`);

console.log("트랜잭션 안에서 CONCURRENTLY:", 동시색인);
```

출력 트랜잭션 안에서 CONCURRENTLY: 25001 CREATE INDEX CONCURRENTLY cannot run inside a transaction block

```
await 연결.query("ROLLBACK").catch(() => {});
```

★ 잠금이 오래 걸릴까 걱정되면 `lock_timeout` 을 걸어 두세요. 못 잡으면 그냥 실패합니다. 서비스를 몇 분씩 멈추는 것보다 낫습니다.

```
SET lock_timeout = '3s';
ALTER TABLE ...;
```

도구를 쓸 것인가

섹션 7

직접 만든 것으로도 충분히 돌아갑니다. 실제로 그렇게 쓰는 팀도 많습니다. 이미 만들어진 것을 쓰고 싶으면 이런 것들이 있습니다.

이름 무엇 특징

node-pg-migrate	Node · Postgres 전용	JS 로도 SQL 로도 씁니다
Umzug	Node · DB 안 가림	Sequelize 계열에서 많이 씁니다
Flyway	Java · 여러 DB SQL 파일 그대로.	오래됐고 튼튼합니다
Liquibase	Java · 여러 DB XML/YAML 로도 씁니다	

Prisma Migrate Prisma 를 쓸 때 스키마 파일에서 자동 생성 (개념05) Drizzle Kit Drizzle 을 쓸 때 스키마 파일에서 자동 생성 (개념05)

★ 이 자료에서는 **설치하지 않습니다**. 원리를 알면 어느 것을 써도 같습니다. 무엇을 고르든 확인할 것은 두 가지입니다.

- ① 적용 기록을 ****DB 안에**** 두는가 (파일에 두면 서버마다 달라집니다)
- ② 한 파일을 트랜잭션으로 감싸는가

뒷정리

```
fs.rmSync(이사폴더, { recursive: true, force: true });
await 연결.query("DROP SCHEMA IF EXISTS 단원08_이사 CASCADE");
await 연결.end();

console.log("끝났습니다.");
```

출력 끝났습니다.

정리

마이그레이션이 하는 일 어떻게

변경을 기록으로 남긴다 번호 붙은 .sql 파일을 저장소에 넣습니다 어디까지 했는지 안다 **_마이그레이션** 표에 이름을 적습니다 여러 번 돌려도 안전하다 이미 적용한 것은 건너뛴다 실패해도 안 깨진다 파일 하나를 트랜잭션으로 감쌌다 동시에 돌아도 안전하다 pg_advisory_lock 으로 한 명씩

위험한 변경 왜 위험한가 안전한 순서

칸 이름 바꾸기 옛 코드가 옛 이름을 찾습니다 확장 → 이전 → 축소 칸 지우기 옛 코드가 그 칸에 씁니다 안 쓰게 만들고 며칠 뒤에 NOT NULL 추가 NULL 이 있으면 실패합니다 기본값 → 채우기 → 걸기 타입 바꾸기 표 전체를 다시 씁니다 작은 표가 아니면 새 칸으로 색인 만들기 표가 잠깁니다 CONCURRENTLY (트랜잭션 밖에서)

★★★ 세 줄 요약

- ① 스키마는 코드입니다. 손으로 고치지 마세요
- ② 한 번 나간 파일은 고치지 말고 새 번호로 하나 더 만드세요
- ③ 배포 중에는 옛 코드와 새 코드가 같이 둡니다. 그 사이에 안 깨지게 나눠서 하세요

직접 해 볼 것

▹ 직접 해보기 1

005_틀린것.sql 의 두 문장 순서를 바꿔 보세요. (틀린 문장을 앞에 두면) 새표가 만들어지나요? 왜 그럴까요?

▹ 직접 해보기 2

파일 이름을 1_설비표.sql, 2_..., 10_... 로 바꿔 보세요. sort() 하면 어떤 순서가 되나요? 왜 001_ 을 쓰는지 알겠나요?

▹ 직접 해보기 3

마이그레이션 도구에 **해시 검사**를 넣어 보세요. crypto.createHash("sha256").update(내용).digest("hex") 를 기록 표에 같이 저장하고, 다음에 돌 때 다르면 경고하세요.

▹ 직접 해보기 4

기록 표의 한 줄을 지우고 다시 돌려 보세요. 001_설비표.sql 을 다시 적용하려 하면 무슨 에러가 나나요? (힌트: 42P07) 이것 때문에 기록 표가 필요합니다.

▹ 직접 해보기 5

트랜잭션을 빼 보세요. BEGIN/COMMIT 을 지우고 005_틀린것.sql 을 다시 넣어 돌리면 새표 가 남나요?

▹ 직접 해보기 6

SET lock_timeout = '1ms' 를 걸고 ALTER TABLE 을 해 보세요. 무슨 코드가 나오나요? (힌트: 55P03)

▹ 직접 해보기 7

두 개의 node 프로세스로 동시에 마이그레이션을 돌려 보세요. pg_advisory_lock 을 지우면 무슨 일이 나나요?

자주 하는 실수

이런 실수를 합니다

이미 나간 마이그레이션 파일을 고침

내 노트북에는 고친 게 적용되고 운영에는 옛것이 적용된 채로 남습니다. 그리고 아무도 그 차이를 모릅니다. 새 번호로 하나 더 만드세요.

이런 실수를 합니다

번호에 자리를 안 맞춤

2_... 와 10_... 을 sort() 하면 10 이 먼저 옵니다. 002_, 010_ 처럼 자리를 맞추세요.

이런 실수를 합니다

적용 기록을 파일이나 코드에 둬

서버마다 파일이 달라집니다. 기록은 **그 데이터베이스 안에** 있어야 “이 DB 가 어디까지 적용됐는가” 를 정확히 알 수 있습니다.

이런 실수를 합니다

마이그레이션을 커넥션 풀로 돌림

`풀.query(“BEGIN”)` 은 다른 연결로 갈 수 있습니다 (개념02 섹션 7). 마이그레이션은 `pg.Client` 하나로 하세요.

이런 실수를 합니다

배포와 동시에 칸을 지움

서버가 두 대 이상이면 배포 중에 옛 코드가 살아 있습니다. 그 순간 터집니다. 지우는 것은 **다음 배포**로 미루세요.

이런 실수를 합니다

큰 표에 `ALTER TABLE ... TYPE` 을 겁

표 전체를 다시 씁니다. 그동안 아무도 그 표를 못 씁니다. 1000만 줄이면 몇 분입니다. 새 칸을 만들어 옮기는 쪽이 안전합니다.

이런 실수를 합니다

`CREATE INDEX` 를 그냥 겁

표가 잠깁니다. 운영에서는 `CONCURRENTLY` 를 쓰세요. 단, 트랜잭션 안에서는 못 씁니다. 그 파일만 예외로 빼야 합니다.

ORM 은 언제 쓰나

```
RUN node 개념05 ORM은_언제_쓰나.js
```

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 이 파일은 새 패키지를 깔지 않습니다.

Prisma-Drizzle 코드는 **주석으로만** 보여 줍니다. 대신 그 동작을 흉내 내서 찍니다.

"요즘은 ORM 쓰지 않나요? SQL 을 왜 배워요?"

다섯 번쯤 듣는 질문입니다. 이 파일에서 답합니다.

ORM(Object-Relational Mapping)은 표의 줄을 객체처럼 다루게 해 주는 도구입니다. 좋습니다. 그런데 **SQL 을 모르는 채로 쓰면** 자기도 모르게 사고를 냅니다.

이 파일에서 할 것:

① 같은 기능을 생 SQL / 쿼리 빌더 / ORM 세 가지로 써 보기 (주석으로) ② ★★ N+1 을 **실제로** 만들어서 얼마나 느린지 재기 ③ 어려운 쿼리에서 ORM 을 탈출하는 법 ④ 무엇을 언제 고르나

```

import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

await 연결.query("DROP SCHEMA IF EXISTS 단원08_orm CASCADE");
await 연결.query("CREATE SCHEMA 단원08_orm");
await 연결.query("SET search_path TO 단원08_orm");

await 연결.query(`
  CREATE TABLE 설비 (
    설비번호 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인      TEXT NOT NULL,
    상태      TEXT NOT NULL DEFAULT '가동'
  )
`);

await 연결.query(`

```



```

CREATE TABLE 점검기록 (
  번호      SERIAL PRIMARY KEY,
  설비번호  TEXT NOT NULL REFERENCES 설비(설비번호),
  결과      TEXT NOT NULL,
  점검일    DATE NOT NULL
)
`);

await 연결.query(` INSERT INTO 설비 (설비번호, 이름, 라인)
  SELECT 'EQ-' || lpad(자리::text, 3, '0'), '설비' || 자리, (ARRAY['A','B','C'])[1
+ 자리 % 3]
  FROM generate_series(1, 100) AS 자리
`);

await 연결.query(` INSERT INTO 점검기록 (설비번호, 결과, 점검일)
  SELECT 'EQ-' || lpad(설비::text, 3, '0'),
        (ARRAY['정상','주의'])[1 + 회차 % 2],
        DATE '2026-01-01' + 회차
  FROM generate_series(1, 100) AS 설비, generate_series(1, 10) AS 회차
`);

await 연결.query("CREATE INDEX 점검기록_설비번호_색인 ON 점검기록 (설비번호);");
await 연결.query("ANALYZE");

console.log("설비 100대, 점검기록 1000건을 만들었습니다.");

```

출력 설비 100대, 점검기록 1000건을 만들었습니다.

쿼리가 몇 번 나갔는지 세려고 `query` 를 감쌉니다.

```

let 쿼리횟수 = 0;
const 원래query = 연결.query.bind(연결);
연결.query = (...인자들) => {
  쿼리횟수 += 1;
  return 원래query(...인자들);
};

const 중앙값 = (젠것) => 젠것.slice().sort((가, 나) => 가 - 나)
[Math.floor(젠것.length / 2)];

```

같은 일을 세 가지 방법으로

섹션 1

“A라인에서 가동 중인 설비 목록” 을 세 가지로 써 봅니다.

① 생 SQL — 우리가 지금까지 한 것

```
const 생SQL = await 연결.query(
  "SELECT 설비번호, 이름 FROM 설비 WHERE 라인 = $1 AND 상태 = $2 ORDER BY 설비번호
  LIMIT 3",
  ["A", "가동"],
);

console.log("생 SQL:", 생SQL.rows.map((줄) => 줄.설비번호).join(", "));
```

출력 생 SQL: EQ-003, EQ-006, EQ-009

② 쿼리 빌더 — SQL 을 함수로 조립합니다. (Kysely, Knex) 나가는 SQL 은 ①과 같습니다. 손으로 문자열을 잊지 않을 뿐입니다.

```
const 목록 = await db
```

```
.selectFrom("설비")
.select(["설비번호", "이름"])
.where("라인", "=", "A")
.where("상태", "=", "가동")
.orderBy("설비번호")
.limit(3)
.execute();
```

★ 좋은 점: 조건을 동적으로 붙이기 쉽습니다.

개념03 직접해보기 6 에서 `if` 가 늘어나는 문제를 봤습니다. 이게 그 답입니다.

```
let 질의 = db.selectFrom("설비").select(["설비번호", "이름"]);
if (라인) 질의 = 질의.where("라인", "=", 라인);
if (상태) 질의 = 질의.where("상태", "=", 상태);
const 목록 = await 질의.execute();
```

③ ORM — 표가 아니라 객체를 다룹니다. (Prisma, TypeORM, Sequelize)

```
const 목록 = await prisma.설비.findMany({
```

```
  where: { 라인: "A", 상태: "가동" },
  select: { 설비번호: true, 이름: true },
  orderBy: { 설비번호: "asc" },
  take: 3,
```

```
});
```

★ SQL 이 한 글자도 없습니다. 그게 장점이자 함정입니다.

****무슨 SQL 이 나가는지 안 보입니다.****

★ 세 가지의 자리를 그림으로 보면 이렇습니다.

내 코드

↓

ORM — 객체로 씁니다. SQL 이 안 보입니다

↓

쿼리 빌더 — SQL 모양 그대로. 조립만 도와줍니다

↓

생 SQL — 문자열

↓

드라이버(pg)

↓

PostgreSQL

위로 갈수록 편하고, 아래로 갈수록 정확합니다.

ORM 이 진짜로 좋은 점

섹션 2

말로만 하면 공평하지 않으니 정리합니다. **이건 진짜 장점입니다.**

① 타입과 자동완성

``설비.이름`` 을 치면 에디터가 알려 줍니다. ``설비.일음`` 이라고 치면 빨간 줄이 뜹니다.
생 SQL 은 문자열이라 오타를 ****실행해 봐야**** 합니다. 이 차이가 큼니다.

② 스키마와 마이그레이션이 한 곳에

스키마 파일을 고치면 마이그레이션을 만들어 줍니다. (개념04 를 대신해 줍니다)

③ 관계를 따라가기

설비.점검기록`처럼 점을 찍어 들어갑니다. JOIN 을 안 써도 됩니다.

④ 반복이 줄어듭니다

CRUD 다섯 개를 매번 손으로 쓰지 않습니다. 개념03 의 저장소를 반쯤 만들어 줍니다.

⑤ DB 를 바꾸기 쉽습니다 — ★ 이걸 반쯤만 사실입니다

단순한 쿼리는 그렇습니다. 어려운 쿼리는 어차피 DB 마다 다르게 씁니다.

그런데 ③이 바로 다음 섹션의 함정으로 이어집니다.

★★★ N+1 — 진짜로 만들어 봅니다

섹션 3

화면 하나를 만든다고 합시다. 설비 목록과 각 설비의 점검기록.

ORM 으로 쓰면 아주 자연스럽게 이렇게 씁니다.

```
const 설비들 = await prisma.설비.findMany();
for (const 설비 of 설비들) {
```

```
  설비.점검 = await prisma.점검기록.findMany({ where: { 설비번호: 설비.설비번호 } });
```

```
}
```

읽기 좋습니다. 그리고 101번 조회합니다.

같은 동작을 생 SQL 로 흉내 내서 재 봅니다. ORM 이 안에서 하는 일 그대로입니다.

```
async function 엔플러스원() {
  const 설비들 = (await 연결.query("SELECT 설비번호, 이름 FROM 설비")).rows;

  for (const 설비 of 설비들) {
    설비.점검 = (await 연결.query(
      "SELECT 결과, 점검일 FROM 점검기록 WHERE 설비번호 = $1",
      [설비.설비번호],
    )).rows;
  }

  return 설비들;
}

async function 조인한번() {
  const 줄들 = (await 연결.query(`
```

```
FROM 설비
  LEFT JOIN 점검기록 ON 점검기록.설비번호 = 설비.설비번호
  ORDER BY 설비.설비번호
`)).rows;
```

줄들을 설비별로 묶습니다. ORM 이 안에서 해 주는 일입니다.

```
const 모음 = new Map();

for (const 줄 of 줄들) {
  if (!모음.has(줄.설비번호)) {
    모음.set(줄.설비번호, { 설비번호: 줄.설비번호, 이름: 줄.이름, 점검: [] });
  }

  if (줄.결과 !== null) 모음.get(줄.설비번호).점검.push({ 결과: 줄.결과, 점검일:
줄.점검일 });
}

return [...모음.values()];
}

async function 두번에() {
  const 설비들 = (await 연결.query("SELECT 설비번호, 이름 FROM 설비")).rows;
  const 번호들 = 설비들.map((설비) => 설비.설비번호);
```

★ IN 대신 = ANY(\$1) 을 씁니다. 배열 하나를 파라미터로 넘길 수 있습니다.

```
const 기록들 = (await 연결.query(
  "SELECT 설비번호, 결과, 점검일 FROM 점검기록 WHERE 설비번호 = ANY($1)",
  [번호들],
)).rows;

const 모음 = new Map(설비들.map((설비) => [설비.설비번호, { ...설비, 점검: [] } ]));

for (const 줄 of 기록들) 모음.get(줄.설비번호).점검.push({ 결과: 줄.결과, 점검일:
줄.점검일 });

return [...모음.values()];
}

async function 재기(할일, 횟수 = 5) {
  const 잔것 = [];
  let 쿼리 = 0;

  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    쿼리횟수 = 0;
    const 시작 = performance.now();
```

```

    잰것.push(performance.now() - 시작);
    쿼리 = 쿼리횟수;
  }

  return { ms: 중앙값(잰것), 쿼리 };
}

const 느린것 = await 재기(엔플러스원);
const 조인 = await 재기(조인한번);
const 두번 = await 재기(두번에);

console.log(`N+1 방식 : ${느린것.ms.toFixed(1)} ms · 쿼리 ${느린것.쿼리}번`);

```

출력 N+1 방식 : 53.9 ms · 쿼리 101번
값이 다를 수 있음

```

console.log(`JOIN 한 번: ${조인.ms.toFixed(1)} ms · 쿼리 ${조인.쿼리}번`);

```

출력 JOIN 한 번: 3.0 ms · 쿼리 1번
값이 다를 수 있음

```

console.log(`ANY 두 번 : ${두번.ms.toFixed(1)} ms · 쿼리 ${두번.쿼리}번`);

```

출력 ANY 두 번 : 2.5 ms · 쿼리 2번
값이 다를 수 있음

```

console.log("N+1 이 가장 느린가:", 느린것.ms > 조인.ms && 느린것.ms > 두번.ms);

```

출력 N+1 이 가장 느린가: true

```

console.log(`몇 배 느린가: ${(느린것.ms / 조인.ms).toFixed(0)} 배`);

```

출력 몇 배 느린가: 18 배
값이 다를 수 있음

```

console.log("결과는 똑같은가:", (await 엔플러스원()).length === (await 조인한번()).length);

```

출력 결과는 똑같은가: true

★★★ 같은 화면인데 쿼리가 101번 대 1번입니다.

그리고 이걸 같은 노트북 안에서 잰 값입니다. 서버와 DB 가 다른 기계면 쿼리 한 번마다 왕복 지연이 붙습니다. 왕복이 1밀리초라면 101번은 그것만으로 100밀리초가 더 붙습니다. 클라우드에서 흔한 값입니다.

★ 이름이 왜 N+1 인가 목록을 가져오는 1번 + 각 줄마다 N번. 그래서 N+1 입니다.

★★ 무서운 점은 작은 데이터에서는 안 보인다는 것입니다. 개발할 때 설비가 3대면 4번입니다. 빠릅니다. 아무 문제 없어 보입니다. 운영에서 설비가 5000대가 되면 5001번이 됩니다. 그때 처음 압니다.

★ 그리고 이걸 **ORM 만의 문제가 아닙니다.** 개념03 의 저장소로도 똑같이 납니다.

```
for (const 설비 of 설비들) {
  설비.점검 = await 점검저장소.목록(풀, 설비.번호); // ← 여기가 N+1 입니다
}
```

저장소 함수를 예쁘게 만들어 두면 for 문 안에서 부르고 싶어집니다. 그게 함정입니다.

ORM 도 고칠 방법을 줍니다

섹션 4

좋은 ORM 은 관계를 **한 번에 가져오는** 방법을 제공합니다.

```
// Prisma
const 설비들 = await prisma.설비.findMany({

include: { 점검기록: true },           // ★ 이 한 줄이 N+1 을 없앱니다

});

// TypeORM
const 설비들 = await 저장소.find({ relations: ["점검기록"] });

// Drizzle
const 설비들 = await db.query.설비.findMany({ with: { 점검기록: true } });
```

★ Prisma 는 **include** 를 쓰면 대개 **쿼리 2번**을 보냅니다. 위에서 **ANY** 두 번 과 같은 방식입니다. JOIN 보다 줄 수가 적어서 유리할 때도 있습니다.

★★ 그런데 문제는 **include** 를 **안 쓰면 조용히 N+1 이 된다는** 것입니다. 에러가 안 납니다. 경고도 없습니다. 그냥 느려집니다. 그래서 **나가는 SQL 을 볼 줄 알아야** 합니다.

```
Prisma: new PrismaClient({ log: ["query"] })
TypeORM: createDataSource({ logging: true })
Drizzle: drizzle(연결, { logger: true })
```

★ 개발할 때 이 로그를 **켜 두세요.** 화면 한 번에 쿼리가 몇 번 나가는지

눈으로 보면 N+1 은 그 자리에서 잡힙니다.

우리 코드에서도 같은 것을 해 봅니다. 쿼리를 세는 것만으로 잡힙니다.

```

쿼리횟수 = 0;
await 엔플러스원();
const 나쁜쿼리수 = 쿼리횟수;

쿼리횟수 = 0;
await 두번에();
const 좋은쿼리수 = 쿼리횟수;

console.log("화면 한 번에 나간 쿼리:", 나쁜쿼리수, "→", 좋은쿼리수);

```

출력 화면 한 번에 나간 쿼리: 101 → 2

```

console.log("쿼리 수가 데이터 개수에 비례하나 (나쁜 신호):", 나쁜쿼리수 > 50);

```

출력 쿼리 수가 데이터 개수에 비례하나 (나쁜 신호): true

★★ 판단 기준은 간단합니다. 쿼리 개수가 데이터 개수에 따라 늘어나면 N+1 입니다. 설비가 100대일 때 101번, 200대일 때 201번이면 확실합니다.

어려운 쿼리는 결국 SQL 로 씁니다

섹션 5

ORM 은 단순한 CRUD 에서 빛납니다. 조금만 어려워지면 막힙니다.

· 라인별로 최근 점검 결과 하나씩만 (윈도우 함수) · 여러 단계로 묶은 집계 · WITH RECURSIVE · INSERT ... ON CONFLICT DO UPDATE · EXPLAIN 을 보면서 힌트를 조정하는 일

이런 것은 ORM 문법으로 쓰면 오히려 더 어렵고, 나가는 SQL 이 느려집니다.

★ 그래서 모든 ORM 에는 탈출구가 있습니다.

```

// Prisma
const 줄들 = await prisma.$queryRaw`SELECT ... WHERE 라인 = ${라인}`;

```

```

// TypeORM
const 줄들 = await 데이터소스.query("SELECT ... WHERE 라인 = $1", [라인]);

```

```

// Drizzle
const 줄들 = await db.execute(sql`SELECT ... WHERE 라인 = ${라인}`);

```

★★ 탈출구를 쓸 때 파라미터 처리 방식이 바뀝니다. 여기서 사고가 납니다. Prisma 의 \$queryRaw 는 태그드 템플릿이라 \${} 가 파라미터가 됩니다. 안전합니다. 그런데 \$queryRawUnsafe(문자열) 은 그냥 문자열입니다. 이어 붙이면 뚫립니다. 이름에 Unsafe 가 붙어 있는 이유가 있습니다.

실제로 어려운 쿼리 하나를 SQL 로 써 봅니다. "라인별 최근 점검 하나씩" 입니다.


```
const 라인별최근 = await 연결.query(`
  SELECT 라인, 설비번호, 결과, 점검일
  FROM (
    SELECT 설비.라인, 설비.설비번호, 점검기록.결과, 점검기록.점검일,
           row_number() OVER (PARTITION BY 설비.라인 ORDER BY 점검기록.점검일 DESC,
설비.설비번호) AS 순위
    FROM 설비
    JOIN 점검기록 ON 점검기록.설비번호 = 설비.설비번호
  ) AS 매긴것
  WHERE 순위 = 1
  ORDER BY 라인
`);

for (const 줄 of 라인별최근.rows) {
  console.log(`${줄.라인}라인 최근 점검 - ${줄.설비번호} · ${줄.결과}`);
}
```

```
출력      A라인 최근 점검 - EQ-003 · 정상
          B라인 최근 점검 - EQ-001 · 정상
          C라인 최근 점검 - EQ-002 · 정상
```

★ 이걸 ORM 문법으로 쓰려면 대부분 방법이 없거나, 있어도 읽기 더 어렵습니다. **SQL 을 아는 사람은 여기서 5분이면 끝납니다.** 모르는 사람은 목록을 전부 가져와서 자바스크립트에서 정렬합니다. 그러면 1000줄을 서버로 옮기고, 999줄을 버립니다.

무엇을 언제 고르나

섹션 6

★ 정답은 없습니다. 판단 기준을 드립니다.

상황 고르면 좋은 것

CRUD 가 대부분인 관리 화면 ORM 팀에 SQL 을 아는 사람이 적음 ORM (단, 로그를 켜 두세요)
 TypeScript 로 타입을 끝까지 쓰고 싶음 ORM 또는 쿼리 빌더 조건이 동적으로 붙는 검색 화면 쿼리 빌더
 집계-리포트가 많음 생 SQL 응답 시간이 밀리초 단위로 중요함 생 SQL 이미 있는 DB 에 붙여야 함 생 SQL
 또는 쿼리 빌더 여러 서비스가 같은 DB 를 씀 생 SQL (ORM 이 스키마를 소유하려 듭니다)

★★ 섞어 쓰는 것이 가장 흔합니다. 그리고 그게 맞습니다. CRUD 는 ORM, 리포트는 생 SQL. 하나만 골라야 한다는 규칙은 없습니다.

★ 규모로 보면 · 혼자, 빨리 만들어야 함 → ORM 이 이깁니다 · 열 명, 오래 갈 서비스 → 규칙을 정하는 것이 도구보다 중요합니다 · 성능이 돈인 서비스 → 생 SQL 을 쓸 자리가 반드시 생깁니다

SQL 을 먼저 배우세요.

이유는 하나입니다.

ORM 은 SQL 을 아는 사람이 쓰면 편한 도구이고, 모르는 사람이 쓰면 지뢰밭입니다.

왜 그런가:

· ORM 이 만든 SQL 이 느릴 때, SQL 을 모르면 왜 느린지 알 수가 없습니다 · N+1 은 ORM 이 안 막아 줍니다. SQL 을 아는 사람만 알아챱니다 · 트랜잭션·격리수준·잠금은 ORM 을 써도 그대로 알아야 합니다 (07단원) · 색인이 왜 안 타는지는 EXPLAIN 을 읽어야 합니다 (06단원) · 어려운 쿼리는 결국 탈출구로 나와야 합니다

★ 반대로, SQL 을 아는 사람에게 ORM 은 아주 좋은 도구입니다. 반복을 없애 주고, 타입을 지켜 주고, 마이그레이션을 만들어 줍니다. 나가는 SQL 이 눈에 보이니 이상하면 바로 알아챱니다.

★★★ 한 줄로: **ORM 을 쓰지 말라는 말이 아닙니다. ORM 이 무슨 SQL 을 보내는지 읽을 수 있는 상태에서 쓰라는 말입니다.**

2026년 기준으로 무엇이 있나

이름 무엇 좋은 점 조심할 점

Prisma ORM 타입이 아주 좋음. 스키마 파일 자체 문법을 새로 배워야 함

하나로 마이그레이션까지 어려운 쿼리는 \$queryRaw 로 탈출

Drizzle ORM/빌더 SQL 에 가깝고 가벼움. 아직 젊습니다. 바뀌는 게 많습니다

나가는 SQL 이 예측됩니다

Kysely 쿼리 빌더 ORM 이 아닙니다. SQL 을 관계를 자동으로 안 가져옵니다

타입 안전하게 조립합니다 (그래서 N+1 이 잘 안 납니다)

TypeORM ORM 오래됐고 자료가 많음 데코레이터 방식. 큰 프로젝트에서

동작이 예측 안 되는 경우가 있음

Sequelize ORM 가장 오래됨. JS 로도 씬 타입 지원이 약함

pg (생 SQL) 드라이버 가장 빠르고 가장 정확함 전부 손으로 씬니다

★ 이 자료는 pg 만 씁니다. 다른 것은 **칼지 않습니다**. 어느 것을 고르든 여기서 배운 것이 그대로 쓰입니다. 도구는 바뀔니다. SQL 은 40년째 그대로입니다.

뒷정리

```
await 연결.query("DROP SCHEMA IF EXISTS 단원08_orm CASCADE");
await 연결.end();

console.log("끝났습니다.");
```

출력 끝났습니다.

정리

층 예 장점 단점

ORM Prisma, TypeORM 빠르게 만듭니다 SQL 이 안 보입니다 쿼리 빌더 Kysely, Knex 조건 조립이 쉽습니다 관계는 직접 처리 생 SQL pg 정확하고 빠릅니다 전부 손으로

N+1 쿼리 101번 ORM 의 대표 함정 include / with 쿼리 2번 ORM 이 주는 해결책 JOIN 한 번 쿼리 1번 SQL 을 알면 이게 보입니다

★★★ 판별법: 쿼리 개수가 데이터 개수를 따라 늘면 N+1 입니다. ★★★ 입장: SQL 을 먼저. ORM 은 SQL 을 읽을 수 있게 된 다음에.

직접 해 볼 것

▫ 직접 해보기 1

설비를 100대에서 500대로 올려 보세요. N+1 은 몇 번이 되나요? JOIN 은요? 시간 차이는 몇 배가 되나요?

▫ 직접 해보기 2

점검기록_설비번호_색인 을 지우고 N+1 을 다시 재 보세요. 얼마나 더 느려지나요? (★ 06단원에서 배운 색인입니다)

▫ 직접 해보기 3

조인한번 이 돌려주는 줄 수와 두번에 가 옮기는 줄 수를 세어 보세요. JOIN 은 설비 이름을 몇 번 반복해서 보내나요? (힌트: 이래서 JOIN 이 항상 이기는 것은 아닙니다)

▫ 직접 해보기 4

연결.query 를 감싼 부분에 SQL 문장도 같이 찍게 해 보세요. N+1 을 돌리면 똑같은 SQL 이 100번 나가는 게 보입니다.

▫ 직접 해보기 5

섹션 5 의 윈도우 함수 쿼리를 N+1 방식으로 다시 써 보세요. (라인마다 설비를 전부 가져와서 자바스크립트로 정렬) 몇 줄이 서버로 오나요?

▫ 직접 해보기 6

SELECT * 와 SELECT 설비번호, 이름 의 시간을 재 보세요. 칸이 4개일 때는 차이가 작습니다. 칸이 40개면 어떨까요?

▫ 직접 해보기 7

여러분이 지금 만들고 있는 것에 무엇을 쓸지 골라 보고, 왜 그런지 세 줄로 적어 보세요. 그게 이 파일의 진짜 숙제입니다.

자주 하는 실수

이런 실수를 합니다

for 문 안에서 조회함

ORM 이든 저장소든 똑같이 N+1 이 납니다. for 안에 await ...query 가 보이면 일단 멈추고 생각하세요.

이런 실수를 합니다

나가는 SQL 을 한 번도 안 봄

ORM 을 쓰면서 로그를 안 켜면 무슨 일이 벌어지는지 알 방법이 없습니다. 개발할 때는 켜 두세요. 화면 한 번에 쿼리가 몇 번 나가는지 세어 보세요.

이런 실수를 합니다

“ORM 을 쓰니까 SQL 은 몰라도 된다”

느려졌을 때 고칠 수가 없습니다. 그리고 반드시 느려집니다. ORM 은 SQL 을 안 쓰게 해 주는 도구지, 모르게 해 주는 도구가 아닙니다.

이런 실수를 합니다

전부 가져와서 자바스크립트에서 거름

findMany() 로 5만 줄을 가져와 .filter() 하는 코드를 자주 봅니다. WHERE 를 쓰면 DB 가 색인으로 걸러 줍니다. 옮기는 양이 다릅니다.

이런 실수를 합니다

\$queryRawUnsafe 에 값을 이어 붙임

\$queryRaw 는 태그드 템플릿이라 안전하고, Unsafe 는 안전하지 않습니다. 이름을 그렇게 지어 놓은 데는 이유가 있습니다.

이런 실수를 합니다

ORM 하나로 전부 하려고 함

리포트 화면 하나 때문에 ORM 문법을 사흘 붙잡고 있는 경우가 많습니다. 탈출구로 나와서 SQL 로 쓰면 30분이면 끝납니다. 섞어 쓰는 게 정상입니다.

이런 실수를 합니다

트랜잭션을 ORM 이 알아서 해 줄 거라고 생각함

ORM 도 트랜잭션은 명시적으로 열어야 합니다. 그리고 개념02 에서 본 "같은 연결" 규칙은 ORM 을 써도 그대로입니다.

애플리케이션과 잇기

RUN node 연습문제.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

// TODO: 를 채우고 실행하세요. 채점이 자동으로 됩니다.

✓ 이면 통과, × 이면 아직입니다.

지금 실행하면 전부 × 입니다. 위에서부터 하나씩 채우세요. 막히면 개념01~05 를 다시 보세요. 정답은 연습문제_정답.js 에 있습니다.

★ 15번까지 있습니다. 13~15번은 [도전] 입니다.

```

import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
};

let 준비;

try {
  준비 = new pg.Client(접속정보);
  await 준비.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

await 준비.query("DROP SCHEMA IF EXISTS 단원08_연습 CASCADE");
await 준비.query("CREATE SCHEMA 단원08_연습");
await 준비.query("SET search_path TO 단원08_연습");
await 준비.query(`
  CREATE TABLE 설비 (
    설비번호 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인      TEXT NOT NULL,
    상태      TEXT NOT NULL DEFAULT '정지',
    도입가    NUMERIC(12, 2) NOT NULL DEFAULT 0,
    설치일    DATE NOT NULL DEFAULT DATE '2026-01-01'
  );

```

```

    )
  `);
  await 준비.query(` CREATE TABLE 점검기록 (
    번호      SERIAL PRIMARY KEY,
    설비번호 TEXT NOT NULL REFERENCES 설비(설비번호),
    결과      TEXT NOT NULL
  )
  `);
  await 준비.query(` INSERT INTO 설비 (설비번호, 이름, 라인, 상태, 도입가, 설치일)
VALUES
  ('EQ-001', '컨베이어 1호', 'A', '가동', 1200000.50, DATE '2026-03-02'),
  ('EQ-002', '프레스 1호', 'B', '정지', 3400000.25, DATE '2026-04-11'),
  ('EQ-003', '용접로봇 1호', 'A', '가동', 980000.25, DATE '2026-05-20')
  `);
  await 준비.query(` INSERT INTO 점검기록 (설비번호, 결과)
  SELECT 'EQ-' || lpad(설비::text, 3, '0'), '정상' FROM generate_series(1, 3) AS
설비, generate_series(1, 4) AS 회차
  `);
  await 준비.query("CREATE TABLE 잔고 (계정 TEXT PRIMARY KEY, 값 INT NOT NULL)");
  await 준비.query("INSERT INTO 잔고 VALUES ('A라인', 100), ('B라인', 100)");

  const 풀 = new pg.Pool({ ...접속정보, max: 5, options: "-c search_path=단원08_연습"
  });

```

채점 도구 (고치지 마세요)

```

const 채점결과 = [];

async function 문제(번호, 제목, 검사) {
  try {
    채점결과.push(`${(await 검사()) ? "✓" : "x"} ${번호}. ${제목}`);
  } catch (에러) {
    채점결과.push(`x ${번호}. ${제목} (${에러.constructor.name})`);
  }
}

```

풀에서 빌려 간 연결을 추적합니다. 학생 코드가 새어도 파일이 끝나게 하려고요.

```

function 감시풀(설정) {
  const 새풀 = new pg.Pool(설정);
  const 빌려준것 = new Set();
  const 원래connect = 새풀.connect.bind(새풀);

  새풀.connect = async (...인자들) => {
    const 연결 = await 원래connect(...인자들);
    const 원래release = 연결.release.bind(연결);

```



```

빌려준것.add(연결);
    연결.release = (...것들) => {
        빌려준것.delete(연결);
        return 원래release(...것들);
    };

    return 연결;
};

새품.강제정리 = () => {
    for (const 연결 of 빌려준것) {
        try {
            연결.release();
        } catch {

```

이미 돌아간 것입니다

```

    }
}

빌려준것.clear();
};

return 새품;
}

```

1. 접속 설정 객체 만들기

docker-compose.yml 의 postgres 는 이렇게 떠 있습니다. 포트 5434 · 계정 factory · 비밀번호 secret · DB factory_db

그 정보로 pg.Client 설정 객체를 만드세요. host 는 "127.0.0.1" 입니다.

```
function 설정만들기() {
```

```
{ host, port, user, password, database } 를 돌려주세요
```

```
return null;
}
```

```
await 문제(1, "접속 설정 객체 만들기", async () => {
    const 설정 = 설정만들기();

```


MySQL과 무엇이 다른가

OPEN 09_MySQL과_무엇이_다른가

```
import pg from "pg";  
let PG = null;  
let MY = null;  
PG = new pg.Client(피지설정);
```

01 왜 둘로 갈렸나

02 옛날 이야기를 뒤집습니다

03 조용히 다른 결과

04 기능과 문법의 차이

05 옮길 때 확인할 것

- 연습문제

왜 둘로 갈렸나

RUN node 개념01_왜_둘로_갈렸나.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

PostgreSQL(5434) 과 MySQL(3307) 을 **동시에** 씁니다.

Docker 가 없으면 안내만 짚고 조용히 끝납니다.

08단원까지는 PostgreSQL 하나만 봤습니다. 그런데 회사에 가면 MySQL 이 깔려 있는 경우가 아주 많습니다. 특히 국내 SI·제조·공공 현장이 그렇습니다.

그래서 이 단원은 **둘을 같이 띄워 놓고 나란히 실행합니다**. 같은 SQL 을 양쪽에 보내고, 결과를 옆에 붙여서 봅니다.

이 파일에서 할 것은 세 가지입니다.

① 두 데이터베이스가 왜 다르게 생겼는지 — 짧게 ② 그 차이가 **지금도 남은 곳**과 **이미 사라진 곳** ③ 이 단원의 실측 환경을 눈으로 확인하기

★ 미리 말해 둡니다. 이 단원은 어느 쪽이 낫다는 이야기가 아닙니다. 둘 다 훌륭한 데이터베이스입니다. **차이를 정확히 아는 것**이 목적입니다. 틀린 지식을 가지고 옮기면 사고가 나기 때문입니다.

```
import pg from "pg";
import mysql from "mysql2/promise";
```

둘에 붙기

섹션 0

이 단원 내내 쓰는 접속 정보입니다.

```
const 피지설정 = { host: "127.0.0.1", port: 5434, user: "factory", password:
"secret", database: "factory_db" };
const 마이설정 = { host: "127.0.0.1", port: 3307, user: "factory", password:
"secret", database: "factory_db" };

let PG = null;
let MY = null;

try {
  PG = new pg.Client(피지설정);
  await PG.connect();
  MY = await mysql.createConnection(마이설정);
```

```

} catch (에러) {
  console.log("★ 데이터베이스에 못 붙었습니다:", 에러.code || 에러.message);
  console.log(" docker compose up -d 를 먼저 실행하세요. (docker compose ps 로 확인)");
  if (PG) await PG.end().catch(() => {});
  process.exit(0);
}

```

★ Postgres 는 07~10단원이 같은 데이터베이스를 씁니다. 단원끼리 표 이름이 부딪히지 않게 **자기 스키마**를 씁니다.

```

await PG.query("CREATE SCHEMA IF NOT EXISTS 단원09");
await PG.query("SET search_path TO 단원09");

```

★★ MySQL 에는 “스키마” 라는 칸막이가 없습니다. MySQL 에서 SCHEMA 는 DATABASE 와 **같은** 말입니다. 그래서 나눌 수가 없습니다. 대신 표 이름 앞에 **구구_** 를 붙여서 09단원 것임을 표시합니다.

★ 이것 자체가 첫 번째 차이입니다.

Postgres 는 데이터베이스 안에 스키마가 여러 개 있습니다.
MySQL 은 데이터베이스가 곧 스키마입니다.

1: 이 단원 내내 쓸 도우미

섹션 0

같은 SQL 을 양쪽에 보내고 결과를 한 줄씩 찍는 함수입니다. **에러도 잡습니다.** 이 단원에서는 에러가 나는 것 자체가 결과입니다.

```

async function 피지(sql, 값 = []) {
  try {
    const 결과 = await PG.query(sql, 값);
    return { 됨: true, 값: 결과.rows };
  } catch (에러) {
    return { 됨: false, 코드: 에러.code, 말: 에러.message };
  }
}

async function 마이(sql, 값 = []) {
  try {
    const [행들] = await MY.query(sql, 값);
    return { 됨: true, 값: 행들 };
  } catch (에러) {

```

★ MySQL 에러는 code 가 글자입니다(ER...). 숫자는 errno 입니다.

```
return { 됴: false, 코드: 에러.code, 번호: 에러.errno, 말: 에러.message };
}
}
```

결과를 한 줄로 줄여 주는 함수입니다.

```
function 한줄로(결과) {
  if (결과.됨) return JSON.stringify(결과.값);
  return `거절 ${결과.코드}${결과.번호 ? "/" + 결과.번호 : ""}`;
}
```

★ 이 세 함수를 이 단원의 다섯 개 파일에서 계속 씁니다. 파일 하나가 스스로 완결돼야 해서 매번 다시 적습니다.

두 DB 의 출신

섹션 1

역사 수업이 아닙니다. **지금 코드에 남아 있는 것만** 짚습니다.

PostgreSQL

1986년 버클리 대학의 연구 과제 (POSTGRES)에서 나왔습니다.
 목적이 "논문에 쓸 만한 제대로 된 관계형 DB" 였습니다.
 그래서 처음부터 ****표준 SQL·타입 시스템·확장성****을 중요하게 봤습니다.

MySQL

1995년 스웨덴의 작은 회사에서 나왔습니다.
 목적이 "웹사이트 뒤에 놓고 빨리 읽는 것" 이였습니다.
 당시 웹은 대부분 ****읽기만**** 했습니다. 게시판, 뉴스, 쇼핑몰 목록.
 그래서 처음에는 트랜잭션조차 없었습니다. 빠르면 됐으니까요.

★ 이 출신 차이가 만든 가장 큰 흔적이 **스토리지 엔진**입니다. MySQL 은 "저장하는 부분"을 갈아 끼울 수 있게 만들었습니다. 초기 기본은 MyISAM 이었고, MyISAM 에는 트랜잭션이 없습니다.

```
const 엔진들 = await 마이(
  "SELECT ENGINE, TRANSACTIONS FROM information_schema.ENGINES WHERE SUPPORT IN
  ('YES','DEFAULT') ORDER BY ENGINE",
);

console.log("MySQL 이 지금 쓸 수 있는 엔진:");
```

출력 MySQL 이 지금 쓸 수 있는 엔진:

```
for (const 줄 of 엔진들.값) {
  console.log(`  · ${줄.ENGINE} - 트랜잭션 ${줄.TRANSACTIONS}`);
}
```

```
출력      · ARCHIVE - 트랜잭션 NO
          · BLACKHOLE - 트랜잭션 NO
          · CSV - 트랜잭션 NO
          · InnoDB - 트랜잭션 YES
          · MEMORY - 트랜잭션 NO
          · MRG_MYISAM - 트랜잭션 NO
          · MyISAM - 트랜잭션 NO
          · PERFORMANCE_SCHEMA - 트랜잭션 NO
```

```
const 기본엔진 = await 마이("SELECT @@default_storage_engine AS e");
console.log("MySQL 8.4 의 기본 엔진:", 기본엔진.값[0].e);
```

```
출력      MySQL 8.4 의 기본 엔진: InnoDB
```

★ 기본이 InnoDB 입니다. InnoDB 는 트랜잭션이 있습니다. MySQL 5.5(2010년) 부터 기본이 바뀌었습니다. “MySQL 은 트랜잭션이 없다” 는 말은 그때 끝난 이야기입니다.

★★ 그런데 MyISAM 은 **지금도 만들 수 있습니다**. 직접 해 봅시다.

```
await 마이("DROP TABLE IF EXISTS 구구_옛날표");
await 마이("CREATE TABLE 구구_옛날표 (값 INT) ENGINE=MyISAM");

await 마이("BEGIN");
await 마이("INSERT INTO 구구_옛날표 VALUES (1)");
await 마이("ROLLBACK");

const 롤백후 = await 마이("SELECT 값 FROM 구구_옛날표");
console.log("MyISAM 표에 넣고 ROLLBACK 한 뒤 남은 것:", 한줄로(롤백후));
```

```
출력      MyISAM 표에 넣고 ROLLBACK 한 뒤 남은 것: [{"값":1}]
```

★★★ ROLLBACK 을 했는데 **줄이 그대로 있습니다**. MyISAM 에는 트랜잭션이 없어서 ROLLBACK 이 그냥 무시됩니다. 에러도 안 납니다. 조용히 무시합니다.

옛날 SI 프로젝트에서 만든 표가 MyISAM 이면 지금도 이렇게 동작합니다. ★ 회사 표를 처음 만나면 엔진부터 확인하세요. 아래 SQL 로 봅시다.

```
const 엔진확인 = await 마이(
  "SELECT TABLE_NAME AS 표, ENGINE AS 엔진 FROM information_schema.TABLES WHERE
  TABLE_SCHEMA='factory_db' AND TABLE_NAME='구구_옛날표'",
);
console.log("표의 엔진 확인:", 한줄로(엔진확인));
```

출력 표의 엔진 확인: [{"표": "구구_옛날표", "엔진": "MyISAM"}]

```
await 마이("DROP TABLE 구구_옛날표");
```

★ PostgreSQL 에는 스토리지 엔진을 고르는 개념이 없습니다. 저장 방식이 하나뿐이고, 그것이 트랜잭션을 지원합니다. “고를 수 없다” 가 여기서는 **틀릴 수 없다** 는 뜻이 됩니다.

라이선스와 소유 — 실무에 영향을 주는 만큼만

섹션 2

```
const 버전주석 = await 마이("SELECT @@version_comment AS c");
console.log("MySQL 이 스스로 밝히는 판(版):", 버전주석.값[0].c);
```

출력 MySQL 이 스스로 밝히는 판(版): MySQL Community Server - GPL

위 한 줄에 이야기가 다 들어 있습니다.

MySQL — 주인이 있습니다.

션(2008) → 오라클(2010) 로 넘어갔습니다.
무료판(Community, GPL) 과 유료판(Enterprise) 이 따로 있습니다.
★ GPL 이라 "MySQL 서버를 내 제품에 넣어서 판다" 면 라이선스를 봐야 합니다.
서버를 그냥 설치해서 쓰는 보통의 웹 서비스는 문제가 없습니다.

PostgreSQL — 주인이 없습니다.

전 세계 개발자 모임이 만듭니다. 회사가 사 갈 대상이 없습니다.
PostgreSQL License(BSD 계열) 라서 제품에 넣어 팔아도 됩니다.

★ MariaDB 가 왜 생겼나 오라클이 MySQL 을 가져가자, 원래 만들던 사람들이 소스를 복사해서 따로 만들기 시작했습니다. 그것이 MariaDB 입니다. (2009년) 초기에는 거의 같았지만 지금은 꽤 갈라졌습니다. 개념05 에서 한 문단 더 다룹니다.

★★ 실무에서 이게 왜 중요한가 “MySQL 이라고 들었는데 접속해 보니 MariaDB” 인 현상이 흔합니다. 버전 문자열을 꼭 눈으로 확인하세요. 문법이 갈리는 곳이 있습니다.

설계 철학의 차이

섹션 3

MySQL 의 기본값 : 속도와 단순함 PostgreSQL 의 기본값 : 정확함과 표준

말로만 하면 안 와닿습니다. 한 줄로 재 보겠습니다.


```
const 나눗셈피지 = await 피지("SELECT 1/0 AS 결과");
const 나눗셈마이 = await 마이("SELECT 1/0 AS 결과");

console.log("1 을 0 으로 나누면 - PG:", 한줄로(나눗셈피지));
```

출력 1 을 0 으로 나누면 - PG: 거절 22012

```
console.log("1 을 0 으로 나누면 - MySQL:", 한줄로(나눗셈마이));
```

출력 1 을 0 으로 나누면 - MySQL: [{"결과":null}]

★ 이 한 줄이 두 철학을 정확히 보여 줍니다.

PostgreSQL : “그건 계산할 수 없습니다.” → 멈춥니다 MySQL : “값이 없네요.” → NULL 을 주고 계속 갑니다

어느 쪽이 옳을까요. **상황에 따라 다릅니다.** 집계 화면이라면 NULL 이 편합니다. 정산이라면 멈추는 쪽이 맞습니다. ★ 중요한 건 **어느 쪽인지 알고 쓰는 것**입니다. 개념03 에서 더 팝니다.

★ 철학이 남은 곳과, 이미 사라진 곳

섹션 4

인터넷에 있는 MySQL 비교 글은 대부분 2010년대 초에 쓴 것입니다. 그때 맞던 말이 지금도 맞는지 확인해야 합니다.

먼저 **이미 사라진 것**부터 봅시다.

```
const 옛말들 = [
  ["MySQL 은 트랜잭션이 없다", "사라짐", "5.5(2010) 부터 기본이 InnoDB"],
  ["MySQL 은 CHECK 제약을 무시한다", "사라짐", "8.0.16(2019) 부터 진짜로 검사"],
  ["MySQL 은 이상한 값을 조용히 넣는다", "거의 사라짐", "5.7/8.0 부터 STRICT 가 기본"],
  ["MySQL 은 GROUP BY 를 아무렇게나 받는다", "사라짐", "5.7 부터 ONLY_FULL_GROUP_BY 가 기본"],
  ["MySQL 은 원도 함수가 없다", "사라짐", "8.0(2018) 부터 있음"],
  ["MySQL 은 CTE(WITH) 가 없다", "사라짐", "8.0 부터 있음"],
];

console.log("[이미 사라진 옛말]");
```

출력 [이미 사라진 옛말]

```
for (const [말, 판정, 이유] of 옛말들) {
  console.log(` · "${말}" → ${판정} (${이유})`);
}
```

```
출력      · "MySQL 은 트랜잭션이 없다" → 사라짐 (5.5(2010) 부터 기본이 InnoDB)
          · "MySQL 은 CHECK 제약을 무시한다" → 사라짐 (8.0.16(2019) 부터 진짜로
          검사)
          · "MySQL 은 이상한 값을 조용히 넣는다" → 거의 사라짐 (5.7/8.0 부터
          STRICT 가 기본)
          · "MySQL 은 GROUP BY 를 아무렇게나 받는다" → 사라짐 (5.7 부터
          ONLY_FULL_GROUP_BY 가 기본)
          · "MySQL 은 윈도우 함수가 없다" → 사라짐 (8.0(2018) 부터 있음)
          · "MySQL 은 CTE(WITH) 가 없다" → 사라짐 (8.0 부터 있음)
```

```
const 남은것들 = [
  ["|| 가 문자열 연결이 아니다", "남음", "MySQL 에서는 OR 입니다"],
  ["글자 비교가 대소문자를 무시한다", "남음", "기본 collation 이 _ci"],
  ["트랜잭션 안에서 DDL 이 안 되돌아간다", "남음", "CREATE/ALTER 가 암묵적 커밋"],
  ["RETURNING 이 없다", "남음", "INSERT 결과를 바로 못 받습니다"],
  ["부분 색인이 없다", "남음", "CREATE INDEX ... WHERE 가 문법 오류"],
  ["표 이름 대소문자가 OS 를 탄다", "남음", "리눅스와 윈도우가 다르게 동작"],
];

console.log("[아직 그대로 남은 것]);
```

```
출력      [아직 그대로 남은 것]
```

```
for (const [말, 판정, 이유] of 남은것들) {
  console.log(` · "${말}" → ${판정} (${이유})`);
}
```

```
출력      · "|| 가 문자열 연결이 아니다" → 남음 (MySQL 에서는 OR 입니다)
          · "글자 비교가 대소문자를 무시한다" → 남음 (기본 collation 이 _ci)
          · "트랜잭션 안에서 DDL 이 안 되돌아간다" → 남음 (CREATE/ALTER 가 암묵적
          커밋)
          · "RETURNING 이 없다" → 남음 (INSERT 결과를 바로 못 받습니다)
          · "부분 색인이 없다" → 남음 (CREATE INDEX ... WHERE 가 문법 오류)
          · "표 이름 대소문자가 OS 를 탄다" → 남음 (리눅스와 윈도우가 다르게 동작)
```

★★ 위 두 목록을 이 단원에서 **전부 직접 재서** 확인합니다. 개념02 가 “사라진 것”, 개념03·04 가 “남은 것” 입니다.

지금 어디에 쓰이나 — 정직하게

섹션 5

국내 현장 이야기를 있는 그대로 적습니다.

MySQL 이 많은 곳

- SI 로 납품한 사내 시스템 (MES, ERP 애드온, 그룹웨어)
- 제조 현장의 설비 관리·품질 시스템 – 이 자료의 도메인이 여기입니다
- 공공기관 시스템 (조달 규격에 들어가 있는 경우가 많습니다)
- 오래된 워드프레스·그누보드 계열 웹

PostgreSQL 이 많은 곳

- 새로 시작하는 스타트업 서비스
- Supabase·Neon·Vercel·Railway 같은 요즘 호스팅
- 지리정보(PostGIS), 분석, 벡터 검색

★ 그래서 이 자료는 PostgreSQL 로 가르치고, MySQL 로 옮기는 법을 알려 줍니다. 반대로 배우면 더 어렵습니다. 이유는 개념05 에서 설명합니다.

★ 취업 관점에서 정직하게 말하면 국내에서 “DB 다룰 줄 안다” 는 자리 중 MySQL 을 쓰는 곳이 아직 더 많습니다. 그런데 **SQL 자체는 90% 가 같습니다.** 다른 10% 가 이 단원입니다.

★ 이 단원의 실측 환경

섹션 6

자료에 적힌 값이 어떤 환경에서 나온 것인지 밝혀 두어야 합니다. 값이 다르면 학생이 자기가 틀린 줄 알고 헤맬니다.

```
const 피지버전 = await 피지("SHOW server_version");
const 마이버전 = await 마이("SELECT version() AS v");

console.log(`PostgreSQL 버전: ${피지버전.값[0].server_version}`);
```

출력 PostgreSQL 버전: 18.6
값이 다를 수 있음

```
console.log(`MySQL 버전: ${마이버전.값[0].v}`);
```

출력 MySQL 버전: 8.4.11
값이 다를 수 있음

★ 버전은 기계마다 다를 수 있습니다. 그래서 판정을 따로 짭니다.

```
const 피지주버전 = Number(피지버전.값[0].server_version.split(".")[0]);
const 마이주버전 = Number(마이버전.값[0].v.split(".")[0]);

console.log("PostgreSQL 이 14 이상인가:", 피지주버전 >= 14);
```

출력 PostgreSQL 이 14 이상인가: true

```
console.log("MySQL 이 8 이상인가:", 마이주버전 >= 8);
```

출력 MySQL 이 8 이상인가: true

★★ MySQL 은 버전보다 설정이 중요합니다. 이 세 줄을 꼭 보세요.

```
const 설정들 = await 마이(  
  "SELECT @@sql_mode AS 모드, @@collation_database AS 정렬규칙,  
  @@transaction_isolation AS 격리수준, @@lower_case_table_names AS 표이름접기",  
);  
const 설정 = 설정들.값[0];  
  
console.log("MySQL sql_mode:", 설정.모드);
```

출력 MySQL sql_mode:
IGNORE_SPACE,ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZE
RO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION

```
console.log("MySQL collation:", 설정.정렬규칙);
```

출력 MySQL collation: utf8mb4_0900_ai_ci

```
console.log("MySQL 격리수준:", 설정.격리수준);
```

출력 MySQL 격리수준: REPEATABLE-READ

```
console.log("MySQL lower_case_table_names:", 설정.표이름접기);
```

출력 MySQL lower_case_table_names: 0

```
const 피지격리 = await 피지("SHOW transaction_isolation");  
console.log("PostgreSQL 격리수준:", 피지격리.값[0].transaction_isolation);
```

출력 PostgreSQL 격리수준: read committed

★ 벌써 두 개가 다릅니다. · 격리수준 기본값이 다릅니다 (read committed vs REPEATABLE-READ) — 개념05 · MySQL 의 collation 이 _ci 로 끝납니다. 대소문자를 무시한다는 뜻입니다 — 개념03

★★ IGNORE_SPACE 가 sql_mode 에 보이는 것이 이상할 수 있습니다. 서버 전역 설정에는 없는데 접속에는 붙어 있습니다. 드라이버(mysql2)가 접속할 때 그 옵션을 켜기 때문입니다. **접속마다 sql_mode 가 다를 수 있다** 는 뜻입니다. 개념02 에서 다시 봅니다.

```
const 전역모드 = await 마이("SELECT @@GLOBAL.sql_mode AS m");  
console.log("서버 전역 sql_mode 에 IGNORE_SPACE 가 있나:", 전역모드.값  
[0].m.includes("IGNORE_SPACE"));
```

출력 서버 전역 sql_mode 에 IGNORE_SPACE 가 있나: false

```
console.log("내 접속 sql_mode 에 IGNORE_SPACE 가 있나:", 설정.
모드.includes("IGNORE_SPACE"));
```

```
출력      내 접속 sql_mode 에 IGNORE_SPACE 가 있나: true
```

90% 는 같습니다

섹션 7

겉먹을 필요 없다는 것을 먼저 보여 드립니다. 03단원~05단원에서 배운 것 대부분이 그대로 통합니다.

```
const 같은것들 = [
  ["SELECT ... WHERE ... ORDER BY", "SELECT 1 AS x WHERE 1=1 ORDER BY x"],
  ["INNER JOIN / LEFT JOIN", "SELECT 1 AS x FROM (SELECT 1) t LEFT JOIN (SELECT 1) u
ON 1=1"],
  ["GROUP BY + HAVING", "SELECT COUNT(*) AS c FROM (SELECT 1 AS v) t GROUP BY v
HAVING COUNT(*)>0"],
  ["WITH (CTE)", "WITH 임시 AS (SELECT 1 AS v) SELECT v FROM 임시"],
  ["윈도 함수", "SELECT ROW_NUMBER() OVER (ORDER BY v) AS r FROM (SELECT 1 AS v) t"],
  ["CASE WHEN", "SELECT CASE WHEN 1=1 THEN 'ㅇ' ELSE '즈' END AS r"],
  ["COALESCE", "SELECT COALESCE(NULL, 7) AS r"],
  ["LIMIT / OFFSET", "SELECT 1 AS x LIMIT 1 OFFSET 0"],
];

let 같이된것 = 0;

for (const [이름, 문장] of 같은것들) {
  const 가 = await 피지(문장);
  const 나 = await 마이(문장);
  const 판정 = 가.됨 && 나.됨 ? "양쪽 다 됨" : `가:${가.됨} 나:${나.됨}`;

  if (가.됨 && 나.됨) 같이된것 += 1;
  console.log(` ${이름} - ${판정}`);
}
```

```
출력      SELECT ... WHERE ... ORDER BY - 양쪽 다 됨
          INNER JOIN / LEFT JOIN - 양쪽 다 됨
          GROUP BY + HAVING - 양쪽 다 됨
          WITH (CTE) - 양쪽 다 됨
          윈도 함수 - 양쪽 다 됨
          CASE WHEN - 양쪽 다 됨
          COALESCE - 양쪽 다 됨
          LIMIT / OFFSET - 양쪽 다 됨
```

```
console.log(`${같은것들.length} 개 중 양쪽에서 그대로 도는 것: ${같이된것} 개`);
```

```
출력      8 개 중 양쪽에서 그대로 도는 것: 8 개
```

★ 이게 이 단원을 배우는 자세입니다. “전혀 다른 것을 새로 배운다” 가 아니라 “90% 는 같고, 다른 10% 를 정확히 외운다” 입니다.

표 이름·칸 이름을 감싸는 따옴표가 다릅니다

섹션 8

이건 아주 자주 걸립니다. 미리 봐 둡니다.

```
const 큰따옴표피지 = await 피지(`SELECT 1 AS "칸 이름"`);
const 큰따옴표마이 = await 마이(`SELECT 1 AS "칸 이름"`);
console.log("큰따옴표를 이름으로 - PG:", 한줄로(큰따옴표피지));
```

출력 큰따옴표를 이름으로 - PG: [{"칸 이름":1}]

```
console.log("큰따옴표를 이름으로 - MySQL:", 한줄로(큰따옴표마이));
```

출력 큰따옴표를 이름으로 - MySQL: [{"칸 이름":1}]

여기까지는 같습니다. 그런데 큰따옴표 안에 글자를 넣으면 갈립니다.

```
const 큰따옴표글자피지 = await 피지(`SELECT "설비" AS r`);
const 큰따옴표글자마이 = await 마이(`SELECT "설비" AS r`);
console.log(`SELECT "설비" - PG:`, 한줄로(큰따옴표글자피지));
```

출력 SELECT "설비" - PG: 거절 42703

```
console.log(`SELECT "설비" - MySQL:`, 한줄로(큰따옴표글자마이));
```

출력 SELECT "설비" - MySQL: [{"r":"설비"}]

★★★ PostgreSQL 은 큰따옴표를 **이름**으로만 봅니다. 그래서 “그런 칸 없다” 고 합니다. MySQL 은 큰따옴표를 **글자**로도 받습니다. → 글자는 항상 작은따옴표를 쓰세요. 그러면 양쪽에서 같습니다.

```
const 역따옴표피지 = await 피지("SELECT 1 AS `칸`");
const 역따옴표마이 = await 마이("SELECT 1 AS `칸`");
console.log("역따옴표(`) - PG:", 한줄로(역따옴표피지));
```

출력 역따옴표(`) - PG: 거절 42601

```
console.log("역따옴표(`) - MySQL:", 한줄로(역따옴표마이));
```

출력 역따옴표(`) - MySQL: [{"칸":1}]

★ 역따옴표는 MySQL 전용입니다. PostgreSQL 은 문법 오류입니다. MySQL 예제를 인터넷에서 복사해 오면 여기서 먼저 터집니다.

정리 — 왜 둘로 갈렸나

항목 PostgreSQL MySQL

출신	1986 대학 연구	1995 웹 회사
목표	정확함·표준	속도·단순함
주인	없음(커뮤니티)	오라클
라이선스	PostgreSQL License GPL(무료) + 상용	

저장 방식 하나뿐 엔진을 고름(기본 InnoDB) 스키마 DB 안에 여러 개 DB 가 곧 스키마 이름 감싸기
 “큰따옴표” 역따옴표 0 으로 나누기 거절 NULL 기본 격리수준 read committed REPEATABLE-READ

★ 그리고 SQL 의 90% 는 같습니다. 위에서 8개를 재서 확인했습니다.

★★ 다음 파일이 이 단원의 핵심입니다. 개념02 에서 “MySQL 은 허술하다”는 옛말을 직접 뒤집습니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 1 의 MyISAM 실험을 InnoDB 로 바꿔 보세요. `ENGINE=MyISAM` 을 `ENGINE=InnoDB` 로만 고치면 됩니다. ROLLBACK 뒤에 줄이 남나요? 남지 않아야 정상입니다.

▫ 직접 해보기 2

섹션 7 의 같은것들 목록에 SQL 을 세 개 더 넣어 보세요. (힌트: `DISTINCT`, `UNION ALL`, `EXISTS`) 양쪽에서 다 되나요? 안 되는 것을 찾으면 그것이 이 단원의 재료입니다.

▫ 직접 해보기 3

`SELECT @@sql_mode` 를 찍고, 각 항목을 검색해 보세요. `NO_ZERO_DATE` 가 무슨 뜻인지 말로 설명할 수 있나요?

▫ 직접 해보기 4

Postgres 에서 `SHOW ALL` 을 실행해 보세요. 설정이 몇 개인가요? MySQL 에서는 `SHOW VARIABLES` 입니다. 어느 쪽이 많나요?

▫ 직접 해보기 5

`SELECT version()` 을 양쪽에서 실행해 보세요. MySQL 쪽 결과에 “MariaDB” 라는 글자가 들어 있나요? (이 자료의 docker 환경은 진짜 MySQL 이라 없습니다. 회사 서버에서는 꼭 확인해 보세요)

섹션 8 을 응용해서, 이름에 공백이 들어간 표를 양쪽에서 만들어 보세요. `CREATE TABLE “점검 기록” (...)` 과 `CREATE TABLE 점검 기록 (...)`. ★ 그리고 실무에서는 이름에 공백을 쓰지 마세요. 고생만 합니다.

자주 하는 실수

이런 실수를 합니다

인터넷의 오래된 비교 글을 그대로 믿는다

“MySQL 은 트랜잭션이 없다”, “CHECK 를 무시한다” 는 끝난 이야기입니다. 글의 작성 연도를 먼저 보세요. 2015년 이전 글은 절반이 틀립니다.

이런 실수를 합니다

MySQL 을 “허술한 DB” 로 깎아내린다

면접에서 이렇게 말하면 감점입니다. 8.4 는 엄격합니다. 차이는 엄격함이 아니라 기본값과 없는 기능입니다.

이런 실수를 합니다

버전만 보고 안심한다

★★ MySQL 은 같은 8.4 라도 `sql_mode` 에 따라 완전히 다르게 동작합니다. 버전이 아니라 설정을 확인하세요. 개념02 에서 직접 재 봅니다.

이런 실수를 합니다

MySQL 인 줄 알았는데 MariaDB

`SELECT version()` 을 안 찍어 봐서 나중에 문법이 안 맞아 고생합니다. 접속하자마자 버전 문자열부터 보는 습관을 들이세요.

이런 실수를 합니다

표 이름을 역따옴표로 감싼 SQL 을 Postgres 에 그대로 붙인다

`SELECT * FROM `설비`` 는 Postgres 에서 42601 문법 오류입니다. 반대로 `SELECT * FROM “설비”` 는 MySQL 에서 돌긴 하지만 `ANSI_QUOTES`` 모드가 아니면 의미가 달라질 수 있습니다.

이런 실수를 합니다

스키마와 데이터베이스를 같은 말로 안다

Postgres 에서는 다릅니다. 하나의 데이터베이스 안에 스키마가 여럿입니다. MySQL 에서는 같은 말입니다. `CREATE SCHEMA = CREATE DATABASE` 입니다.


```
await PG.end();  
await MY.end();
```

옛날 이야기를 뒤집습니다

RUN node 개념02_옛날_이야기를_뒤집습니다.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

인터넷에서 이런 문장을 보셨을 겁니다.

“MySQL 은 관대해서 이상한 값이 조용히 들어간다.” “숫자 칸에 글자를 넣으면 0 이 들어간다.”

“VARCHAR(5) 에 10글자를 넣으면 앞 5글자만 남고 잘린다.” “GROUP BY 에 없는 칸을 아무렇게나 골라도 받아 준다.”

이 문장들은 MySQL 8.4 에서는 전부 틀렸습니다.

옛날 MySQL(5.6 이하) 이야기입니다. 5.7 과 8.0 을 지나며 `STRICT_TRANS_TABLES` 와 `ONLY_FULL_GROUP_BY` 가 **기본값**이 되었습니다. 지금은 PostgreSQL 과 거의 똑같이 거절합니다.

믿으라고 하지 않겠습니다. **직접 여섯 가지를 넣어 보겠습니다.**

그리고 이 파일의 진짜 결론은 뒷부분에 있습니다. ★★ 그 엄격함은 **설정**이라서 끌 수 있습니다.

끄면 옛날 그대로 조용히 잘립니다. 그것도 직접 재 봅니다.

```
import pg from "pg";
import mysql from "mysql2/promise";
```

불기·도우미

섹션 0

```
let PG = null;
let MY = null;

try {
  PG = new pg.Client({ host: "127.0.0.1", port: 5434, user: "factory", password:
"secret", database: "factory_db" });
  await PG.connect();
  MY = await mysql.createConnection({ host: "127.0.0.1", port: 3307, user: "factory",
password: "secret", database: "factory_db" });
} catch (에러) {
  console.log("★ 데이터베이스에 못 붙었습니다:", 에러.code || 에러.message);
  console.log(" docker compose up -d 를 먼저 실행하세요.");
  if (PG) await PG.end().catch(() => {});
  process.exit(0);
}
```

```

}

await PG.query("CREATE SCHEMA IF NOT EXISTS 단원09");
await PG.query("SET search_path TO 단원09");

async function 피지(sql, 값 = []) {
  try {
    const 결과 = await PG.query(sql, 값);
  }
}

```

★ 조회면 줄을 보여 주고, 넣기·고치기면 몇 건인지 보여 줍니다.

```

const 조회인가 = 결과.command === "SELECT" || 결과.command === "SHOW";

return { 됴: true, 값: 결과.rows, 조회: 조회인가, 건수: 결과.rowCount ?? 0 };
} catch (에러) {
  return { 됴: false, 코드: 에러.code };
}
}

async function 마이(sql, 값 = []) {
  try {
    const [행들] = await MY.query(sql, 값);
  }
}

```

★ mysql2 는 조회면 배열을, 넣기·고치기면 요약 객체를 줍니다.

```

const 조회인가 = Array.isArray(행들);

return {
  됴: true,
  값: 조회인가 ? 행들 : [],
  조회: 조회인가,
  건수: 조회인가 ? 행들.length : 행들.affectedRows,
};
} catch (에러) {
}

```

★ MySQL 에러는 code 가 글자입니다(ER...). 숫자는 errno 입니다.

```

return { 됴: false, 코드: 에러.code, 번호: 에러.errno };
}
}

function 짧게(결과) {
  if (!결과.됴) return `거절 ${결과.코드}${결과.번호 === undefined ? "" : "/" + 결과.번호}`;
}

```

조회이거나, RETURNING 처럼 돌려준 줄이 있으면 그 줄을 보여 줍니다.

```

if (결과.조회 || 결과.값.length > 0) return JSON.stringify(결과.값);
return `성공 ${결과.건수}건`;
}

```

같은 뜻의 SQL 을 양쪽에 보내고 나란히 찍습니다. 이 단위 내내 씁니다.

```

async function 나란히(제목, 피자문, 마이문) {
  const 가 = await 피자(피자문);
  const 나 = await 마이(마이문 === undefined ? 피자문 : 마이문);

  console.log(`· ${제목}`);
  console.log(`    PostgreSQL - ${짧게(가)}`);
  console.log(`    MySQL      - ${짧게(나)}`);

  return { 가, 나 };
}

```

같은 표를 양쪽에 만듭니다

섹션 1

★ 여러 번 돌려도 같은 결과가 나오게, 매번 표를 새로 만듭니다.

```

await 피자("DROP TABLE IF EXISTS 설비");
await 마이("DROP TABLE IF EXISTS 구구_설비");

await 피자(`
  CREATE TABLE 설비 (
    id      SERIAL PRIMARY KEY,
    이름    VARCHAR(5),
    수량    INT,
    도입일  DATE,
    라인    VARCHAR(10)
  )
`);

await 마이(`
  CREATE TABLE 구구_설비 (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    이름    VARCHAR(5),
    수량    INT,
    도입일  DATE,
    라인    VARCHAR(10)
  )
`);

```

★ 두 CREATE 문에서 다른 곳은 **자동 번호를 만드는 방법 한 군데**뿐입니다. SERIAL vs INT AUTO_INCREMENT. 나머지는 글자 하나까지 같습니다. (자동 번호 이야기는 개념04 에서 따로 합니다)

```
console.log("표를 양쪽에 만들었습니다.");
```

출력 표를 양쪽에 만들었습니다.

★★★ 여섯 가지 시험

섹션 2

```
console.log("— 시험 1. 숫자 칸에 글자를 넣으면 —");
```

출력 — 시험 1. 숫자 칸에 글자를 넣으면 —

```
await 나란히(
  "INT 칸에 '백개' 넣기",
  "INSERT INTO 설비(수량) VALUES ('백개')",
  "INSERT INTO 구구_설비(수량) VALUES ('백개')",
);
```

출력 · INT 칸에 '백개' 넣기
 PostgreSQL – 거절 22P02
 MySQL – 거절 ER_TRUNCATED_WRONG_VALUE_FOR_FIELD/1366

★ 옛말: “MySQL 은 0 을 넣는다” → 실제: **거절합니다**. PostgreSQL 의 22P02 는 “입력 문법이 타입에 안 맞음” 이라는 뜻입니다. MySQL 의 1366 은 “칸에 맞지 않는 값” 이라는 뜻입니다. 이름만 다르고 결과는 같습니다.

```
console.log("— 시험 2. VARCHAR(5) 에 10글자를 넣으면 —");
```

출력 — 시험 2. VARCHAR(5) 에 10글자를 넣으면 —

```
await 나란히(
  "VARCHAR(5) 에 '가나다라마바사아자차'",
  "INSERT INTO 설비(이름) VALUES ('가나다라마바사아자차')",
  "INSERT INTO 구구_설비(이름) VALUES ('가나다라마바사아자차')",
);
```

출력 · VARCHAR(5) 에 '가나다라마바사아자차'
 PostgreSQL – 거절 22001
 MySQL – 거절 ER_DATA_TOO_LONG/1406

★ 옛말: “MySQL 은 앞 5글자만 남기고 자른다” → 실제: **거절합니다**. ★ 이게 가장 무서운 옛날 동작이었습니다.

주민번호나 계좌번호가 조용히 잘린 채로 저장되는 사고가 실제로 있었습니다.

```
console.log("— 시험 3. INT 범위를 넘기면 —");
```

출력 — 시험 3. INT 범위를 넘기면 —

```
await 나란히(  
  "INT 칸에 999999999",  
  "INSERT INTO 설비(수량) VALUES (999999999)",  
  "INSERT INTO 구구_설비(수량) VALUES (999999999)",  
);
```

출력 · INT 칸에 999999999
 PostgreSQL – 거절 22003
 MySQL – 거절 ER_WARN_DATA_OUT_OF_RANGE/1264

★ INT 는 21억(2,147,483,647) 까지입니다. 그 위는 BIGINT 를 써야 합니다. 옛날 MySQL 은 21억을 넣고 말았습니다. 지금은 거절합니다.

```
console.log("— 시험 4. 있을 수 없는 날짜를 넣으면 —");
```

출력 — 시험 4. 있을 수 없는 날짜를 넣으면 —

```
await 나란히(  
  "날짜 칸에 '0000-00-00'",  
  "INSERT INTO 설비(도입일) VALUES ('0000-00-00')",  
  "INSERT INTO 구구_설비(도입일) VALUES ('0000-00-00')",  
);
```

출력 · 날짜 칸에 '0000-00-00'
 PostgreSQL – 거절 22008
 MySQL – 거절 ER_TRUNCATED_WRONG_VALUE/1292

```
await 나란히(  
  "날짜 칸에 '2026-02-30'",  
  "INSERT INTO 설비(도입일) VALUES ('2026-02-30')",  
  "INSERT INTO 구구_설비(도입일) VALUES ('2026-02-30')",  
);
```

출력 · 날짜 칸에 '2026-02-30'
 PostgreSQL – 거절 22008
 MySQL – 거절 ER_TRUNCATED_WRONG_VALUE/1292

★ ‘0000-00-00’ 은 옛날 MySQL 의 상징 같은 값이었습니다. “날짜 없음” 을 NULL 대신 이걸로 쓰던 습관이 있었습니다. 그 값을 자바스크립트 `new Date()` 에 넣으면 **Invalid Date** 가 나옵니다. 지금은 `NO_ZERO_DATE` 가 기본이라 아예 안 들어갑니다.

```
console.log("— 시험 5. GROUP BY 에 없는 칸을 SELECT 하면 —");
```

```
출력      — 시험 5. GROUP BY 에 없는 칸을 SELECT 하면 —
```

먼저 정상 데이터를 넣습니다.

```
await 피지("INSERT INTO 설비(이름,수량,라인) VALUES ('가',10,'A'),('나',20,'A'),
('다',30,'B')");
await 마이("INSERT INTO 구구_설비(이름,수량,라인) VALUES ('가',10,'A'),('나',20,'A'),
('다',30,'B')");

await 나란히(
  "GROUP BY 라인 인데 이름도 SELECT",
  "SELECT 라인, 이름, SUM(수량) AS 합 FROM 설비 GROUP BY 라인",
  "SELECT 라인, 이름, SUM(수량) AS 합 FROM 구구_설비 GROUP BY 라인",
);
```

```
출력      · GROUP BY 라인 인데 이름도 SELECT
          PostgreSQL – 거절 42803
          MySQL      – 거절 ER_WRONG_FIELD_WITH_GROUP/1055
```

★ 라인 A 에는 이름이 '가' 와 '나' 두 개 있습니다. 어느 것을 보여 줘야 할까요? 답이 없습니다. 그래서 **양쪽 다 거절합니다**. 옛날 MySQL 은 아무거나 하나 골라서 줬습니다. 그래서 화면에 엉뚱한 이름이 떴습니다.

```
console.log("— 시험 6. CHECK 제약을 어기면 —");
```

```
출력      — 시험 6. CHECK 제약을 어기면 —
```

```
await 피지("DROP TABLE IF EXISTS 재고");
await 마이("DROP TABLE IF EXISTS 구구_재고");
await 피지("CREATE TABLE 재고 (부품 VARCHAR(20), 수량 INT CHECK (수량 >= 0))");
await 마이("CREATE TABLE 구구_재고 (부품 VARCHAR(20), 수량 INT CHECK (수량 >= 0))");

await 나란히(
  "수량 >= 0 인데 -5 를 넣기",
  "INSERT INTO 재고 VALUES ('베어링', -5)",
  "INSERT INTO 구구_재고 VALUES ('베어링', -5)",
);
```

```
출력      · 수량 >= 0 인데 -5 를 넣기
          PostgreSQL – 거절 23514
          MySQL      – 거절 ER_CHECK_CONSTRAINT_VIOLATED/3819
```

★★ 이걸 정말 최근에 바뀐 것입니다. MySQL 은 8.0.16(2019년) 전까지 **CHECK** 를 **문법만 받고 무시했습니다**. 에러도 안 났습니다. 그래서 “MySQL 에서 CHECK 는 장식” 이라는 말이 돌았습니다. 지금은 진짜로 검사합니다.

```
console.log("— 덤. NOT NULL 은 어떤가 —");
```

출력 — 덤. NOT NULL 은 어떤가 —

```
await 피지("DROP TABLE IF EXISTS 필수");
await 마이("DROP TABLE IF EXISTS 구구_필수");
await 피지("CREATE TABLE 필수 (이름 VARCHAR(20) NOT NULL)");
await 마이("CREATE TABLE 구구_필수 (이름 VARCHAR(20) NOT NULL)");

await 나란히(
  "NOT NULL 칸에 NULL 넣기",
  "INSERT INTO 필수 VALUES (NULL)",
  "INSERT INTO 구구_필수 VALUES (NULL)",
);
```

출력 · NOT NULL 칸에 NULL 넣기
 PostgreSQL – 거절 23502
 MySQL – 거절 ER_BAD_NULL_ERROR/1048

```
await 나란히(
  "NOT NULL 칸을 아예 빼고 넣기",
  "INSERT INTO 필수 DEFAULT VALUES",
  "INSERT INTO 구구_필수 () VALUES ()",
);
```

출력 · NOT NULL 칸을 아예 빼고 넣기
 PostgreSQL – 거절 23502
 MySQL – 거절 ER_NO_DEFAULT_FOR_FIELD/1364

★ 에러 이름이 다르지만(23502 vs 1364) 결과는 같습니다. 둘 다 막습니다.

몇 대 몇이었나

섹션 3

```
const 시험표 = [
  ["INT 칸에 '백개'", "22P02", "1366"],
  ["VARCHAR(5) 에 10글자", "22001", "1406"],
  ["INT 범위 초과", "22003", "1264"],
  ["날짜 '0000-00-00'", "22008", "1292"],
  ["GROUP BY 위반", "42803", "1055"],
  ["CHECK 위반", "23514", "3819"],
  ["NOT NULL 위반", "23502", "1048"],
];

console.log("[일곱 가지 시험 결과]");
```


출력 [일곱 가지 시험 결과]

```
for (const [무엇, 피지코드, 마이코드] of 시험표) {
  console.log(` ${무엇} - PG ${피지코드} 거절 · MySQL ${마이코드} 거절`);
}
```

출력 INT 칸에 '백개' - PG 22P02 거절 · MySQL 1366 거절
 VARCHAR(5) 에 10글자 - PG 22001 거절 · MySQL 1406 거절
 INT 범위 초과 - PG 22003 거절 · MySQL 1264 거절
 날짜 '0000-00-00' - PG 22008 거절 · MySQL 1292 거절
 GROUP BY 위반 - PG 42803 거절 · MySQL 1055 거절
 CHECK 위반 - PG 23514 거절 · MySQL 3819 거절
 NOT NULL 위반 - PG 23502 거절 · MySQL 1048 거절

```
console.log(`양쪽이 똑같이 거절한 시험: ${시험표.length} / ${시험표.length}`);
```

출력 양쪽이 똑같이 거절한 시험: 7 / 7

★★★ 7 대 7 입니다. **한 개도 다르지 않았습니**다. “MySQL 은 허술하다” 는 말은 여기서 끝났습니다.

왜 그런가 — sql_mode 를 봅니다

섹션 4

```
const 모드결과 = await 마이("SELECT @@SESSION.sql_mode AS m");
const 지금모드 = 모드결과.값[0].m;

console.log("지금 접속의 sql_mode:");
```

출력 지금 접속의 sql_mode:

```
for (const 항목 of 지금모드.split(",")) {
  console.log(` · ${항목}`);
}
```

출력

- IGNORE_SPACE
- ONLY_FULL_GROUP_BY
- STRICT_TRANS_TABLES
- NO_ZERO_IN_DATE
- NO_ZERO_DATE
- ERROR_FOR_DIVISION_BY_ZERO
- NO_ENGINE_SUBSTITUTION

각 항목이 무슨 뜻인지 정리합니다.

STRICT_TRANS_TABLES	*** 잘못된 값을 잘라 넣지 말고 **거절**하라
ONLY_FULL_GROUP_BY	** GROUP BY 에 없는 칸을 SELECT 하지 못하게
NO_ZERO_DATE	'0000-00-00' 을 막음
NO_ZERO_IN_DATE	'2026-00-15' 처럼 달·일이 0 인 것을 막음
ERROR_FOR_DIVISION_BY_ZERO	0 으로 나누기를 경고로 알림 (INSERT 때 의미가 있음)
NO_ENGINE_SUBSTITUTION	없는 엔진을 달라고 하면 단 것으로 바꾸지 말고 에러
IGNORE_SPACE	함수 이름과 괄호 사이 공백을 허용 (드라이버가 켜)

★ 앞의 두 개가 이 파일의 시험 일곱 개를 전부 설명합니다.

```
console.log("STRICT_TRANS_TABLES 켜져 있나:",
지금모드.includes("STRICT_TRANS_TABLES"));
```

출력 STRICT_TRANS_TABLES 켜져 있나: true

```
console.log("ONLY_FULL_GROUP_BY 켜져 있나:",
지금모드.includes("ONLY_FULL_GROUP_BY"));
```

출력 ONLY_FULL_GROUP_BY 켜져 있나: true

★★★ 그런데 이건 설정입니다. 끝 수 있습니다

섹션 5

여기가 이 파일에서 제일 중요한 부분입니다.

위의 엄격함은 MySQL 이라는 프로그램의 성질이 아닙니다. **설정값**입니다. 설정은 바꿀 수 있습니다. 그리고 **옛 서버에는 꺼져 있습니다**.

★ 안전하게 실험하는 법: SET SESSION 을 씁니다. SET GLOBAL 은 서버 전체를 바꿉니다. 남의 접속까지 바꿉니다. 쓰지 마세요. SET SESSION 은 **내 접속에만** 적용됩니다. 접속을 끊으면 사라집니다.

```
const 원래모드 = 지금모드;
```

2010년대 서버에 흔했던 설정으로 바꿔 봅니다.

```
await 마이("SET SESSION sql_mode='NO_ENGINE_SUBSTITUTION'");

const 바뀐모드 = await 마이("SELECT @@SESSION.sql_mode AS m");
console.log("바꾼 뒤 sql_mode:", 바뀐모드.값[0].m);
```

출력 바꾼 뒤 sql_mode: NO_ENGINE_SUBSTITUTION

같은 것을 다시 넣어 봅니다. 이번에는 표를 비우고 시작합니다.

```
await 마이("DELETE FROM 구구_설비");
await 마이("ALTER TABLE 구구_설비 AUTO_INCREMENT = 1");

const 다시넣기 = [
  ["INT 칸에 '백개'", "INSERT INTO 구구_설비(수량) VALUES ('백개')"],
  ["VARCHAR(5) 에 10글자", "INSERT INTO 구구_설비(이름) VALUES ('가나다라마바사아자차')"],
  ["INT 범위 초과", "INSERT INTO 구구_설비(수량) VALUES (9999999999)"],
  ["날짜 '0000-00-00'", "INSERT INTO 구구_설비(도입일) VALUES ('0000-00-00')"],
  ["날짜 '2026-02-30'", "INSERT INTO 구구_설비(도입일) VALUES ('2026-02-30')"],
];

console.log("[STRICT 를 끈 뒤 같은 값을 다시 넣으면]");
```

출력 [STRICT 를 끈 뒤 같은 값을 다시 넣으면]

```
for (const [무엇, 문장] of 다시넣기) {
  const 결과 = await 마이(문장);
  const 경고 = await 마이("SHOW WARNINGS");
  const 경고말 = 경고.값.length === 0 ? "경고조차 없음" : `경고 ${경고.값[0].Code}`;

  console.log(` ${무엇} - ${결과.됨 ? "들어감" : "거절"} · ${경고말}`);
}
```

출력 INT 칸에 '백개' - 들어감 · 경고 1366
 VARCHAR(5) 에 10글자 - 들어감 · 경고 1265
 INT 범위 초과 - 들어감 · 경고 1264
 날짜 '0000-00-00' - 들어감 · 경고조차 없음
 날짜 '2026-02-30' - 들어감 · 경고 1264

★★★ 다섯 개가 전부 들어갔습니다. 그럼 무엇이 들어갔을까요?

```
const 망가진것 = await 마이(
  "SELECT id, 이름, 수량, CAST(도입일 AS CHAR) AS 도입일 FROM 구구_설비 ORDER BY id",
);

console.log("[실제로 저장된 값]");
```

출력 [실제로 저장된 값]

```
for (const 줄 of 망가진것.값) {
  console.log(` ${줄.id}번 - 이름 ${JSON.stringify(줄.이름)} · 수량 ${JSON.stringify(줄.수량)} · 도입일 ${JSON.stringify(줄.도입일)}`);
}
```

```
출력      1번 - 이름 null · 수량 0 · 도입일 null
          2번 - 이름 "가나다라마" · 수량 null · 도입일 null
          3번 - 이름 null · 수량 2147483647 · 도입일 null
          4번 - 이름 null · 수량 null · 도입일 "0000-00-00"
          5번 - 이름 null · 수량 null · 도입일 "0000-00-00"
```

★★★ 옛말이 전부 사실이 됐습니다.

‘백개’ → 0 ‘가나다라마바사아자차’ → ‘가나다라마’ (뒤 다섯 글자가 사라졌습니다)

```
9999999999          → **2147483647** (INT 의 최댓값으로 높였습니다)
'0000-00-00'        → 그대로 들어갔고 **경고조차 없습니다**
'2026-02-30'        → **'0000-00-00'** 이 됐습니다
```

★ 마지막 두 개가 제일 무섭습니다. 2월 30일을 넣었는데 0년 0월 0일이 됐습니다. 그리고 ‘0000-00-00’은 경고도 안 나옵니다. **완전히 조용합니다.**

★★ “그래도 경고는 나오잖아요?” SHOW WARNINGS 를 따로 실행해야 보입니다. 응용 프로그램에서 그걸 확인하는 코드를 짜는 사람은 거의 없습니다. 그래서 실질적으로는 **아무도 모릅니다.**

```
console.log("잘린 이름의 글자 수:", 망가진것.값[1].이름.length);
```

```
출력      잘린 이름의 글자 수: 5
```

```
console.log("원래 넣으려던 글자 수:", "가나다라마바사아자차".length);
```

```
출력      원래 넣으려던 글자 수: 10
```

무엇이 안 꺼지나

섹션 6

★ 좋은 소식도 있습니다. sql_mode 로 못 끄는 것이 있습니다.

```
await 마이("SET SESSION sql_mode='');

const 체크시험 = await 마이("INSERT INTO 구구_재고 VALUES ('베어링', -5)");
console.log("sql_mode 를 통째로 비운 뒤 CHECK 위반:", 짧게(체크시험));
```

```
출력      sql_mode 를 통째로 비운 뒤 CHECK 위반: 거절
          ER_CHECK_CONSTRAINT_VIOLATED/3819
```

```
const 널시험 = await 마이("INSERT INTO 구구_필수 VALUES (NULL)");
console.log("sql_mode 를 통째로 비운 뒤 NOT NULL 위반:", 짧게(널시험));
```

```
출력      sql_mode 를 통째로 비운 뒤 NOT NULL 위반: 거절 ER_BAD_NULL_ERROR/1048
```

★ CHECK 와 NOT NULL 은 **제약(constraint)** 입니다. 설정이 아닙니다. sql_mode 를 아무리 비워도 안 꺼집니다. → 그래서 **중요한 규칙은 sql_mode 가 아니라 제약으로 걸어야 합니다.**

(제약은 02단원에서 했습니다)

★ 반드시 되돌려 놓습니다

섹션 7

```
await 마이("SET SESSION sql_mode=?", [원래모드]);

const 되돌린모드 = await 마이("SELECT @@SESSION.sql_mode AS m");
console.log("되돌린 sql_mode 가 원래와 같나:", 되돌린모드.값[0].m === 원래모드);
```

출력 되돌린 sql_mode 가 원래와 같나: true

```
const 다시거절 = await 마이("INSERT INTO 구구_설비(수량) VALUES ('백개')");
console.log("되돌린 뒤 '백개' 넣기:", 짧게(다시거절));
```

출력 되돌린 뒤 '백개' 넣기: 거절 ER_TRUNCATED_WRONG_VALUE_FOR_FIELD/1366

★ 실험 뒤에는 반드시 원래대로 돌려놓으세요. SET SESSION 만 썼으니 접속을 끊어도 알아서 사라지지만, 같은 접속을 계속 쓰는 코드에서는 다음 쿼리가 이상해집니다.

★★★ 그래서 무엇을 해야 하나

섹션 8

결론은 하나입니다.

MySQL 서버를 처음 만나면 SELECT @@sql_mode 부터 찍어 보세요.

버전이 8.4 라도 설정이 꺼져 있으면 옛날처럼 동작합니다. 반대로 5.7 이라도 설정이 켜져 있으면 엄격합니다. **버전이 아니라 설정이 문제입니다.**

진단하는 함수를 만들어 두면 편합니다.

```
function 엄격한가(모드문자열) {
  const 있어야할것 = ["STRICT_TRANS_TABLES", "ONLY_FULL_GROUP_BY", "NO_ZERO_DATE",
    "NO_ZERO_IN_DATE"];
  const 빠진것 = 있어야할것.filter((항목) => !모드문자열.includes(항목));

  return { 안전한가: 빠진것.length === 0, 빠진것 };
}

const 지금진단 = 엄격한가(원래모드);
console.log("지금 접속 진단 - 안전한가:", 지금진단.안전한가);
```

출력 지금 접속 진단 - 안전한가: true

```
console.log("지금 접속 진단 - 빠진 것:", JSON.stringify(지금진단.빠진것));
```

출력 지금 접속 진단 - 빠진 것: []

```
const 옛서버진단 = 엄격한가("NO_ENGINE_SUBSTITUTION");  
console.log("옛 서버 진단 - 안전한가:", 옛서버진단.안전한가);
```

출력 옛 서버 진단 - 안전한가: false

```
console.log("옛 서버 진단 - 빠진 것:", JSON.stringify(옛서버진단.빠진것));
```

출력 옛 서버 진단 - 빠진 것:
["STRICT_TRANS_TABLES", "ONLY_FULL_GROUP_BY", "NO_ZERO_DATE", "NO_ZERO_IN_DATE"]

★ 빠진 것이 있으면 서버 설정 파일(my.cnf)에서 이렇게 겁니다.

— mysqld

```
sql_mode =  
STRICT_TRANS_TABLES,ONLY_FULL_GROUP_BY,NO_ZERO_DATE,NO_ZERO_IN_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION
```

★★ 그런데 운영 중인 서버에서 갑자기 켜면 **돌던 코드가 멈춥니다**. 지금까지 조용히 잘려 들어가던 INSERT 가 에러가 되기 때문입니다. 순서는 이렇습니다.

① 개발 환경에서 먼저 켜다 ② 터지는 곳을 전부 고친다 ③ 데이터를 점검한다 ('0000-00-00' 이 이미 들어 있을 수 있습니다) ④ 그 다음에 운영에 켜다

★ 접속할 때마다 강제로 켜는 방법도 있습니다. 서버를 못 건드릴 때 씁니다. mysql2 에서는 접속 옵션으로 줍니다.

```
mysql.createConnection({ ..., flags: [] })    // 드라이버 옵션은 따로  
await 연결.query("SET SESSION sql_mode='STRICT_TRANS_TABLES,ONLY_FULL_GROUP_BY'");
```

접속을 만든 직후에 위 한 줄을 실행하면 그 접속만 안전해집니다.

뒷정리

```
await 피지("DROP TABLE IF EXISTS 설비");  
await 피지("DROP TABLE IF EXISTS 재고");  
await 피지("DROP TABLE IF EXISTS 필수");  
await 마이("DROP TABLE IF EXISTS 구구_설비");  
await 마이("DROP TABLE IF EXISTS 구구_재고");  
await 마이("DROP TABLE IF EXISTS 구구_필수");
```

정리 — 옛말은 어디까지 맞나

시험 PostgreSQL 18 MySQL 8.4 (기본) MySQL 8.4 (STRICT 끄)

INT 칸에 '백개' 거절 22P02 거절 1366 0 이 들어감 VARCHAR(5) 에 10글자 거절 22001 거절 1406 앞 5글자만 남음 INT 범위 초과 거절 22003 거절 1264 2147483647 로 넘림 날짜 '0000-00-00' 거절 22008 거절 1292 그대로 · 경고도 없음 날짜 '2026-02-30' 거절 22008 거절 1292 '0000-00-00' 이 됨 GROUP BY 위반 거절 42803 거절 1055 아무 값이나 골라 줌 CHECK 위반 거절 23514 거절 3819 거절 (안 꺼짐) NOT NULL 위반 거절 23502 거절 1048 거절 (안 꺼짐)

★ 왼쪽 두 칸이 같습니다. **MySQL 8.4 는 PostgreSQL 만큼 엄격합니다.** ★★ 오른쪽 칸이 이 파일의 진짜 교훈입니다. **설정 하나로 옛날로 돌아갑니다.**

→ MySQL 을 만나면 `SELECT @@sql_mode` 를 찍으세요.

버전이 아니라 설정입니다.

★ 다음 파일이 더 위험합니다. 개념03 에서는 **에러조차 안 나면서 결과가 다른 것들**을 봅니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 5 에서 `sql_mode` 를 '`STRICT_ALL_TABLES`' 로 바꿔 보세요. `STRICT_TRANS_TABLES` 와 무엇이 다른가요? (힌트: `TRANS` 는 트랜잭션을 지원하는 엔진에만 적용됩니다. `MyISAM` 표에서는 `STRICT_TRANS_TABLES` 가 약하게 동작합니다)

▫ 직접 해보기 2

`STRICT` 를 끈 채로 `TINYINT` 칸에 300 을 넣어 보세요. 무슨 값이 들어가나요? (힌트: `TINYINT` 는 127 까지입니다)

▫ 직접 해보기 3

`STRICT` 를 끈 채로 `UPDATE` 를 해 보세요. `UPDATE` 구구_설비 SET 수량='백개' 도 조용히 0 이 되나요? ★ `INSERT` 만 조심하면 된다고 생각하기 쉽습니다. 아닙니다.

▫ 직접 해보기 4

`NO_ZERO_DATE` 만 빼고 나머지는 켜 상태를 만들어 보세요. 그 상태에서 '0000-00-00' 이 들어가나요? 설정을 **하나씩** 껴다 켜면서 무엇이 무엇을 막는지 확인해 보세요.

⇒ 직접 해보기 5

PostgreSQL 에는 `sql_mode` 같은 것이 있을까요? `SHOW ALL` 에서 찾아보세요. 없습니다. ★ 없다는 것이 이 비교의 핵심입니다. Postgres 는 고를 수 없어서 안전합니다.

⇒ 직접 해보기 6

이미 '0000-00-00' 이 들어 있는 표를 어떻게 고칠까요? `UPDATE ... SET 도입일 = NULL WHERE 도입일 = '0000-00-00'` 을 `STRICT` 를 켜 상태에서 실행하면 될까요? 직접 해 보세요.

⇒ 직접 해보기 7

섹션 8 의 `엄격한가()` 함수에 `ANSI_QUOTES` 검사도 넣어 보세요. 그 모드가 켜지면 큰따옴표의 뜻이 바뀝니다 (개념01 섹션 8 참고).

자주 하는 실수

이런 실수를 합니다

"MySQL 8 이니까 안전하다" 고 끝낸다

★★★ 버전이 아니라 설정입니다. 회사 서버에서 `SELECT @@sql_mode` 를 찍어 보면 비어 있는 경우가 실제로 있습니다. 옛 버전에서 올린 서버는 옛 설정을 그대로 들고 옵니다.

이런 실수를 합니다

전역 설정만 보고 안심한다

`@@GLOBAL.sql_mode` 와 `@@SESSION.sql_mode` 는 다를 수 있습니다. 드라이버가 붙을 때 바꾸기도 하고, 프레임워크가 바꾸기도 합니다. ★ 실제로 이 자료의 접속도 `IGNORE_SPACE` 가 붙어 있습니다 (개념01 섹션 6). 내 접속의 값을 보세요.

이런 실수를 합니다

SET GLOBAL 로 실험한다

서버 전체가 바뀝니다. 다른 사람 접속까지 영향을 받습니다. 실험은 항상 `SET SESSION` 으로 하세요. 그리고 되돌려 놓으세요.

이런 실수를 합니다

경고를 안 본다

`STRICT` 가 꺼진 서버에서는 값이 망가져도 **에러가 아니라 경고**입니다. `SHOW WARNINGS` 를 안 보면 영원히 모릅니다. ★ 더 무서운 것: '0000-00-00' 은 경고조차 안 났습니다. 위에서 확인했습니다.

이런 실수를 합니다

운영 서버에서 갑자기 STRICT 를 켜다

지금까지 조용히 들어가던 INSERT 가 전부 에러가 됩니다. 새벽에 배포하고 아침에 장애가 납니다. 개발 환경에서 먼저 켜세요.

이런 실수를 합니다

중요한 규칙을 sql_mode 에 맡긴다

sql_mode 는 꺾다 켤 수 있습니다. **제약(NOT NULL, CHECK, UNIQUE, 외래키)**은 못 끔니다. 섹션 6 에서 확인했습니다. 규칙은 제약으로 거세요.

이런 실수를 합니다

옛 데이터를 안 보고 설정만 고친다

설정을 켜도 이미 들어가 있는 망가진 값은 그대로 남습니다. WHERE 도입일 = '0000-00-00' 이나 WHERE 수량 = 2147483647 로 먼저 세어 보세요.

```
await PG.end();
await MY.end();
```

조용히 다른 결과

RUN node 개념03_조용히_다른_결과.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

개념02 에서 본 것들은 사실 **안전한 차이**였습니다. 에러가 났으니까요. 에러는 고칠 수 있습니다.

이 파일에서 보는 것은 **에러가 안 나는 차이**입니다.

같은 SQL 을 보냈는데 양쪽 다 성공했다고 하고 **값이 다릅니다**.

이게 진짜 사고를 냅니다. 옮긴 날에는 아무 일도 없습니다. 테스트도 다 통과합니다. 몇 주 뒤에 "왜 목록이 이상하죠?" 라는 전화가 옵니다.

이 파일에서 여섯 가지를 봅니다. 전부 실제로 양쪽에서 돌립니다.

① || 가 문자열 연결이 아니다 ★★★ ② 글자 비교가 대소문자를 무시한다 ★★★ ③ 뒤 공백을 무시할 때가 있다 ★★ ④ 0 으로 나누면 NULL 이 나온다 ★★ ⑤ 글자를 숫자로 알아서 바꿔 준다 ★★★ ⑥ 글자 수를 세면 바이트가 나온다 ★

```
import pg from "pg";
import mysql from "mysql2/promise";
```

불기 · 도우미

섹션 0

```
let PG = null;
let MY = null;

try {
  PG = new pg.Client({ host: "127.0.0.1", port: 5434, user: "factory", password:
"secret", database: "factory_db" });
  await PG.connect();
  MY = await mysql.createConnection({ host: "127.0.0.1", port: 3307, user: "factory",
password: "secret", database: "factory_db" });
} catch (에러) {
  console.log("★ 데이터베이스에 못 붙었습니다:", 에러.code || 에러.message);
  console.log(" docker compose up -d 를 먼저 실행하세요.");
  if (PG) await PG.end().catch(() => {});
  process.exit(0);
}
```

```

await PG.query("CREATE SCHEMA IF NOT EXISTS 단위09");
await PG.query("SET search_path TO 단위09");

async function 피지(sql, 값 = []) {
  try {
    const 결과 = await PG.query(sql, 값);
  }
}

```

★ 조회면 줄을 보여 주고, 넣기·고치기면 몇 건인지 보여 줍니다.

```

const 조회인가 = 결과.command === "SELECT" || 결과.command === "SHOW";

return { 됨: true, 값: 결과.rows, 조회: 조회인가, 건수: 결과.rowCount ?? 0 };
} catch (에러) {
  return { 됨: false, 코드: 에러.code };
}
}

async function 마이(sql, 값 = []) {
  try {
    const [행들] = await MY.query(sql, 값);
  }
}

```

★ mysql2 는 조회면 배열을, 넣기·고치기면 요약 객체를 줍니다.

```

const 조회인가 = Array.isArray(행들);

return {
  됨: true,
  값: 조회인가 ? 행들 : [],
  조회: 조회인가,
  건수: 조회인가 ? 행들.length : 행들.affectedRows,
};
} catch (에러) {
}

```

★ MySQL 에러는 code 가 글자입니다(ER...). 숫자는 errno 입니다.

```

return { 됨: false, 코드: 에러.code, 번호: 에러.errno };
}
}

function 짧게(결과) {
  if (!결과.됨) return `거절 ${결과.코드}${결과.번호 === undefined ? "" : "/" + 결과.번호}`;
}

```

조회이거나, RETURNING 처럼 돌려준 줄이 있으면 그 줄을 보여 줍니다.

```

if (결과.조회 || 결과.값.length > 0) return JSON.stringify(결과.값);
return `성공 ${결과.건수}건`;

```

```

}

async function 나란히(제목, 피자문, 마이문) {
  const 가 = await 피자(피자문);
  const 나 = await 마이(마이문 === undefined ? 피자문 : 마이문);

  console.log(`· ${제목}`);
  console.log(`    PostgreSQL - ${짧게(가)}`);
  console.log(`    MySQL      - ${짧게(나)}`);

  return { 가, 나 };
}

```

★ MySQL 은 참/거짓이 1/0 으로 옵니다. 비교하기 편하게 바꿔 주는 함수입니다.

```

function 참인가(값) {
  return 값 === true || 값 === 1;
}

```

★★★ || — 같은 기호, 완전히 다른 뜻

섹션 1

```

const 이어붙이기 = await 나란히("설비 이름 만들기 - 'A동' || 'B동'", "SELECT 'A동' || 'B동' AS 결과");

```

```

출력      · 설비 이름 만들기 - 'A동' || 'B동'
           PostgreSQL - [{"결과":"A동B동"}]
           MySQL      - [{"결과":0}]

```

★★★ PostgreSQL 은 ‘A동B동’ 을 찾고, MySQL 은 0 을 찾습니다.

왜 그런가:

표준 SQL 에서 `||` 는 **문자열 연결**입니다. PostgreSQL 은 그대로 따릅니다.
 MySQL 은 `||` 를 **OR** 로 씁니다. C 언어 습관입니다.
 그래서 `A동 || B동` 은 `A동 OR B동` 이 됩니다.
 문자를 참/거짓으로 바꿀 때 숫자로 읽을 수 없으면 0(거짓)이라서 결과가 0 입니다.

★★ 에러가 안 납니다. 이게 핵심입니다. 설비 이름을 만들어 붙이는 코드가 있었다면, 옮긴 뒤 이름이 전부 0 이 됩니다.

```

console.log("PG 결과가 글자인가:", typeof 이어붙이기.가.값[0].결과 === "string");

```

```

출력      PG 결과가 글자인가: true

```

```

console.log("MySQL 결과가 숫자인가:", typeof 이어붙이기.나.값[0].결과 === "number");

```

출력 MySQL 결과가 숫자인가: true

★★★ 그리고 값에 따라 0 이 되기도 하고 1 이 되기도 합니다. 더 나쁩니다.

```
await 나란히("숫자로 시작하는 글자를 붙이면 - 'A동' || '3호'", "SELECT 'A동' || '3호' AS 결과");
```

출력 · 숫자로 시작하는 글자를 붙이면 - 'A동' || '3호'

PostgreSQL - [{"결과": "A동3호"}]

MySQL - [{"결과": 1}]

★ '3호' 는 숫자 3 으로 읽힙니다. 0 OR 3 은 참이라서 1 이 됐습니다. 같은 코드가 데이터에 따라 0 을 주기도 1 을 주기도 합니다. "값이 0 이면 문제" 라고 찍어 보는 점검 코드도 이걸 못 잡습니다.

```
await 나란히("정말 OR 이 맞는지 - 1 || 0", "SELECT 1 || 0 AS 결과");
```

출력 · 정말 OR 이 맞는지 - 1 || 0

PostgreSQL - 거절 42883

MySQL - [{"결과": 1}]

★ 1 || 0 이 MySQL 에서 1 입니다. 1 OR 0 이 참이니깐요. OR 이 맞습니다. PostgreSQL 은 "숫자 || 숫자 라는 연산자는 없다"(42883) 고 합니다.

— 어떻게 맞추나 ㉠: MySQL 에서 || 를 연결로 만들기

```
const 원래모드 = (await 마이("SELECT @@SESSION.sql_mode AS m")).값[0].m;

await 마이("SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode, ',PIPES_AS_CONCAT')");

const کنڈ = await 마이("SELECT 'A동' || 'B동' AS 결과");
console.log("PIPES_AS_CONCAT کن ڈ MySQL:", 짧게(کنڈ));
```

출력 PIPES_AS_CONCAT کن ڈ MySQL: [{"결과": "A동B동"}]

```
await 마이("SET SESSION sql_mode = ?", [원래모드]);

const 끈ڈ = await 마이("SELECT 'A동' || 'B동' AS 결과");
console.log("되돌린 뒤 MySQL:", 짧게(끈ڈ));
```

출력 되돌린 뒤 MySQL: [{"결과": 0}]

★ PIPES_AS_CONCAT 을 켜면 됩니다. 그런데 **기본이 아닙니다**. 그리고 이 설정을 켜면 || 를 OR 로 쓰던 옛 코드가 망가집니다. 운영 중인 서버에서 함부로 못 켵니다.

— 어떻게 맞추나 ㉡: 양쪽에서 다 되는 문법을 쓰기

```
await 나란히("concat() 으로 쓰면", "SELECT concat('A동', 'B동') AS 결과");
```

출력 · concat() 으로 쓰면
 PostgreSQL – [{"결과":"A동B동"}]
 MySQL – [{"결과":"A동B동"}]

★★★ 이게 정답입니다. `concat()` 은 양쪽에서 똑같이 동작합니다. 두 DB 를 다 지원해야 하는 코드라면 `||` 를 아예 쓰지 마세요.

★ 주의: NULL 을 다루는 방식은 다릅니다. PostgreSQL 의 `concat()` 은 NULL 을 빈 글자로 봅니다. PostgreSQL 의 `||` 는 NULL 이 하나라도 있으면 전체가 NULL 입니다.

```
await 나란히("concat 에 NULL 이 섞이면", "SELECT concat('A', NULL, 'B') AS 결과");
```

출력 · concat 에 NULL 이 섞이면
 PostgreSQL – [{"결과":"AB"}]
 MySQL – [{"결과":null}]

★★ 여기도 다릅니다. PostgreSQL 은 'AB', MySQL 은 NULL 입니다. MySQL 의 `CONCAT()` 은 NULL 이 하나라도 있으면 전체가 NULL 입니다. → NULL 이 섞일 수 있으면 `COALESCE(칸, '')` 로 감싸세요. 양쪽에서 같아집니다.

```
await 나란히("COALESCE 로 감싸면", "SELECT concat('A', COALESCE(NULL, ''), 'B') AS 결과");
```

출력 · COALESCE 로 감싸면
 PostgreSQL – [{"결과":"AB"}]
 MySQL – [{"결과":"AB"}]

★★★ 대소문자 — 로그인과 중복확인이 걸립니다

섹션 2

```
await 나란히("'ABC' = 'abc'", "SELECT ('ABC' = 'abc') AS 결과");
```

출력 · 'ABC' = 'abc'
 PostgreSQL – [{"결과":false}]
 MySQL – [{"결과":1}]

왜 그런가 — **collation(정렬·비교 규칙)** 때문입니다.

collation 은 “글자를 어떻게 비교하고 어떻게 정렬할지” 를 정한 규칙표입니다. MySQL 의 기본은 `utf8mb4_0900_ai_ci` 입니다. 이름 뒤 두 글자가 핵심입니다.

ai = accent insensitive – 악센트를 무시합니다 (e 와 é 를 같게 봅니다)
 ci = case insensitive – 대소문자를 무시합니다 (A 와 a 를 같게 봅니다)

PostgreSQL 의 기본은 운영체제 규칙을 따르는 정렬이고, **글자 그대로 비교**합니다.

```
const 콜레이션 = await 마이("SELECT @@collation_database AS c");
console.log("MySQL 표의 기본 collation:", 콜레이션.값[0].c);
```

```
출력      MySQL 표의 기본 collation: utf8mb4_0900_ai_ci
```

```
await 나란히("악센트도 무시할까 - 'e' = 'é'", "SELECT ('e' = 'é') AS 결과");
```

```
출력      · 악센트도 무시할까 - 'e' = 'é'
          PostgreSQL - [{"결과":false}]
          MySQL      - [{"결과":1}]
```

★ ai 가 진짜로 동작합니다. 프랑스어·독일어 이름을 다루면 이게 문제가 됩니다.

★★ 실제 시나리오: 아이디 중복확인

현장 코드가 보통 이렇게 생겼습니다.

① SELECT COUNT(*) FROM 작업자 WHERE 아이디 = ? ← 중복확인 ② 0 이면 INSERT

이 코드가 양쪽에서 **정반대로** 동작합니다. 직접 해 봅니다.

```
await 피지("DROP TABLE IF EXISTS 작업자");
await 마이("DROP TABLE IF EXISTS 구구_작업자");
await 피지("CREATE TABLE 작업자 (아이디 VARCHAR(30) UNIQUE, 이름 VARCHAR(20))");
await 마이("CREATE TABLE 구구_작업자 (아이디 VARCHAR(30) UNIQUE, 이름 VARCHAR(20))");
```

관리자가 'Admin' 으로 먼저 가입합니다.

```
await 피지("INSERT INTO 작업자 VALUES ('Admin', '김반장')");
await 마이("INSERT INTO 구구_작업자 VALUES ('Admin', '김반장')");
```

이제 누군가 'admin' 으로 가입하려고 합니다. 먼저 중복확인을 합니다.

```
const 확인피지 = await 피지("SELECT COUNT(*)::int AS 개수 FROM 작업자 WHERE 아이디 = 'admin'");
const 확인마이 = await 마이("SELECT COUNT(*) AS 개수 FROM 구구_작업자 WHERE 아이디 = 'admin'");

console.log("중복확인 결과 - PG:", 확인피지.값[0].개수, "· MySQL:", 확인마이.값[0].개수);
```

```
출력      중복확인 결과 - PG: 0 · MySQL: 1
```

★★★ 벌써 같았습니다. PostgreSQL 은 “그런 아이디 없습니다” 라고 합니다 → 가입 화면이 통과됩니다.
MySQL 은 “이미 있습니다” 라고 합니다 → 막습니다.

```
const 가입피지 = await 피지("INSERT INTO 작업자 VALUES ('admin', '이반장')");
const 가입마이 = await 마이("INSERT INTO 구구_작업자 VALUES ('admin', '이반장')");

console.log("실제 가입 - PG:", 짧게(가입피지));
```

출력 실제 가입 - PG: 성공 1건

```
console.log("실제 가입 - MySQL:", 짧게(가입마이));
```

출력 실제 가입 - MySQL: 거절 ER_DUP_ENTRY/1062

```
const 목록피지 = await 피지("SELECT 아이디, 이름 FROM 작업자 ORDER BY 아이디");
const 목록마이 = await 마이("SELECT 아이디, 이름 FROM 구구_작업자 ORDER BY 아이디");

console.log("PG 에 남은 계정:", 짧게(목록피지));
```

출력 PG 에 남은 계정: [{"아이디":"Admin","이름":"김반장"},
{"아이디":"admin","이름":"이반장"}]

```
console.log("MySQL 에 남은 계정:", 짧게(목록마이));
```

출력 MySQL 에 남은 계정: [{"아이디":"Admin","이름":"김반장"}]

★★★ 결과를 정리하면 이렇습니다.

PostgreSQL — ‘Admin’ 과 ‘admin’ 이 **다른 계정 두 개**가 됐습니다. MySQL — UNIQUE 색인이 같은 값으로 보고 **막았습니다**.

★ 놀랐을 겁니다. “MySQL 이 허술하다” 는 예상과 정반대입니다. 여기서는 **MySQL 이 더 안전하게 동작했습니다**. 대신 MySQL 은 대소문자를 구분해야 하는 곳(비밀번호 해시, 토큰, 상품 코드)에서 같은 이유로 위험합니다. 좋고 나쁨의 문제가 아니라 **다름**의 문제입니다.

★★ 옮길 때 무슨 일이 나나 — 양쪽 다 사고입니다.

MySQL → PostgreSQL 로 옮기면

```
지금까지 막히던 'Admin'/'admin' 이 **둘 다 가입됩니다.**
로그인 화면에서 어느 계정으로 들어갈지 알 수 없게 됩니다. 보안 사고입니다.
```

PostgreSQL → MySQL 로 옮기면

```
이미 들어 있는 'Admin' 과 'admin' 을 옮기다가
**UNIQUE 충돌(1062)로 이관이 멈춥니다.**
밤새 데이터를 옮기다 새벽 3시에 여기서 막힙니다.
```


어떻게 맞추나

방법 ① PostgreSQL 쪽을 MySQL 처럼 만들기 — 소문자 유일 색인

```
await 피지("DELETE FROM 작업자");
await 피지("CREATE UNIQUE INDEX 작업자_아이디_소문자 ON 작업자 (lower(아이디))");

const 소문자1 = await 피지("INSERT INTO 작업자 VALUES ('Admin','김반장')");
const 소문자2 = await 피지("INSERT INTO 작업자 VALUES ('admin','이반장')");

console.log("소문자 유일 색인 뒤 'Admin':", 짧게(소문자1));
```

출력 소문자 유일 색인 뒤 'Admin': 성공 1건

```
console.log("소문자 유일 색인 뒤 'admin':", 짧게(소문자2));
```

출력 소문자 유일 색인 뒤 'admin': 거절 23505

★★ 이제 PostgreSQL 도 MySQL 처럼 막습니다. ★ 이때 조회도 `WHERE lower(아이디) = lower($1)` 로 해야 색인을 탑니다. ★ `citext` 확장을 쓰는 방법도 있습니다. 칸 타입을 아예 대소문자 무시로 만듭니다.

```
const 시텍스트 = await 피지("SELECT COUNT(*)::int AS c FROM pg_available_extensions
WHERE name='citext'");
console.log("이 서버에 citext 확장이 있나:", 시텍스트.값[0].c > 0);
```

출력 이 서버에 citext 확장이 있나: true

방법 ② MySQL 쪽을 PostgreSQL 처럼 만들기 — collation 을 바꾸기

```
await 마이("DROP TABLE IF EXISTS 구구_구분작업자");
await 마이("CREATE TABLE 구구_구분작업자 (아이디 VARCHAR(30) COLLATE
utf8mb4_0900_as_cs UNIQUE)");

const 구분1 = await 마이("INSERT INTO 구구_구분작업자 VALUES ('Admin')");
const 구분2 = await 마이("INSERT INTO 구구_구분작업자 VALUES ('admin')");

console.log("as_cs collation 에서 'Admin':", 짧게(구분1));
```

출력 as_cs collation 에서 'Admin': 성공 1건

```
console.log("as_cs collation 에서 'admin':", 짧게(구분2));
```

출력 as_cs collation 에서 'admin': 성공 1건

```
const 구분목록 = await 마이("SELECT 아이디 FROM 구구_구분작업자 ORDER BY BINARY
아이디");
console.log("as_cs 표에 남은 것:", 짧게(구분목록));
```

```
출력      as_cs 표에 남은 것: [{"아이디":"Admin"}, {"아이디":"admin"}]
```

★ as_cs = accent sensitive, case sensitive. 이제 PostgreSQL 과 같아졌습니다. ★★ 칸 하나씩 지정할 수도 있고, 표·데이터베이스 단위로도 지정합니다.

그런데 이미 데이터가 들어 있는 표의 collation 을 바꾸면 색인을 다시 만들어야 합니다.
설계할 때 정하세요.

★ 쿼리 한 번만 다르게 비교하고 싶을 때는 COLLATE 를 붙입니다.

```
const 한번만 = await 마이("SELECT ('ABC' COLLATE utf8mb4_0900_as_cs = 'abc') AS
결과");
console.log("한 번만 구분해서 비교:", 짧게(한번만));
```

```
출력      한 번만 구분해서 비교: [{"결과":0}]
```

★★ 뒤 공백 — 여기서 실측이 예상을 뒤집었습니다

섹션 3

많은 자료에 “MySQL 은 뒤 공백을 무시한다” 고 적혀 있습니다. 재 봤더니 **반만 맞았습니다**. 어느 collation 이냐에 따라 다릅니다.

```
await 나란히("글자끼리 비교 - 'a ' = 'a'", "SELECT ('a ' = 'a') AS 결과");
```

```
출력      · 글자끼리 비교 - 'a ' = 'a'
           PostgreSQL - [{"결과":false}]
           MySQL      - [{"결과":1}]
```

여기까지는 “무시한다” 가 맞아 보입니다. 그런데 **표의 칸**으로 하면 다릅니다.

```

await 마이("DROP TABLE IF EXISTS 구구_새칸");
await 마이("DROP TABLE IF EXISTS 구구_옛칸");
await 마이("CREATE TABLE 구구_새칸 (값 VARCHAR(20) COLLATE utf8mb4_0900_ai_ci
UNIQUE)");
await 마이("CREATE TABLE 구구_옛칸 (값 VARCHAR(20) COLLATE utf8mb4_general_ci
UNIQUE)");

await 마이("INSERT INTO 구구_새칸 VALUES ('admin')");
await 마이("INSERT INTO 구구_옛칸 VALUES ('admin')");

const 새칸공백 = await 마이("INSERT INTO 구구_새칸 VALUES ('admin ')");
const 옛칸공백 = await 마이("INSERT INTO 구구_옛칸 VALUES ('admin ')");

console.log("MySQL 8 기본 collation 칸에 'admin ' 추가:", 짧게(새칸공백));

```

출력 MySQL 8 기본 collation 칸에 'admin ' 추가: 성공 1건

```

console.log("MySQL 5.7 기본 collation 칸에 'admin ' 추가:", 짧게(옛칸공백));

```

출력 MySQL 5.7 기본 collation 칸에 'admin ' 추가: 거절 ER_DUP_ENTRY/1062

★★★ 같은 서버, 같은 SQL 인데 칸의 collation 에 따라 갑니다.

utf8mb4_general_ci	(5.7 시절 기본) → PAD SPACE	→ 뒤 공백을 무시합니다
utf8mb4_0900_ai_ci	(8.0 이후 기본) → NO PAD	→ 뒤 공백을 **구분합니다**

PAD_ATTRIBUTE 라는 성질입니다. 직접 확인해 봅니다.

```

const 패드 = await 마이(
  "SELECT COLLATION_NAME AS 이름, PAD_ATTRIBUTE AS 공백처리 FROM
  information_schema.COLLATIONS " +
  "WHERE COLLATION_NAME IN ('utf8mb4_general_ci','utf8mb4_0900_ai_ci') ORDER BY 1",
);

for (const 줄 of 패드.값) {
  console.log(` ${줄.이름} - ${줄.공백처리}`);
}

```

출력 utf8mb4_0900_ai_ci - NO PAD
 utf8mb4_general_ci - PAD SPACE

★ 그럼 아까 'a ' = 'a' 가 왜 참이었을까요? 글자 리터럴끼리의 비교는 접속(connection)의 collation 을 씁니다. 드라이버가 접속에 어떤 collation 을 쓰느냐에 따라 달라집니다.

```
const 접속콜 = await 마이("SELECT @@collation_connection AS c, COLLATION('a') AS l");
console.log("접속 collation:", 접속콜.값[0].c, "· 리터럴 collation:", 접속콜.값[0].l);
```

출력 접속 collation: utf8mb4_unicode_ci · 리터럴 collation: utf8mb4_unicode_ci

★★★ 이 자료의 드라이버(mysql2)는 utf8mb4_unicode_ci 로 붙습니다. 그런데 표의 기본은 utf8mb4_0900_ai_ci 입니다. **둘이 다릅니다.** utf8mb4_unicode_ci 는 PAD SPACE 라서 리터럴 비교에서 뒤 공백을 무시한 것입니다.

★ 여기서 배울 것은 이겁니다. **“MySQL 은 이렇다” 가 아니라 “이 칸의 collation 은 이렇다” 로 생각하세요.** 서버·데이터베이스·표·칸·접속마다 collation 이 다를 수 있습니다.

★ 그리고 PostgreSQL 에도 뒤 공백 함정이 있습니다. CHAR(n) 입니다.

```
await 피지("DROP TABLE IF EXISTS 공백시험");
await 피지("CREATE TABLE 공백시험 (변하는칸 VARCHAR(20), 고정칸 CHAR(20))");
await 피지("INSERT INTO 공백시험 VALUES ('a ', 'a ')");

const 변하는 = await 피지("SELECT COUNT(*)::int AS c FROM 공백시험 WHERE 변하는칸 = 'a'");
const 고정 = await 피지("SELECT COUNT(*)::int AS c FROM 공백시험 WHERE 고정칸 = 'a'");

console.log("PG VARCHAR 칸에 'a ' 를 넣고 'a' 로 찾기:", 변하는.값[0].c, "건");
```

출력 PG VARCHAR 칸에 'a ' 를 넣고 'a' 로 찾기: 0 건

```
console.log("PG CHAR(20) 칸에 'a ' 를 넣고 'a' 로 찾기:", 고정.값[0].c, "건");
```

출력 PG CHAR(20) 칸에 'a ' 를 넣고 'a' 로 찾기: 1 건

★★ PostgreSQL 도 CHAR(n) 은 뒤 공백을 무시합니다. 표준이 그렇게 정해 놨습니다. → **CHAR(n) 을 쓰지 마세요.** 거의 항상 VARCHAR 나 TEXT 가 낫습니다. → 그리고 아이디·코드는 저장하기 전에 trim() 하세요. 양쪽 문제가 다 사라집니다.

★★ 0 으로 나누기

섹션 4

```
await 나란히("SELECT 1/0", "SELECT 1/0 AS 결과");
```

출력 · SELECT 1/0
 PostgreSQL – 거절 22012
 MySQL – [{"결과":null}]

```
await 나란히("나머지 연산 MOD(1,0)", "SELECT MOD(1,0) AS 결과");
```

출력 · 나머지 연산 MOD(1,0)
 PostgreSQL – 거절 22012
 MySQL – [{"결과":null}]

★ MySQL 은 NULL 을 줍니다. 그리고 NULL 은 **전역됩니다**. `SELECT 생산량 / 가동시간 * 100` 에서 가동시간이 0 이면 결과가 NULL 이 되고, 그 값을 화면에 그리면 빈 칸이 나옵니다. 에러는 없습니다.

★★ 그래서 어느 쪽이든 **직접 막는 것이** 정답입니다. 양쪽에서 다 됩니다.

```
await 나란히(
  "NULLIF 로 0 을 걸러내기",
  "SELECT COALESCE(100 / NULLIF(0,0), 0) AS 가동률",
);
```

출력 · NULLIF 로 0 을 걸러내기
 PostgreSQL – [{"가동률":0}]
 MySQL – [{"가동률":"0.0000"}]

★ `NULLIF(a, b)` 는 a 와 b 가 같으면 NULL, 아니면 a 를 줍니다. `x / NULLIF(y, 0)` 은 y 가 0 이면 NULL 이 되고, **에러가 안 납니다**. 그걸 `COALESCE(..., 0)` 로 받으면 0 이 나옵니다. **양쪽에서 같은 답입니다**.

★★ 그런데 자바스크립트로 온 모양이 다릅니다. 0 과 “0.0000” 입니다. MySQL 의 / 는 결과를 DECIMAL 로 만들고, 드라이버는 DECIMAL 을 **글자로** 줍니다. (PostgreSQL 의 NUMERIC 도 글자로 옵니다. 02단원에서 했습니다) → 계산에 쓰려면 `Number(...)` 로 바꿔야 합니다. 안 그러면 “0.0000” + 1 이

“0.00001” 이 됩니다. ****정산 코드에서 사고가 나는 지점입니다.****

```
const 나눗셈타입 = await 마이("SELECT 100 / 3 AS 몫");
console.log("MySQL 나눗셈 결과의 자바스크립트 타입:", typeof 나눗셈타입.값[0].몫);
```

출력 MySQL 나눗셈 결과의 자바스크립트 타입: string

```
console.log("정수로 받고 싶으면 CAST:", 짧게(await 마이("SELECT CAST(100/3 AS SIGNED) AS 몫")));
```

출력 정수로 받고 싶으면 CAST: [{"몫":33}]

★★★ 글자를 숫자로 알아서 바꿔 줍니다

섹션 5

이게 개인적으로 제일 무섭다고 생각하는 차이입니다.

```
await 나란히("SELECT 'abc' + 1", "SELECT 'abc' + 1 AS 결과");
```

출력 · SELECT 'abc' + 1
 PostgreSQL – 거절 22P02
 MySQL – [{"결과":1}]

```
await 나란히("SELECT '12개' + 1", "SELECT '12개' + 1 AS 결과");
```

```
출력      · SELECT '12개' + 1
           PostgreSQL - 거절 22P02
           MySQL      - [{"결과":13}]
```

★★ MySQL 은 'abc' 를 0 으로, '12개' 를 12 로 바꿉니다. 앞에서 읽을 수 있는 데까지 읽고 나머지를 버립니다.

★★★ 그리고 비교에서도 그렇게 합니다. 이게 진짜 문제입니다.

```
await 나란히("'1' = 1 인가", "SELECT ('1' = 1) AS 결과");
```

```
출력      · '1' = 1 인가
           PostgreSQL - [{"결과":true}]
           MySQL      - [{"결과":1}]
```

```
await 나란히("'1a' = 1 인가", "SELECT ('1a' = 1) AS 결과");
```

```
출력      · '1a' = 1 인가
           PostgreSQL - 거절 22P02
           MySQL      - [{"결과":1}]
```

★★★ MySQL 에서 '1a' = 1 이 참입니다.

설비번호가 INT 인 표에서 WHERE 설비번호 = '1a' 를 보내면 PostgreSQL 은 22P02 로 거절합니다. MySQL 은 1번 설비를 돌려줍니다.

★ 사용자가 화면에 이상한 값을 넣었을 때 양쪽 반응이 완전히 다릅니다.

PostgreSQL 은 "잘못된 입력" 이라고 알려 주고,
MySQL 은 엉뚱한 줄을 조용히 보여 줍니다.

★★ 막는 법: 파라미터를 쓰고, 앱에서 타입을 검사하세요. WHERE 설비번호 = ? 에 문자열을 그냥 넣지 말고 Number.isInteger(값) 을 먼저 확인한 뒤 숫자로 넣으세요. (08단원에서 파라미터 쓰는 법을 했습니다)

★ 글자 수를 세면 바이트가 나옵니다

섹션 6

```
await 나란히("길이 재기 - LENGTH('설비🔧')", "SELECT LENGTH('설비🔧') AS 결과");
```

```
출력      · 길이 재기 - LENGTH('설비🔧')
           PostgreSQL - [{"결과":3}]
           MySQL      - [{"결과":10}]
```

★★ 같은 함수 이름인데 답이 다릅니다.

PostgreSQL 의 `length()` = 글자 수 (설비 → 3) MySQL 의 `LENGTH()` = 바이트 수 (설비 → 10)

한글 한 자는 UTF-8 에서 3바이트, 이 이모지는 4바이트라서 3+3+4 = 10 입니다.

★ 각자 반대편에 해당하는 함수가 따로 있습니다.

```
const 길이비교 = await 나란히(
  "글자 수 세기 - 양쪽에서 통하는 이름",
  "SELECT char_length('설비') AS 결과",
);
```

```
출력      · 글자 수 세기 - 양쪽에서 통하는 이름
           PostgreSQL - [{"결과":3}]
           MySQL      - [{"결과":3}]
```

```
console.log("char_length 는 양쪽이 같은가:", 길이비교.가.값[0].결과 === 길이비교.나.
값[0].결과);
```

```
출력      char_length 는 양쪽이 같은가: true
```

★★ `char_length()` 를 쓰세요. 양쪽에서 글자 수를 줍니다. `LENGTH()` 는 두 DB 에서 뜻이 달라서 옮기면 조용히 틀립니다. “이름이 20자 넘으면 자르기” 같은 코드가 여기서 깨집니다.

★ NULL 이 정렬에서 어디로 가나

섹션 7

```
await 나란히(
  "ORDER BY 에서 NULL 의 자리",
  "SELECT x FROM (SELECT 1 AS x UNION ALL SELECT NULL UNION ALL SELECT 3) AS t ORDER
BY x",
);
```

```
출력      · ORDER BY 에서 NULL 의 자리
           PostgreSQL - [{"x":1},{x":3},{x":null}]
           MySQL      - [{"x":null},{x":1},{x":3}]
```

★★ 오름차순인데 NULL 의 자리가 반대입니다.

PostgreSQL : NULL 을 **제일 큰 값**으로 봅니다 → 맨 뒤 MySQL : NULL 을 **제일 작은 값**으로 봅니다 → 맨 앞

★ “미점검 설비” 목록의 순서가 통째로 달라집니다. 에러는 안 납니다.

```
const 널문법 = await 마이("SELECT 1 AS x ORDER BY x NULLS LAST");
console.log("MySQL 에서 NULLS LAST 문법:", 짧게(널문법));
```

```
출력      MySQL 에서 NULLS LAST 문법: 거절 ER_PARSE_ERROR/1064
```

★ PostgreSQL 에는 ORDER BY x NULLS LAST 문법이 있습니다. MySQL 에는 없습니다. MySQL 에서는 이렇게 흉내 냅니다.

```
await 나란히(  
  "양쪽에서 NULL 을 맨 뒤로 보내기",  
  "SELECT x FROM (SELECT 1 AS x UNION ALL SELECT NULL UNION ALL SELECT 3) AS t ORDER  
  BY (x IS NULL), x",  
);
```

출력

- 양쪽에서 NULL 을 맨 뒤로 보내기

```
PostgreSQL - [{"x":1},{x":3},{x":null}]  
MySQL      - [{"x":1},{x":3},{x":null}]
```

★ ORDER BY (x IS NULL), x 는 양쪽에서 똑같이 동작합니다. x IS NULL 이 거짓(0)인 줄이 먼저 오고, 그 안에서 x 로 정렬됩니다.

★★ utf8 은 utf8 이 아닙니다

섹션 8

MySQL 에서 utf8 이라고 쓰면 진짜 UTF-8 이 아닙니다. 한 글자당 최대 3바이트까지만 받는 반쪽짜리입니다. 정식 이름은 utf8mb3 입니다.

한글은 3바이트라 들어갑니다. 그런데 이모지는 4바이트라 안 들어갑니다. 직접 해 봅니다.

```
await 마이("DROP TABLE IF EXISTS 구구_옛문자");  
await 마이("CREATE TABLE 구구_옛문자 (메모 VARCHAR(50)) CHARACTER SET utf8mb3");  
  
const 한글넣기 = await 마이("INSERT INTO 구구_옛문자 VALUES ('점검 완료')");  
console.log("utf8mb3 칸에 한글:", 짧게(한글넣기));
```

출력

```
utf8mb3 칸에 한글: 성공 1건
```

```
const 이모지넣기 = await 마이("INSERT INTO 구구_옛문자 VALUES ('점검 완료 🗑')");  
console.log("utf8mb3 칸에 이모지:", 짧게(이모지넣기));
```

출력

```
utf8mb3 칸에 이모지: 거절 ER_TRUNCATED_WRONG_VALUE_FOR_FIELD/1366
```

★ STRICT 가 켜져 있어서 거절했습니다. 개념02 를 기억하세요. ★★ STRICT 를 끄면 어떻게 될까요. 이게 진짜 옛날에 벌어지던 일입니다.

```
await 마이("SET SESSION sql_mode = '');  
await 마이("INSERT INTO 구구_옛문자 VALUES ('점검 완료 🗑 이상없음')");  
  
const 망가진메모 = await 마이("SELECT 메모 FROM 구구_옛문자 ORDER BY 메모");  
console.log("STRICT 끄고 넣은 뒤 표 내용:", 짧게(망가진메모));
```


출력 STRICT 고고 넣은 뒤 표 내용: [{"메모":"점검 완료"}, {"메모":"점검 완료 ? 이상없음"}]

```
await 마이("SET SESSION sql_mode = ?", [원래모드]);
```

★★★ 이모지가 물음표로 바뀌었습니다. 에러도 없습니다. 사용자가 쓴 글이 조용히 훼손됩니다.

2010년대에 “이모지 쓰면 글이 깨져요” 라는 문의가 흔했던 이유가 이것입니다.

★ utf8mb4 칸에서는 잘 들어갑니다.

```
await 마이("DROP TABLE IF EXISTS 구구_새문자");
await 마이("CREATE TABLE 구구_새문자 (메모 VARCHAR(50)) CHARACTER SET utf8mb4");
await 마이("INSERT INTO 구구_새문자 VALUES ('점검 완료 🛠 이상없음')");

const 멀정한메모 = await 마이("SELECT 메모 FROM 구구_새문자");
console.log("utf8mb4 칸에 넣은 뒤:", 짧게(멀정한메모));
```

출력 utf8mb4 칸에 넣은 뒤: [{"메모":"점검 완료 🛠 이상없음"}]

★ MySQL 8.0 부터 서버 기본이 utf8mb4 라서 새로 만들면 괜찮습니다. 문제는 **5.x 시절에 만든 옛** 표입니다. 그때 기본이 utf8(=utf8mb3) 이었습니다. 회사 표를 만나면 문자셋부터 확인하세요.

```
const 문자셋확인 = await 마이(
  "SELECT TABLE_NAME AS 표, TABLE_COLLATION AS 규칙 FROM information_schema.TABLES "
  +
  "WHERE TABLE_SCHEMA='factory_db' AND TABLE_NAME IN ('구구_옛문자', '구구_새문자')
  ORDER BY 1",
);

for (const 줄 of 문자셋확인.값) {
  console.log(` ${줄.표} - ${줄.규칙}`);
}
```

출력 구구_새문자 - utf8mb4_0900_ai_ci
구구_옛문자 - utf8mb3_general_ci

★★ PostgreSQL 에는 이 함정이 없습니다. 데이터베이스를 만들 때 인코딩을 UTF8 로 정하면 그냥 진짜 UTF-8 입니다. 반쪽짜리 UTF-8 이라는 개념 자체가 없습니다.

```
const 피지인코딩 = await 피지("SHOW server_encoding");
console.log("PostgreSQL 인코딩:", 피지인코딩.값[0].server_encoding);
```

출력 PostgreSQL 인코딩: UTF8

```
const 피지이모지 = await 피지("SELECT length('점검 완료 🛠 이상없음') AS 글자수");
console.log("PostgreSQL 에서 이모지가 든 글의 글자 수:", 피지이모지.값[0].글자수);
```

뒷정리

```
await 피지("DROP TABLE IF EXISTS 작업자");
await 피지("DROP TABLE IF EXISTS 공백시험");
await 마이("DROP TABLE IF EXISTS 구구_작업자");
await 마이("DROP TABLE IF EXISTS 구구_구분작업자");
await 마이("DROP TABLE IF EXISTS 구구_새칸");
await 마이("DROP TABLE IF EXISTS 구구_옛칸");
await 마이("DROP TABLE IF EXISTS 구구_옛문자");
await 마이("DROP TABLE IF EXISTS 구구_새문자");

const 마무리모드 = await 마이("SELECT @@SESSION.sql_mode AS m");
console.log("sql_mode 를 원래대로 돌려놨나:", 마무리모드.값[0].m === 원래모드);
```

정리 — 에러 없이 달라지는 것들

시험 PostgreSQL MySQL 8.4 맞추는 법

'A' || 'B' 'AB' 0 (OR) `concat()` 을 쓴다 `concat('A',NULL,'B')` 'AB' NULL COALESCE 로 감싼다 'ABC' = 'abc' `false true lower()` 유일 색인 / `as_cs 'e' = 'é' false true as_cs collation 'a' = 'a'` (리터럴) `false true` 저장 전에 `trim()` 'a' = 'a' (8.0 칸) `false false` (이미 같습니다) 'a' = 'a' (5.7 칸) `false true collation` 을 확인한다 1/0 거절 22012 NULL NULLIF(y,0) 로 막는다 'abc' + 1 거절 22P02 1 앱에서 타입을 검사한다 '1a' = 1 거절 22P02 `true` 파라미터 + 타입 검사 `LENGTH('설비')` 3 (글자) 10 (바이트) `char_length()` 를 쓴다 ORDER BY 의 NULL 맨 뒤 맨 앞 ORDER BY (x IS NULL), x utf8 칸에 이모지 (해당 없음) 안 들어감 utf8mb4 를 쓴다

★★★ 오른쪽 칸이 이 파일의 값어치입니다. **전부 양쪽에서 통하는 방법이 있습니다.** 두 DB 를 다 지원해야 한다면 오른쪽 칸대로 쓰세요.

★ 그리고 하나 더 기억하세요. **어느 쪽이 옳은 게 아닙니다.** 규칙이 다를 뿐입니다. 대소문자 무시가 아이디에는 좋고 상품 코드에는 나쁩니다. 중요한 건 **내 칸이 어떤 규칙인지 아는 것**입니다.

직접 해 볼 것

⇒ 직접 해보기

1

섹션 1 에서 '설비' || 3 을 양쪽에 보내 보세요. MySQL 은 무엇을 주나요? 왜 그럴까요?

▫ 직접 해보기 2

섹션 2 의 중복확인 시나리오를 **비밀번호**로 바꿔 보세요. 'Secret1!' 로 가입하고 'secret1!' 로 로그인하면 MySQL 에서 들어가지나요? ★ 이걸 진짜 사고입니다. (실제 서비스에서는 비밀번호를 그대로 저장하지 않습니다. 해시를 저장합니다. 그런데 해시 문자열 비교에서 같은 문제가 납니다)

▫ 직접 해보기 3

섹션 3 에서 utf8mb4_bin collation 으로 칸을 만들어 보세요. 'admin' 과 'admin ' 이 같은 값으로 보이나요? `information_schema.COLLATIONS` 로 PAD_ATTRIBUTE 를 확인해 보세요.

▫ 직접 해보기 4

섹션 5 를 표로 해 보세요. 설비 표를 만들고 `WHERE id = '1등급'` 을 양쪽에 보내면? MySQL 이 몇 번 설비를 줄까요?

▫ 직접 해보기 5

한글 정렬은 어떨까요? '하','가','나' 를 넣고 `ORDER BY` 해 보세요. ★ 재 보면 양쪽이 같습니다. 안 다른 것도 확인해 두면 좋습니다.

▫ 직접 해보기 6

섹션 8 에서 `VARCHAR(50)` 인 utf8mb3 칸에 한글 50자를 넣어 보세요. 들어가요? ★ MySQL 의 `VARCHAR(n)` 은 **글자** 수입입니다. 바이트가 아닙니다. (옛날 latin1 시절 이야기와 헷갈리는 사람이 많습니다)

▫ 직접 해보기 7

`SELECT 'a' = 'A' COLLATE utf8mb4_bin` 을 실행해 보세요. `_bin` 은 무엇을 하는 collation 인가요?

자주 하는 실수

이런 실수를 합니다

|| 를 그대로 옮긴다

★★★ 에러가 안 나서 못 찾습니다. 이름 붙이는 코드가 전부 0 이 됩니다. 옮기기 전에 || 를 전부 찾아서 `concat()` 으로 바꾸세요. 개념05 에서 찾는 도구를 만듭니다.

이런 실수를 합니다

"MySQL 은 대소문자를 무시하니까 편하다" 고 기대고 코드를 짠다

그 코드를 PostgreSQL 로 옮기면 **중복 계정이 생깁니다**. ★ 앱에서 `아이디.toLowerCase()` 를 하고 저장하세요. 그러면 어느 DB 든 같습니다.

이런 실수를 합니다

뒤 공백을 “MySQL 은 무시한다” 고 외운다

★★ 반만 맞습니다. 8.0 기본 collation 은 구분합니다. 외우지 말고 `information_schema.COLLATIONS` 에서 확인하세요. 그리고 저장 전에 `trim()` 하면 이 고민 자체가 사라집니다.

이런 실수를 합니다

`LENGTH()` 로 글자 수를 센다

MySQL 에서는 바이트가 나옵니다. 한글은 3배가 됩니다. “20자 넘으면 자르기” 코드가 7자에서 잘립니다. `char_length()` 를 쓰세요.

이런 실수를 합니다

0 나누기를 DB 에 맡긴다

PostgreSQL 은 멈추고 MySQL 은 NULL 을 줍니다. ★ 어느 쪽도 원하는 동작이 아닙니다. `NULLIF(y,0)` 로 직접 막으세요.

이런 실수를 합니다

숫자 칸에 문자열 파라미터를 넣는다

MySQL 은 ‘1a’ 를 1 로 바꿔서 엉뚱한 줄을 줍니다. ★ 앱에서 타입을 검사하세요. DB 가 막아 줄 거라고 믿으면 안 됩니다.

이런 실수를 합니다

MySQL 에서 `utf8` 이라고 적는다

그건 `utf8mb3` 입니다. 이모지가 안 들어갑니다. ★ 항상 `utf8mb4` 라고 적으세요. 세 글자 더 쓰면 됩니다.

이런 실수를 합니다

옳고 나서 “에러 없으니 성공” 이라고 판단한다

★★★ 이 파일의 모든 차이는 에러가 안 납니다. 옳긴 뒤에는 결과 값을 직접 비교해 보세요. 같은 쿼리를 양쪽에 보내고 답을 맞춰 보는 겁니다.

```
await PG.end();
await MY.end();
```

기능과 문법의 차이

```
RUN node 개념04_기능과_문법의_차이.js
```

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

개념03 은 **에러가 안 나는데 답이 다른 것**들이었습니다. 제일 위험합니다. 이 파일은 반대입니다. **아예 문법 오류가 나는 것**들입니다.

이건 오히려 마음이 편합니다. 옮기면 바로 터지니까 못 넘어갑니다. 대신 **개수가 많습니다**. 하나하나 어떻게 바꿔 쓰는지 알아야 합니다.

① 파라미터 기호가 다릅니다 \$1 vs ? ② 자동 번호 만드는 법이 다릅니다 SERIAL vs AUTO_INCREMENT ③ RETURNING 이 MySQL 에 없습니다 ★★ ④ 부분 색인·배열·진짜 BOOLEAN 이 없습니다 ⑤ 트랜잭션 안의 DDL 이 안 되돌아갑니다 ★★★ ⑥ 표 이름 대소문자 규칙이 반대입니다 ★★ ⑦ UPSERT 문법이 다릅니다 ⑧ JSON 다루는 법이 다릅니다

```
import pg from "pg";
import mysql from "mysql2/promise";
```

```

let PG = null;
let MY = null;

try {
  PG = new pg.Client({ host: "127.0.0.1", port: 5434, user: "factory", password:
"secret", database: "factory_db" });
  await PG.connect();
  MY = await mysql.createConnection({ host: "127.0.0.1", port: 3307, user: "factory",
password: "secret", database: "factory_db" });
} catch (에러) {
  console.log("★ 데이터베이스에 못 붙었습니다:", 에러.code || 에러.message);
  console.log(" docker compose up -d 를 먼저 실행하세요.");
  if (PG) await PG.end().catch(() => {});
  process.exit(0);
}

await PG.query("CREATE SCHEMA IF NOT EXISTS 단원09");
await PG.query("SET search_path TO 단원09");

async function 피지(sql, 값 = []) {
  try {
    const 결과 = await PG.query(sql, 값);
    const 조회인가 = 결과.command === "SELECT" || 결과.command === "SHOW";

    return { 됨: true, 값: 결과.rows, 조회: 조회인가, 건수: 결과.rowCount ?? 0 };
  } catch (에러) {
    return { 됨: false, 코드: 에러.code };
  }
}

async function 마이(sql, 값 = []) {
  try {
    const [행들] = await MY.query(sql, 값);
    const 조회인가 = Array.isArray(행들);

    return {
      됨: true,
      값: 조회인가 ? 행들 : [],
    };
  }
}

```

```

    조회: 조회인가,
    건수: 조회인가 ? 행들.length : 행들.affectedRows,
    새번호: 조회인가 ? null : 행들.insertId,
  };
} catch (에러) {
  return { 됨: false, 코드: 에러.code, 번호: 에러.errno };
}
}

function 짧게(결과) {
  if (!결과.됨) return `거절 ${결과.코드}${결과.번호 === undefined ? "" : "/" + 결과.번호}`;
}

```

조회이거나, RETURNING 처럼 돌려준 줄이 있으면 그 줄을 보여 줍니다.

```

if (결과.조회 || 결과.값.length > 0) return JSON.stringify(결과.값);
return `성공 ${결과.건수}건`;
}

async function 나란히(제목, 피지문, 마이문) {
  const 가 = await 피지(피지문);
  const 나 = await 마이(마이문 === undefined ? 피지문 : 마이문);

  console.log(`· ${제목}`);
  console.log(`  PostgreSQL - ${짧게(가)}`);
  console.log(`  MySQL      - ${짧게(나)}`);

  return { 가, 나 };
}

```

09

파라미터 기호

섹션 1

08단원에서 “SQL 에 값을 직접 붙이지 말고 파라미터로 넘기라” 고 했습니다. 그 기호가 다릅니다.

```

const 피지파라 = await 피지("SELECT $1::int + 1 AS 결과", [10]);
console.log("PostgreSQL 에 $1 로 보내기:", 짧게(피지파라));

```

```
출력      PostgreSQL 에 $1 로 보내기: [{"결과":11}]
```

```

const 피지물음 = await 피지("SELECT ? + 1 AS 결과", [10]);
console.log("PostgreSQL 에 ? 로 보내기:", 짧게(피지물음));

```

```
출력      PostgreSQL 에 ? 로 보내기: 거절 42883
```

```
const 마이물음 = await 마이("SELECT ? + 1 AS 결과", [10]);
console.log("MySQL 에 ? 로 보내기:", 짧게(마이물음));
```

출력 MySQL 에 ? 로 보내기: [{"결과":11}]

```
const 마이달러 = await 마이("SELECT $1 + 1 AS 결과", [10]);
console.log("MySQL 에 $1 로 보내기:", 짧게(마이달러));
```

출력 MySQL 에 \$1 로 보내기: 거절 ER_BAD_FIELD_ERROR/1054

★ 코드를 옮기면 모든 쿼리의 파라미터를 다 바꿔야 합니다. 쿼리가 200개면 200개를 다 뽐니다. 이게 옮길 때 가장 지루한 작업입니다.

★★ 차이가 하나 더 있습니다. 번호가 있느냐 없느냐 입니다.

PostgreSQL : \$1, \$2 — 번호가 있어서 같은 값을 여러 번 쓸 수 있습니다 MySQL : ? — 번호가 없어서 나오는 순서대로 값을 하나씩 씁니다

```
const 두번쓰기피지 = await 피지("SELECT $1::int AS 처음, $1::int * 2 AS 두배", [7]);
console.log("PostgreSQL 에서 $1 을 두 번 쓰기:", 짧게(두번쓰기피지));
```

출력 PostgreSQL 에서 \$1 을 두 번 쓰기: [{"처음":7,"두배":14}]

```
const 두번쓰기마이 = await 마이("SELECT ? AS 처음, ? * 2 AS 두배", [7, 7]);
console.log("MySQL 에서는 값을 두 번 넘겨야 함:", 짧게(두번쓰기마이));
```

출력 MySQL 에서는 값을 두 번 넘겨야 함: [{"처음":7,"두배":14}]

★ 옮길 때 \$1 을 기계적으로 ? 로 바꾸면 값 개수가 안 맞아서 터집니다. 같은 번호를 두 번 쓴 곳이 있으면 값 배열도 고쳐야 합니다.

자동 번호

섹션 2

```
await 피지("DROP TABLE IF EXISTS 점검");
await 마이("DROP TABLE IF EXISTS 구구_점검");

await 피지("CREATE TABLE 점검 (id SERIAL PRIMARY KEY, 내용 VARCHAR(30))");
await 마이("CREATE TABLE 구구_점검 (id INT AUTO_INCREMENT PRIMARY KEY, 내용 VARCHAR(30))");
```

★ PostgreSQL 에는 표준 문법도 있습니다. 새 코드에서는 이쪽을 권합니다.


```
const 아이덴티티 = await 피지(
  "CREATE TABLE 표준자동번호 (id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, v
  VARCHAR(10))",
);
console.log("PostgreSQL 의 GENERATED ... AS IDENTITY:", 짧게(아이덴티티));
```

출력 PostgreSQL 의 GENERATED ... AS IDENTITY: 성공 0건

```
const 마이아이덴 = await 마이(
  "CREATE TABLE 구구_표준자동번호 (id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, v
  VARCHAR(10))",
);
console.log("MySQL 에서 같은 문법:", 짧게(마이아이덴));
```

출력 MySQL 에서 같은 문법: 거절 ER_PARSE_ERROR/1064

```
await 피지("DROP TABLE IF EXISTS 표준자동번호");
```

★★ 그럼 롤백하면 번호가 어떻게 되나요? 이걸 자주 나오는 질문입니다. "MySQL 은 번호가 꼬인다" 는 말도 있는데, 재 봤습니다.

```
await 피지("INSERT INTO 점검(내용) VALUES ('첫 점검')");
await 피지("BEGIN");
await 피지("INSERT INTO 점검(내용) VALUES ('취소될 점검')");
await 피지("ROLLBACK");
await 피지("INSERT INTO 점검(내용) VALUES ('세 번째 점검')");

await 마이("INSERT INTO 구구_점검(내용) VALUES ('첫 점검')");
await 마이("BEGIN");
await 마이("INSERT INTO 구구_점검(내용) VALUES ('취소될 점검')");
await 마이("ROLLBACK");
await 마이("INSERT INTO 구구_점검(내용) VALUES ('세 번째 점검')");

const 번호피지 = await 피지("SELECT id, 내용 FROM 점검 ORDER BY id");
const 번호마이 = await 마이("SELECT id, 내용 FROM 구구_점검 ORDER BY id");

console.log("롤백 뒤 PostgreSQL 의 번호:", 짧게(번호피지));
```

출력 롤백 뒤 PostgreSQL 의 번호: [{"id":1,"내용":"첫 점검"}, {"id":3,"내용":"세 번째 점검"}]

```
console.log("롤백 뒤 MySQL 의 번호:", 짧게(번호마이));
```

출력 롤백 뒤 MySQL 의 번호: [{"id":1,"내용":"첫 점검"}, {"id":3,"내용":"세 번째 점검"}]

★★★ 똑같습니다. 양쪽 다 2번을 건너뛰었습니다.

왜 그럴까요. 자동 번호는 **트랜잭션 밖에서** 돌아가기 때문입니다. 번호를 되돌리려면 다른 트랜잭션이 그 번호를 못 쓰게 기다려야 합니다. 그러면 동시에 INSERT 하는 사람들이 전부 줄을 서야 합니다. 너무 느립니다.

★ 그래서 이걸 외우세요. **자동 번호에 “빠진 번호가 없다” 고 기대하지 마세요.** 1, 2, 3 다음에 7 이 나올 수 있습니다. 정상입니다. “오늘 몇 건 들어왔나” 를 $MAX(id) - MIN(id)$ 로 세면 안 됩니다.

★★ RETURNING 이 MySQL 에 없습니다

섹션 3

```
const 반환피지 = await 피지("INSERT INTO 점검(내용) VALUES ('네 번째') RETURNING id, 내용");
console.log("PostgreSQL 의 RETURNING:", 짧게(반환피지));
```

출력 PostgreSQL 의 RETURNING: [{"id":4,"내용":"네 번째"}]

```
const 반환마이 = await 마이("INSERT INTO 구구_점검(내용) VALUES ('네 번째') RETURNING id, 내용");
console.log("MySQL 에서 RETURNING:", 짧게(반환마이));
```

출력 MySQL 에서 RETURNING: 거절 ER_PARSE_ERROR/1064

★ PostgreSQL 은 UPDATE·DELETE 에도 RETURNING 을 씁니다. 아주 편합니다.

```
const 고침반환 = await 피지("UPDATE 점검 SET 내용 = '고친 내용' WHERE id = 1 RETURNING id, 내용");
console.log("UPDATE ... RETURNING:", 짧게(고침반환));
```

출력 UPDATE ... RETURNING: [{"id":1,"내용":"고친 내용"}]

```
const 지움반환 = await 피지("DELETE FROM 점검 WHERE id = 1 RETURNING id, 내용");
console.log("DELETE ... RETURNING:", 짧게(지움반환));
```

출력 DELETE ... RETURNING: [{"id":1,"내용":"고친 내용"}]

★★ MySQL 에서는 어떻게 방금 넣은 id 를 받아 드라이버가 주는 insertId 를 씁니다. SQL 로는 LAST_INSERT_ID() 입니다.

```
const 한건 = await 마이("INSERT INTO 구구_점검(내용) VALUES ('다섯 번째')");
console.log("mysql2 가 준 insertId:", 한건.새번호);
```

출력 mysql2 가 준 insertId: 4

```
const 함수로 = await 마이("SELECT LAST_INSERT_ID() AS 마지막");
console.log("LAST_INSERT_ID() 로 받기:", 짧게(함수로));
```

```
출력      LAST_INSERT_ID() 로 받기: [{"마지막":4}]
```

★ `LAST_INSERT_ID()` 는 **접속마다 따로** 기억합니다. 다른 사람이 그 사이에 INSERT 해도 내 값은 안 바뀝니다. 그건 안전합니다.

★★★ 그런데 **여러 건을 한 번에 넣으면 위험합니다.**

```
const 여러건 = await 마이("INSERT INTO 구구_점검(내용) VALUES ('여섯'),('일곱'),
('여덟')");
console.log("세 건 넣었을 때 insertId:", 여러건.새번호, "· 넣은 건수:", 여러건.건수);
```

```
출력      세 건 넣었을 때 insertId: 5 · 넣은 건수: 3
```

```
const 실제번호 = await 마이("SELECT id FROM 구구_점검 WHERE 내용 IN
('여섯','일곱','여덟') ORDER BY id");
console.log("실제로 들어간 번호들:", 짧게(실제번호));
```

```
출력      실제로 들어간 번호들: [{"id":5},{"id":6},{"id":7}]
```

★★★ `insertId` 는 5 입니다. **첫 번째 번호만** 줍니다. 나머지 6, 7 은 알려 주지 않습니다. "연속으로 붙었을 테니 5,6,7 이겠지" 라고 계산하는 코드를 짜면 안 됩니다. ★ 실제로 `innodb_autoinc_lock_mode` 설정에 따라 **연속이 아닐 수** 있습니다.

여러 서버가 동시에 넣거나, 복제 설정이 다르면 띄엄띄엄 붙습니다.

★ 그래서 MySQL 에서 여러 건을 넣고 번호를 알아야 하면 **한 건씩 넣고 매번 insertId 를 받는 수밖에 없습니다.** 느립니다. PostgreSQL 은 `RETURNING` 한 줄로 끝납니다.

```
const 피지여러건 = await 피지("INSERT INTO 점검(내용) VALUES ('여섯'),('일곱'),
('여덟') RETURNING id, 내용");
console.log("PostgreSQL 은 세 건의 번호를 다 줍니다:", 짧게(피지여러건));
```

```
출력      PostgreSQL 은 세 건의 번호를 다 줍니다: [{"id":5,"내용":"여섯"},
{"id":6,"내용":"일곱"}, {"id":7,"내용":"여덟"}]
```

없는 기능들

섹션 4

```
console.log("— 부분 색인 —");
```

```
출력      — 부분 색인 —
```

```
await 나란히(
  "미완료 점검만 색인하기",
  "CREATE INDEX 미완료색인 ON 점검(id) WHERE 내용 IS NULL",
  "CREATE INDEX 미완료색인 ON 구구_점검(id) WHERE 내용 IS NULL",
);
```

```
출력      · 미완료 점검만 색인하기
           PostgreSQL - 성공 0건
           MySQL      - 거절 ER_PARSE_ERROR/1064
```

★ 부분 색인(partial index)은 **조건에 맞는 줄만** 색인에 담습니다. “처리 안 된 주문” 처럼 전체의 1% 만 자주 찾는 경우에 아주 효율적입니다. (색인 이야기는 06단원에서 했습니다)

★ MySQL 에서 대신하는 법: 생성 칸(generated column) + 그 칸에 색인. 또는 그냥 전체 색인을 만들고 크기를 감수합니다.

```
console.log("— 표현식 색인 —");
```

```
출력      — 표현식 색인 —
```

```
await 나란히(
  "소문자로 만든 값에 색인",
  "CREATE INDEX 소문자색인 ON 점검(lower(내용))",
  "CREATE INDEX 소문자색인 ON 구구_점검(lower(내용))",
);
```

```
출력      · 소문자로 만든 값에 색인
           PostgreSQL - 성공 0건
           MySQL      - 성공 0건
```

★ 표현식 색인은 **양쪽 다 됩니다**. MySQL 은 8.0.13 부터 생겼습니다. 괄호를 두 겹 쓰는 것까지 문법이 같습니다.

```
console.log("— 배열 타입 —");
```

```
출력      — 배열 타입 —
```

```
await 나란히("ARRAY[1,2,3] 만들기", "SELECT ARRAY[1,2,3] AS 목록");
```

```
출력      · ARRAY[1,2,3] 만들기
           PostgreSQL - [{"목록": [1,2,3]}]
           MySQL      - 거절 ER_PARSE_ERROR/1064
```

```
await 나란히(
  "배열 칸 만들기",
  "CREATE TABLE 태그시험 (태그 TEXT[])",
  "CREATE TABLE 구구_태그시험 (태그 TEXT[])",
);
```

```
출력      · 배열 칸 만들기
           PostgreSQL - 성공 0건
           MySQL      - 거절 ER_PARSE_ERROR/1064
```

```
await 피지("DROP TABLE IF EXISTS 태그시험");
```

★ MySQL 에서 여러 값을 한 칸에 넣고 싶으면 JSON 배열을 씁니다. 또는 **표를 따로 만드는 것이** 정석입니다 (04단원 정규화). ★ 사실 배열 칸은 PostgreSQL 에서도 남용하면 안 됩니다.

검색·조인·제약이 어려워집니다.

```
console.log("— 진짜 BOOLEAN 이 있나 —");
```

```
출력      — 진짜 BOOLEAN 이 있나 —
```

```
await 피지("DROP TABLE IF EXISTS 가동상태");
await 마이("DROP TABLE IF EXISTS 구구_가동상태");
await 피지("CREATE TABLE 가동상태 (설비 VARCHAR(10), 가동 BOOLEAN)");
await 마이("CREATE TABLE 구구_가동상태 (설비 VARCHAR(10), 가동 BOOLEAN)");
```

```
await 피지("INSERT INTO 가동상태 VALUES ('프레스', true)");
await 마이("INSERT INTO 구구_가동상태 VALUES ('프레스', true)");
```

```
await 나란히(
  "true 를 넣고 다시 읽기",
  "SELECT 가동 FROM 가동상태",
  "SELECT 가동 FROM 구구_가동상태",
);
```

```
출력      · true 를 넣고 다시 읽기
           PostgreSQL - [{"가동":true}]
           MySQL      - [{"가동":1}]
```

```
const 피지타입 = await 피지(
  "SELECT data_type AS 타입 FROM information_schema.columns WHERE
  table_schema='단원09' AND table_name='가동상태' AND column_name='가동'",
);
const 마이타입 = await 마이(
  "SELECT COLUMN_TYPE AS 타입 FROM information_schema.columns WHERE
  TABLE_SCHEMA='factory_db' AND TABLE_NAME='구구_가동상태' AND COLUMN_NAME='가동'",
);

console.log("PostgreSQL 이 실제로 만든 타입:", 피지타입.값[0].타입);
```

출력 PostgreSQL 이 실제로 만든 타입: boolean

```
console.log("MySQL 이 실제로 만든 타입:", 마이타입.값[0].타입);
```

출력 MySQL 이 실제로 만든 타입: tinyint(1)

★★ MySQL 의 BOOLEAN 은 별명입니다. 진짜 타입은 TINYINT(1) 입니다. 그래서 참·거짓 말고 다른 숫자도 들어갑니다.

```
const 오오넣기 = await 마이("INSERT INTO 구구_가동상태 VALUES ('용접로봇', 5)");
console.log("MySQL 의 BOOLEAN 칸에 5 넣기:", 짧게(오오넣기));
```

출력 MySQL 의 BOOLEAN 칸에 5 넣기: 성공 1건

```
const 오오피지 = await 피지("INSERT INTO 가동상태 VALUES ('용접로봇', 5)");
console.log("PostgreSQL 의 BOOLEAN 칸에 5 넣기:", 짧게(오오피지));
```

출력 PostgreSQL 의 BOOLEAN 칸에 5 넣기: 거절 42804

★★★ 그리고 5 는 = true 로 안 잡힙니다. 이게 진짜 함정입니다.

```
const 참비교 = await 마이("SELECT COUNT(*) AS 개수 FROM 구구_가동상태 WHERE 가동 =
true");
const 참자체 = await 마이("SELECT COUNT(*) AS 개수 FROM 구구_가동상태 WHERE 가동");

console.log("MySQL 에서 `가동 = true` 로 센 것:", 참비교.값[0].개수, "건");
```

출력 MySQL 에서 `가동 = true` 로 센 것: 1 건

```
console.log("MySQL 에서 `WHERE 가동` 으로 센 것:", 참자체.값[0].개수, "건");
```

출력 MySQL 에서 `WHERE 가동` 으로 센 것: 2 건

★★★ 같은 표에서 답이 1 건과 2 건으로 갈립니다. = true 는 = 1 이라서 5 를 놓칩니다. WHERE 가동 은 0 이 아닌 값을 다 잡습니다. → MySQL 에서는 = true 대신 WHERE 가동 <> 0 이나 WHERE 가동 을 쓰세요. →

더 좋은 방법은 앱에서 0/1 만 넣도록 막고, CHECK (가동 IN (0,1)) 를 거는 것입니다.

★★★ 트랜잭션 안의 DDL — 마이그레이션이 여기서 갑니다

섹션 5

08단원에서 스키마 변경(마이그레이션)을 했습니다. 스키마 변경은 보통 여러 문장을 이어서 실행합니다.

① 표를 만든다 ② 칸을 추가한다 ③ 데이터를 옮긴다 ④ 옛 칸을 지운다

③에서 실패하면 어떻게 될까요?

```
await 피지("DROP TABLE IF EXISTS 이력");
await 마이("DROP TABLE IF EXISTS 구구_이력");
await 피지("CREATE TABLE 이력 (기록 VARCHAR(30))");
await 마이("CREATE TABLE 구구_이력 (기록 VARCHAR(30))");

await 피지("DROP TABLE IF EXISTS 새표");
await 마이("DROP TABLE IF EXISTS 구구_새표");
```

PostgreSQL — 트랜잭션 안에서 INSERT + CREATE TABLE 을 하고 롤백

```
await 피지("BEGIN");
await 피지("INSERT INTO 이력 VALUES ('마이그레이션 시작')");
await 피지("CREATE TABLE 새표 (id INT)");
await 피지("ROLLBACK");
```

MySQL — 똑같이

```
await 마이("BEGIN");
await 마이("INSERT INTO 구구_이력 VALUES ('마이그레이션 시작')");
await 마이("CREATE TABLE 구구_새표 (id INT)");
await 마이("ROLLBACK");

const 표남았나피지 = await 피지("SELECT to_regclass('단원09.새표') IS NOT NULL AS
남음");
const 표남았나마이 = await 마이(
  "SELECT COUNT(*) > 0 AS 남음 FROM information_schema.TABLES WHERE
TABLE_SCHEMA='factory_db' AND TABLE_NAME='구구_새표'",
);

console.log("ROLLBACK 뒤 새 표가 남았나 - PostgreSQL:", 표남았나피지.값[0].남음);
```

출력 ROLLBACK 뒤 새 표가 남았나 - PostgreSQL: false

```
console.log("ROLLBACK 뒤 새 표가 남았나 - MySQL:", 표남았나마이.값[0].남음 === 1);
```

출력 ROLLBACK 뒤 새 표가 남았나 - MySQL: true

★★★ 그리고 더 무서운 것이 있습니다. **앞의 INSERT 도 같이 커밋됩니다.**

```
const 이력피지 = await 피지("SELECT 기록 FROM 이력");
const 이력마이 = await 마이("SELECT 기록 FROM 구구_이력");

console.log("ROLLBACK 뒤 INSERT 가 남았나 - PostgreSQL:", 짧게(이력피지));
```

출력 ROLLBACK 뒤 INSERT 가 남았나 - PostgreSQL: []

```
console.log("ROLLBACK 뒤 INSERT 가 남았나 - MySQL:", 짧게(이력마이));
```

출력 ROLLBACK 뒤 INSERT 가 남았나 - MySQL: [{"기록": "마이그레이션 시작"}]

★★★ MySQL 은 CREATE TABLE 을 만나는 순간 **암묵적 커밋(implicit commit)** 을 합니다. 지금까지의 트랜잭션이 그 자리에서 확정됩니다. 그 뒤의 ROLLBACK 은 되돌릴 것이 없어서 아무 일도 안 합니다.

ALTER TABLE, DROP TABLE, CREATE INDEX, TRUNCATE 도 마찬가지입니다.

★★ 이게 실무에서 무슨 뜻인가

PostgreSQL — 마이그레이션 스크립트를 BEGIN ... COMMIT 으로 감싸면

중간에 실패해도 **통째로 원래대로 돌아갑니다.** 다시 돌리면 됩니다.

MySQL — 중간에 실패하면 **반쯤 바뀐 상태로 멈춥니다.**

어디까지 됐는지 직접 확인하고 손으로 되돌려야 합니다.
새벽 배포 중에 이 상황이 오면 정말 곤란합니다.

★ MySQL 에서 대비하는 법 ① 마이그레이션을 **한 파일에 한 가지 변경만** 담습니다. 실패 지점이 분명해집니다 ② 되돌리는 스크립트(down)를 **미리 짜 둡니다** ③ 운영에 하기 전에 **똑같은 데이터로 연습합니다** ④ Flyway·Liquibase 같은 도구를 씁니다. 어디까지 됐는지 기록해 줍니다

★★ 표 이름 대소문자

섹션 6

```
await 피지("DROP TABLE IF EXISTS SeolBi");
await 피지("DROP TABLE IF EXISTS seolbi");
await 마이("DROP TABLE IF EXISTS SeolBi");
await 마이("DROP TABLE IF EXISTS seolbi");

await 피지("CREATE TABLE SeolBi (id INT)");
await 마이("CREATE TABLE SeolBi (id INT)");
```

소문자로 찾아봅니다.


```
const 소문자피지 = await 피지("SELECT COUNT(*) AS c FROM seolbi");
const 소문자마이 = await 마이("SELECT COUNT(*) AS c FROM seolbi");

console.log("대문자로 만들고 소문자로 찾기 - PostgreSQL:", 짧게(소문자피지));
```

```
출력      대문자로 만들고 소문자로 찾기 - PostgreSQL: [{"c":"0"}]
```

```
console.log("대문자로 만들고 소문자로 찾기 - MySQL:", 짧게(소문자마이));
```

```
출력      대문자로 만들고 소문자로 찾기 - MySQL: 거절 ER_NO_SUCH_TABLE/1146
```

큰따옴표로 원래 대소문자 그대로 찾아봅니다.

```
const 그대로피지 = await 피지(`SELECT COUNT(*) AS c FROM "SeolBi"`);
console.log("PostgreSQL 에서 \"SeolBi\" 로 찾기:", 짧게(그대로피지));
```

```
출력      PostgreSQL 에서 "SeolBi" 로 찾기: 거절 42P01
```

```
const 이름피지 = await 피지(
  "SELECT tablename AS 이름 FROM pg_tables WHERE schemaname='단원09' AND
  lower(tablename)='seolbi'",
);
const 이름마이 = await 마이(
  "SELECT TABLE_NAME AS 이름 FROM information_schema.TABLES WHERE
  TABLE_SCHEMA='factory_db' AND LOWER(TABLE_NAME)='seolbi'",
);

console.log("실제로 저장된 표 이름 - PostgreSQL:", 짧게(이름피지));
```

```
출력      실제로 저장된 표 이름 - PostgreSQL: [{"이름":"seolbi"}]
```

```
console.log("실제로 저장된 표 이름 - MySQL:", 짧게(이름마이));
```

```
출력      실제로 저장된 표 이름 - MySQL: [{"이름":"SeolBi"}]
```

★★ 규칙이 정확히 반대입니다.

PostgreSQL — 따옴표가 없으면 **소문자로 접어서** 저장합니다.

```
`SeolBi` 로 만들면 실제 이름은 `seolbi` 입니다.
그래서 `SELECT * FROM SEOLBI` 도 되고 `seolbi` 도 됩니다.
반대로 `"SeolBi"` 라고 따옴표를 붙이면 **없는 표**입니다.
```

MySQL — 적은 그대로 저장합니다. `SeolBi` 는 `SeolBi` 입니다.

```
그래서 `seolbi` 로는 못 찾습니다.
```

★★★ 그런데 MySQL 은 운영체제를 탐니다. 이게 진짜 문제입니다.

```
const 접기설정 = await 마이("SELECT @@lower_case_table_names AS 값");
console.log("이 서버의 lower_case_table_names:", 접기설정.값[0].값);
```

출력 이 서버의 lower_case_table_names: 0

lower_case_table_names 값의 뜻

- 0 - 적은 그대로 저장하고 **구분해서** 찾습니다 (리눅스 기본)
- 1 - 전부 소문자로 바꿔서 저장하고 구분하지 않습니다 (윈도 기본)
- 2 - 적은 대로 저장하지만 찾을 때는 구분하지 않습니다 (macOS 기본)

★★★ 개발은 맥에서 하고 배포는 리눅스에 하는 팀이 여기서 터집니다.

맥에서는 `SELECT * FROM Seolbi` 가 잘 됩니다 (구분 안 함). 같은 코드를 리눅스 서버에 올리면 `Table doesn't exist` 가 납니다. ★ 이 자료의 docker 는 리눅스 컨테이너라서 값이 0 입니다. **구분합니다.**

★ 예방하는 법 — 아주 간단합니다. **표 이름·칸 이름을 전부 소문자로 쓰세요.** 그러면 어느 쪽에서도 안 터집니다. (이 자료는 읽기 쉬우라고 한글 이름을 쓰지만, 회사에서는 소문자 영어가 많습니다)

★ 칸 이름은 어떨까요. 여기는 MySQL 이 더 너그럽습니다.

```
const 칸대소문자 = await 마이("SELECT COUNT(ID) AS c FROM SeolBi");
console.log("MySQL 에서 칸 이름을 대문자로 써도 되나:", 짧게(칸대소문자));
```

출력 MySQL 에서 칸 이름을 대문자로 써도 되나: [{"c":0}]

★ MySQL 에서 **칸 이름은 항상 대소문자를 구분하지 않습니다.** OS 와 무관합니다. PostgreSQL 은 칸 이름도 소문자로 접습니다. 결과적으로 둘 다 대소문자를 안 가립니다.

```
await 피지("DROP TABLE IF EXISTS seolbi");
await 마이("DROP TABLE IF EXISTS SeolBi");
```

UPSERT — 있으면 고치고 없으면 넣기

섹션 7

```
await 피지("DROP TABLE IF EXISTS 부품재고");
await 마이("DROP TABLE IF EXISTS 구구_부품재고");
await 피지("CREATE TABLE 부품재고 (부품 VARCHAR(20) PRIMARY KEY, 수량 INT)");
await 마이("CREATE TABLE 구구_부품재고 (부품 VARCHAR(20) PRIMARY KEY, 수량 INT)");
```

PostgreSQL — ON CONFLICT

```
const 피지업1 = await 피지(
  "INSERT INTO 부품재고 VALUES ('베어링', 10) ON CONFLICT (부품) DO UPDATE SET 수량 =
  부품재고.수량 + EXCLUDED.수량 RETURNING *",
);
const 피지업2 = await 피지(
  "INSERT INTO 부품재고 VALUES ('베어링', 5) ON CONFLICT (부품) DO UPDATE SET 수량 =
  부품재고.수량 + EXCLUDED.수량 RETURNING *",
);

console.log("PostgreSQL 첫 번째(새로 넣기):", 짧게(피지업1));
```

출력 PostgreSQL 첫 번째(새로 넣기): [{"부품":"베어링","수량":10}]

```
console.log("PostgreSQL 두 번째(더하기):", 짧게(피지업2));
```

출력 PostgreSQL 두 번째(더하기): [{"부품":"베어링","수량":15}]

MySQL — ON DUPLICATE KEY UPDATE ★ 8.0.20 부터 AS 별명 문법을 씁니다. 옛 문법 VALUES(칸)은 곧 없어집니다.

```
const 마이업1 = await 마이(
  "INSERT INTO 구구_부품재고 VALUES ('베어링', 10) AS 새값 ON DUPLICATE KEY UPDATE
  수량 = 구구_부품재고.수량 + 새값.수량",
);
const 마이업2 = await 마이(
  "INSERT INTO 구구_부품재고 VALUES ('베어링', 5) AS 새값 ON DUPLICATE KEY UPDATE
  수량 = 구구_부품재고.수량 + 새값.수량",
);

console.log("MySQL 첫 번째 affectedRows:", 마이업1.건수);
```

출력 MySQL 첫 번째 affectedRows: 1

```
console.log("MySQL 두 번째 affectedRows:", 마이업2.건수);
```

출력 MySQL 두 번째 affectedRows: 2

```
const 마이재고 = await 마이("SELECT 부품, 수량 FROM 구구_부품재고");
console.log("MySQL 결과:", 짧게(마이재고));
```

출력 MySQL 결과: [{"부품":"베어링","수량":15}]

★★ affectedRows 가 1 과 2 로 다릅니다. 이게 MySQL 의 신호입니다.

1 = 새로 넣었음 · 2 = 이미 있어서 고쳤음

★ 문서에는 “값이 그대로면 0” 이라고 적혀 있는데, 드라이버 설정에 따라 다릅니다.

이 자료의 mysql2 에서는 값이 안 바뀌어도 1 이 나옵니다. **민지 마세요.**

→ “새로 만들어졌는지” 를 알아야 하면 PostgreSQL 의 RETURNING 이 훨씬 확실합니다.

★ “있으면 그냥 넘어가기” 도 문법이 다릅니다.

```
const 피지무시 = await 피지("INSERT INTO 부품재고 VALUES ('베어링', 99) ON CONFLICT DO NOTHING");
const 마이무시 = await 마이("INSERT IGNORE INTO 구구_부품재고 VALUES ('베어링', 99)");

console.log("PostgreSQL ON CONFLICT DO NOTHING:", 짧게(피지무시));
```

출력 PostgreSQL ON CONFLICT DO NOTHING: 성공 0건

```
console.log("MySQL INSERT IGNORE:", 짧게(마이무시));
```

출력 MySQL INSERT IGNORE: 성공 0건

★★ INSERT IGNORE 는 조심해서 쓰세요. 중복만 무시하는 게 아니라 다른 에러도 경고로 바꿔 버립니다.

개념02 에서 본 “값이 잘려 들어가는” 일이 INSERT IGNORE 에서는 STRICT 를 켜도 일어납니다. → 중복만 넘기고 싶으면 ON DUPLICATE KEY UPDATE 부품 = 부품 처럼 쓰는 편이 안전합니다.

JSON

섹션 8

```
await 피지("DROP TABLE IF EXISTS 센서");
await 마이("DROP TABLE IF EXISTS 구구_센서");
await 피지("CREATE TABLE 센서 (id INT, 값 JSONB)");
await 마이("CREATE TABLE 구구_센서 (id INT, 값 JSON)");

const 센서값 = '{"온도":78,"진동":0.4}';
await 피지("INSERT INTO 센서 VALUES (1, $1)", [센서값]);
await 마이("INSERT INTO 구구_센서 VALUES (1, ?)", [센서값]);

const 피지꺼내기 = await 피지("SELECT 값->>'온도' AS 온도 FROM 센서");
const 마이꺼내기 = await 마이(`SELECT 값->>'$.온도' AS 온도 FROM 구구_센서`);

console.log("PostgreSQL 에서 값 꺼내기:", 짧게(피지꺼내기));
```

출력 PostgreSQL 에서 값 꺼내기: [{"온도":78}]

```
console.log("MySQL 에서 값 꺼내기:", 짧게(마이꺼내기));
```

출력 MySQL 에서 값 꺼내기: [{"온도":78}]

★ -> 와 ->> 기호는 같습니다. 그런데 **가리키는 방법이 다릅니다.**

PostgreSQL	- `값->>'온도'`	(키 이름을 그냥 적습니다)
MySQL	- `값->>'\$.온도'`	(JSON 경로 문법입니다)

★★ MySQL 의 경로에서 한글 키는 **큰따옴표로 감싸야 합니다.** 안 그러면 3143 에러입니다.

```
const 경로실수 = await 마이("SELECT 값->>'$.온도' AS 온도 FROM 구구_센서");
console.log("MySQL 에서 한글 키를 따옴표 없이:", 짧게(경로실수));
```

출력 MySQL 에서 한글 키를 따옴표 없이: 거절 ER_INVALID_JSON_PATH/3143

포함 검사

```
const 피지포함 = await 피지(`SELECT COUNT(*)::int AS c FROM 센서 WHERE 값 @>
'{"온도":78}'`);
const 마이포함 = await 마이(`SELECT COUNT(*) AS c FROM 구구_센서 WHERE
JSON_CONTAINS(값, '{"온도":78}')`);
const 마이꼭쇠 = await 마이(`SELECT COUNT(*) AS c FROM 구구_센서 WHERE 값 @>
'{"온도":78}'`);

console.log("PostgreSQL 포함 검사(@>):", 짧게(피지포함));
```

출력 PostgreSQL 포함 검사(@>): [{"c":1}]

```
console.log("MySQL 포함 검사(JSON_CONTAINS):", 짧게(마이포함));
```

출력 MySQL 포함 검사(JSON_CONTAINS): [{"c":1}]

```
console.log("MySQL 에서 @> 를 쓰면:", 짧게(마이꼭쇠));
```

출력 MySQL 에서 @> 를 쓰면: 거절 ER_PARSE_ERROR/1064

색인

```
const 피지진 = await 피지("CREATE INDEX 센서색인 ON 센서 USING GIN (값)");
const 마이직접 = await 마이("CREATE INDEX 센서색인 ON 구구_센서 (값)");

console.log("PostgreSQL 의 GIN 색인:", 짧게(피지진));
```

출력 PostgreSQL 의 GIN 색인: 성공 0건

```
console.log("MySQL 에서 JSON 칸에 직접 색인:", 짧게(마이직접));
```

출력 MySQL 에서 JSON 칸에 직접 색인: 거절 ER_JSON_USED_AS_KEY/3152

```
const 마이생성칸 = await 마이(
  `ALTER TABLE 구구_센서 ADD COLUMN 온도 INT AS (값->>'$.온도') STORED, ADD INDEX
  (온도)`
);
console.log("MySQL 은 생성 칸을 만들어 색인합니다:", 짧게(마이생성칸));
```

출력 MySQL 은 생성 칸을 만들어 색인합니다: 성공 1건

```
const 생성칸값 = await 마이("SELECT id, 온도 FROM 구구_센서");
console.log("생성 칸에 들어간 값:", 짧게(생성칸값));
```

출력 생성 칸에 들어간 값: [{"id":1,"온도":78}]

★ 정리하면 PostgreSQL — JSONB 는 **통째로 색인**할 수 있습니다(GIN). 어떤 키로 찾아도 빠릅니다
MySQL — JSON 칸 자체에는 색인을 못 겁니다.

자주 찾는 키마다 ****생성 칸을 만들고 거기에 색인****을 겁니다

★★ 그래서 “무엇이 들어올지 모르는 데이터” 는 PostgreSQL 이 편합니다. 미리 정해진 몇 개만 찾을
거라면 MySQL 도 충분합니다. ★ 어느 쪽이든, **자주 찾는 값은 JSON 밖으로 꺼내서 진짜 칸으로 만드는
것이 정석**입니다.

뒷정리

```
for (const 표 of ["점검", "가동상태", "이력", "새표", "부품재고", "센서", "seolbi"])
{
  await 피지(`DROP TABLE IF EXISTS ${표}`);
}
for (const 표 of ["구구_점검", "구구_가동상태", "구구_이력", "구구_새표", "구구_
부품재고", "구구_센서", "SeolBi"]) {
  await 마이(`DROP TABLE IF EXISTS ${표}`);
}

console.log("실습에 쓴 표를 모두 지웠습니다.");
```

출력 실습에 쓴 표를 모두 지웠습니다.

정리 — 문법·기능 대응표

하고 싶은 것 PostgreSQL MySQL 8.4

파라미터 \$1, \$2 (번호 있음) ? (순서대로) 자동 번호 SERIAL / IDENTITY AUTO_INCREMENT 롤백 뒤
번호 건너뛴 건너뛴 (같음) 넣은 뒤 id 받기 RETURNING id insertId / **LAST_INSERT_ID()** 여러 건 넣고 id

RETURNING 이 전부 줌 첫 번째만 알려 줌 ★ 부분 색인 CREATE INDEX ... WHERE × 없음 표현식 색인 ○ ○ (8.0.13+) 배열 타입 TEXT[] , ARRAY[...] × 없음 (JSON 으로 대신) BOOLEAN 진짜 타입 TINYINT(1) 별명 ★ 트랜잭션 안의 DDL 롤백됨 암묵적 커밋 ★★★ 표 이름 대소문자 소문자로 접음 그대로 · OS 를 따 ★★ 칸 이름 대소문자 소문자로 접음 안 가림 UPSERT ON CONFLICT DO UPDATE ON DUPLICATE KEY UPDATE 중복 무시 ON CONFLICT DO NOTHING INSERT IGNORE (주의) JSON 꺼내기 값→>'키' 값→>'\$.키' JSON 포함 @> JSON_CONTAINS() JSON 색인 GIN 으로 통째로 생성 칸 + 색인 이름 감싸기 “큰따옴표” 역따옴표

★★★ 이 표에서 별표가 붙은 다섯 개만 확실히 외우면 됩니다. 나머지는 옮길 때 문법 오류가 나서 어차피 눈에 띄니까.

★ 다음 파일에서 옮기기 전에 확인할 목록을 코드로 만듭니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 2 에서 롤백 대신 **에러가 난 INSERT** 를 넣어 보세요. (예: NOT NULL 위반) 번호가 건너뛰나요?

▫ 직접 해보기 2

MySQL 에서 ALTER TABLE 구구_점검 AUTO_INCREMENT = 100 을 해 보세요. 다음 번호가 100 이 되나요? PostgreSQL 에서는 ALTER SEQUENCE 점검_id_seq RESTART WITH 100 입니다.

▫ 직접 해보기 3

섹션 3 을 응용해서, MySQL 에서 세 건을 넣고 각각의 id 를 **정확히** 받는 코드를 짜 보세요. (힌트: 한 건씩 넣는 수밖에 없습니다. 몇 줄이 더 필요한가요?)

▫ 직접 해보기 4

섹션 5 를 ALTER TABLE 로 바꿔 보세요. BEGIN → INSERT → ALTER TABLE ... ADD COLUMN → ROLLBACK.
★ MySQL 에서 칸이 남나요? INSERT 도 남나요?

▫ 직접 해보기 5

섹션 6 에서 CREATE TABLE “설비 목록” 처럼 공백이 든 이름을 양쪽에서 만들어 보세요. 어떻게 감싸야 하나요?

▫ 직접 해보기 6

섹션 7 의 UPSERT 를 **여러 건 한꺼번에** 해 보세요. VALUES ('a',1),('b',2),('c',3) 에서 b 만 이미 있으면 MySQL 의 affectedRows 는 얼마일까요?

▹ 직접 해보기 7

섹션 8 에서 JSON 배열을 넣어 보세요. ‘{“태그”:[“점검”,“긴급”]}’ 에서 ‘긴급’ 이 들어 있는 줄을 양쪽에서 찾는 SQL 을 각각 써 보세요.

▹ 직접 해보기 8

PostgreSQL 의 TEXT[] 칸을 MySQL 로 옮긴다면 어떻게 설계하시겠습니까? 방법 두 가지를 적어 보세요.

자주 하는 실수

이런 실수를 합니다

\$1 을 ? 로 기계적으로 바꾼다

같은 번호를 두 번 쓴 쿼리가 있으면 값 개수가 안 맞아 터집니다. 반대로 ? 를 \$1,\$2,... 로 바꿀 때는 순서를 세어야 합니다.

이런 실수를 합니다

자동 번호가 연속일 거라고 믿는다

★ 양쪽 다 건너뛴니다. 롤백해도 번호는 안 돌아옵니다. “번호 = 건수” 로 계산하는 코드는 언젠가 틀립니다.

이런 실수를 합니다

MySQL 에서 여러 건 넣고 insertId + i 로 번호를 계산한다

★★★ 연속이라는 보장이 없습니다. 설정과 동시성에 따라 띄엄띄엄 붙습니다. 운영에서 사용자가 몰릴 때만 틀려서 재현이 안 됩니다. 최악의 버그입니다.

이런 실수를 합니다

마이그레이션을 트랜잭션으로 감싸면 MySQL 도 안전할 거라고 믿는다

★★★ 안 됩니다. DDL 이 암묵적 커밋을 일으킵니다. 섹션 5 에서 확인했습니다. INSERT 까지 같이 커밋됩니다.

이런 실수를 합니다

맥에서 개발하고 리눅스에 배포한다 + 표 이름에 대문자를 쓴다

★★ 맥에서는 잘 돌고 서버에서만 Table doesn't exist 가 납니다. 표 이름은 소문자로 통일하세요.

이런 실수를 합니다

MySQL의 BOOLEAN을 진짜 BOOLEAN이라고 믿는다

TINYINT(1)입니다. 5도 들어가고, 그러면 = true로 안 잡힙니다. WHERE 가동이나 WHERE 가동 <> 0을 쓰세요.

이런 실수를 합니다

INSERT IGNORE를 “중복만 넘기는 것”으로 안다

다른 에러까지 경고로 바꿉니다. 값이 조용히 잘려 들어갈 수 있습니다.

이런 실수를 합니다

MySQL의 JSON 경로에 한글 키를 그냥 쓴다

\$.온도는 3143 에러입니다. \$.“온도”로 감싸야 합니다. 영어 키만 쓰면 이 문제가 안 생깁니다.

```
await PG.end();
await MY.end();
```

옳길 때 확인할 것

```
RUN node 개념05_옳길_때_확인할_것.js
```

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 잠금이 풀리기를 기다리는 실험이 있어서 5초쯤 걸립니다.

개념02~04 에서 차이를 다 봤습니다. 이제 **실제로 옳길 때** 할 일입니다.

① 옳길 SQL 을 자동으로 훑어서 문제될 곳을 찾는 도구를 만듭니다 ② 격리수준 기본값이 다른 것을 확인합니다 (07단원과 이어집니다) ③ MySQL 의 갭 락을 실제로 재현합니다 — 잠금 범위가 다릅니다 ④ 성능 비교를 왜 하면 안 되는지 이야기합니다 ⑤ 어느 쪽을 고를지 정직하게 정리합니다 ⑥ MariaDB 는 무엇인지 한 문단

```
import pg from "pg";
import mysql from "mysql2/promise";
```

불기 · 도우미

섹션 0

★ 이 파일은 잠금 실험을 하느라 **양쪽 다 두 개씩** 붙습니다.

```

const 피지설정 = { host: "127.0.0.1", port: 5434, user: "factory", password:
"secret", database: "factory_db" };
const 마이설정 = { host: "127.0.0.1", port: 3307, user: "factory", password:
"secret", database: "factory_db" };

let PG김 = null;
let PGOI = null;
let MY김 = null;
let MYOI = null;

try {
  PG김 = new pg.Client(피지설정);
  await PG김.connect();
  PGOI = new pg.Client(피지설정);
  await PGOI.connect();
  MY김 = await mysql.createConnection(마이설정);
  MYOI = await mysql.createConnection(마이설정);
} catch (에러) {
  console.log("★ 데이터베이스에 못 붙었습니다:", 에러.code || 에러.message);
  console.log(" docker compose up -d 를 먼저 실행하세요.");
  for (const 연결 of [PG김, PGOI]) if (연결) await 연결.end().catch(() => {});
  for (const 연결 of [MY김, MYOI]) if (연결) await 연결.end().catch(() => {});
  process.exit(0);
}

await PG김.query("CREATE SCHEMA IF NOT EXISTS 단원09");
await PG김.query("SET search_path TO 단원09");
await PGOI.query("SET search_path TO 단원09");

async function 피지(연결, sql, 값 = []) {
  try {
    const 결과 = await 연결.query(sql, 값);
    const 조회인가 = 결과.command === "SELECT" || 결과.command === "SHOW";

    return { 됨: true, 값: 결과.rows, 조회: 조회인가, 건수: 결과.rowCount ?? 0 };
  } catch (에러) {
    return { 됨: false, 코드: 에러.code };
  }
}

```

```

}

async function 마이(연결, sql, 값 = []) {
  try {
    const [행들] = await 연결.query(sql, 값);
    const 조회인가 = Array.isArray(행들);

    return {
      됨: true,
      값: 조회인가 ? 행들 : [],
      조회: 조회인가,
      건수: 조회인가 ? 행들.length : 행들.affectedRows,
    };
  } catch (에러) {
    return { 됨: false, 코드: 에러.code, 번호: 에러.errno };
  }
}

function 짧게(결과) {
  if (!결과.됨) return `거절 ${결과.코드}${결과.번호 === undefined ? "" : "/" + 결과.번호}`;
  if (결과.조회 || 결과.값.length > 0) return JSON.stringify(결과.값);
  return `성공 ${결과.건수}건`;
}

```

★ 옮길 SQL 을 훑는 점검 도구

섹션 1

09

쿼리가 수백 개인 프로젝트를 눈으로 훑을 수는 없습니다. **찾아 주는 코드**를 만듭니다. 완벽하지 않아도 90%는 잡아 줍니다.

```
const 옳길때규칙 = [
  {
    이름: "|| 로 글자 잇기",
    정규식: /\|\|/,
    심각도: "***",
    문제: "MySQL 에서는 OR 로 해석되어 숫자가 나옵니다. 에러가 안 납니다",
    해결: "concat() 으로 바꾸세요",
  },
  {
    이름: "RETURNING",
    정규식: /\bRETURNING\b/i,
    심각도: "**",
    문제: "MySQL 에 없는 문법입니다 (1064 문법 오류)",
    해결: "insertId 를 받거나, 넣은 뒤 다시 SELECT 하세요",
  },
  {
    이름: "$1 파라미터",
    정규식: /\$d+/,
    심각도: "**",
    문제: "MySQL 은 ? 만 씁니다",
    해결: "? 로 바꾸고, 같은 번호를 두 번 쓴 곳은 값도 두 번 넘기세요",
  },
  {
    이름: "SERIAL 자동번호",
    정규식: /\b(BIG|SMALL)?SERIAL\b/i,
    심각도: "*",
    문제: "MySQL 에는 SERIAL 이 다른 뜻입니다 (BIGINT UNSIGNED AUTO_INCREMENT UNIQUE)",
    해결: "INT AUTO_INCREMENT PRIMARY KEY 로 바꾸세요",
  },
  {
    이름: "GENERATED ... AS IDENTITY",
    정규식: /GENERATED\s+(ALWAYS|BY\s+DEFAULT)\s+AS\s+IDENTITY/i,
    심각도: "*",
    문제: "MySQL 에 없는 문법입니다",
    해결: "AUTO_INCREMENT 로 바꾸세요",
  },
]
```

```

{
  이름: "부분 색인 (CREATE INDEX ... WHERE)",
  정규식: /CREATE\s+(UNIQUE\s+)?INDEX[^\s;]*\bWHERE\b/i,
  심각도: "★★",
  문제: "MySQL 에 부분 색인이 없습니다",
  해결: "조건을 뺀 전체 색인으로 만들거나, 생성 칸을 만들어 색인하세요",
},
{
  이름: "배열 타입",
  정규식: /\bARRAY\s*\[\s*\w+\s*\[\s*\s*\]/,
  심각도: "★★",
  문제: "MySQL 에 배열 타입이 없습니다",
  해결: "표를 따로 만들거나 JSON 배열로 바꾸세요",
},
{
  이름: "JSONB",
  정규식: /\bJSONB\b/i,
  심각도: "★",
  문제: "MySQL 은 JSON 하나뿐입니다. 색인 방법이 다릅니다",
  해결: "JSON 으로 바꾸고, 자주 찾는 키는 생성 칸 + 색인으로 빼세요",
},
{
  이름: "@> 포함 연산자",
  정규식: /@>/,
  심각도: "★★",
  문제: "MySQL 에 없는 연산자입니다 (1064)",
  해결: "JSON_CONTAINS() 로 바꾸세요",
},
{
  이름: "= true / = false 비교",
  정규식: /=\s*(TRUE|FALSE)\b/i,
  심각도: "★★",
  문제: "MySQL 의 BOOLEAN 은 TINYINT(1) 이라 1·0 이 아닌 값을 놓칩니다",
  해결: "WHERE 칸 또는 WHERE 칸 <> 0 으로 바꾸세요",
},
{

```

```

    이름: "ILIKE",
    정규식: /\bILIKE\b/i,
    심각도: "*",
    문제: "MySQL 에 없는 문법입니다",
    해결: "그냥 LIKE 를 쓰세요. MySQL 기본 collation 은 이미 대소문자를 무시합니다",
  },
  {
    이름: "NULLS FIRST / NULLS LAST",
    정규식: /\bNULLS\s+(FIRST|LAST)\b/i,
    심각도: "**",
    문제: "MySQL 에 없는 문법이고, NULL 의 기본 자리도 반대입니다",
    해결: "ORDER BY (칸 IS NULL), 칸 으로 바꾸세요",
  },
  {
    이름: ":: 타입 바꾸기",
    정규식: /:::/,
    심각도: "*",
    문제: "MySQL 에 없는 문법입니다",
    해결: "CAST(값 AS 타입) 으로 바꾸세요. 양쪽에서 다 됩니다",
  },
  {
    이름: "ON CONFLICT",
    정규식: /\bON\s+CONFLICT\b/i,
    심각도: "*",
    문제: "MySQL 은 ON DUPLICATE KEY UPDATE 입니다",
    해결: "ON DUPLICATE KEY UPDATE 로 바꾸세요. 어느 칸이 충돌인지는 못 고릅니다",
  },
  {
    이름: "length() 로 길이 재기",
    정규식: /\blength\s*\(/i,
    심각도: "**",
    문제: "MySQL 의 LENGTH() 는 바이트를 셉니다. 한글은 3배가 됩니다",
    해결: "char_length() 로 바꾸세요",
  },
  {
    이름: "트랜잭션 안의 DDL",
    정규식: /\bBEGIN\b[\s\S]*\b(CREATE|ALTER|DROP|TRUNCATE)\s+TABLE\b/i,
    심각도: "***",
    문제: "MySQL 은 DDL 에서 암묵적 커밋이 일어나 롤백이 안 됩니다",
    해결: "변경을 파일 하나에 하나씩 나누고, 되돌리는 스크립트를 미리 만드세요",
  },
];

function 옳기기점검(SQL모음) {
  return 옳길때규칙.filter((규칙) => 규칙.정규식.test(SQL모음));
}

```

실제 프로젝트에서 흔히 보는 SQL 을 넣어 봅니다.

```
const 옮길SQL = `
  CREATE TABLE 설비 (
    id      SERIAL PRIMARY KEY,
    이름    VARCHAR(50) NOT NULL,
    라인    VARCHAR(10),
    태그    TEXT[],
    점검정보 JSONB,
    가동    BOOLEAN DEFAULT true
  );

  CREATE INDEX 설비_미점검 ON 설비 (id) WHERE 점검정보 IS NULL;

  INSERT INTO 설비 (이름, 라인) VALUES ($1, $2) RETURNING id;

  SELECT 이름 || ' (' || 라인 || ')' AS 표시이름
  FROM 설비
  WHERE 가동 = true
        AND length(이름) <= 20;

  SELECT * FROM 설비 WHERE 이름 ILIKE $1 ORDER BY 라인 NULLS LAST;

  UPDATE 설비 SET 점검정보 = $1::jsonb WHERE id = $2;

  SELECT * FROM 설비 WHERE 점검정보 @> '{"상태":"불량"}';

  INSERT INTO 재고 (부품, 수량) VALUES ($1, $2)
  ON CONFLICT (부품) DO UPDATE SET 수량 = 재고.수량 + EXCLUDED.수량;
`;

const 걸린것 = 옮기기점검(옮길SQL);

console.log(`[점검 결과] 규칙 ${옮길때규칙.length} 개 중 ${걸린것.length} 개가
걸렸습니다`);
```

출력 [점검 결과] 규칙 16 개 중 14 개가 걸렸습니다

```
for (const 규칙 of 걸린것) {
  console.log(` ${규칙.심각도} ${규칙.이름}`);
  console.log(`     문제 - ${규칙.문제}`);
  console.log(`     해결 - ${규칙.해결}`);
}
```

출력 *** || 로 글자 잇기
 문제 - MySQL 에서는 OR 로 해석되어 숫자가 나옵니다. 예러가 안

해결 - concat() 으로 바꾸세요

- ★★ RETURNING
 - 문제 - MySQL 에 없는 문법입니다 (1064 문법 오류)
 - 해결 - insertId 를 받거나, 넣은 뒤 다시 SELECT 하세요
- ★★ \$1 파라미터
 - 문제 - MySQL 은 ? 만 씁니다
 - 해결 - ? 로 바꾸고, 같은 번호를 두 번 쓴 곳은 값도 두 번 넘기세요
- ★ SERIAL 자동번호
 - 문제 - MySQL 에는 SERIAL 이 다른 뜻입니다 (BIGINT UNSIGNED AUTO_INCREMENT UNIQUE)
 - 해결 - INT AUTO_INCREMENT PRIMARY KEY 로 바꾸세요
- ★★ 부분 색인 (CREATE INDEX ... WHERE)
 - 문제 - MySQL 에 부분 색인이 없습니다
 - 해결 - 조건을 뺀 전체 색인으로 만들거나, 생성 칸을 만들어 색인하세요
- ★★ 배열 타입
 - 문제 - MySQL 에 배열 타입이 없습니다
 - 해결 - 표를 따로 만들거나 JSON 배열로 바꾸세요
- ★ JSONB
 - 문제 - MySQL 은 JSON 하나뿐입니다. 색인 방법이 다릅니다
 - 해결 - JSON 으로 바꾸고, 자주 찾는 키는 생성 칸 + 색인으로 빼세요
- ★★ @> 포함 연산자
 - 문제 - MySQL 에 없는 연산자입니다 (1064)
 - 해결 - JSON_CONTAINS() 로 바꾸세요
- ★★ = true / = false 비교
 - 문제 - MySQL 의 BOOLEAN 은 TINYINT(1) 이라 1·0 이 아닌 값을 놓칩니다
 - 해결 - WHERE 칸 또는 WHERE 칸 <> 0 으로 바꾸세요
- ★ ILIKE
 - 문제 - MySQL 에 없는 문법입니다
 - 해결 - 그냥 LIKE 를 쓰세요. MySQL 기본 collation 은 이미 대소문자를 무시합니다
- ★★ NULLS FIRST / NULLS LAST
 - 문제 - MySQL 에 없는 문법이고, NULL 의 기본 자리도 반대입니다
 - 해결 - ORDER BY (칸 IS NULL), 칸 으로 바꾸세요
- ★ :: 타입 바꾸기
 - 문제 - MySQL 에 없는 문법입니다
 - 해결 - CAST(값 AS 타입) 으로 바꾸세요. 양쪽에서 다 됩니다
- ★ ON CONFLICT
 - 문제 - MySQL 은 ON DUPLICATE KEY UPDATE 입니다
 - 해결 - ON DUPLICATE KEY UPDATE 로 바꾸세요. 어느 칸이 충돌인지는 못 고릅니다
- ★★ length() 로 길이 재기
 - 문제 - MySQL 의 LENGTH() 는 바이트를 셉니다. 한글은 3배가 됩니다
 - 해결 - char_length() 로 바꾸세요

★ 이 도구가 진짜 쓸모 있는 곳은 **개수**입니다. “고칠 곳이 13개 있다” 가 아니라 “★★★★ 짜리가 하나 있다” 를 먼저 봅니다.

```
const 심각한것 = 걸린것.filter((규칙) => 규칙.심각도 === "★★★★");
console.log("에러 없이 조용히 틀리는 것(★★★★) 개수:", 심각한것.length);
```

출력 여러 없이 조용히 틀리는 것(★★) 개수: 1

```
console.log("그것이 무엇인가:", 심각한것.map((규칙) => 규칙.이름).join(", "));
```

출력 그것이 무엇인가: || 로 글자 잇기

★★★ 나머지는 옮기면 **문법 오류로 터져서** 어차피 눈에 띄니다. || 하나만 조용히 틀립니다. 그래서 가장 먼저 고쳐야 합니다.

★ 이 도구의 한계도 정직하게 적어 둡니다.

· 정규식이라 **글자 안에 든 것도 잡습니다**. 'a || b' 같은 문자열도 걸립니다 · 반대로 여러 줄에 나뉜 문법은 놓칠 수 있습니다 · 진짜 파서가 아니라서 **거짓 경고가 납니다**

그래도 씁니다. 수백 개 쿼리에서 **볼 곳을 좁혀 주는 것만으로** 충분합니다. ★ 경고가 하나도 없다고 안전한 게 아닙니다. 개념03 의 조용한 차이는 이걸로 못 잡습니다.

거짓 경고가 어떻게 나는지 직접 봅니다.

```
const 거짓경고SQL = "SELECT '규칙: a || b 는 OR 입니다' AS 안내문";
const 거짓걸림 = 옮기기점검(거짓경고SQL);

console.log("글자 안에 든 || 도 걸리나:", 거짓걸림.map((규칙) => 규칙.이름).join(", "));
```

출력 글자 안에 든 || 도 걸리나: || 로 글자 잇기

★ 이 SQL 은 MySQL 에서도 잘 됩니다. 그런데 경고가 났습니다. → 사람이 한 번 보고 판단해야 합니다. 도구는 **후보를 좁혀 줄 뿐**입니다.

반대로 못 잡는 것도 봅니다.

```
const 못잡는SQL = "SELECT 아이디 FROM 작업자 WHERE 아이디 = '설비관리자'";
console.log("대소문자 차이를 잡아내나:", 옮기기점검(못잡는SQL).length, "개 걸림");
```

출력 대소문자 차이를 잡아내나: 0 개 걸림

★★★ 이 쿼리는 **양쪽에서 답이 다를 수 있습니다**. (개념03 섹션 2) 그런데 도구는 아무 말도 안 합니다. 문법에는 아무 문제가 없으니까요.

격리수준 기본값이 다릅니다

섹션 2

```
const 피지격리 = await 피지(PG김, "SHOW transaction_isolation");
const 마이격리 = await 마이(MY김, "SELECT @@SESSION.transaction_isolation AS
격리수준");

console.log("PostgreSQL 기본 격리수준:", 피지격리.값[0].transaction_isolation);
```

출력 PostgreSQL 기본 격리수준: read committed

```
console.log("MySQL 기본 격리수준:", 마이격리.값[0].격리수준);
```

출력 MySQL 기본 격리수준: REPEATABLE-READ

★★ 07단원에서 격리수준을 했습니다. 거기서 확인한 것을 다시 씁니다.

READ COMMITTED — 매 SELECT 마다 그 순간의 최신 커밋을 봅니다 REPEATABLE READ — 트랜잭션이 시작한 순간의 사진을 계속 봅니다

★ 그래서 옳기면 무슨 일이 나나

PostgreSQL → MySQL

긴 트랜잭션 안에서 같은 SELECT 를 두 번 하면 **답이 안 바뀌게 됩니다.**
"왜 방금 다른 사람이 넣은 게 안 보이죠?" 라는 문의가 옵니다.

MySQL → PostgreSQL

같은 SELECT 가 **다른 답을 주기 시작합니다.**
한 트랜잭션 안에서 두 번 세어서 비교하던 코드가 어긋납니다.

★★ 맞추는 법은 간단합니다. **접속할 때 격리수준을 명시하세요.** 기본값에 기대지 마세요. 어차피 서버마다 다를 수 있습니다.

```
await 피지(PG김, "BEGIN ISOLATION LEVEL REPEATABLE READ");
const 바꾼피지 = await 피지(PG김, "SHOW transaction_isolation");
console.log("PostgreSQL 에서 명시하면:", 바꾼피지.값[0].transaction_isolation);
```

출력 PostgreSQL 에서 명시하면: repeatable read

```
await 피지(PG김, "ROLLBACK");

await 마이(MY김, "SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED");
const 바꾼마이 = await 마이(MY김, "SELECT @@SESSION.transaction_isolation AS
격리수준");
console.log("MySQL 에서 명시하면:", 바꾼마이.값[0].격리수준);
```

출력 MySQL 에서 명시하면: READ-COMMITTED

```
await 마이(MY김, "SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ");
```

★★ 같은 이름, 다른 방법 — 갭 락

섹션 3

07단원에서 이런 것을 봤습니다. PostgreSQL 의 REPEATABLE READ 는 팬텀 읽기까지 막습니다. MySQL 의 REPEATABLE READ 도 막습니다. 그런데 막는 방법이 다릅니다.

PostgreSQL — 스냅샷(MVCC). 시작 순간의 사진을 봅니다. 아무것도 안 잠급니다 MySQL — 넥스트키 락(next-key lock). 줄과 줄 사이의 빈틈까지 잠급니다

평범한 SELECT 만 하면 결과가 같아 보입니다. 먼저 그걸 확인합니다.

```
await 피지(PG김, "DROP TABLE IF EXISTS 생산실적");
await 마이(MY김, "DROP TABLE IF EXISTS 구구_생산실적");
await 피지(PG김, "CREATE TABLE 생산실적 (번호 INT PRIMARY KEY, 수량 INT)");
await 마이(MY김, "CREATE TABLE 구구_생산실적 (번호 INT PRIMARY KEY, 수량 INT)");
await 피지(PG김, "INSERT INTO 생산실적 VALUES (10, 100), (20, 200)");
await 마이(MY김, "INSERT INTO 구구_생산실적 VALUES (10, 100), (20, 200)");
```

김반장이 트랜잭션을 열고 두 번 셉니다. 그 사이에 이반장이 한 건 넣습니다.

```
await 피지(PG김, "BEGIN ISOLATION LEVEL REPEATABLE READ");
const 피지첫번째 = await 피지(PG김, "SELECT COUNT(*)::int AS 개수 FROM 생산실적");
await 피지(PG김, "INSERT INTO 생산실적 VALUES (30, 300)");
const 피지두번째 = await 피지(PG김, "SELECT COUNT(*)::int AS 개수 FROM 생산실적");
await 피지(PG김, "COMMIT");
```

```
await 마이(MY김, "BEGIN");
const 마이첫번째 = await 마이(MY김, "SELECT COUNT(*) AS 개수 FROM 구구_생산실적");
await 마이(MY김, "INSERT INTO 구구_생산실적 VALUES (30, 300)");
const 마이두번째 = await 마이(MY김, "SELECT COUNT(*) AS 개수 FROM 구구_생산실적");
await 마이(MY김, "COMMIT");
```

```
console.log("PostgreSQL - 처음", 피지첫번째.값[0].개수, "건 · 나중", 피지두번째.값[0].개수, "건");
```

출력 PostgreSQL - 처음 2 건 · 나중 2 건

```
console.log("MySQL - 처음", 마이첫번째.값[0].개수, "건 · 나중", 마이두번째.값[0].개수, "건");
```

출력 MySQL - 처음 2 건 · 나중 2 건

★ 여기까지는 결과가 같습니다. 둘 다 팬텀을 막았습니다. 그런데 이반장의 INSERT 는 **양쪽 다 바로 성공했습니다**. 안 막혔습니다. 평범한 SELECT 는 아무것도 잠그지 않기 때문입니다.

★★★ SELECT ... FOR UPDATE 로 바꾸면 완전히 달라집니다.

기다리는 시간을 짧게 잡습니다. 안 그러면 MySQL 은 50초를 기다립니다.

```
await 피지(PG이, "SET lock_timeout = '1s'");
await 마이(MY이, "SET SESSION innodb_lock_wait_timeout = 1");

await 피지(PG김, "DELETE FROM 생산실적 WHERE 번호 = 30");
await 마이(MY김, "DELETE FROM 구구_생산실적 WHERE 번호 = 30");
```

— PostgreSQL: 10~20 번을 FOR UPDATE 로 잠그고, 그 사이(15번)에 넣어 봅니다

```
await 피지(PG김, "BEGIN");
await 피지(PG김, "SELECT * FROM 생산실적 WHERE 번호 BETWEEN 10 AND 20 FOR UPDATE");

const 피지시작 = process.hrtime.bigint();
const 피지사이 = await 피지(PG이, "INSERT INTO 생산실적 VALUES (15, 150)");
const 피지걸린 = Number(process.hrtime.bigint() - 피지시작) / 1e6;

console.log("PostgreSQL - 잠긴 범위 사이(15번)에 INSERT:", 짧게(피지사이));
```

출력 PostgreSQL - 잠긴 범위 사이(15번)에 INSERT: 성공 1건

```
console.log(`PostgreSQL - 걸린 시간 ${피지걸린.toFixed(0)} ms`);
```

출력 PostgreSQL - 걸린 시간 1 ms
값이 다를 수 있음

```
await 피지(PG김, "ROLLBACK");
await 피지(PG김, "DELETE FROM 생산실적 WHERE 번호 = 15");
```

— MySQL: 똑같이 해 봅니다

```
await 마이(MY김, "BEGIN");
await 마이(MY김, "SELECT * FROM 구구_생산실적 WHERE 번호 BETWEEN 10 AND 20 FOR UPDATE");

const 마이시작 = process.hrtime.bigint();
const 마이사이 = await 마이(MY이, "INSERT INTO 구구_생산실적 VALUES (15, 150)");
const 마이걸린 = Number(process.hrtime.bigint() - 마이시작) / 1e6;

console.log("MySQL - 잠긴 범위 사이(15번)에 INSERT:", 짧게(마이사이));
```

출력 MySQL - 잠긴 범위 사이(15번)에 INSERT: 거절
ER_LOCK_WAIT_TIMEOUT/1205

```
console.log(`MySQL      - 걸린 시간 ${마이걸린.toFixed(0)} ms`);
```

```
출력      MySQL      - 걸린 시간 1004 ms  
값이 다를 수 있음
```

```
console.log("PostgreSQL 은 안 막히고 MySQL 은 막혔나:", 피지사이.됨 === true &&  
마이사이.됨 === false);
```

```
출력      PostgreSQL 은 안 막히고 MySQL 은 막혔나: true
```

★★★ 같은 SQL, 같은 격리수준인데 **한쪽만 막혔습니다.**

MySQL 은 10 과 20 사이의 **빈 자리(갭)** 까지 잠급니다. 거기에 새 줄이 들어오면 아까 읽은 범위가 달라지기 때문입니다. PostgreSQL 은 잠글 필요가 없습니다. 어차피 사진을 보고 있으니까요.

★★ 더 놀라운 것: **범위 밖도 막힙니다.**

```
await 마이(MY김, "ROLLBACK");  
await 마이(MY김, "DELETE FROM 구구_생산실적 WHERE 번호 = 15");  
  
await 마이(MY김, "BEGIN");  
await 마이(MY김, "SELECT * FROM 구구_생산실적 WHERE 번호 BETWEEN 10 AND 20 FOR  
UPDATE");  
  
const 마이바깥 = await 마이(MY이, "INSERT INTO 구구_생산실적 VALUES (100, 999)");  
console.log("MySQL      - 범위 한참 밖(100번)에 INSERT:", 짧게(마이바깥));
```

```
출력      MySQL      - 범위 한참 밖(100번)에 INSERT: 거절 ER_LOCK_WAIT_TIMEOUT/1205
```

```
await 마이(MY김, "ROLLBACK");
```

★★★ 100번은 20번보다 훨씬 큰데도 막혔습니다. 범위를 훑다가 20번 다음에 아무것도 없는 것을 확인하면서 **"20번 뒤의 모든 자리"** 를 잠갔기 때문입니다.

→ MySQL 의 REPEATABLE READ 에서 `FOR UPDATE` 나 `UPDATE ... WHERE` 범위 는

```
생각보다 **훨씬 넓게 잠급니다.** 동시 처리량이 여기서 떨어집니다.
```

★ 어떻게 줄이나 ① 잠그는 범위를 **정확한 값 하나로** 좁힙니다 (`WHERE 번호 = 15`) ② 색인이 없으면 MySQL 은 **표 전체를** 잠급니다. 색인을 꼭 만드세요 ③ 정말 필요하면 격리수준을 `READ COMMITTED` 로 낮춥니다. 갭 락이 거의 사라집니다 ④ 트랜잭션을 짧게 잡습니다. 이건 어느 DB 에서나 맞는 말입니다

★★ 반대 방향도 조심하세요. MySQL 에서 "갭 락이 막아 주겠지" 하고 짤 코드를 PostgreSQL 로 옮기면 **아무도 안 막아 줍니다.** 중복이 들어갑니다. → 중복을 막는 것은 잠금이 아니라 **UNIQUE 제약**입니다. 어느 DB 든 그게 정답입니다.

★ 성능 비교는 하지 않습니다

섹션 4

“그래서 어느 쪽이 빠른가요?” 를 꼭 묻습니다. 이 자료는 그 숫자를 신지 않습니다. **공정하게 잴 수가 없기** 때문입니다.

왜 어려운지 실제 설정 값으로 보여 드립니다.

```
const 마이설정값 = await 마이(
  MY김,
  "SELECT @@innodb_buffer_pool_size AS 버퍼, @@innodb_flush_log_at_trx_commit AS
  로그동기화, @@max_connections AS 최대접속",
);
const 피지설정값 = await 피지(
  PG김,
  "SELECT current_setting('shared_buffers') AS 버퍼,
  current_setting('synchronous_commit') AS 로그동기화,
  current_setting('max_connections') AS 최대접속",
);

console.log("MySQL      - 버퍼", 마이설정값.값[0].버퍼, ". 로그동기화", 마이설정값.값
[0].로그동기화, ". 최대접속", 마이설정값.값[0].최대접속);
```

출력 MySQL - 버퍼 134217728 · 로그동기화 1 · 최대접속 151

```
console.log("PostgreSQL - 버퍼", 피지설정값.값[0].버퍼, ". 로그동기화", 피지설정값.값
[0].로그동기화, ". 최대접속", 피지설정값.값[0].최대접속);
```

출력 PostgreSQL - 버퍼 128MB · 로그동기화 on · 최대접속 100

★ 숫자가 비슷해 보입니다. 그런데 **뜻이 다릅니다**.

MySQL 의 버퍼(innodb_buffer_pool_size) — 데이터를 담는 **거의 유일한 캐시**입니다.

그래서 실무에서는 메모리의 60~70% 를 줍니다.

PostgreSQL 의 버퍼(shared_buffers) — **운영체제 캐시와 나눠 씁니다**.

그래서 25% 정도만 주는 것이 보통입니다.

이 상태로 같은 쿼리를 재면 **설정 차이를 재는 것**이지 DB 를 재는 게 아닙니다.

★★ 공정한 비교가 어려운 이유를 정리하면

① 설정 — 위에서 봤습니다. 기본값의 철학이 다릅니다 ② 하드웨어 — 디스크가 SSD 인지, fsync 가 진짜
도는데 따라 몇 배가 갑니다 ③ 워크로드 — 읽기가 많은지 쓰기가 많은지, 조인이 깊은지에 따라 순위가

바뀝니다 ④ 데이터 크기 — 메모리에 다 들어가면 둘 다 빠릅니다. 안 들어갈 때 걸립니다 ⑤ 버전 — 매년 바뀝니다. 작년 벤치마크는 이미 낡았습니다

★★★ 그래서 이렇게 하세요.

우리 데이터, 우리 쿼리, 우리 서버로 직접 재세요.

① 운영 데이터를 복사해서 (개인정보는 가리고) 양쪽에 넣습니다 ② 실제로 많이 도는 쿼리 10개를 고릅니다 (느린 쿼리 로그를 보면 나옵니다) ③ 같은 색인을 만듭니다. ★ 이걸 빼먹으면 비교가 아예 무의미합니다 ④ 위밍업을 한 번 돌리고, 그 다음부터 여러 번 재서 **중앙값**을 씁니다 ⑤ 응답 시간뿐 아니라 **동시 접속을 늘렸을 때** 어떻게 되는지도 봅니다

★ 인터넷의 “MySQL 이 X% 빠르다” 는 글은 대부분 ③ 을 안 지켰습니다.

★ 어느 쪽을 고를까

섹션 5

```
const 고르기 = [
  ["새로 시작하는 서비스", "PostgreSQL", "기능이 넓고 표준을 잘 지킵니다. 요즘 호스팅도 Postgres 쪽이 많습니다"],
  ["회사에 이미 MySQL 이 있음", "MySQL", "DB 를 바꾸는 건 큰 결정입니다. 8.4 는 충분히 엄격합니다"],
  ["복잡한 조회·분석이 많음", "PostgreSQL", "CTE·윈도 함수·부분 색인·표현식 색인이 다 있습니다"],
  ["단순 조회가 아주 많음", "둘 다 괜찮음", "이 영역은 설정과 색인이 DB 선택보다 훨씬 큼니다"],
  ["JSON 을 많이 씁니다", "PostgreSQL", "JSONB 는 통째로 색인이 됩니다"],
  ["지리정보를 다룹니다", "PostgreSQL", "PostGIS 가 사실상 표준입니다"],
  ["마이그레이션을 자주 합니다", "PostgreSQL", "트랜잭션 DDL 이 있어서 실패해도 되돌아갑니다"],
  ["운영할 사람이 MySQL 만 압니다", "MySQL", "★ 이게 생각보다 큰 이유입니다. 새벽에 고칠 사람이 있어야 합니다"],
  ["국내 SI·공공 납품", "MySQL 이 많음", "규격에 들어가 있는 경우가 많습니다. 확인하고 시작하세요"],
];
```

```
console.log("[상황별로 무엇을 고를까]");
```

출력 [상황별로 무엇을 고를까]

```
for (const [상황, 추천, 이유] of 고르기) {
  console.log(`  · ${상황} → ${추천}`);
  console.log(`    ${이유}`);
}
```

출력 · 새로 시작하는 서비스 → PostgreSQL
 기능이 넓고 표준을 잘 지킵니다. 요즘 호스팅도 Postgres 쪽이

- 회사에 이미 MySQL 이 있음 → MySQL
DB 를 바꾸는 건 큰 결정입니다. 8.4 는 충분히 엄격합니다
- 복잡한 조회·분석이 많음 → PostgreSQL
CTE·윈도 함수·부분 색인·표현식 색인이 다 있습니다
- 단순 조회가 아주 많음 → 둘 다 괜찮음
이 영역은 설정과 색인이 DB 선택보다 훨씬 큼니다
- JSON 을 많이 씁니다 → PostgreSQL
JSONB 는 통째로 색인이 됩니다
- 지리정보를 다룹니다 → PostgreSQL
PostGIS 가 사실상 표준입니다
- 마이그레이션을 자주 합니다 → PostgreSQL
트랜잭션 DDL 이 있어서 실패해도 되돌아갑니다
- 운영할 사람이 MySQL 만 압니다 → MySQL
★ 이게 생각보다 큰 이유입니다. 새벽에 고칠 사람이 있어야 합니다
- 국내 SI·공공 납품 → MySQL 이 많음
규격에 들어가 있는 경우가 많습니다. 확인하고 시작하세요

★★ 이 자료가 PostgreSQL 로 가르치는 이유 — 실측에 근거해서 정리하면

① **설치가 0 입니다.** 01~06단원은 PGlite 로 `npm install` 만 하면 됩니다.

MySQL 은 반드시 서버나 Docker 가 필요합니다.
수업 첫 시간을 설치로 날리지 않는 것이 큼니다

② **배울 것이 더 많습니다.** RETURNING·부분 색인·배열·트랜잭션 DDL·JSONB 는

MySQL 에 없습니다. 없는 쪽에서 배우면 있는 것을 모르고 지나갑니다

③ **표준에 가깝습니다.** 개념03 에서 봤듯이 `||`, `LENGTH`, `NULL` 정렬,

대소문자 비교 모두 PostgreSQL 쪽이 표준입니다.
표준을 익히면 다른 DB 로 갈 때 고칠 것이 적습니다

★★★ 그런데 **MySQL 을 무시하면 안 됩니다.** 이것도 실측으로 확인했습니다.

① 개념02 에서 일곱 가지를 재 봤습니다. **7 대 7 로 똑같이 거절했습니다.**

"MySQL 은 허술하다" 는 평가는 낡았습니다

② 개념03 의 중복확인 시나리오에서는 **MySQL 이 더 안전하게 동작했습니다** ③ 국내 제조·SI·공공 현장에 여전히 많습니다. 이 자료의 도메인이 특히 그렇습니다

★ 정리하면 **Postgres 로 배우고, MySQL 을 만나면 이 단원의 표를 확인하세요.** 반대 방향(MySQL 로 배우고 Postgres 로 가기)이 더 어렵습니다. MySQL 에서만 통하는 습관(`||` 를 OR 로 쓰기, 대소문자 무시해 기대기, `WHERE 가동 = true` 로 `TINYINT` 비교하기)이 몸에 배기 때문입니다.

```
const 버전문자열 = await 마이(MY김, "SELECT version() AS v");
const 마리아인가 = 버전문자열.값[0].v.includes("MariaDB");

console.log("이 서버의 버전 문자열:", 버전문자열.값[0].v);
```

```
출력      이 서버의 버전 문자열: 8.4.11
값이 다를 수 있음
```

```
console.log("MariaDB 인가:", 마리아인가);
```

```
출력      MariaDB 인가: false
```

2010년에 오라클이 MySQL 을 가져가자, 원래 만들던 사람들이 소스를 복사해서 따로 만들기 시작한 것이 MariaDB 입니다. 처음에는 MySQL 5.5 와 거의 같아서 “그냥 바꿔 끼우면 된다” 고 했습니다. 지금은 꽤 달라졌습니다. MariaDB 에는 `RETURNING` 이 있고(MySQL 에는 없습니다), 시퀀스도 있고, 스토리지 엔진도 더 많습니다. 반대로 MySQL 8 의 JSON 함수·CTE 최적화·데이터 디크셔너리는 MariaDB 와 구현이 다릅니다. ★ 실무에서 중요한 건 하나입니다. **“MySQL 이라고 들었는데 MariaDB” 인 현장이 많습니다.** 접속하자마자 `SELECT version()` 을 찍어서 확인하세요. 위 한 줄이면 됩니다.

뒷정리

```
await 피지(PG김, "DROP TABLE IF EXISTS 생산실적");
await 마이(MY김, "DROP TABLE IF EXISTS 구구_생산실적");

console.log("실습에 쓴 표를 모두 지웠습니다.");
```

```
출력      실습에 쓴 표를 모두 지웠습니다.
```

정리 — 옮기기 전 점검 목록

□ 1. `SELECT @@sql_mode` 를 찍었는가 (개념02)

```
STRICT_TRANS_TABLES · ONLY_FULL_GROUP_BY 가 있는가
```

□ 2. `||` 를 전부 `concat()` 으로 바꿨는가 (개념03) ★★★

□ 3. 대소문자 비교에 기대는 곳을 찾았는가 (개념03) ★★★

```
아이디·코드는 앱에서 소문자로 맞춰 저장하는 것이 안전합니다
```

□ 4. `LENGTH()` 를 `char_length()` 로 바꿨는가 (개념03)

□ 5. `ORDER BY` 의 NULL 자리를 확인했는가 (개념03)

□ 6. 문자셋이 `utf8mb4` 인가 (개념03)

□ 7. 파라미터를 \$1 에서 ? 로 바꿨는가 (개념04)

같은 번호를 두 번 쓴 곳의 값 개수도 맞췄는가

□ 8. RETURNING 을 쓰는 곳을 전부 고쳤는가 (개념04) ★★

여러 건 넣고 id 를 받는 코드가 있으면 특히 위험합니다

□ 9. 마이그레이션이 실패했을 때 되돌릴 방법이 있는가 (개념04) ★★★

MySQL 은 DDL 이 암묵적 커밋입니다

□ 10. 표 이름을 전부 소문자로 통일했는가 (개념04) ★★

□ 11. 격리수준을 접속할 때 명시했는가 (개념05)

□ 12. 범위를 잡는 쿼리에 색인이 있는가 (개념05)

색인이 없으면 MySQL 은 표 전체를 잠급니다

□ 13. 성능은 우리 데이터로 직접 잰는가 (개념05)

★ 그리고 마지막으로, 가장 확실한 검증 방법을 적어 둡니다. 옳긴 뒤에 같은 쿼리를 양쪽에 보내고 결과를 비교하세요. 이 단원 내내 쓴 나란히() 함수가 바로 그 도구입니다. 에러가 안 나는 차이는 그 방법으로만 잡힙니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 1 의 거짓 경고를 줄여 보세요. 작은따옴표로 감싼 글자를 먼저 지우고 검사하면 됩니다. ★ 힌트: `SQL.replace(/'(?![^'\\]|\\.)*'/g, '')` (이 자료의 출력검증 도구도 같은 방법을 씁니다)

▫ 직접 해보기 2

옳기기점검() 이 몇 번째 줄에서 걸렸는지도 알려 주게 고쳐 보세요. (힌트: SQL 을 `split("\n")` 해서 줄마다 검사하면 됩니다)

▫ 직접 해보기 3

규칙을 세 개 더 추가해 보세요. (힌트: `DISTINCT ON, generate_series, INTERVAL '1 day'`) 각각 MySQL 에서 어떻게 바꿔야 하는지도 적으세요.

▫ 직접 해보기 4

섹션 3 의 갭 락 실험에서 `BETWEEN 10 AND 20` 을 `= 10` 으로 바꿔 보세요. 15번 INSERT 가 막히나요? ★ 안 막힐 것입니다. 왜일까요?

▫ 직접 해보기 5

섹션 3 을 색인 없는 칸으로 해 보세요. 수량 칸(색인 없음)에 WHERE 수량 = 100 FOR UPDATE 를 걸고 전혀 상관없는 줄을 INSERT 해 보세요. 막히나요? ★ 색인이 없으면 MySQL 은 훨씬 넓게 잠급니다.

▫ 직접 해보기 6

MySQL 의 격리수준을 READ COMMITTED 로 바꾸고 섹션 3 의 갭 락 실험을 다시 해 보세요. 여전히 막히나요?

▫ 직접 해보기 7

여러분 회사(또는 연습 프로젝트)의 SQL 을 파일로 읽어서 윗기검() 에 넣어 보세요. 몇 개나 걸리나요?

▫ 직접 해보기 8

섹션 4 의 안내대로 간단한 벤치마크를 직접 짜 보세요. 같은 표에 10만 건을 넣고, 같은 색인을 만들고, 같은 SELECT 를 100번 돌려서 중앙값을 재세요. ★ 그리고 그 숫자를 남에게 이야기할 때는 설정도 같이 말하세요.

자주 하는 실수

이런 실수를 합니다

점검 도구를 돌리고 “경고 0개니까 안전하다” 고 판단한다

★★★ 개념03 의 조용한 차이는 정규식으로 못 잡습니다. 대소문자 비교, NULL 정렬 순서, 자동 형변환은 SQL 만 봐서는 안 보입니다. 결과를 양쪽에서 비교하는 것이 유일한 확인 방법입니다.

이런 실수를 합니다

격리수준 기본값이 같을 거라고 생각한다

PostgreSQL 은 read committed, MySQL 은 REPEATABLE READ 입니다. ★ 접속할 때 명시하세요. 기본값에 기대면 서버가 바뀔 때 동작이 바뀝니다.

이런 실수를 합니다

MySQL 의 갭 락을 모르고 범위 UPDATE 를 남발한다

★★ UPDATE ... WHERE 날짜 BETWEEN ... 하나가 그 뒤 전부를 잠글 수 있습니다. 동시에 넣는 쪽이 1205(잠금 대기 시간 초과)로 죽습니다.

이런 실수를 합니다

색인 없이 범위를 잠근다

MySQL 은 색인이 없으면 훑은 줄을 전부 잠급니다. 사실상 표 잠금입니다. ★ 잠그는 쿼리의 WHERE 절에는 반드시 색인이 있어야 합니다.

이런 실수를 합니다

인터넷 벤치마크를 근거로 DB 를 고른다

★ 설정·하드웨어·워크로드가 다르면 순위가 바뀝니다. 특히 색인을 양쪽에 똑같이 안 만든 비교가 아주 많습니다.

이런 실수를 합니다

“누가 더 좋은가” 로 결론을 낸다

★★ 운영할 사람이 아는 DB 가 제일 좋은 DB 입니다. 새벽 3시에 장애가 났을 때 고칠 수 있느냐가 기능표보다 중요합니다.

이런 실수를 합니다

MariaDB 를 MySQL 과 같다고 본다

지금은 꽤 갈라졌습니다. RETURNING 처럼 MariaDB 에만 있는 것도 있습니다. ★ 접속하면 `SELECT version()` 부터 찍으세요.

이런 실수를 합니다

데이터를 옮기기 전에 제약 충돌을 확인하지 않는다

★★★ 개념03 에서 봤습니다. PostgreSQL 에 있던 'Admin' 과 'admin' 을 MySQL 로 옮기면 UNIQUE 충돌(1062)로 이관이 중간에 멈춥니다. 옮기기 전에 `SELECT lower(아이디), COUNT(*) ... HAVING COUNT(*) > 1` 로 세어 보세요.

```
await PG김.end();
await PGOI.end();
await MY김.end();
await MYOI.end();
```

MySQL 과 무엇이 다른가

RUN node 연습문제.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 13번 문제는 잠금을 기다려서 1~2초쯤 걸립니다.

// TODO: 자리를 채우고 실행하세요. 채점 코드가 진짜 데이터베이스 두 개에 SQL 을 보내서 확인합니다.

☐ 아직 안 풀었습니다 — 답을 안 채웠습니다 ☒ 정답 — 통과했습니다 ☒ 다시 — 답은 채웠는데 통과 못했습니다

답은 하나가 아닌 문제도 있습니다. 채점을 통과하면 맞은 것입니다. 막히면 연습문제_정답.js 를 보세요. 왜 그런지까지 적어 두었습니다.

```
import pg from "pg";
import mysql from "mysql2/promise";
```

준비

```

const 피지설정 = { host: "127.0.0.1", port: 5434, user: "factory", password:
"secret", database: "factory_db" };
const 마이설정 = { host: "127.0.0.1", port: 3307, user: "factory", password:
"secret", database: "factory_db" };

let PG = null;
let MY = null;
let MY2 = null;

try {
  PG = new pg.Client(피지설정);
  await PG.connect();
  MY = await mysql.createConnection(마이설정);
  MY2 = await mysql.createConnection(마이설정);
} catch (에러) {
  console.log("* 데이터베이스에 못 붙었습니다:", 에러.code || 에러.message);
  console.log(" docker compose up -d 를 먼저 실행하세요.");
  if (PG) await PG.end().catch(() => {});
  if (MY) await MY.end().catch(() => {});
  process.exit(0);
}

await PG.query("CREATE SCHEMA IF NOT EXISTS 단원09");
await PG.query("SET search_path TO 단원09");

async function 피지(sql, 값 = []) {
  const 결과 = await PG.query(sql, 값);
  return 결과.rows;
}

async function 마이(sql, 값 = []) {
  const [행들] = await MY.query(sql, 값);
  return 행들;
}

const 안품답 = Symbol("안품");

async function 채점(번호, 제목, 답, 검사) {

```

```
const 비었나 = 답 === 안푼답 || 답 === null || 답 === undefined || (typeof 답 ===
"string" && 답.trim() === "");

if (비었나) {
  console.log(`문제 ${번호} -  아직 안 풀었습니다 · ${제목}`);
  return;
}

let 통과 = false;

try {
  통과 = (await 검사(답)) === true;
} catch (에러) {
  통과 = false;
}

console.log(`문제 ${번호} - ${통과 ? "✅ 정답" : "❌ 다시"} · ${제목}`);
}

console.log("=== 09단원 연습문제 ===");
```

출력 === 09단원 연습문제 ===

문제 1 — MySQL 서버가 엄격한지 검사하기

MySQL 서버를 처음 만나면 `SELECT @@sql_mode` 를 봐야 합니다 (개념02). `sql_mode` 문자열을 받아서, `**STRICT_TRANS_TABLES` 와 `ONLY_FULL_GROUP_BY` 가 둘 다 켜져 있으면 `true` 를 주는 함수를 만드세요.

아래 `안푼답` 을 함수로 바꾸세요.

예) `const 답1 = (모드) => { ... };`


```
const 답1 = 안푼답;

await 채점(1, "엄격한 sql_mode 인지 검사", 답1, (답) => {
  return (
    답("IGNORE_SPACE,ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_DATE") === true
    &&
    답("NO_ENGINE_SUBSTITUTION") === false &&
    답("STRICT_TRANS_TABLES") === false
  );
});
```

출력 문제 1 - ☐ 아직 안 풀었습니다 · 엄격한 sql_mode 인지 검사

문제 2 — || 를 양쪽에서 통하게 고치기

PostgreSQL 에서 `SELECT '설비' || 'A' AS 결과` 는 '설비A' 를 줍니다. MySQL 에 그대로 보내면 0 이 나옵니다 (|| 가 OR 이라서).

양쪽에서 똑같이 '설비A' 가 나오는 SQL 을 쓰세요. 결과 칸 이름은 **결과** 여야 합니다.

SQL 을 문자열로 쓰세요.

```
const 답2 = 안푼답;

await 채점(2, "양쪽에서 글자 잇기", 답2, async (답) => {
  const 가 = await 피지(답);
  const 나 = await 마이(답);
  return 가[0].결과 === "설비A" && 나[0].결과 === "설비A";
});
```

출력 문제 2 - ☐ 아직 안 풀었습니다 · 양쪽에서 글자 잇기

문제 3 — 글자 수를 양쪽에서 똑같이 세기

'설비' 는 글자 3 개입니다. PostgreSQL 의 `length()` 는 3 을 주지만 MySQL 의 `LENGTH()` 는 10 을 줍니다(바이트).

양쪽에서 3 이 나오는 SQL 을 쓰세요. 칸 이름은 **개수** 입니다.

TODO

```
const 답3 = 안푼답;

await 채점(3, "글자 수 세기", 답3, async (답) => {
  const 가 = await 피지(답);
  const 나 = await 마이(답);
  return Number(가[0].개수) === 3 && Number(나[0].개수) === 3;
});
```

출력 문제 3 - ☐ 아직 안 풀었습니다 · 글자 수 세기

문제 4 — NULL 을 맨 뒤로 보내기

아래 자료를 정렬하면 PostgreSQL 은 NULL 을 맨 뒤에, MySQL 은 맨 앞에 둡니다.

SELECT x FROM (SELECT 1 AS x UNION ALL SELECT NULL UNION ALL SELECT 3) AS t

양쪽에서 1, 3, NULL 순서가 나오는 SQL 을 쓰세요. ★ MySQL 에는 NULLS LAST 문법이 없습니다.

TODO

```
const 답4 = 안푼답;

await 채점(4, "NULL 을 맨 뒤로", 답4, async (답) => {
  const 바른순서 = (행들) => 행들.length === 3 && 행들[0].x === 1 && 행들[1].x === 3
  && 행들[2].x === null;
  return 바른순서(await 피지(답)) && 바른순서(await 마이(답));
});
```

출력 문제 4 - ☐ 아직 안 풀었습니다 · NULL 을 맨 뒤로

문제 5 — 0 으로 나누기를 안전하게

SELECT 100 / 0 은 PostgreSQL 에서 에러(22012), MySQL 에서 NULL 입니다.

가동시간이 0 이어도 에러 없이 0 이 나오는 SQL 을 쓰세요. 아래 식을 고치면 됩니다. 칸 이름은 가동률 입니다.

SELECT 100 / 0 AS 가동률

TODO

```
const 답5 = 안푼답;

await 채점(5, "0 으로 나누기 막기", 답5, async (답) => {
  const 가 = await 피지(답);
  const 나 = await 마이(답);
  return Number(가[0].가동률) === 0 && Number(나[0].가동률) === 0;
});
```

출력 문제 5 - 아직 안 풀었습니다 · 0 으로 나누기 막기

문제 6 — PostgreSQL 의 CREATE TABLE 을 MySQL 용으로 바꾸기

아래는 PostgreSQL 용 표 정의입니다.

CREATE TABLE 구구_문제6 (

```
id    SERIAL PRIMARY KEY,
이름  VARCHAR(50) NOT NULL,
가동  BOOLEAN DEFAULT true,
메모  JSONB
```

)

MySQL 에서 그대로 실행되는 CREATE TABLE 문으로 바꿔 쓰세요. 표 이름은 `구구_문제6` 그대로 두세요.

★ 조건: id 는 자동 번호 + 기본키, 이름은 NOT NULL, 메모는 JSON 이어야 합니다.

TODO

```
const 답6 = 안푼답;

await 채점(6, "CREATE TABLE 을 MySQL 용으로", 답6, async (답) => {
  await 마이("DROP TABLE IF EXISTS 구구_문제6");
  await 마이(답);

  const 칸들 = await 마이(
    "SELECT COLUMN_NAME AS 이름, DATA_TYPE AS 타입, IS_NULLABLE AS 널허용, EXTRA AS
    덤 " +
    "FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='factory_db' AND
    TABLE_NAME='구구_문제6'",
  );
  const 찾기 = (이름) => 칸들.find((칸) => 칸.이름 === 이름);

  return (
    칸들.length === 4 &&
    찾기("id").덤 === "auto_increment" &&
    찾기("이름").널허용 === "NO" &&
    찾기("메모").타입 === "json"
  );
});
```

출력 문제 6 - 아직 안 풀었습니다 · CREATE TABLE 을 MySQL 용으로

문제 7 — 파라미터 기호를 바꿔 주는 함수

PostgreSQL 의 \$1, \$2, \$3 을 MySQL 의 ? 로 바꾸는 함수를 만드세요.

"SELECT * FROM 설비 WHERE 라인=\$1 AND 가동=\$2"

→ "SELECT * FROM 설비 WHERE 라인=? AND 가동=?"

함수로 바꾸세요.

```
const 답7 = 안푼답;

await 채점(7, "$1 을 ? 로 바꾸기", 답7, (답) => {
  return (
    답("SELECT * FROM 설비 WHERE 라인=$1 AND 가동=$2") === "SELECT * FROM 설비 WHERE
라인=? AND 가동=?" &&
    답("INSERT INTO 점검(설비,내용) VALUES ($1,$2)") === "INSERT INTO 점검(설비,내용)
VALUES (?,?)" &&
    답("SELECT $10, $2") === "SELECT ?, ?"
  );
});
```

출력 문제 7 - ☐ 아직 안 풀었습니다 · \$1 을 ? 로 바꾸기

문제 8 — RETURNING 없이 방금 넣은 id 받기

PostgreSQL 은 INSERT ... RETURNING id 로 한 번에 받습니다. MySQL 에는 없습니다.

아래 함수를 완성하세요. **MySQL 에 한 건을 넣고 그 id 를 돌려줘야** 합니다. 표는 채점 코드가 미리 만들어
둡니다: 구구_문제8 (id INT AUTO_INCREMENT PRIMARY KEY, 내용 VARCHAR(30))

async (연결, 내용) => 새 id

```
const 답8 = 안푼답;

await 채점(8, "MySQL 에서 새 id 받기", 답8, async (답) => {
  await 마이("DROP TABLE IF EXISTS 구구_문제8");
  await 마이("CREATE TABLE 구구_문제8 (id INT AUTO_INCREMENT PRIMARY KEY, 내용
VARCHAR(30))");
```

운영과 종합 실습

OPEN 10_운영과_종합_실습

```
import pg from "pg";  
import fs from "node:fs";  
import os from "node:os";  
import path from "node:path";
```

01 백업과 복구

02 느린 쿼리 찾기

03 청소와 통계

04 멈추지 않고 스키마 바꾸기

05 종합 실습

- 연습문제

백업과 복구

RUN node 개념01_백업과_복구.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

pg_dump / pg_restore 를 컨테이너 안에서 실제로 돌립니다.

★ 표를 진짜로 지우고 진짜로 되살립니다. 10초쯤 걸립니다.

여기서부터 마지막 단원입니다. 01~09단원이 만드는 법이었다면, 10단원은 망가뜨리지 않고 굴리는 법입니다.

그 첫 번째가 백업입니다. 규칙은 한 문장입니다.

★★★ 백업은 복구를 해 봐야 백업입니다.

복구해 본 적 없는 백업은 백업이 아니라 백업이라는 믿음입니다. 매일 밤 백업이 돌고 있었는데 3년째 빈 파일이었다는 이야기는 흔합니다.

01단원이 이런 빛을 남겼습니다. 섹션 1 에서 눈으로 갇습니다.

이런 실수를 합니다

백업을 파일 복사로 함

복사하는 중에 누가 쓰면 반쯤 쓰인 파일 이 복사됩니다. 그 백업은 복구할 때 깨져 있습니다. 10단원에서 제대로 합니다.

```
import pg from "pg";
import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import { execSync } from "node:child_process";
```

연결

섹션 0

```
const 접속정보 = {
  host: "127.0.0.1",
  port: 5434, // * 5432 가 아닙니다. 이미 깔린 DB 와 안 부딪히게 비켜 났습니다 user:
  "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit10-backup", // * 이름표는 영어로. 한글은 16진수로 깨져
```

```

보입니다
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

```

★★ 07~10단원이 같은 데이터베이스를 씁니다. 단원끼리 표 이름이 부딪히지 않게 **자기 스키마**를 씁니다.

```

await 연결.query("CREATE SCHEMA IF NOT EXISTS 단원10");
await 연결.query("SET search_path TO 단원10");

```

pg_dump 는 컨테이너 **안에** 들어 있습니다. docker exec 로 부릅니다.

```

const 컨테이너 = "db-material-postgres";

function 도커(명령) {
  return execSync(`docker exec ${컨테이너} sh -c ${JSON.stringify(명령)}`, {
    encoding: "utf8",
    timeout: 120000,
    maxBuffer: 32 * 1024 * 1024, // * 백업 파일은 큼니다. 기본 1MB 로는 터집니다
  });
}

try {
  도커("echo ok");
} catch {
  console.log(`docker exec 를 쓸 수 없습니다. 컨테이너 이름이 ${컨테이너} 인지 확인하세요.`);
  console.log(" docker compose up -d");
  await 연결.end();
  process.exit(0);
}

console.log("연결됐습니다.");

```

```

출력      연결됐습니다.

```

01단원의 빛을 갉습니다.

“데이터베이스 파일을 그냥 cp 로 복사해 두면 되지 않나요?” 안 됩니다. 두 가지 이유가 있고, 둘 다 여기서 직접 보여 드립니다.

```
const 실습방 = fs.mkdtempSync(path.join(os.tmpdir(), "db10-백업-"));
```

① 반쯤 쓰인 파일이 복사됩니다

프로그램은 파일을 **한 번에** 쓰지 않습니다. 조각을 여러 번 나눠 씁니다. 데이터베이스 파일은 수기가바이트라 더 잘게 나뉩니다. 그 조각과 조각 **사이에** 백업이 복사를 하면 어떻게 될까요.

```
const 원본파일 = path.join(실습방, "설비.json");
const 쓸내용 = JSON.stringify(
  [
    { 번호: 1, 이름: "프레스1호" },
    { 번호: 2, 이름: "프레스2호" },
    { 번호: 3, 이름: "절삭기1호" },
  ],
  null,
  1,
);

const 파일손잡이 = fs.openSync(원본파일, "w");
const 절반 = Math.floor(쓸내용.length / 2);

fs.writeFileSync(파일손잡이, 쓸내용.slice(0, 절반)); // ① 앞부분을 씁니다
const 중간에복사한것 = fs.readFileSync(원본파일, "utf8"); // * 백업이 하필 지금 복사합니다
fs.writeFileSync(파일손잡이, 쓸내용.slice(절반)); // ② 나머지를 씁니다
fs.closeSync(파일손잡이);

const 다쓴것 = fs.readFileSync(원본파일, "utf8");

console.log(`다 쓴 파일: ${다쓴것.length}글자 / 중간에 복사한 것:
${중간에복사한것.length}글자`);
```

출력 다 쓴 파일: 104글자 / 중간에 복사한 것: 52글자


```
const 읽어보기 = (글) => {
  try {
    JSON.parse(글);
    return "읽힘";
  } catch (에러) {
    return 에러.constructor.name;
  }
};

console.log("다 쓴 것을 읽으면:", 읽어보기(다쓴것));
```

출력 다 쓴 것을 읽으면: 읽힘

```
console.log("중간에 복사한 것을 읽으면:", 읽어보기(중간에복사한것));
```

출력 중간에 복사한 것을 읽으면: SyntaxError

★ 절반만 복사됐습니다. 이 파일은 열리지 않습니다. 그런데 **복사는 성공했습니다**. 에러도 안 났습니다. 크기도 그럴듯합니다. 깨진 걸 아는 순간은 복구가 필요한 그날 새벽 3시입니다.

② 표가 여러 개면 더 나쁩니다

이게 진짜 무서운 쪽입니다.

Postgres 는 표마다 파일이 따로 있습니다. `cp -r` 은 **하나씩 차례대로** 복사합니다. 첫 파일과 마지막 파일 사이에 몇 초가 흐르고, 그 사이에 트랜잭션이 지나갑니다.

```
const 참고파일 = path.join(실습방, "참고.json");
const 라인파일 = path.join(실습방, "라인.json");

fs.writeFileSync(참고파일, JSON.stringify({ 수량: 100 }));
fs.writeFileSync(라인파일, JSON.stringify({ 수량: 0 }));

const 진짜합계 = () => JSON.parse(fs.readFileSync(참고파일, "utf8")).수량 +
  JSON.parse(fs.readFileSync(라인파일, "utf8")).수량;

console.log("웁기기 전 - 참고 100 + 라인 0 = 총", 진짜합계(), "개");
```

출력 웁기기 전 - 참고 100 + 라인 0 = 총 100 개

백업이 참고.json 을 먼저 복사합니다.

```
const 참고사본 = fs.readFileSync(참고파일, "utf8");
```

★ 하필 그 사이에 “참고에서 라인으로 30개 웁기기” 가 일어납니다. 두 파일을 고쳐야 하는 한 건의 작업입니다. (07단원의 그 트랜잭션입니다)

```
fs.writeFileSync(참고파일, JSON.stringify({ 수량: 70 }));
fs.writeFileSync(라인파일, JSON.stringify({ 수량: 30 }));
```

백업이 이어서 라인.json 을 복사합니다.

```
const 라인사본 = fs.readFileSync(라인파일, "utf8");

const 사본합계 = JSON.parse(참고사본).수량 + JSON.parse(라인사본).수량;

console.log("옮긴 뒤 진짜 총 수량:", 진짜합계(), "개 / 백업 사본의 총 수량:",
사본합계, "개");
```

```
출력      옮긴 뒤 진짜 총 수량: 100 개 / 백업 사본의 총 수량: 130 개
```

```
console.log("백업 사본에서 수량이 맞나:", 사본합계 === 100);
```

```
출력      백업 사본에서 수량이 맞나: false
```

```
console.log("없던 30개가 생겼나:", 사본합계 === 130);
```

```
출력      없던 30개가 생겼나: true
```

★★★ 백업 안에 **없던 재고 30개**가 생겼습니다.

참고는 옮기기 **전**(100) 으로 찍혔고, 라인은 옮긴 **후**(30) 로 찍혔습니다. 둘 다 각각은 멀쩡한 파일입니다. 열립니다. 에러도 없습니다. 그런데 합치면 말이 안 됩니다.

이걸 돈으로 바꿔 보세요. 계좌 A 에서 계좌 B 로 이체하는 중에 백업을 뒀습니다. 복구하면 **돈이 복제돼 있습니다**. 그리고 아무도 눈치채지 못합니다.

★★ 그래서 백업 도구가 따로 있는 것입니다. 백업 도구는 **한 시점**을 잡습니다. 07단원에서 배운 그 스냅샷입니다. pg_dump 는 트랜잭션을 하나 열고 그 안에서 전부 읽습니다. 그래서 50개 표를 10분에 걸쳐 읽어도 전부 **같은 순간**의 모습입니다.

pg_dump — 논리 백업 만들기

섹션 2

실습할 데이터를 만듭니다. 05단원에서 쓰던 그 제조 현장 표입니다.

```

await 연결.query("DROP TABLE IF EXISTS 점검기록, 설비, 라인 CASCADE");

await 연결.query(`
    CREATE TABLE 라인 (
        라인코드 TEXT PRIMARY KEY,
        이름      TEXT NOT NULL,
        동        TEXT NOT NULL
    );

    CREATE TABLE 설비 (
        설비번호 INT PRIMARY KEY,
        이름      TEXT NOT NULL,
        라인코드 TEXT REFERENCES 라인(라인코드),
        도입가   NUMERIC(12,2) NOT NULL CHECK (도입가 > 0)
    );

    CREATE TABLE 점검기록 (
        점검번호 INT PRIMARY KEY,
        설비번호 INT REFERENCES 설비(설비번호),
        점검일    DATE NOT NULL,
        결과      TEXT NOT NULL
    );
`);

await 연결.query(`
    INSERT INTO 라인 VALUES ('A','조립1라인','1동'), ('B','가공2라인','1동'),
    ('C','포장3라인','2동');

    INSERT INTO 설비 VALUES
        (1,'컨베이어 1호','A',12000000.00),
        (2,'프레스 1호', 'A',35400000.00),
        (3,'용접로봇 1호','B',88000000.00),
        (4,'포장기 1호', 'C',9500000.00);

    INSERT INTO 점검기록
    SELECT g, (g % 4) + 1, DATE '2026-01-01' + (g % 90), (ARRAY['정상','주의','고장'])
    [(g % 3) + 1]
    FROM generate_series(1, 2000) g;
`);

```

```
`);

const 원래건수 = await 연결.query(` SELECT (SELECT count(*)::int FROM 라인)      AS
라인,
      (SELECT count(*)::int FROM 설비)      AS 설비,
      (SELECT count(*)::int FROM 점검기록) AS 점검기록
`);

console.log("백업 전 건수:", JSON.stringify(원래건수.rows[0]));
```

```
출력      백업 전 건수: {"라인":3,"설비":4,"점검기록":2000}
```

★ pg_dump 는 **SQL 문장으로** 백업합니다. 이걸 논리 백업이라고 합니다. 결과물은 “이 표를 만들고, 이 값들을 넣어라” 라는 지시서입니다.

```
const 백업경로 = "/tmp/단원10백업";
도커(`rm -rf ${백업경로} && mkdir -p ${백업경로}`);
```

★ -t 로 이 파일이 만든 표 세 개만 지정합니다. -n 단원10 만 주면 이 단원의 다른 파일이 만든 표까지 딸려 옵니다.

```
const 표지정 = "-t 단원10.라인 -t 단원10.설비 -t 단원10.점검기록";

도커(
  `PGPASSWORD=secret pg_dump -U factory -d factory_db ${표지정} ` +
  `--format=plain -f ${백업경로}/전체.sql`,
);
```

백업 파일 안에 무엇이 들어 있는지 봅니다. ★ 파일 전체를 읽어 오면 큼니다. 컨테이너 안에서 필요한 줄만 골라 옵니다.

```
const 문장들 = 도커(`grep -E '^(CREATE TABLE|COPY)' ${백업경로}/전체.sql`)
  .split("\n")
  .filter((줄) => 줄.trim());

for (const 줄 of 문장들.slice(0, 6)) console.log(".", 줄.slice(0, 62));
```

```
출력      • CREATE TABLE "단원10"."라인" (
      • CREATE TABLE "단원10"."설비" (
      • CREATE TABLE "단원10"."점검기록" (
      • COPY "단원10"."라인" ("라인코드", "이름", "동") FROM stdin;
      • COPY "단원10"."설비" ("설비번호", "이름", "라인코드", "도입가") FROM
      stdin;
      • COPY "단원10"."점검기록" ("점검번호", "설비번호", "점검일", "결과")
      FROM stdin;
```

```
console.log("백업이 사람이 읽을 수 있는 SQL 인가:", 문장들.some((줄) =>
  줄.startsWith("CREATE TABLE")));
```

```
출력      백업이 사람이 읽을 수 있는 SQL 인가: true
```

★ 값은 INSERT 가 아니라 COPY 로 들어갑니다. COPY 가 훨씬 빠릅니다. (03단원) `--inserts` 를 주면 INSERT 문으로 뽑을 수 있지만 아주 느립니다. 다른 데이터베이스로 옮길 때가 아니면 쓰지 마세요.

형식 세 가지

섹션 3

pg_dump 는 형식이 세 가지입니다. 셋 다 만들어 보고 크기를 잽니다.

```
도커(
  `PGPASSWORD=secret pg_dump -U factory -d factory_db ${표지정} ` +
  `--format=custom -f ${백업경로}/전체.dump`,
);

도커(
  `PGPASSWORD=secret pg_dump -U factory -d factory_db ${표지정} ` +
  `--format=directory -f ${백업경로}/전체디렉터리`,
);

const 크기재기 = (경로) => Number(도커(`du -sk ${경로} | cut -f1`).trim());

const 평문크기 = 크기재기(`${백업경로}/전체.sql`);
const 커스텀크기 = 크기재기(`${백업경로}/전체.dump`);
const 디렉터리크기 = 크기재기(`${백업경로}/전체디렉터리`);

console.log(`plain ${평문크기}KB · custom ${커스텀크기}KB · directory
${디렉터리크기}KB`);
```

```
출력      plain 52KB · custom 12KB · directory 28KB
값이 다를 수 있음
```

```
console.log("custom 이 plain 보다 작은가:", 커스텀크기 < 평문크기);
```

```
출력      custom 이 plain 보다 작은가: true
```

★ 안의 파일 이름은 내부 번호라 뽑을 때마다 바뀝니다. 개수와 목차 파일만 봅니다.

```
const 디렉터리안 = 도커(`ls ${백업경로}/전체디렉터리`).trim().split("\n");

console.log(`directory 안 - 자료파일 ${디렉터리안.filter((이름) =>
  이름.endsWith(".dat.gz")).length}개 + 목차(toc.dat)
${디렉터리안.includes("toc.dat")}`);
```

출력 directory 안 - 자료파일 3개 + 목차(toc.dat) true

형식 확장자 복구 도구 골라 넣기 압축 병렬

plain	.sql	psql	x	x	x
custom	.dump	pg_restore	✓	✓	복구만
directory	폴더	pg_restore	✓	✓	✓ 양쪽

★ 기본으로 **custom** 을 쓰세요. 압축되고, 표 하나만 골라 복구할 수 있고, 목록을 볼 수 있습니다.

★ 표가 수백 개인 큰 DB 는 **directory** 에 -j 4 를 붙이세요. 백업도 병렬로 됩니다.

★ plain 은 psql < 파일 로 밀어 넣는 것 말고는 할 수 있는 게 없습니다. 대신 사람이 읽을 수 있어서 “그때 그 표가 어떻게 생겼더라” 를 확인하기 좋습니다.

custom 형식은 목록을 볼 수 있습니다. 이게 유용합니다.

```
const 목차 = 도커(`pg_restore -l ${백업경로}/전체.dump`)
  .split("\n")
  .filter((줄) => 줄.trim() && !줄.startsWith(";"))
```

★ 앞의 숫자들은 내부 번호(OID)라 돌릴 때마다 바뀝니다. 종류와 이름만 봅니다.

```
.map((줄) => 줄.split(" ").slice(3).join(" "))
.filter((줄) => 줄.startsWith("TABLE") || 줄.startsWith("CONSTRAINT") ||
줄.startsWith("FK"));

for (const 줄 of 목차) console.log(".", 줄);
```

출력

- TABLE 단위10 라인 factory
- TABLE 단위10 설비 factory
- TABLE 단위10 점검기록 factory
- TABLE DATA 단위10 라인 factory
- TABLE DATA 단위10 설비 factory
- TABLE DATA 단위10 점검기록 factory
- CONSTRAINT 단위10 라인 라인_pkey factory
- CONSTRAINT 단위10 설비 설비_pkey factory
- CONSTRAINT 단위10 점검기록 점검기록_pkey factory
- FK CONSTRAINT 단위10 설비 설비_라인코드_fkey factory
- FK CONSTRAINT 단위10 점검기록 점검기록_설비번호_fkey factory

★★★ 진짜로 날리고 진짜로 되살리기

섹션 4

여기가 이 파일에서 제일 중요한 섹션입니다.

백업 파일이 생겼고 크기도 그럴듯합니다. 그런데 이게 진짜 백업인지는 아직 모릅니다. 확인하는 방법은 복구해 보는 것입니다.

```
await 연결.query("DROP TABLE 점검기록, 설비, 라인 CASCADE");
```

정말 없어졌는지 확인합니다.

```
let 지운뒤코드 = null;

try {
  await 연결.query("SELECT count(*) FROM 점검기록");
} catch (에러) {
  지운뒤코드 = 에러.code;
}

console.log("표를 지운 뒤 조회하면 에러코드:", 지운뒤코드, "(42P01 = 없는 표)");
```

```
출력      표를 지운 뒤 조회하면 에러코드: 42P01 (42P01 = 없는 표)
```

★ 이제 정말 없습니다. 2000건이 사라졌습니다. 운영에서 이런 일이 나면 심장이 멎습니다. 백업으로 되살립니다.

```
도커(`PGPASSWORD=secret pg_restore -U factory -d factory_db ${백업경로}/전체.dump`);

const 복구건수 = await 연결.query(`
  SELECT (SELECT count(*)::int FROM 라인)      AS 라인,
         (SELECT count(*)::int FROM 설비)      AS 설비,
         (SELECT count(*)::int FROM 점검기록) AS 점검기록
`);

console.log("복구 후 건수:", JSON.stringify(복구건수.rows[0]));
```

```
출력      복구 후 건수: {"라인":3,"설비":4,"점검기록":2000}
```

```
console.log("건수가 원래와 똑같은가:", JSON.stringify(복구건수.rows[0]) ===
JSON.stringify(원래건수.rows[0]));
```

```
출력      건수가 원래와 똑같은가: true
```

건수만 같으면 안 됩니다. 값도 봐야 합니다.

```
const 복구값 = await 연결.query("SELECT 이름, 도입가 FROM 설비 WHERE 설비번호 = 3");

console.log("복구된 설비 3번:", JSON.stringify(복구값.rows[0]));
```

```
출력      복구된 설비 3번: {"이름":"용접로봇 1호","도입가":"88000000.00"}
```

★ 도입가 가 문자열 "88000000.00" 으로 옵니다. 숫자가 아닙니다. NUMERIC 은 pg 에서 항상 string 입니다. 02단원에서 본 그대로입니다. 돈은 NUMERIC 으로 저장하고 문자열로 받아서 계산도 문자열 기반으로 하세요.

계약도 같이 살아났는지 봅니다. 이걸 안 보고 넘어가는 사람이 많습니다.

```
const 제약수 = await 연결.query(`
  SELECT count(*)::int AS 개수 FROM pg_constraint
  WHERE connamespace = '단원10'::regnamespace AND contype IN ('p','f','c')
`);

console.log("살아난 제약(기본키·외래키·CHECK) 개수:", 제약수.rows[0].개수);
```

출력 살아난 제약(기본키·외래키·CHECK) 개수: 6

외래키가 진짜로 도는지 찔러 봅니다.

```
let 외래키코드 = null;

try {
  await 연결.query("INSERT INTO 설비 VALUES (99, '없는라인설비', 'Z', 1000.00)");
} catch (에러) {
  외래키코드 = 에러.code;
}

console.log("없는 라인으로 넣어 보면:", 외래키코드, "(23503 = 외래키 위반)");
```

출력 없는 라인으로 넣어 보면: 23503 (23503 = 외래키 위반)

★★ 이제 이걸 백업입니다. 복구해 봤으니까요. 건수 · 값 · 제약이 전부 돌아왔습니다.

표 하나만 골라 복구하기

섹션 5

실수로 표 하나만 날렸는데 전체를 되돌리면 다른 표의 최신 데이터가 사라집니다. custom / directory 형식은 골라서 복구할 수 있습니다.

```
await 연결.query("DELETE FROM 점검기록");

console.log("점검기록을 비운 뒤:", (await 연결.query("SELECT count(*)::int AS 수 FROM 점검기록")).rows[0].수, "건");
```

출력 점검기록을 비운 뒤: 0 건

★ --data-only 는 표를 다시 만들지 않고 값만 넣습니다. 표가 이미 있으니까요. 이걸 빼면 "이미 있다"(42P07) 로 실패합니다.


```

도커(
  `PGPASSWORD=secret pg_restore -U factory -d factory_db ` +
  `--data-only --table=점검기록 ${백업경로}/전체.dump`,
);

console.log("점검기록만 복구한 뒤:", (await 연결.query("SELECT count(*)::int AS 수
FROM 점검기록")).rows[0].수, "건");

```

출력 점검기록만 복구한 뒤: 2000 건

```

console.log("다른 표는 그대로인가 (설비 4건):", (await 연결.query("SELECT
count(*)::int AS 수 FROM 설비")).rows[0].수 === 4);

```

출력 다른 표는 그대로인가 (설비 4건): true

★★ `--data-only` 로 값을 넣을 때 **이미 값이 있으면 중복**이 됩니다. 기본키가 있으면 23505 로 실패하고, 없으면 조용히 두 배가 됩니다. 비우고 넣거나, 새 스키마에 복구해서 옮기세요.

논리 백업 vs 물리 백업

섹션 6

지금까지 한 것은 **논리 백업**입니다. SQL 문장으로 뽑았습니다. 다른 방법이 있습니다. **물리 백업** — 데이터 파일을 통째로 가져가는 것입니다.

“그거 섹션 1 에서 안 된다고 하지 않았나요?” 맞습니다. **그냥 cp 로 하면** 안 됩니다. 그래서 전용 도구 `pg_basebackup` 을 씁니다.

```

const 물리도구 = 도커("pg_basebackup --version").trim().split(" ")[0];

console.log("물리 백업 도구가 컨테이너에 있나:", 물리도구 === "pg_basebackup");

```

출력 물리 백업 도구가 컨테이너에 있나: true

★ `pg_basebackup` 이 `cp` 와 다른 점

① 시작할 때 서버에 “지금부터 백업이야” 라고 알립니다 (백업 모드) ② 파일을 복사하는 **동안 일어난 변경**을 WAL 로 같이 받아 갑니다 ③ 그래서 복구할 때 그 WAL 을 다시 적용해 **찢어진 부분을 꿰매**니다

섹션 1 의 “창고 100 + 라인 30 = 130” 문제를 ③ 이 고칩니다. 찢어진 채로 복사되긴 하는데, 꿰매 실을 같이 가져가는 것입니다.

항목 논리 백업 (`pg_dump`) 물리 백업 (`pg_basebackup`)

담는 것 SQL 문장 데이터 파일 통째로

크기	작다 (압축·색인 제외)	크다 (색인까지 전부)
속도	느리다 (읽어서 다시 씀)	빠르다 (그냥 복사)

복구 단위 표 하나도 가능 DB 전체만 버전 이동 ✓ 18 → 19 로 옮길 수 있다 × 같은 버전이어야 한다 다른 기계 ✓ 어디든 × 같은 아키텍처만 시점 복구 × 백업 시점만 ✓ WAL 과 합치면 아무 순간

★★ 언제 무엇을 쓰나 · 100GB 미만 · 하루 한 번이면 충분 → pg_dump (custom) · 수백 GB 이상 · 복구가 몇 분 안이어야 → pg_basebackup + WAL 보관 · 버전을 올리거나 다른 서버로 이사 → pg_dump (이것뿐입니다) · 중요한 서비스 → ★ **둘 다** 하세요

★ WAL 과 시점 복구(PITR)

섹션 7

07단원이 “갑자기 꺼져도 WAL 을 읽어 복구합니다. 10단원에서 다시 봅니다” 라고 미웠던 것입니다.

Postgres 는 표를 고치기 **전에** “이렇게 고칠 거야” 를 먼저 다른 파일에 적습니다. 그게 WAL (Write-Ahead Log, 미리 쓰는 기록)입니다. 순서가 핵심입니다. **적고 나서 고칩니다.** 반대가 아닙니다.

```
const 왈설정 = await 연결.query(`
  SELECT current_setting('wal_level') AS 수준,
         current_setting('archive_mode') AS 보관,
         current_setting('fsync') AS 확실히쓰기
`);

console.log("WAL 설정:", JSON.stringify(왈설정.rows[0]));
```

출력 WAL 설정: {"수준":"replica","보관":"off","확실히쓰기":"on"}

WAL 이 진짜로 쌓이는지 재 봅니다.

```
await 연결.query("CREATE TABLE IF NOT EXISTS 왈시험 (번호 INT, 값 TEXT)");

const 넣기전 = (await 연결.query("SELECT pg_current_wal_lsn() AS 위치")).rows[0].위치;
await 연결.query("INSERT INTO 왈시험 SELECT g, '값' || g FROM generate_series(1, 20000) g");
const 넣은뒤 = (await 연결.query("SELECT pg_current_wal_lsn() AS 위치")).rows[0].위치;

const 늘어난양 = await 연결.query("SELECT pg_wal_lsn_diff($1, $2)::int AS 바이트", [넣은뒤, 넣기전]);

console.log(`2만 건 넣었더니 WAL 이 ${Math.round(늘어난양.rows[0].바이트 / 1024)}KB 늘었습니다`);
```

출력 2만 건 넣었더니 WAL 이 1410KB 늘었습니다
값이 다를 수 있음

```
console.log("WAL 이 늘어났나:", 늘어난양.rows[0].바이트 > 0);
```

```
출력      WAL 이 늘어났나: true
```

```
await 연결.query("DROP TABLE 왓시힘");
```

★★ 이 WAL 을 버리지 않고 모아 두면 놀라운 일이 가능해집니다.

일요일 새벽 2시 pg_basebackup 으로 통째로 백업 그 뒤 계속 WAL 을 전부 다른 곳에 보관
(archive_mode = on)

수요일 오후 3시 10분에 누가 DELETE FROM 주문; 을 WHERE 없이 쳤습니다.

① 일요일 백업을 복구합니다 ② 보관해 둔 WAL 을 오후 3시 09분까지만 다시 적용합니다 ③ 3시 09분의
데이터베이스가 됩니다. 그 DELETE 는 없던 일이 됩니다

이걸 **시점 복구(PITR, Point-In-Time Recovery)** 라고 합니다. “어제 오후 3시로 돌려 주세요” 가 진짜로
되는 이유가 이것입니다.

★ 설정은 이렇게 생겼습니다. (이 자료의 컨테이너는 꺼 났습니다)

postgresql.conf

```
wal_level = replica
archive_mode = on
archive_command = 'test ! -f /보관/%f && cp %p /보관/%f'
```

복구할 때 recovery 설정

```
restore_command = 'cp /보관/%f %p'
recovery_target_time = '2026-08-26 15:09:00'
```

★★ WAL 을 보관하지 않으면 PITR 은 불가능합니다. 나중에 켤 수도 없습니다. 지난주 WAL 은 이미
지워졌으니까요. 켜 두는 것 말고는 방법이 없습니다.

★ 요즘은 손으로 안 합니다. pgBackRest, Barman, WAL-G 같은 도구가 백업 · WAL 보관 · 보관 기간
관리 · 복구 검증까지 해 줍니다. 운영이라면 쓰세요.

백업 점검 목록

섹션 8

백업은 만드는 것보다 **굴리는 것**이 어렵습니다.

☐ 주기 얼마나 짧아도 되나? 하루? 1시간? 그게 백업 주기입니다 ☐ 보관 기간 어제 것만 있으면, 3일 전에
생긴 오류는 못 되돌립니다 ☐ 다른 곳 같은 서버-같은 디스크에 두면 그 디스크가 죽을 때 같이 죽습니다 ☐
자동 확인 백업 파일 크기가 갑자기 0 이 되면 알림이 와야 합니다 ☐ ★ 복구 연습 분기에 한 번, **다른 서버에**

진짜로 복구해서 건수를 세 보세요 □ 시간 재기 복구에 몇 시간 걸리나요? 그게 서비스 정지 시간입니다 □ 암호화 고객 데이터가 든 백업 파일은 그냥 두면 유출 사고 그 자체입니다

★★ 두 가지 숫자를 정하고 시작하세요. 회사가 답을 갖고 있어야 합니다.

RPO (얼마나 잃어도 되나) — 1시간이면 백업을 1시간마다 하거나 WAL 을 보관해야 합니다 RTO (얼마나 멈춰도 되나) — 30분이면 3시간 걸리는 복구 절차는 실패입니다

이 둘을 안 정하고 “백업은 하고 있습니다” 라고 하면 아무것도 안 정한 것입니다.

뒷정리

```
도커(`rm -rf ${백업경로}`);
fs.rmSync(실습방, { recursive: true, force: true });
await 연결.query("DROP TABLE IF EXISTS 점검기록, 설비, 라인 CASCADE");
await 연결.end();

console.log("끝났습니다.");
```

출력 끝났습니다.

정리

도구 무엇을 하나 복구 도구

pg_dump	한 DB 를 SQL 로 뺏는다	psql / pg_restore
pg_dumpall	모든 DB + 역할·권한을 뺏는다	psql
pg_restore	custom/directory 백업을 넣는다	-
pg_basebackup	데이터 파일을 통째로 가져간다	서버가 직접

pg_dump 옵션 하는 일

```
--format=custom      압축되고 골라 복구할 수 있음 (기본으로 이걸 쓰세요)
--format=directory    -j 로 병렬. 표가 아주 많을 때
```

-n 스키마이름 그 스키마만 -t 표이름 그 표만

```
--schema-only      구조만 (값 없이)
--data-only        값만 (구조 없이)
```

pg_restore 옵션 하는 일

-l	안에 뭐가 있는지 목록만 봄
--table=이름	그 표만 복구
--data-only	표는 그대로 두고 값만

-j 4 병렬로 복구 (directory/custom)

에러코드 뜻

42P01	없는 표 (지운 뒤 조회)
42P07	이미 있는 표 (--data-only 를 빼먹었을 때)
23503	외래키 위반 (제약이 살아 있다는 증거)
23505	중복 (--data-only 를 비우지 않고 두 번 넣었을 때)

★★★ 꼭 기억할 것 ① 복구해 본 적 없는 백업은 백업이 아닙니다. 분기에 한 번은 해 보세요 ② cp 로 복사하면 반쯤 쓰인 파일이 복사됩니다. 게다가 표끼리 시점이 어긋납니다 ③ pg_dump 는 한 트랜잭션 안에서 읽어서 **전부 같은 순간**입니다 ④ 기본은 --format=custom. 표 하나만 골라 되살릴 수 있습니다 ⑤ WAL 을 보관해 두면 “어제 오후 3시” 로 돌아갈 수 있습니다 (PITR) ⑥ 백업을 같은 디스크에 두지 마세요. 그 디스크가 죽는 게 백업이 필요한 날입니다

직접 해 볼 것

▫ 직접 해보기 1

섹션 3 의 --format=custom 을 지우고 다시 뽑아 보세요. (기본 형식은 plain 입니다) pg_restore -i 이 되나요? (힌트: plain 은 그냥 텍스트라 목록이라는 개념이 없습니다)

▫ 직접 해보기 2

섹션 4 의 pg_restore 를 두 번 연달아 돌려 보세요. 두 번째에 어떤 에러가 나나요? 왜 그럴까요? (힌트: 표를 또 만들려고 합니다. 42P07)

▫ 직접 해보기 3

섹션 5 에서 --data-only 를 빼고 돌려 보세요. 무슨 에러가 나나요? 반대로 점검기록을 비우지 않고 --data-only 로 넣으면 어떻게 되나요?

▫ 직접 해보기 4

--schema-only 로 뽑아 보세요. 파일 크기가 얼마인가요? “표 구조만 다른 팀에 보내 주세요” 일 때 쓰는 방법입니다.

▫ 직접 해보기 5

섹션 1의 ②에서 창고와 라인을 복사하는 **순서를 바꿔** 보세요. 이번엔 총 수량이 몇 개가 되나요? 30개가 사라지나요?

▫ 직접 해보기 6

섹션 7에서 `generate_series(1, 20000)` 을 200000 으로 바꿔 보세요. WAL 이 10배 늘어나나요? 정확히 비례하나요?

▫ 직접 해보기 7

새 스키마에 복구해 보세요. `pg_restore ... --schema=단원10` 은 안 됩니다. 대신 plain 백업의 "단원10" 을 다른 이름으로 바꿔서 `psql` 로 넣어 보세요. 운영에서 "지우지 않고 옆에 복구해서 비교하기" 가 이 방법입니다.

자주 하는 실수

이런 실수를 합니다

백업만 하고 복구를 안 해 봄

이 파일에서 제일 중요한 실수입니다. 백업 스크립트가 3년째 빈 파일을 만들고 있어도 아무도 모릅니다. ★ 달력에 적어 두고 분기마다 다른 서버에 복구해서 건수를 세 보세요.

이런 실수를 합니다

cp / rsync 로 데이터 폴더를 복사함

섹션 1에서 본 그대로입니다. 반쯤 쓰인 파일이 복사되고, 표끼리 시점이 어긋납니다. ★ 서버를 끄고 복사하면 됩니다. 그런데 그동안 서비스가 멈춥니다. `pg_basebackup` 은 안 끄고도 됩니다. 그래서 그걸 씁니다.

이런 실수를 합니다

백업을 같은 서버에 둬

디스크가 죽는 날이 바로 백업이 필요한 날입니다. 같이 죽습니다. ★ 최소한 다른 기계, 되도록 다른 지역에 두세요.

이런 실수를 합니다

pg_dump 만 하고 역할·권한을 안 뽑음

`pg_dump` 는 그 데이터베이스만 뽑습니다. 계정과 권한은 안 들어갑니다. 복구했는데 아무도 로그인을 못 합니다. ★ `pg_dumpall --globals-only` 를 같이 뽑아 두세요.

이런 실수를 합니다

보관 기간을 안 정함

어제 백업만 갖고 있으면, 지난주에 생긴 데이터 오염은 못 되돌립니다. ★ 일 7개 · 주 4개 · 월 12개 처럼 계단식으로 두는 것이 보통입니다.

이런 실수를 합니다

복구 시간을 안 재 봄

500GB 를 복구하는 데 6시간 걸린다면, “30분 안에 복구” 라는 약속은 거짓말입니다. ★ 한 번 재 보고 그 숫자를 회사에 알려세요. 그게 진짜 RTO 입니다.

이런 실수를 합니다

EXPLAIN 하듯 운영에서 바로 복구해 봄

“복구 연습” 을 운영 DB 에 하면 그게 사고입니다. ★ 반드시 다른 서버 · 다른 데이터베이스에 복구하세요.

이런 실수를 합니다

백업 파일을 암호화하지 않음

백업 파일에는 고객 데이터가 전부 들어 있습니다. 그냥 두면 그 자체가 유출 통로입니다. ★ 저장할 때 암호화하고, 누가 가져갈 수 있는지 권한을 좁히세요.

느린 쿼리 찾기

RUN node 개념02_느린_쿼리_찾기.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 일부러 기다리는 실험이 있어서 15초쯤 걸립니다.

“서비스가 느려요.”

이 말을 들으면 사람들은 대개 이렇게 합니다. 코드를 들여다보다가 “이 쿼리가 좀 이상한데” 하고 고칩니다. 그리고 아무것도 안 빨라집니다.

★★★ 감으로 찾지 마세요. 재서 찾으세요.

눈으로 봐서 느려 보이는 쿼리와 실제로 시간을 잡아먹는 쿼리는 거의 항상 다릅니다. 이 파일에서 그럴 숫자로 보여 드립니다.

06단원에서 EXPLAIN ANALYZE 로 한 쿼리가 왜 느린지 보는 법을 배웠습니다. 그런데 그건 “어느 쿼리를 볼지” 를 이미 알고 있을 때 이야기입니다. 이 파일은 그 앞 단계입니다. 어느 쿼리를 볼지 찾는 법입니다.

① pg_stat_statements — 쌓인 통계로 범인 찾기 ② 느린 쿼리 로그 — 오래 걸린 것만 기록으로 남기기
③ pg_stat_activity — 지금 이 순간 돌고 있는 것 보기 ④ auto_explain — 느린 것의 실행계획을 자동으로 남기기

```
import pg from "pg";
import { execSync } from "node:child_process";
```

연결

섹션 0

```
const 접속정보 = {
  host: "127.0.0.1",
  port: 5434, // * 5432 가 아닙니다 user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit10-slow", // * 영어로. 한글을 넣으면 16진수로 깨져 보입니다
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
```



```

} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

```

★★ 07~10단원이 같은 데이터베이스를 씁니다. 자기 스키마를 씁니다.

```

await 연결.query("CREATE SCHEMA IF NOT EXISTS 단위10");
await 연결.query("SET search_path TO 단위10");

async function 새연결(이름표) {
  const 새것 = new pg.Client({ ...접속정보, application_name: 이름표 });
  await 새것.connect();
  await 새것.query("SET search_path TO 단위10");
  return 새것;
}

const 잠깐 = (밀리초) => new Promise((끝) => setTimeout(끝, 밀리초));

console.log("연결됐습니다.");

```

```

출력      연결됐습니다.

```

실습용 표를 매번 새로 만듭니다.

```

await 연결.query("DROP TABLE IF EXISTS 느린점검 CASCADE");

await 연결.query(`
  CREATE TABLE 느린점검 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT NOT NULL,
    라인      TEXT NOT NULL,
    결과      TEXT NOT NULL,
    점검일    DATE NOT NULL
  )
`);

await 연결.query(`
  INSERT INTO 느린점검
  SELECT g, (g % 500) + 1, (ARRAY['A','B','C'][(g % 3) + 1],
    (ARRAY['정상','주의','고장'][(g % 3) + 1],
    DATE '2026-01-01' + (g % 200)
  FROM generate_series(1, 120000) g
`);

```

```
await 연결.query("ANALYZE 느린점검");

console.log("실습용 표:", (await 연결.query("SELECT count(*)::int AS 수 FROM
느린점검")).rows[0].수, "건");
```

출력 실습용 표: 120000 건

pg_stat_statements — 켜는 법

섹션 1

Postgres 에는 모든 쿼리의 통계를 자동으로 쌓아 주는 확장이 있습니다. 06단원에서 "10단원에서 pg_stat_statements 로 합니다" 라고 미뤄 뒀던 그것입니다.

먼저 확장을 만들어 봅니다.

```
let 만들기결과 = "성공";

try {
  await 연결.query("CREATE EXTENSION IF NOT EXISTS pg_stat_statements");
} catch (에러) {
  만들기결과 = 에러.code;
}

console.log("CREATE EXTENSION 결과:", 만들기결과);
```

출력 CREATE EXTENSION 결과: 성공

만들어졌습니다. 그럼 조회해 봅니다.

```
let 조회에러 = null;

try {
  await 연결.query("SELECT count(*) FROM pg_stat_statements");
} catch (에러) {
  조회에러 = `${에러.code} - ${에러.message}`;
}

console.log("조회하면:", 조회에러);
```

출력 조회하면: 55000 - pg_stat_statements must be loaded via "shared_preload_libraries"

★★★ 이게 이 확장에서 제일 많이 만나는 에러입니다.

CREATE EXTENSION 은 성공합니다. 그런데 읽으면 실패합니다. 확장은 만들어졌는데 통계를 모으는 부분이 안 켜져 있기 때문입니다.

pg_stat_statements 는 모든 쿼리를 가로채야 해서 **서버가 시작할 때** 메모리에 올라와 있어야 합니다. 나중에 켤 수가 없습니다. 그래서 재시작이 필요합니다.

★ 켜는 법 (postgresql.conf)

```
shared_preload_libraries = 'pg_stat_statements'
pg_stat_statements.max = 10000      ← 몇 종류까지 기억할지
pg_stat_statements.track = top      ← top(직접 부른 것만) / all(함수 안까지)
```

그리고 **서버를 재시작합니다**. 그 뒤에 CREATE EXTENSION 을 합니다.

★★ 이 자료의 컨테이너는 다른 단원들이 같이 쓰고 있어서 재시작을 하지 않습니다. 그래서 섹션 2 는 **pg_stat_statements** 와 똑같은 방식으로 세는 사본을 만들어 그 통계를 어떻게 읽는지 연습합니다. 읽는 법은 완전히 같습니다.

★ 여러분의 서버에서는 위 설정을 넣고 재시작한 뒤 진짜로 써 보세요.

```
await 연결.query("DROP EXTENSION IF EXISTS pg_stat_statements");
```

pg_stat_statements 가 켜져 있으면 이런 칸들이 보입니다.

칸 이름 뜻

query	정규화된 쿼리 (\$1, \$2 로 바뀐 모양)
calls	몇 번 불렀나
total_exec_time	★ 전부 합쳐 몇 ms 를 썼나 ← 제일 중요합니다
mean_exec_time	한 번에 평균 몇 ms
max_exec_time	제일 오래 걸린 한 번
rows	돌려준 줄 수의 합
shared_blk_read_time	디스크에서 읽느라 쓴 시간

★★★ 총 시간이 긴 쿼리가 범인입니다

섹션 2

여기가 이 파일에서 제일 중요한 섹션입니다.

pg_stat_statements 와 똑같은 방식으로 세는 작은 사본을 만듭니다. 쿼리를 정규화해서(값을 \$1 로 바꿔서) 같은 모양끼리 묶고, 호출 횟수와 걸린 시간을 더합니다. 진짜 확장이 하는 일이 이것입니다.

```
const 통계 = new Map();
```

★ 정규화 — 값이 다른 같은 모양의 쿼리를 하나로 묶습니다. WHERE 설비번호 = 3 과 WHERE 설비번호 = 7 은 같은 쿼리입니다.

```
function 정규화(문장) {
  return 문장
    .replace(/\s+/g, " ")
```

★ 따옴표로 감싼 값과 숫자를 한 번에 \$1 로 바꿉니다. ★★ 바꿀 글자에 쓰는 \$\$ 는 달러 하나를 뜻합니다. 그냥 "\$1" 로 쓰면

자바스크립트가 **첫 번째 괄호 그룹**으로 알아들어서 엉뚱하게 나옵니다.

★★★ (?<!\\$) 는 "앞에 달러가 없는 숫자만" 이라는 뜻입니다.

이게 없으면 이미 파라미터인 ` \$1 ` 의 1 까지 바뀌어서 ` \$\$1 ` 이 됩니다. 실제로 이 실수를 했고, 출력에 ` \$\$1 ` 이 찍혀서 알았습니다.

```
.replace(/'[^']*'|(?<!\$)\b\d+\b/g, "$$1")
.trim();
}

async function 재면서질의(문장, 값들 = []) {
  const 시작 = performance.now();
  const 결과 = await 연결.query(문장, 값들);
  const 걸린 = performance.now() - 시작;

  const 열쇠 = 정규화(문장);
  const 이전 = 통계.get(열쇠) ?? { 호출: 0, 총시간: 0, 최대: 0 };

  통계.set(열쇠, {
    호출: 이전.호출 + 1,
    총시간: 이전.총시간 + 걸린,
    최대: Math.max(이전.최대, 걸린),
  });

  return 결과;
}
```

두 종류의 쿼리를 섞어서 돌립니다

① 목록 쿼리 — 아주 빠릅니다. 그런데 화면을 열 때마다 부릅니다 (2000번) ② 월간 집계 — 눈에 띄게 느립니다. 그런데 하루에 몇 번 안 부릅니다 (5번)

어느 쪽이 서버 시간을 더 먹고 있을까요?

```
await 연결.query("CREATE INDEX 느린점검_설비 ON 느린점검 (설비번호)");
await 연결.query("ANALYZE 느린점검");
```

```

for (let 회차 = 0; 회차 < 2000; 회차 += 1) {
  await 재면서질의("SELECT 점검번호, 결과 FROM 느린점검 WHERE 설비번호 = $1 LIMIT
10", [(회차 % 500) + 1]);
}

for (let 회차 = 0; 회차 < 5; 회차 += 1) {
  await 재면서질의(`
SELECT 라인, count(*)::int AS 건수
FROM 느린점검 WHERE 결과 <> '정상'
GROUP BY 라인 ORDER BY 건수 DESC
`);
}

const 쌓인것 = [...통계.entries()].map(([문장, 값]) => ({
  문장: 문장.slice(0, 42),
  호출: 값.호출,
  총시간: 값.총시간,
  평균: 값.총시간 / 값.호출,
})));

```

평균으로 줄 세우면

```
console.log("[평균이 긴 순서]");
```

```
출력      [평균이 긴 순서]
```

```

for (const 줄 of [...쌓인것].sort((가, 나) => 나.평균 - 가.평균)) {
  console.log(` ${줄.평균.toFixed(2)} ms/회 · ${줄.호출}회 · ${줄.문장}`);
}

```

```

출력      11.62 ms/회 · 5회 · SELECT 라인, count(*)::int AS 건수 FROM 느린점검
값이 다를 수 있음      W
                     0.57 ms/회 · 2000회 · SELECT 점검번호, 결과 FROM 느린점검 WHERE
                     설비번호 = $1

```

총 시간으로 줄 세우면

```
console.log("[총 시간이 긴 순서]");
```

```
출력      [총 시간이 긴 순서]
```

```

for (const 줄 of [...쌓인것].sort((가, 나) => 나.총시간 - 가.총시간)) {
  console.log(` 총 ${줄.총시간.toFixed(0)} ms · ${줄.호출}회 · ${줄.문장}`);
}

```

출력 총 1131 ms · 2000회 · SELECT 점검번호, 결과 FROM 느린점검 WHERE
값이 다를 수 있음 설비번호 = \$1
 총 58 ms · 5회 · SELECT 라인, count(*)::int AS 건수 FROM 느린점검 W

```
const 평균1등 = [...쌓인것].sort((가, 나) => 나.평균 - 가.평균)[0];
const 총시간1등 = [...쌓인것].sort((가, 나) => 나.총시간 - 가.총시간)[0];

console.log("평균 1등과 총시간 1등이 다른가:", 평균1등.문장 !== 총시간1등.문장);
```

출력 평균 1등과 총시간 1등이 다른가: true

```
console.log("총시간 1등이 1회당으로는 더 빠른가:", 총시간1등.평균 < 평균1등.평균);
```

출력 총시간 1등이 1회당으로는 더 빠른가: true

```
console.log(`총시간 1등은 ${총시간1등.호출}회, 평균 1등은 ${평균1등.호출}회
불렸습니다`);
```

출력 총시간 1등은 2000회, 평균 1등은 5회 불렸습니다

```
console.log(`총시간 1등이 전체의 ${Math.round((총시간1등.총시간 / 쌓인것.reduce((합,
줄) => 합 + 줄.총시간, 0)) * 100)}퍼센트를 씹`);
```

출력 총시간 1등이 전체의 95퍼센트를 씹
값이 다를 수 있음

★★★ 0.2ms 짜리가 범인입니다.

“7ms 걸리는 집계 쿼리” 를 보면 누구나 저게 문제라고 생각합니다. 그런데 그건 5번밖에 안 불립니다. 합쳐서 35ms 입니다.

0.2ms 짜리는 빠릅니다. 눈으로 보면 아무 문제 없습니다. 그런데 2000번 불립니다. 합쳐서 424ms 입니다. **12배**입니다.

운영에서는 이 비율이 훨씬 극단적입니다. 0.1ms 짜리가 하루 100만 번이면 100초입니다. 1초짜리가 하루 10번이면 10초입니다. **10배 차이**입니다.

★★ 그래서 pg_stat_statements 를 볼 때는 **total_exec_time** 으로 정렬합니다. mean 으로 정렬하면 하루에 한 번 도는 배치 쿼리만 잔뜩 나옵니다.

```
SELECT query, calls, round(total_exec_time) AS 총ms,
```

```
round(mean_exec_time::numeric, 2) AS 평균ms
```

```
FROM pg_stat_statements ORDER BY total_exec_time DESC
```

```
LIMIT 20;
```

★ 그리고 고치는 방법도 다릅니다. · 총시간 1등이 **호출이 많아서** 라면 → 캐시하거나, 아예 덜 부르게 만듭니다. · 총시간 1등이 **한 번이 느려서** 라면 → 색인이나 쿼리를 고칩니다 (06단원)

1등이 호출 200만 번짜리면 색인을 아무리 잘 걸어도 소용이 적습니다. **부르는 횟수를 줄이는 게** 답입니다. 08단원의 N+1 문제가 대표적입니다.

느린 쿼리 로그

섹션 3

pg_stat_statements 는 **합계**를 봅니다. 개별 사건은 안 남습니다. “어제 오후 3시에 뭐가 느렸나” 를 보려면 로그가 필요합니다.

log_min_duration_statement 를 켜면 그 시간을 넘긴 쿼리만 로그에 남습니다.

```
console.log("기본값:", (await 연결.query("SHOW
log_min_duration_statement")).rows[0].log_min_duration_statement, "(-1 = 안 남김)");
```

출력 기본값: -1 (-1 = 안 남김)

★ 이 설정은 세션에서도 바꿀 수 있습니다. 실제로 켜고 느린 쿼리를 하나 만듭니다.

```
await 연결.query("SET log_min_duration_statement = '200ms'");

const 표식 = "SLOWQUERY_MARK";
```

★ 아래 두 줄은 SQL 에 글자를 끼워 넣습니다. 보통은 하면 안 되는 일입니다(03단원). 여기서는 **로그에서 이 줄을 찾으려고** 표식을 문장 안에 박는 것이고, **표식** 은 바로 위에서 우리가 정한 상수입니다. 사용자 입력이 아닙니다. ★★ 사용자에게서 온 값은 어떤 경우에도 이렇게 넣으면 안 됩니다. **\$1** 을 쓰세요.

```
await 연결.query(`SELECT '${표식}' AS 표식, pg_sleep(0.35)`);
await 연결.query(`SELECT '${표식}' AS 표식, 1 AS 빠른것`);

await 잠깐(1200); // 로그가 흘러나올 시간을 줍니다
```

이 컨테이너는 로그를 표준출력으로 내보냅니다. docker logs 로 읽습니다. ★ **--since 3s** 로 **최근 3초만** 봅니다. **--tail** 로 보면 지난번 실행 때 남은 줄까지 같이 잡혀서 개수가 안 맞습니다.

```
const 로그 = execSync("docker logs --since 3s db-material-postgres 2>&1", {
  encoding: "utf8",
  maxBuffer: 32 * 1024 * 1024,
});

const 남은줄 = 로그.split("\n").filter((줄) => 줄.includes(표식) &&
줄.includes("duration:"));

console.log("로그에 남은 줄 수:", 남은줄.length);
```

출력 로그에 남은 줄 수: 1

```
console.log("남은 것이 pg_sleep 쪽인가:", 남은줄[0].includes("pg_sleep"));
```

출력 남은 것이 pg_sleep 쪽인가: true

```
const 걸린시간 = Number(남은줄[0].match(/duration: ([\d.]+) ms/)[1]);
```

```
console.log(`로그가 적어 둔 시간: ${걸린시간.toFixed(0)} ms`);
```

출력 로그가 적어 둔 시간: 358 ms

값이 다를 수 있음

```
console.log("200ms 를 넘겼나:", 걸린시간 > 200);
```

출력 200ms 를 넘겼나: true

```
await 연결.query("SET log_min_duration_statement = -1");
```

★★ 값을 얼마로 잡을지가 중요합니다.

0 모든 쿼리를 남깁니다. ★ 절대 운영에서 쓰지 마세요.

	로그 쓰느라 서버가 더 느려지고 디스크가 하루 만에 찹니다
200ms	보통 여기서 시작합니다. 사람이 느끼기 시작하는 지점입니다
1s	로그가 너무 많이 나올 때 올립니다

★ 짝이 되는 설정

log_min_duration_sample / log_statement_sample_rate	- 일부만 샘플로 남기기
log_lock_waits = on	- 잠금을 오래 기다린 것 (07단원)
log_temp_files = 0	- 임시 파일을 쓴 쿼리 (메모리 부족 신호)
log_autovacuum_min_duration = 0	- 오래 걸린 청소 (개념03)

지금 돌고 있는 것 보기

섹션 4

위의 둘은 **지나간 일**입니다. "지금 서비스가 멈춰 있어요" 일 때는 지금 뭐가 도는지 봐야 합니다.

pg_stat_activity 입니다.

```
const 느린놈 = await 새연결("unit10-victim");
```

★ await 를 안 겁니다. 띄워만 두고 아래로 내려갑니다. (07단원에서 쓴 방법입니다)


```
const 오래도는쿼리 = 느린놈
  .query("SELECT pg_sleep(30), count(*) FROM 느린점검")
  .then(() => "혼자 끝남")
  .catch((에러) => 에러.code);

await 잠깐(600);
```

★ 이 DB 는 다른 단원도 같이 씁니다. 내 이름표가 붙은 것만 봅니다.

```
const 들고있는것 = await 연결.query(`
  SELECT state,
         left(query, 24)           AS 문장,
         (now() - query_start) > interval '0.3 s' AS 오래됨,
         wait_event_type           AS 무엇을기다리나
  FROM pg_stat_activity
  WHERE application_name = 'unit10-victim'
`);

console.log("지금 들고 있는 것:", JSON.stringify(들고있는것.rows[0]));
```

```
출력      지금 들고 있는 것: {"state":"active","문장":"SELECT pg_sleep(30),
                        cou","오래됨":true,"무엇을기다리나":"Timeout"}
```

★ 실무에서 쓰는 질의는 이렇게 생겼습니다. **그대로 쓰세요.**

```
SELECT pid, now() - query_start AS 걸린시간, state, wait_event_type, left(query, 80) FROM
pg_stat_activity WHERE state <> 'idle' AND pid <> pg_backend_pid()
```

```
ORDER BY query_start;
```

★★ state 를 꼭 보세요. 셋의 뜻이 완전히 다릅니다.

```
active      지금 진짜 일하는 중
idle        아무것도 안 함. 정상입니다 (연결 풀이 잡고 있는 것)
```

idle in transaction ★★ BEGIN 만 해 놓고 노는 중 — **제일 위험합니다**

```
잠금을 쥐고, 청소를 막습니다 (개념03 에서 봅니다)
```

죽이기

섹션 5

찾았으면 죽여야 할 때가 있습니다. 두 가지가 있고 **많이 다릅니다.**

```
const 잡힌pid = (
  await 연결.query("SELECT pid FROM pg_stat_activity WHERE application_name =
'unit10-victim'")
).rows[0].pid;
```

① pg_cancel_backend — **쿼리만** 취소합니다. 연결은 살아 있습니다.

```
const 취소했나 = await 연결.query("SELECT pg_cancel_backend($1) AS 결과", [잡힌pid]);

console.log("pg_cancel_backend 가 참을 돌려줬나:", 취소했나.rows[0].결과);
```

```
출력      pg_cancel_backend 가 참을 돌려줬나: true
```

```
console.log("취소된 쿼리의 에러코드:", await 오래도는쿼리, "(57014 = 취소됨)");
```

```
출력      취소된 쿼리의 에러코드: 57014 (57014 = 취소됨)
```

연결이 아직 살아 있는지 확인합니다.

```
console.log("취소 뒤에도 그 연결을 쓸 수 있나:", (await 느린놈.query("SELECT 1 AS
값")).rows[0].값 === 1);
```

```
출력      취소 뒤에도 그 연결을 쓸 수 있나: true
```

② pg_terminate_backend — **연결을 끊습니다**. 훨씬 셉니다.

★★ 서버가 연결을 끊으면 pg 드라이버가 'error' 이벤트를 냅니다. 이것 안 받아 두면 **노드 프로세스가 통째로 죽습니다**. 반드시 달아 두세요.

```
느린놈.on("error", () => {});

const 두번째쿼리 = 느린놈
  .query("SELECT pg_sleep(30)")
  .then(() => "혼자 끝남")
  .catch((에러) => 에러.code ?? "연결끊김");

await 잠깐(500);
await 연결.query("SELECT pg_terminate_backend($1)", [잡힌pid]);

console.log("terminate 뒤 에러코드:", await 두번째쿼리, "(57P01 = 관리자가 끊음)");
```

```
출력      terminate 뒤 에러코드: 57P01 (57P01 = 관리자가 끊음)
```

```
await 느린놈.end().catch(() => {}); // 이미 서버가 끊었을 수 있습니다
```

★★★ 어느 것을 쓰나

pg_cancel_backend 먼저 이걸 쓰세요. 쿼리만 멈추고 연결은 삽니다.

트랜잭션은 열린 채로 남습니다 (그래서 안 풀릴 수도)

pg_terminate_backend 안 죽을 때만. 연결을 끊고 트랜잭션도 롤백됩니다.

★ 애플리케이션 쪽에서 에러가 납니다. 각오하고 쓰세요

★ 사람이 손으로 죽이지 않아도 되게 타임아웃을 걸어 두는 게 낫습니다. (07단원)

```
statement_timeout = '30s'
idle_in_transaction_session_timeout = '60s'
lock_timeout = '3s'
```

찾았으면 원인을 봅니다 — 06단원으로 이어집니다

섹션 6

범인을 찾았습니다. 이제 왜 느린지 봅니다. 여기서부터 06단원입니다.

```
async function 계획(sql) {
  const 결과 = await 연결.query(`EXPLAIN (ANALYZE) ${sql}`);
  return 결과.rows.map((줄) => 줄["QUERY PLAN"]).join("\n");
}
```

색인이 없는 칸으로 찾으면 어떻게 되는지 봅니다.

```
const 색인없는계획 = await 계획("SELECT count(*) FROM 느린점검 WHERE 라인 = 'A'");

console.log("라인으로 찾을 때 순차 훑기가 나오나:", 색인없는계획.includes("Seq Scan"));
```

출력 라인으로 찾을 때 순차 훑기가 나오나: true

```
const 색인있는계획 = await 계획("SELECT count(*) FROM 느린점검 WHERE 설비번호 = 42");

console.log("설비번호로 찾을 때 색인을 쓰나:", 색인있는계획.includes("Index"));
```

출력 설비번호로 찾을 때 색인을 쓰나: true

```
const 실행시간 = (계획글) => Number(계획글.match(/Execution Time: ([\d.]+) ms/)[1]);

console.log(`색인 없이 ${실행시간(색인없는계획).toFixed(1)} ms / 색인으로 ${실행시간(색인있는계획).toFixed(2)} ms`);
```

출력 색인 없이 12.3 ms / 색인으로 0.26 ms
값이 다를 수 있음

```
console.log("색인 쪽이 더 빠른가:", 실행시간(색인있는계획) < 실행시간(색인없는계획));
```

```
출력      색인 쪽이 더 빠른가: true
```

★★★ 06단원에서 배운 그 경고를 다시 합니다. **EXPLAIN ANALYZE** 는 **진짜로 실행합니다**. UPDATE 나 DELETE 에 붙이면 진짜로 고치고 진짜로 지웁니다. 운영에서 이걸로 사고가 납니다. 반드시 이렇게 하세요.

```
BEGIN;  
EXPLAIN (ANALYZE) DELETE ...;  
ROLLBACK;
```

```
const 지우기전 = (await 연결.query("SELECT count(*)::int AS 수 FROM  
느린점검")).rows[0].수;  
  
await 연결.query("BEGIN");  
await 연결.query("EXPLAIN (ANALYZE) DELETE FROM 느린점검 WHERE 라인 = 'A'");  
const 롤백전 = (await 연결.query("SELECT count(*)::int AS 수 FROM  
느린점검")).rows[0].수;  
await 연결.query("ROLLBACK");  
  
const 롤백후 = (await 연결.query("SELECT count(*)::int AS 수 FROM  
느린점검")).rows[0].수;  
  
console.log(`계획만 봤는데 ${지우기전 - 롤백전}건이 진짜로 지워졌습니다`);
```

```
출력      계획만 봤는데 40000건이 진짜로 지워졌습니다
```

```
console.log("ROLLBACK 으로 되살아났나:", 롤백후 === 지우기전);
```

```
출력      ROLLBACK 으로 되살아났나: true
```

auto_explain — 느린 것의 계획을 자동으로 남기기

섹션 7

문제는 이겁니다. 새벽 3시에 느렸던 쿼리를 낮에 다시 돌리면 빠릅니다. 그때는 데이터가 달랐고, 캐시가 달랐고, 통계가 달랐습니다.

`auto_explain` 은 **느릴 때 그 자리에서** 실행계획을 로그에 남겨 줍니다.

```
await 연결.query("LOAD 'auto_explain'"); // * 세션에서 켜 볼 수 있습니다  
await 연결.query("SET auto_explain.log_min_duration = '5ms'");  
await 연결.query("SET auto_explain.log_analyze = true");  
  
const 표식2 = "AUTOEXPLAIN_MARK";
```

★ 색인이 없는 칸으로 12만 건을 훑고 정렬까지 시킵니다. 진짜로 느립니다. ★★ `pg_sleep` 을 WHERE 에 넣지 마세요. **줄마다** 실행됩니다.

4만 줄이면 4만 번 잡니다. 영원히 안 끝납니다.

```
await 연결.query(`
  SELECT '${표식2}' AS 표식, 결과, count(*)::int AS 건수
  FROM 느린점검 WHERE 라인 <> 'Z'
  GROUP BY 결과 ORDER BY 건수 DESC
`);

await 잠깐(1200);

const 로그2 = execSync("docker logs --since 3s db-material-postgres 2>&1", {
  encoding: "utf8",
  maxBuffer: 32 * 1024 * 1024,
});

console.log("auto_explain 이 계획을 남겼나:", 로그2.includes("Seq Scan on") &&
로그2.includes(표식2));
```

출력 auto_explain 이 계획을 남겼나: true

```
await 연결.query("SET auto_explain.log_min_duration = -1");
```

★ 운영에서 켜는 법 (postgresql.conf) — 재시작 없이 되는 방법도 있습니다

session_preload_libraries = 'auto_explain'	← 새 연결부터 적용 (재시작 불필요)
auto_explain.log_min_duration = '500ms'	
auto_explain.log_analyze = on	← ★ 실제 시간이 찍힙니다
auto_explain.log_buffers = on	
auto_explain.log_nested_statements = on	← 함수 안의 쿼리까지

★★ `log_analyze = on` 은 모든 쿼리에 계속 비용이 불립니다. 느린 것만 로그에 남지만, 재는 것은 전부 잡니다. 처음엔 끄고 쓰다가, 필요할 때만 켜세요.

뒷정리

```
await 연결.query("DROP TABLE IF EXISTS 느린점검 CASCADE");
await 연결.end();

console.log("끝났습니다.");
```

출력 끝났습니다.

정리

도구 무엇을 보나 언제 쓰나

pg_stat_statements 쿼리 종류별 누적 통계 “뭘 고쳐야 하지?” 느린 쿼리 로그 느렸던 개별 사건 “어제 3시에 뭐가 느렸지?”

pg_stat_activity	지금 들고 있는 것	"지금 멈췄어요!"
auto_explain	느린 것의 실행계획	"왜 그때만 느리지?"

EXPLAIN ANALYZE 한 쿼리를 파고들기 (06단원) “이건 왜 느리지?”

설정 기본값 추천

log_min_duration_statement	-1	200ms 부터 시작
log_lock_waits	off	on (07단원의 잠금 대기)
log_temp_files	-1 0	(메모리 부족 신호)
log_autovacuum_min_duration	10min 0	(개념03)
auto_explain.log_min_duration	-1	500ms (필요할 때만)

에러코드 뜻

55000	확장이 shared_preload_libraries 에 안 올라와 있음
57014	쿼리가 취소됨 (pg_cancel_backend 또는 statement_timeout)
57P01	관리자가 연결을 끊음 (pg_terminate_backend)

★★★ 꼭 기억할 것 ① 감으로 찾지 마세요. 재서 찾으세요. 눈에 띄는 쿼리는 대개 범인이 아닙니다 ② pg_stat_statements 는 **total_exec_time** 으로 정렬합니다. mean 이 아닙니다 ③ 0.2ms × 2000번이 7ms × 5번보다 12배 나뉩니다. 실제로 재 봤습니다 ④ pg_stat_statements 는 재시작이 필요합니다. 미리 켜 두세요 ⑤ pg_cancel_backend 를 먼저, 안 되면 pg_terminate_backend ⑥ idle in transaction 을 보면 바로 쫓아가세요. 제일 위험한 상태입니다

직접 해 볼 것

⇒ 직접 해보기

1

섹션 2 의 2000회를 200회로 줄여 보세요. 이제 총시간 1등이 바뀌나요? 몇 회부터 뒤집히나요?

▹ 직접 해보기 2

섹션 2의 **정규화** 함수에서 숫자 치환 줄을 지워 보세요. 통계가 몇 종류로 늘어나나요? 왜 정규화가 필요한지 보입니다.

▹ 직접 해보기 3

섹션 3의 **200ms**를 **0**으로 바꿔 보세요. 로그에 몇 줄이 남나요? 운영에서 왜 위험한지 알 수 있습니다.

▹ 직접 해보기 4

섹션 4에서 **BEGIN**만 하고 쿼리를 안 던지는 연결을 만들어 보세요. **state**가 뭐라고 나오나요? (힌트: idle in transaction)

▹ 직접 해보기 5

섹션 5에서 **pg_cancel_backend**를 **pg_terminate_backend**로 바꿔 보세요. 그 뒤 **느린놈.query("SELECT 1")**이 되나요?

▹ 직접 해보기 6

섹션 6의 **느린점검_설비** 색인을 지우고 다시 돌려 보세요. 설비번호 쿼리의 계획이 어떻게 바뀌나요?

▹ 직접 해보기 7

여러분의 서버에 **shared_preload_libraries**를 설정하고 재시작한 뒤 진짜 **pg_stat_statements**를 켜 보세요. **total_exec_time**순으로 정렬하면 1등이 무엇인가요?

자주 하는 실수

이런 실수를 합니다

눈으로 코드를 훑어 느린 쿼리를 찾음

이 파일의 주제입니다. 눈에 띄는 쿼리와 시간을 먹는 쿼리는 다릅니다. ★ 반드시 재세요.
pg_stat_statements 하나면 몇 분 안에 끝납니다.

이런 실수를 합니다

mean_exec_time으로 정렬함

하루에 한 번 도는 배치 쿼리만 1등으로 올라옵니다. 고쳐도 아무도 안 빨라집니다. ★ **total_exec_time**입니다. 예외 없습니다.

이런 실수를 합니다

`log_min_duration_statement = 0` 을 운영에 켜 둡

모든 쿼리가 로그에 남습니다. 로그 쓰느라 더 느려지고 디스크가 찹니다. ★ 200ms 부터 시작해서 로그가 너무 많으면 올리세요.

이런 실수를 합니다

`pg_stat_statements` 를 사고 난 뒤에 켜려고 함

재시작이 필요합니다. 장애 중에 재시작을 하면 그게 두 번째 장애입니다. ★ 서버를 만들 때 미리 켜 두세요. 부담은 거의 없습니다.

이런 실수를 합니다

`pg_terminate_backend` 를 먼저 쏜

연결이 끊기면서 애플리케이션 쪽에 에러가 터집니다. ★ `pg_cancel_backend` 를 먼저 쓰세요. 대개 그걸로 됩니다.

이런 실수를 합니다

통계를 한 번도 초기화하지 않음

서버를 켜 뒤로 계속 쌓여서, 지난달 배치가 아직도 1등에 있습니다. ★ 고친 뒤에는 `SELECT pg_stat_statements_reset();` 하고 다시 재세요.

이런 실수를 합니다

느린 쿼리를 찾고 바로 색인부터 만들

호출이 200만 번이면 색인으로는 별로 안 줄어듭니다. ★ 먼저 물어보세요. "이걸 이렇게 많이 부를 필요가 있나?" 그 다음이 색인입니다. (08단원의 N+1)

이런 실수를 합니다

`pg_stat_activity` 에서 남의 세션까지 죽임

이 DB 는 여러 단원·여러 서비스가 같이 씁니다. ★ `application_name` 으로 이름표를 달고 그것만 거르세요.

청소와 통계

RUN node 개념03_청소와_통계.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ VACUUM 을 여러 번 돌려서 20초쯤 걸립니다.

03단원에서 이런 걸 했습니다.

10만 건을 한 문장으로 고쳤더니 표 크기가 7.2 MB → 14.3 MB. **정확히 두 배**가 됐습니다.

그리고 이렇게 설명했습니다. "Postgres 는 줄을 고칠 때 그 자리를 덮어쓰지 않습니다. 새 줄을 뒤에 쓰고, 옛 줄에 '죽었다' 고 표시합니다."

07단원에서는 이렇게 미뤘습니다. "누군가 치워야 합니다. 그게 VACUUM 입니다. 그리고 **긴 트랜잭션이 열려 있으면 치우지 못합니다.**" "VACUUM 과 부풀림을 제대로 다루는 것은 10단원에서 합니다."

그 빛을 여기서 전부 갉습니다. 재면서 갉습니다.

① 왜 죽은 줄이 생기는가 (MVCC) ② 얼마나 부푸는가 — 직접 재기 ③ VACUUM 이 하는 일 — ★ 파일을 안 줄입니다. 그런데도 효과가 있습니다 ④ VACUUM FULL 이 하는 일 — 줄입니다. 대신 서비스가 멈춥니다 ⑤ ★★ 긴 트랜잭션이 청소를 막는 것 — 눈으로 보여 드립니다 ⑥ autovacuum 과 ANALYZE

```
import pg from "pg";
```

연결

섹션 0

```
const 접속정보 = {
  host: "127.0.0.1",
  port: 5434, // * 5432 가 아닙니다 user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit10-vacuum", // * 영어로
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
}
```

멈추지 않고 스키마 바꾸기

RUN node 개념04_멈추지_않고_스키마_바꾸기.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 표를 다시 쓰는 변경을 실제로 재서 20초쯤 걸립니다.

08단원에서 마이그레이션을 만들었습니다. 번호 붙은 .sql 파일, 적용 기록 표, 한 파일 = 한 트랜잭션. 그리고 이런 경고를 남겼습니다.

시각 서버1 서버2 데이터베이스

10:00	옛 코드	옛 코드	옛 스키마
10:01	옛 코드	옛 코드	★ 새 스키마 (마이그레이션 적용)
10:02	새 코드	옛 코드	새 스키마 ← 여기가 위험합니다
10:03	새 코드	새 코드	새 스키마

07단원에서는 이렇게 미뤘습니다. "그래서 운영 중 스키마 변경은 lock_timeout 을 짧게 걸고, 안 잡히면 포기하고, CREATE INDEX CONCURRENTLY 처럼 안 막는 방법을 씁니다. 10단원에서 다시 다룹니다."

이 파일이 그 답입니다. 두 가지를 다룹니다.

① 얼마나 오래 잠그나 — 변경 종류별로 직접 잡니다 ② 어떻게 안 멈추고 바꾸나 — 확장 → 이전 → 축소를 실제 코드로

★★★ 배포 중에는 옛 코드와 새 코드가 같이 둡니다. 이 한 문장이 이 파일 전부입니다. ALTER TABLE ... RENAME COLUMN 한 줄이면 되는 일을 왜 세 번에 나눠서 며칠에 걸쳐 하는지, 그 이유가 이것입니다.

```
import pg from "pg";
```

연결

섹션 0

```
const 접속정보 = {
  host: "127.0.0.1",
  port: 5434, // ★ 5432 가 아닙니다 user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit10-ddl", // ★ 영어로
```

```
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}
```

★★ 07~10단원이 같은 데이터베이스를 씁니다. 자기 스키마를 씁니다.

```
await 연결.query("CREATE SCHEMA IF NOT EXISTS 단원10");
await 연결.query("SET search_path TO 단원10");

async function 새연결(이름표) {
  const 새것 = new pg.Client({ ...접속정보, application_name: 이름표 });
  await 새것.connect();
  await 새것.query("SET search_path TO 단원10");
  return 새것;
}

const 잠깐 = (밀리초) => new Promise((끝) => setTimeout(끝, 밀리초));

console.log("연결됐습니다.");
```

출력 연결됐습니다.

★★ 어떤 변경이 표를 잠그나 — 직접 재기

섹션 1

08단원에서 표로만 보여 줬던 것을 여기서는 **잡니다**. 20만 건짜리 표를 만들고, 변경마다 몇 ms 걸리는지 봅니다.

★ 걸린 시간 = 그 표가 잠겨 있던 시간입니다. ALTER TABLE 은 거의 다 ACCESS EXCLUSIVE 를 잡습니다. (07단원) 그동안 그 표는 읽지도 쓰지도 못합니다.

```
async function 시험표만들기() {
  await 연결.query("DROP TABLE IF EXISTS 변경시험 CASCADE");

  await 연결.query(`
```

```

번호 INT PRIMARY KEY,
    이름 VARCHAR(20) NOT NULL,
    수량 INT NOT NULL
)
`);

await 연결.query("INSERT INTO 변경시험 SELECT g, '설비' || g, g % 100 FROM
generate_series(1, 200000) g");
await 연결.query("VACUUM ANALYZE 변경시험");
}

await 시험표만들기();

async function 재기(sql) {
    const 시작 = performance.now();
    await 연결.query(sql);
    return performance.now() - 시작;
}

```

★★★ 시간만 재면 안 됩니다. 기계가 바쁘면 몇 배씩 흔들립니다. 표를 다시 썼는지는 흔들리지 않습니다. 그걸 같이 봅니다.

relfilenode 는 그 표가 담긴 **디스크 파일의 번호**입니다. 표를 통째로 다시 쓰면 새 파일에 쓰고 옛 파일을 버리므로 **번호가 바뀝니다**. 카탈로그만 고치는 변경은 번호가 그대로입니다. ★ 이게 “이 ALTER 가 비싼가”를 재는 가장 정확한 방법입니다.

```

const 파일번호 = async () =>
(
    await 연결.query("SELECT relfilenode::int AS 번호 FROM pg_class WHERE oid =
'단원10.변경시험'::regclass")
).rows[0].번호;

async function 재고다시썼나보기(sql) {
    const 전 = await 파일번호();
    const 걸린 = await 재기(sql);
    const 후 = await 파일번호();

    return { 걸린, 다시썼나: 전 !== 후 };
}

const 칸추가 = await 재고다시썼나보기("ALTER TABLE 변경시험 ADD COLUMN 비고 TEXT");
const 기본값칸추가 = await 재고다시썼나보기("ALTER TABLE 변경시험 ADD COLUMN 등급
TEXT DEFAULT 'B'");
const 낫널추가 = await 재고다시썼나보기("ALTER TABLE 변경시험 ALTER COLUMN 등급 SET
NOT NULL");
const 타입넓히기 = await 재고다시썼나보기("ALTER TABLE 변경시험 ALTER COLUMN 이름

```

```
TYPE VARCHAR(50)");
const 타입좁히기 = await 재고다시썼나보기("ALTER TABLE 변경시험 ALTER COLUMN 이름
TYPE VARCHAR(30)");
const 타입바꾸기 = await 재고다시썼나보기("ALTER TABLE 변경시험 ALTER COLUMN 수량
TYPE TEXT");
```

★ 색인 두 가지는 차이가 작아서 한 번만 재면 뒤집힙니다. 제작 지침대로 **세 번 재고 중앙값**을 씁니다. 실제로 한 번만 썼다가 뒤집혔습니다.

```
async function 중앙값으로재기(sql) {
  const 잔것 = [];

  for (let 회차 = 0; 회차 < 3; 회차 += 1) {
    잔것.push(await 재기(sql));
    await 연결.query("DROP INDEX 변경_수량");
  }

  return 잔것.sort((가, 나) => 가 - 나)[1];
}

const 색인만들기 = await 중앙값으로재기("CREATE INDEX 변경_수량 ON 변경시험 (수량)");
const 동시색인 = await 중앙값으로재기("CREATE INDEX CONCURRENTLY 변경_수량 ON
변경시험 (수량)");

await 연결.query("CREATE INDEX 변경_수량 ON 변경시험 (수량)"); // 표를 다시 만들
때까지 자리를 맞춰 둡니다
await 연결.query("DROP INDEX 변경_수량");

const 잔것 = [
  ["칸 추가 (기본값 없음)", 칸추가.걸린],
  ["칸 추가 (기본값 있음)", 기본값칸추가.걸린],
  ["NOT NULL 걸기", 낫널추가.걸린],
  ["VARCHAR(20) → VARCHAR(50) 넓히기", 타입넓히기.걸린],
  ["VARCHAR(50) → VARCHAR(30) 좁히기", 타입좁히기.걸린],
  ["INT → TEXT 바꾸기", 타입바꾸기.걸린],
  ["색인 만들기 (보통)", 색인만들기],
  ["색인 만들기 CONCURRENTLY", 동시색인],
];

for (const [이름, 걸린] of 잔것) console.log(` ${걸린.toFixed(1)} ms - ${이름}`);
```

출력	2.5 ms - 칸 추가 (기본값 없음)
값이 다를 수 있음	1.9 ms - 칸 추가 (기본값 있음)
	10.1 ms - NOT NULL 걸기
	5.6 ms - VARCHAR(20) → VARCHAR(50) 넓히기
	177.6 ms - VARCHAR(50) → VARCHAR(30) 좁히기
	174.5 ms - INT → TEXT 바꾸기
	106.0 ms - 색인 만들기 (보통)
	142.2 ms - 색인 만들기 CONCURRENTLY

판정은 시간이 아니라 표를 다시 썼는지로 합니다

★ 시간은 기계와 부하에 따라 흔들립니다. 파일 번호가 바뀌었는지는 안 흔들립니다.

```
const 다시쓴것 = [
  ["칸 추가 (기본값 없음)", 칸추가],
  ["칸 추가 (기본값 있음)", 기본값칸추가],
  ["NOT NULL 걸기", 낮널추가],
  ["VARCHAR 넓히기", 타입넓히기],
  ["VARCHAR 좁히기", 타입좁히기],
  ["INT → TEXT", 타입바꾸기],
];

for (const [이름, 결과] of 다시쓴것) {
  console.log(` ${결과.다시썼나 ? "표를 다시 씀" : "카탈로그만 고침"} - ${이름}`);
}
```

출력	카탈로그만 고침 - 칸 추가 (기본값 없음)
	카탈로그만 고침 - 칸 추가 (기본값 있음)
	카탈로그만 고침 - NOT NULL 걸기
	카탈로그만 고침 - VARCHAR 넓히기
	표를 다시 씀 - VARCHAR 좁히기
	표를 다시 씀 - INT → TEXT

```
console.log("칸 추가는 표를 안 다시 쓰나:", 칸추가.다시썼나 === false &&
기본값칸추가.다시썼나 === false);
```

출력	칸 추가는 표를 안 다시 쓰나: true
----	------------------------

```
console.log("넓히기는 안 다시 쓰고, 좁히기는 다시 쓰나:", 타입넓히기.다시썼나 ===
false && 타입좁히기.다시썼나 === true);
```

출력	넓히기는 안 다시 쓰고, 좁히기는 다시 쓰나: true
----	--------------------------------

```
console.log("타입 바꾸기는 표를 다시 쓰나:", 타입바꾸기.다시썼나);
```

출력	타입 바꾸기는 표를 다시 쓰나: true
----	------------------------

```
console.log(`참고 - 타입 바꾸기가 칸 추가보다 ${Math.round(타입바꾸기.걸린 / 칸추가.걸린)}배 걸렸습니다`);
```

출력 참고 - 타입 바꾸기가 칸 추가보다 69배 걸렸습니다
값이 다를 수 있음

★★★ 이 숫자들이 말하는 것

칸 추가는 공짜입니다. 몇 ms 입니다. 20만 건이든 2억 건이든 같습니다. Postgres 11 부터는 기본값이 있어도 그렇습니다. 카탈로그만 고치고 끝냅니다.

타입을 바꾸면 표를 통째로 다시 씁니다. 수십~수백 배 걸립니다. ★ 그리고 이건 **줄 수에 비례합니다.** 20만 건에 0.2초면 2억 건에는 200초입니다.

칸 추가는 아무리 커도 그대로인데, 타입 바꾸기는 표가 클수록 정직하게 늘어납니다. 그동안 그 표는 읽지도 쓰지도 못합니다.

★ 넓히기(VARCHAR(20) → VARCHAR(50))는 빠릅니다. 값이 그대로 들어가니까요.

좁히기(50 → 30)는 느립니다. **한 줄씩 확인해야** 하니까요.
"안 넘치는지 봐야 한다" 가 그대로 시간이 됩니다.

★★ 그러니 규칙은 이렇습니다.

- **넓히는 방향은** 대체로 안전합니다
- **좁히거나 바꾸는 방향은** 표를 다시 씁니다. 큰 표면 못 합니다

★★★ 잠금 큐 — 진짜 사고는 여기서 납니다

섹션 2

위에서 "1.8초 멈춘다" 고 했습니다. 그 정도면 참을 만하다고 생각할 수 있습니다. 그런데 진짜 사고는 다른 데서 납니다.

ALTER TABLE 이 잠금을 못 잡고 기다리는 동안, 그 뒤에 줄 선 모든 쿼리가 같이 막힙니다. 읽기까지 포함해서요.

```
await 시험표만들기();

const 오래도는쿼리 = await 새연결("unit10-queue");
const 바꾸는사람 = await 새연결("unit10-queue");
const 읽는사람 = await 새연결("unit10-queue");
```

① 누군가 그 표를 읽는 트랜잭션을 열어 둡니다. 아주 흔한 일입니다.

```
await 오래도논쿼리.query("BEGIN");
await 오래도논쿼리.query("SELECT count(*) FROM 변경시험");

const 시작시각 = performance.now();
```

② ALTER 를 겁니다. ①이 안 끝나서 잠금을 못 잡습니다. 기다립니다. ★ await 를 안 겁니다. 띄워만 둡니다. (07단원에서 쓴 방법)

```
const 알터대기 = 바꾸는사람
  .query("ALTER TABLE 변경시험 ADD COLUMN 새칸 TEXT")
  .then(() => performance.now() - 시작시각);

await 잠깐(400);
```

③ ★★★ 그냥 평범한 SELECT 하나가 뒤에 옵니다. SELECT 는 SELECT 를 절대 안 막습니다. ①과는 아무 문제가 없습니다. 그런데 ②가 앞에 줄 서 있어서 같이 막힙니다.

```
const 읽기대기 = 읽는사람
  .query("SELECT count(*) FROM 변경시험")
  .then(() => performance.now() - 시작시각);

await 잠깐(400);

const 출선킷 = await 연결.query(`
  SELECT left(query, 26)                AS 문장,
         wait_event_type                AS 무엇을기다리나,
         cardinality(pg_blocking_pids(pid)) > 0 AS 막혀있나
  FROM pg_stat_activity
  WHERE application_name = 'unit10-queue' AND state <> 'idle'
  ORDER BY query_start
`);

for (const 줄 of 출선킷.rows) {
  console.log(` ${줄.막혀있나 ? "막힘" : "안막힘"} · ${줄.무엇을기다리나 ?? "없음"}
· ${줄.문장}`);
}
```

```
출력      안막힘 · Client · SELECT count(*) FROM 변경시험
          막힘 · Lock · ALTER TABLE 변경시험 ADD COLUM
          막힘 · Lock · SELECT count(*) FROM 변경시험
```

①이 놓아 줍니다.


```
await 오래도는쿼리.query("COMMIT");

const 알터가기다린 = await 알터대기;
const 읽기가기다린 = await 읽기대기;

console.log(`ALTER 가 기다린 시간: ${알터가기다린.toFixed(0)} ms`);
```

출력 ALTER 가 기다린 시간: 814 ms
값이 다를 수 있음

```
console.log(`뒤에 섰 SELECT 가 기다린 시간: ${읽기가기다린.toFixed(0)} ms`);
```

출력 뒤에 섰 SELECT 가 기다린 시간: 821 ms
값이 다를 수 있음

```
console.log("죄 없는 SELECT 도 같이 막혔나:", 읽기가기다린 > 300);
```

출력 죄 없는 SELECT 도 같이 막혔나: true

```
await 오래도는쿼리.end();
await 바꾸는사람.end();
await 읽는사람.end();
```

★★★ 이게 운영 장애의 전형적인 모양입니다.

- ① 누가 그 표를 읽는 긴 트랜잭션을 열어 뒀습니다 (보고서 쿼리, 백업, 사람이 연 psql) ② 배포 스크립트가 ALTER TABLE 을 겁니다. 잠금을 못 잡고 기다립니다 ③ ★ 그 뒤로 오는 모든 요청이 줄을 섭니다. 읽기도요 ④ 몇 초 만에 커넥션 풀이 다 찹니다 (08단원) ⑤ 서비스 전체가 죽습니다

ALTER 자체는 2ms 짜리였습니다. 문제는 기다리는 동안입니다.

★★ 왜 뒤의 SELECT 가 막히나 Postgres 의 잠금은 먼저 온 순서를 지킵니다. ALTER 가 ACCESS EXCLUSIVE 를 기다리는 중이면, 그 뒤에 온 요청은 ALTER 와 부딪히는지부터 봅니다. 그래서 같이 섭니다. 새치기를 허용하면 ALTER 가 영원히 못 들어가니까요.

lock_timeout — 못 잡으면 포기하게

섹션 3

답은 간단합니다. 오래 기다리지 않게 하면 됩니다.

```
await 시험표만들기();

const 불잡는사람 = await 새연결("unit10-lt");

await 불잡는사람.query("BEGIN");
await 불잡는사람.query("SELECT count(*) FROM 변경시험");
```

```

await 연결.query("SET lock_timeout = '150ms'");

const 포기시작 = performance.now();
let 포기결과 = "잠갔음";

try {
  await 연결.query("ALTER TABLE 변경시험 ADD COLUMN 못들어감 TEXT");
} catch (에러) {
  포기결과 = 에러.code;
}

const 포기까지 = performance.now() - 포기시작;

console.log(`lock_timeout: ${포기결과} - ${포기까지.toFixed(0)} ms 만에 포기`);

```

```

출력          lock_timeout: 55P03 - 156 ms 만에 포기
값이 다를 수 있음

```

```

console.log("150ms 안팎에 포기했나:", 포기까지 >= 140 && 포기까지 < 3000);

```

```

출력          150ms 안팎에 포기했나: true

```

```

console.log("에러코드가 55P03 인가:", 포기결과 === "55P03");

```

```

출력          에러코드가 55P03 인가: true

```

```

await 연결.query("SET lock_timeout = 0");
await 불잡는사람.query("ROLLBACK");
await 불잡는사람.end();

```

★★ 운영 마이그레이션의 표준 서식입니다. 외워 두세요.

```

SET lock_timeout = '3s';
ALTER TABLE 설비 ADD COLUMN 담당자 TEXT;

```

못 잡으면 3초 만에 실패합니다. 실패는 괜찮습니다. **다시 하면 됩니다.** 기다리는 것이 나쁩니다. 뒤에 줄이 서니까요.

★ 재시도까지 붙이면 이렇게 됩니다.

```

for (let 시도 = 0; 시도 < 10; 시도 += 1) {

```

```
try {
  await 연결.query("SET lock_timeout = '3s'");
  await 연결.query("ALTER TABLE ...");
  break;
} catch (에러) {
  if (에러.code !== "55P03") throw 에러;
  await 잠깐(5000); // 조금 쉬었다 다시
}
```

```
}
```

★★★ 그리고 `statement_timeout` 과 헛갈리지 마세요. (07단원)

<code>lock_timeout</code>	잠금을 **기다리는** 시간만 제한
<code>statement_timeout</code>	문장 **전체** 시간을 제한

→ ALTER 에 `statement_timeout` 을 걸면, 표를 다시 쓰는 중에 잘려서

그 시간이 통째로 날아갑니다. `lock_timeout` 을 쓰세요.

CREATE INDEX CONCURRENTLY

섹션 4

색인 만들기는 그 자체로 오래 걸립니다. 큰 표면 몇십 분입니다. 보통 `CREATE INDEX` 는 그동안 **쓰기를 막습니다**. `CONCURRENTLY` 를 붙이면 안 막습니다.

```
await 시험표만들기();
```

색인을 만드는 **동안** 쓰기가 되는지 확인해 봅니다.

```
const 쓰는사람 = await 새연결("unit10-cic");

const 색인시작 = performance.now();

const 색인만드는중 = 연결
  .query("CREATE INDEX CONCURRENTLY 변경_이름 ON 변경시험 (이름)")
  .then(() => "색인 완성");

await 잠깐(30);

let 쓰기됐나 = "뒀";

try {
  await 쓰는사람.query("SET lock_timeout = '2s'");
  await 쓰는사람.query("INSERT INTO 변경시험 VALUES (900001, '색인만드는중에넣음',
```

```
7)");
} catch (에러) {
    쓰기됐나 = 에러.code;
}

console.log("CONCURRENTLY 로 색인 만드는 중에 INSERT:", 쓰기됐나);
```

출력 CONCURRENTLY 로 색인 만드는 중에 INSERT: 됨

```
console.log(await 색인만드는중, `${(performance.now() - 색인시작).toFixed(0)} ms`);
```

출력 색인 완성 (198 ms)
값이 다를 수 있음

```
await 쓰는사람.end();
```

왜 하나는 막고 하나는 안 막나 — 잠금 종류가 다릅니다

위의 실험은 “색인을 만드는 동안” 을 노려야 해서 타이밍을 맞춰야 합니다. 표가 작으면 색인이 순식간에 끝나서 실험 자체가 어긋납니다.

★ 그래서 색인이 잡는 것과 같은 잠금을 직접 걸어 봅니다. 타이밍이 필요 없습니다.

CREATE INDEX → SHARE ★ 쓰기를 막습니다 CREATE INDEX CONCURRENTLY → SHARE
UPDATE EXCLUSIVE 안 막습니다

07단원의 잠금 표를 다시 보세요. SHARE 는 INSERT 가 잡는 ROW EXCLUSIVE 와 부딪히고, SHARE
UPDATE EXCLUSIVE 는 안 부딪힙니다.

```
async function 이잠금중에쓰기가되나(잠금종류, 번호) {
    const 쓰려는이 = await 새연결("unit10-lockmode");

    await 연결.query("BEGIN");
    await 연결.query(`LOCK TABLE 변경시험 IN ${잠금종류} MODE`);

    await 쓰려는이.query("SET lock_timeout = '400ms'");

    let 됐나 = true;

    try {
        await 쓰려는이.query("INSERT INTO 변경시험 VALUES ($1, '잠금시험', 1)", [번호]);
    } catch {
        됐나 = false; // 55P03 - 잠금을 못 잡았습니다
    }

    await 연결.query("ROLLBACK");
    await 쓰려는이.end();
}
```

종합 실습

RUN node 개념05_종합_실습.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 시스템을 통째로 만들고 점검까지 해서 20초쯤 걸립니다.

이 자료의 마지막 개념 파일입니다.

지금까지 아홉 단원을 지나왔습니다. 각 단원은 한 가지씩 다뤘습니다. 여기서는 그걸 **전부 이어서 한 시스템**을 만듭니다.

설계 04단원 — 무엇을 표로 만들 것인가 표 만들기 02단원 — 타입과 제약

데이터	03단원 — 넣기, 대량 처리
조회	05단원 — 여러 표 잇기, 묶어 세기
색인	06단원 — 실행계획을 보고 거는가
트랜잭션	07단원 — 업무 한 건을 통째로

앱 연결 08단원 — 풀과 저장소 계층 운영 10단원 — 백업 · 느린 쿼리 · 청소 · 무중단 변경

그리고 마지막에 **점검 도구**를 만듭니다. 이 자료에서 배운 것을 데이터베이스에서 **자동으로 찾아 주는** 함수입니다. 여러분 회사 DB 에 그대로 돌려 보셔도 됩니다.

```
import pg from "pg";
```

연결

섹션 0

```
const 접속정보 = {
  host: "127.0.0.1",
  port: 5434, // * 5432 가 아닙니다 user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit10-final", // * 영어로
};

let 준비;

try {
  준비 = new pg.Client(접속정보);
  await 준비.connect();
```

```

} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

```

★ 이 파일은 시스템을 통째로 만듭니다. 전용 스키마를 새로 파고 시작합니다. 여러 번 돌려도 같은 결과가 나오게 매번 새로 만듭니다. (08단원의 그 방식)

```

const 스키마 = "단원10_종합";

await 준비.query(`DROP SCHEMA IF EXISTS ${스키마} CASCADE`);
await 준비.query(`CREATE SCHEMA ${스키마}`);
await 준비.query(`SET search_path TO ${스키마}`);

console.log("연결됐습니다.");

```

출력 연결됐습니다.

설계 (04단원)

섹션 1

요구사항입니다.

"공장에 라인이 여러 개 있습니다. 라인마다 설비가 있고, 작업자가 배치됩니다. 설비는 정기적으로 점검하고 결과를 남깁니다. 라인별로 하루 생산량을 봅니다."

04단원에서 배운 대로 명사를 뽑고, 관계를 봅니다.

명사 → 표가 될 후보

라인	→ 표. 설비와 작업자가 여기에 속합니다
설비	→ 표. 라인에 속합니다 (여러 설비 : 한 라인)
작업자	→ 표. 라인에 속합니다
점검기록	→ 표. 설비에 속하고 작업자가 합니다
생산실적	→ 표. 라인의 하루치

★ 04단원의 규칙 — **같은 사실을 두 군데 적지 않습니다.** 점검기록에 **라인이름** 을 또 적으면 안 됩니다. 설비를 통해 알 수 있으니까요. 라인 이름이 바뀌면 두 군데를 고쳐야 합니다.

★ 그리고 04단원이 남긴 반대쪽 조연도 있습니다. **가끔은 일부러 중복합니다.** 매번 이어 붙이는 게 너무 비싸면요. 그때는 "왜 중복했는지" 를 주석으로 남기고, 어긋나지 않게 지키는 방법도 정합니다.

표 만들기 (02단원)

섹션 2

02단원에서 배운 것을 씁니다. 타입을 고르고, 제약으로 규칙을 박습니다.

```
await 준비.query(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY CHECK (라인코드 ~ '^[A-Z]$'),
    이름      TEXT NOT NULL UNIQUE,
    동        TEXT NOT NULL
  );

  CREATE TABLE 작업자 (
    사번      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드),
    입사일    DATE NOT NULL DEFAULT CURRENT_DATE
  );

  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL UNIQUE,
    라인코드 TEXT NOT NULL REFERENCES 라인(라인코드),
    도입가    NUMERIC(12,2) NOT NULL CHECK (도입가 > 0),
    상태      TEXT NOT NULL DEFAULT '정지'
              CHECK (상태 IN ('가동','정지','점검','고장'))
  );

  CREATE TABLE 점검기록 (
    점검번호 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    설비번호 INT NOT NULL REFERENCES 설비(설비번호),
    담당사번 INT REFERENCES 작업자(사번),
    점검일    DATE NOT NULL,
    결과      TEXT NOT NULL CHECK (결과 IN ('정상','주의','고장')),
    비고      TEXT
  );

  CREATE TABLE 생산실적 (
    라인코드 TEXT NOT NULL REFERENCES 라인(라인코드),
    생산일    DATE NOT NULL,
    수량      INT NOT NULL CHECK (수량 >= 0),
```

```

    불량      INT NOT NULL DEFAULT 0 CHECK (불량 >= 0),
    CONSTRAINT 불량은수량이하 CHECK (불량 <= 수량),
    PRIMARY KEY (라인코드, 생산일)
);
`);

const 만든표 = await 준비.query(` SELECT relname::text AS 표 FROM pg_class WHERE
relnamespace = '${스키마}':regnamespace AND relkind = 'r' ORDER BY relname
`);

console.log("만든 표:", 만든표.rows.map((줄) => 줄.표).join(", "));

```

출력 만든 표: 라인, 생산실적, 설비, 작업자, 점검기록

★ 02단원에서 고른 것들 · 돈(도입가)은 **NUMERIC**. FLOAT 는 절대 안 됩니다. 반올림이 어긋납니다 · 상태 같은 정해진 값은 **CHECK**. 오타가 들어갈 자리를 없앱니다 · 점검번호는 **IDENTITY**. SERIAL 보다 표준이고 실수로 넣는 것을 막습니다 · 생산실적은 **복합 기본키** (라인코드, 생산일). 하루에 한 줄만 있으면 되니까요 · 불량 <= 수량 처럼 **두 칸 사이의 규칙**도 CHECK 로 박습니다

데이터 넣기 (03단원)

섹션 3

```

await 준비.query(`
  INSERT INTO 라인 VALUES
    ('A','조립1라인','1동'), ('B','가공2라인','1동'),
    ('C','포장3라인','2동'), ('D','신설4라인','2동');

  INSERT INTO 작업자 (사번, 이름, 소속라인, 입사일) VALUES
    (101,'김반장','A',DATE '2018-03-02'), (102,'이기사','B',DATE '2020-07-15'),
    (103,'박주임','B',DATE '2021-01-04'), (104,'최사원','C',DATE '2024-09-01'),
    (105,'정신입',NULL,DATE '2026-02-01');

  INSERT INTO 설비 VALUES
    (1,'컨베이어 1호','A',12000000.00,'가동'),
    (2,'프레스 1호','A',35400000.00,'가동'),
    (3,'용접로봇 1호','B',88000000.00,'점검'),
    (4,'검사기 1호','B',15750000.00,'가동'),
    (5,'포장기 1호','C',9500000.00,'정지');
`);

```

★ 03단원: 대량은 한 줄씩 INSERT 하지 않습니다. generate_series 로 한 문장에 넣습니다.

```

await 준비.query(`
  INSERT INTO 점검기록 (설비번호, 담당사번, 점검일, 결과, 비고)
  SELECT (g % 5) + 1,
         101 + (g % 4),

```



```
(ARRAY['정상', '정상', '정상', '주의', '고장'])(g % 5) + 1],
    '점검 ' || g
FROM generate_series(1, 60000) g;
```

```
INSERT INTO 생산실적 (라인코드, 생산일, 수량, 불량)
SELECT 라인.라인코드,
    DATE '2026-01-01' + 일자,
    800 + ((일자 * 7) % 400),
    ((일자 * 3) % 25)
FROM 라인, generate_series(0, 179) AS 일자
WHERE 라인.라인코드 <> 'D';
`);
```

```
const 건수 = await 준비.query(`
    SELECT (SELECT count(*)::int FROM 라인)      AS 라인,
           (SELECT count(*)::int FROM 설비)      AS 설비,
           (SELECT count(*)::int FROM 작업자)    AS 작업자,
           (SELECT count(*)::int FROM 점검기록) AS 점검기록,
           (SELECT count(*)::int FROM 생산실적) AS 생산실적
`);

console.log("넣은 건수:", JSON.stringify(건수.rows[0]));
```

출력 넣은 건수: {"라인":4,"설비":5,"작업자":5,"점검기록":60000,"생산실적":540}

★ count(*)::int 를 쓴 이유 — pg 에서 count(*) 는 문자열로 옵니다. ::int 를 안 붙이면 “60000” 이 와서 덧셈이 이어붙이기가 됩니다. (02단원)

제약이 진짜로 도는지 찢러 봅니다. 02단원에서 한 그대로입니다.

```
const 찢러보기 = async (설명, sql) => {
    try {
        await 준비.query(sql);
        return `${설명}: 들어감`;
    } catch (에러) {
        return `${설명}: ${에러.code}`;
    }
};

console.log(await 찢러보기("없는 라인의 설비 ", "INSERT INTO 설비 VALUES
(99,'유령','Z',100.00,'가동')"));
```

출력 없는 라인의 설비 : 23503

```
console.log(await 찢러보기("이상한 상태 ", "INSERT INTO 설비 VALUES
(98,'이상','A',100.00,'폭발')"));
```

출력 이상한 상태 : 23514

```
console.log(await 찰러보기("불량이 수량보다 큼", "INSERT INTO 생산실적 VALUES ('A',DATE '2027-01-01',10,99)"));
```

출력 불량이 수량보다 큼: 23514

```
console.log(await 찰러보기("IDENTITY 에 직접 ", "INSERT INTO 점검기록 (점검번호, 설비번호, 점검일, 결과) VALUES (1,1,CURRENT_DATE, '정상')"));
```

출력 IDENTITY 에 직접 : 428C9

조회 (05단원)

섹션 4

05단원에서 배운 것으로 실제 보고서를 만듭니다.

```
const 라인별현황 = await 준비.query(`
  SELECT 라인.이름                                AS 라인,
         count(설비.설비번호)::int                AS 설비수,
         COALESCE(sum(설비.도입가), 0)::text        AS 설비총액
  FROM 라인
  LEFT JOIN 설비 ON 설비.라인코드 = 라인.라인코드
  GROUP BY 라인.라인코드, 라인.이름
  ORDER BY 라인.라인코드
`);

for (const 줄 of 라인별현황.rows) console.log(` ${줄.라인} - 설비 ${줄.설비수}대 ·
${줄.설비총액}원`);
```

출력 조립1라인 - 설비 2대 · 47400000.00원
 가공2라인 - 설비 2대 · 103750000.00원
 포장3라인 - 설비 1대 · 9500000.00원
 신설4라인 - 설비 0대 · 0원

★★ LEFT JOIN 이라서 설비가 없는 신설4라인도 나옵니다. 그냥 JOIN 이면 사라집니다. 05단원에서 본 그 차이입니다. 그리고 count(설비.설비번호) 이지 count(*) 가 아닙니다. count(*) 로 하면 설비가 없는 라인이 1로 나옵니다.

윈도우 함수 — 라인별로 불량률이 가장 높았던 날 (05단원)

```
const 최악의날 = await 준비.query(`
  SELECT 라인, 생산일::text AS 날짜, 불량률
  FROM (
    SELECT 라인.이름 AS 라인, 생산실적.생산일,
           round(생산실적.불량 * 100.0 / 생산실적.수량, 2) AS 불량률,
```

```
ORDER BY 생산실적.불량 * 1.0 / 생산실적.수량 DESC,
                생산실적.생산일) AS 순위
FROM 생산실적
JOIN 라인 ON 라인.라인코드 = 생산실적.라인코드
) 매긴것
WHERE 순위 = 1
ORDER BY 라인
`);

for (const 줄 of 최악의날.rows) console.log(` ${줄.라인} - ${줄.날짜} 불량률 ${줄.
불량률}퍼센트`);
```

```
출력      가공2라인 - 2026-02-28 불량률 2.98퍼센트
          조립1라인 - 2026-02-28 불량률 2.98퍼센트
          포장3라인 - 2026-02-28 불량률 2.98퍼센트
```

★ `round(...)` 결과가 NUMERIC 이라 문자열로 옵니다. 02단원의 그 표입니다. 불량률 을 숫자로 계산에 쓰려면 `Number()` 로 감싸야 합니다.

색인 (06단원)

섹션 5

아직 색인이 하나도 없습니다. 기본키가 만들어 준 것 말고는요. 06단원에서 배운 대로 실행계획을 보고 겁니다. 감으로 걸지 않습니다.

```
await 준비.query("ANALYZE");

const 계획 = async (sql) =>
  (await 준비.query(`EXPLAIN (ANALYZE) ${sql}`)).rows.map((줄) => 줄["QUERY
PLAN"]).join("\n");

const 자주쓰는질의 = `
SELECT 결과, count(*)::int FROM 점검기록
WHERE 설비번호 = 3 AND 점검일 >= DATE '2026-03-01'
GROUP BY 결과
`;

const 색인전 = await 계획(자주쓰는질의);

console.log("색인 전 - 순차 훑기인가:", 색인전.includes("Seq Scan"));
```

```
출력      색인 전 - 순차 훑기인가: true
```

★ 06단원의 복합 색인 순서 규칙: **같다(=)** 로 쓰는 칸을 먼저.

```
await 준비.query("CREATE INDEX 점검_설비_점검일 ON 점검기록 (설비번호, 점검일)");
await 준비.query("ANALYZE 점검기록");

const 색인후 = await 계획(자주쓰는질의);

console.log("색인 후 - 색인을 쓰나:", 색인후.includes("Index"));
```

출력 색인 후 - 색인을 쓰나: true

```
const 실행시간 = (계획글) => Number(계획글.match(/Execution Time: ([\d.]+) ms/)[1]);

console.log(`색인 전 ${실행시간(색인전).toFixed(1)} ms → 색인 후 ${실행시간(색인후).toFixed(2)} ms`);
```

출력 색인 전 3.5 ms → 색인 후 1.95 ms
값이 다를 수 있음

```
console.log("색인 쪽이 더 빠른가:", 실행시간(색인후) < 실행시간(색인전));
```

출력 색인 쪽이 더 빠른가: true

트랜잭션 (07단원)

섹션 6

업무 한 건을 통째로 묶습니다. SQL 한 줄이 아니라 업무가 단위입니다.

“설비가 고장 났습니다” 는 두 가지 일입니다. ① 점검기록에 ‘고장’ 을 남긴다 ② 설비의 상태를 ‘고장’ 으로 바꾼다 둘 중 하나만 되면 데이터가 어긋납니다.

```
const 풀 = new pg.Pool({ ...접속정보, max: 5, options: { -c search_path=${스키마} } });
```

★★ 08단원에서 배운 그 껍데기입니다. 반드시 풀.connect() 로 잡은 같은 연결에서 BEGIN/COMMIT 을 해야 합니다. 풀.query("BEGIN") 은 롤백이 안 됩니다.

```
async function 트랜잭션(할일) {
  const 연결 = await 풀.connect();

  try {
    await 연결.query("BEGIN");
    const 값 = await 할일(연결);
    await 연결.query("COMMIT");
    return 값;
  } catch (에러) {
    await 연결.query("ROLLBACK");
    throw 에러;
  }
}
```

```

    } finally {
        연결.release();
    }
}

async function 고장신고(설비번호, 담당사번, 비고) {
    return 트랜잭션(async (연결) => {
        await 연결.query(
            "INSERT INTO 점검기록 (설비번호, 담당사번, 점검일, 결과, 비고) VALUES
            ($1,$2,CURRENT_DATE, '고장', $3)",
            [설비번호, 담당사번, 비고],
        );

        const 고침 = await 연결.query("UPDATE 설비 SET 상태 = '고장' WHERE 설비번호 =
        $1", [설비번호]);

        if (고침.rowCount === 0) throw new Error(`설비 ${설비번호} 가 없습니다`);

        return "신고됨";
    });
}

console.log("고장 신고:", await 고장신고(2, 101, "이상 소음"));

```

출력 고장 신고: 신고됨

```

const 상태확인 = await 준비.query("SELECT 상태 FROM 설비 WHERE 설비번호 = 2");

console.log("설비 2번 상태:", 상태확인.rows[0].상태);

```

출력 설비 2번 상태: 고장

중간에 실패하면 **통째로** 없던 일이 되는지 확인합니다.

```

const 신고전건수 = (await 준비.query("SELECT count(*)::int AS 수 FROM
점검기록")).rows[0].수;

let 실패결과 = "성공함";

try {
    await 고장신고(999, 101, "없는 설비");
} catch (에러) {
    실패결과 = 에러.code ?? "직접던진에러";
}

const 신고후건수 = (await 준비.query("SELECT count(*)::int AS 수 FROM

```

```
점검기록")).rows[0].수;

console.log("없는 설비에 신고하면:", 실패결과);
```

```
출력      없는 설비에 신고하면: 23503
```

```
console.log("점검기록이 안 늘었나:", 신고전건수 === 신고후건수);
```

```
출력      점검기록이 안 늘었나: true
```

★★ 외래키가 먼저 막아 줬습니다(23503). 만약 외래키가 없었다면 점검기록만 들어가고 설비는 안 바뀌는 **어긋난 상태**가 됐을 것입니다. 제약과 트랜잭션은 같이 일합니다.

앱 연결 (08단원)

섹션 7

08단원의 저장소 계층입니다. **첫 인자가 항상 실행자**입니다. 그래야 풀에서 바로 쓸 수도 있고, 트랜잭션 안에서 쓸 수도 있습니다.

```
const 설비저장소 = {
  async 라인별로(실행자, 라인코드) {
    const 결과 = await 실행자.query(
```

★ **SELECT *** 를 안 씁니다. 필요한 칸만 적습니다 (섹션 8 에서 이유를 봅니다)

```
"SELECT 설비번호, 이름, 상태 FROM 설비 WHERE 라인코드 = $1 ORDER BY 설비번호",
  [라인코드], // * 값은 항상 $1 로. 이어 붙이면 SQL 인젝션입니다 (03단원)
);

return 결과.rows;
},

async 고장난것세기(실행자) {
  const 결과 = await 실행자.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE
상태 = '고장'");
  return 결과.rows[0].건수;
},
};

const A라인설비 = await 설비저장소.라인별로(풀, "A");

console.log("A라인 설비:", A라인설비.map((줄) => `${줄.이름}(${줄.상태})`).join(",
"));
```

```
출력      A라인 설비: 컨베이어 1호(가동), 프레스 1호(고장)
```

```
console.log("고장난 설비 수:", await 설비저장소.고장난것세기(풀));
```

출력 고장난 설비 수: 1

★ 03단원에서 뚫어 본 그 인젝션을 다시 확인합니다.

```
const 나쁜입력 = "A' OR '1'='1";

const 안전한결과 = await 설비저장소.라인별로(풀, 나쁜입력);

console.log("공격 문자열을 $1 로 넘기면 나오는 줄 수:", 안전한결과.length);
```

출력 공격 문자열을 \$1 로 넘기면 나오는 줄 수: 0

★★ 0줄입니다. A' OR '1'='1 이라는 **라인코드를 찾은 것**이기 때문입니다. 그런 라인은 없습니다. 공격이 아니라 그냥 값이 됐습니다. 이게 파라미터의 힘입니다.

★★ 점검 도구 만들기

섹션 8

여기가 이 파일의 핵심입니다.

이 자료에서 배운 것들을 **데이터베이스에 직접 물어봐서** 찾아 주는 함수를 만듭니다. 사람이 눈으로 훑는 것은 놓칩니다. 기계로 훑어야 합니다.

```
async function 스키마점검(연결, 대상스키마) {
  const 지적 = [];

  const 담기 = (종류, 줄들, 설명) => {
    for (const 줄 of 줄들) 지적.push({ 종류, 대상: 줄.대상, 설명 });
  };
}
```

① 색인 없는 외래키 (06단원) 부모를 지우거나 고칠 때마다 자식 표를 통째로 훑습니다. ★
conrelid::regclass::text 를 쓰면 한글 이름에 따옴표가 붙습니다("설비").

읽기 좋게 `relname` 으로 직접 있습니다.

```
const 색인없는외래키 = await 연결.query(
  `SELECT (t.relname || '.' || a.attname) AS 대상
  FROM pg_constraint c
  JOIN pg_class t ON t.oid = c.conrelid
  JOIN pg_attribute a ON a.attrelid = c.conrelid AND a.attnum = c.conkey[1]
  WHERE c.contype = 'f' AND c.connamespace = $1::regnamespace
  AND NOT EXISTS (
    SELECT 1 FROM pg_index i
    WHERE i.indrelid = c.conrelid AND i.indkey[0] = c.conkey[1]
  )
  ORDER BY 1`,
```

```
);
```

```
    담기("색인없는외래키", 색인없는외래키.rows, "부모를 지울 때마다 자식 표를 전부  
    뺏습니다");
```

② 기본키 없는 표 (02단원) 중복을 못 막고, 한 줄을 꼭 집을 수가 없습니다.

```
const 기본키없음 = await 연결.query(  
    `SELECT c.relname::text AS 대상  
    FROM pg_class c  
    WHERE c.relnamespace = $1::regnamespace AND c.relkind = 'r'  
    AND NOT EXISTS (SELECT 1 FROM pg_constraint k WHERE k.conrelid = c.oid AND  
    k.contype = 'p')  
    ORDER BY 1`,  
    [대상스키마],  
);  
  
    담기("기본키없음", 기본키없음.rows, "중복을 못 막고 한 줄을 꼭 집을 수 없습니다");
```

③ 돈처럼 보이는데 NUMERIC 이 아닌 칸 (02단원) FLOAT 로 돈을 담으면 더할수록 어긋납니다.

```
const 돈이아닌돈 = await 연결.query(  
    `SELECT (c.relname || '.' || a.attname || ' (' || format_type(a.atttypid,  
    a.atttypmod) || ')') AS 대상  
    FROM pg_attribute a  
    JOIN pg_class c ON c.oid = a.attrelid  
    WHERE c.relnamespace = $1::regnamespace AND c.relkind = 'r'  
    AND a.attnum > 0 AND NOT a.attisdropped  
    AND (a.attname LIKE '%가%' OR a.attname LIKE '%금액%' OR a.attname LIKE '%  
    단가%'  
    OR a.attname LIKE '%비용%' OR a.attname LIKE '%원가%')  
    AND a.atttypid <> 'numeric'::regtype  
    ORDER BY 1`,  
    [대상스키마],  
);  
  
    담기("돈이NUMERIC아님", 돈이아닌돈.rows, "FLOAT 로 돈을 담으면 더할수록  
    어긋납니다");
```

④ 한 번도 안 쓰인 색인 (06단원) INSERT 마다 같이 써야 하고, 백업에도 들어가고, 캐시도 먹습니다.


```
const 안쓰는색인 = await 연결.query(
  `SELECT (indexrelname || ' (' || pg_size_pretty(pg_relation_size(indexrelid))
  || ')') AS 대상
  FROM pg_stat_user_indexes
  WHERE schemaname = $1 AND idx_scan = 0
  AND indexrelid NOT IN (SELECT conindid FROM pg_constraint WHERE contype IN
  ('p','u'))
  ORDER BY 1`,
  [대상스키마],
);

답기("안쓰이는색인", 안쓰는색인.rows, "쓰이지도 않으면서 쓰기를 느리게 합니다");
```

⑤ 부폰 표 (개념03)

```
const 부폰표 = await 연결.query(
  `SELECT (relname || ' - 죽은 줄 ' || n_dead_tup || '개') AS 대상
  FROM pg_stat_user_tables
  WHERE schemaname = $1 AND n_dead_tup > 1000 AND n_dead_tup > n_live_tup
  ORDER BY 1`,
  [대상스키마],
);

답기("부폰표", 부폰표.rows, "죽은 줄이 산 줄보다 많습니다. 청소가 막혔는지 보세요");
```

⑥ 오래된 트랜잭션 (개념03 · 07단원) ★ 이걸 스키마와 상관없이 서버 전체를 봅니다.

```
const 오래된트랜잭션 = await 연결.query(
  `SELECT (application_name || ' - ' || state || ' ' ||
  round(extract(epoch FROM now() - xact_start))::text || '초') AS 대상
  FROM pg_stat_activity
  WHERE xact_start IS NOT NULL AND pid <> pg_backend_pid()
  AND now() - xact_start > interval '10 seconds'
  ORDER BY xact_start`,
);

답기("오래된트랜잭션", 오래된트랜잭션.rows, "청소를 막고 잠금을칩니다");
```

⑦ 못 쓰는 색인 (개념04)

```
const 망가진색인 = await 연결.query(
  `SELECT x.relname::text AS 대상
  FROM pg_index i
  JOIN pg_class c ON c.oid = i.indrelid
  JOIN pg_class x ON x.oid = i.indexrelid
```

```
WHERE NOT i.indisvalid AND c.relnamespace = $1::regnamespace
    ORDER BY 1`,
    [대상스키마],
);

담기("못쓰는색인", 망가진색인.rows, "CONCURRENTLY 가 실패해서 남은 것입니다. 지우고
다시 만드세요");

return 지적;
}
```

코드도 점검합니다

위의 것들은 데이터베이스에 물어봤습니다. `SELECT *` 나 이어 붙인 SQL 은 **코드 안에** 있습니다. 글자로 찾습니다.

```
function 코드점검(파일이름, 소스) {
    const 지적 = [];

    소스.split("\n").forEach((줄, 자리) => {
        const 어디 = `${파일이름}:${자리 + 1}`;

```

① `SELECT *` (05단원 · 08단원)

```
if (/select\s+*/i.test(줄)) {
    지적.push({ 종류: "SELECT별표", 대상: 어디, 설명: "칸이 늘면 조용히 같이 달려
웁니다" });
}
```

② 글자로 이어 붙인 SQL (03단원) — ★ 제일 위험합니다

★ 정규식 안의 따옴표를 `\x22(") \x27(') \x60(`)` 로 적었습니다.

따옴표를 그대로 쓰면 ****코드를 훑는 도구들이 문자열의 시작으로 오해합니다.****
 실제로 이 자료의 검증 도구가 그렇게 오해해서 파일 뒷부분을 통째로 놓쳤습니다.
 정규식에 따옴표를 넣을 일이 있으면 이스케이프로 적는 편이 안전합니다.

```
if (/select|insert|update|delete)[^\x22\x27\x60]*$/i.test(줄) ||
/(where|values)[^\x22\x27\x60]*\x22\s+*/i.test(줄)) {
    지적.push({ 종류: "SQL이어붙임", 대상: 어디, 설명: "★ SQL 인젝션입니다. $1 을
쓰세요" });
}
```

③ 폴에 직접 `BEGIN` (08단원)

```
if (/풀\.query\(\s*[\x22\x27\x60]\s*(BEGIN|COMMIT|ROLLBACK)/i.test(줄) ||
/pool\.query\(\s*[\x22\x27\x60]\s*BEGIN/i.test(줄)) {
    지적.push({ 종류: "풀에BEGIN", 대상: 어디, 설명: "문장마다 다른 연결로 갑니다.
    롤백이 안 됩니다" });
}
});

return 지적;
}
```

일부러 문제를 심어 둔 코드 조각입니다. 점검 도구가 찾아내는지 봅시다.

```
const 나쁜코드 = [
    'const 목록 = await 풀.query("SELECT * FROM 설비");',
    'const 하나 = await 풀.query(`SELECT 이름 FROM 설비 WHERE 라인코드 = ${입력}`);',
    'await 풀.query("BEGIN");',
    'const 안전 = await 풀.query("SELECT 이름 FROM 설비 WHERE 라인코드 = $1",
    [입력]);',
].join("\n");

const 코드지적 = 코드점검("나쁜예시.js", 나쁜코드);

for (const 줄 of 코드지적) console.log(` [${줄.종류}] ${줄.대상} - ${줄.설명}`);
```

출력 [SELECT별표] 나쁜예시.js:1 - 칸이 늘면 조용히 같이 딸려 옵니다
 [SQL이어붙임] 나쁜예시.js:2 - * SQL 인젝션입니다. \$1 을 쓰세요
 [풀에BEGIN] 나쁜예시.js:3 - 문장마다 다른 연결로 갑니다. 롤백이 안
 됩니다

```
console.log("마지막 줄($1 을 쓴 것)은 안 걸렸나:", !코드지적.some((줄) => 줄.
대상.endsWith(":4")));
```

출력 마지막 줄(\$1 을 쓴 것)은 안 걸렸나: true

진짜 스키마에 돌려 봅니다

우리가 방금 만든 시스템입니다. 잘 만들었다고 생각했는데 어떨까요.

색인 통계가 쌓이도록 실제 질의를 몇 번 돌립니다. ★ 06단원에서 배운 것: 통계는 **진짜로 실행한 질의**만 셉니다. EXPLAIN 은 안 셉니다.

```
for (let 회차 = 0; 회차 < 3; 회차 += 1) {
  await 준비.query("SELECT 결과 FROM 점검기록 WHERE 설비번호 = 3 AND 점검일 >= DATE
'2026-03-01' LIMIT 5");
}

await 준비.query("ANALYZE");
```

★ 색인 사용 통계는 조금 늦게 반영됩니다. 잠깐 기다렸다 봅시다. (개념03 에서 본 그것)

```
await new Promise((끝) => setTimeout(끝, 1000));

const 첫점검 = await 스키마점검(준비, 스키마);
```

★ 종류별로 몇 건인지 먼저 봅시다. 총 건수는 통계가 언제 반영되느냐에 따라 달라집니다.

```
const 종류별 = {};

for (const 줄 of 첫점검) 종류별[줄.종류] = (종류별[줄.종류] ?? 0) + 1;

console.log("점검 결과(종류별):", JSON.stringify(종류별));
```

출력 점검 결과(종류별): {"색인없는외래키":3}

값이 다를 수 있음

```
const 외래키지적 = 첫점검.filter((줄) => 줄.종류 === "색인없는외래키");

for (const 줄 of 외래키지적) console.log(`  [${줄.종류}] ${줄.대상} - ${줄.설명}`);
```

출력 [색인없는외래키] 설비.라인코드 - 부모를 지울 때마다 자식 표를 전부
 훑습니다
 [색인없는외래키] 작업자.소속라인 - 부모를 지울 때마다 자식 표를 전부
 훑습니다
 [색인없는외래키] 점검기록.담당사번 - 부모를 지울 때마다 자식 표를 전부
 훑습니다

```
console.log("색인 없는 외래키가 잡혔나:", 외래키지적.length > 0);
```

출력 색인 없는 외래키가 잡혔나: true

★ 생산실적.라인코드 는 안 걸렸습니다. 왜일까요. 기본키가 (라인코드, 생산일) 복합키라서 첫 칸이 라인코드입니다. 그 기본키 색인을 외래키 검사에 그대로 쓸 수 있습니다. 그래서 따로 안 걸어도 됩니다.

★★ 06단원의 “복합 색인은 왼쪽부터 쓴다” 가 여기서도 그대로 통합니다.

★★ 우리가 방금 만든 표에서 문제가 나왔습니다. 외래키를 걸었으면 그 칸에 색인도 걸어야 합니다. Postgres 는 기본키·UNIQUE 에는 색인을 자동으로 만들어 주지만 외래키에는 안 만들어 줍니다. 이게 제일 흔한 누락입니다.

왜 필요한가: DELETE FROM 라인 WHERE 라인코드 = 'D' 를 할 때 Postgres 는 “이 라인을 쓰는 설비가 있나” 를 확인해야 합니다. 색인이 없으면 설비 표를 **통째로 훑습니다**. 그동안 잠금도칩니다.

고칩니다.

```
await 준비.query(`
  CREATE INDEX 설비_라인코드 ON 설비 (라인코드);
  CREATE INDEX 작업자_소속라인 ON 작업자 (소속라인);
  CREATE INDEX 점검_담당사번 ON 점검기록 (담당사번);
  CREATE INDEX 생산_라인코드 ON 생산실적 (라인코드);
`);

const 둘째점검 = await 스키마점검(준비, 스키마);
const 남은외래키지적 = 둘째점검.filter((줄) => 줄.종류 === "색인없는외래키");

console.log("고친 뒤 색인 없는 외래키:", 남은외래키지적.length, "건");
```

출력 고친 뒤 색인 없는 외래키: 0 건

★ 그런데 이번엔 **다른 게** 걸립니다. 방금 만든 색인들이 아직 안 쓰였으니까요.

```
const 안쓰인것 = 둘째점검.filter((줄) => 줄.종류 === "안쓰이는색인");

console.log("이번엔 '안 쓰이는 색인' 이 잡혔나:", 안쓰인것.length > 0);
```

출력 이번엔 '안 쓰이는 색인' 이 잡혔나: true

★★★ **점검 도구를 맹신하면 안 됩니다**. 방금 만든 색인이 안 쓰였다고 나오는 게 당연합니다. 아직 아무도 안 썼으니까요. 06단원에서 경고했습니다. “머칠 돌려 본 통계로 판단하면 안 됩니다.”

점검 도구는 **볼 곳을 알려 주는 것**이지 답을 주는 게 아닙니다. 각 항목마다 “왜 이런가” 를 사람이 판단해야 합니다.

오래된 트랜잭션도 잡히는지 확인합니다.

```
const 게으른연결 = new pg.Client({ ...접속정보, application_name: "unit10-lazy" });
await 게으른연결.connect();
await 게으른연결.query("BEGIN");
await 게으른연결.query("SELECT 1");
```

★ 10초가 지나야 잡히므로, 시간을 훑내 내서 같은 질의를 0초 기준으로 돌려 봅니다.

```
const 지금열린것 = await 준비.query(`
  SELECT application_name AS 이름표, state
  FROM pg_stat_activity
  WHERE xact_start IS NOT NULL AND application_name = 'unit10-lazy'
`);

console.log("열어 두고 노는 트랜잭션:", JSON.stringify(지금열린것.rows[0]));
```

```
출력      열어 두고 노는 트랜잭션: {"이름표":"unit10-lazy","state":"idle in
        transaction"}
```

```
await 게으른연결.query("ROLLBACK");
await 게으른연결.end();
```

운영 점검 목록

섹션 9

배포 전에 보는 것 / 배포 후에 보는 것을 나눠서 적습니다.

배포 전

☐ 마이그레이션에 `SET lock_timeout` 이 있는가 (개념04) ☐ 칸 이름 바꾸기·지우기가 확장 → 이전 → 축소로 나뉘었는가 (개념04) ☐ `CREATE INDEX` 에 `CONCURRENTLY` 가 붙었고 트랜잭션 밖인가 (개념04) ☐ 새 쿼리의 `EXPLAIN` 을 봤는가. Seq Scan 이 남아 있는가 (06단원) ☐ 새 외래키에 색인을 같이 걸었는가 (섹션 8) ☐ 되돌릴 방법이 있는가 (옛 칸을 아직 안 지웠는가) (개념04) ☐ ★ 백업이 최근 것이고, 복구해 본 적이 있는가 (개념01)

배포 후

☐ 느린 쿼리 순위가 바뀌었는가 (개념02) ☐ 에러 로그에 새 에러코드가 있는가 (23503 · 42703 · 23502) ☐ 커넥션 풀이 꽉 차지 않았는가 (08단원) ☐ 잠금을 오래 기다린 것이 있는가 (`log_lock_waits`) (07단원) ☐ 죽은 줄이 갑자기 늘지 않았는가 (개념03) ☐ 못 쓰는 색인이 남지 않았는가 (`indisvalid = false`) (개념04)

★ 01단원에서 “GRANT · REVOKE 는 10단원에서” 라고 했습니다. 여기서 갓습니다.

권한도 운영의 일부입니다. **애플리케이션 계정에 관리자 권한을 주지 마세요.**

★★ 역할은 그냥 `DROP ROLE` 이 안 됩니다. 그 역할에 준 권한이 남아 있으면 `2BP01` 로 거절합니다. `DROP OWNED BY` 로 권한부터 걷어 내야 합니다. 실제로 이 에러를 만났습니다.

```
async function 역할지우기(이름) {
  const 있나 = await 준비.query("SELECT 1 FROM pg_roles WHERE rolname = $1", [이름]);

  if (있나.rowCount === 0) return;
```

운영과 종합 실습

RUN node 연습문제.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 채점에 VACUUM 과 스키마 변경이 들어가서 25초쯤 걸립니다.

// TODO: 를 채우고 실행하면 채점 결과가 나옵니다. 아무것도 안 채우고 실행하면 전부 실패로 나옵니다.
그게 정상입니다.

14문제입니다. 뒤로 갈수록 어렵습니다. 마지막 세 개는 [도전] 입니다. 막히면 연습문제_정답.js 를
보세요. 답만 있는 게 아니라 왜 그런지가 적혀 있습니다.

```
import pg from "pg";
import { execSync } from "node:child_process";
```

```

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434, // * 5432 가 아닙니다 user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit10-exercise", // * 영어로
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

const 스키마 = "단원10_연습";

await 연결.query(`DROP SCHEMA IF EXISTS ${스키마} CASCADE`);
await 연결.query(`CREATE SCHEMA ${스키마}`);
await 연결.query(`SET search_path TO ${스키마}`);

async function 새연결(이름표) {
  const 새것 = new pg.Client({ ...접속정보, application_name: 이름표 });
  await 새것.connect();
  await 새것.query(`SET search_path TO ${스키마}`);
  return 새것;
}

const 컨테이너 = "db-material-postgres";

function 도커(명령) {
  return execSync(`docker exec ${컨테이너} sh -c ${JSON.stringify(명령)}`, {
    encoding: "utf8",
    timeout: 120000,
    maxBuffer: 32 * 1024 * 1024,
  });
}

```

실습용 표입니다. 05단원에서 쓰던 제조 현장 표입니다.


```

await 연결.query(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL
  );

  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT NOT NULL REFERENCES 라인(라인코드),
    도입가   NUMERIC(12,2) NOT NULL
  );

  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT NOT NULL REFERENCES 설비(설비번호),
    점검일   DATE NOT NULL,
    결과      TEXT NOT NULL
  );

  CREATE TABLE 임시메모 (
    내용 TEXT
  );
`);

await 연결.query(`
  INSERT INTO 라인 VALUES ('A','조립1라인'), ('B','가공2라인'), ('C','포장3라인');

  INSERT INTO 설비 VALUES
    (1,'컨베이어 1호','A',12000000.00),
    (2,'프레스 1호','A',35400000.00),
    (3,'용접로봇 1호','B',88000000.00);

  INSERT INTO 점검기록
  SELECT g, (g % 3) + 1, DATE '2026-01-01' + (g % 120),
    (ARRAY['정상','주의','고장'])[(g % 3) + 1]
  FROM generate_series(1, 40000) g;
`);

await 연결.query("VACUUM ANALYZE");

const 잠깐 = (밀리초) => new Promise((끝) => setTimeout(끝, 밀리초));

```

문제 1 — 백업 명령의 형식 고르기

표 하나만 골라 되살릴 수 있고, 압축도 되는 pg_dump 형식은 무엇인가요? "plain" / "custom" / "directory" 중에 하나를 문자열로 적으세요.

```
const 답1 = null; // TODO: 형식 이름을 문자열로
```

문제 2 — 백업을 진짜로 만들기

단원10_연습.설비 표 하나만 custom 형식으로 뽑는 pg_dump 명령을 완성하세요. 결과 파일은 /tmp/연습백업.dump 입니다.

힌트: -t 스키마.표이름, --format=custom, -f 파일경로

앞부분은 이미 적어 뒀습니다.

```
const 백업명령 = null; // TODO: 문자열로. `PGPASSWORD=secret pg_dump -U factory -d
factory_db ...`
```

문제 3 — 총 시간으로 줄 세우기

pg_stat_statements 를 훑내 낸 배열입니다. 서버 시간을 제일 많이 잡아먹는 쿼리의 이름을 돌려주세요.

★ 주의: 평균이 긴 것이 아니라 총 시간이 긴 것입니다.

```
const 쿼리통계 = [
  { 이름: "목록조회", 호출: 500000, 평균ms: 0.4 },
  { 이름: "월간집계", 호출: 12, 평균ms: 900 },
  { 이름: "상세조회", 호출: 80000, 평균ms: 1.1 },
];
```

```
function 진짜범인(통계) {
```

총 시간(호출 × 평균ms)이 가장 큰 것의 이름을 돌려주세요

```
return null;
}
```

문제 4 — 지금 오래 돌고 있는 쿼리 찾기

pg_stat_activity 에서 지금 실행 중이면서 1초 넘게 돌고 있는 세션을 찾는 SQL 을 쓰세요.
application_name 이 'unit10-slowvictim' 인 것만 봅니다.

돌려줄 칸: 문장 하나 (query 의 앞 20글자)

힌트: state = 'active', now() - query_start > interval '1 second'

```
left(query, 20) AS 문장
```

```
const 돌고있는것찾기 = null; // TODO: SQL 문자열
```

문제 5 — 취소인가 강제 종료인가

아래 상황에서 무엇을 써야 할까요. "pg_cancel_backend" 또는 "pg_terminate_backend" 를 문자열로 적으세요.

"보고서 쿼리가 10분째 돌고 있습니다. 멈추고 싶은데, 그 애플리케이션의 연결은 살려 두고 싶습니다."

```
const 답5 = null; // TODO
```

문제 6 — 표 크기 재기

점검기록 표의 **본체 크기**(색인 제외)를 바이트로 돌려주는 SQL 을 쓰세요. 돌려줄 칸 이름은 **바이트** 이고 정수여야 합니다.

힌트: pg_relation_size('스키마.표 '::regclass)::int AS 바이트 ★ pg_relation_size(\$1) 처럼 파라미터로 넘기면 타입 추론이 깨집니다.

문자열로 직접 적으세요.

```
const 표크기SQL = null; // TODO: SQL 문자열
```

문제 7 — VACUUM 인가 VACUUM FULL 인가

두 상황에 각각 "VACUUM" 또는 "VACUUM FULL" 을 넣으세요.

상황 가: "운영 중입니다. 죽은 줄이 쌓여서 표가 계속 커집니다.

더 커지지만 얹게 하고 싶습니다."

상황 나: "새벽에 서비스를 30분 세울 수 있습니다.

디스크를 진짜로 되찾고 싶습니다."

```
const 답7가 = null; // TODO
```

```
const 답7나 = null; // TODO
```

문제 8 — 죽은 줄이 많은 표 찾기

단원10_연습 스키마에서 **죽은 줄이 1000개를 넘는** 표를 찾는 SQL 을 쓰세요. 돌려줄 칸: 표 (표 이름)

힌트: pg_stat_user_tables 의 schemaname, relname, n_dead_tup

```
const 부품표찾기 = null; // TODO: SQL 문자열
```

문제 9 — 긴 트랜잭션 찾기

청소를 막고 있을 만한 세션을 찾습니다. `application_name` 이 'unit10-blocker2' 이고 **트랜잭션이 열려 있는** 세션의 `state` 를 돌려주는 SQL 을 쓰세요.

돌려줄 칸: `state`

힌트: `xact_start IS NOT NULL` 이면 트랜잭션이 열려 있는 것입니다

```
const 긴트랜잭션찾기 = null; // TODO: SQL 문자열
```

문제 10 — 안전한 칸 추가

운영 중인 `설비` 표에 `담당자` `TEXT` 칸을 추가하려고 합니다. **잠금을 못 잡으면 2초 만에 포기**하도록 만드세요.

두 문장을 순서대로 배열로 돌려주세요. ["SET lock_timeout = ...", "ALTER TABLE ..."]

```
function 안전한칸추가() {
```

문장 두 개가 든 배열을 돌려주세요

```
return null;  
}
```

문제 11 — 안 막는 색인 만들기

점검기록(`설비번호`) 에 색인을 만듭니다. 이름은 `연습_설비번호` 입니다. **쓰기를 막지 않는** 방법으로 만드는 SQL 한 줄을 쓰세요.

```
const 안막는색인 = null; // TODO: SQL 문자열
```

문제 12 — [도전] 확장 → 이전 → 축소

`설비.이름` 을 `설비.설비명` 으로 바꾸려고 합니다. 무중단으로 하려면 어떤 순서여야 할까요.

아래 다섯 가지 일을 **안전한 순서**로 배열에 담으세요. 문자열을 그대로 쓰면 됩니다.

```
"옛칸지우기" ALTER TABLE 설비 DROP COLUMN 이름
"새칸추가" ALTER TABLE 설비 ADD COLUMN 설비명 TEXT
"값복사" UPDATE 설비 SET 설비명 = 이름
"새코드배포"      새 코드가 설비명을 읽고 씁니다
"동기화트리거"    양쪽 칸을 자동으로 맞춰 줍니다
```

★ 힌트: 새 코드가 배포되기 **전에** 새 칸이 있어야 합니다. 그리고 옛 칸은 **모든 서버가 새 코드가 된 뒤에** 지웁니다.

```
const 무중단순서 = null; // TODO: 문자열 5개가 든 배열
```

문제 13 — [도전] 색인 없는 외래키 찾기

외래키를 걸었는데 그 칸에 색인이 없는 곳을 찾는 SQL 을 쓰세요. [단원10_연습](#) 스키마만 봅니다.

돌려줄 칸: [표](#) 와 [칸](#)

힌트: pg_constraint (contype='f', conkey[1] 이 첫 칸)

```
pg_index 에 indrelid 가 같고 indkey[0] = conkey[1] 인 것이 없으면 색인 없음
pg_attribute 로 칸 번호를 이름으로 바꿉니다
```

```
const 색인없는외래키찾기 = null; // TODO: SQL 문자열
```

문제 14 — [도전] 코드 점검 도구

소스 코드 문자열을 받아서 위험한 줄을 찾아 주는 함수를 만드세요.

찾을 것 두 가지 ① `SELECT *` → 종류 "별표" ② 템플릿 문자열로 이어 붙인 SQL → 종류 "이어붙임"

```
(SELECT/INSERT/UPDATE/DELETE 가 있는 줄에 `${` 가 있으면)
```

돌려줄 것: { [줄번호](#), [종류](#) } 객체의 배열. 줄번호는 1부터 씁니다. 한 줄에 둘 다 있으면 두 개를 담습니다. 순서는 ① 다음 ② 입니다.

```
function 코드점검(소스) {
```

```
  { 줄번호, 종류 } 배열을 돌려주세요
```

```
  return null;
}
```

```
const 채점결과 = [];

async function 검사(번호, 이름, 할일) {
  try {
    const 통과 = await 할일();
    채점결과.push({ 번호, 이름, 통과: 통과 === true });
  } catch {
    채점결과.push({ 번호, 이름, 통과: false });
  }
}

await 검사(1, "백업 형식 고르기", async () => 답1 === "custom");

await 검사(2, "백업을 진짜로 만들기", async () => {
  if (typeof 백업명령 !== "string") return false;

  도커("rm -f /tmp/연습백업.dump");
  도커(백업명령);

  const 목록 = 도커("pg_restore -l /tmp/연습백업.dump");

  return 목록.includes("설비") && !목록.includes("점검기록");
});

await 검사(3, "총 시간으로 줄 세우기", async () => 진짜범인(쿼리통계) ===
"목록조회");

await 검사(4, "오래 돌고 있는 쿼리 찾기", async () => {
  if (typeof 돌고있는것찾기 !== "string") return false;

  const 피해자 = await 새연결("unit10-slowvictim");
  const 던짐 = 피해자.query("SELECT pg_sleep(20)").catch(() => {});

  await 잠깐(1400);

  const 결과 = await 연결.query(돌고있는것찾기);
```

```

const pid = (
    await 연결.query("SELECT pid FROM pg_stat_activity WHERE application_name =
'unit10-slowvictim'")
    ).rows[0].pid;

    await 연결.query("SELECT pg_terminate_backend($1)", [pid]);
    피해자.on("error", () => {});
    await 던짐;
    await 피해자.end().catch(() => {});

    return 결과.rows.length === 1 && String(결과.rows[0].문장).includes("pg_sleep");
});

await 검사(5, "취소인가 강제 종료인가", async () => 답5 === "pg_cancel_backend");

await 검사(6, "표 크기 재기", async () => {
    if (typeof 표크기SQL !== "string") return false;

    const 결과 = await 연결.query(표크기SQL);

    return Number.isInteger(결과.rows[0].바이트) && 결과.rows[0].바이트 > 100000;
});

await 검사(7, "VACUUM 인가 VACUUM FULL 인가", async () => 답7가 === "VACUUM" && 답7나
=== "VACUUM FULL");

await 검사(8, "죽은 줄이 많은 표 찾기", async () => {
    if (typeof 부폰표찾기 !== "string") return false;

    await 연결.query("UPDATE 점검기록 SET 결과 = 결과");
    await 연결.query("ANALYZE 점검기록");

    let 결과 = await 연결.query(부폰표찾기);

    for (let 다시 = 0; 결과.rows.length === 0 && 다시 < 10; 다시 += 1) {
        await 잠깐(500);
        await 연결.query("ANALYZE 점검기록");
        결과 = await 연결.query(부폰표찾기);
    }
}

```

```

    }

    await 연결.query("VACUUM 점검기록");

    return 결과.rows.some((줄) => 줄.표 === "점검기록");
  });

  await 검사(9, "긴 트랜잭션 찾기", async () => {
    if (typeof 긴트랜잭션찾기 !== "string") return false;

    const 방해꾼 = await 새연결("unit10-blocker2");
    await 방해꾼.query("BEGIN");
    await 방해꾼.query("SELECT 1");

    const 결과 = await 연결.query(긴트랜잭션찾기);

    await 방해꾼.query("ROLLBACK");
    await 방해꾼.end();

    return 결과.rows.length === 1 && 결과.rows[0].state === "idle in transaction";
  });

  await 검사(10, "안전한 칸 추가", async () => {
    const 문장들 = 안전한칸추가();

    if (!Array.isArray(문장들) || 문장들.length !== 2) return false;
    if (!/lock_timeout/i.test(문장들[0])) return false;
    if (!/2\s*s|2000\s*ms/i.test(문장들[0])) return false;

    for (const 문장 of 문장들) await 연결.query(문장);

    await 연결.query("SET lock_timeout = 0");

    const 있나 = await 연결.query(`
      SELECT count(*)::int AS 개수 FROM information_schema.columns
      WHERE table_schema = '${스키마}' AND table_name = '설비' AND column_name =
      '담당자'
    `);
  });
}

```



```

`);

return 있나.rows[0].개수 === 1;
});

await 검사(11, "안 막는 색인 만들기", async () => {
  if (typeof 안막는색인 !== "string") return false;
  if (!/CONCURRENTLY/i.test(안막는색인)) return false;

  await 연결.query(안막는색인);

  const 있나 = await 연결.query(` SELECT indisvalid FROM pg_index WHERE indexrelid =
'${스키마}.연습_설비번호'::regclass
`);

  return 있나.rows[0].indisvalid === true;
});

await 검사(12, "[도전] 확장 → 이전 → 축소", async () => {
  if (!Array.isArray(무중단순서) || 무중단순서.length !== 5) return false;

  const 자리 = (이름) => 무중단순서.indexOf(이름);

  return (
    자리("새칸추가") === 0 &&
    자리("동기화트리거") === 1 &&
    자리("값복사") === 2 &&
    자리("새코드배포") === 3 &&
    자리("옛칸지우기") === 4
  );
});

await 검사(13, "[도전] 색인 없는 외래키 찾기", async () => {
  if (typeof 색인없는외래키찾기 !== "string") return false;

  const 결과 = await 연결.query(색인없는외래키찾기);

  const 찾은것 = 결과.rows.map((줄) => `${줄.표}.${줄.칸}`).sort();

```

설비.라인코드는 색인이 없습니다. 점검기록.설비번호는 문제 11에서 만들었습니다.

```

return 찾은것.includes("설비.라인코드") && !찾은것.includes("점검기록.설비번호");
});

await 검사(14, "[도전] 코드 점검 도구", async () => {
  const 소스 = [
    'const a = await 풀.query("SELECT * FROM 설비");',
    'const b = await 풀.query(`SELECT 이름 FROM 설비 WHERE 라인코드 = ${입력}`);',

```


부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

06단원 연습문제 정답 색인과 실행계획

```
RUN node 연습문제_정답.js
```

★ 이 파일은 20초쯤 걸립니다.

답만 보지 마세요. 왜 그 답인지, 그리고 다른 답은 왜 안 되는지 가 각 문제 뒤에 길게 적혀 있습니다. 맞힌 문제도 설명은 읽어 보세요. 맞았지만 이유가 다를 수 있고, 그러면 다음번에 틀립니다.

```

import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();

await db.exec(`
  CREATE TABLE 설비 (
    id          SERIAL PRIMARY KEY,
    이름        TEXT NOT NULL,
    라인        TEXT NOT NULL,
    관리번호    TEXT NOT NULL
  );
  INSERT INTO 설비 (이름, 라인, 관리번호)
  SELECT '설비' || i, (ARRAY['A','B','C','D'])[i % 4 + 1], lpad(i::text, 8, '0')
  FROM generate_series(1, 20000) AS i;

  CREATE TABLE 점검기록 (
    id          SERIAL PRIMARY KEY,
    설비번호    INT  NOT NULL REFERENCES 설비(id),
    상태        TEXT NOT NULL,
    점검자      TEXT NOT NULL,
    소속반      TEXT NOT NULL,
    점검일      DATE NOT NULL,
    비고        TEXT
  );
  INSERT INTO 점검기록 (설비번호, 상태, 점검자, 소속반, 점검일, 비고)
  SELECT (i % 20000) + 1,
         (ARRAY['정상','주의','고장'])[i % 3 + 1],
         '점검원' || ((i % 50) + 1),
         '반' || ((i % 50) + 1),
         DATE '2024-01-01' + ((i % 365) || ' days')::interval,
         '특이사항 없음. 기록번호 ' || i
  FROM generate_series(1, 200000) AS i;
  ANALYZE;
`);

const 결과 = [];

```

```

async function 계획(sql, 옵션 = "") {
  return (await db.query(`EXPLAIN ${옵션} ${sql}`)).rows.map((줄) => 줄["QUERY
PLAN"]).join("\n");
}

async function 채점(번호, 제목, 답, 검사) {
  const 비었나 = 답 === null || 답 === undefined || (typeof 답 === "string" &&
  답.trim() === "");

  if (비었나) {
    결과.push(false);
    console.log(`${번호}. ${제목} - 아직 안 풀었습니다`);
    return;
  }

  try {
    const 통과 = await 검사(답);
    결과.push(통과 === true);
    console.log(`${번호}. ${제목} - ${통과 === true ? "통과" : "틀렸습니다"}`);
  } catch (에러) {
    결과.push(false);
    console.log(`${번호}. ${제목} - 에러가 났습니다 (${에러.code ?? "?"})`);
  }
}

```

정답 1 — 색인 만들기

```

const 답1 = "CREATE INDEX 점검_점검자 ON 점검기록 (점검자)";

await 채점(1, "점검자에 색인 만들기", 답1, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 점검기록");
  const 계획글 = await 계획("SELECT * FROM 점검기록 WHERE 점검자 = '점검원7'");
  return 계획글.includes("점검_점검자");
});

```

출력 1. 점검자에 색인 만들기 - 통과

★ 왜 CREATE INDEX 이름 ON 표 (칸) 이 전부입니다. 이름을 안 주면 PostgreSQL 이 점검기록_점검자_idx 로 지어 줍니다. 그것도 됩니다. 그런데 회사에서는 이름 규칙을 정해 두는 게 낫습니다. 나중에 “이 색인 지워도 되나?” 를 판단할 때 이름만 보고 알아야 하기 때문입니다.

★ 만든 뒤에 ANALYZE 를 한 이유 색인을 만들면 그 색인의 통계도 새로 필요합니다. CREATE INDEX 는 색인 자체 통계는 만들어 주지만, 대량 작업 뒤에는 습관적으로 ANALYZE 를 붙이는 게 안전합니다.

★ 자주 하는 실수 CREATE INDEX ON 점검기록 점검자 — 괄호를 빼면 문법 에러(42601)입니다.

정답 2 — Seq Scan 인지 확인하기

```
const 답2 = "Seq Scan";

await 채점(2, "어떤 스캔이 나오나", 답2, async (이름) => {
  const 계획글 = await 계획("SELECT * FROM 점검기록 WHERE 비고 = '없음'");
  return 계획글.split("\n")[0].startsWith(이름.trim());
});
```

출력 2. 어떤 스캔이 나오나 - 통과

★ 왜 비고 칸에는 색인이 없습니다. 그러니 표를 처음부터 끝까지 읽는 수밖에 없습니다. 그게 Seq Scan(순차 훑기)입니다.

★ 여기서 놓치기 쉬운 것 "Seq Scan 이 나왔으니 색인을 걸어야겠다" 는 **자동 반응이 아닙니다**. 개념01 섹션 6 에서 봤듯이, 표가 작거나 대부분을 읽는 질의라면 Seq Scan 이 웁습니다. 봐야 할 것은 Rows Removed by Filter 입니다. 그 값이 결과 줄 수보다 훨씬 크면 그때 색인을 생각합니다.

정답 3 — 외래키에 색인 걸기

```
const 답3 = "CREATE INDEX 점검_설비번호 ON 점검기록 (설비번호)";

await 채점(3, "외래키에 색인 걸기", 답3, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 점검기록");
  const 계획글 = await 계획("SELECT * FROM 점검기록 WHERE 설비번호 = 777");
  return 계획글.includes("점검_설비번호") && !계획글.includes("Seq Scan");
});
```

출력 3. 외래키에 색인 걸기 - 통과

★ 왜 이게 중요한가 (개념02 섹션 9) REFERENCES 설비(id) 라고 썼다고 색인이 생기지 않습니다. 기본키 쪽(설비.id)에는 자동으로 생기지만 **참조하는 쪽에는 안 생깁니다**.

문제가 두 군데서 터집니다.

① 조인이 느립니다

```
`설비 JOIN 점검기록 ON 점검기록.설비번호 = 설비.id` 는 거의 모든 화면에서 씁니다
```

② 부모를 지울 때 자식 표를 통째로 훑습니다

```
`DELETE FROM 설비 WHERE id = 777` 을 하면 PostgreSQL 은
"이 설비를 참조하는 점검기록이 있나" 를 확인해야 합니다.
색인이 없으면 20만 줄을 다 봅니다. 자식이 1000만 건이면 그대로 멈춥니다.
```

★ MySQL 은 다릅니다 InnoDB 는 외래키를 만들면 색인을 **자동으로** 만들어 줍니다. 그래서 MySQL 하다 온 사람이 PostgreSQL 에서 특히 많이 당합니다.

```
const 답4 = "CREATE INDEX 점검_복합 ON 점검기록 (상태, 점검일)";

await 채점(4, "복합 색인 순서", 답4, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 점검기록");

  const 계획글 = await 계획(
    `SELECT * FROM 점검기록
     WHERE 상태 = '고장' AND 점검일 BETWEEN DATE '2024-03-01' AND DATE '2024-03-31'`,
  );

  const 색인조건 = 계획글.split("\n").filter((줄) => 줄.includes("Index
  Cond")).join(" ");
  return 계획글.includes("점검_복합") && 색인조건.includes("상태") &&
    색인조건.includes("점검일");
});
```

출력 4. 복합 색인 순서 - 통과

★ 왜 (상태, 점검일) 이고 (점검일, 상태) 가 아닌가

등호 조건을 앞에, 범위 조건을 뒤에. 이게 규칙입니다.

색인은 정렬된 목록입니다. (상태, 점검일) 로 만들면 이렇게 정렬됩니다.

```
고장 · 2024-01-01
고장 · 2024-01-02
...
고장 · 2024-12-31
정상 · 2024-01-01
...
```

상태 = '고장' 으로 '고장' 구역에 점프하면, 그 구역 안이 점검일로 정렬돼 있습니다. 3월 구간만 딱 읽으면 끝입니다.

반대로 (점검일, 상태) 로 만들면 이렇습니다.

```
2024-01-01 · 고장
2024-01-01 · 정상
2024-01-01 · 주의
2024-01-02 · 고장
...
```

3월 구간으로 점프하면 그 안에 고장·정상·주의가 섞여 있습니다. 3월치를 전부 읽고 고장만 골라내야 합니다.

★ 개념04 섹션 4 에서 실제로 재 봤습니다. 색인에서 읽는 쪽 수가 3배쯤 차이 났습니다.

★ 정리하면 순서 정하는 규칙은 셋입니다. ① 등호(=, IN) 조건 칸을 앞에 ② 등호끼리는 값 종류가 많은 칸 (많이 걸러지는 칸)을 앞에 ③ 범위 조건과 ORDER BY 칸은 뒤에

정답 5 — 부분 색인

```
const 답5 = "CREATE INDEX 점검_고장부분 ON 점검기록 (점검일) WHERE 상태 = '고장'";

await 채점(5, "부분 색인", 답5, async (sql) => {
  await db.exec(sql);
  await db.exec("CREATE INDEX 비교용_전체 ON 점검기록 (점검일)");
  await db.exec("ANALYZE 점검기록");

  const 크기 = (
    await db.query("SELECT pg_relation_size('점검_고장부분') AS 부분,
pg_relation_size('비교용_전체') AS 전체")
  ).rows[0];

  const 계획글 = await 계획("SELECT * FROM 점검기록 WHERE 상태 = '고장' AND 점검일 =
DATE '2024-03-01'");

  await db.exec("DROP INDEX 비교용_전체");

  return 계획글.includes("점검_고장부분") && Number(크기.부분) < Number(크기.전체) /
2;
});
```

출력 5. 부분 색인 - 통과

★ 왜 WHERE 상태 = '고장' 이 붙으면 **고장인 줄만** 색인에 들어갑니다. 전체의 3분의 1이니 색인도 3분의 1 크기가 됩니다.

★ 좋은 점 두 가지 ① 작습니다 → 메모리 캐시를 덜 먹고 뒤지는 깊이도 얕습니다 ② **정상·주의 기록을** 넣거나 고칠 때는 이 색인을 안 건드립니다

→ INSERT 가 덜 느려집니다. 전체 색인이라면 매번 써야 합니다

★ 반드시 지켜야 할 것 질의에도 WHERE 상태 = '고장' 이 있어야 씁니다. SELECT * FROM 점검기록 WHERE 점검일 = ... 만 하면 이 색인은 못 씁니다. 플래너가 "이 색인에 그 줄이 들어 있다는 보장이 없다" 고 판단하기 때문입니다.

★ 현장에서 잘 맞는 곳

- 상태가 '진행중' 인 주문만 (끝난 주문은 목록에 안 나옴)
- 삭제표시 = false 인 출만
- 오류 로그만 (전체 로그의 0.1%)

★ MySQL 에는 부분 색인이 없습니다. PostgreSQL 만 있습니다.

정답 6 — 표현식 색인

```
const 답6 = "CREATE INDEX 설비_소문자이름 ON 설비 (lower(이름));

await 채점(6, "표현식 색인", 답6, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 설비");
  const 계획글 = await 계획("SELECT * FROM 설비 WHERE lower(이름) = '설비777'");
  return 계획글.includes("설비_소문자이름");
});
```

출력 6. 표현식 색인 - 통과

★ 왜 그냥 (이름) 색인으로서는 안 되나 색인 안에는 설비777 이라는 **저장된 값 그대로** 들어 있습니다.

lower(설비777) 은 색인 어디에도 없습니다. 색인은 정렬된 목록인데, 함수를 씌우면 그 정렬이 소용없어집니다.

lower(이름) 으로 색인을 만들면 **함수를 씌운 결과가 정렬돼서** 들어갑니다. 그러면 WHERE lower(이름) = ... 을 그대로 찾을 수 있습니다.

★ 반드시 지켜야 할 것: 질의의 표현식과 **글자 그대로** 같아야 합니다

색인 lower(이름) ← 질의 lower(이름)	쓰임
색인 lower(이름) ← 질의 upper(이름)	안 쓰임
색인 lower(이름) ← 질의 lower(trim(이름))	안 쓰임
색인 lower(이름) ← 질의 이름	안 쓰임

★ 함수가 IMMUTABLE 이어야 합니다 같은 입력이면 언제나 같은 출력이 나와야 색인에 담을 수 있습니다.

now() 나 시간대에 따라 달라지는 함수로는 색인을 못 만듭니다. (timestampz 를 date 로 바꾸는 것이 대표적으로 막히는 경우입니다)

★ 쓸 데가 많습니다

```
CREATE INDEX ON 회원 (lower(전자우편));      -- 대소문자 무시 로그인
CREATE INDEX ON 주문 (date_trunc('day', 시각)); -- 날짜별 집계
CREATE INDEX ON 설비 ((설정 ->> '제조사'));   -- JSONB 안의 값
```

정답 7 — 커버링 색인으로 Index Only Scan 만들기

```
const 답7 = "CREATE INDEX 점검_커버 ON 점검기록 (설비번호) INCLUDE (상태)";

await 채점(7, "Index Only Scan 만들기", 답7, async (sql) => {
  await db.exec(sql);
  await db.exec("VACUUM ANALYZE 점검기록");
  const 계획글 = await 계획("SELECT 설비번호, 상태 FROM 점검기록 WHERE 설비번호 = 777", "(ANALYZE)");
  return 계획글.includes("Index Only Scan") && 계획글.includes("Heap Fetches: 0");
});
```

출력 7. Index Only Scan 만들기 - 통과

★ CREATE INDEX 점검_커버 ON 점검기록 (설비번호, 상태) 도 통과합니다. 둘 다 "필요한 칸이 색인 안에 다 있다" 는 조건을 채웁니다.

차이는 이렇습니다.

(설비번호, 상태)	상태로도 **찾을 수** 있습니다. 정렬 순서에 들어갑니다
(설비번호) INCLUDE (상태)	상태는 얹혀만 있습니다. **꺼내는 데만** 씁니다

찾는 조건에 안 쓰는 칸은 INCLUDE 가 낫습니다. 색인이 조금 더 작고, UNIQUE 색인에 없을 때 유일성 판정에 영향을 안 줍니다.

★★ VACUUM 이 핵심입니다. 색인에는 "이 줄이 지금 보여도 되는 줄인지" 가 안 적혀 있습니다. 누가 지웠는데 아직 정리가 안 된 줄일 수도 있으니까요. 그래서 PostgreSQL 은 가시성 지도(visibility map) 를 따로 뱉습니다. 그 지도를 채우는 것이 VACUUM 입니다.

VACUUM 을 안 하면 커버링 색인을 만들어도 Bitmap Heap Scan 이 나옵니다. 개념03 섹션 5 에서 실제로 확인했습니다.

★ Heap Fetches 를 꼭 보세요 Index Only Scan 이라고 나와도 Heap Fetches 가 크면 반쪽짜리입니다. 결국 표를 그만큼 본 것입니다. autovacuum 이 안 따라오고 있다는 뜻입니다.

정답 8 — 형변환 때문에 죽은 색인 살리기

```
const 답8 = "SELECT * FROM 점검기록 WHERE 설비번호 = 777";

await 채점(8, "형변환 때문에 죽은 색인 살리기", 답8, async (sql) => {
  const 계획글 = await 계획(sql);
  const 출수 = (await db.query(sql)).rows.length;
  return !계획글.includes("Seq Scan") && 출수 === 10;
});
```

출력 8. 형변환 때문에 죽은 색인 살리기 - 통과

★ 왜 설비번호::text = '777' 은 색인을 못 쓰나 ::text 는 칸 쪽에 붙은 함수입니다. 정답 6 과 똑같은 함정입니다. 색인에는 정수 777 이 들어 있지 문자열 '777' 이 없습니다.

★ WHERE 설비번호 = '777' 도 통과합니다 이건 값 쪽에 형변환이 붙습니다. PostgreSQL 이 '777' 을 정수 777 로 바꿔 줍니다. 칸은 그대로이니 색인이 살아 있습니다.

★★ 규칙: 형변환은 값 쪽에서 하세요. 칸 쪽에 붙이면 색인이 죽습니다.

```
나쁨: WHERE CAST(설비번호 AS TEXT) = $1
좋은: WHERE 설비번호 = $1::int
```

★ 이게 애플리케이션에서 잘 생깁니다 화면에서 온 값은 전부 문자열입니다. 그걸 그대로 맞추려고 칸을 문자열로 바꾸는 코드가 자연스럽게 나옵니다. **값을 숫자로 바꾸세요. 칸을 문자로 바꾸지 마세요.**

정답 9 — LIKE 앞을 고정하기

```
await db.exec("CREATE INDEX 설비_이름 ON 설비 (이름);");
await db.exec("ANALYZE 설비");

const 답9 = "SELECT * FROM 설비 WHERE 이름 LIKE '설비777%'";

await 채점(9, "LIKE 앞을 고정하기", 답9, async (sql) => {
  const 계획글 = await 계획(sql);
  const 줄수 = (await db.query(sql)).rows.length;
  return 계획글.includes("설비_이름") && 줄수 >= 1;
});
```

출력 9. LIKE 앞을 고정하기 - 통과

★ 왜 앞이 고정돼야 하나 PostgreSQL 은 LIKE '설비777%' 를 보면 범위 조건으로 바꿉니다.

```
이름 >= '설비777' AND 이름 < '설비778'
```

계획의 Index Cond 에 이 모양이 그대로 찍힙니다. 개념05 함정 2 에서 봤습니다. 색인은 정렬돼 있으니 그 구간만 읽으면 됩니다.

'%777%' 은 그렇게 못 바꿉니다. 777 이 앞에 있는지 가운데 있는지 모르니 전부 봐야 합니다.

전화번호부에서 "이름에 철 자가 들어가는 사람" 을 찾는 것과 같습니다.

★ LIKE '설비777' (와일드카드 없음) 도 통과합니다. 그건 그냥 등호와 같습니다.

★ 앞을 못 고정하는 진짜 검색이라면 ① 뒤에서부터 찾는 거면 거꾸로 저장한 칸을 만들고 색인 (reverse(이름)) ② 진짜 전문 검색이면 GIN 색인 + pg_trgm 확장

(PGlite 에는 확장이 없어 실습은 못 합니다)

★ 대소문자 무시까지 하려면 표현식 색인과 합칩니다

```
CREATE INDEX ON 설비 (lower(이름) text_pattern_ops);
WHERE lower(이름) LIKE '설비777%'
```

정답 10 — 두 표에 걸친 OR 고치기

```
const 답10 = `
  SELECT p.id FROM 점검기록 p JOIN 설비 s ON p.설비번호 = s.id WHERE s.이름 =
  '설비77'
  UNION
  SELECT p.id FROM 점검기록 p JOIN 설비 s ON p.설비번호 = s.id WHERE p.점검자 =
  '점검원7';

await 채점(10, "두 표에 걸친 OR 고치기", 답10, async (sql) => {
  const 원래 = (
    await db.query(`
      SELECT p.id FROM 점검기록 p JOIN 설비 s ON p.설비번호 = s.id
      WHERE s.이름 = '설비77' OR p.점검자 = '점검원7'`)
  ).rows.length;

  const 고친것 = (await db.query(sql)).rows.length;
  const 계획글 = await 계획(sql);

  return 원래 === 고친것 && !계획글.includes("Join Filter");
});
```

출력 10. 두 표에 걸친 OR 고치기 - 통과

★ 왜 OR 이 두 표에 걸치면 나쁜가 WHERE s.이름 = ? OR p.점검자 = ? 은 **양쪽 표를 다 봐야** 참인지 알 수 있습니다. 그래서 플래너는 두 표를 전부 이어 붙인 뒤에 Join Filter 로 거릅니다. 개념05 함정 4 에서 Rows Removed by Join Filter: 195990 을 봤습니다. 20만 줄을 이어 붙이고 19만 6천 줄을 버렸습니다.

★ UNION 으로 쪼개면 각 가지가 **한 표의 조건**이 됩니다. 그러면 그 표의 색인을 쓸 수 있습니다. 개념05 에서 재 봤을 때 4~5배 차이가 났습니다.

★★ UNION 과 UNION ALL 중 무엇을 쓰나 OR 은 같은 줄을 두 번 내보내지 않습니다. 두 조건을 다 만족하는 줄이 있으면 UNION ALL 은 그 줄을 **두 번** 냅니다. 그러면 결과가 달라집니다. 그래서 여기서는 **UNION** 을 써야 합니다. 겹치는 줄이 절대 없다고 확신할 때만 UNION ALL 이 낫습니다(중복 제거 비용이 없으니까요).

★ 한 표 안의 OR 은 괜찮습니다 WHERE 설비번호 = ? OR 점검자 = ? 는 양쪽에 색인이 있으면 PostgreSQL 이 BitmapOr 로 둘 다 씁니다. 굳이 UNION 으로 안 바뀌도 됩니다. **“OR 은 무조건 나쁘다” 는 외우지 마세요.**

정답 11 — 안 쓰이는 색인 찾기

```
await db.exec("CREATE INDEX 점검_쓸모없음 ON 점검기록 (비고)");

const 답11 = `
  SELECT relname AS 표, indexrelname AS 색인, pg_relation_size(indexrelid) AS 크기
  FROM pg_stat_user_indexes
```

```
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC`;

await 채점(11, "안 쓰이는 색인 찾기", 답11, async (sql) => {
  const 줄들 = (await db.query(sql)).rows;
  const 칸이름 = Object.keys(줄들[0] ?? {});
  return (
    칸이름.includes("표") && 칸이름.includes("색인") && 칸이름.includes("크기") &&
    줄들.some((줄) => 줄.색인 === "점검_쓸모없음")
  );
});
```

출력 11. 안 쓰이는 색인 찾기 - 통과

★ 운영에서는 여기에 두 가지를 더 붙입니다

SELECT relname AS 표, indexrelname AS 색인, idx_scan AS 사용횟수,

```
pg_size_pretty(pg_relation_size(indexrelid)) AS 크기
```

FROM pg_stat_user_indexes WHERE idx_scan = 0

```
AND indexrelid NOT IN (SELECT conindid FROM pg_constraint WHERE contype IN ('p','u'))
```

```
ORDER BY pg_relation_size(indexrelid) DESC;
```

· `pg_size_pretty` 로 사람이 읽는 단위로 (8912 kB 처럼) · **기본키·UNIQUE 색인은 빼야 합니다.** 사용횟수가 0 이어도 규칙을 지키는 역할이 있습니다

★★ 이 숫자를 믿기 전에 확인할 것 ① `idx_scan` 은 **통계를 마지막으로 초기화한** 뒤부터 셉니다.

```
서버를 다시 켜거나 `pg_stat_reset()` 을 하면 0 부터 시작합니다
```

② EXPLAIN 만 해서는 안 늘어납니다. **진짜로 실행한 질의**만 셉니다 ③ 최소 몇 주는 지켜보세요. 월말 결산에만 쓰는 색인이 있습니다

★ 지울 때는 `DROP INDEX CONCURRENTLY 이름;` 을 쓰세요. 표를 잠그지 않습니다.

정답 12 — N+1 을 두 번으로 줄이기

```

const 답12 = async function 빠른방법() {
  const 설비들 = (await db.query("SELECT id FROM 설비 WHERE 라인 = 'A' LIMIT
50")).rows;

  const 기록들 = (
    await db.query("SELECT 설비번호, id FROM 점검기록 WHERE 설비번호 = ANY($1)", [
      설비들.map((설비) => 설비.id),
    ])
  ).rows;

  const 묶음 = new Map();
  for (const 기록 of 기록들) {
    묶음.set(기록.설비번호, (묶음.get(기록.설비번호) ?? 0) + 1);
  }

  return 설비들.map((설비) => ({ 설비번호: 설비.id, 기록수: 묶음.get(설비.id) ?? 0
})));
};

await 채점(12, "N+1 을 두 번으로 줄이기", 답12, async (함수) => {
  let 왕복 = 0;
  const 원래질의 = db.query.bind(db);
  db.query = (...인자) => { 왕복 += 1; return 원래질의(...인자); };

  let 받은것;
  try {
    받은것 = await 함수();
  } finally {
    db.query = 원래질의;
  }

  const 정답 = (
    await db.query(`
      SELECT s.id AS 설비번호, count(p.id) AS 기록수
      FROM (SELECT id FROM 설비 WHERE 라인 = 'A' LIMIT 50) AS s
      LEFT JOIN 점검기록 p ON p.설비번호 = s.id
      GROUP BY s.id`)
  ).rows;

```

```
const 정답표 = new Map(정답.map((줄) => [Number(줄.설비번호), Number(줄.기록수)]));

return (
  왕복 <= 2 &&
  Array.isArray(받은것) &&
  받은것.length === 50 &&
  받은것.every((줄) => 정답표.get(Number(줄.설비번호)) === Number(줄.기록수))
);
});
```

출력 12. N+1 을 두 번으로 줄이기 - 통과

★ JOIN 한 번으로 푸는 답도 맞습니다

```
const 답12 = async () => (await db.query(`
SELECT s.id AS 설비번호, count(p.id) AS 기록수
FROM (SELECT id FROM 설비 WHERE 라인 = 'A' LIMIT 50) AS s
LEFT JOIN 점검기록 p ON p.설비번호 = s.id
GROUP BY s.id`)).rows;
```

왕복이 **한 번**이라 더 좋습니다. 대신 조인 결과가 커질 수 있습니다. 설비 정보가 기록 수만큼 반복해서 옵니다. (여기서는 count 로 줄여서 괜찮습니다)

★★ = ANY(\$1) 대 IN (...) 결과는 같습니다. 그런데 파라미터로 넘길 때는 = ANY 가 낫습니다. IN 은 개수만큼 \$1, \$2, \$3 ... 을 만들어야 해서 **개수가 바뀔 때마다 질의 글자가 달라집니다**. 그러면 데이터베이스가 계획을 매번 새로 세웁니다. = ANY(\$1) 은 배열 하나를 넘기니 질의 글자가 항상 같습니다.

★ 한 번에 몇 개까지 넘길 수 있나 기술적으로는 아주 많이 됩니다. 그런데 배열이 커지면 계획이 나빠집니다. **1000개쯤에서 나눠 보내는 게 안전합니다**.

★ 왜 N+1 이 무서운가 (개념05 후반부) 같은 기계에서는 5배쯤 차이입니다. “별거 아니네” 싶습니다. 그런데 데이터베이스가 다른 서버에 있으면 왕복마다 몇 ms 가 붙습니다. 왕복 3ms 로 흥내 내 봤더니 **30~50배**가 됐습니다. **질의는 하나도 안 느린데 화면이 느립니다**. 그래서 못 찾습니다.

정답 13 — 진단하기: 왜 느린가

```
const 진단질의 = "SELECT count(*) FROM 점검기록 WHERE 점검자 = '점검원7' AND 소속반 = '반7'";

async function 짐작과실제() {
  const 계획글 = await 계획(진단질의, "(ANALYZE)");
  const 줄 = 계획글.split("\n").find((한줄) => /Scan on/.test(한줄) && /actual/.test(한줄));

  return {
```



```

    짐작: Number(줄.match(/rows=(\d+) width/)[1]),
    실제: Number(줄.match(/rows=(\d.+)+ loops/)[1]),
    계획: 계획글,
  };
}

const 처음 = await 짐작과실제();

console.log(`(참고) 짐작 ${처음.짐작} 줄 / 실제 ${처음.실제} 줄`);

```

출력 (참고) 짐작 79 줄 / 실제 4000 줄
값이 다를 수 있음

```

const 답13 = "추정어긋남";

await 채점(13, "[도전] 왜 느린지 진단하기", 답13, async (원인) => {
  const 어긋남 = 처음.실제 / Math.max(처음.짐작, 1);
  const 훔었나 = 처음.계획.includes("Seq Scan");

  const 진짜원인 = 어긋남 > 10 ? "추정어긋남" : 훔었나 ? "색인없음" : "대부분읽음";
  return 원인.trim() === 진짜원인;
});

```

출력 13. [도전] 왜 느린지 진단하기 - 통과

★ 어떻게 알아내나

계획을 보면 이렇게 나옵니다.

```

Bitmap Heap Scan on "점검기록" (cost=... rows=80 width=0)
    (actual time=... rows=4000.00 loops=1)
    Recheck Cond: ("점검자" = '점검원7'::text)
    Filter: ("소속반" = '반7'::text)

```

rows=80 은 짐작, actual ... rows=4000 은 실제입니다. 50배 어긋났습니다.

★ 왜 어긋나나 플래너는 두 칸을 **따로따로** 보고 곱합니다.

```

점검자 = '점검원7' → 50분의 1
소속반 = '반7' → 50분의 1
둘 다니까 200000 × (1/50) × (1/50) = **80 줄**

```

그런데 이 표에서 점검원7 은 **반드시** 반7 소속입니다. 소속반 조건은 하나도 안 걸러냅니다. 그래서 4000 줄이 그대로 나옵니다.

플래너가 왜 모르나: **칸들이 서로 독립이라고 가정**하기 때문입니다. 현실 표에는 이런 짝이 아주 많습니다. 우편번호-시, 모델명-제조사, 라인-공장.

★ 다른 답이 왜 아닌가

"색인없음" - 점검자에 색인이 있고 실제로 쓰이고 있습니다
"순서틀림" - 복합 색인 자체가 없습니다

"대부분읽음" — $4000 / 200000 = 2\%$ 입니다. 대부분이 아닙니다

★★ 느린 질의를 볼 때 가장 먼저 볼 것이 이 어긋남입니다. 짐작이 틀렸으면 그 위에 세운 계획도 전부 틀립니다. 특히 조인 안쪽에 들어가면 "80번 돌겠지" 가 "4000번 돌기" 가 됩니다.

정답 14 — 짐작을 실제에 맞추기

```
const 답14 = "CREATE STATISTICS 점검_점검자소속반 (dependencies) ON 점검자, 소속반  
FROM 점검기록";  
  
await 채점(14, "[도전] 짐작을 실제에 맞추기", 답14, async (sql) => {  
  await db.exec(sql);  
  await db.exec("ANALYZE 점검기록");  
  
  const 나중 = await 짐작과실제();  
  return 나중.짐작 >= 나중.실제 / 2 && 나중.짐작 <= 나중.실제 * 2;  
});
```

출력 14. [도전] 짐작을 실제에 맞추기 - 통과

★ 확장 통계(extended statistics) 는 PostgreSQL 10 부터입니다.

```
CREATE STATISTICS 이름 (종류) ON 칸1, 칸2 FROM 표;
```

종류는 세 가지입니다.

dependencies - "칸1 을 알면 칸2 도 안다" 는 관계. 이 문제에 딱 맞습니다
ndistinct - 여러 칸을 묶었을 때 값 조합이 몇 가지인가. GROUP BY 추정에 쓰인다
mcv - 자주 나오는 값 **조합**의 목록. 가장 정확하지만 자리를 많이 먹습니다

CREATE STATISTICS 이름 ON 점검자, 소속반 FROM 점검기록 처럼 종류를 빼면 세 가지를 전부 만듭니다.
그것도 통과합니다.

★★ 만든 다음에 ANALYZE 를 꼭 해야 합니다. CREATE STATISTICS 는 "이 관계를 봐 달라" 고 등록만 하는 것입니다. 실제로 값을 채우는 건 ANALYZE 입니다. 안 하면 아무것도 안 바뀝니다.

★ 다른 답: 복합 색인 (점검자, 소속반) 을 만드는 것 이것도 질의는 빨라집니다. 두 조건이 모두 색인 안에서 걸러지니까요. 하지만 짐작 자체는 여전히 어긋납니다. 이 문제의 통과 조건은 못 채웁니다.

★ 둘의 차이를 알아 두세요.

색인 - 그 질의를 빠르게 합니다
확장 통계 - **플래너의 짐작을 고칩니다.** 그 칸들을 쓰는 모든 질의에 영향을 줍니다

짐작이 어긋난 게 원인이라면 색인을 아무리 만들어도 다른 데서 또 터집니다. 특히 조인 방식(Nested Loop 대 Hash Join)이 짐작 하나로 뒤집힙니다.

정답 15 — 정렬을 색인으로 없애기

```
const 답15 = "CREATE INDEX 점검_정렬용 ON 점검기록 (상태, 점검일 DESC)";

await 채점(15, "[도전] Sort 노드 없애기", 답15, async (sql) => {
  await db.exec(sql);
  await db.exec("ANALYZE 점검기록");

  const 계획글 = await 계획(
    "SELECT * FROM 점검기록 WHERE 상태 = '고장' ORDER BY 점검일 DESC LIMIT 20",
  );

  return 계획글.includes("점검_정렬용") && !계획글.includes("Sort");
});
```

출력 15. [도전] Sort 노드 없애기 - 통과

★ 왜 (상태, 점검일 DESC) 인가

① 상태 = '고장' 은 등호 조건 → 앞에 ② ORDER BY 점검일 DESC 는 정렬 → 뒤에

그러면 색인이 이렇게 정렬됩니다.

```
고장 · 2024-12-31
고장 · 2024-12-30
...
고장 · 2024-01-01
정상 · 2024-12-31
...
```

'고장' 구역으로 점프해서 앞에서 20줄만 읽고 멈추면 끝입니다.

★ 색인이 없으면 무슨 일이 생기나 6만 6천 줄을 전부 읽어서 정렬하고, 그중 20줄만 씁니다. 계획에 Sort 노드가 보이고 그 밑에 Seq Scan 이 있습니다. 개념04 섹션 8 에서 재 봤을 때 수백 배 차이가 났습니다.

★★ ORDER BY ... LIMIT 은 색인 효과가 가장 극적으로 나는 곳입니다. 화면에 20줄 보여 주려고 6만 줄을 줄 세우는 게 흔한 낭비입니다.

★ DESC 를 안 붙여도 대개 됩니다 (상태, 점검일) 오름차순 색인이어도 PostgreSQL 은 색인을 거꾸로 읽을 수 있습니다. 계획에 Index Scan Backward 라고 나옵니다.

★ 문제는 방향이 섞일 때입니다.

```
`ORDER BY 상태 ASC, 점검일 DESC` 는 거꾸로 읽기로 해결이 안 됩니다.
이때는 `(상태, 점검일 DESC)` 처럼 **색인에 방향을 박아 넣어야** 합니다.
```

채점 결과

```
const 맞은개수 = 결과.filter(Boolean).length;

console.log(`- ${결과.length}문제 중 ${맞은개수}개 통과 -`);
```

출력 - 15문제 중 15개 통과 -

```
console.log(맞은개수 === 결과.length ? "전부 통과했습니다." : "아직 남았습니다.");
```

출력 전부 통과했습니다.

이 단원에서 꼭 남길 것

① 색인은 읽기를 사고 쓰기를 파는 거래다. 둘 다 재고 정한다 ② Seq Scan 이 나쁜 게 아니다. 필요 없는 Seq Scan 이 나쁘다 ③ 시간보다 읽은 쪽 수(Buffers) 를 믿어라. 기계마다 안 흔들린다 ④ 느린 질의를 볼 때 순서:

왕복 횟수(N+1) → rows 어긋남 → 색인이 쓰이나 → 색인이 있나

⑤ 복합 색인은 왼쪽부터 이어지는 만큼만 쓰인다. 등호 먼저, 범위 나중 ⑥ 칸 쪽에 함수·형변환을 붙이면 색인이 죽는다. 값 쪽에서 해라 ⑦ EXPLAIN ANALYZE 는 진짜로 실행한다. UPDATE/DELETE 는 BEGIN ... ROLLBACK

★ 다음은 07단원입니다. 여러 사람이 동시에 쓸 때 무슨 일이 생기는지 봅니다. PGlite 는 연결이 하나뿐이라 거기서부터는 Docker 로 진짜 PostgreSQL 을 띄웁니다.

```
await db.close();
```

07단원 · 연습문제 정답

07단원 연습문제 정답

트랜잭션과 동시성

`RUN node 연습문제_정답.js`

★ 이 파일은 Docker 가 필요합니다. `docker compose up -d`

★ 먼저 직접 풀어 보고 나서 여기를 보세요. 동시성은 “답을 읽어서” 는 안 익혀집니다. **틀려 봐야** 익혀집니다.

각 문제마다 답 아래에 **왜 그런지**를 길게 적었습니다. 답보다 그 설명이 중요합니다.

```

import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit07-answer",
};

let 가;

try {
  가 = new pg.Client(접속정보);
  await 가.connect();
} catch (에러) {
  console.log("Docker Postgres 에 못 붙었습니다. 먼저 이걸 실행하세요:");
  console.log(" docker compose up -d");
  console.log(` (원인: ${에러.message})`);
  process.exit(0);
}

await 가.query("CREATE SCHEMA IF NOT EXISTS 단원07");
await 가.query("SET search_path TO 단원07");

async function 새연결() {
  const 연결 = new pg.Client(접속정보);
  await 연결.connect();
  await 연결.query("SET search_path TO 단원07");
  return 연결;
}

const 나 = await 새연결();
const 모든연결 = [가, 나];
const 잠깐 = (밀리초) => new Promise((끝) => setTimeout(끝, 밀리초));

```

```

const 한값 = async (문장, 값들 = []) => Object.values((await 가.query(문장,
값들)).rows[0])[0];

await 가.query("DROP TABLE IF EXISTS 답_점검, 답_설비, 답_재고, 답_큐, 답_당직
CASCADE");
await 가.query("CREATE TABLE 답_설비 (번호 INT PRIMARY KEY, 이름 TEXT NOT NULL, 상태
TEXT NOT NULL)");
await 가.query("CREATE TABLE 답_점검 (번호 SERIAL PRIMARY KEY, 설비번호 INT NOT NULL
REFERENCES 답_설비(번호), 내용 TEXT NOT NULL)");
await 가.query("CREATE TABLE 답_재고 (부품번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
수량 INT NOT NULL, 버전 INT NOT NULL DEFAULT 0)");
await 가.query("CREATE TABLE 답_큐 (번호 INT PRIMARY KEY, 내용 TEXT NOT NULL, 상태
TEXT NOT NULL DEFAULT '대기', 처리자 TEXT)");
await 가.query("CREATE TABLE 답_당직 (번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
당직여부 BOOLEAN NOT NULL)");

async function 초기화() {
  for (const 연결 of 모든연결) {
    try {
      await 연결.query("ROLLBACK");
    } catch {

```

열린 트랜잭션이 없으면 넘어갑니다

```

    }
  }

  await 가.query("TRUNCATE 답_점검, 답_설비, 답_재고, 답_큐, 답_당직 RESTART IDENTITY CASCADE");
  await 가.query("INSERT INTO 답_설비 VALUES (1, '컨베이어 1호', '가동'), (2, '프레스 1호', '가동')");
  await 가.query("INSERT INTO 답_점검 (설비번호, 내용) VALUES (1, '벨트 점검'), (2, '유압 점검')");
  await 가.query("INSERT INTO 답_재고 VALUES (1, '베어링', 100, 0), (2, '벨트', 100, 0)");
  await 가.query("INSERT INTO 답_큐 (번호, 내용) SELECT g, '작업' || g FROM generate_series(1, 9) g");
  await 가.query("INSERT INTO 답_당직 VALUES (1, '김반장', true), (2, '이반장', true)");
}

async function 연결여럿(개수, 할일) {
  const 연결들 = [];

  for (let 자리 = 0; 자리 < 개수; 자리 += 1) 연결들.push(await 새연결());

  try {
    return await 할일(연결들);
  } finally {
    for (const 연결 of 연결들) await 연결.end().catch(() => {});
  }
}

let 통과수 = 0;
const 전체문제수 = 15;

async function 채점(번호, 제목, 검사) {
  let 결과 = "실패";

  try {
    await 초기화();
    결과 = (await 검사()) === true ? "통과" : "실패";
  } catch (에러) {
    결과 = `실패 (${에러.code ?? String(에러.message).slice(0, 30)})`;
  }

  if (결과 === "통과") 통과수 += 1;
  console.log(`${String(번호).padStart(2)}번 · ${제목} - ${결과}`);
}

```

정답 1 — 되돌리기


```

async function 문제1(연결) {
  await 연결.query("BEGIN");
  await 연결.query("UPDATE 답_설비 SET 상태 = '폐기' WHERE 번호 = 1");

  const 내가본것 = (await 연결.query("SELECT 상태 FROM 답_설비 WHERE 번호 = 1")).rows[0].상태;

  await 연결.query("ROLLBACK");
  return 내가본것;
}

```

★ 왜 이렇게 되나 UPDATE 직후에 읽으면 '폐기' 입니다. 아직 커밋을 안 했는데도요. **내 트랜잭션 안에서는 내가 한 변경이 보입니다.** 남에게만 안 보입니다. 그래서 "고치고 → 읽고 → 판단" 하는 코드가 트랜잭션 안에서 정상 동작합니다.

★ 되돌리기는 테스트에서 유용합니다. 각 테스트를 BEGIN 으로 시작하고

ROLLBACK 으로 끝내면 정리 코드 없이 매번 깨끗한 상태에서 시작합니다.

정답 2 — 자동 커밋

```

async function 문제2(연결) {

```

BEGIN 을 안 씁니다. 이 UPDATE 는 그 자체로 하나의 트랜잭션입니다.

```

  await 연결.query("UPDATE 답_재고 SET 수량 = 50 WHERE 부품번호 = 1");

  try {
    await 연결.query("SELECT * FROM 없는표"); // 42P01
  } catch {

```

이 에러는 앞의 UPDATE 와 아무 상관이 없습니다

```

  }
}

```

★ 왜 UPDATE 가 살아남나 BEGIN 을 안 썼으므로 UPDATE 는 보내는 순간 시작하고 **끝나는 순간 커밋**됩니다. 그 다음에 난 에러는 이미 끝난 트랜잭션을 되돌릴 수 없습니다.

★★ 개념01 섹션1 의 함정이 이것입니다. "설비 지우고 → 점검기록 지우고 →

대장에 넣고" 를 BEGIN 없이 하면 세 번째가 실패해도 앞의 둘은 이미 커밋돼 있습니다. 반쯤 된 상태가 됩니다.

★ BEGIN 을 썼다면? 에러 뒤로 트랜잭션이 오염돼(25P02) COMMIT 도 롤백됩니다.

****BEGIN 하나로 결과가 정반대가 됩니다.****

정답 3 — 오염된 트랜잭션 되살리기

```
async function 문제3(연결) {  
    await 연결.query("BEGIN");  
  
    try {  
        await 연결.query("SELECT * FROM 없는표");  
    } catch {
```

★ 이 한 줄이 답입니다.

```
        await 연결.query("ROLLBACK");  
    }  
}
```

★ 왜 ROLLBACK 이 필요한가 트랜잭션 안에서 에러가 나면 그 트랜잭션은 **오염(aborted)** 됩니다. 그 뒤로는 SELECT 1 조차 25P02 로 거부됩니다. 오염을 푸는 방법은 딱 둘입니다. ROLLBACK 하거나, ROLLBACK TO SAVEPOINT 하거나. (COMMIT 을 보내도 되지만, 그건 롤백으로 처리됩니다)

★★★ 이걸 안 하면 오염된 연결이 풀로 돌아갑니다. 다음 요청이 그 연결을 받아

****아무 잘못 없이 25P02 로 죽습니다.**** 로그에는 전혀 다른 코드가 찍힙니다.
그래서 트랜잭션은 반드시 감싼 함수로 쓰세요(개념01 섹션7).

정답 4 — SAVEPOINT

```
async function 문제4(연결) {  
    await 연결.query("BEGIN");  
    await 연결.query("INSERT INTO 답_점검 (설비번호, 내용) VALUES (1, '정상 1')");  
  
    await 연결.query("SAVEPOINT 여기까지는_확실");  
  
    try {  
        await 연결.query("INSERT INTO 답_점검 (설비번호, 내용) VALUES (99, '잘못')"); //  
23503  
    } catch {
```

★ 세이브포인트로 되돌리면 오염이 **풀립니다**. 그 뒤로 다시 일할 수 있습니다.

```

await 연결.query("ROLLBACK TO SAVEPOINT 여기까지는_확실");
}

await 연결.query("INSERT INTO 답_점검 (설비번호, 내용) VALUES (2, '정상 2')");
await 연결.query("COMMIT");
}

```

★ 왜 '정상 1'은 살아남나 ROLLBACK TO SAVEPOINT 는 **세이브포인트를 찍은 뒤에 한 일만** 되돌립니다. 그 앞에서 한 '정상 1'은 그대로 남습니다.

★ 정답 3 의 ROLLBACK 과 다른 점

```

ROLLBACK          - 트랜잭션 전체를 버립니다
ROLLBACK TO SP    - 일부만 버리고 트랜잭션은 계속 살립니다

```

★★ 남발하지 마세요. 세이브포인트 하나가 내부 트랜잭션 하나입니다.

"건별 실패를 허용해야 한다" 는 요구가 진짜 있을 때만 쓰세요.

정답 5 — 원자적 UPDATE

```

async function 문제5(연결) {
  await 연결.query("UPDATE 답_재고 SET 수량 = 수량 + 1 WHERE 부품번호 = 1");
}

```

★ 왜 이게 안전한가 값을 자바스크립트로 가져오지 않습니다. **읽고 더하고 쓰는 것을 DB 안에서** 합니다. UPDATE 는 그 줄에 잠금을 걸고, 남은 그 앞에서 기다립니다. 기다렸다 풀리면 **새로 바뀐 값을 다시 읽어서** 더합니다(개념02 섹션4).

★★ 할 수 있으면 무조건 이 방법을 쓰세요. 제일 빠르고 코드도 제일 짧습니다.

조회수, 좋아요, 잔액, 재고 - 대부분 이걸로 끝납니다.

정답 6 — 재고 부족 판정

```

async function 문제6(연결, 빨수량) {
  const 결과 = await 연결.query(
    "UPDATE 답_재고 SET 수량 = 수량 - $1 WHERE 부품번호 = 1 AND 수량 >= $1",
    [빨수량],
  );

  return 결과.rowCount === 1;
}

```

★ 핵심은 두 가지입니다. ① 조건을 **WHERE 안에** 넣었습니다. 확인과 수정이 한 문장입니다.

"SELECT 로 확인하고 UPDATE" 는 그 사이에 남이 다 가져갈 수 있습니다.

② rowCount 를 봤습니다.

조건이 안 맞으면 0줄이 고쳐집니다. **에러가 안 납니다.**
안 보고 넘어가면 재고가 없는데 "출고 완료" 를 답하게 됩니다.

★★ 왜 음수가 안 되나 — 남이 먼저 커밋했으면 내 UPDATE 는 잠금이 풀린 뒤

값을 다시 읽고 `수량 >= \$1` 을 **다시 확인**해서 안 맞으면 건너뛰니다.

정답 7 — 비관적 잠금

```
async function 문제7(연결) {  
  await 연결.query("BEGIN");
```

★ FOR UPDATE 여섯 글자가 전부입니다.

```
const 지금 = (await 연결.query("SELECT 수량 FROM 답_재고 WHERE 부품번호 = 1 FOR  
UPDATE")).rows[0].수량;  
  
await 잠깐(3);  
  
await 연결.query("UPDATE 답_재고 SET 수량 = $1 WHERE 부품번호 = 1", [지금 + 1]);  
await 연결.query("COMMIT");  
}
```

★ FOR UPDATE 가 없으면 열 명이 전부 같은 값을 읽고 같은 값을 씁니다. 붙이면 첫 사람이 커밋할 때까지 나머지 아홉 명이 SELECT 에서 멈춥니다. 깨어난 사람은 **바뀐 값**을 읽습니다.

★★ BEGIN 이 없으면 의미가 없습니다. 자동 커밋이면 SELECT 가 끝나는 순간 풀립니다. ★ 대가: 열 명이 한 줄로 섭니다. 잠깐(3) 이 100ms 라면 열 명이 1초를 씁니다.

그래서 **잠금을 켜 동안에는 최소한의 일만** 하세요.

정답 8 — 낙관적 잠금

```

async function 문제8(연결) {
  for (let 시도 = 1; 시도 <= 100; 시도 += 1) {
    const 읽은것 = (await 연결.query("SELECT 수량, 버전 FROM 답_재고 WHERE 부품번호 = 1")).rows[0];

    const 결과 = await 연결.query(
      "UPDATE 답_재고 SET 수량 = $1, 버전 = 버전 + 1 WHERE 부품번호 = 1 AND 버전 = $2",
      [읽은것.수량 + 1, 읽은것.버전],
    );

    if (결과.rowCount === 1) return 시도; // 성공
  }
}

```

rowCount 가 0 이면 내가 읽은 뒤 남이 먼저 고친 것입니다. 다시 읽고 다시 합니다.

```

}

throw new Error("100번 시도했는데 실패");
}

```

★ 버전 칸이 하는 일 "내가 읽었을 때의 버전이 아직 그대로일 때만 써라" 는 조건입니다. 남이 먼저 썼으면 버전이 올라가 있으므로 내 UPDATE 는 0줄을 고칩니다. 그러면 처음부터 다시 합니다.

★ 잠그지 않으니 기다림이 없습니다. 대신 **헛일**이 생깁니다.

개념02 섹션8 에서 20명이 한 줄에 몰렸을 때 헛시도가 130회였습니다.

★★ 언제 쓰나 — 충돌이 드물 때, 그리고 **화면이 오래 열려 있을 때**.

수정 폼을 열고 5분 뒤에 저장하는 경우를 생각해 보세요. FOR UPDATE 로 5분을 잠글 수는 없습니다. 버전 칸이면 잠그지 않고도 "그사이 다른 사용자가 수정했습니다" 를 알려 줍니다.

★ 수정시각(updated_at)으로도 하지만, 같은 밀리초에 두 번 고치면 못 잡습니다.

정답 9 — 반복 불가 읽기 막기

```

async function 문제9(연결, 사이에할일) {
  await 연결.query("BEGIN ISOLATION LEVEL REPEATABLE READ");

  const 첫번째 = (await 연결.query("SELECT 수량 FROM 답_재고 WHERE 부품번호 = 1")).rows[0].수량;
  await 사이에할일();
  const 두번째 = (await 연결.query("SELECT 수량 FROM 답_재고 WHERE 부품번호 = 1")).rows[0].수량;

  await 연결.query("COMMIT");
  return { 첫번째, 두번째 };
}

```

★ 왜 REPEATABLE READ 인가 기본값 READ COMMITTED 는 **문장마다** 새로 사진을 찍습니다. 그래서 문장 사이에 남이 커밋하면 다음 문장은 새 값을 봅니다.

REPEATABLE READ 는 **트랜잭션의 첫 문장에서 찍은 사진 한 장**을 끝까지 봅니다. 남이 뭘 하든 내 사진은 안 바뀝니다.

★ 사진은 BEGIN 이 아니라 **첫 문장**에서 찍힙니다(개념03 섹션8). ★★ 대가: 내가 본 줄을 남이 먼저 고치면 내 UPDATE 가 40001 로 거부됩니다.

읽기만 하면 괜찮지만 쓰기까지 한다면 재시도 코드가 필요합니다(정답 14).

★ 언제 쓰나: 월말 정산, 대사 작업, 여러 표를 한 시점 기준으로 읽는 보고서.

정답 10 — 작업 큐

```

async function 문제10(연결, 일꾼이름) {
  await 연결.query("BEGIN");

  const 집기 = await 연결.query(`
    SELECT 번호 FROM 답_큐
    WHERE 상태 = '대기'
    ORDER BY 번호
    LIMIT 1
    FOR UPDATE SKIP LOCKED
  `);

  if (집기.rowCount === 0) {
    await 연결.query("COMMIT");
    return null;
  }

  const 번호 = 집기.rows[0].번호;
  await 잠깐(5);
  await 연결.query("UPDATE 답_큐 SET 상태 = '완료', 처리자 = $1 WHERE 번호 = $2",

```

```
[일꾼이름, 번호]);
    await 연결.query("COMMIT");
    return 번호;
}
```

★ FOR UPDATE 와 SKIP LOCKED 가 각각 하는 일이 다릅니다. FOR UPDATE → 내가 집은 줄을 남이 못 집게 잠급니다 (중복 처리를 막습니다) SKIP LOCKED → 남이 잡은 줄은 건너뛰고 다음 것을 줍니다 (기다리지 않습니다)

★ 둘 중 하나만 쓰면 이렇게 됩니다.

아무것도 안 쓰면 → 세 명이 같은 1번을 집습니다. 세 번 처리됩니다
FOR UPDATE 만 → 세 명이 1번 앞에 줄을 섭니다. 결과는 맞지만 한 명씩만 일합니다

★★ 잠금은 COMMIT 할 때 풀립니다. 처리와 상태 변경이 같은 트랜잭션 안에 있어야

"처리는 했는데 상태를 못 바꾼" 상태가 안 생깁니다.

★ 일꾼이 중간에 죽으면 롤백되어 그 작업은 '대기' 로 돌아옵니다. 좋은 일입니다.

대신 **같은 일을 두 번 해도 괜찮게** 만들어 주세요.

정답 11 — 기다리지 않기

```
async function 문제11(연결) {
    await 연결.query("BEGIN");

    try {
```

★ NOWAIT: 잠겨 있으면 기다리지 말고 바로 55P03 을 내려

```
await 연결.query("SELECT * FROM 답_재고 WHERE 부품번호 = 1 FOR UPDATE NOWAIT");
    await 연결.query("ROLLBACK");
    return "잡음";
} catch {
    await 연결.query("ROLLBACK");
    return "못잡음";
}
}
```

★ 방법이 두 가지 있습니다. ① FOR UPDATE NOWAIT — 잠겨 있으면 즉시 55P03 ② SET lock_timeout = '100ms' — 100ms 만 기다리고 55P03

★ NOWAIT 은 0초, lock_timeout 은 "조금은 기다려 준다" 입니다.

대부분의 대기는 짧게 끝나므로 실무에서는 lock_timeout 이 더 쓸모 있습니다.

★★ 언제 필요한가: 손님을 기다리게 하면 안 되는 화면,

운영 중 ALTER TABLE, 이번엔 건너뛰어도 되는 배치.

정답 12 — ★ 동시성 버그 찾아 고치기

```
async function 문제12(연결, 주문수) {  
  await 연결.query("BEGIN");
```

★ 확인과 차감을 한 문장으로 합칩니다.

```
const 결과 = await 연결.query(  
  "UPDATE 답_재고 SET 수량 = 수량 - $1 WHERE 부품번호 = 1 AND 수량 >= $1",  
  [주문수],  
);  
  
if (결과.rowCount === 0) {  
  await 연결.query("ROLLBACK");  
  throw new Error("재고 부족");  
}  
  
await 잠깐(10); // 배송지 확인 같은 다른 일 await 연결.query("COMMIT");  
}
```

★★★ 원래 코드의 버그가 무엇이었나

const 지금 = SELECT 수량 ← 두 사람 다 100 을 읽습니다 if (지금 < 주문수) 실패 ← 두 사람 다 "100 >= 60, 괜찮네" 로 통과합니다 UPDATE 수량 = 지금 - 주문수 ← 두 사람 다 40 을 씁니다

두 명이 60씩 가져갔는데 재고는 40 입니다. **60개가 공짜로 나갔습니다.** 더 나쁜 경우: 세 번째 사람이 끼면 -20 이 됩니다. 재고가 음수가 됩니다.

★ 트랜잭션이 왜 못 막았나

둘 다 자기 트랜잭션은 완벽하게 끝냈습니다. 트랜잭션은 아무 잘못이 없습니다.
문제는 **읽은 값이 쓸 때까지 그대로일 거라고 믿은 것** 입니다.
이게 개념02 의 핵심입니다.

★★ 고치는 방법은 셋 다 됩니다. 상황에 따라 고르세요.

- ① 위 정답처럼 한 문장으로 (제일 좋음)
`UPDATE ... WHERE 수량 >= $1 → rowCount` 로 판정
 읽기가 아예 없으니 틈이 없습니다.
- ② `FOR UPDATE` 로 읽을 때부터 잠그기
`const 지금 = SELECT 수량 ... FOR UPDATE`
 읽은 값으로 복잡한 계산을 해야 할 때 씁니다.
- ③ 버전 칸으로 낙관적 잠금 + 재시도
 충돌이 드문 곳에 씁니다. 재고는 충돌이 잦으니 여기엔 안 맞습니다.

★ 찾는 요령: 코드에서 **SELECT** 와 **UPDATE** 사이를 보세요.

"읽어서 → 판단해서 → 쓴다" 가 보이면 일단 의심하세요.
 혼자 테스트하면 절대 안 잡힙니다. 반드시 두 연결로 겹쳐서 시험하세요.

정답 13 — [도전] 데드락 안 나게

```
async function 문제13(연결, 부품번호들) {
  await 연결.query("BEGIN");
```

★ 이 한 줄이 답입니다. 잠그기 전에 **정렬**합니다.

```
const 순서대로 = [...부품번호들].sort((하나, 둘) => 하나 - 둘);

for (const 번호 of 순서대로) {
  await 연결.query("UPDATE 답_재고 SET 수량 = 수량 - 1 WHERE 부품번호 = $1",
[번호]);
  await 잠깐(60);
}

await 연결.query("COMMIT");
}
```

★ 왜 정렬만으로 데드락이 사라지나 데드락은 **잡그는 순서가 서로 다를 때만** 생깁니다(개념05 섹션2).
 모두가 작은 번호부터 잡으면 고리가 안 만들어집니다.

두 번째 사람은 1번에서 기다립니다. 첫 번째가 커밋하면 1번, 2번을 차례로 잡습니다. **기다림은 있지만 서로 기다리지는 않습니다.**

★★ [...부품번호들] 로 복사한 것에 주의하세요.

`sort()` 는 원본 배열을 ****제자리에서**** 바꿉니다.
 부르는 쪽의 배열을 몰래 바꿔 버리면 나중에 이상한 버그가 납니다.

★ 순서를 맞추는 방법: 항상 기본키 오름차순, 표에도 순서 정하기,

한 문장으로 잠그기(WHERE 번호 IN (...) ORDER BY 번호 FOR UPDATE).

★★★ 그래도 데드락은 완전히 사라지지 않습니다.

외래키, 트리거, UNIQUE INSERT 로도 납니다(개념05 섹션5).
그래서 정답 14 가 반드시 함께 있어야 합니다.

정답 14 — [도전] 재시도 함수

```
const 다시해볼만한코드 = new Set(["40001", "40P01"]);

async function 문제14(연결, 할일, 최대시도 = 20) {
  let 마지막에러;

  for (let 시도 = 1; 시도 <= 최대시도; 시도 += 1) {
    try {
      await 연결.query("BEGIN");
      await 할일(연결);
      await 연결.query("COMMIT");
      return { 시도 };
    } catch (에러) {
      마지막에러 = 에러;
    }
  }
}
```

★ 반드시 롤백합니다. 안 하면 이 연결이 오염된 채 남습니다.

```
try {
  await 연결.query("ROLLBACK");
} catch {
```

서버가 이미 롤백했으면 할 일이 없습니다

```
}
```

★ 다시 해도 소용없는 에러는 바로 던집니다.

```
if (!다시해볼만한코드.has(에러.code)) throw 에러;
```

★★ 점점 오래 기다리고(백오프), 무작위로 흔들니다(지터).

```
await 잠깐(Math.min(200, 10 * 2 ** (시도 - 1)) + Math.random() * 20);
}
}

throw 마지막에러;
}
```

★ 40001 과 40P01 을 왜 같이 묶나 둘 다 “네 잘못이 아니다. 지금 남과 부딪혔을 뿐이다” 라는 뜻입니다. 그래서 **다시 하면 대개 성공합니다**. 상대가 이미 끝났으니까요.

40001 직렬화 실패 – REPEATABLE READ / SERIALIZABLE 에서 남과 충돌
40P01 데드락 – 서로 기다리다 한쪽이 죽음

★★ 재시도 코드에서 절대 빠뜨리면 안 되는 다섯 가지

① 트랜잭션 **전체**를 처음부터 다시 하세요.

실패한 문장만 다시 하면 안 됩니다. 앞 문장들이 이미 롤백됐습니다.

② 재시도해도 되는 에러만 하세요.

23505(중복키)를 재시도하면 백 번 해도 실패합니다. 응답만 느려집니다.

③ 반드시 ROLLBACK 하세요. 안 하면 연결이 오염된 채 풀로 돌아갑니다.

④ 상한을 두세요. 실무에서는 3~5회면 충분합니다.

(이 문제는 여섯 명이 일부러 어긋나게 만든 최악의 상황이라 20 을 줍니다)

⑤ 기다리는 시간을 늘리고 **흔드세요**.

똑같이 기다리면 둘이 동시에 다시 시작해서 또 부딪힙니다.
Math.random() 을 빼고 돌려 보면 평균 시도 횟수가 늘어납니다.

★★★ 그리고 하나 더 — 트랜잭션 안에 **되돌릴 수 없는 일**을 넣지 마세요. 메일 발송이 할일() 안에 있으면 재시도할 때마다 메일이 또 갑니다. DB 롤백은 메일을 회수해 주지 않습니다. 그런 일은 커밋에 성공한 **뒤에** 트랜잭션 밖에서 하세요.

정답 15 — [도전] 쓰기 왜곡 막기

```
async function 문제15(연결, 내번호) {
  await 연결.query("BEGIN ISOLATION LEVEL SERIALIZABLE");

  const 당직수 = Number((await 연결.query("SELECT count(*) FROM 답_당직 WHERE
  당직여부")).rows[0].count);

  if (당직수 <= 1) {
    await 연결.query("ROLLBACK");
    throw new Error("당직이 한 명뿐입니다");
  }

  await 잠깐(30);
  await 연결.query("UPDATE 답_당직 SET 당직여부 = false WHERE 번호 = $1", [내번호]);
}
```

08단원 연습문제 정답 애플리케이션과 잇기

```
RUN node 연습문제_정답.js
```

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

정답과 왜 그런지를 같이 적었습니다.

★ 답만 베끼면 남는 게 없습니다. 먼저 연습문제.js 를 스스로 풀고, 막힌 것만 여기서 보세요. 특히 10번·11번(버그 찾기)은 설명이 깁니다. 실무에서 가장 자주 나는 사고입니다.

```

import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import pg from "pg";

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434,
  user: "factory",
  password: "secret",
  database: "factory_db",
};

let 준비;

try {
  준비 = new pg.Client(접속정보);
  await 준비.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

await 준비.query("DROP SCHEMA IF EXISTS 단원08_연습 CASCADE");
await 준비.query("CREATE SCHEMA 단원08_연습");
await 준비.query("SET search_path TO 단원08_연습");
await 준비.query(`
  CREATE TABLE 설비 (
    설비번호 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인      TEXT NOT NULL,
    상태      TEXT NOT NULL DEFAULT '정지',
    도입가    NUMERIC(12, 2) NOT NULL DEFAULT 0,
    설치일    DATE NOT NULL DEFAULT DATE '2026-01-01'
  );
`);

```

```

    )
  `);
  await 준비.query(` CREATE TABLE 점검기록 (
    번호      SERIAL PRIMARY KEY,
    설비번호 TEXT NOT NULL REFERENCES 설비(설비번호),
    결과      TEXT NOT NULL
  )
  `);
  await 준비.query(` INSERT INTO 설비 (설비번호, 이름, 라인, 상태, 도입가, 설치일)
VALUES
  ('EQ-001', '컨베이어 1호', 'A', '가동', 1200000.50, DATE '2026-03-02'),
  ('EQ-002', '프레스 1호', 'B', '정지', 3400000.25, DATE '2026-04-11'),
  ('EQ-003', '용접로봇 1호', 'A', '가동', 980000.25, DATE '2026-05-20')
  `);
  await 준비.query(` INSERT INTO 점검기록 (설비번호, 결과)
  SELECT 'EQ-' || lpad(설비::text, 3, '0'), '정상' FROM generate_series(1, 3) AS
설비, generate_series(1, 4) AS 회차
  `);
  await 준비.query("CREATE TABLE 잔고 (계정 TEXT PRIMARY KEY, 값 INT NOT NULL)");
  await 준비.query("INSERT INTO 잔고 VALUES ('A라인', 100), ('B라인', 100)");

  const 풀 = new pg.Pool({ ...접속정보, max: 5, options: "-c search_path=단원08_연습"
  });

```

채점 도구 (고치지 마세요)

```

const 채점결과 = [];

async function 문제(번호, 제목, 검사) {
  try {
    채점결과.push(`${(await 검사()) ? "✓" : "x"} ${번호}. ${제목}`);
  } catch (에러) {
    채점결과.push(`x ${번호}. ${제목} (${에러.constructor.name})`);
  }
}

```

풀에서 빌려 간 연결을 추적합니다. 학생 코드가 새어도 파일이 끝나게 하려고요.

```

function 감시풀(설정) {
  const 새풀 = new pg.Pool(설정);
  const 빌려준것 = new Set();
  const 원래connect = 새풀.connect.bind(새풀);

  새풀.connect = async (...인자들) => {
    const 연결 = await 원래connect(...인자들);
    const 원래release = 연결.release.bind(연결);

```

```

빌려준것.add(연결);
    연결.release = (...것들) => {
        빌려준것.delete(연결);
        return 원래release(...것들);
    };

    return 연결;
};

새품.강제정리 = () => {
    for (const 연결 of 빌려준것) {
        try {
            연결.release();
        } catch {

```

이미 돌아간 것입니다

```

    }
}

빌려준것.clear();
};

return 새품;
}

```

1. 접속 설정 객체 만들기

docker-compose.yml 의 postgres 는 이렇게 떠 있습니다. 포트 5434 · 계정 factory · 비밀번호 secret · DB factory_db

그 정보로 pg.Client 설정 객체를 만드세요. host 는 "127.0.0.1" 입니다.

```

function 설정만들기() {
    return {
        host: "127.0.0.1",
        port: 5434,           // * 5432 가 아닙니다 user: "factory",
        password: "secret",
        database: "factory_db",
    };
}

```

왜 _____

port 를 안 적으면 pg 는 **5432** 로 붙으려 합니다. 우리 컨테이너는 5434 입니다. 그러면 **ECONNREFUSED** 가 납니다. (이미 5432 에 다른 Postgres 가 떠 있으면 더 나쁩니다. **영똥한 데이터베이스**에 붙어서 표가 없다고 나옵니다)

★ 실무에서는 이 값을 코드에 적지 않습니다. `process.env.DATABASE_URL` 로 빼고, `.env` 는 `.gitignore` 에 넣습니다. 여기서는 연습이라 그대로 적었습니다.

```
await 문제(1, "접속 설정 객체 만들기", async () => {
  const 설정 = 설정만들기();
  if (설정 === null) return false;

  const 시험 = new pg.Client(설정);
  await 시험.connect();
  const 값 = (await 시험.query("SELECT 1 AS 값")).rows[0].값;
  await 시험.end();

  return 값 === 1;
});
```

2. 연결 문자열 만들기

설정 객체를 `postgres://사용자:비밀번호@호스트:포트/DB이름` 으로 바꾸세요. ★ 비밀번호에 `@` 나 `/` 가 들어갈 수 있습니다. `encodeURIComponent` 를 쓰세요.

```
function 연결문자열만들기({ host, port, user, password, database }) {
  const 안전한비번 = encodeURIComponent(password);

  return `postgres://${user}:${안전한비번}@${host}:${port}/${database}`;
}
```

왜 _____

`encodeURIComponent` 가 핵심입니다.

비밀번호가 `a@b/c` 인데 그대로 이으면 이렇게 됩니다.

`postgres://factory:a@b/c@127.0.0.1:5434/factory_db`

URL 파서는 첫 번째 `@` 까지를 계정 부분으로 봅니다. 그래서 호스트가 `b`, 경로가 `/c@127.0.0.1:5434/factory_db` 가 됩니다. `ENOTFOUND b` 같은 엉뚱한 에러가 나고, 원인을 찾는 데 한참 걸립니다.

★ 운영에서 비밀번호를 자동 생성하면 특수문자가 섞여 나옵니다. 그날 배포가 터집니다. 그래서 이 한 줄이 필요합니다.

★ 사용자 이름에도 특수문자가 들어갈 수 있습니다. 엄밀하게는 그쪽도 감싸야 합니다.


```
await 문제(2, "연결 문자열 만들기", () => {
  const 보통 = 연결문자열만들기(접속정보);
  const 특수 = 연결문자열만들기({ ...접속정보, password: "a@b/c" });

  return (
    보통 === "postgres://factory:secret@127.0.0.1:5434/factory_db" &&
    특수 === "postgres://factory:a%40b%2Fc@127.0.0.1:5434/factory_db"
  );
});
```

3. 비밀번호 가리기

로그에 연결 문자열을 찍을 때 비밀번호만 **** 로 바꾸세요.

postgres://factory:secret@127.0.0.1:5434/factory_db →

postgres://factory:****@127.0.0.1:5434/factory_db

```
function 비밀가리기(문자열) {
  return 문자열.replace(/:\w\/([\^:]+):(.*)@/, "://$1:****@");
}
```

왜 —————

두 번째 그룹이 (.*?) 인 것을 눈여겨보세요. ([^@]*) 로 쓰면 비밀번호에 @ 가 들어간 두 번째 시험 (kim:p@ss@db...)에서 틀립니다.

.* 는 가장 긴 것을 먹으므로 마지막 @ 까지 갑니다. 연결 문자열에서 마지막 @ 뒤가 호스트이므로 이게 맞습니다.

★ 왜 이런 걸 만드나 서버가 뜰 때 “어디에 붙었는지” 를 로그로 남기는 것은 좋은 습관입니다. 그런데 연결 문자열을 그대로 찍으면 **비밀번호가 로그 파일에 영원히 남습니다**. 로그는 대개 여러 사람이 보고, 외부 로그 서비스로도 나갑니다.

★ 에러 객체를 통째로 찍는 것도 같은 이유로 위험합니다. `에러.code` 와 `에러.message` 만 찍으세요.

```
await 문제(3, "비밀번호 가리기", () =>
  비밀가리기("postgres://factory:secret@127.0.0.1:5434/factory_db") ===
    "postgres://factory:****@127.0.0.1:5434/factory_db" &&
  비밀가리기("postgres://kim:p@ss@db.example.com:5432/prod") ===
    "postgres://kim:****@db.example.com:5432/prod",
);
```

4. 연결 에러를 사람 말로 바꾸기

연결에 실패했을 때 나오는 코드를 보고 한국어 안내를 돌려주세요.

ECONNREFUSED → “데이터베이스가 안 떠 있거나 포트가 틀립니다”

28P01	→ "계정이나 비밀번호가 틀립니다"
3D000	→ "그 이름의 데이터베이스가 없습니다"

그 밖에 → "알 수 없는 오류입니다"

```
const 연결에러표 = {
  ECONNREFUSED: "데이터베이스가 안 떠 있거나 포트가 틀립니다",
  "28P01": "계정이나 비밀번호가 틀립니다",
  "3D000": "그 이름의 데이터베이스가 없습니다",
};

function 연결에러설명(코드) {
  return 연결에러표[코드] ?? "알 수 없는 오류입니다";
}
```

왜 —————

if 를 늘어놓아도 됩니다. 표로 두면 코드를 추가하기가 쉽습니다.

★★ 중요한 것은 두 종류의 코드가 섞여 있다는 점입니다.

ECONNREFUSED / ENOTFOUND / ETIMEDOUT ← Node 가 낸 것. DB 까지 못 갔습니다 28P01 / 3D000 / 42601 ← Postgres 가 낸 것. DB 는 만났습니다

이 구분이 진단의 절반입니다. 앞쪽이면 **네트워크·컨테이너**를 보고, 뒤쪽이면 **계정·DB 이름·SQL** 을 봅니다.

★ 없는 계정으로 붙어도 3D000 이 아니라 28P01 이 납니다. 계정이 있는지 없는지를 밖에서 알아내지 못하게 일부러 그렇게 만든 것입니다.

```
await 문제(4, "연결 에러를 사람 말로", () =>
  연결에러설명("ECONNREFUSED") === "데이터베이스가 안 떠 있거나 포트가 틀립니다" &&
  연결에러설명("28P01") === "계정이나 비밀번호가 틀립니다" &&
  연결에러설명("3D000") === "그 이름의 데이터베이스가 없습니다" &&
  연결에러설명("42601") === "알 수 없는 오류입니다",
);
```

5. NUMERIC 을 제대로 더하기

도입가는 NUMERIC 이라 자바스크립트에 **문자열**로 옵니다. 그냥 더하면 글자가 이어 붙습니다.

설비 세 대의 도입가 합계를 **숫자**로 돌려주세요. (정답: 5580001)

```
async function 도입가합계(실행자) {
```

방법 ①: DB 에서 더하고 숫자로 바꿔 받습니다 (권장)

```
const 결과 = await 실행자.query("SELECT sum(도입가) AS 합 FROM 설비");

return Number(결과.rows[0].합);
}
```

방법 ②: 자바스크립트에서 더한다면 반드시 Number 로 바꾼 뒤에 더합니다.

```
const 줄들 = (await 실행자.query("SELECT 도입가 FROM 설비")).rows;
return 줄들.reduce((합, 줄) => 합 + Number(줄.도입가), 0);
```

왜 _____

NUMERIC 은 pg 에서 **문자열**로 옵니다. sum() 의 결과도 문자열입니다. (sum(NUMERIC) 은 NUMERIC 이니까요)

그래서 Number(...) 를 안 붙이면 이렇게 됩니다.

줄들.reduce((합, 줄) => 합 + 줄.도입가, 0) → 0 + "1200000.50" = "01200000.50" →
"01200000.50" + "3400000.25" = "01200000.503400000.25"

예러가 안 납니다. 화면에 이상한 숫자가 나올 뿐입니다. 금액이라면 몇 달 뒤에 발견됩니다.

★ 왜 문자열로 주나 NUMERIC 은 자릿수 제한이 없습니다. 자바스크립트 number 로는 못 담습니다.

드라이버가 마음대로 바꾸면 **조용히 값이 망가집니다.** 그래서 판단을 넘긴 것입니다.

★★ 돈이라면 Number 도 완전하지 않습니다. (0.1 + 0.2 !== 0.3) 가장 안전한 것은 **DB 안에서 NUMERIC 으로 계산**해서 결과만 받는 것입니다. 자바스크립트에서 다뤄야 한다면 원 단위 정수로 바꿔서 다루세요.

```
await 문제(5, "NUMERIC 을 제대로 더하기", async () => {
  const 합 = await 도입가합계(풀);
  return typeof 합 === "number" && Math.abs(합 - 5580001) < 0.001;
});
```

6. DATE 를 날짜 글자로 (시간대 함정 피하기)

EQ-001 의 설치일은 2026-03-02 입니다. 그런데 설치일.toString().slice(0, 10) 을 하면
한국에서는 2026-03-01 이 나옵니다.

"2026-03-02" 를 정확히 돌려주세요. ★ 힌트: SQL 쪽에서 글자로 만들어 받으면 시간대를 안 탑니다.

```

async function 설치일글자(실행자, 설비번호) {
  const 결과 = await 실행자.query(
    "SELECT to_char(설치일, 'YYYY-MM-DD') AS 날 FROM 설비 WHERE 설비번호 = $1",
    [설비번호],
  );

  return 결과.rows.length === 0 ? null : 결과.rows[0].날;
}

```

왜 —————

pg 는 DATE 를 그 기계의 로컬 자정으로 만든 Date 로 줍니다.

DB 값 : 2026-03-02 JS 값 (한국) : Mon Mar 02 2026 00:00:00 GMT+0900 toISOString():
2026-03-01T15:00:00.000Z ← ★ 하루 밀렸습니다

toISOString().slice(0, 10) 은 그래서 **2026-03-01** 을 줍니다.

★★ 최악인 점은 **UTC 서버에서는 안 밀린다**는 것입니다. 한국 노트북에서만 틀리고 CI 에서는 통과합니다.
재현이 어렵습니다. 시간대가 UTC 서쪽(미국 등)이면 하루 **뒤로** 밀립니다.

★ 해결책 세 가지 ① to_char(칸, 'YYYY-MM-DD') 로 SQL 에서 글자를 만듭니다 — 가장 확실합니다 ②
파서를 바꿉니다 — pg.types.setTypeParser(1082, (글자) => 글자) ③ 로컬 기준으로 직접 조립합니다
— getFullYear()/getMonth()+1/getDate()

★ **TIMESTAMPZ** 는 이 문제가 없습니다. 시각까지 정확히 옵니다. 문제가 되는 것은 **시각이 없는 DATE**
입니다.

```

await 문제(6, "DATE 를 날짜 글자로", async () =>
  (await 설치일글자(풀, "EQ-001")) === "2026-03-02" &&
  (await 설치일글자(풀, "EQ-002")) === "2026-04-11",
);

```

7. 저장소 — 하나 조회하기

설비번호로 한 대를 찾습니다. · 있으면 { 번호, 이름, 라인 } 을 돌려주세요 (칸 이름을 바꿉니다) · ★ 없으면
null 을 돌려주세요. undefined 가 아닙니다

```

async function 설비하나(실행자, 설비번호) {
  const 결과 = await 실행자.query(
    "SELECT 설비번호, 이름, 라인 FROM 설비 WHERE 설비번호 = $1",
    [설비번호],
  );

  if (결과.rows.length === 0) return null;

  const 줄 = 결과.rows[0];

  return { 번호: 줄.설비번호, 이름: 줄.이름, 라인: 줄.라인 };
}

```

왜 —————

① `rows[0]` 을 그냥 돌려주면 안 됩니다. 없을 때 `undefined` 가 나오는데, `JSON.stringify` 는 `undefined` 인 키를 통째로 빼 버립니다.

```

JSON.stringify({ 설비: undefined }) → "{}"
JSON.stringify({ 설비: null })      → '{"설비":null}'

```

앞은 “그런 걸 물어본 적 없다” 처럼 읽히고, 뒤는 “찾아봤는데 없다” 입니다. API 를 쓰는 쪽에서는 완전히 다른 뜻입니다. `null` 로 통일하세요.

② ****SELECT *** 를 쓰지 않았습니다.** 지금은 칸이 여섯 개인데 셋만 골랐습니다. 나중에 비밀번호해시 같은 칸이 추가돼도 **아무 일도 안 일어납니다**. **SELECT *** 였다면 그날부터 API 응답에 딸려 나갑니다.

③ **칸 이름을 바꿔서 내보냈습니다**. (**설비번호** → **번호**) DB 의 칸 이름이 앱 전체로 새어 나가지 않습니다. 나중에 칸 이름을 바꿔도 **이 함수만** 고치면 됩니다.

```

await 문제(7, "저장소 - 하나 조회하기", async () => {
  const 있는것 = await 설비하나(풀, "EQ-001");
  const 없는것 = await 설비하나(풀, "EQ-999");

  return (
    없는것 === null &&
    있는것 !== null &&
    있는것.번호 === "EQ-001" &&
    있는것.이름 === "컨베이어 1호" &&
    있는것.라인 === "A" &&
    Object.keys(있는것).length === 3
  );
});

```

8. 저장소 — 없는 줄을 고쳤을 때

상태를 바꿉니다. · 바꿨으면 바뀐 설비 객체 { 번호, 상태 } 를 돌려주세요 · ★ 그 설비가 없으면 `null` 을 돌려주세요 (rowCount 를 보세요)

이걸 안 하면 “수정했습니다” 라고 200 을 주고 화면은 안 바뀝니다.

```
async function 상태바꾸기(실행자, 설비번호, 새상태) {
  const 결과 = await 실행자.query(
    "UPDATE 설비 SET 상태 = $2 WHERE 설비번호 = $1 RETURNING 설비번호, 상태",
    [설비번호, 새상태],
  );

  if (결과.rowCount === 0) return null;

  return { 번호: 결과.rows[0].설비번호, 상태: 결과.rows[0].상태 };
}
```

왜

★★ 없는 줄을 UPDATE 해도 에러가 안 납니다. rowCount 만 0 입니다.

그래서 확인하지 않으면 이렇게 됩니다.

```
PATCH /equipments/EQ-999 → 200 OK {"메시지":"수정되었습니다"}
```

사용자는 성공했다고 믿고, 화면은 안 바뀝니다. “저장이 안 돼요” 라는 문의가 들어오는데 로그에는 에러가 없습니다.

rowCount === 0 이면 null 을 돌려주고, 라우트에서 404 를 주세요.

★ RETURNING 을 쓰면 **고친 뒤의 값**을 한 번에 받습니다. UPDATE 하고 다시 SELECT 하면 쿼리가 두 번이고, 그 사이에 남이 또 고칠 수도 있습니다. (07단원의 경험) RETURNING 은 MySQL 에는 없습니다. Postgres 의 무기입니다.

★ rowCount 가 2 이상이면 **WHERE 가 잘못된 것**입니다. 기본키로 찾는데 2가 나오면 그때는 던지는 편이 낫습니다.

```

await 문제(8, "저장소 - 없는 줄을 고쳤을 때", async () => {
  const 바뀐것 = await 상태바꾸기(풀, "EQ-002", "가동");
  const 없는것 = await 상태바꾸기(풀, "EQ-999", "가동");

  await 풀.query("UPDATE 설비 SET 상태 = '정지' WHERE 설비번호 = 'EQ-002'");

  return 없는것 === null && 바뀐것 !== null && 바뀐것.번호 === "EQ-002" && 바뀐것.
    상태 === "가동";
});

```

9. 에러 코드를 도메인 말로

이미 있는 설비번호를 또 넣으면 23505 가 납니다. 그 에러를 잡아서 `Error("이미 있는 설비번호입니다")`로 바꿔 던지세요. ★ 다른 코드의 에러는 **손대지 말고 그대로** 던져야 합니다.

```

async function 설비추가(실행자, { 번호, 이름, 라인 }) {
  try {
    const 결과 = await 실행자.query(
      "INSERT INTO 설비 (설비번호, 이름, 라인) VALUES ($1, $2, $3) RETURNING 설비번호",
      [번호, 이름, 라인],
    );

    return { 번호: 결과.rows[0].설비번호 };
  } catch (에러) {
    if (에러.code === "23505") {
      throw new Error("이미 있는 설비번호입니다");
    }

    throw 에러; // * 모르는 에러는 손대지 말고 그대로 올립니다
  }
}

```

왜 —————

① 낱것 메시지를 사용자에게 보여 주면 안 됩니다.

```
duplicate key value violates unique constraint "설비_pkey"
```

사용자는 못 알아듣고, 공격자는 표 이름과 제약 이름을 알게 됩니다. 그리고 나중에 제약 이름을 바꾸면 화면 문구가 같이 바뀝니다.

② ★★ 모르는 에러를 삼키면 안 됩니다. `catch` 안에서 무조건 “이미 있는 설비번호입니다”를 던지면, 이름이 NULL 이라 난 23502 도 “이미 있는 설비번호”가 됩니다. 그럼 아무도 진짜 원인을 못 찾습니다. 이 문제의 세 번째 검사가 그것입니다.

조건에 안 맞으면 **원래 에러를 그대로** 던지세요.

③ ★★★ 미리 **SELECT** 로 확인하면 안 됩니다.

```
const 있나 = await 실행자.query("SELECT 1 FROM 설비 WHERE 설비번호 = $1", [번호]);
if (있나.rowCount > 0) throw new Error("이미 있는 설비번호입니다");
await 실행자.query("INSERT ...");
```

확인과 INSERT **사이에** 남이 넣을 수 있습니다. 그러면 INSERT 가 터집니다. 혼자 테스트하면 절대 안 납니다. 사람이 몰릴 때만 납니다. 23505 를 잡는 쪽이 **유일하게 정확한** 방법입니다. (07단원의 경험)

```
await 문제(9, "에러 코드를 도메인 말로", async () => {
  const 새것 = await 설비추가(풀, { 번호: "EQ-100", 이름: "새설비", 라인: "C" });

  const 중복 = await 설비추가(풀, { 번호: "EQ-100", 이름: "또새설비", 라인: "C" })
    .then(() => "성공")
    .catch((에러) => 에러.message);

  const 낫널 = await 설비추가(풀, { 번호: "EQ-101", 이름: null, 라인: "C" })
    .then(() => "성공")
    .catch((에러) => 에러.code);

  await 풀.query("DELETE FROM 설비 WHERE 설비번호 IN ('EQ-100', 'EQ-101')");

  return 새것 !== null && 중복 === "이미 있는 설비번호입니다" && 낫널 === "23502";
});
```

10. ★ 버그 찾기 — 풀이 마릅니다

아래 코드는 잘 돌아가는 것처럼 보이는데, 몇 번 부르면 서비스가 멈춥니다.

```
async function 설비수세기(풀) {

  const 연결 = await 풀.connect();
  const 결과 = await 연결.query("SELECT count(*)::int AS 건수 FROM 설비");
  연결.release();
  return 결과.rows[0].건수;

}
```

무엇이 문제인지 찾아서 고친 것을 아래에 쓰세요. ★ 힌트: 쿼리가 **실패하면** 어떻게 되나요?


```

async function 설비수세기(풀, SQL = "SELECT count(*)::int AS 건수 FROM 설비") {
  const 연결 = await 풀.connect();

  try {
    return (await 연결.query(SQL)).rows[0].건수;
  } finally {
    연결.release(); // * 성공하든 던지든 반드시 돌아갑니다
  }
}

```

왜 _____

원래 코드의 문제는 딱 한 줄입니다.

```

const 결과 = await 연결.query(...); // ← 여기서 던지면
연결.release();                      // ← 이 줄에 못 옵니다

```

정상 경로에만 `release()` 가 있습니다.

★★★ 이게 운영에서 어떻게 보이는지가 중요합니다.

· 배포 직후에는 멀쩡합니다. 며칠 잘 돕니다 · 어느 날 갑자기 **모든 요청이 멈춥니다** · CPU 도 메모리도 한가합니다. DB 도 한가합니다 · 재시작하면 잠깐 괜찮아졌다가 또 멈춥니다 · 로그에는 `timeout exceeded when trying to connect` 만 쌓입니다

원인은 “에러가 몇 번 났다” 입니다. 에러가 `max` 번 나면 풀이 완전히 마릅니다. `max: 10` 이면 **딱 10번**의 실패로 서비스 전체가 멈춥니다.

★ 그리고 `풀.end()` 도 안 끝납니다. 빌려준 연결이 다 돌아올 때까지 기다리기 때문입니다. “테스트가 안 끝나요” 의 흔한 원인입니다.

★★ 더 나은 답: **한 문장이면 `풀.query(...)` 를 쓰세요.**

```

return (await 풀.query(SQL)).rows[0].건수;

```

빌리고 돌려주는 것을 풀이 알아서 합니다. 빠뜨릴 수가 없습니다. `풀.connect()` 는 **같은 연결이 꼭 필요할 때만 쓰세요** (트랜잭션 등). (이 문제는 `finally` 연습이라 `connect()` 로 풀었습니다)

```

await 문제(10, "버그 찾기 - 풀이 마릅니다", async () => {
  const 작은풀 = 감시풀({ ...접속정보, max: 2, connectionTimeoutMillis: 1000,
    options: "-c search_path=단원08_연습" });

  try {

```

★ 일부러 세 번 실패시킵니다. 그래도 연결이 돌아와야 합니다.

```

for (let 회차 = 0; 회차 < 3; 회차 += 1) {
    await 설비수세기(작은풀, "SELEC 1").catch(() => {});
}

const 건수 = await 설비수세기(작은풀);

return 건수 === 3 && 작은풀.idleCount > 0;
} finally {
    작은풀.강제정리();
    await 작은풀.end();
}
});

```

11. ★ 버그 찾기 — 롤백이 안 됩니다

아래 코드는 테스트에서는 잘 되는데 운영에서 가끔 돈이 사라집니다.

```

async function 이체(풀, 보내는곳, 받는곳, 금액) {

```

```

    await 풀.query("BEGIN");
    await 풀.query("UPDATE 잔고 SET 값 = 값 - $2 WHERE 계정 = $1", [보내는곳, 금액]);
    await 풀.query("UPDATE 잔고 SET 값 = 값 + $2 WHERE 계정 = $1", [받는곳, 금액]);
    await 풀.query("COMMIT");

```

```

}

```

무엇이 문제인지 찾아서 고치세요. ★ 중간에 에러가 나면 **아무것도 안 바뀌어야** 합니다.

```

async function 이체(풀, 보내는곳, 받는곳, 금액) {
    const 연결 = await 풀.connect(); // * 연결을 하나 빌려서 try {
        await 연결.query("BEGIN"); // * 그 연결에만 BEGIN 을 보냅니다 await
        연결.query("UPDATE 잔고 SET 값 = 값 - $2 WHERE 계정 = $1", [보내는곳, 금액]);

        if (금액 <= 0) throw new Error("금액이 잘못됐습니다");

        await 연결.query("UPDATE 잔고 SET 값 = 값 + $2 WHERE 계정 = $1", [받는곳, 금액]);
        await 연결.query("COMMIT");
    } catch (에러) {
        await 연결.query("ROLLBACK");
        throw 에러;
    } finally {
        연결.release();
    }
}

```

왜 _____

★★★ 풀.query 는 매번 다른 연결을 쓸 수 있습니다.

트랜잭션은 **연결에 붙어 있는 상태**입니다. BEGIN 을 A 연결에 보내고 UPDATE 를 B 연결에 보내면, B 는 트랜잭션 밖이라 **그 자리에서 커밋**됩니다.

풀.query("BEGIN") → A 연결 풀.query("UPDATE ... 빼기") → B 연결 ★ 곧바로 커밋됩니다

풀.query("UPDATE ... 더하기") → A 연결 풀.query("COMMIT") → C 연결 ★ 트랜잭션도 아닌데 커밋

중간에 터지면 **뺀 것만 남고 더한 것은 없습니다.** 돈이 사라집니다.

★★ 최악인 점은 **부하가 없으면 안 난다**는 것입니다. 혼자 테스트하면 요청이 안 겹치니 풀이 늘 같은 연결을 줍니다. 잘 됩니다. 손님이 몰리는 순간에만 다른 연결이 잡혀서 깨집니다. 그래서 테스트로는 절대 못 잡습니다.

★ 개념02 에서 이걸 실제로 깨 봤습니다. 롤백했는데 값이 999 로 남았습니다.

★ throw 를 UPDATE 뒤에 둔 것을 눈여겨보세요. 이미 한 번 UPDATE 한 뒤에 던져도 ROLLBACK 이 그것까지 되돌립니다. 그게 원자성입니다.

★ 실무에서는 이 껍데기를 함수 하나로 만들어 두고 **그것만 씁니다.** BEGIN 을 손으로 치는 곳이 코드에 남아 있으면 언젠가 이 사고가 납니다. (12번 문제가 그 껍데기를 만드는 것입니다)

```
await 문제(11, "버그 찾기 - 롤백이 안 됩니다", async () => {
  await 풀.query("UPDATE 잔고 SET 값 = 100");

  await 이체(풀, "A라인", "B라인", 30);

  const 정상뒤 = (await 풀.query("SELECT 계정, 값 FROM 잔고 ORDER BY 계정")).rows;
```

실패해야 하는 이체 — 금액이 0 이하입니다

```
const 실패 = await 이체(풀, "A라인", "B라인", -50).then(() => "성공").catch(() => "실패");

const 실패뒤 = (await 풀.query("SELECT 계정, 값 FROM 잔고 ORDER BY 계정")).rows;

return (
  정상뒤[0].값 === 70 && 정상뒤[1].값 === 130 &&
  실패 === "실패" &&
  실패뒤[0].값 === 70 && 실패뒤[1].값 === 130
);
});
```

12. 여러 저장소를 한 트랜잭션으로 묶기

설비를 등록하면서 첫 점검기록도 같이 남깁니다. ★ 둘 다 성공하거나 둘 다 안 되어야 합니다. ★ 저장소 함수에는 **빌린 연결**을 넘겨야 합니다.

```
async function 트랜잭션으로(풀, 할일) {
  const 연결 = await 풀.connect();

  try {
    await 연결.query("BEGIN");
    const 값 = await 할일(연결);    // ★ 빌린 연결을 그대로 넘깁니다 await
    연결.query("COMMIT");
    return 값;
  } catch (에러) {
    await 연결.query("ROLLBACK");
    throw 에러;
  } finally {
    연결.release();
  }
}

async function 등록하고첫점검(풀, 설비, 첫결과) {
  return 트랜잭션으로(풀, async (연결) => {
```

★ 두 저장소 함수 모두에 **같은 연결**을 넘깁니다

```
const 만든것 = await 설비추가(연결, 설비);

await 연결.query(
  "INSERT INTO 점검기록 (설비번호, 결과) VALUES ($1, $2)",
  [만든것.번호, 첫결과],
);

return 만든것;
});
}
```

왜 _____

★★ 저장소 함수의 **첫 인자**를 **실행자**로 만들어 둔 것이 여기서 값을 합니다.

설비추가(풀, ...) → 풀이 알아서 아무 연결이나 씁니다 (트랜잭션 밖) 설비추가(연결, ...) → 우리가 준 연결을 씁니다 (트랜잭션 안)

같은 함수인데 부르는 쪽이 정합니다. 저장소는 트랜잭션을 **모릅니다**.

★★★ 만약 **설비추가(풀, ...)** 라고 쓰면 이렇게 됩니다. 설비 INSERT 는 풀의 다른 연결에서 **곧바로 커밋**되고, 점검기록 INSERT 만 트랜잭션 안에서 롤백됩니다.

출력 ****설비만 남습니다.**** 이 문제의 두 번째 검사가 그것을 봅니다.

★ 트랜잭션 경계는 **업무 한 건**입니다. SQL 한 줄이 아닙니다. 그래서 저장소가 아니라 **서비스 층**이 시작합니다.

층 하는 일 트랜잭션

라우트	HTTP 를 받고 응답을 만들	모릅니다
서비스	업무 규칙. 여러 저장소를 엮음	★ 여기서 시작
저장소	SQL 한 덩어리	받아서 씁니다

★ 저장소 함수 안에서 **BEGIN** 을 하면 안 됩니다. 묶을 수가 없어집니다.

```
await 문제(12, "여러 저장소를 한 트랜잭션으로", async () => {
  await 등록하고첫점검(풀, { 번호: "EQ-200", 이름: "포장기", 라인: "C" }, "정상");

  const 좋은쪽 = (await 풀.query(
    "SELECT count(*)::int AS 건수 FROM 점검기록 WHERE 설비번호 = 'EQ-200'",
  )).rows[0].건수;
```

점검기록의 결과는 NOT NULL 입니다. 여기서 터져야 합니다.

```
await 등록하고첫점검(풀, { 번호: "EQ-201", 이름: "포장기2", 라인: "C" },
null).catch(() => {});

const 남았나 = (await 풀.query(
  "SELECT count(*)::int AS 건수 FROM 설비 WHERE 설비번호 = 'EQ-201'",
)).rows[0].건수;

await 풀.query("DELETE FROM 점검기록 WHERE 설비번호 = 'EQ-200'");
await 풀.query("DELETE FROM 설비 WHERE 설비번호 IN ('EQ-200', 'EQ-201')");

return 좋은쪽 === 1 && 남았나 === 0;
});
```

13. [도전] 테스트를 롤백으로 뒷정리하기

시험 하나를 트랜잭션으로 감싸고, **성공해도 롤백**하는 껍데기를 만드세요. 시험이 넣은 데이터가 하나도 안 남아야 합니다.

· 시험이 통과하면 **true**, 던지면 **false** 를 돌려주세요 · 어느 쪽이든 **ROLLBACK** 하고 연결을 돌려주세요

```
async function 시험하기(풀, 할일) {
  const 연결 = await 풀.connect();
```

09단원 연습문제 정답 MySQL과 무엇이 다른가

RUN node 연습문제_정답.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

정답과 **왜 그런지**를 함께 적었습니다. 답만 베끼지 말고 설명을 읽으세요. 이 단원의 값어치는 설명에 있습니다.

★ 답이 하나가 아닌 문제도 있습니다. 다른 방법도 함께 적어 두었습니다.

```
import pg from "pg";  
import mysql from "mysql2/promise";
```

준비

```

const 피지설정 = { host: "127.0.0.1", port: 5434, user: "factory", password:
"secret", database: "factory_db" };
const 마이설정 = { host: "127.0.0.1", port: 3307, user: "factory", password:
"secret", database: "factory_db" };

let PG = null;
let MY = null;
let MY2 = null;

try {
  PG = new pg.Client(피지설정);
  await PG.connect();
  MY = await mysql.createConnection(마이설정);
  MY2 = await mysql.createConnection(마이설정);
} catch (에러) {
  console.log("* 데이터베이스에 못 붙었습니다:", 에러.code || 에러.message);
  console.log(" docker compose up -d 를 먼저 실행하세요.");
  if (PG) await PG.end().catch(() => {});
  if (MY) await MY.end().catch(() => {});
  process.exit(0);
}

await PG.query("CREATE SCHEMA IF NOT EXISTS 단원09");
await PG.query("SET search_path TO 단원09");

async function 피지(sql, 값 = []) {
  const 결과 = await PG.query(sql, 값);
  return 결과.rows;
}

async function 마이(sql, 값 = []) {
  const [행들] = await MY.query(sql, 값);
  return 행들;
}

async function 채점(번호, 제목, 답, 검사) {
  let 통과 = false;

```

```

try {
    통과 = (await 검사(답)) === true;
} catch (에러) {
    통과 = false;
}

console.log(` 문제 ${번호} - ${통과 ? "✅ 정답" : "❌ 다시"} · ${제목}`);
}

console.log("=== 09단원 연습문제 정답 ===");

```

출력 === 09단원 연습문제 정답 ===

문제 1 — MySQL 서버가 엄격한지 검사하기

```

const 답1 = (모드) => 모드.includes("STRICT_TRANS_TABLES") &&
모드.includes("ONLY_FULL_GROUP_BY");

await 채점(1, "엄격한 sql_mode 인지 검사", 답1, (답) => {
    return (
        답("IGNORE_SPACE,ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_DATE") === true
        &&
        답("NO_ENGINE_SUBSTITUTION") === false &&
        답("STRICT_TRANS_TABLES") === false
    );
});

```

출력 문제 1 - ✅ 정답 · 엄격한 sql_mode 인지 검사

★ 왜 이 검사가 중요한가

개념02 에서 재 봤습니다. MySQL 8.4 는 기본 설정에서 PostgreSQL 만큼 엄격합니다. 그런데 그 엄격함은 **버전의 성질이 아니라 설정값**입니다. `SET SESSION sql_mode='NO_ENGINE_SUBSTITUTION'` 한 줄이면 옛날로 돌아갑니다.

그래서 MySQL 서버를 처음 만나면 버전이 아니라 **sql_mode** 를 봐야 합니다. 옛 버전에서 올린 서버는 옛 설정을 그대로 들고 옵니다.

★ 두 개만 봤는데, 실무에서는 이것도 같이 봅니다.

```

NO_ZERO_DATE, NO_ZERO_IN_DATE - '0000-00-00' 을 막습니다
ERROR_FOR_DIVISION_BY_ZERO - INSERT 안에서 0 나누기를 경고합니다

```

★★ 그리고 **접속마다 다를 수 있습니다.** `@GLOBAL.sql_mode` 가 아니라 `@SESSION.sql_mode` 를 보세요. 드라이버나 프레임워크가 붙을 때 바꾸는 경우가 있습니다.

문제 2 — || 를 양쪽에서 통하게 고치기


```
const 답2 = "SELECT concat('설비', 'A') AS 결과";

await 채점(2, "양쪽에서 글자 잇기", 답2, async (답) => {
  const 가 = await 피지(답);
  const 나 = await 마이(답);
  return 가[0].결과 === "설비A" && 나[0].결과 === "설비A";
});
```

출력 문제 2 -  정답 · 양쪽에서 글자 잇기

★ 왜 `||` 를 쓰면 안 되나

표준 SQL 에서 `||` 는 문자열 연결입니다. PostgreSQL 은 표준을 따릅니다. MySQL 은 `||` 를 **OR** 로 씁니다. C 언어 습관에서 온 것입니다.

‘설비’ `||` ‘A’ → MySQL 은 ‘설비’ OR ‘A’ 로 읽습니다. 글자를 숫자로 바꾸면 둘 다 0 이라 결과가 0 입니다.

★★★ 예러가 안 납니다. 이게 핵심입니다. 옮긴 날에는 아무도 모릅니다. 화면에 설비 이름 대신 0 이 뜬 뒤에야 압니다.

★ 다른 방법도 있습니다. 그런데 권하지 않습니다.

```
SET SESSION sql_mode = CONCAT(@@SESSION.sql_mode, ',PIPES_AS_CONCAT');
```

이러면 MySQL 에서도 `||` 가 연결이 됩니다. 하지만

- 접속마다 설정해야 하고
- 이미 ``||`` 를 OR 로 쓰던 옛 코드가 망가집니다

운영 중인 서버에서는 못 겁니다.

★★ NULL 이 섞이면 또 걸립니다. 이것도 외워 두세요.

PostgreSQL 의	<code>concat('A', NULL, 'B')</code>	→ 'AB' (NULL 을 빈 글자로 봅니다)
MySQL 의	<code>CONCAT('A', NULL, 'B')</code>	→ NULL (전체가 NULL 이 됩니다)

→ NULL 이 들어올 수 있으면 `COALESCE(칸, '')` 로 감싸세요.

문제 3 — 글자 수를 양쪽에서 똑같이 세기

```
const 답3 = "SELECT char_length('설비') AS 개수";

await 채점(3, "글자 수 세기", 답3, async (답) => {
  const 가 = await 피지(답);
  const 나 = await 마이(답);
  return Number(가[0].개수) === 3 && Number(나[0].개수) === 3;
});
```

출력 문제 3 - ☒ 정답 · 글자 수 세기

★ 같은 이름의 함수인데 뜻이 다릅니다

PostgreSQL 의 length() = 글자 수 → '설비' 은 3
MySQL 의 LENGTH() = 바이트 수 → '설비' 은 10

한글 한 자는 UTF-8 에서 3바이트, 이 이모지는 4바이트라서 3+3+4 = 10 입니다.

★ 반대편에 해당하는 함수도 있습니다.

PostgreSQL 에서 바이트를 세려면 → octet_length()
MySQL 에서 글자를 세려면 → CHAR_LENGTH()

char_length() 는 양쪽에 다 있습니다. 그래서 이걸 쓰면 됩니다.

★★ 이게 왜 사고가 되나 "제목이 20자 넘으면 자르기" 같은 코드를 생각해 보세요. MySQL 로 옮기면 한글 제목이 7자에서 잘립니다. 에러는 없습니다. 그냥 화면이 이상해집니다.

문제 4 — NULL 을 맨 뒤로 보내기

```
const 답4 =
  "SELECT x FROM (SELECT 1 AS x UNION ALL SELECT NULL UNION ALL SELECT 3) AS t ORDER
  BY (x IS NULL), x";

await 채점(4, "NULL 을 맨 뒤로", 답4, async (답) => {
  const 바른순서 = (행들) => 행들.length === 3 && 행들[0].x === 1 && 행들[1].x === 3
  && 행들[2].x === null;
  return 바른순서(await 피지(답)) && 바른순서(await 마이(답));
});
```

출력 문제 4 - ☒ 정답 · NULL 을 맨 뒤로

★ 기본 자리가 반대입니다

PostgreSQL - NULL 을 ****가장 큰 값****으로 봅니다 → 오름차순에서 맨 뒤
 MySQL - NULL 을 ****가장 작은 값****으로 봅니다 → 오름차순에서 맨 앞

표준 SQL 은 “구현이 정하라” 고만 해 놨습니다. 그래서 둘 다 틀린 게 아닙니다.

★ PostgreSQL 에는 ORDER BY x NULLS LAST 문법이 있습니다. **MySQL 에는 없습니다.** 1064 문법 오류가 납니다.

★★ 그래서 ORDER BY (x IS NULL), x 를 씁니다. x IS NULL 은 참/거짓입니다. 거짓(0)이 먼저 오므로 값이 있는 줄이 앞에 옵니다. 그 안에서 다시 x 로 정렬됩니다. **양쪽에서 똑같이 동작합니다.**

★ NULL 을 맨 앞으로 보내고 싶으면 ORDER BY (x IS NOT NULL), x 입니다.

★★ 실무에서 어디서 터지나 “마지막 점검일” 처럼 NULL 이 있는 칸으로 정렬하는 목록 화면입니다. 미점검 설비가 맨 위에 있다가 맨 아래로 내려갑니다. 페이지를 나눠서 보여 주면 **1페이지 내용이 통째로 바뀝니다.**

문제 5 — 0 으로 나누기를 안전하게

```
const 답5 = "SELECT COALESCE(100 / NULLIF(0, 0), 0) AS 가동률";

await 채점(5, "0 으로 나누기 막기", 답5, async (답) => {
  const 가 = await 피지(답);
  const 나 = await 마이(답);
  return Number(가[0].가동률) === 0 && Number(나[0].가동률) === 0;
});
```

출력 문제 5 - ☒ 정답 · 0 으로 나누기 막기

★ 두 DB 의 철학이 정확히 갈리는 자리입니다

PostgreSQL - "그건 계산할 수 없습니다" → 22012 로 ****멈춥니다****
 MySQL - "값이 없네요" → ****NULL**** 을 주고 계속 갑니다

어느 쪽도 우리가 원하는 답은 아닙니다. 화면에는 “0%” 라고 뜨는 게 맞지, 에러 페이지나 빈 칸이 맞는 게 아닙니다.

★ 그래서 **직접 막습니다.**

NULLIF(a, b) - a 와 b 가 같으면 NULL, 아니면 a
 COALESCE(...) - 앞에서부터 NULL 이 아닌 첫 값

100 / NULLIF(0,0) → 나누는 수가 NULL 이 되어 결과가 NULL (에러 없음) COALESCE(NULL, 0) → 0

두 함수 다 **양쪽에 있습니다.** 표준 함수입니다.

★★ 결과가 자바스크립트로 올 때 모양이 다릅니다. 조심하세요.

```
PostgreSQL → 0      (숫자)
MySQL      → "0.0000" (글자)
```

MySQL 의 / 는 결과를 DECIMAL 로 만들고, 드라이버는 DECIMAL 을 글자로 줍니다. 그래서 채점 코드에서 `Number(...)` 로 감쌌습니다. ★ 계산에 쓰기 전에 반드시 숫자로 바꾸세요. “0.0000” + 1 은 “0.00001” 입니다.

문제 6 — CREATE TABLE 을 MySQL 용으로 바꾸기

```
const 답6 = `
  CREATE TABLE 구구_문제6 (
    id INT AUTO_INCREMENT PRIMARY KEY,
    이름 VARCHAR(50) NOT NULL,
    가동 BOOLEAN DEFAULT true,
    메모 JSON
  )
`;

await 채점(6, "CREATE TABLE 을 MySQL 용으로", 답6, async (답) => {
  await 마이("DROP TABLE IF EXISTS 구구_문제6");
  await 마이(답);

  const 칸들 = await 마이(
    "SELECT COLUMN_NAME AS 이름, DATA_TYPE AS 타입, IS_NULLABLE AS 널허용, EXTRA AS
    덤 " +
    "FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='factory_db' AND
    TABLE_NAME='구구_문제6'",
  );
  const 찾기 = (이름) => 칸들.find((칸) => 칸.이름 === 이름);

  return (
    칸들.length === 4 &&
    찾기("id").덤 === "auto_increment" &&
    찾기("이름").널허용 === "NO" &&
    찾기("메모").타입 === "json"
  );
});
```

출력 문제 6 - ☒ 정답 · CREATE TABLE 을 MySQL 용으로

★ 바꾼 곳을 하나씩 봅니다

① SERIAL PRIMARY KEY → INT AUTO_INCREMENT PRIMARY KEY

★★ MySQL 에도 ``SERIAL`` 이라는 말이 **있습니다.** 그런데 뜻이 다릅니다.
 MySQL 의 `SERIAL = `BIGINT UNSIGNED NOT NULL AUTO_INCREMENT UNIQUE``
 기본키가 아니라 `UNIQUE` 이고, `INT` 가 아니라 `BIGINT` 입니다.
 "돌긴 도는데 다른 표가 만들어지는" 최악의 경우입니다. 조심하세요.

② JSONB → JSON

MySQL 에 `JSONB` 는 없습니다. `JSON` 하나뿐입니다.
 * 이름만 바꾸면 되는 게 아닙니다. **색인 방법이 다릅니다.**
 PostgreSQL 은 ``USING GIN (칸)`` 으로 통째로 색인합니다.
 MySQL 은 `JSON` 칸에 직접 색인을 못 겁니다(3152 예러).
 자주 찾는 키마다 생성 칸을 만들고 거기에 색인을 겁니다.

③ BOOLEAN 은 그대로 써도 됩니다. 그런데 진짜 BOOLEAN 이 아닙니다.

MySQL 이 만드는 실제 타입은 ``TINYINT(1)`` 입니다.
 그래서 5 나 -1 도 들어갑니다. 그리고 그 값은 ``= true`` 로 안 잡힙니다.
 * ``WHERE 가동`` 이나 ``WHERE 가동 <> 0`` 을 쓰세요.
 * 더 좋은 방법: ``가동 TINYINT(1) NOT NULL DEFAULT 1 CHECK (가동 IN (0,1))``

★★ 그리고 실무에서는 문자셋도 명시합니다 (문제 11 참고).

```
... ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
```

문제 7 — 파라미터 기호를 바꿔 주는 함수

```
const 답7 = (sql) => sql.replace(/\$d+/g, "?");

await 채점(7, "$1 을 ? 로 바꾸기", 답7, (답) => {
  return (
    답("SELECT * FROM 설비 WHERE 라인=$1 AND 가동=$2") === "SELECT * FROM 설비 WHERE
라인=? AND 가동=?" &&
    답("INSERT INTO 점검(설비,내용) VALUES ($1,$2)") === "INSERT INTO 점검(설비,내용)
VALUES (?,?)" &&
    답("SELECT $10, $2") === "SELECT ?, ?"
  );
});
```

출력 문제 7 -  정답 · \$1 을 ? 로 바꾸기

★ `\d+` 를 쓴 이유 `\$d` 라고만 쓰면 `$10` 이 `?0` 이 됩니다. 파라미터가 10개를 넘는 쿼리는 흔합니다. + 를 붙여서 숫자를 끝까지 먹게 해야 합니다.

★★★ 그런데 이 함수만으로는 부족합니다. 가장 중요한 함정이 남아 있습니다.

PostgreSQL 의 `$1` 은 번호가 있어서 같은 값을 여러 번 쓸 수 있습니다.

```
SELECT * FROM 설비 WHERE 이름 = $1 OR 별칭 = $1
→ 값은 한 개만 넘깁니다: [이름]
```

이걸 기계적으로 바꾸면

```
SELECT * FROM 설비 WHERE 이름 = ? OR 별칭 = ?
→ 값은 **두 개**를 넘겨야 합니다: [이름, 이름]
```

SQL 만 바꾸고 값 배열을 안 고치면 실행할 때 터집니다. ★ 그래서 진짜 변환 도구는 값 배열도 같이 만들어 줘야 합니다.

```
function 바꾸기(sql, 값들) {
  const 새값 = [];
  const 새sql = sql.replace(/\$(\d+)/g, (_, 번호) => {
    새값.push(값들[Number(번호) - 1]);
    return "?";
  });
  return { sql: 새sql, 값: 새값 };
}
```

★ 또 하나: 글자 안에 든 \$1 도 바뀝니다. SELECT ‘가격은 \$100 입니다’ 가 ‘가격은 ?00 입니다’ 가 됩니다. 정규식으로 SQL 을 다루는 도구의 한계입니다. 사람이 한 번 봐야 합니다.

문제 8 — RETURNING 없이 방금 넣은 id 받기

```
const 답8 = async (연결, 내용) => {
  const [결과] = await 연결.query("INSERT INTO 구구_문제8(내용) VALUES (?)", [내용]);
  return 결과.insertId;
};

await 채점(8, "MySQL 에서 새 id 받기", 답8, async (답) => {
  await 마이("DROP TABLE IF EXISTS 구구_문제8");
  await 마이("CREATE TABLE 구구_문제8 (id INT AUTO_INCREMENT PRIMARY KEY, 내용 VARCHAR(30))");

  const 첫번째 = await 답(MY, "첫 점검");
  const 두번째 = await 답(MY, "둘째 점검");

  const 확인 = await 마이("SELECT id, 내용 FROM 구구_문제8 ORDER BY id");

  return (
    Number(첫번째) === 확인[0].id &&
    Number(두번째) === 확인[1].id &&
  );
});
```

```
확인[0].내용 === "첫 점검" &&
  확인[1].내용 === "둘째 점검"
);
});
```

출력 문제 8 -  정답 · MySQL 에서 새 id 받기

★ SQL 로 받는 방법도 있습니다. 결과는 같습니다.

```
await 연결.query("INSERT INTO 구구_문제8(내용) VALUES (?)", [내용]);
const [행] = await 연결.query("SELECT LAST_INSERT_ID() AS id");
return 행[0].id;
```

쿼리를 한 번 더 보내는 것뿐이라 insertId 를 쓰는 편이 낫습니다.

★ LAST_INSERT_ID() 는 **접속마다 따로** 기억합니다. 그 사이에 다른 사람이 INSERT 해도 내 값은 안 바뀝니다. 그건 안전합니다.

★★★ 진짜 위험한 곳은 **여러 건을 한 번에 넣을 때**입니다.

```
INSERT INTO 구구_점검(내용) VALUES ('여섯'),('일곱'),('여덟');
→ insertId 는 **첫 번째 번호 하나**만 줍니다. 나머지는 안 알려 줍니다.
```

“연속으로 붙었을 테니 +1, +2 겠지” 라고 계산하면 안 됩니다. innodb_autoinc_lock_mode 설정과 동시성에 따라 **띄엄띄엄 붙습니다**. ★ 서버가 여러 대일 때만 틀려서 재현이 안 됩니다. 최악의 버그입니다.

→ MySQL 에서 여러 건의 id 를 다 알아야 하면 **한 건씩 넣는 수밖에** 없습니다. → PostgreSQL 은 INSERT ... VALUES (...),(...),(...) RETURNING id 한 줄로 끝납니다.

이게 RETURNING 이 없어서 생기는 가장 큰 차이입니다.

문제 9 — PostgreSQL 에서 대소문자를 무시하는 중복 막기

```
const 답9 = "CREATE UNIQUE INDEX 작업자9_소문자 ON 작업자9 (lower(아이디))";

await 채점(9, "PG 에서 대소문자 무시 중복 막기", 답9, async (답) => {
  await 피지("DROP TABLE IF EXISTS 작업자9");
  await 피지("CREATE TABLE 작업자9 (아이디 VARCHAR(30))");
  await 피지(답);
  await 피지("INSERT INTO 작업자9 VALUES ('Admin')");

  try {
    await 피지("INSERT INTO 작업자9 VALUES ('admin')");
    return false;
  } catch (에러) {
    return 에러.code === "23505";
  }
});
```

```
}  
});
```

출력 문제 9 -  정답 · PG 에서 대소문자 무시 중복 막기

★ 이 문제는 개념03 에서 예상이 뒤집힌 자리입니다

“MySQL 이 허술하다” 고 배웠다면 MySQL 쪽이 뚫릴 거라고 생각했을 겁니다. **반대입니다.**

```
MySQL      - 기본 collation 이 `utf8mb4_0900_ai_ci` (ci = 대소문자 무시)  
            → UNIQUE 색인이 'Admin' 과 'admin' 을 같게 보고 **막습니다**  
PostgreSQL - 글자 그대로 비교합니다  
            → 둘 다 들어갑니다. **계정이 두 개** 생깁니다
```

여기서는 MySQL 이 더 안전하게 동작했습니다. ★ 좋고 나쁨의 문제가 아닙니다. 아이디어는 `_ci` 가 좋고,

상품 코드나 토큰에는 ``_ci`` 가 나뉩니다. ****다르다는 것을 아는 게 중요합니다.****

★★ 옮길 때 무슨 일이 하나 — 양쪽 다 사고입니다.

MySQL → PostgreSQL : 지금까지 막히던 'Admin'/'admin' 이 둘 다 가입됩니다 PostgreSQL → MySQL : 이미 들어 있는 두 계정을 옮기다가

****UNIQUE 충돌(1062)로 이관이 멈춥니다****

★ 그래서 옮기기 전에 반드시 세어 보세요.

```
SELECT lower(아이디), COUNT(*) FROM 작업자  
GROUP BY lower(아이디) HAVING COUNT(*) > 1;
```

★ 다른 방법도 있습니다.

① `citext` 확장 — 칸 타입을 아예 대소문자 무시로 만듭니다

```
CREATE EXTENSION IF NOT EXISTS citext;  
CREATE TABLE 작업자 (아이디 citext UNIQUE);  
* 확장을 깔 권한이 필요합니다. 관리형 서비스에서는 안 될 수 있습니다.
```

② PostgreSQL 12+ 의 대소문자 무시 collation 을 칸에 지정

③ ★★ 가장 확실한 방법: 앱에서 소문자로 바꿔서 저장합니다.

```
`아이디.trim().toLowerCase()` 한 줄이면 DB 가 무엇이든 알아줍니다.  
뒤 공백 문제(개념03 섹션 3)까지 같이 해결됩니다.
```

문제 10 — MySQL 에서 UPSERT


```
const 답10 =
  "INSERT INTO 구구_재고10 VALUES ('베어링', 5) AS 새값 " +
  "ON DUPLICATE KEY UPDATE 수량 = 구구_재고10.수량 + 새값.수량";

await 채점(10, "MySQL 의 UPSERT", 답10, async (답) => {
  await 마이("DROP TABLE IF EXISTS 구구_재고10");
  await 마이("CREATE TABLE 구구_재고10 (부품 VARCHAR(20) PRIMARY KEY, 수량 INT)");

  await 마이(답);
  await 마이(답);
  await 마이(답);

  const 확인 = await 마이("SELECT 부품, 수량 FROM 구구_재고10");
  return 확인.length === 1 && 확인[0].수량 === 15;
});
```

출력 문제 10 -  정답 · MySQL 의 UPSERT

★ 문법 대응

PostgreSQL	: ON CONFLICT (부품) DO UPDATE SET 수량 = 재고.수량 + EXCLUDED.수량
MySQL	: AS 새값 ON DUPLICATE KEY UPDATE 수량 = 구구_재고10.수량 + 새값.수량

“넣으려던 값” 을 가리키는 이름이 다릅니다.

PostgreSQL	- `EXCLUDED` (고정된 이름입니다)
MySQL	- `AS 별명` 으로 **내가 이름을 붙입니다** (8.0.20 부터)

★ 옛 MySQL 문법도 아직 돕니다. 그런데 경고가 뜨고, 곧 없어집니다.

```
ON DUPLICATE KEY UPDATE 수량 = 수량 + VALUES(수량)
```

새 코드에는 AS 별명 을 쓰세요.

★★ 진짜 차이는 어느 제약과 충돌했는지 고를 수 있느냐입니다.

PostgreSQL 은 ON CONFLICT (부품) 처럼 칸을 지정합니다. 그래서 “부품이 겹칠 때만 고치고, 다른 UNIQUE 가 겹치면 에러” 를 만들 수 있습니다.

MySQL 의 ON DUPLICATE KEY 는 아무 UNIQUE 든 겹치면 동작합니다. 표에 UNIQUE 가 두 개 있으면 의도하지 않은 줄이 고쳐질 수 있습니다. ★ 이걸 조용히 데이터가 망가지는 종류의 차이입니다.

★ affectedRows 로 무엇이 일어났는지 압니다.

1 = 새로 넣었음 · 2 = 이미 있어서 고쳤음

★★ 문서에는 “값이 그대로면 0” 이라고 적혀 있는데 드라이버 설정에 따라 다릅니다.

이 자료의 mysql2 에서는 값이 안 바뀌어도 1 이 나옵니다. 믿지 마세요.
"새로 만들어졌는지" 를 확실히 알아야 하면 PostgreSQL 의 RETURNING 이 낫습니다.

문제 11 — 이모지가 들어가는 MySQL 표 만들기

```
const 답11 = "CREATE TABLE 구구_문제11 (메모 VARCHAR(50)) CHARACTER SET utf8mb4";

await 채점(11, "이모지가 들어가는 표", 답11, async (답) => {
  await 마이("DROP TABLE IF EXISTS 구구_문제11");
  await 마이(답);
  await 마이("INSERT INTO 구구_문제11 VALUES ('점검 🛠️')");

  const 확인 = await 마이("SELECT 메모 FROM 구구_문제11");
  const 문자셋 = await 마이(
    "SELECT CHARACTER_SET_NAME AS c FROM information_schema.COLUMNS " +
    "WHERE TABLE_SCHEMA='factory_db' AND TABLE_NAME='구구_문제11' AND COLUMN_NAME='메모'",
  );

  return 확인[0].메모 === "점검 🛠️" && 문자셋[0].c === "utf8mb4";
});
```

출력 문제 11 -  정답 · 이모지가 들어가는 표

★★★ MySQL 에서 utf8 은 진짜 UTF-8 이 아닙니다

MySQL 의 utf8 = utf8mb3 = 한 글자당 최대 3바이트입니다. 한글은 3바이트라 들어갑니다. 이모지는 4바이트라 안 들어갑니다. utf8mb4 가 진짜 UTF-8 입니다.

★ STRICT 가 켜져 있으면 거절합니다 (1366). STRICT 가 꺼져 있으면 이모지가 물음표로 바뀝니다. 예러도 없습니다. 개념03 섹션 8 에서 직접 확인했습니다: '점검 완료 🛠️ 이상없음' → '점검 완료 ? 이상없음'

★ MySQL 8.0 부터는 서버 기본이 utf8mb4 라서 새로 만들면 대개 괜찮습니다. 문제는 5.x 시절에 만든 옛 표입니다. 그때 기본이 utf8(=utf8mb3) 이었습니다. 회사 표를 만나면 이걸로 확인하세요.

```
SELECT TABLE_NAME, TABLE_COLLATION FROM information_schema.TABLES WHERE TABLE_SCHEMA = '내디비';
```

★★ 그래서 이 문제는 "명시해서 쓰라" 고 했습니다. 기본값에 기대면 서버가 바뀔 때 조용히 달라집니다. 실무에서는 데이터베이스·표·칸·접속 네 곳 다 utf8mb4 로 맞춥니다.

★ PostgreSQL 에는 이 함정이 없습니다. 데이터베이스 인코딩을 UTF8 로 만들면 그냥 진짜 UTF-8 입니다. "반쪽짜리 UTF-8" 이라는 개념 자체가 없습니다.

문제 12 [도전] — 옮기기 점검 도구 만들기

```

const 답12 = (sql) => {
  const 규칙들 = [
    ["||", /\|\|/],
    ["RETURNING", /\bRETURNING\b/i],
    ["$파라미터", /\$d+/,],
  ];

  return 규칙들.filter(([, 정규식]) => 정규식.test(sql)).map(([이름]) => 이름);
};

await 채점(12, "[도전] 옮기기 점검 도구", 답12, (답) => {
  const 같나 = (가, 나) => JSON.stringify(가) === JSON.stringify(나);

  return (
    같나(답("INSERT INTO t(a) VALUES ($1) RETURNING id"), ["RETURNING", "$파라미터"])
    &&
    같나(답("SELECT a || b FROM t"), ["||"]) &&
    같나(답("SELECT concat(a,b) FROM t WHERE id = 3"), []) &&
    같나(답("SELECT a || b FROM t WHERE id=$1 RETURNING x"), ["||", "RETURNING",
"$파라미터"])
  );
});

```

출력 문제 12 -  정답 · [도전] 옮기기 점검 도구

★ 설계 요령

규칙을 **배열**로 만들고 **filter**로 거르면 순서가 저절로 지켜집니다. **if**를 세 번 쓰면 규칙이 늘어날 때마다 코드를 고쳐야 합니다. 규칙만 추가하면 되게 만드세요. 개념05의 **옮길때규칙**이 이 구조입니다.

★ 정규식 하나씩 보면

/\|\|/ - `|` 는 정규식에서 특별한 뜻이라 `` 로 막아야 합니다
 /\bRETURNING\b/i - ``b` 로 단어 경계를 잡아야 `RETURNINGS` 를 안 잡습니다
 /\\$d+/ - `\$` 도 특별한 뜻(문자열 끝)이라 막아야 합니다. `d+` 는 문제 7 과 같은 이유

★★ 이 도구의 한계를 정확히 알고 쓰세요

① 글자 안에 든 것도 잡습니다. SELECT 'a || b' 도 경고가 납니다

→ 작은따옴표 안을 먼저 지우면 줄어듭니다:
``sql.replace(/'(?:[^`\\]|\\.)*'/g, '')``

② 진짜 파서가 아닙니다. 여러 줄에 나뉜 문법을 놓칠 수 있습니다 ③ ★★★ 개념03의 조용한 차이는 못 잡습니다.

`WHERE 아이디 = '설비관리자'` 는 문법에 아무 문제가 없습니다.
그런데 대소문자 규칙 때문에 양쪽 답이 다를 수 있습니다.

그래도 씁니다. 쿼리가 수백 개면 **볼 곳을 좁혀 주는 것만으로** 충분합니다. ★ 경고 0개가 “안전하다” 는 뜻이 아니라는 것만 기억하세요.

문제 13 [도전] — 갭 락을 피하기

```
const 답13 = "SELECT * FROM 구구_생산13 WHERE 번호 = 10 FOR UPDATE";

await 채점(13, "[도전] 갭 락 피하기", 답13, async (답) => {
  await 마이("DROP TABLE IF EXISTS 구구_생산13");
  await 마이("CREATE TABLE 구구_생산13 (번호 INT PRIMARY KEY, 수량 INT)");
  await 마이("INSERT INTO 구구_생산13 VALUES (10,100),(20,200)");

  await MY2.query("SET SESSION innodb_lock_wait_timeout = 1");

  await 마이("BEGIN");
  const 잠근것 = await 마이(답);

  let 넣었나 = false;

  try {
    await MY2.query("INSERT INTO 구구_생산13 VALUES (15, 150)");
    넣었나 = true;
  } catch (에러) {
    넣었나 = false;
  }

  await 마이("ROLLBACK");

  return 넣었나 === true && 잠근것.length === 1 && 잠근것[0].번호 === 10;
});
```

출력 문제 13 -  정답 · [도전] 갭 락 피하기

★ 왜 범위로 잠그면 막히나

MySQL 의 REPEATABLE READ 는 팬텀 읽기를 **넥스트키 락**으로 막습니다. 줄뿐 아니라 **줄과 줄 사이의 빈 자리(갭)** 까지 잠급니다. **BETWEEN 10 AND 20** 으로 훑으면 10 과 20 사이가 통째로 잠깁니다. 그래서 다른 접속이 15번을 못 넣고 1205(잠금 대기 시간 초과)로 죽습니다.

★★ 개념05 에서 더 놀라운 것도 봤습니다. 범위 밖인 **100번도 막혔습니다**. 20번 다음에 아무것도 없는 것을 확인하면서 “20번 뒤의 모든 자리” 를 잠갔기 때문입니다.

★ **번호 = 10** 은 왜 안 막히나 기본키에 **정확히 하나**를 찍었기 때문입니다. 찾는 값이 유일 색인에 딱 하나 있으면 InnoDB 는 갭을 안 잡고 그 줄 하나만 잠급니다(record lock).

★★ 실무 규칙 네 가지 ① 잠글 때는 **정확한 값 하나로** 좁히세요 ② 잠그는 쿼리의 WHERE 절에 **반드시 색인이** 있어야 합니다.

색인이 없으면 **흙은 줄**을 전부 잠급니다. 사실상 표 잠금입니다

③ 정말 넓게 잠가야 하면 격리수준을 **READ COMMITTED** 로 낮춥니다.

갭 락이 거의 사라집니다. 대신 팬텀은 허용됩니다

④ 트랜잭션을 짧게 잡으세요. 이건 어느 DB 에서나 맞는 말입니다

★★★ 반대 방향도 조심하세요. MySQL 에서 “갭 락이 중복을 막아 주겠지” 하고 짤 코드를 PostgreSQL 로 옮기면 **아무도 안 막아 줍니다**. PostgreSQL 은 스냅샷으로 팬텀을 막기 때문에 빈 자리를 잠그지 않습니다. → 중복을 막는 것은 잠금이 아니라 **UNIQUE 제약**입니다. 어느 DB 든 그게 정답입니다.

문제 14 [도전] — 양쪽 결과를 비교하는 함수

```
const 답14 = async (피지SQL, 마이SQL) => {
  const 가 = await 피지(피지SQL);
  const 나 = await 마이(마이SQL);

  return JSON.stringify(가) === JSON.stringify(나);
};

await 채점(14, "[도전] 양쪽 결과 비교하기", 답14, async (답) => {
  const 같은것 = await 답("SELECT 1 AS x", "SELECT 1 AS x");
  const 다른것 = await 답("SELECT 'a' || 'b' AS x", "SELECT 'a' || 'b' AS x");
  const 또같은것 = await 답("SELECT concat('a','b') AS x", "SELECT concat('a','b') AS x");

  return 같은것 === true && 다른것 === false && 또같은것 === true;
});
```

출력 문제 14 -  정답 · [도전] 양쪽 결과 비교하기

★★★ 이게 이 단원 전체의 결론입니다

개념02 의 차이(문법 오류)는 옮기면 바로 터져서 어차피 눈에 띄니다. 개념03 의 차이(조용히 다른 답)는 **아무도 알려 주지 않습니다**.

그걸 잡는 유일한 방법이 **같은 쿼리를 양쪽에 보내고 답을 맞춰 보는 것**입니다. 위 함수가 그 도구입니다.

개념02~05 의 **나란히()** 함수와 같은 생각입니다.

★ 실무에서 쓰려면 이렇게 키웁니다

① 실제로 도는 쿼리 목록을 모읍니다 (느린 쿼리 로그·코드 검색) ② 같은 데이터를 양쪽에 넣습니다 ③ 쿼리마다 이 함수를 돌려서 **다른 것만** 뽑습니다 ④ 다른 것을 하나씩 봅니다. 대부분 이 단원의 표에 나오는 것들입니다

★★ 비교할 때 조심할 점

① 타입이 다르게 옵니다. MySQL 은 참/거짓을 1/0 으로, DECIMAL 을 글자로 줍니다.

`JSON.stringify` 로 그냥 비교하면 "값은 같은데 다르다" 는 거짓 경고가 납니다.
→ 숫자는 `Number(...)` , 참거짓은 `!!...` 로 맞춘 뒤 비교하는 게 좋습니다.

② 줄 순서가 다를 수 있습니다. ORDER BY 가 없으면 순서는 보장이 없습니다.

비교하기 전에 정렬하거나, 쿼리에 `ORDER BY` 를 넣으세요.

③ NULL 의 자리가 다릅니다 (문제 4). 이것도 순서 문제로 나타납니다.

★ 그래서 이 함수가 "다르다" 고 하면 두 가지 중 하나입니다.

- 진짜 의미가 다르다 → 고쳐야 합니다
- 모양만 다르다 → 비교 방법을 다듬어야 합니다

둘 다 직접 눈으로 봐야 판단할 수 있습니다. 자동화의 끝은 사람입니다.

뒷정리

```
await PG.query("DROP TABLE IF EXISTS 작업자9");

for (const 표 of ["구구_문제6", "구구_문제8", "구구_재고10", "구구_문제11", "구구_
생산13"]) {
  await MY.query(`DROP TABLE IF EXISTS ${표}`);
}

console.log("=== 정답 확인 끝 ===");
```

출력 === 정답 확인 끝 ===

한 장 요약 — 이 단원에서 꼭 가져갈 것

① "MySQL 은 허술하다" 는 **낮은** 말입니다.

8.4 는 잘못된 타입·길이 초과·범위 초과·GROUP BY 위반·CHECK 위반을 전부 거절합니다.
개념02 에서 일곱 가지를 재서 **7 대 7** 로 확인했습니다.

② 대신 **설정을 보세요**. SELECT @@sql_mode 한 줄입니다.

STRICT 를 끄면 5분 만에 옛날로 돌아갑니다. 버전이 아니라 설정입니다.

③ 진짜 위험한 것은 **에러가 안 나는 차이**입니다.

`||`, 대소문자, 뒤 공백, 0 나누기, 자동 형변환, LENGTH, NULL 정렬.
전부 양쪽에서 통하는 대체 문법이 있습니다. 그걸 쓰세요.

④ 없는 기능은 오히려 안전합니다. 옮기면 터지니까요.

RETURNING · 부분 색인 · 배열 · 트랜잭션 DDL.
★ 이 중 **트랜잭션 DDL** 만은 미리 대비해야 합니다. 마이그레이션이 반쯤 멈춥니다.

⑤ 마지막 확인은 **양쪽에 같은 쿼리를 보내고 답을 맞춰 보는 것**입니다.

문제 14 의 함수가 그것입니다.

★ 그리고 둘 다 훌륭한 데이터베이스입니다.

운영할 사람이 아는 DB 가 제일 좋은 DB 입니다.

```
await PG.end();
await MY.end();
await MY2.end();
```

10단원 연습문제 정답 운영과 종합 실습

RUN node 연습문제_정답.js

★ 이 파일은 Docker 가 필요합니다. docker compose up -d

★ 채점에 VACUUM 과 스키마 변경이 들어가서 25초쯤 걸립니다.

정답만 적어 두면 다음에 또 틀립니다. **왜 그런지**를 길게 적었습니다. 답을 맞혔더라도 설명은 읽어 보세요.

운영은 “이렇게 하면 된다” 보다 “**이렇게 하면 사고가 난다**” 를 아는 것이 중요합니다. 그래서 각 문제마다 틀린 답이 왜 위험한지도 같이 적었습니다.

```
import pg from "pg";
import { execSync } from "node:child_process";
```


준비

```

const 접속정보 = {
  host: "127.0.0.1",
  port: 5434, // * 5432 가 아닙니다 user: "factory",
  password: "secret",
  database: "factory_db",
  application_name: "unit10-answer", // * 영어로
};

let 연결;

try {
  연결 = new pg.Client(접속정보);
  await 연결.connect();
} catch (에러) {
  console.log("데이터베이스에 연결하지 못했습니다.", 에러.code ?? 에러.message);
  console.log("이 파일은 Docker 가 필요합니다. 아래를 치고 다시 실행하세요.");
  console.log(" docker compose up -d");
  process.exit(0);
}

const 스키마 = "단원10_정답";

await 연결.query(`DROP SCHEMA IF EXISTS ${스키마} CASCADE`);
await 연결.query(`CREATE SCHEMA ${스키마}`);
await 연결.query(`SET search_path TO ${스키마}`);

async function 새연결(이름표) {
  const 새것 = new pg.Client({ ...접속정보, application_name: 이름표 });
  await 새것.connect();
  await 새것.query(`SET search_path TO ${스키마}`);
  return 새것;
}

const 컨테이너 = "db-material-postgres";

function 도커(명령) {
  return execSync(`docker exec ${컨테이너} sh -c ${JSON.stringify(명령)}`, {
    encoding: "utf8",
    timeout: 120000,
    maxBuffer: 32 * 1024 * 1024,
  });
}

```

실습용 표입니다. 05단원에서 쓰던 제조 현장 표입니다.

```

await 연결.query(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL
  );

  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT NOT NULL REFERENCES 라인(라인코드),
    도입가    NUMERIC(12,2) NOT NULL
  );

  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT NOT NULL REFERENCES 설비(설비번호),
    점검일    DATE NOT NULL,
    결과      TEXT NOT NULL
  );

  CREATE TABLE 임시메모 (
    내용 TEXT
  );
`);

await 연결.query(`
  INSERT INTO 라인 VALUES ('A','조립1라인'), ('B','가공2라인'), ('C','포장3라인');

  INSERT INTO 설비 VALUES
    (1,'컨베이어 1호','A',12000000.00),
    (2,'프레스 1호','A',35400000.00),
    (3,'용접로봇 1호','B',88000000.00);

  INSERT INTO 점검기록
  SELECT g, (g % 3) + 1, DATE '2026-01-01' + (g % 120),
    (ARRAY['정상','주의','고장'])[(g % 3) + 1]
  FROM generate_series(1, 40000) g;
`);

await 연결.query("VACUUM ANALYZE");

const 잠깐 = (밀리초) => new Promise((끝) => setTimeout(끝, 밀리초));

```

문제 1 — 백업 명령의 형식 고르기

표 하나만 골라 되살릴 수 있고, 압축도 되는 pg_dump 형식은 무엇인가요? "plain" / "custom" / "directory" 중에 하나를 문자열로 적으세요.

```
const 답1 = "custom";
```

★ 왜 custom 인가

plain 그냥 SQL 텍스트입니다. psql < 파일 로 통째로 밀어 넣는 것 말고는

할 수 있는 게 없습니다. 압축도 안 됩니다.

custom ✓ 압축됩니다. pg_restore -l 로 목록을 볼 수 있고,

```
`--table=이름` 으로 표 하나만 골라 되살릴 수 있습니다.
```

directory custom 과 능력은 같은데 폴더로 나옵니다.

```
`-j 4` 로 **백업도 병렬**로 됩니다. 표가 수백 개인 큰 DB 용입니다.
```

★★ 실무 기본값은 custom 입니다. 고민되면 이걸 쓰세요. directory 는 "백업 자체가 너무 오래 걸린다" 는 문제가 생겼을 때 넘어가면 됩니다.

문제 2 — 백업을 진짜로 만들기

단원10_정답.설비 표 하나만 custom 형식으로 뽑는 pg_dump 명령을 완성하세요. 결과 파일은 /tmp/정답백업.dump 입니다.

힌트: -t 스키마.표이름, --format=custom, -f 파일경로

앞부분은 이미 적어 뒀습니다.

```
const 백업명령 =
  `PGPASSWORD=secret pg_dump -U factory -d factory_db ` +
  `-t ${스키마}.설비 --format=custom -f /tmp/정답백업.dump`;
```

★ 옵션 하나씩 -U factory 접속 계정 -d factory_db 데이터베이스 이름 -t 스키마.표 ★ 이 표만. 안 주면 데이터베이스 전체가 나옵니다 --format=custom 문제 1 에서 고른 형식 -f 파일경로 어디에 쓸지. 안 주면 화면으로 쏟아집니다

★★ -t 없이 뽑으면 다른 단원 표까지 딸려 옵니다. 이 자료는 07~10단원이 같은 데이터베이스를 쓰기 때문입니다. 실무에서도 "이 표만" 이 필요한 순간이 자주 옵니다. 옵션을 기억해 두세요.

★ 비밀번호를 PGPASSWORD= 로 넘겼습니다. 편하지만 명령 기록에 남습니다. 운영에서는 ~/.pgpass 파일 (권한 600)이나 환경 변수 파일을 쓰세요.

문제 3 — 총 시간으로 줄 세우기

pg_stat_statements 를 흉내 낸 배열입니다. 서버 시간을 제일 많이 잡아먹는 쿼리의 이름을 돌려주세요.

★ 주의: 평균이 긴 것이 아니라 **총 시간**이 긴 것입니다.

```
const 쿼리통계 = [
  { 이름: "목록조회", 호출: 500000, 평균ms: 0.4 },
  { 이름: "월간집계", 호출: 12, 평균ms: 900 },
  { 이름: "상세조회", 호출: 80000, 평균ms: 1.1 },
];

function 진짜범인(통계) {
  return [...통계].sort((가, 나) => 나.호출 * 나.평균ms - 가.호출 * 가.평균ms)[0].
  이름;
}
```

★★★ 이 문제가 개념02 의 전부입니다. 숫자를 직접 보세요.

이름 호출 평균 총 시간

목록조회	500,000	0.4ms	200,000ms	← ★ 범인
상세조회	80,000	1.1ms	88,000ms	
월간집계	12	900ms	10,800ms	

눈에 띄는 것은 **월간집계**입니다. 한 번에 0.9초니까요. 그런데 서버 시간의 **2퍼센트**밖에 안 씁니다.

목록조회는 0.4ms 입니다. 아무도 느리다고 생각 안 합니다. 그런데 서버 시간의 **67퍼센트**를 씁니다.

월간집계의 18배입니다.

★★ 그래서 pg_stat_statements 는 이렇게 봅니다.

```
ORDER BY total_exec_time DESC    ← ✓
ORDER BY mean_exec_time DESC     ← ✕ 배치 쿼리만 나옵니다
```

★ 그리고 **고치는 방법이 다릅니다**. 목록조회를 색인으로 0.4ms → 0.2ms 로 만들면 10만ms 를 아깁니다. 좋습니다. 그런데 더 좋은 것은 **50만 번 부르는 것을 줄이는 것**입니다. 대개 08단원의 N+1 입니다. 목록 하나 그리는데 줄마다 쿼리를 날리는 것이죠. 그걸 한 번의 JOIN 으로 바꾸면 50만 번이 1000번이 됩니다.

문제 4 — 지금 오래 돌고 있는 쿼리 찾기

pg_stat_activity 에서 **지금 실행 중이면서 1초 넘게 돌고 있는** 세션을 찾는 SQL 을 쓰세요.

application_name 이 'unit10-answervictim' 인 것만 봅니다.

돌려줄 칸: 문장 하나 (query 의 앞 20글자)

힌트: state = 'active', now() - query_start > interval '1 second'

left(query, 20) AS 문장

```
const 들고있는것찾기 = `
  SELECT left(query, 20) AS 문장
  FROM pg_stat_activity
  WHERE application_name = 'unit10-answervictim'
        AND state = 'active'
        AND now() - query_start > interval '1 second'
`;
```

★ 세 조건이 다 필요합니다.

application_name = '...' ★ 이 DB 는 다른 단원도 같이 씁니다.

내 것만 봐야 합니다

state = 'active' 지금 **진짜 일하는 중인 것만** now() - query_start > interval '1 second' 1초 넘게 들고 있는 것만

★★ state 를 안 걸면 놓고 있는 연결까지 다 나옵니다. 연결 풀을 쓰면 idle 이 수십 개씩 떠 있습니다. 그건 정상입니다. (08단원)

★★★ 운영에서 쓰는 진짜 질의는 이렇습니다. 외워 두면 장애 때 큰 도움이 됩니다.

```
SELECT pid,
       now() - query_start AS 걸린시간,
       state, wait_event_type,
       left(query, 80) AS 문장
FROM pg_stat_activity
WHERE state <> 'idle' AND pid <> pg_backend_pid()
ORDER BY query_start;
```

pid <> pg_backend_pid() 는 나 자신을 빼는 것입니다. 안 빼면 이 조회 자체가 목록에 나와서 헛갈립니다.

문제 5 — 취소인가 강제 종료인가

아래 상황에서 무엇을 써야 할까요. "pg_cancel_backend" 또는 "pg_terminate_backend" 를 문자열로 적으세요.

"보고서 쿼리가 10분째 들고 있습니다. 멈추고 싶은데, 그 애플리케이션의 연결은 살려 두고 싶습니다."

```
const 답5 = "pg_cancel_backend";
```

★ 둘의 차이가 이렇게 다릅니다.

쿼리	연결	열린 트랜잭션	앱에서 보이는 것
----	----	---------	-----------

pg_cancel_backend	멈춤	살아 있음	★ 그대로 열림	쿼리 에러 (57014)
pg_terminate_backend	멈춤	끊김	롤백됨	연결 끊김 (57P01)

★★ 문제는 “연결은 살려 두고 싶다” 였으므로 `pg_cancel_backend` 입니다.

★★★ 그런데 `pg_cancel_backend` 에는 함정이 있습니다. 트랜잭션이 **열린 채로 남습니다**. 그 세션이 `idle in transaction` 이 됩니다. 그러면 잠금을 계속 쥐고, **청소를 막습니다**. (개념03 섹션 5)

그래서 취소한 뒤에는 반드시 확인하세요.

- 그 앱이 ROLLBACK 을 보내는가
- 안 보내면 ``idle_in_transaction_session_timeout`` 이 걸려 있는가

★ 사람이 손으로 죽이는 것 자체가 이미 늦은 것입니다. `statement_timeout` 을 걸어 두면 서버가 알아서 자릅니다. (07단원)

문제 6 — 표 크기 재기

점검기록 표의 **본체 크기**(색인 제외)를 바이트로 돌려주는 SQL 을 쓰세요. 돌려줄 칸 이름은 **바이트** 이고 정수여야 합니다.

힌트: `pg_relation_size('스키마.표 '::regclass)::int AS 바이트` ★ `pg_relation_size($1)` 처럼 파라미터로 넘기면 타입 추론이 깨집니다.

문자열로 직접 적으세요.

```
const 표크기SQL = `SELECT pg_relation_size('${스키마}.점검기록 '::regclass)::int AS 바이트`;
```

★ 크기 함수가 세 가지입니다. 자주 헷갈립니다.

<code>pg_relation_size(표)</code>	표 **본체만** (색인 제외)
<code>pg_indexes_size(표)</code>	그 표의 **색인들만**
<code>pg_total_relation_size(표)</code>	★ 본체 + 색인 + TOAST 전부

03단원에서 “정확히 두 배” 를 잴 때는 `pg_total_relation_size` 를 썼습니다. 개념03 에서 표가 부푸는 것만 볼 때는 `pg_relation_size` 를 썼습니다. **무엇을 보고 싶은지에 따라 고르세요**.

★★ `::int` 를 붙인 이유 이 함수들은 `BIGINT` 를 돌려줍니다. pg 에서 `BIGINT` 는 **문자열**입니다. “1163264” 가 와서 > 100000 비교가 이상해집니다. 02단원의 그 표입니다.

★★★ 왜 파라미터로 넘기면 안 되나 — 이 자료를 만들며 실제로 걸렸습니다.

```
await 연결.query("SELECT pg_relation_size($1)", ["단원10_정답.점검기록"]);
```

이러면 Postgres 가 \$1 의 타입을 정하지 못합니다. `pg_relation_size` 는 `regclass` 를 받는 것과 `text` 를 받는 것이 따로 있어서 어느 쪽인지 모릅니다. 42725(모호함) 나 42883(함수 없음)이 납니다. ★ 표 이름처럼 구조를 가리키는 것은 파라미터로 못 넘깁니다.

값이 아니라 이름이니깐요. 문자열로 만들되, 사용자 입력을 그대로 넣지 마세요.

문제 7 — VACUUM 인가 VACUUM FULL 인가

두 상황에 각각 "VACUUM" 또는 "VACUUM FULL" 을 넣으세요.

상황 가: "운영 중입니다. 죽은 줄이 쌓여서 표가 계속 커집니다.

더 커지지만 않게 하고 싶습니다."

상황 나: "새벽에 서비스를 30분 세울 수 있습니다.

디스크를 진짜로 되찾고 싶습니다."

```
const 답7가 = "VACUUM";
const 답7나 = "VACUUM FULL";
```

★ 개념03 에서 실제로 했던 것을 다시 정리하면 이렇습니다.

파일 크기	잠금	운영 중에 되나
-------	----	----------

VACUUM ★ 안 줄어듭니다 거의 안 막음 ✓ 됩니다 VACUUM FULL 줄어듭니다 ★ 전부 막음 × 서비스가 멈춥니다

★★ 상황 가 — “더 커지지만 않게” 이게 VACUUM 이 하는 일 그대로입니다. 죽은 줄을 지워서 빈자리로 만듭니다. 다음 UPDATE 가 그 자리를 씁니다. 개념03 에서 했습니다. VACUUM 뒤에는 고쳐도 표가 안 커졌습니다.

★★ 상황 나 — “디스크를 진짜로 되찾고 싶다” VACUUM FULL 만 파일을 줄입니다. 표를 통째로 옆에 새로 써서 옛 것을 버립니다. 그래서 디스크가 잠깐 두 배 필요합니다. 이게 함정입니다. 디스크가 꽉 차서 줄이려는 상황이면 VACUUM FULL 마저 실패합니다.

★★★ 상황 나에도 더 좋은 답이 있습니다. `pg_repack` 입니다. VACUUM FULL 과 같은 일을 하는데 거의 안 잠급니다. 서비스를 안 세우고 부푼 표를 줄일 수 있습니다. 이 자료는 확장 설치를 안 하므로 이름만 알려드립니다.

문제 8 — 죽은 줄이 많은 표 찾기

단원10_정답 스키마에서 죽은 줄이 1000개를 넘는 표를 찾는 SQL 을 쓰세요. 돌려줄 칸: 표 (표 이름)

힌트: `pg_stat_user_tables` 의 `schemaname`, `relname`, `n_dead_tup`

```
const 부폰표찾기 = `
    SELECT relname::text AS 표
    FROM pg_stat_user_tables
    WHERE schemaname = '${스키마}' AND n_dead_tup > 1000
    ORDER BY 표
`;
```

★ `pg_stat_user_tables` 에서 꼭 볼 칸들입니다.

<code>n_live_tup</code>	산 줄 (대략)
<code>n_dead_tup</code>	* 죽은 줄 - 청소해야 할 것
<code>last_vacuum</code>	마지막으로 **손으로** 돌린 시각
<code>last_autovacuum</code>	마지막으로 **알아서** 돈 시각

`last_analyze` / `last_autoanalyze` 통계를 언제 모았나

★★ 이 숫자들은 **대략**이고 **늦게** 옵니다. 통계는 따로 모아서 나중에 반영됩니다. 그래서 채점 코드가 `ANALYZE` 를 돌리고 채워질 때까지 몇 번 다시 봅니다. 이 자료를 만들 때도 바로 읽었다가 0 이 나와서 해했습니다.

★★★ 운영에서 진짜 쓰는 질의는 **비율**로 봅니다. 절대 개수는 표마다 의미가 달라서요.

```
SELECT relname,
       n_live_tup, n_dead_tup,
       round(n_dead_tup * 100.0 / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS 죽은비율,
       last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC
LIMIT 20;
```

`NULLIF(..., 0)` 은 **0 으로 나누기를 막는 것**입니다. 빈 표에서 터지지 않게요.

★ 죽은 줄이 많은데 `last_autovacuum` 이 오래됐다면 → 긴 트랜잭션이 막고 있는지 먼저 보세요. (문제 9)

문제 9 — 긴 트랜잭션 찾기

청소를 막고 있을 만한 세션을 찾습니다. `application_name` 이 'unit10-answerblocker' 이고 **트랜잭션이 열려 있는** 세션의 `state` 를 돌려주는 SQL 을 쓰세요.

돌려줄 칸: `state`

힌트: `xact_start IS NOT NULL` 이면 트랜잭션이 열려 있는 것입니다


```
const 긴트랜잭션찾기 = `
    SELECT state
    FROM pg_stat_activity
    WHERE application_name = 'unit10-answerblocker'
    AND xact_start IS NOT NULL
`;
```

★ 세 가지 시각이 있고 뜻이 다릅니다. 이걸 구분하는 게 핵심입니다.

```
backend_start    그 연결이 **붙은** 시각
xact_start       * 지금 열려 있는 **트랜잭션이 시작된** 시각
query_start      지금 문장이 시작된 시각
```

`xact_start IS NOT NULL` = **트랜잭션이 열려 있다** 문장은 안 돌고 있어도(state = 'idle in transaction') 열려 있을 수 있습니다.

★★★ `idle in transaction` 이 제일 위험합니다. 아무 일도 안 하면서 잠금을 쥐고, **청소를 막습니다**. (개념03 섹션 5) 개념03 에서 봤듯이, 그 세션이 우리 표를 건드리지 않았어도 막습니다.

★★ 운영에서 쓰는 질의입니다.

```
SELECT pid, application_name, state,
       now() - xact_start AS 트랜잭션나이,
       left(query, 60) AS 마지막문장
FROM pg_stat_activity
WHERE xact_start < now() - interval '5 minutes' AND pid <> pg_backend_pid()
ORDER BY xact_start;
```

★ 그리고 아예 못 생기게 막으세요. 이 한 줄이 사고를 많이 막습니다.

```
idle_in_transaction_session_timeout = '60s'
```

문제 10 — 안전한 칸 추가

운영 중인 **설비** 표에 **담당자** TEXT 칸을 추가하려고 합니다. **잠금을 못 잡으면 2초 만에 포기**하도록 만드세요.

두 문장을 순서대로 배열로 돌려주세요. ["SET lock_timeout = ...", "ALTER TABLE ..."]

```
function 안전한칸추가() {
    return ["SET lock_timeout = '2s'", "ALTER TABLE 설비 ADD COLUMN 담당자 TEXT"];
}
```

★★★ 왜 `lock_timeout` 이 그렇게 중요한가 — 개념04 섹션 2 를 다시 보세요.

칸 추가 자체는 2ms 입니다. 아무 문제 없습니다. 문제는 **잠금을 못 잡고 기다리는 동안**입니다.

① 누가 그 표를 읽는 긴 트랜잭션을 열어 뒀습니다 ② ALTER 가 ACCESS EXCLUSIVE 를 기다립니다 ③

★ 그 뒤에 오는 **모든 요청이 줄을 섭니다**. 평범한 SELECT 까지요 ④ 몇 초 만에 커넥션 풀이 다 찹니다

(08단원) ⑤ 서비스 전체가 죽습니다

개념04 에서 실제로 했습니다. ALTER 가 806ms 기다리는 동안 죄 없는 SELECT 도 810ms 를 같이 기다렸습니다.

★★ `lock_timeout` 을 걸면 못 잡을 때 **2초 만에 55P03 으로 실패합니다.** 실패는 괜찮습니다. 다시 하면 됩니다. 기다리는 것이 나쁩니다.

★ 재시도까지 붙인 실무용 모양입니다.

```
for (let 시도 = 0; 시도 < 10; 시도 += 1) {
  try {
    await 연결.query("SET lock_timeout = '2s'");
    await 연결.query("ALTER TABLE 설비 ADD COLUMN 담당자 TEXT");
    break;
  } catch (에러) {
    if (에러.code !== "55P03") throw 에러;    // 다른 에러면 진짜 문제입니다 await
    잠깐(5000);
  }
}
```

★★ `statement_timeout` 을 쓰면 안 됩니다. 그건 문장 **전체** 시간을 재서, 표를 다시 쓰는 중에 잘라 버립니다. 그러면 그때까지 한 일이 통째로 날아갑니다. `lock_timeout` 은 **기다리는 시간만** 잡니다.

문제 11 — 안 막는 색인 만들기

점검기록(설비번호) 에 색인을 만듭니다. 이름은 `연습_설비번호` 입니다. **쓰기를 막지 않는** 방법으로 만드는 SQL 한 줄을 쓰세요.

```
const 안막는색인 = "CREATE INDEX CONCURRENTLY 연습_설비번호 ON 점검기록 (설비번호);"
```

★ `CONCURRENTLY` 한 단어 차이입니다. 그런데 하는 일이 많이 다릅니다.

쓰기를 막나 표를 몇 번 훑나 트랜잭션 안에서

CREATE INDEX ★ 막습니다 한 번 ✓ 됩니다 CREATE INDEX 안 막습니다 두 번 ★ × 25001

CONCURRENTLY

★★ 대가가 있습니다. · 표를 두 번 훑어서 대체로 더 오래 걸립니다 · ★ 중간에 실패하면 **못 쓰는 색인**이 조용히 남습니다

못 쓰는 색인은 자리는 차지하고 INSERT 마다 써야 하는데 **조회에는 안 쓰입니다.** 최악입니다. 그런데 아무 표시도 안 남니다.

★★★ 그래서 `CONCURRENTLY` 를 쓴 뒤에는 **반드시 확인하세요.**

```
SELECT indexrelid::regclass AS 색인
FROM pg_index WHERE NOT indisvalid;
```

나오면 `DROP INDEX CONCURRENTLY` 이름; 으로 지우고 다시 만듭니다.

★ 그리고 **트랜잭션 안에서는 못 씁니다** (25001). 08단원의 마이그레이션 도구는 파일마다 `BEGIN` 을 겁니다. 그래서 이 문장만 “트랜잭션 없이” 표시를 해서 따로 돌려야 합니다.

문제 12 — [도전] 확장 → 이전 → 축소

설비.이름 을 설비.설비명 으로 바꾸려고 합니다. 무중단으로 하려면 어떤 순서여야 할까요.

아래 다섯 가지 일을 **안전한 순서**로 배열에 답으세요. 문자열을 그대로 쓰면 됩니다.

```
"옛칸지우기" ALTER TABLE 설비 DROP COLUMN 이름
"새칸추가" ALTER TABLE 설비 ADD COLUMN 설비명 TEXT
"값복사" UPDATE 설비 SET 설비명 = 이름
"새코드배포"      새 코드가 설비명을 읽고 씁니다
"동기화트리거"    양쪽 칸을 자동으로 맞춰 줍니다
```

★ 힌트: 새 코드가 배포되기 **전에** 새 칸이 있어야 합니다. 그리고 옛 칸은 **모든 서버가 새 코드가 된 뒤에** 지웁니다.

```
const 무중단순서 = ["새칸추가", "동기화트리거", "값복사", "새코드배포",
"옛칸지우기"];
```

★★★ 왜 이 순서인가 — 한 단계씩 “지금 옛 코드가 살아 있나” 를 물어보면 됩니다.

순서 그 직후에 옛 코드는? 새 코드는? 왜 이 자리인가

① 새칸추가 ✓ 삽니다 아직 없음 새 코드가 오기 전에

칸이 먼저 있어야 합니다

② 동기화트리거 ✓ 삽니다 아직 없음 ★ 값복사보다 먼저!

아래 설명을 보세요

③ 값복사 ✓ 삽니다 아직 없음 옛 값을 새 칸으로 ④ 새코드배포 ✓ 아직 삽니다 ✓ 삽니다 ★ 두 버전이 같이 도는 구간 ⑤ 옛칸지우기 × 죽습니다 ✓ 삽니다 모두 새 코드가 된 뒤

★★ ②가 ③보다 먼저인 것이 제일 헷갈리는 부분입니다.

트리거를 나중에 달면, 값복사와 트리거 사이의 틈에 들어온 변경이 새 니다. 그 사이에 옛 코드가 이름 을 고치면 설비명 은 옛 값 그대로입니다. 나중에 아무도 그 어긋남을 못 찾습니다. 몇 건이라 눈에 안 띄니까요.

★ 순서를 지키면 틈이 없습니다. 트리거가 먼저 있으니 그 뒤의 모든 변경이 양쪽에 갑니다.

★★ ⑤를 같은 날 하면 안 됩니다. ④에서 새 코드에 문제가 생기면 옛 코드로 되돌려야 합니다. 옛 칸이 아직 있으면 되돌릴 수 있습니다. 지웠으면 못 돌아갑니다. ★ 보통 며칠에서 한두 주 뒤에 지웁니다. 급할 이유가 하나도 없습니다.

★ 트리거가 왜 꼭 필요한가 이름이 NOT NULL 이면, 새 코드가 설비명만 채워서 INSERT 할 때 이름이 비어서 23502로 실패합니다. 트리거가 BEFORE INSERT 에서 채워 주기 때문에 통과합니다.

트리거 대신 새 코드가 양쪽에 다 쓰게 만들어도 됩니다. 대신 그 표를 건드리는 모든 코드를 빠짐없이 고쳐야 합니다. 경로가 많으면 트리거가 안전합니다. 놓칠 곳이 없으니까요.

문제 13 — [도전] 색인 없는 외래키 찾기

외래키를 걸었는데 그 칸에 색인이 없는 곳을 찾는 SQL 을 쓰세요. [단원10_정답](#) 스키마만 봅니다.

돌려줄 칸: 표와 칸

힌트: pg_constraint (contype='f', conkey[1] 이 첫 칸)

pg_index 에 indrelid 가 같고 indkey[0] = conkey[1] 인 것이 없으면 색인 없음
pg_attribute 로 칸 번호를 이름으로 바꿉니다

```
const 색인없는외래키찾기 = `
SELECT t.relname::text AS 표, a.attname::text AS 칸
FROM pg_constraint c
JOIN pg_class t ON t.oid = c.conrelid
JOIN pg_attribute a ON a.attrelid = c.conrelid AND a.attnum = c.conkey[1]
WHERE c.contype = 'f'
      AND c.connamespace = '${스키마}::regnamespace
      AND NOT EXISTS (
        SELECT 1 FROM pg_index i
        WHERE i.indrelid = c.conrelid AND i.indkey[0] = c.conkey[1]
      )
ORDER BY 표, 칸
`;
```

★ 카탈로그 표들이 무엇인지부터

pg_constraint 제약들. contype='f' 가 외래키입니다

`conrelid`	= 자식 표, `conkey` = 자식 쪽 칸 번호 배열
pg_class	표·색인 같은 것들. `relname` 이 이름입니다
pg_attribute	칸들. `attnum` 이 번호, `attname` 이 이름입니다
pg_index	색인들. `indkey` 가 색인이 걸린 칸 번호 배열입니다

★★ indkey[0] = conkey[1] 이 핵심입니다. 번호가 헛갈립니다. conkey 는 1부터 세는 SQL 배열이라 첫 칸이 conkey[1] 입니다. indkey 는 0부터 세는 내부 벡터라 첫 칸이 indkey[0] 입니다. 둘 다 “첫 번째 칸” 을

가리킵니다.

★★★ 왜 첫 칸만 보나 — 06단원의 복합 색인 규칙 때문입니다. 색인 (라인코드, 생산일) 은 라인코드 만으로 찾을 때도 쓸 수 있습니다. 왼쪽부터 쓰니까요. 그래서 외래키 칸이 색인의 첫 칸이기만 하면 됩니다. 개념05 에서 생산실적.라인코드 가 안 걸린 이유가 이것입니다. 복합 기본키 (라인코드, 생산일) 의 첫 칸이었으니까요.

★★ 왜 이게 중요한가 — 색인이 없으면 이렇게 됩니다.

```
DELETE FROM 라인 WHERE 라인코드 = 'D';
```

Postgres 는 “이 라인을 쓰는 설비가 있나” 를 확인해야 합니다. 색인이 없으면 설비 표를 통째로 훑습니다. 그동안 잠금도 씁니다. 설비가 1000만 건이면 라인 하나 지우는 데 몇십 초가 걸립니다.

★ 기본키와 UNIQUE 에는 색인이 자동으로 생깁니다. 외래키에는 안 생깁니다. 이게 실무에서 제일 흔한 누락입니다.

문제 14 — [도전] 코드 점검 도구

소스 코드 문자열을 받아서 위험한 줄을 찾아 주는 함수를 만드세요.

찾을 것 두 가지 ① SELECT * → 종류 “별표” ② 템플릿 문자열로 이어 붙인 SQL → 종류 “이어붙임”

```
(SELECT/INSERT/UPDATE/DELETE 가 있는 줄에 `${` 가 있으면)
```

돌려줄 것: { 줄번호, 종류 } 객체의 배열. 줄번호는 1부터 셉니다. 한 줄에 둘 다 있으면 두 개를 담습니다. 순서는 ① 다음 ② 입니다.

```
function 코드점검(소스) {
  const 찾은것 = [];

  소스.split("\n").forEach((줄, 자리) => {
    const 줄번호 = 자리 + 1;
```

① SELECT * — 대소문자를 가리지 않게 i, 공백은 하나 이상이라 \s+

```
if (/select\s+*/i.test(줄)) {
  찾은것.push({ 줄번호, 종류: "별표" });
}
```

② 템플릿 문자열로 이어 붙인 SQL

```

if (/^(select|insert|update|delete)/i.test(줄) && 줄.includes("${}") {
    찾은것.push({ 줄번호, 종류: "이어붙임" });
}
});

return 찾은것;
}

```

★ 왜 이 두 가지를 잡나

① SELECT * 가 왜 나쁜가

· 나중에 칸이 늘면 **조용히 같이 달려 옵니다**. 네트워크로 다 넘어옵니다 · 큰 TEXT 칸 하나가 추가되면 목록 조회가 갑자기 느려집니다 · ★ 06단원의 **Index Only Scan** 이 안 됩니다. 색인에 없는 칸을 달라고 하니까요 · 칸 순서가 바뀌면 위치로 읽는 코드가 조용히 틀립니다

→ 필요한 칸만 적으세요. 귀찮지만 그게 맞습니다.

② 이어 붙인 SQL 이 왜 나쁜가

★★★ 이건 나쁜 정도가 아니라 **구멍**입니다. 03단원에서 직접 뚫어 봤습니다.

```
`SELECT * FROM 설비 WHERE 라인코드 = ${입력}`
```

입력 이 'A' OR '1'='1 이면 전부 나옵니다. 'A'; DROP TABLE 설비; -- 면 표가 날아갑니다.

→ \$1 을 쓰고 값은 배열로 넘기세요. 예외가 없습니다.

그러면 값이 **문장이 아니라 값**으로만 취급됩니다.
개념05 섹션 7 에서 확인했습니다. 공격 문자열이 그냥 "그런 이름의 라인" 이 됩니다.

★★ 이 도구의 한계도 알아 두세요. · 여러 줄에 걸친 SQL 은 못 잡습니다 · 주석 안의 예시도 잡습니다 (거짓 양성) · + 로 이어 붙인 것은 이 정규식으로는 못 잡습니다

★ 진짜로 쓰려면 ESLint 규칙이나 `eslint-plugin-security` 같은 것을 쓰세요.

이 함수는 "이런 걸 기계로 찾을 수 있다" 는 것을 보여 주는 예입니다.

채점

```

const 채점결과 = [];

async function 검사(번호, 이름, 할일) {
  try {
    const 통과 = await 할일();
    채점결과.push({ 번호, 이름, 통과: 통과 === true });
  } catch {
    채점결과.push({ 번호, 이름, 통과: false });
  }
}

await 검사(1, "백업 형식 고르기", async () => 답1 === "custom");

await 검사(2, "백업을 진짜로 만들기", async () => {
  if (typeof 백업명령 !== "string") return false;

  도커("rm -f /tmp/정답백업.dump");
  도커(백업명령);

  const 목록 = 도커("pg_restore -l /tmp/정답백업.dump");

  return 목록.includes("설비") && !목록.includes("점검기록");
});

await 검사(3, "총 시간으로 줄 세우기", async () => 진짜범인(쿼리통계) ===
"목록조회");

await 검사(4, "오래 돌고 있는 쿼리 찾기", async () => {
  if (typeof 돌고있는것찾기 !== "string") return false;

  const 피해자 = await 새연결("unit10-answervictim");
  const 던짐 = 피해자.query("SELECT pg_sleep(20)").catch(() => {});

  await 잠깐(1400);

  const 결과 = await 연결.query(돌고있는것찾기);

```

```

const pid = (
    await 연결.query("SELECT pid FROM pg_stat_activity WHERE application_name =
'unit10-answervictim'")
    ).rows[0].pid;

    await 연결.query("SELECT pg_terminate_backend($1)", [pid]);
    피해자.on("error", () => {});
    await 던짐;
    await 피해자.end().catch(() => {});

    return 결과.rows.length === 1 && String(결과.rows[0].문장).includes("pg_sleep");
});

await 검사(5, "취소인가 강제 종료인가", async () => 답5 === "pg_cancel_backend");

await 검사(6, "표 크기 재기", async () => {
    if (typeof 표크기SQL !== "string") return false;

    const 결과 = await 연결.query(표크기SQL);

    return Number.isInteger(결과.rows[0].바이트) && 결과.rows[0].바이트 > 100000;
});

await 검사(7, "VACUUM 인가 VACUUM FULL 인가", async () => 답7가 === "VACUUM" && 답7나
=== "VACUUM FULL");

await 검사(8, "죽은 줄이 많은 표 찾기", async () => {
    if (typeof 부폰표찾기 !== "string") return false;

    await 연결.query("UPDATE 점검기록 SET 결과 = 결과");
    await 연결.query("ANALYZE 점검기록");

    let 결과 = await 연결.query(부폰표찾기);

    for (let 다시 = 0; 결과.rows.length === 0 && 다시 < 10; 다시 += 1) {
        await 잠깐(500);
        await 연결.query("ANALYZE 점검기록");
        결과 = await 연결.query(부폰표찾기);
    }
}

```



```

    }

    await 연결.query("VACUUM 점검기록");

    return 결과.rows.some((줄) => 줄.표 === "점검기록");
  });

  await 검사(9, "긴 트랜잭션 찾기", async () => {
    if (typeof 긴트랜잭션찾기 !== "string") return false;

    const 방해꾼 = await 새연결("unit10-answerblocker");
    await 방해꾼.query("BEGIN");
    await 방해꾼.query("SELECT 1");

    const 결과 = await 연결.query(긴트랜잭션찾기);

    await 방해꾼.query("ROLLBACK");
    await 방해꾼.end();

    return 결과.rows.length === 1 && 결과.rows[0].state === "idle in transaction";
  });

  await 검사(10, "안전한 칸 추가", async () => {
    const 문장들 = 안전한칸추가();

    if (!Array.isArray(문장들) || 문장들.length !== 2) return false;
    if (!/lock_timeout/i.test(문장들[0])) return false;
    if (!/2\s*s|2000\s*ms/i.test(문장들[0])) return false;

    for (const 문장 of 문장들) await 연결.query(문장);

    await 연결.query("SET lock_timeout = 0");

    const 있나 = await 연결.query(`
      SELECT count(*)::int AS 개수 FROM information_schema.columns
      WHERE table_schema = '${스키마}' AND table_name = '설비' AND column_name =
      '담당자'
    `);
  });

```

```

`);

return 있나.rows[0].개수 === 1;
});

await 검사(11, "안 막는 색인 만들기", async () => {
  if (typeof 안막는색인 !== "string") return false;
  if (!/CONCURRENTLY/i.test(안막는색인)) return false;

  await 연결.query(안막는색인);

  const 있나 = await 연결.query(` SELECT indisvalid FROM pg_index WHERE indexrelid =
'${스키마}.연습_설비번호'::regclass
`);

  return 있나.rows[0].indisvalid === true;
});

await 검사(12, "[도전] 확장 → 이전 → 축소", async () => {
  if (!Array.isArray(무중단순서) || 무중단순서.length !== 5) return false;

  const 자리 = (이름) => 무중단순서.indexOf(이름);

  return (
    자리("새칸추가") === 0 &&
    자리("동기화트리거") === 1 &&
    자리("값복사") === 2 &&
    자리("새코드배포") === 3 &&
    자리("옛칸지우기") === 4
  );
});

await 검사(13, "[도전] 색인 없는 외래키 찾기", async () => {
  if (typeof 색인없는외래키찾기 !== "string") return false;

  const 결과 = await 연결.query(색인없는외래키찾기);

  const 찾은것 = 결과.rows.map((줄) => `${줄.표}.${줄.칸}`).sort();

```

설비.라인코드는 색인이 없습니다. 점검기록.설비번호는 문제 11에서 만들었습니다.

```

return 찾은것.includes("설비.라인코드") && !찾은것.includes("점검기록.설비번호");
});

await 검사(14, "[도전] 코드 점검 도구", async () => {
  const 소스 = [
    'const a = await 풀.query("SELECT * FROM 설비");',
    'const b = await 풀.query(`SELECT 이름 FROM 설비 WHERE 라인코드 = ${입력}`);',

```

부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.